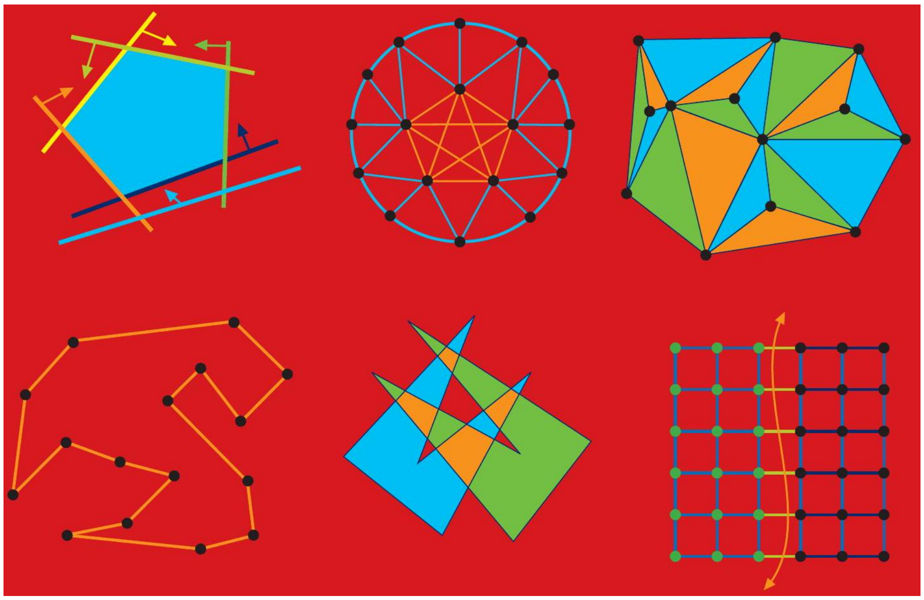
Contents

Texts in Computer Science

Steven S. Skiena
Department of Computer Science
Stony Brook University
Stony Brook, NY 11794-2424
http://www.cs.stonybrook.edu/~skiena
August 2020

TEXTS IN COMPUTER SCIENCE THE Algorithm Design MANUAL



Steven S. Skiena Springer

David Gries, Department of Computer Science, Cornell University, Ithaca, NY, USA
Orit Hazzan (10), Faculty of Education in Technology and Science, Technion-Israel Institute of Technology, Haifa, Israel

Titles in this series now included in the Thomson Reuters Book Citation Index!
'Texts in Computer Science' (TCS) delivers high-quality instructional content for undergraduates and graduates in all areas of computing and information science, with a strong emphasis on core foundational and theoretical material but inclusive of some prominent applications-related content. TCS books should be reasonably self-contained and aim to provide students with modern and clear accounts of topics ranging across the computing curriculum. As a result, the books are ideal for semester courses or for individual self-study in cases where people need to expand their knowledge. All texts are authored by established experts in their fields, reviewed internally and by the series editors, and provide numerous examples, problems, and other pedagogical tools; many contain fully worked solutions.

The TCS series is comprised of high-quality, self-contained books that have broad and comprehensive coverage and are generally in hardback format and sometimes contain color. For undergraduate textbooks that are likely to be more brief and modular in their approach, require only black and white, and are under 275 pages, Springer offers the flexibly designed Undergraduate Topics in Computer Science series, to which we refer potential authors.

More information about this series at http://www.springer.com/series/3191
The Algorithm Design Manual Third Edition
Springer
ISSN 1868-0941
ISSN 1868-095X (electronic)
Texts in Computer Science
ISBN 978-3-030-54255-9
ISBN 978-3-030-54256-6 (eBook)
https://doi.org/10.1007/978-3-030-54256-6

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

0.1 Preface

Many professional programmers are not well prepared to tackle algorithm design problems. This is a pity, because the techniques of algorithm design form one of the core practical *technologies* of computer science.

This book is intended as a manual on algorithm design, providing access to combinatorial algorithm technology for both students and computer professionals. It is divided into two parts: *Techniques* and *Resources*. The former is a general introduction to the design and analysis of computer algorithms. The Resources section is intended for browsing and reference, and comprises the catalog of algorithmic resources, implementations, and an extensive bibliography.

To the Reader I have been gratified by the warm reception previous editions of *The Algorithm Design Manual* have received, with over 60,000 copies sold in various formats since first being published by Springer-Verlag in 1997. Translations have appeared in Chinese, Japanese, and Russian. It has been recognized as a unique guide to using algorithmic techniques to solve problems that often arise in practice.

Much has changed in the world since the second edition of The Algorithm Design Manual was published in 2008. The popularity of my book soared as software companies increasingly emphasized algorithmic questions during employment interviews, and many successful job candidates have trusted The Algorithm Design Manual to help them prepare for their interviews.

Although algorithm design is perhaps the most classical area of computer science, it continues to advance and change. Randomized algorithms and data structures have become increasingly important, particularly techniques based on hashing. Recent breakthroughs have reduced the algorithmic complexity of the best algorithms known for such fundamental problems as finding minimum spanning trees, graph isomorphism, and network flows. Indeed, if we date the origins of modern algorithm design and analysis to about 1970, then roughly 20% of modern algorithmic history has happened since the second edition of The Algorithm Design Manual.

The time has come for a new edition of my book, incorporating changes in the algorithmic and industrial world plus the feedback I have received from hundreds of readers. My major goals for the third edition are:

- To introduce or expand coverage of important topics like hashing, randomized algorithms, divide and conquer, approximation algorithms, and quantum computing in the first part of the book (Practical Algorithm Design).
- To update the reference material for all the catalog problems in the second part of the book (*The Hitchhiker's Guide to Algorithms*).
- To take advantage of advances in color printing to produce more informative and eye-catching illustrations.

Three aspects of *The Algorithm Design Manual* have been particularly beloved: (1) the hitchhiker's guide to algorithms, (2) the war stories, and (3) the electronic component of the book. These features have been preserved and strengthened in this third edition:

- The Hitchhiker's Guide to Algorithms - Since finding out what is known about an algorithmic problem can be a difficult task, I provide a catalog of the seventy-five most important algorithmic problems arising in practice. By browsing through this catalog, the student or practitioner can quickly identify what their problem is called, what is known about it, and how they should proceed to solve it.

I have updated every section in response to the latest research results and applications. Particular attention has been paid to updating discussion of available software implementations for each problem, reflecting sources such as GitHub, which have emerged since the previous edition.

- *War stories* - To provide a better perspective on how algorithm problems arise in the real world, I include a collection of "war stories", tales from my experience on real problems. The moral of these stories is that algorithm design and analysis is not just theory, but an important tool to be pulled out and used as needed.

The new edition of the book updates the best of the old war stories, plus adds new tales on randomized algorithms, divide and conquer, and dynamic programming.

- *Online component* - Full lecture notes and a problem solution Wiki is available on my website WWW.algorist.com. My algorithm lecture videos have been watched over 900,000 times on YouTube. This website has been updated in parallel with the book.

Equally important is what is not done in this book. I do not stress the mathematical analysis of algorithms, leaving most of the analysis as informal arguments. You will not find a single theorem anywhere in this book. When more details are needed, the reader should study the cited programs or references. The goal of this manual is to get you going in the right direction as quickly as possible.

Chapter 1

To the Instructor

This book covers enough material for a standard *Introduction to Algorithms* course. It is assumed that the reader has completed the equivalent of a second programming course, typically titled *Data Structures* or *Computer Science II.*

A full set of lecture slides for teaching this course are available online at www.algorist.com. Further, I make available online video lectures using these slides to teach a full-semester algorithm course. Let me help teach your course, through the magic of the Internet!

I have made several pedagogical improvements throughout the book, including:

- *New material* - To reflect recent developments in algorithm design, I have added new chapters on randomized algorithms, divide and conquer, and approximation algorithms. I also delve deeper into topics such as hashing. But I have been careful to heed the readers who begged me to keep the book of modest length. I have (painfully) removed less important material to keep total expansion by page count under 10% over the previous edition.

- *Clearer exposition* - Reading through my text ten years later, I was thrilled to find many sections where my writing seemed ethereal, but other places that were a muddled mess. Every page in this manuscript has been edited or rewritten for greater clarity, correctness and flow.

- *More interview resources* - The Algorithm Design Manual remains very popular for interview prep, but this is a fast-paced world. I include more and fresher interview problems, plus coding challenges associated with interview sites like LeetCode and Hackerrank. I also include a new section with advice on how to best prepare for interviews.

- *Stop and think* - Each of my course lectures begins with a "Problem of the Day," where I illustrate my thought process as I solve a topic-specific homework problem - false starts and all. This edition had more Stop and Think sections, which perform a similar mission for the reader.

- *More and better homework problems* - The third edition of The Algorithm Design Manual has more and better homework exercises than the previous one. I have added over a hundred exciting new problems, pruned some less interesting problems, and clarified exercises that proved confusing or ambiguous.

- *Updated code style* - The second edition featured algorithm implementations in C, replacing or augmenting pseudocode descriptions. These have generally been well received, although certain aspects of my programming have been condemned by some as old fashioned. All programs have been revised and updated, and are structurally highlighted in color.

- *Color images* - My companion book *The Data Science Design Manual* was printed with color images, and I was excited by how much this made concepts clearer. Every single image in the The Algorithm Design Manual is now rendered in living color, and the process of review has improved the contents of most figures in the text.

Chapter 2

Acknowledgments

Updating a book dedication every ten years focuses attention on the effects of time. Over the lifespan of this book, Renee became my wife and then the mother of our two children, Bonnie and Abby, who are now no longer children. My father has left this world, but Mom and my brothers Len and Rob remain a vital presence in my life. I dedicate this book to my family, new and old, here and departed.

I would like to thank several people for their concrete contributions to this new edition. Michael Alvin, Omar Amin, Emily Barker, and Jack Zheng were critical to building the website infrastructure and dealing with a variety of manuscript preparation issues. Their roles were played by Ricky Bradley, Andrew Gaun, Zhong Li, Betson Thomas, and Dario Vlah on previous editions. The world's most careful reader, Robert Piché of Tampere University, and Stony Brook students Peter Duffy, Olesia Elfimova, and Robert Matsibekker read early versions of this edition, and saved both you and me the trouble of dealing with many errata. Thanks also to my Springer-Verlag editors, Wayne Wheeler and Simon Rees.

Several exercises were originated by colleagues or inspired by other texts. Reconstructing the original sources years later can be challenging, but credits for each problem (to the best of my recollection) appear on the website.

Much of what I know about algorithms I learned along with my graduate students. Several of them (Yaw-Ling Lin, Sundaram Gopalakrishnan, Ting Chen, Francine Evans, Harald Rau, Ricky Bradley, and Dimitris Margaritis) are the real heroes of the war stories related within. My Stony Brook friends and algorithm colleagues Estie Arkin, Michael Bender, Jing Chen, Rezaul Chowdhury, Jie Gao, Joe Mitchell, and Rob Patro have always been a pleasure to work with.

Chapter 3

Caveat

It is traditional for the author to magnanimously accept the blame for whatever deficiencies remain. I don't. Any errors, deficiencies, or problems in this book are somebody else's fault, but I would appreciate knowing about them so as to determine who is to blame.

3.1 Contents

Part I ***Practical Algorithm Design*** *Chapter 1*

Chapter 4

Introduction to Algorithm Design

What is an algorithm? An algorithm is a procedure to accomplish a specific task. An algorithm is the idea behind any reasonable computer program.

To be interesting, an algorithm must solve a general, well-specified problem. An algorithmic problem is specified by describing the complete set of instances it must work on and of its output after running on one of these instances. This distinction, between a problem and an instance of a problem, is fundamental. For example, the algorithmic problem known as sorting is defined as follows:

Problem: Sorting Input: A sequence of n *keys* $a_1, \ldots, a_n$.
Output: The permutation (reordering) of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_{n-1} \leq a'_n$.

An instance of sorting might be an array of names, like {Mike, Bob, Sally, Jill, Jan}, or a list of numbers like $\{154, 245, 568, 324, 654, 324\}$. *Determining that you are dealing with a general problem instead of an instance is your first step towards solving it.*

An algorithm is a procedure that takes any of the possible input instances and transforms it to the desired output. There are many different algorithms that can solve the problem of sorting. For example, insertion sort is a method that starts with a single element (thus trivially forming a sorted list) and then incrementally inserts the remaining elements so that the list remains sorted. An animation of the logical flow of this algorithm on a particular instance (the letters in the word "INSERTIONSORT") is given in Figure 1.1.

This algorithm, implemented in C, is described below:

I N S ER T I O N S OR
R
I NSERTIONSOR
EINSRTIONSOR
EINRSTIONSOR
EINRSTIONSOR
EIINRSTONSOR
EIINORSTNSOR
EIINNORSTSOR
EI IN NORSSTOR
EIINNOORSSTR
EIINNOORRS S T

Figure 1.1: Animation of insertion sort in action (time flows downward).

```
void insertion_sort(item_type s[], int n) {
    int i, j; /* counters */
    for (i = 1; i < n; i++) {
        j = i;
        while ((j > 0) && (s[j] < s[j - 1])) {
            swap(&s[j], &s[j - 1]);
            j = j-1;
        }
    }
}
```

Note the generality of this algorithm. It works just as well on names as it does on numbers. Or anything else, given the appropriate comparison operation ($<$) *to test which of two keys should appear first in sorted order. It can be readily verified that this algorithm correctly orders every possible input instance according to our definition of the sorting problem.*

There are three desirable properties for a good algorithm. We seek algorithms that are correct and efficient, while being easy to implement. These goals may not be simultaneously achievable. In industrial settings, any program that seems to give good enough answers without slowing the application down is often acceptable, regardless of whether a better algorithm exists. The issue of finding the best possible answer or achieving maximum efficiency usually arises in industry only after serious performance or legal troubles.

This chapter will focus on algorithm correctness, with our discussion of efficiency concerns deferred to Chapter 2 It is seldom obvious whether a given algorithm correctly solves a given problem. Correct algorithms usually come with a proof of correctness, which is an explanation of why we know that the algorithm must take every instance of the problem to the desired result. But before we go further, it is important to demonstrate why it's obvious never suffices as a proof of correctness, and is usually flat-out wrong.

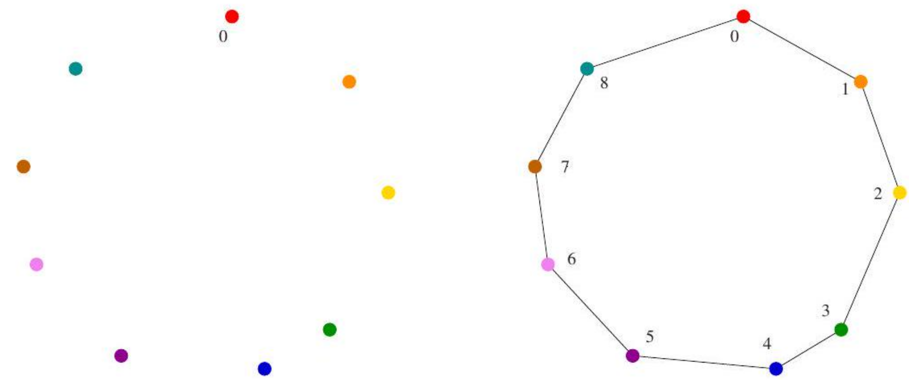


Figure 4.1: A good instance for the *nearest-neighbor* heuristic. The rainbow coloring (red to violet) reflects the order of incorporation.

4.1 Robot Tour Optimization

Let's consider a problem that arises often in manufacturing, transportation, and testing applications. Suppose we are given a robot arm equipped with a tool, say a soldering iron. When manufacturing circuit boards, all the chips and other components must be fastened onto the substrate. More specifically, each chip has a set of contact points (or wires) that need be soldered to the board. To program the robot arm for this job, we must first construct an ordering of the contact points so that the robot visits (and solders) the first contact point, then the second point, third, and so forth until the job is done. The robot arm then proceeds back to the first contact point to prepare for the next board, thus turning the tool-path into a closed tour, or cycle.

Robots are expensive devices, so we want the tour that minimizes the time it takes to assemble the circuit board. A reasonable assumption is that the robot arm moves with fixed speed, so the time to travel between two points is proportional to their distance. In short, we must solve the following algorithm problem:

Problem: Robot Tour Optimization Input: A set S of n points in the plane. Output: What is the shortest cycle tour that visits each point in the set S ? You are given the job of programming the robot arm. Stop right now and think up an algorithm to solve this problem. I'll be happy to wait for you...

Several algorithms might come to mind to solve this problem. Perhaps the most popular idea is the nearest-neighbor heuristic. Starting from some point p_0, we walk first to its nearest neighbor p_1. From p_1, we walk to its nearest unvisited neighbor, thus excluding only p_0 as a candidate. We now repeat this process until we run out of unvisited points, after which we return to p_0 to close off the tour. Written in pseudo-code, the nearest-neighbor heuristic looks like

this:

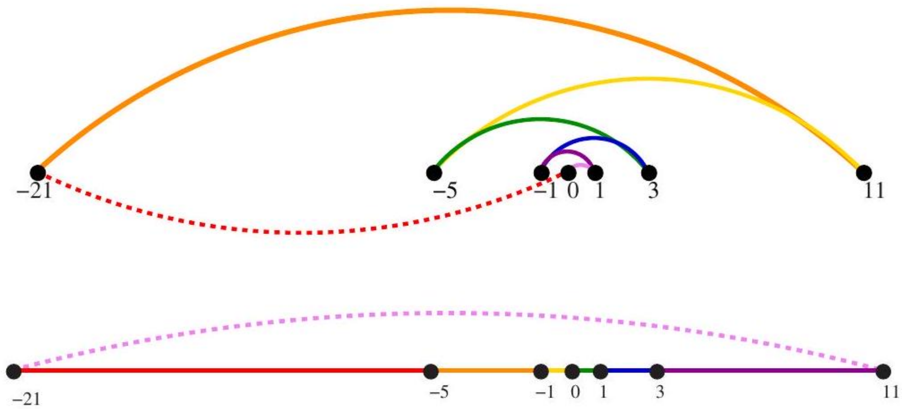


Figure 4.2: A bad instance for the nearest-neighbor heuristic, with the optimal solution. Colors are sequenced as ordered in the rainbow.

NearestNeighbor (P)
Pick and visit an initial point p_0 *from* P
$p = p_0$
$i = 0$
While there are still unvisited points
$i = i + 1$
Select p_i *to be the closest unvisited point to* p_{i-1}
Visit p_i
Return to p_0 *from* p_{n-1}

This algorithm has a lot to recommend it. It is simple to understand and implement. It makes sense to visit nearby points before we visit faraway points to reduce the total travel time. The algorithm works perfectly on the example in Figure 1.2. The nearest-neighbor rule is reasonably efficient, for it looks at each pair of points (p_i, p_j) *at most twice: once when adding* p_i *to the tour, the other when adding* p_j*. Against all these positives there is only one problem. This algorithm is completely wrong.*

Wrong? How can it be wrong? The algorithm always finds a tour, but it doesn't necessarily find the shortest possible tour. It doesn't necessarily even come close. Consider the set of points in Figure 1.3 all of which lie along a line. The numbers describe the distance that each point lies to the left or right of the point labeled " 0 ". When we start from the point " 0 " and repeatedly walk to the nearest unvisited neighbor, we might keep jumping left-right-left-right over " 0 " as the algorithm offers no advice on how to break ties. A much better (indeed optimal) tour for these points starts from the left-most point and visits *each point as we walk right before returning at the left-most point.*

Try now to imagine your boss's delight as she watches a demo of your robot arm hopscotching left-right-left-right during the assembly of such a simple board.

"But wait," you might be saying. "The problem was in starting at point " 0. ." Instead, why don't we start the nearest-neighbor rule using the left-most point as the initial point p_0 *? By doing this, we will find the optimal solution on this instance."*

That is 100% *true, at least until we rotate our example by 90 degrees. Now all points are equally left-most. If the point " 0 " were moved just slightly to the left, it would be picked as the starting point. Now the robot arm will hopscotch up-down-up-down instead of left-right-left-right, but the travel time will be just as bad as before. No matter what you do to pick the first point, the nearest-neighbor rule is doomed to work incorrectly on certain point sets.*

Maybe what we need is a different approach. Always walking to the closest point is too restrictive, since that seems to trap us into making moves we didn't want. A different idea might repeatedly connect the closest pair of endpoints whose connection will not create a problem, such as premature termination of the cycle. Each vertex begins as its own single vertex chain. After merging everything together, we will end up with a single chain containing all the points in it. Connecting the final two endpoints gives us a cycle. At any step during the execution of this closest-pair heuristic, we will have a set of single vertices and the end of vertex-disjoint chains available to merge. In pseudocode:

ClosestPair (P) Let n *be the number of points in set* P.
For $i = 1$ *to* $n - 1$ *do*
$d = \infty$
For each pair of endpoints (s, t *) from distinct vertex chains if* $\operatorname{dist}(s,t) \leq d$ *then* $s_m = s, t_m = t$, *and* $d = \operatorname{dist}(s,t)$ *Connect* (s_m, t_m) *by an edge*
Connect the two endpoints by an edge
This closest-pair rule does the right thing in the example in Figure 1.3 It starts by connecting " 0 " to its two immediate neighbors, the points 1 and -1 . Subsequently, the next closest pair will alternate left-right, growing the central path by one link at a time. The closest-pair heuristic is somewhat more complicated and less efficient than the previous one, but at least it gives the right answer in this example.

But not on all examples. Consider what this algorithm does on the point set in Figure 1.4 (l). It consists of two rows of equally spaced points, with the rows slightly closer together (distance $1 - \epsilon$ *) than the neighboring points are spaced within each row (distance* $1 + \epsilon$ *). Thus, the closest pairs of points stretch across the gap, not around the boundary. After we pair off these points, the closest remaining pairs will connect these pairs alternately around the boundary. The total path length of the closest-pair tour is* $3(1-\epsilon) + 2(1+\epsilon) + \sqrt{(1-\epsilon)^2 + (2+2\epsilon)^2}$. *Compared to the tour shown in Figure 1.4(r), we travel over* 20% *farther than necessary when* $\epsilon \to 0$. *Examples exist where the penalty is considerably worse than this.*

Thus, this second algorithm is also wrong. Which one of these algorithms

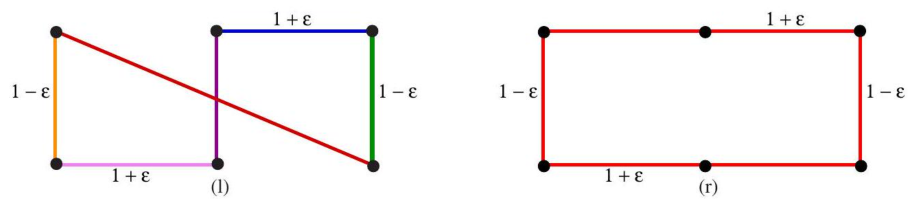


Figure 4.3: A bad instance for the closest-pair heuristic, with the optimal solution.

performs better? You can't tell just by looking at them. Clearly, both heuristics can end up with very bad tours on innocent-looking input.

At this point, you might wonder what a correct algorithm for our problem looks like. Well, we could try enumerating all possible orderings of the set of points, and then select the one that minimizes the total length:

OptimalTSP (P)

$d = \infty$

For each of the $n!$ *permutations* P_i *of point set* P

If $(cost(P_i) \leq d)$ *then* $d = \text{cost}\,(P_i)$ *and* $P_{\min} = P_i$

Return $P_{\min}$

Since all possible orderings are considered, we are guaranteed to end up with the shortest possible tour. This algorithm is correct, since we pick the best of all the possibilities. But it is also extremely slow. Even a powerful computer couldn't hope to enumerate all the $20! = 2,432,902,008,176,640,000$ *orderings of 20 points within a day. For real circuit boards, where* $n \approx 1,000$*, forget about it. All of the world's computers working full time wouldn't come close to finishing the problem before the end of the universe, at which point it presumably becomes moot.*

The quest for an efficient algorithm to solve this problem, called the traveling salesman problem (TSP), will take us through much of this book. If you need to know how the story ends, check out the catalog entry for TSP in Section 19.4 (page 594).

Take-Home Lesson: There is a fundamental difference between algorithms, procedures that always produce a correct result, and heuristics, which may usually do a good job but provide no guarantee of correctness.

4.2 Selecting the Right Jobs

Now consider the following scheduling problem. Imagine you are a highly in demand actor, who has been presented with offers to star in n different movie *projects under development. Each offer comes specified with the first and last*

Tarjan of the Jungle		*The Four Volume Problem*
The President's Algorist	*Steiner's Tree*	*Process Terminated*
	Halting State	*Programming Challenges*
"Discrete" Mathematics		*Calculated Bets*

Figure 1.5: An instance of the non-overlapping movie scheduling problem. The four red titles define an optimal solution.

day of filming. Whenever you accept a job, you must commit to being available throughout this entire period. Thus, you cannot accept two jobs whose intervals overlap.

For an artist such as yourself, the criterion for job acceptance is clear: you want to make as much money as possible. Because each film pays the same fee, this implies you seek the largest possible set of jobs (intervals) such that no two of them conflict with each other.

For example, consider the available projects in Figure 1.5 You can star in at most four films, namely "Discrete" Mathematics, Programming Challenges, Calculated Bets, and one of either Halting State or Steiner's Tree.

You (or your agent) must solve the following algorithmic scheduling problem:

Problem: Movie Scheduling Problem

Input: A set I of n intervals on the line.

Output: What is the largest subset of mutually non-overlapping intervals that can be selected from I ?

Now you (the algorist) are given the job of developing a scheduling algorithm for this task. Stop right now and try to find one. Again, I'll be happy to wait. . .

There are several ideas that may come to mind. One is based on the notion that it is best to work whenever work is available. This implies that you should start with the job with the earliest start date - after all, there is no other job you can work on then, at least during the beginning of this period:

EarliestJobFirst(I)

Accept the earliest starting job j from I that does not overlap any previously accepted job, and repeat until no more such jobs remain.

This idea makes sense, at least until we realize that accepting the earliest job might block us from taking many other jobs if that first job is long. Check out Figure 1.6(1), where the epic War and Peace is both the first job available and long enough to kill off all other prospects.

This bad example naturally suggests another idea. The problem with War and Peace is that it is too long. Perhaps we should instead start by taking the shortest job, and keep seeking the shortest available job at every turn. Maximizing the number of jobs we do in a given period is clearly connected to the notion of banging them out as quickly as possible. This yields the heuristic:

```
ShortestJobFirst(I)
```

Figure 4.4: Bad instances for the (l) earliest job first and (r) shortest job first heuristics. The optimal solutions are in red.

```
While ( }I\not=\emptyset\mathrm{ ) do
Accept the shortest possible job j from I.
Delete j, and any interval that intersects j, from I.
```

Again this idea makes sense, at least until we realize that accepting the shortest job might block us from taking two other jobs, as shown in Figure 1.6(r). While the maximum potential loss here seems smaller than with the previous heuristic, it can still limit us to half the optimal payoff.

At this point, an algorithm where we try all possibilities may start to look good. As with the TSP problem, we can be certain exhaustive search is correct. If we ignore the details of testing whether a set of intervals are in fact disjoint, it looks something like this:

```
ExhaustiveScheduling $(I)$
    $j=0$
    $S_{\text {max }}=\emptyset$
    For each of the $2^{n}$ subsets $S_{i}$ of intervals $I$
        If ( $S_{i}$ is mutually non-overlapping) and $\left(\operatorname{size}\left(S
            then $j=\operatorname{size}\left(S_{i}\right)$ and $S_{\max }=S_{i}$.
    Return $S_{\text {max }}$
```

But how slow is it? The key limitation is enumerating the 2^n *subsets of* n *things. The good news is that this is much better than enumerating all* n *! orders of* n *things, as proposed for the robot tour optimization problem. There are only about one million subsets when* $n = 20$*, which can be enumerated within seconds on a decent computer. However, when fed* $n = 100$ *movies, we get* 2^{100} *subsets, which is much much greater than the 20 ! that made our robot cry "uncle" in the previous problem.*

The difference between our scheduling and robotics problems is that there is an algorithm that solves movie scheduling both correctly and efficiently. Think about the first job to terminate - that is, the interval x *whose right endpoint is left-most among all intervals. This role is played by "Discrete" Mathematics in Figure 1.5. Other jobs may well have started before* x*, but all of these must at least partially overlap each other. Thus, we can select at most one from the group. The first of these jobs to terminate is* x*, so any of the overlapping jobs potentially block out other opportunities to the right of it. Clearly we can never lose by picking* x*. This suggests the following correct, efficient algorithm:*

OptimalScheduling (I)

While ($I \neq \emptyset$) do
Accept the job j from I with the earliest completion date.
Delete j, and any interval which intersects j, from I.
Ensuring the optimal answer over all possible inputs is a difficult but often achievable goal. Seeking counterexamples that break pretender algorithms is an important part of the algorithm design process. Efficient algorithms are often lurking out there; this book will develop your skills to help you find them.

Take-Home Lesson: Reasonable-looking algorithms can easily be incorrect. Algorithm correctness is a property that must be carefully demonstrated.

4.3 Reasoning about Correctness

Hopefully, the previous examples have opened your eyes to the subtleties of algorithm correctness. We need tools to distinguish correct algorithms from incorrect ones, the primary one of which is called a proof.

A proper mathematical proof consists of several parts. First, there is a clear, precise statement of what you are trying to prove. Second, there is a set of assumptions of things that are taken to be true, and hence can be used as part of the proof. Third, there is a chain of reasoning that takes you from these assumptions to the statement you are trying to prove. Finally, there is a little square ($\mathbf{\Pi}$) or QED at the bottom to denote that you have finished, representing the Latin phrase for "thus it is demonstrated."

This book is not going to emphasize formal proofs of correctness, because they are very difficult to do right and quite misleading when you do them wrong. A proof is indeed a demonstration. Proofs are useful only when they are honest, crisp arguments that explain why an algorithm satisfies a non-trivial correctness property. Correct algorithms require careful exposition, and efforts to show both correctness and not incorrectness.

4.3.1 Problems and Properties

Before we start thinking about algorithms, we need a careful description of the problem that needs to be solved. Problem specifications have two parts: (1) the set of allowed input instances, and (2) the required properties of the algorithm's output. It is impossible to prove the correctness of an algorithm for a fuzzilystated problem. Put another way, ask the wrong question and you will get the wrong answer.

Some problem specifications allow too broad a class of input instances. Suppose we had allowed film projects in our movie scheduling problem to have gaps in production (e.g. filming in September and November but a hiatus in October). Then the schedule associated with any particular film would consist of a given set of intervals. Our star would be free to take on two interleaving but not *overlapping projects (such as the above-mentioned film nested with one filming*

in August and October). The earliest completion algorithm would not work for such a generalized scheduling problem. Indeed, no efficient algorithm exists for this generalized problem, as we will see in Section 11.3.2

Take-Home Lesson: An important and honorable technique in algorithm design is to narrow the set of allowable instances until there is a correct and efficient algorithm. For example, we can restrict a graph problem from general graphs down to trees, or a geometric problem from two dimensions down to one.

There are two common traps when specifying the output requirements of a problem. The first is asking an ill-defined question. Asking for the best route between two places on a map is a silly question, unless you define what best means. Do you mean the shortest route in total distance, or the fastest route, or the one minimizing the number of turns? All of these are liable to be different things.

The second trap involves creating compound goals. The three route-planning criteria mentioned above are all well-defined goals that lead to correct, efficient optimization algorithms. But you must pick a single criterion. A goal like Find the shortest route from a to b that doesn't use more than twice as many turns as necessary is perfectly well defined, but complicated to reason about and solve.

I encourage you to check out the problem statements for each of the seventyfive catalog problems in Part II of this book. Finding the right formulation for your problem is an important part of solving it. And studying the definition of all these classic algorithm problems will help you recognize when someone else has thought about similar problems before you.

4.3.2 Expressing Algorithms

Reasoning about an algorithm is impossible without a careful description of the sequence of steps that are to be performed. The three most common forms of algorithmic notation are (1) English, (2) pseudocode, or (3) a real programming language. Pseudocode is perhaps the most mysterious of the bunch, but it is best defined as a programming language that never complains about syntax errors.

All three methods are useful because there is a natural tradeoff between greater ease of expression and precision. English is the most natural but least precise programming language, while Java and C/C++ are precise but difficult to write and understand. Pseudocode is generally useful because it represents a happy medium.

The choice of which notation is best depends upon which method you are most comfortable with. I usually prefer to describe the ideas of an algorithm in English (with pictures!), moving to a more formal, programming-language-like pseudocode or even real code to clarify sufficiently tricky details.

A common mistake my students make is to use pseudocode to dress up an ill-defined idea so that it looks more formal. Clarity should be the goal. For *example, the ExhaustiveScheduling algorithm on page 10 would have better been written in English as:*

ExhaustiveScheduling (I)

Test all 2^n subsets of intervals from I, and return the largest subset consisting of mutually non-overlapping intervals.

Take-Home Lesson: The heart of any algorithm is an idea. If your idea is not clearly revealed when you express an algorithm, then you are using too low-level a notation to describe it.

4.3.3 Demonstrating Incorrectness

The best way to prove that an algorithm is incorrect is to produce an instance on which it yields an incorrect answer. Such instances are called counterexamples. No rational person will ever defend the correctness of an algorithm after a counter-example has been identified. Very simple instances can instantly defeat reasonable-looking heuristics with a quick touché. Good counterexamples have two important properties:

- *Verifiability - To demonstrate that a particular instance is a counterexample to a particular algorithm, you must be able to (1) calculate what answer your algorithm will give in this instance, and (2) display a better answer so as to prove that the algorithm didn't find it.*

- *Simplicity - Good counter-examples have all unnecessary details stripped away. They make clear exactly why the proposed algorithm fails. Simplicity is important because you must be able to hold the given instance in your head in order to reason about it. Once a counterexample has been found, it is worth simplifying it down to its essence. For example, the counterexample of Figure 1.6 (1) could have been made simpler and better by reducing the number of overlapped segments from five to two.*

Hunting for counterexamples is a skill worth developing. It bears some similarity to the task of developing test sets for computer programs, but relies more on inspiration than exhaustion. Here are some techniques to aid your quest:

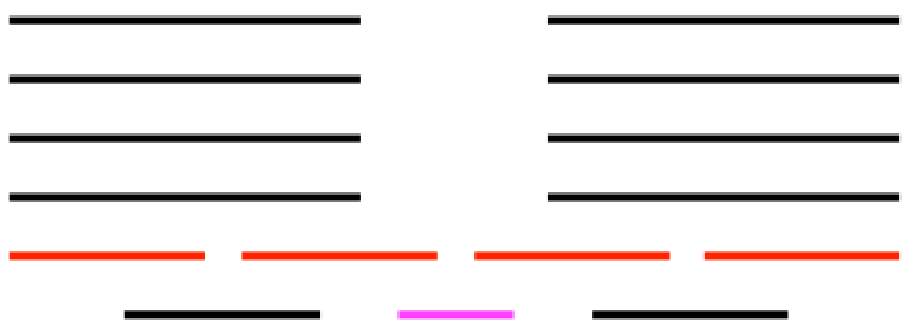
- *Think small - Note that the robot tour counter-examples I presented boiled down to six points or less, and the scheduling counter-examples to only three intervals. This is indicative of the fact that when algorithms fail, there is usually a very simple example on which they fail. Amateur algorists tend to draw a big messy instance and then stare at it helplessly. The pros look carefully at several small examples, because they are easier to verify and reason about.*

- *Think exhaustively - There are usually only a small number of possible instances for the first non-trivial value of n. For example, there are only three distinct ways two intervals on the line can occur: as disjoint intervals, as overlapping intervals, and as properly nesting intervals, one within* *the other. All cases of three intervals (including counter-examples to both of the movie heuristics) can be systematically constructed by adding a third segment in each possible way to these three instances.*

- *Hunt for the weakness - If a proposed algorithm is of the form "always take the biggest" (better known as the greedy algorithm), think about why that might prove to be the wrong thing to do. In particular, ...*

- *Go for a tie - A devious way to break a greedy heuristic is to provide instances where everything is the same size. Suddenly the heuristic has nothing to base its decision on, and perhaps has the freedom to return something suboptimal as the answer.*

- *Seek extremes - Many counter-examples are mixtures of huge and tiny, left and right, few and many, near and far. It is usually easier to verify or reason about extreme examples than more muddled ones. Consider two tightly bunched clouds of points separated by a much larger distance d. The optimal TSP tour will be essentially $2d$ regardless of the number of points, because what happens within each cloud doesn't really matter.*



Take-Home Lesson: Searching for counterexamples is the best way to disprove the correctness of a heuristic.

Stop and Think: Greedy Movie Stars? *Problem: Recall the movie star scheduling problem, where we seek to find the largest possible set of non-overlapping intervals in a given set S. A natural greedy heuristic selects the interval i, which overlaps the smallest number of other intervals in S, removes them, and repeats until no intervals remain.*

Give a counter-example to this proposed algorithm.

Solution: Consider the counter-example in Figure 1.7 The largest possible independent set consists of the four intervals in red, but the interval of lowest degree (shown in pink) overlaps two of these, After we grab it, we are doomed to finding a solution of only three intervals.

But how would you go about constructing such an example? My thought process started with an odd-length chain of intervals, each of which overlaps one *interval to the left and one to the right. Picking an even-length chain would mess up the optimal solution (hunt for the weakness). All intervals overlap two others, except for the left and right-most intervals (go for the tie). To make these terminal intervals unattractive, we can pile other intervals on top of them*

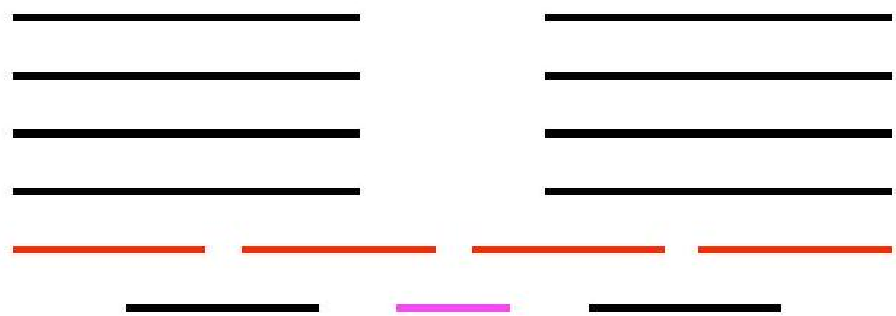


Figure 4.5: Counter-example to the greedy heuristic for movie star scheduling. Picking the pink interval, which intersects the fewest others, blocks us from the optimal solution (the four red intervals).

(seek extremes). The length of our chain (7) is the shortest that permits this construction to work.

4.4 Induction and Recursion

Failure to find a counterexample to a given algorithm does not mean "it is obvious" that the algorithm is correct. A proof or demonstration of correctness is needed. Often mathematical induction is the method of choice.

When I first learned about mathematical induction it seemed like complete magic. You proved a formula like $\sum_{i=1}^{n} i = n(n+1)/2$ *for some basis case like* $n = 1$ *or 2 , then assumed it was true all the way to* $n - 1$ *before proving it was in fact true for general* n *using the assumption. That was a proof? Ridiculous!*

When I first learned the programming technique of recursion it also seemed like complete magic. The program tested whether the input argument was some basis case like 1 or 2 . If not, you solved the bigger case by breaking it into pieces and calling the subprogram itself to solve these pieces. That was a program? Ridiculous!

The reason both seemed like magic is because recursion is mathematical induction in action. In both, we have general and boundary conditions, with the general condition breaking the problem into smaller and smaller pieces. The initial or boundary condition terminates the recursion. Once you understand either recursion or induction, you should be able to see why the other one also works.

I've heard it said that a computer scientist is a mathematician who only knows how to prove things by induction. This is partially true because computer scientists are lousy at proving things, but primarily because so many of the algorithms we study are either recursive or incremental.

Consider the correctness of insertion sort, which we introduced at the beginning of this chapter. The reason it is correct can be shown inductively:

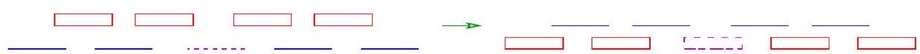


Figure 4.6: Large-scale changes in the optimal solution (red boxes) after inserting a single interval (dashed) into the instance.

- *The basis case consists of a single element, and by definition a one-element array is completely sorted.*
- *We assume that the first $n-1$ elements of array A are completely sorted after $n-1$ iterations of insertion sort.*
- *To insert one last element x to A, we find where it goes, namely the unique spot between the biggest element less than or equal to x and the smallest element greater than x. This is done by moving all the greater elements back by one position, creating room for x in the desired location.*

One must be suspicious of inductive proofs, however, because very subtle reasoning errors can creep in. The first are boundary errors. For example, our insertion sort correctness proof above boldly stated that there was a unique place to insert x between two elements, when our basis case was a single-element array. Greater care is needed to properly deal with the special cases of inserting the minimum or maximum elements.

The second and more common class of inductive proof errors concerns cavalier extension claims. Adding one extra item to a given problem instance might cause the entire optimal solution to change. This was the case in our scheduling problem (see Figure 1.8). The optimal schedule after inserting a new segment may contain none of the segments of any particular optimal solution prior to insertion. Boldly ignoring such difficulties can lead to very convincing inductive proofs of incorrect algorithms.

Take-Home Lesson: Mathematical induction is usually the right way to verify the correctness of a recursive or incremental insertion algorithm.

Stop and Think: Incremental Correctness Problem: Prove the correctness of the following recursive algorithm for incrementing natural numbers, that is, $y \rightarrow y+1$:

```
Increment ( $y$ )
    if $(y=0)$ then return(1) else
```

```
if $(y \bmod 2)=1$ then
    return( $2 \cdot \operatorname{Increment}(\lfloor y / 2\rfloor))$
    else return $(y+1)$
```

Solution: The correctness of this algorithm is certainly not obvious to me. But as it is recursive and I am a computer scientist, my natural instinct is to try to prove it by induction. The basis case of $y = 0$ *is obviously correctly handled. Clearly the value 1 is returned, and* $0 + 1 = 1$.

Now assume the function works correctly for the general case of $y = n - 1$. *Given this, we must demonstrate the truth for the case of* $y = n$. *The cases corresponding to even numbers are obvious, because* $y + 1$ *is explicitly returned when* $(y \bmod 2) = 0$.

For odd numbers, the answer depends on what Increment $(\lfloor y/2 \rfloor)$ *returns. Here we want to use our inductive assumption, but it isn't quite right. We have assumed that Increment worked correctly for* $y = n - 1$, *but not for a value that is about half of it. We can fix this problem by strengthening our assumption to declare that the general case holds for all* $y \leq n - 1$. *This costs us nothing in principle, but is necessary to establish the correctness of the algorithm.*

Now, the case of odd y (i.e. $y = 2m + 1$ *for some integer m) can be dealt with as:*

$$\begin{aligned} 2 \cdot \textit{ Increment } (\lfloor (2m+1)/2 \rfloor) &= 2 \cdot \text{Increment}(\lfloor m + 1/2 \rfloor) \\ &= 2 \cdot \text{Increment}(m) \\ &= 2(m+1) \\ &= 2m + 2 = y + 1 \end{aligned}$$

and the general case is resolved.

Modeling the Problem Modeling is the art of formulating your application in terms of precisely described, well-understood problems. Proper modeling is the key to applying algorithmic design techniques to real-world problems. Indeed, proper modeling can eliminate the need to design or even implement algorithms, by relating your application to what has been done before. Proper modeling is the key to effectively using the "Hitchhiker's Guide" in Part II of this book.

Real-world applications involve real-world objects. You might be working on a system to route traffic in a network, to find the best way to schedule classrooms in a university, or to search for patterns in a corporate database. Most algorithms, however, are designed to work on rigorously defined abstract structures such as permutations, graphs, and sets. To exploit the algorithms literature, you must learn to describe your problem abstractly, in terms of procedures on such fundamental structures.

4.4.1 Combinatorial Objects

Odds are very good that others have probably stumbled upon any algorithmic problem you care about, perhaps in substantially different contexts. But to find

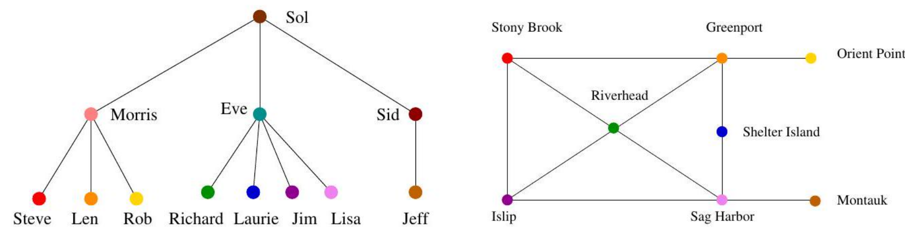


Figure 4.7: Modeling real-world structures with trees and graphs.

out what is known about your particular "widget optimization problem," you can't hope to find it in a book under widget. You must first formulate widget optimization in terms of computing properties of common structures such as those described below:

- *Permutations are arrangements, or orderings, of items. For example,* $\{1, 4, 3, 2\}$ *and* $\{4, 3, 2, 1\}$ *are two distinct permutations of the same set of four integers. We have already seen permutations in the robot optimization problem, and in sorting. Permutations are likely the object in question whenever your problem seeks an "arrangement," "tour," "ordering," or "sequence."*

- *Subsets represent selections from a set of items. For example,* $\{1, 3, 4\}$ *and* $\{2\}$ *are two distinct subsets of the first four integers. Order does not matter in subsets the way it does with permutations, so the subsets* $\{1, 3, 4\}$ *and* $\{4, 3, 1\}$ *would be considered identical. Subsets arose as candidate solutions in the movie scheduling problem. They are likely the object in question whenever your problem seeks a "cluster," "collection," "committee," "group," "packaging," or "selection."*

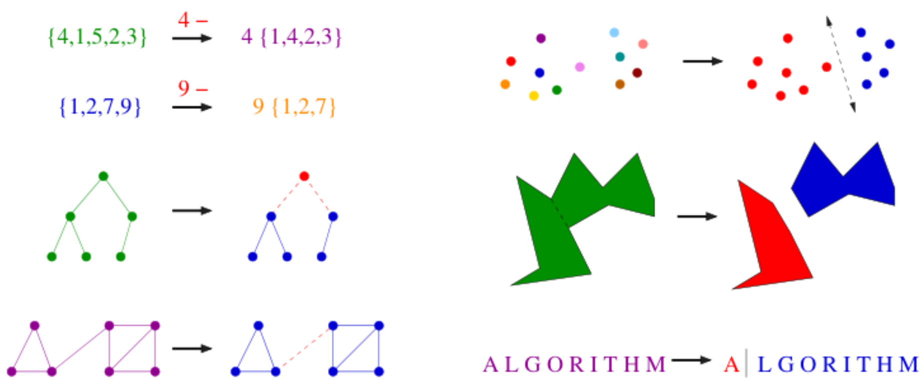
- *Trees represent hierarchical relationships between items. Figure 1.9 a) shows part of the family tree of the Skiena clan. Trees are likely the object in question whenever your problem seeks a "hierarchy," "dominance relationship," "ancestor/descendant relationship," or "taxonomy."*

- *Graphs represent relationships between arbitrary pairs of objects. Figure 1.9 (b) models a network of roads as a graph, where the vertices are cities and the edges are roads connecting pairs of cities. Graphs are likely the object in question whenever you seek a "network," "circuit," "web," or "relationship."*

- *Points define locations in some geometric space. For example, the locations of McDonald's restaurants can be described by points on a map/plane. Points are likely the object in question whenever your problems work on "sites," "positions," "data records," or "locations."*

- *Polygons define regions in some geometric spaces. For example, the borders of a country can be described by a polygon on a map/plane. Polygons and polyhedra are likely the object in question whenever you are working on "shapes," "regions," "configurations," or "boundaries."*

- *Strings represent sequences of characters, or patterns. For example, the names of students in a class can be represented by strings. Strings are likely the object in question whenever you are dealing with "text," "characters," "patterns," or "labels."*



These fundamental structures all have associated algorithm problems, which are presented in the catalog of Part II. Familiarity with these problems is important, because they provide the language we use to model applications. To become fluent in this vocabulary, browse through the catalog and study the input and output pictures for each problem. Understanding these problems, even at a cartoon/definition level, will enable you to know where to look later when the problem arises in your application.

Examples of successful application modeling will be presented in the war stories spaced throughout this book. However, some words of caution are in order. The act of modeling reduces your application to one of a small number of existing problems and structures. Such a process is inherently constraining, and certain details might not fit easily into the given target problem. Also, certain problems can be modeled in several different ways, some much better than others.

Modeling is only the first step in designing an algorithm for a problem. Be alert for how the details of your applications differ from a candidate model, but don't be too quick to say that your problem is unique and special. Temporarily ignoring details that don't fit can free the mind to ask whether they really were fundamental in the first place.

Take-Home Lesson: Modeling your application in terms of well-defined structures and algorithms is the most important single step towards a solution.

4.4.2 Recursive Objects

Learning to think recursively is learning to look for big things that are made from smaller things of exactly the same type as the big thing. If you think of houses as sets of rooms, then adding or deleting a room still leaves a house behind.

Recursive structures occur everywhere in the algorithmic world. Indeed, each of the abstract structures described above can be thought about recursively. You just have to see how you can break them down, as shown in Figure 1.10

- *Permutations - Delete the first element of a permutation of* n *things* $\{1, \ldots, n\}$ *and you get a permutation of the remaining* $n-1$ *things. This may require renumbering to keep the object a permutation of consecutive integers. For example, removing the first element of* $\{4, 1, 5, 2, 3\}$ *and*

ALGORITHM $\longrightarrow$ *A* | *LGORITHM*

Figure 1.10: Recursive decompositions of combinatorial objects. Permutations, subsets, trees, and graphs (left column). Point sets, polygons, and strings (right column). Note that the elements of a permutation of $\{1, \ldots, n\}$ *get renumbered after element deletion in order to remain a permutation of* $\{1, \ldots, n-1\}$*.*
renumbering gives $\{1, 4, 2, 3\}$*, a permutation of* $\{1, 2, 3, 4\}$*. Permutations are recursive objects.*

- *Subsets - Every subset of* $\{1, \ldots, n\}$ *contains a subset of* $\{1, \ldots, n-1\}$ *obtained by deleting element* n*, if it is present. Subsets are recursive objects.*

- *Trees - Delete the root of a tree and what do you get? A collection of smaller trees. Delete any leaf of a tree and what do you get? A slightly smaller tree. Trees are recursive objects.*

- *Graphs - Delete any vertex from a graph, and you get a smaller graph. Now divide the vertices of a graph into two groups, left and right. Cut through all edges that span from left to right, and what do you get? Two smaller graphs, and a bunch of broken edges. Graphs are recursive objects.*

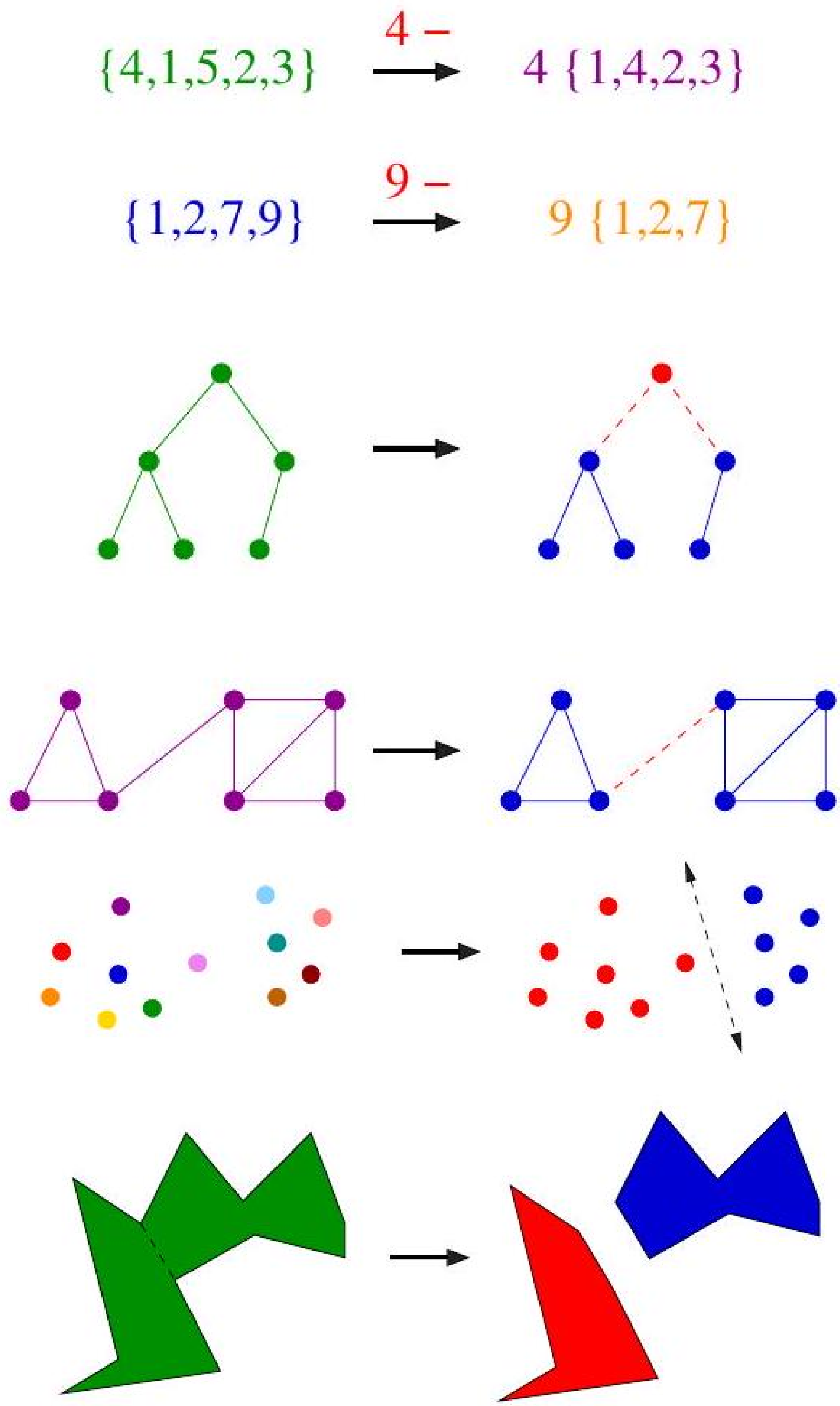
- *Points - Take a cloud of points, and separate them into two groups by drawing a line. Now you have two smaller clouds of points. Point sets are recursive objects.*

- *Polygons - Inserting any internal chord between two non-adjacent vertices of a simple polygon cuts it into two smaller polygons. Polygons are recursive objects.*

- *Strings - Delete the first character from a string, and what do you get? A shorter string. Strings are recursive objects.* [1]

Recursive descriptions of objects require both decomposition rules and basis cases, namely the specification of the smallest and simplest objects where the

[1] An alert reader of the previous edition observed that salads are recursive objects, in ways that hamburgers are not. Take a bite (or an ingredient) out of a salad, and what is left is a smaller salad. Take a bite out of a hamburger, and what is left is something disgusting.


4 –
{4,1,5,2,3}
4 {1,4,2,3}
9 –
{1,2,7,9}
9 {1,2,7}

decomposition stops. These basis cases are usually easily defined. Permutations and subsets of zero things presumably look like }. The smallest interesting tree or graph consists of a single vertex, while the smallest interesting point cloud consists of a single point. Polygons are a little trickier; the smallest genuine simple polygon is a triangle. Finally, the empty string has zero characters in it. The decision of whether the basis case contains zero or one element is more a question of taste and convenience than any fundamental principle.

Recursive decompositions will define many of the algorithms we will see in this book. Keep your eyes open for them.

4.5 Proof by Contradiction

Although some computer scientists only know how to prove things by induction, this isn't true of everyone. The best sometimes use contradiction.

The basic scheme of a contradiction argument is as follows:

- *Assume that the hypothesis (the statement you want to prove) is false.*
- *Develop some logical consequences of this assumption.*
- *Show that one consequence is demonstrably false, thereby showing that the assumption is incorrect and the hypothesis is true.*

The classic contradiction argument is Euclid's proof that there are an infinite number of prime numbers: integers n like $2, 3, 5, 7, 11, \ldots$ *that have no nontrivial factors, only 1 and n itself. The negation of the claim would be that there are only a finite number of primes, say* m*, which can be listed as* $p_1, \ldots, p_m$*. So let's assume this is the case and work with it.*

Prime numbers have particular properties with respect to division. Suppose we construct the integer formed as the product of "all" of the listed primes:

$$N = \prod_{i=1}^{m} p_i$$

This integer N *has the property that it is divisible by each and every one of the known primes, because of how it was built.*

But consider the integer $N + 1$*. It can't be divisible by* $p_1 = 2$*, because* N *is. The same is true for* $p_2 = 3$ *and every other listed prime. Since* $N + 1$ *doesn't have any non-trivial factor, this means it must be prime. But you asserted that there are exactly* m *prime numbers, none of which are* $N + 1$*. This assertion is false, so there cannot be a bounded number of primes. Touché!*

For a contradiction argument to be convincing, the final consequence must be clearly, ridiculously false. Muddy outcomes are not convincing. It is also important that this contradiction be a logical consequence of the assumption. We will see contradiction arguments for minimum spanning tree algorithms in Section 8.1

4.6 About the War Stories

The best way to learn how careful algorithm design can have a huge impact on performance is to look at real-world case studies. By carefully studying other people's experiences, we learn how they might apply to our work.

Scattered throughout this text are several of my own algorithmic war stories, presenting our successful (and occasionally unsuccessful) algorithm design efforts on real applications. I hope that you will be able to internalize these experiences so that they will serve as models for your own attacks on problems.

Every one of the war stories is true. Of course, the stories improve somewhat in the retelling, and the dialogue has been punched up to make them more interesting to read. However, I have tried to honestly trace the process of going from a raw problem to a solution, so you can watch how this process unfolded.

The Oxford English Dictionary defines an algorist as "one skillful in reckonings or figuring." In these stories, I have tried to capture some of the mindset of the algorist in action as they attack a problem.

The war stories often involve at least one problem from the problem catalog in Part II. I reference the appropriate section of the catalog when such a problem occurs. This emphasizes the benefits of modeling your application in terms of standard algorithm problems. By using the catalog, you will be able to pull out what is known about any given problem whenever it is needed.

4.7 War Story: Psychic Modeling

The call came for me out of the blue as I sat in my office.
"Professor Skiena, I hope you can help me. I'm the President of Lotto Systems Group Inc., and we need an algorithm for a problem arising in our latest product."
"Sure," I replied. After all, the dean of my engineering school is always encouraging our faculty to interact more with industry.
"At Lotto Systems Group, we market a program designed to improve our customers' psychic ability to predict winning lottery numbers. [2] *In a standard lottery, each ticket consists of six numbers selected from, say, 1 to 44. However, after proper training, our clients can visualize (say) fifteen numbers out of the 44 and be certain that at least four of them will be on the winning ticket. Are you with me so far?"*
"Probably not," I replied. But then I recalled how my dean encourages us to interact with industry.
"Our problem is this. After the psychic has narrowed the choices down to fifteen numbers and is certain that at least four of them will be on the winning ticket, we must find the most efficient way to exploit this information. Suppose a cash prize is awarded whenever you pick at least three of the correct numbers on your ticket. We need an algorithm to construct the smallest set of tickets that we must buy in order to guarantee that we win at least one prize."

Tickets	Winning Pairs			
123	12	2	3	3
145	13	2	4	35
245	14	2	5	45
345	15			

Figure 1.11: Covering all pairs of $\{1,2,3,4,5\}$ *with tickets* $\{1,2,3\},\{1,4,5\},\{2,4,5\},\{3,4,5\}$. *Pair color reflects the covering ticket.*

"Assuming the psychic is correct?"

"Yes, assuming the psychic is correct. We need a program that prints out a list of all the tickets that the psychic should buy in order to minimize their investment. Can you help us?"

Maybe they did have psychic ability, for they had come to the right place. Identifying the best subset of tickets to buy was very much a combinatorial algorithm problem. It was going to be some type of covering problem, where each ticket bought would "cover" some of the possible 4-element subsets of the psychic's set. Finding the absolute smallest set of tickets to cover everything was a special instance of the NP-complete problem set cover (discussed in Section 21.1 (page 678), and presumably computationally intractable.

It was indeed a special instance of set cover, completely specified by only four numbers: the size n *of the candidate set* S *(typically* $n \approx 15$ *), the number of slots* k *for numbers on each ticket (typically* $k \approx 6$ *), the number of psychicallypromised correct numbers* j *from* S *(say* $j = 4$ *), and finally, the number of matching numbers* l *necessary to win a prize (say* $l = 3$ *). Figure 1.11 illustrates a covering of a smaller instance, where* $n = 5, k = 3$, *and* $l = 2$, *and no psychic contribution (meaning* $j = 5$ *).*

"Although it will be hard to find the exact minimum set of tickets to buy, with heuristics I should be able to get you pretty close to the cheapest covering ticket set," I told him. "Will that be good enough?"

"So long as it generates better ticket sets than my competitor's program, that will be fine. His system doesn't always guarantee a win. I really appreciate your help on this, Professor Skiena."

"One last thing. If your program can train people to pick lottery winners, why don't you use it to win the lottery yourself?"

"I look forward to talking to you again real soon, Professor Skiena. Thanks for the help."

I hung up the phone and got back to thinking. It seemed like the perfect project to give to a bright undergraduate. After modeling it in terms of sets and subsets, the basic components of a solution seemed fairly straightforward:

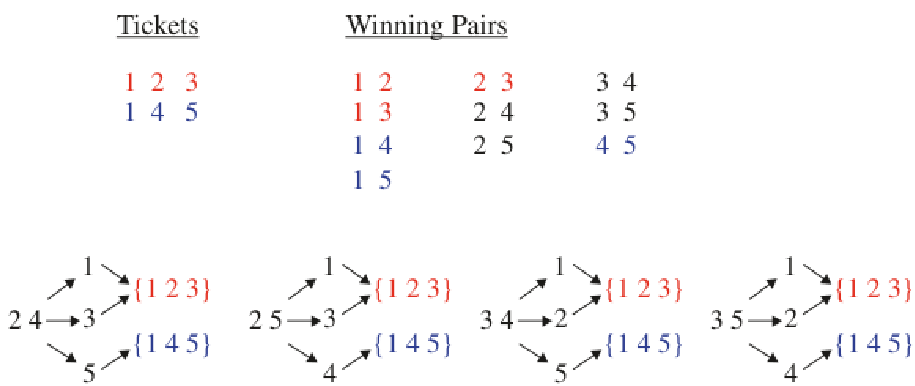
- *We needed the ability to generate all subsets of* k *numbers from the candidate set* S. *Algorithms for generating and ranking/unranking subsets of sets are presented in Section 17.5 (page 521 .*

[2] Yes, this is a true story.

- *We needed the right formulation of what it meant to have a covering set of purchased tickets. The obvious criteria would be to pick a small set of tickets such that we have purchased at least one ticket containing each of the* $\binom{n}{l}$ *l-subsets of S that might pay off with the prize.*

- *We needed to keep track of which prize combinations we have thus far covered. We seek tickets to cover as many thus-far-uncovered prize combinations as possible. The currently covered combinations are a subset of all possible combinations. Data structures for subsets are discussed in Section 15.5 (page 456). The best candidate seemed to be a bit vector, which would answer in constant time "is this combination already covered?"*

- *We needed a search mechanism to decide which ticket to buy next. For small enough set sizes, we could do an exhaustive search over all possible subsets of tickets and pick the smallest one. For larger problems, a randomized search process like simulated annealing (see Section 12.6.3 (page 406) would select tickets-to-buy to cover as many uncovered combinations as possible. By repeating this randomized procedure several times and picking the best solution, we would be likely to come up with a good set of tickets.*



The bright undergraduate, Fayyaz Younas, rose to the challenge. Based on this framework, he implemented a brute-force search algorithm and found optimal solutions for problems with $n \leq 5$ in a reasonable time. He implemented a random search procedure to solve larger problems, tweaking it for a while before settling on the best variant. Finally, the day arrived when we could call Lotto Systems Group and announce that we had solved the problem.
"Our program found an optimal solution for $n = 15, k = 6, j = 4, l = 3$ meant buying 28 tickets."
"Twenty-eight tickets!" complained the president. "You must have a bug. Look, these five tickets will suffice to cover everything twice over: $\{2, 4, 8, 10, 13, 14\}$, $\{4, 5, 7, 8, 12, 15\}$, $\{1, 2, 3, 6, 11, 13\}$, $\{3, 5, 6, 9, 10, 15\}$, $\{1, 7, 9, 11, 12, 14\}$." We fiddled with this example for a while before admitting that he was right.

We hadn't modeled the problem correctly! In fact, we didn't need to explicitly cover all possible winning combinations. Figure 1.12 illustrates the principle by giving a two-ticket solution to our previous four-ticket example. Although the pairs $\{2,4\},\{2,5\},\{3,4\}$, or $\{3,5\}$ do not explicitly appear in one of our two tickets, these pairs plus any possible third ticket number must create a pair in either $\{1,2,3\}$ or $\{1,4,5\}$. We were trying to cover too many combinations, and the penny-pinching psychics were unwilling to pay for such extravagance.

Fortunately, this story has a happy ending. The general outline of our search-based solution still holds for the real problem. All we must fix is which subsets we get credit for covering with a given set of tickets. After this modification, we obtained the kind of results they were hoping for. Lotto Systems Group gratefully accepted our program to incorporate into their product, and we hope they hit the jackpot with it.

Tickets Winning Pairs
123 1 2 2 3 4
14 5

The moral of this story is to make sure that you model your problem correctly before trying to solve it. In our case, we came up with a reasonable model, but didn't work hard enough to validate it before we started to program. Our misinterpretation would have become obvious had we worked out a small example by hand and bounced it off our sponsor before beginning work. Our success in recovering from this error is a tribute to the basic correctness of our initial formulation, and our use of well-defined abstractions for such tasks as (1) ranking/unranking k-subsets, (2) the set data structure, and (3) combinatorial search.

4.8 Estimation

When you don't know the right answer, the best thing to do is guess. Principled guessing is called estimation. The ability to make back-of-the-envelope estimates of diverse quantities such as the running time of a program is a valuable skill in algorithm design, as it is in any technical enterprise.

Estimation problems are best solved through some kind of logical reasoning process, typically a mix of principled calculations and analogies. Principled calculations give the answer as a function of quantities that either you already know, can look up on Google, or feel confident enough to guess. Analogies reference your past experiences, recalling those that seem similar to some aspect of the problem at hand.

I once asked my class to estimate the number of pennies in a hefty glass jar, and got answers ranging from 250 to 15,000 . Both answers will seem pretty silly if you make the right analogies:

- A penny roll holds 50 coins in a tube roughly the length and width of your biggest finger. So five such rolls can easily be held in your hand, with no need for a hefty jar.

- *15,000 pennies means* \$150 *in value. I have never managed to accumulate that much value in coins even in a hefty pile of change - and here I am only using pennies!*

But the class average estimate proved very close to the right answer, which turned out to be 1879. There are at least three principled ways I can think of to estimate the number of coins in the jar:

- *Volume - The jar was a cylinder with about 5 inches diameter, and the coins probably reached a level equal to about ten pennies tall stacked end to end. Figure a penny is ten times longer than it is thick. The bottom layer of the jar was a circle of radius about five pennies. So*

$$(10 \times 10) \times \left(\pi \times 2.5^2\right) \approx 1962.5$$

- *Weight - Lugging the jar felt like carrying around a bowling ball. Multiplying the number of US pennies in a pound (181 when I looked it up) times a 10 lb . ball gave a frighteningly accurate estimate of 1810.*

- *Analogy - The coins had a height of about 8 inches in the jar, or twice that of a penny roll. Figure I could stack about two layers of ten rolls per layer in the jar, or a total estimate of 1,000 coins.*

A best practice in estimation is to try to solve the problem in different ways and see if the answers generally agree in magnitude. All of these are within a factor of two of each other, giving me confidence that my answer is about right.

Try some of the estimation exercises at the end of this chapter, and see how many different ways you can approach them. If you do things right, the ratio between your high and low estimates should be somewhere within a factor of two to ten, depending upon the nature of the problem. A sound reasoning process matters a lot more here than the actual numbers you get.

***Chapter Notes** Every algorithm book reflects the design philosophy of its author. For students seeking alternative presentations and viewpoints, I particularly recommend the books of Cormen, et al.*
citecormen2009introduction, Kleinberg and Tardos [
citekleinberg2006algorithm], Manber
citemanber1989introduction, and Roughgarden
citeroughgarden2017algorithms.

Formal proofs of algorithm correctness are important, and deserve a fuller discussion than this chapter is able to provide. See Gries
citegries1989science] for a thorough introduction to the techniques of program verification.

The movie scheduling problem represents a very special case of the general independent set problem, which will be discussed in Section 19.2 (page 589 . The restriction limits the allowable input instances to interval graphs, where the vertices of the graph G can be represented by intervals on the line and (i, j) is *an edge of G iff the intervals overlap. Golumbic citegolumbic2004algorithmic provides a full treatment of this interesting and important class of graphs.*

Jon Bentley's Programming Pearls columns are probably the best known collection of algorithmic "war stories," collected in two books citeben90 citebentley1999programming. Brooks's The Mythical Man Month citebrooks1995mythical] is another wonderful collection of war stories that, although focused more on software engineering than algorithm design, remain a source of considerable wisdom. Every programmer should read these books for pleasure as well as insight.

Our solution to the lotto ticket set covering problem is presented in more detail in Younas and Skiena citeyounas1996randomized.

4.9 Exercises

Finding Counterexamples 1-1. [3] Show that $a + b$ *can be less than* $\min(a, b)$.
$1 - 2$. *[3] Show that* $a \times b$ *can be less than* $\min(a, b)$.
1-3. [5] Design/draw a road network with two points a and b such that the fastest route between a and b is not the shortest route.
1-4. [5] Design/draw a road network with two points a and b such that the shortest route between a and b is not the route with the fewest turns.
1-5. [4] The knapsack problem is as follows: given a set of integers $S = \{s_1, s_2, \ldots, s_n\}$, *and a target number* T, *find a subset of* S *that adds up exactly to* T. *For example, there exists a subset within* $S = \{1, 2, 5, 9, 10\}$ *that adds up to* $T = 22$ *but not* $T = 23$.
Find counterexamples to each of the following algorithms for the knapsack problem. That is, give an S and T where the algorithm does not find a solution that leaves the knapsack completely full, even though a full-knapsack solution exists.
(a) Put the elements of S in the knapsack in left to right order if they fit, that is, the first-fit algorithm.
(b) Put the elements of S in the knapsack from smallest to largest, that is, the best-fit algorithm.
(c) Put the elements of S in the knapsack from largest to smallest.

1-6. [5] The set cover problem is as follows: given a set S *of subsets* $S_1, \ldots, S_m$ *of the universal set* $U = \{1, \ldots, n\}$, *find the smallest subset of subsets* $T \subseteq S$ *such that* $\cup_{t_i \in T} t_i = U$. *For example, consider the subsets* $S_1 = \{1, 3, 5\}, S_2 = \{2, 4\}$, $S_3 = \{1, 4\}$, *and* $S_4 = \{2, 5\}$. *The set cover of* $\{1, \ldots, 5\}$ *would then be* S_1 *and* S_2.
Find a counterexample for the following algorithm: Select the largest subset for the cover, and then delete all its elements from the universal set. Repeat by adding the subset containing the largest number of uncovered elements until all are covered.
1-7. [5] The maximum clique problem in a graph $G = (V, E)$ *asks for the*

largest subset C of vertices V such that there is an edge in E between every pair of vertices in C. Find a counterexample for the following algorithm: Sort the vertices of G from highest to lowest degree. Considering the vertices in order

of degree, for each vertex add it to the clique if it is a neighbor of all vertices currently in the clique. Repeat until all vertices have been considered.

Proofs of Correctness 1-8. [3] Prove the correctness of the following recursive algorithm to multiply two natural numbers, for all integer constants $c \geq 2$.

```
$\operatorname{Multiply}(y, z)$
if $z=0$ then return(0) else
    return(Multiply $(c y,\lfloor z / c\rfloor)+y \cdot(z \bmod c))$
```

1-9. [3] Prove the correctness of the following algorithm for evaluating a polynomial $a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$.

Horner (a, x)

$p = a_n$
for i from $n-1$ to 0

$$p = p \cdot x + a_i$$

return p
1-10. [3] Prove the correctness of the following sorting algorithm.
Bubblesort (A)

for i from n to 1
for j from 1 to $i-1$
if $(A[j] > A[j+1])$
swap the values of $A[j]$ and $A[j+1]$

1-11. [5] The greatest common divisor of positive integers x and y is the largest integer d such that d divides x and d divides y. Euclid's algorithm to compute $\gcd(x, y)$ where $x > y$ reduces the task to a smaller problem:

$$\gcd(x, y) = \gcd(y, x \bmod y)$$

Prove that Euclid's algorithm is correct.

Induction 1-12. [3] Prove that $\sum_{i=1}^{n} i = n(n+1)/2$ for $n \geq 0$, by induction.
1-13. [3] Prove that $\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$ for $n \geq 0$, by induction.
1-14. [3] Prove that $\sum_{i=1}^{n} i^3 = n^2(n+1)^2/4$ for $n \geq 0$, by induction.
1-15. [3] Prove that

$$\sum_{i=1}^{n} i(i+1)(i+2) = n(n+1)(n+2)(n+3)/4$$

1-16. [5] Prove by induction on $n \geq 1$ that for every $a \neq 1$,

$$\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$$

1-17. [3] Prove by induction that for $n \geq 1$,

$$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \frac{n}{n+1}$$

1-18. [3] Prove by induction that $n^3 + 2n$ is divisible by 3 for all $n \geq 0$.
1-19. [3] Prove by induction that a tree with n vertices has exactly $n-1$ edges.
1-20. [3] Prove by induction that the sum of the cubes of the first n positive integers is equal to the square of the sum of these integers, that is,

$$\sum_{i=1}^{n} i^3 = \left(\sum_{i=1}^{n} i\right)^2$$

Estimation 1-21. [3] Do all the books you own total at least one million pages? How many total pages are stored in your school library?

1-22. [3] How many words are there in this textbook?
1-23. [3] How many hours are one million seconds? How many days? Answer these questions by doing all arithmetic in your head.

1-24. [3] Estimate how many cities and towns there are in the United States.
1-25. [3] Estimate how many cubic miles of water flow out of the mouth of the Mississippi River each day. Do not look up any supplemental facts. Describe all assumptions you made in arriving at your answer.
1-26. [3] How many Starbucks or McDonald's locations are there in your country?
1-27. [3] How long would it take to empty a bathtub with a drinking straw?
1-28. [3] Is disk drive access time normally measured in milliseconds (thousandths of a second) or microseconds (millionths of a second)? Does your RAM memory access a word in more or less than a microsecond? How many instructions can your CPU execute in one year if the machine is left running all the time?
1-29. [4] A sorting algorithm takes 1 second to sort 1,000 items on your machine. How long will it take to sort 10,000 items...
(a) if you believe that the algorithm takes time proportional to n^2, and
(b) if you believe that the algorithm takes time roughly proportional to $n \log n$?

Implementation Projects 1-30. [5] Implement the two TSP heuristics of Section 1.1 (page 5. Which of them gives better solutions in practice? Can you devise a heuristic that works better than both of them?

1-31. [5] Describe how to test whether a given set of tickets establishes sufficient coverage in the Lotto problem of Section 1.8 (page 22). Write a program to find good ticket sets.

*Interview Problems 1-32. [5] Write a function to perform integer division without using either the / or * operators. Find a fast way to do it.*

1-33. [5] There are twenty-five horses. At most, five horses can race together at a time. You must determine the fastest, second fastest, and third fastest horses. Find the minimum number of races in which this can be done.

1-34. [3] How many piano tuners are there in the entire world?

1-35. [3] How many gas stations are there in the United States?

1-36. [3] How much does the ice in a hockey rink weigh?

1-37. [3] How many miles of road are there in the United States?

1-38. [3] On average, how many times would you have to flip open the Manhattan phone book at random in order to find a specific name?

LeetCode *1-1. https://leetcode.com/problems/daily-temperatures/*

1-2. https://leetcode.com/problems/rotate-list/

1-3. https://leetcode.com/problems/wiggle-sort-ii/

HackerRank *1-1. https://www.hackerrank.com/challenges/array-left-rotation/*

1-2. https://www.hackerrank.com/challenges/kangaroo/

1-3. https://www.hackerrank.com/challenges/hackerland-radio-transmitters/

Programming Challenges *These programming challenge problems with robot judging are available at https://onlinejudge.org*

1-1. "The $3n+1$ *Problem"-Chapter 1, problem 100.*

1-2. "The Trip"-Chapter 1, problem 10137.

1-3. "Australian Voting"-Chapter 1, problem 10142.

Chapter 2

Chapter 5

Algorithm Analysis

Algorithms are the most important and durable part of computer science because they can be studied in a language- and machine-independent way. This means we need techniques that let us compare the efficiency of algorithms without implementing them. Our two most important tools are (1) the RAM model of computation and (2) the asymptotic analysis of computational complexity.

Assessing algorithmic performance makes use of the "Big Oh" notation that proves essential to compare algorithms, and design better ones. This method of keeping score will be the most mathematically demanding part of this book. But once you understand the intuition behind this formalism it becomes a lot easier to deal with.

5.1 The RAM Model of Computation

Machine-independent algorithm design depends upon a hypothetical computer called the Random Access Machine, or RAM. Under this model of computation, we are confronted with a computer where:

- *Each simple operation* ($+, ^*, -, =$, *if, call) takes exactly one time step.*
- *Loops and subroutines are not considered simple operations. Instead, they are the composition of many single-step operations. It makes no sense for sort to be a single-step operation, since sorting 1,000,000 items will certainly take much longer than sorting ten items. The time it takes to run through a loop or execute a subprogram depends upon the number of loop iterations or the specific nature of the subprogram.*
- *Each memory access takes exactly one time step. Furthermore, we have as much memory as we need. The RAM model takes no notice of whether an item is in cache or on the disk.*

Under the RAM model, we measure run time by counting the number of steps an algorithm takes on a given problem instance. If we assume that our

RAM executes a given number of steps per second, this operation count converts naturally to the actual running time.

The RAM is a simple model of how computers perform. Perhaps it sounds too simple. After all, multiplying two numbers takes more time than adding two numbers on most processors, which violates the first assumption of the model. Fancy compiler loop unrolling and hyperthreading may well violate the second assumption. And certainly memory-access times differ greatly depending on where your data sits in the storage hierarchy. This makes us zero for three on the truth of our basic assumptions.

And yet, despite these objections, the RAM proves an excellent model for understanding how an algorithm will perform on a real computer. It strikes a fine balance by capturing the essential behavior of computers while being simple to work with. We use the RAM model because it is useful in practice.

Every model in science has a size range over which it is useful. Take, for example, the model that the Earth is flat. You might argue that this is a bad model, since it is quite well established that the Earth is round. But, when laying the foundation of a house, the flat Earth model is sufficiently accurate that it can be reliably used. It is so much easier to manipulate a flat-Earth model that it is inconceivable that you would try to think spherically when you don't have to 1

The same situation is true with the RAM model of computation. We make an abstraction that is generally very useful. It is difficult to design an algorithm where the RAM model gives you substantially misleading results. The robustness of this model enables us to analyze algorithms in a machine-independent way.

Take-Home Lesson: Algorithms can be understood and studied in a language- and machine-independent manner.

5.1.1 Best-Case, Worst-Case, and Average-Case Complexity

Using the RAM model of computation, we can count how many steps our algorithm takes on any given input instance by executing it. However, to understand how good or bad an algorithm is in general, we must know how it works over all possible instances.

To understand the notions of the best, worst, and average-case complexity, think about running an algorithm over all possible instances of data that can be fed to it. For the problem of sorting, the set of possible input instances includes every possible arrangement of n keys, for all possible values of n. We can represent each input instance as a point on a graph (shown in Figure 2.1) where the x-axis represents the size of the input problem (for sorting, the number of items to sort), and the y-axis denotes the number of steps taken by the algorithm in this instance.

[1] The Earth is not completely spherical either, but a spherical Earth provides a useful model for such things as longitude and latitude.

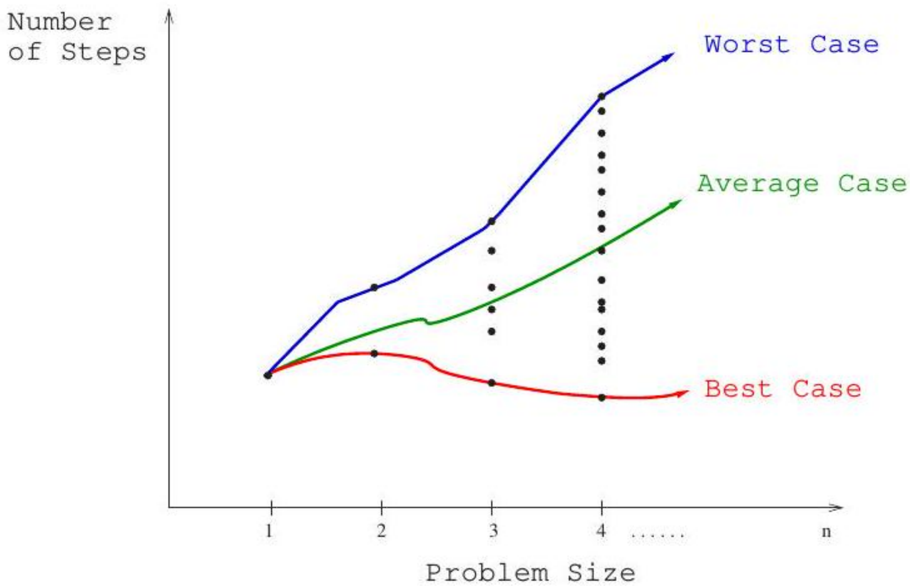


Figure 5.1: Best-case, worst-case, and average-case complexity.

These points naturally align themselves into columns, because only integers represent possible input sizes (e.g., it makes no sense to sort 10.57 items). We can define three interesting functions over the plot of these points:

- *The worst-case complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n. This represents the curve passing through the highest point in each column.*
- *The best-case complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n. This represents the curve passing through the lowest point of each column.*
- *The average-case complexity or expected time of the algorithm, which is the function defined by the average number of steps over all instances of size n.*

The worst-case complexity generally proves to be most useful of these three measures in practice. Many people find this counterintuitive. To illustrate why, try to project what will happen if you bring \$$n$ into a casino to gamble. The best case, that you walk out owning the place, is so unlikely that you should not even think about it. The worst case, that you lose all \$$n$, is easy to calculate and distressingly likely to happen.

The average case, that the typical bettor loses 87.32% *of the money that he or she brings to the casino, is both difficult to establish and its meaning subject to debate. What exactly does average mean? Stupid people lose more than smart* *people, so are you smarter or stupider than the average person, and by how much? Card counters at blackjack do better on average than customers who accept three or more free drinks. We avoid all these complexities and obtain a very useful result by considering the worst case.*

That said, average-case analysis for expected running time will prove very important with respect to randomized algorithms, which use random numbers to make decisions within the algorithm. If you make n independent \$1 redblack bets on roulette in the casino, your expected loss is indeed well defined at \$$(2n/38)$, because American roulette wheels have eighteen red, eighteen black, and two green slots 0 and 00 where every bet loses.

Take-Home Lesson: Each of these time complexities defines a numerical function for any given algorithm, representing running time as a function of input size. These functions are just as well defined as any other numerical function, be it $y = x^2 - 2x + 1$ or the price of Alphabet stock as a function of time. But time complexities are such complicated functions that we must simplify them for analysis using the "Big Oh" notation.

5.2 The Big Oh Notation

The best-case, worst-case, and average-case time complexities for any given algorithm are numerical functions over the size of possible problem instances. However, it is very difficult to work precisely with these functions, because they tend to:

- *Have too many bumps - An algorithm such as binary search typically runs a bit faster for arrays of size exactly $n = 2^k - 1$ (where k is an integer), because the array partitions work out nicely. This detail is not particularly important, but it warns us that the exact time complexity function for any algorithm is liable to be very complicated, with lots of little up and down bumps as shown in Figure 2.2*
- *Require too much detail to specify precisely - Counting the exact number of RAM instructions executed in the worst case requires the algorithm be specified to the detail of a complete computer program. Furthermore, the precise answer depends upon uninteresting coding details (e.g. did the code use a case statement or nested ifs?). Performing a precise worst-case analysis like*

$$T(n) = 12754n^2 + 4353n + 834 \lg_2 n + 13546$$

would clearly be very difficult work, but provides us little extra information than the observation that "the time grows quadratically with n."

It proves to be much easier to talk in terms of simple upper and lower bounds of time-complexity functions using the Big Oh notation. The Big Oh simplifies our analysis by ignoring levels of detail that do not impact our comparison of algorithms.

The Big Oh notation ignores the difference between multiplicative constants. The functions $f(n) = 2n$ and $g(n) = n$ are identical in Big Oh analysis. This makes sense given our application. Suppose a given algorithm in (say) C language ran twice as fast as one with the same algorithm written in Java. This multiplicative factor of two can tell us nothing about the algorithm itself, because both programs implement exactly the same algorithm. We should ignore such constant factors when comparing two algorithms.

The formal definitions associated with the Big Oh notation are as follows:

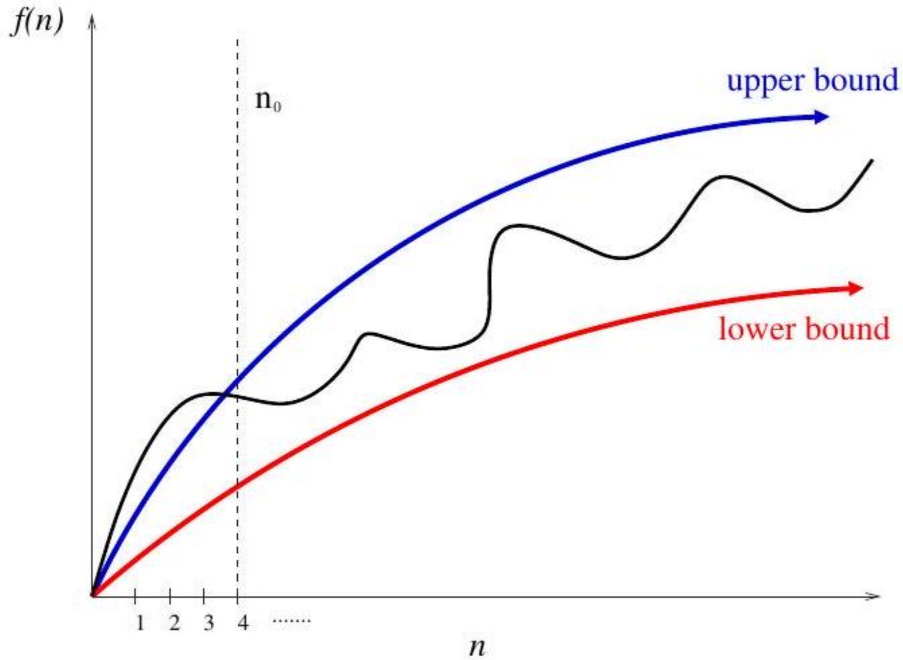


Figure 5.2: Upper and lower bounds valid for $n > n_0$ smooth out the behavior of complex functions.

Figure 2.3: Illustrating notations: (a) $f(n) = O(g(n))$, (b) $f(n) = \Omega(g(n))$, and (c) $f(n) = \Theta(g(n))$.

- *$f(n) = O(g(n))$ means $c \cdot g(n)$ is an upper bound on $f(n)$. Thus, there exists some constant c such that $f(n) \leq c \cdot g(n)$ for every large enough n (that is, for all $n \geq n_0$, for some constant n_0).*
- *$f(n) = \Omega(g(n))$ means $c \cdot g(n)$ is a lower bound on $f(n)$. Thus, there exists some constant c such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.*

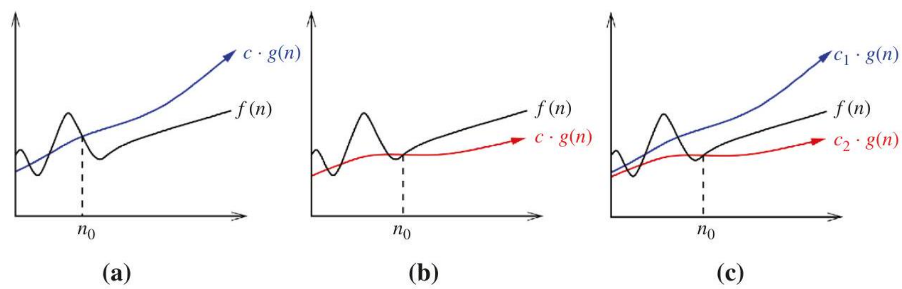


Figure 5.3: Upper and lower bounds valid for $n > n_0$ smooth out the behavior of complex functions.

- *$f(n) = \Theta(g(n))$ means $c_1 \cdot g(n)$ is an upper bound on $f(n)$ and $c_2 \cdot g(n)$ is a lower bound on $f(n)$, for all $n \geq n_0$. Thus, there exist constants c_1 and c_2 such that $f(n) \leq c_1 \cdot g(n)$ and $f(n) \geq c_2 \cdot g(n)$ for all $n \geq n_0$. This means that $g(n)$ provides a nice, tight bound on $f(n)$.*

Got it? These definitions are illustrated in Figure 2.3 Each of these definitions assumes there is a constant n_0 beyond which they are satisfied. We are not concerned about small values of n, anything to the left of n_0. After all, we don't really care whether one sorting algorithm sorts six items faster than

another, but we do need to know which algorithm proves faster when sorting 10,000 or $1,000,000$ *items. The Big Oh notation enables us to ignore details and focus on the big picture.*

Take-Home Lesson: The Big Oh notation and worst-case analysis are tools that greatly simplify our ability to compare the efficiency of algorithms.

Make sure you understand this notation by working through the following examples. Certain constants (c and n_0) are chosen in the explanations below because they work and make a point, but other pairs of constants will do exactly the same job. You are free to choose any constants that maintain the same inequality (ideally constants that make it obvious that the inequality holds):

$f(n) = 3n^2 - 100n + 6 = O\left(n^2\right)$, *because for* $c = 3, 3n^2 > f(n)$;

$f(n) = 3n^2 - 100n + 6 = O\left(n^3\right)$, *because for* $c = 1, n^3 > f(n)$ *when* $n > 3$;

$f(n) = 3n^2 - 100n + 6 \neq O(n)$, *because for any* $c > 0, cn < f(n)$ *when* $n > (c + 100)/3$,

since $n > (c + 100)/3 \Rightarrow 3n > c + 100 \Rightarrow 3n^2 > cn + 100n > cn + 100n - 6$
$\Rightarrow 3n^2 - 100n + 6 = f(n) > cn$;

$f(n) = 3n^2 - 100n + 6 = \Omega\left(n^2\right)$, *because for* $c = 2, 2n^2 < f(n)$ *when* $n > 100$;

$f(n) = 3n^2 - 100n + 6 \neq \Omega\left(n^3\right)$, *because for any* $c > 0, f(n) < c \cdot n^3$ *when* $n > 3/c + 3$;

$f(n) = 3n^2 - 100n + 6 = \Omega(n)$, *because for any* $c > 0, f(n) < 3n^2 + 6n^2 = 9n^2$, *which is* $< cn^3$ *when* $n > \max(9/c, 1)$;

$f(n)=3n^2-100n+6=\Theta\left(n^2\right)$, because both O and Ω apply;
$f(n)=3n^2-100n+6\neq\Theta\left(n^3\right)$, because only O applies;
$f(n)=3n^2-100n+6\neq\Theta(n)$, because only Ω applies.

The Big Oh notation provides for a rough notion of equality when comparing functions. It is somewhat jarring to see an expression like $n^2=O\left(n^3\right)$, but its meaning can always be resolved by going back to the definitions in terms of upper and lower bounds. It is perhaps most instructive to read the "=" here as meaning one of the functions that are. Clearly, n^2 is one of the functions that are $O\left(n^3\right)$.

***Stop and Think: Back to the Definition** Problem: Is $2^{n+1}=\Theta\left(2^n\right)$?*

Solution: Designing novel algorithms requires cleverness and inspiration. However, applying the Big Oh notation is best done by swallowing any creative instincts you may have. All Big Oh problems can be correctly solved by going back to the definition and working with that.

- *Is $2^{n+1}=O\left(2^n\right)$? Well, $f(n)=O(g(n))$ iff there exists a constant c such that for all sufficiently large $n, f(n)\leq c\cdot g(n)$. Is there? Yes, because $2^{n+1}=2\cdot 2^n$, and clearly $2\cdot 2^n\leq c\cdot 2^n$ for any $c\geq 2$.*

- *Is $2^{n+1}=\Omega\left(2^n\right)$? Go back to the definition. $f(n)=\Omega(g(n))$ iff there exists a constant $c>0$ such that for all sufficiently large $nf(n)\geq c\cdot g(n)$. This would be satisfied for any $0<c\leq 2$. Together the Big Oh and Ω bounds imply $2^{n+1}=\Theta\left(2^n\right)$.*

***Stop and Think: Hip to the Squares?** Problem: Is $(x+y)^2=O\left(x^2+y^2\right)$?*

Solution: Working with the Big Oh means going back to the definition at the slightest sign of confusion. By definition, this expression is valid iff we can find some c such that $(x+y)^2\leq c\left(x^2+y^2\right)$ for all sufficiently large x and y.

My first move would be to expand the left side of the equation, that is, $(x+y)^2=x^2+2xy+y^2$. If the middle $2xy$ term wasn't there, the inequality would clearly hold for any $c>1$. But it is there, so we need to relate $2xy$ to x^2+y^2. What if $x\leq y$? Then $2xy\leq 2y^2\leq 2\left(x^2+y^2\right)$. What if $x\geq y$? Then $2xy\leq 2x^2\leq 2\left(x^2+y^2\right)$. Either way, we now can bound $2xy$ by two times the right-side function x^2+y^2. This means that $(x+y)^2\leq 3\left(x^2+y^2\right)$, and so the result holds.

5.3 Growth Rates and Dominance Relations

With the Big Oh notation, we cavalierly discard the multiplicative constants. Thus, the functions $f(n)=0.001n^2$ and $g(n)=1000n^2$ are treated identically, even though $g(n)$ is a million times larger than $f(n)$ for all values of n.

The reason why we are content with such coarse Big Oh analysis is provided by Figure 2.4, which shows the growth rate of several common time analysis functions. In particular, it shows how long algorithms that use $f(n)$ operations *take to run on a fast computer, where each operation costs one nanosecond (10^{-9} seconds). The following conclusions can be drawn from this table:*

- *All such algorithms take roughly the same time for $n = 10$.*
- *Any algorithm with n ! running time becomes useless for $n \geq 20$.*

n	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10	0.003μ s	0.01μ s	0.033μ s	0.1μ s	1μ s	*3.63 ms*
20	0.004μ s	0.02μ s	0.086μ s	0.4μ s	*1 ms*	*77.1 years*
30	0.005μ s	0.03μ s	0.147μ s	0.9μ s	*1 sec*	8.4×10^{15}yrs
40	0.005μ s	0.04μ s	0.213μ s	1.6μ s	*18.3 min*	
50	0.006μ s	0.05μ s	0.282μ s	2.5μ s	*13 days*	
100	0.007μ s	0.1μ s	0.644μ s	10μ s	4×10^{13}yrs	
1,000	0.010μ s	1.00μ s	9.966μ s	*1 ms*		
10,000	0.013μ s	10μ s	130μ s	*100 ms*		
100,000	0.017μ s	*0.10 ms*	*1.67 ms*	*10 sec*		
$1,000,000$	0.020μ s	*1 ms*	*19.93 ms*	*16.7 min*		
$10,000,000$	0.023μ s	*0.01 sec*	*0.23 sec*	*1.16 days*		
$100,000,000$	0.027μ s	*0.10 sec*	*2.66 sec*	*115.7 days*		
$1,000,000,000$	0.030μ s	*1 sec*	*29.90 sec*	*31.7 years*		

Figure 2.4: Running times of common functions measured in nanoseconds. The function $\lg n$ denotes the base- 2 logarithm of n.

- *Algorithms whose running time is 2^n have a greater operating range, but become impractical for $n > 40$.*
- *Quadratic-time algorithms, whose running time is n^2, remain usable up to about $n = 10,000$, but quickly deteriorate with larger inputs. They are likely to be hopeless for $n > 1,000,000$.*
- *Linear-time and $n \lg n$ algorithms remain practical on inputs of one billion items.*
- *An $O(\lg n)$ algorithm hardly sweats for any imaginable value of n.*

The bottom line is that even ignoring constant factors, we get an excellent idea of whether a given algorithm is appropriate for a problem of a given size.

5.3.1 Dominance Relations

The Big Oh notation groups functions into a set of classes, such that all the functions within a particular class are essentially equivalent. Functions $f(n) = 0.34n$ and $g(n) = 234,234n$ belong in the same class, namely those that are order $\Theta(n)$. Furthermore, when two functions f and g belong to different classes, they are different with respect to our notation, meaning either $f(n) = O(g(n))$ or $g(n) = O(f(n))$, but not both.

We say that a faster growing function dominates a slower growing one, just as a faster growing company eventually comes to dominate the laggard. When f *and* g *belong to different classes (i.e.* $f(n) \neq \Theta(g(n))$ *), we say* g *dominates* f *when* $f(n) = O(g(n))$*. This is sometimes written* $g \gg f$.

The good news is that only a few different function classes tend to occur in the course of basic algorithm analysis. These suffice to cover almost all the algorithms we will discuss in this text, and are listed in order of increasing dominance:

- *Constant functions,* $f(n) = 1$ *: Such functions might measure the cost of adding two numbers, printing out "The Star Spangled Banner," or the growth realized by functions such as* $f(n) = \min(n, 100)$*. In the big picture, there is no dependence on the parameter* n.

- *Logarithmic functions,* $f(n) = \log n$ *: Logarithmic time complexity shows up in algorithms such as binary search. Such functions grow quite slowly as* n *gets big, but faster than the constant function (which is standing still, after all). Logarithms will be discussed in more detail in Section 2.7 (page 48).*

- *Linear functions,* $f(n) = n$ *: Such functions measure the cost of looking at each item once (or twice, or ten times) in an* n*-element array, say to identify the biggest item, the smallest item, or compute the average value.*

- *Superlinear functions,* $f(n) = n \lg n$ *: This important class of functions arises in such algorithms as quicksort and mergesort. They grow just a little faster than linear (recall Figure 2.4, but enough so to rise to a higher dominance class.*

- *Quadratic functions,* $f(n) = n^2$ *: Such functions measure the cost of looking at most or all pairs of items in an* n*-element universe. These arise in algorithms such as insertion sort and selection sort.*

- *Cubic functions,* $f(n) = n^3$ *: Such functions enumerate all triples of items in an* n*-element universe. These also arise in certain dynamic programming algorithms, to be developed in Chapter 10*

- *Exponential functions,* $f(n) = c^n$ *for a given constant* $c > 1$ *: Functions like* 2^n *arise when enumerating all subsets of* n *items. As we have seen, exponential algorithms become useless fast, but not as fast as...*

- *Factorial functions,* $f(n) = n$ *!: Functions like* n *! arise when generating all permutations or orderings of* n *items.*

The intricacies of dominance relations will be further discussed in Section 2.10 .2 (page 58). However, all you really need to understand is that:

$$n! \gg 2^n \gg n^3 \gg n^2 \gg n \log n \gg n \gg \log n \gg 1$$

Take-Home Lesson: Although esoteric functions arise in advanced algorithm analysis, a small set of time complexities suffice for most algorithms we will see in this book.

5.4 Working with the Big Oh

You learned how to do simplifications of algebraic expressions back in high school. Working with the Big Oh requires dusting off these tools. Most of what you learned there still holds in working with the Big Oh, but not everything.

5.4.1 Adding Functions

The sum of two functions is governed by the dominant one, namely:

$$f(n) + g(n) \to \Theta(\max(f(n), g(n)))$$

This is very useful in simplifying expressions: for example it gives us that $n^3+n^2+n+1 = \Theta\left(n^3\right)$*. Everything else is small potatoes besides the dominant term.*

The intuition is as follows. At least half the bulk of $f(n) + g(n)$ *must come from the larger value. The dominant function will, by definition, provide the larger value as* $n \to \infty$*. Thus, dropping the smaller function from consideration reduces the value by at most a factor of* $1/2$*, which is just a multiplicative constant. For example, if* $f(n) = O\left(n^2\right)$ *and* $g(n) = O\left(n^2\right)$*, then* $f(n) + g(n) = O\left(n^2\right)$ *as well.*

5.4.2 Multiplying Functions

Multiplication is like repeated addition. Consider multiplication by any constant $c > 0$*, be it 1.02 or* $1,000,000$*. Multiplying a function by a constant cannot affect its asymptotic behavior, because we can multiply the bounding constants in the Big Oh analysis to account for it. Thus,*

$$O(c \cdot f(n)) \to O(f(n))$$
$$\Omega(c \cdot f(n)) \to \Omega(f(n))$$
$$\Theta(c \cdot f(n)) \to \Theta(f(n))$$

Of course, c *must be strictly positive (i.e.* $c > 0$ *) to avoid any funny business, since we can wipe out even the fastest growing function by multiplying it by zero.*

On the other hand, when two functions in a product are increasing, both are important. An $O(n! \log n)$ *function dominates* $n!$ *by just as much as* $\log n$ *dominates 1. In general,*

$$O(f(n)) \cdot O(g(n)) \to O(f(n) \cdot g(n))$$
$$\Omega(f(n)) \cdot \Omega(g(n)) \to \Omega(f(n) \cdot g(n))$$
$$\Theta(f(n)) \cdot \Theta(g(n)) \to \Theta(f(n) \cdot g(n))$$

Stop and Think: Transitive Experience *Problem: Show that Big Oh relationships are transitive. That is, if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.*

Solution: We always go back to the definition when working with the Big Oh. What we need to show here is that $f(n) \leq c_3 \cdot h(n)$ for $n > n_3$ given that $f(n) \leq c_1 \cdot g(n)$ and $g(n) \leq c_2 \cdot h(n)$, for $n > n_1$ and $n > n_2$, respectively. Cascading these inequalities, we get that

$$f(n) \leq c_1 \cdot g(n) \leq c_1 c_2 \cdot h(n)$$

for $n > n_3 = \max(n_1, n_2)$.

Reasoning about Efficiency Coarse reasoning about an algorithm's running time is usually easy, given a precise description of the algorithm. In this section, I will work through several examples, perhaps in greater detail than necessary.

5.4.3 Selection Sort

Here we'll analyze the selection sort algorithm, which repeatedly identifies the smallest remaining unsorted element and puts it at the end of the sorted portion of the array. An animation of selection sort in action appears in Figure 2.5 and the code is shown below:

```
void selection_sort(item_type s[], int n) {
    int i, j; /* counters */
    int min; /* index of minimum */
    for (i = 0; i < n; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (s[j] < s[min]) {
                min = j;
            }
        }
        swap(&s[i], &s[min]);
    }
}
```

The outer for loop goes around n times. The nested inner loop goes around $n - (i + 1)$ times, where i is the index of the outer loop. The exact number of times the if statement is executed is given by:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-1} n - i - 1$$

What this sum is doing is adding up the non-negative integers in decreasing order starting from $n-1$*, that is,*

$$T(n) = (n-1) + (n-2) + (n-3) + \ldots + 2 + 1$$

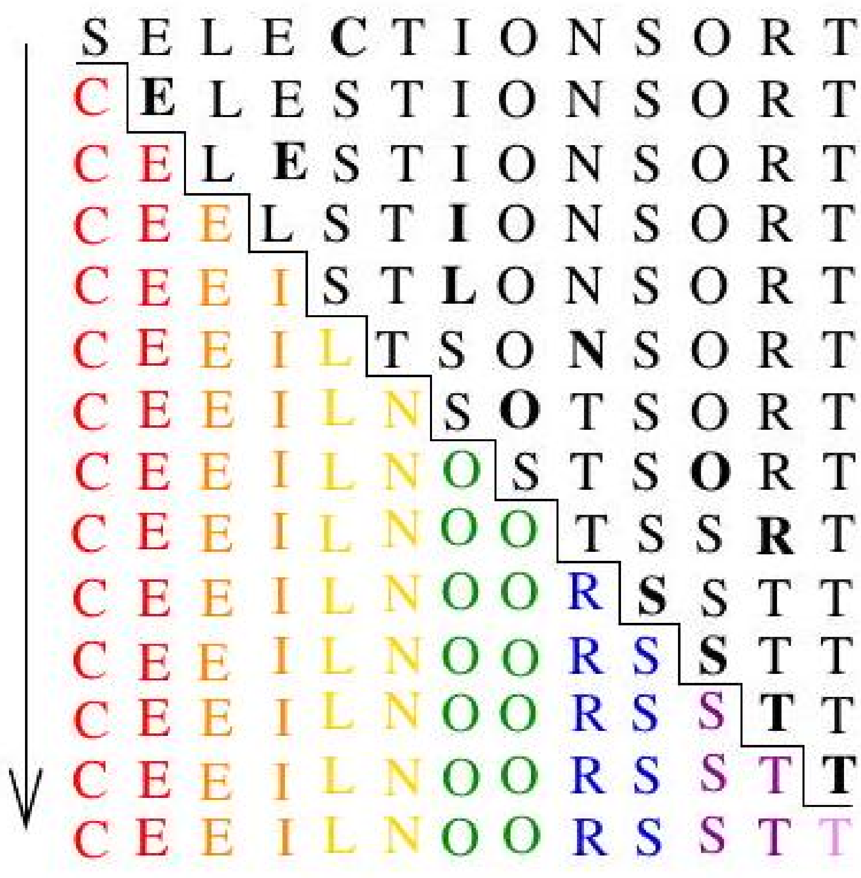


Figure 5.4: Animation of selection sort in action.

How can we reason about such a formula? We must solve the summation formula using the techniques of Section 2.6 (page 46 to get an exact value. But, with the Big Oh, we are only interested in the order of the expression. One way to think about it is that we are adding up $n-1$ *terms, whose average value is about* $n/2$*. This yields* $T(n) \approx (n-1)n/2 = O\left(n^2\right)$*.*

Proving the Theta *Another way to think about this algorithm's running time is in terms of upper and lower bounds. We have* n *terms at most, each of* *which is at most $n-1$. Thus, $T(n) \leq n(n-1) = O\left(n^2\right)$. The Big Oh is an upper bound.*

The Big Ω is a lower bound. Looking at the sum again, we have $n/2$ terms each of which is bigger than $n/2$, followed by $n/2$ terms each greater than zero.. Thus, $T(n) \geq (n/2) \cdot (n/2) + (n/2) \cdot 0 = \Omega\left(n^2\right)$. Together with the Big Oh result, this tells us that the running time is $\Theta\left(n^2\right)$, meaning that selection sort is quadratic.

Generally speaking, turning a Big Oh worst-case analysis into a Big Θ *involves identifying a bad input instance that forces the algorithm to perform as poorly as possible. But selection sort is distinctive among sorting algorithms in that it takes exactly the same time on all n ! possible input instances. Since $T(n) = n(n-1)/2$ for all $n \geq 0, T(n) = \Theta\left(n^2\right)$.*

5.4.4 Insertion Sort

A basic rule of thumb in Big Oh analysis is that worst-case running time follows from multiplying the largest number of times each nested loop can iterate. Consider the insertion sort algorithm presented on page 3 whose inner loops are repeated here:

```
for (i = 1; i < n; i++) {
    j = i;
    while ((j > 0) && (s[j] < s[j - 1])) {
        swap(&s[j], &s[j - 1]);
        j = j-1;
    }
}
```

How often does the inner while loop iterate? This is tricky because there are two different stopping conditions: one to prevent us from running off the bounds of the array ($j > 0$) and the other to mark when the element finds its proper place in sorted order ($s[j] < s[j-1]$). Since worst-case analysis seeks an upper bound on the running time, we ignore the early termination and assume that this loop always goes around i times. In fact, we can simplify further and assume it always goes around n times since $i < n$. Since the outer loop goes around n times, insertion sort must be a quadratic-time algorithm, that is, $O\left(n^2\right)$.

This crude "round it up" analysis always does the job, in that the Big Oh running time bound you get will always be correct. Occasionally, it might be too pessimistic, meaning the actual worst-case time might be of a lower order than implied by such analysis. Still, I strongly encourage this kind of reasoning as a basis for simple algorithm analysis.

Proving the Theta The worst case for insertion sort occurs when each newly inserted element must slide all the way to the front of the sorted region. This happens if the input is given in reverse sorted order. Each of the last $n/2$ elements of the input must slide over at least $n/2$ elements to find the correct position, taking at least $(n/2)^2 = \Omega\left(n^2\right)$ time.

5.4.5 String Pattern Matching

Pattern matching is the most fundamental algorithmic operation on text strings. This algorithm implements the find command available in any web browser or text editor:

Problem: Substring Pattern Matching Input: A text string t and a pattern string p.
Output: Does t contain the pattern p as a substring, and if so, where?
Perhaps you are interested in finding where "Skiena" appears in a given news article (well, I would be interested in such a thing). This is an instance of string pattern matching with t as the news article and p = "Skiena".

There is a fairly straightforward algorithm for string pattern matching that considers the possibility that p may start at each possible position in t and then tests if this is so.

```
a b b a
a b b a
a b b a
a b b a
a b b a
a b b a
```

abaababbaba
Figure 2.6: Searching for the substring abba in the text abaababbaba. Blue characters represent pattern-text matches, with red characters mismatches. The search stops as soon as a match is found.

```
int findmatch(char *p, char *t) {
    int i, j; /* counters */
    int plen, tlen; /* string lengths */
    plen = strlen(p);
    tlen = strlen(t);
    for (i = 0; i <= (tlen-plen); i = i + 1) {
        j = 0;
        while ((j < plen) && (t[i + j] == p[j])) {
            j = j + 1;
        }
        if (j == plen) {
            return(i); /* location of the first match */
        }
    }
    return(-1); /* there is no match */
}
```

What is the worst-case running time of these two nested loops? The inner while loop goes around at most m times, and potentially far less when the pattern match fails. This, plus two other statements, lies within the outer for loop. The *outer loop goes around at most $n - m$ times, since no complete alignment is possible once we get too far to the right of the text. The time complexity of nested loops multiplies, so this gives a worst-case running time of $O((n - m)(m + 2))$.*

We did not count the time it takes to compute the length of the strings using the function strlen. Since the implementation of strlen is not given, we must guess how long it should take. If it explicitly counts the number of characters until it hits the end of the string, this will take time linear in the length of the string. Thus, the total worst-case running time is $O(n + m + (n - m)(m + 2))$.

Let's use our knowledge of the Big Oh to simplify things. Since $m + 2 = \Theta(m)$, the " +2 " isn't interesting, so we are left with $O(n + m + (n - m)m)$.

Multiplying this out yields $O\left(n + m + nm - m^2\right)$, which still seems kind of ugly.

However, in any interesting problem we know that $n \geq m$, since p can't be a substring of t when the pattern is longer than the text itself. One consequence of this is that $n + m \leq 2n = \Theta(n)$. Thus, our worst-case running time simplifies further to $O\left(n + nm - m^2\right)$.

Two more observations and we are done. First, note that $n \leq nm$, since $m \geq 1$ in any interesting pattern. Thus, $n + nm = \Theta(nm)$, and we can drop the additive n, simplifying our analysis to $O\left(nm - m^2\right)$.

Finally, observe that the $-m^2$ term is negative, and thus only serves to lower the value within. Since the Big Oh gives an upper bound, we can drop any negative term without invalidating the upper bound. The inequality $n \geq m$ implies that $mn \geq m^2$, so the negative term is not big enough to cancel the term that is left. Thus, we can express the worst-case running time of this algorithm simply as $O(nm)$.

After you get enough experience, you will be able to do such an algorithm analysis in your head without even writing the algorithm down. After all, algorithm design for a given task involves mentally rifling through different possibilities and selecting the best approach. This kind of fluency comes with practice, but if you are confused about why a given algorithm runs in $O(f(n))$ time, start by writing the algorithm out carefully and then employ the kind of reasoning we used in this section.

Proving the Theta *The analysis above gives a quadratic-time upper bound on the running time of this simple pattern matching algorithm. To prove the theta, we must show an example where it actually does take $\Omega(mn)$ time.*

Consider what happens when the text t = "aaaa ...aaaa" is a string of n a 's, and the pattern p = "aaaa ...aaab" is a string of $m - 1a$'s followed by a b. Wherever the pattern is positioned on the text, the while loop will successfully match the first $m - 1$ characters before failing on the last one. There are $n - m + 1$ possible positions where p can sit on t without overhanging the end, so the running time is:

$$(n - m + 1)(m) = mn - m^2 + m = \Omega(mn)$$

Thus, this string matching algorithm runs in worst-case $\Theta(nm)$ time. Faster *algorithms do exist: indeed we will see an expected linear-time algorithm for this problem in Section 6.7 .*

5.4.6 Matrix Multiplication

Nested summations often arise in the analysis of algorithms with nested loops. Consider the problem of matrix multiplication:

Problem: Matrix Multiplication Input: Two matrices, A (of dimension $x \times y$) and B (dimension $y \times z$).
Output: An $x \times z$ matrix C where $C[i][j]$ is the dot product of the i th row of A and the j th column of B.

Matrix multiplication is a fundamental operation in linear algebra, presented with an entry in the catalog in Section 16.3 (page 472 . That said, the elementary algorithm for matrix multiplication is implemented as three nested loops:

```
for (i = 1; i <= a->rows; i++) {
    for (j = 1; j <= b->columns; j++) {
        c->m[i][j] = 0;
        for (k = 1; k <= b->rows; k++) {
            c->m[i][j] += a->m[i][k] * b->m[k][j];
            }
    }
}
```

How can we analyze the time complexity of this algorithm? Three nested loops should smell $O\left(n^3\right)$ to you by this point, but let's be precise. The number of multiplications $M(x, y, z)$ is given by the following summation:

$$M(x, y, z) = \sum_{i=1}^{x} \sum_{j=1}^{y} \sum_{k=1}^{z} 1$$

Sums get evaluated from the right inward. The sum of z ones is z, so

$$M(x, y, z) = \sum_{i=1}^{x} \sum_{j=1}^{y} z$$

The sum of yz 's is just as simple, yz, so

$$M(x, y, z) = \sum_{i=1}^{x} yz$$

Finally, the sum of xyz 's is xyz.
Thus, the running of this matrix multiplication algorithm is $O(xyz)$. If we consider the common case where all three dimensions are the same, this becomes $O\left(n^3\right)$. The same analysis holds for an $\Omega\left(n^3\right)$ lower bound, because the matrix dimensions govern the number of iterations of the for loops. Simple matrix multiplication is a cubic algorithm that runs in $\Theta\left(n^3\right)$ time. Faster algorithms exist: see Section 16.3 .

5.5 Summations

Mathematical summation formulae are important to us for two reasons. First, they often arise in algorithm analysis. Second, proving the correctness of such

formulae is a classic application of mathematical induction. Several exercises on inductive proofs of summations appear as exercises at the end of this chapter. To make them more accessible, I review the basics of summations here.

Summation formulae are concise expressions describing the addition of an arbitrarily large set of numbers, in particular the formula

$$\sum_{i=1}^{n} f(i) = f(1) + f(2) + \ldots + f(n)$$

Simple closed forms exist for summations of many algebraic functions. For example, since the sum of n *ones is* n,

$$\sum_{i=1}^{n} 1 = n$$

When n *is even, the sum of the first* $n = 2k$ *integers can be seen by pairing up the* i *th and* $(n - i + 1)$ *th integers:*

$$\sum_{i=1}^{n} i = \sum_{i=1}^{k} (i + (2k - i + 1)) = k(2k + 1) = n(n + 1)/2$$

The same result holds for odd n *with slightly more careful analysis.*
Recognizing two basic classes of summation formulae will get you a long way in algorithm analysis:

- *Sum of a power of integers - We encountered the sum of the first* n *positive integers* $S(n) = \sum_{i=1}^{n} i = n(n+1)/2$ *in the analysis of selection sort. From the big picture perspective, the important thing is that the sum is quadratic, not that the constant is* $1/2$. *In general,*

$$S(n, p) = \sum_{i=1}^{n} i^p = \Theta\left(n^{p+1}\right)$$

for $p \geq 0$. *Thus, the sum of squares is cubic, and the sum of cubes is quartic (if you use such a word).*
For $p < -1$, *this sum* $S(n, p)$ *always converges to a constant as* $n \to \infty$, *while for* $p \geq 0$ *it diverges. The interesting case between these is the Harmonic numbers,* $H(n) = \sum_{i=1}^{n} 1/i = \Theta(\log n)$.

- *Sum of a geometric progression - In geometric progressions, the index of the loop affects the exponent, that is,*

$$G(n, a)=\sum_{i=0}^{n} a^{i}=\left(a^{n+1}-1\right) /(a-1)$$

How we interpret this sum depends upon the base of the progression, in this case a. *When* $|a|<1, G(n, a)$ *converges to a constant as* $n \rightarrow \infty$.

This series convergence proves to be the great "free lunch" of algorithm analysis. It means that the sum of a linear number of things can be constant, not linear. For example, $1+1/2+1/4+1/8+\ldots \leq 2$ *no matter how many terms we add up.*

When $a>1$, *the sum grows rapidly with each new term, as in* $1+2+4+8+16+32=63$. *Indeed,* $G(n, a)=\Theta\left(a^{n+1}\right)$ *for* $a>1$.

Stop and Think: Factorial Formulae *Problem: Prove that* $\sum_{i=1}^{n} i \times i!=(n+1)!-1$ *by induction.*

Solution: The inductive paradigm is straightforward. First verify the basis case. The case $n=0$ *gives an empty sum, which by definition evaluates to 0 . Alternately we can do* $n=1$ *:*

$$\sum_{i=1}^{1} i \times i!=1 \text { and }(1+1)!-1=2-1=1$$

Now assume the statement is true up to n. *To prove the general case of* $n+1$, *observe that separating out the largest term*

$$\sum_{i=1}^{n+1} i \times i!=(n+1) \times(n+1)!+\sum_{i=1}^{n} i \times i!$$

reveals the left side of our inductive assumption. Substituting the right side gives us

$$\begin{aligned}
\sum_{i=1}^{n+1} i \times i! & =(n+1) \times(n+1)!+(n+1)!-1 \\
& =(n+1)! \times((n+1)+1)-1 \\
& =(n+2)!-1
\end{aligned}$$

This general trick of separating out the largest term from the summation to reveal an instance of the inductive assumption lies at the heart of all such proofs.

5.6 Logarithms and Their Applications

Logarithm is an anagram of algorithm, but that's not why we need to know what logarithms are. You've seen the button on your calculator, but may have forgotten why it is there. A logarithm is simply an inverse exponential function. Saying $b^{x}=y$ *is equivalent to saying that* $x=\log_{b} y$. *Further, this equivalence is the same as saying* $b^{\log_{b} y}=y$.

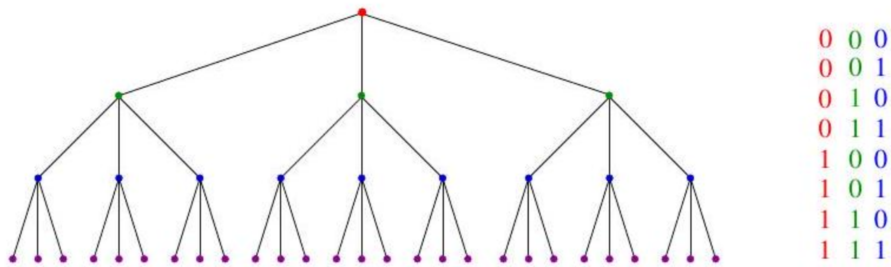


Figure 5.5: A height h tree with d children per node has d^h leaves. Here $h = 3$ and $d = 3$ (left). The number of bit patterns grows exponentially with pattern length (right). These would be described by the root-to-leaf paths of a *binary* tree of height $h = 3$.

Exponential functions grow at a distressingly fast rate, as anyone who has ever tried to pay off a credit card balance understands. Thus, inverse exponential functions (logarithms) grow refreshingly slowly. Logarithms arise in any process where things are repeatedly halved. We'll now look at several examples.

5.6.1 Logarithms and Binary Search

Binary search is a good example of an $O(\log n)$ *algorithm. To locate a particular person* p *in a telephone book* 2^2 *containing* n *names, you start by comparing* p *against the middle, or (* $n/2$ *)nd name, say Monroe, Marilyn. Regardless of whether* p *belongs before this middle name (Dean, James) or after it (Presley, Elvis), after just one comparison you can discard one half of all the names in the book. The number of steps the algorithm takes equals the number of times we can halve* n *until only one name is left. By definition, this is exactly* $\log_2 n$*. Thus, twenty comparisons suffice to find any name in the million-name Manhattan phone book!*

Binary search is one of the most powerful ideas in algorithm design. This power becomes apparent if we imagine trying to find a name in an unsorted telephone book.

5.6.2 Logarithms and Trees

A binary tree of height 1 can have up to 2 leaf nodes, while a tree of height 2 can have up to 4 leaves. What is the height h *of a rooted binary tree with* n *leaf nodes? Note that the number of leaves doubles every time we increase the height by 1 . To account for* n *leaves,* $n = 2^h$*, which implies that* $h = \log_2 n$*.*

What if we generalize to trees that have d *children, where* $d = 2$ *for the case of binary trees? A tree of height 1 can have up to* d *leaf nodes, while one of height 2 can have up to* d^2 *leaves. The number of possible leaves multiplies by* *d every time we increase the height by 1 , so to account for n leaves,* $n = d^h$, *which implies that* $h = \log_d n$, *as shown in Figure 2.7*

The punch line here is that short trees can have very many leaves, which is the main reason why binary trees prove fundamental to the design of fast data structures.

5.6.3 Logarithms and Bits

There are two bit patterns of length $1(0$ *and 1*$)$*, four of length* $2(00, 01, 10,$ *and 11), and eight of length 3 (see Figure 2.7 (right)). How many bits* w *do we need to represent any one of* n *different possibilities, be it one of* n *items or the integers from 0 to* $n-1$ *?*

The key observation is that there must be at least n *different bit patterns of length* w*. Since the number of different bit patterns doubles as you add each bit, we need at least* w *bits where* $2^w = n$*. In other words, we need* $w = \log_2 n$ *bits.*

5.6.4 Logarithms and Multiplication

Logarithms were particularly important in the days before pocket calculators. They provided the easiest way to multiply big numbers by hand, either implicitly using a slide rule or explicitly by using a book of logarithms.

Logarithms are still useful for multiplication, particularly for exponentiation. Recall that $\log_a(xy) = \log_a(x) + \log_a(y)$*; that is, the log of a product is the sum of the logs. A direct consequence of this is*

$$\log_a n^b = b \cdot \log_a n$$

How can we compute a^b *for any* a *and* b *using the* $\exp(x)$ *and* $\ln(x)$ *functions on your calculator, where* $\exp(x) = e^x$ *and* $\ln(x) = \log_e(x)$ *? We know*

$$a^b = \exp\left(\ln\left(a^b\right)\right) = \exp(b \ln(a))$$

so the problem is reduced to one multiplication plus one call to each of these functions.

5.6.5 Fast Exponentiation

Suppose that we need to exactly compute the value of a^n *for some reasonably large* n*. Such problems occur in primality testing for cryptography, as discussed in Section 16.8 (page 490). Issues of numerical precision prevent us from applying the formula above.*

The simplest algorithm performs $n-1$ *multiplications, by computing* $a \times a \times \ldots \times a$*. However, we can do better by observing that* $n = \lfloor n/2 \rfloor + \lceil n/2 \rceil$*. If* n *is even, then* $a^n = \left(a^{n/2}\right)^2$*. If* n *is odd, then* $a^n = a\left(a^{\lfloor n/2 \rfloor}\right)^2$*. In either case, we have halved the size of our exponent at the cost of, at most, two multiplications, so* $O(\lg n)$ *multiplications suffice to compute the final value.*

[2] If necessary, ask your grandmother what telephone books were.

```
function power $(a, n)$
    if $(n=0)$ return $(1)$
    $x=\operatorname{power}(a,\lfloor n / 2\rfloor)$
    if ( $n$ is even) then return $\left(x^{2}\right)$
        else return $\left(a \times x^{2}\right)$
```

This simple algorithm illustrates an important principle of divide and conquer. It always pays to divide a job as evenly as possible. When n is not a power of two, the problem cannot always be divided perfectly evenly, but a difference of one element between the two sides as shown here cannot cause any serious imbalance.

5.6.6 Logarithms and Summations

The Harmonic numbers arise as a special case of a sum of a power of integers, namely $H(n) = S(n, -1)$*. They are the sum of the progression of simple reciprocals, namely,*

$$H(n) = \sum_{i=1}^{n} 1/i = \Theta(\log n)$$

The Harmonic numbers prove important, because they usually explain "where the log comes from" when one magically pops out from algebraic manipulation. For example, the key to analyzing the average-case complexity of quicksort is the summation $n \sum_{i=1}^{n} 1/i$*. Employing the Harmonic number's* Θ *bound immediately reduces this to* $\Theta(n \log n)$*.*

5.6.7 Logarithms and Criminal Justice

Figure 2.8 will be our final example of logarithms in action. This table appears in the Federal Sentencing Guidelines, used by courts throughout the United States. These guidelines are an attempt to standardize criminal sentences, so that a felon convicted of a crime before one judge receives the same sentence that they would before a different judge. To accomplish this, the judges have prepared an intricate point function to score the depravity of each crime and map it to time-to-serve.

Figure 2.8 gives the actual point function for fraud - a table mapping dollars stolen to points. Notice that the punishment increases by one level each time the amount of money stolen roughly doubles. That means that the level of punishment (which maps roughly linearly to the amount of time served) grows logarithmically with the amount of money stolen.

Think for a moment about the consequences of this. Many a corrupt CEO certainly has. It means that your total sentence grows extremely slowly with the amount of money you steal. Embezzling an additional $100,000 *gets you 3 additional punishment levels if you've already stolen* $10,000*, adds only 1 level if you've stolen* $50,000*, and has no effect if you've stolen a million. The corresponding benefit of stealing really large amounts of money becomes even*

Loss (apply the greatest)	*Increase in level*
(A) $\$2,000$ *or less*	*no increase*
(B) More than $\$2,000$	*add 1*
(C) More than $\$5,000$	*add 2*
(D) More than $\$10,000$	*add 3*
(E) More than $\$20,000$	*add 4*
(F) More than $\$40,000$	*add 5*
(G) More than $\$70,000$	*add 6*
(H) More than $\$120,000$	*add 7*
(I) More than $\$200,000$	*add 8*
(J) More than $\$350,000$	*add 10*
(K) More than $\$500,000$	*add 11*
(L) More than $\$800,000$	*add 12*
(M) More than $\$1,500,000$	*add 13*
(N) More than $\$2,500,000$	*add 14*
(O) More than $\$5,000,000$	*add 15*
(P) More than $\$10,000,000$	*add 16*
(Q) More than $\$20,000,000$	*add 17*
(R) More than $\$40,000,000$	*add 18*
(S) More than $\$80,000,000$	

Figure 2.8: The Federal Sentencing Guidelines for fraud
greater. The moral of logarithmic growth is clear: If you're gonna do the crime, make it worth the time $^{\beta}$

Take-Home Lesson: Logarithms arise whenever things are repeatedly halved or doubled.

5.7 Properties of Logarithms

As we have seen, stating $b^x = y$ *is equivalent to saying that* $x = \log_b y$. *The* b *term is known as the base of the logarithm. Three bases are of particular importance for mathematical and historical reasons:*

- *Base* $b = 2$ *: The binary logarithm, usually denoted* $\lg x$, *is a base 2 logarithm. We have seen how this base arises whenever repeated halving (i.e., binary search) or doubling (i.e., nodes in trees) occurs. Most algorithmic applications of logarithms imply binary logarithms.*

- *Base* $b = e$ *: The natural logarithm, usually denoted* $\ln x$, *is a base* $e = 2.71828\ldots$ *logarithm. The inverse of* $\ln x$ *is the exponential function* $\exp(x) = e^x$ *on your calculator. Thus, composing these functions gives us the identity function,*

$$\exp(\ln x) = x \text{ and } \ln(\exp x) = x$$

- *Base $b = 10$: Less common today is the base-10 or common logarithm, usually denoted as $\log x$. This base was employed in slide rules and logarithm books in the days before pocket calculators.*

We have already seen one important property of logarithms, namely that

$$\log_a(xy) = \log_a(x) + \log_a(y)$$

The other important fact to remember is that it is easy to convert a logarithm from one base to another. This is a consequence of the following formula:

$$\log_a b = \frac{\log_c b}{\log_c a}$$

Thus, changing the base of $\log b$ from base- a to base- c simply involves multiplying by $\log_c a$. It is easy to convert a common $\log$ function to a natural $\log$ function, and vice versa.

Two implications of these properties of logarithms are important to appreciate from an algorithmic perspective:

- *The base of the logarithm has no real impact on the growth rate: Compare the following three values: $\log_2(1,000,000) = 19.9316, \log_3(1,000,000) =$ 12.5754, and $\log_{100}(1,000,000) = 3$. A big change in the base of the logarithm produces little difference in the value of the log. Changing the base of the $\log$ from a to c involves multiplying by $\log_c a$. This conversion factor is absorbed in the Big Oh notation whenever a and c are constants. Thus, we are usually justified in ignoring the base of the logarithm when analyzing algorithms.*

- *Logarithms cut any function down to size: The growth rate of the logarithm of any polynomial function is $O(\lg n)$. This follows because*

$$\log_a n^b = b \cdot \log_a n$$

The effectiveness of binary search on a wide range of problems is a consequence of this observation. Note that performing a binary search on a sorted array of n^2 things requires only twice as many comparisons as a binary search on n things.

Logarithms efficiently cut any function down to size. It is hard to do arithmetic on factorials except after taking logarithms, since

$$n! = \prod_{i=1}^{n} i \rightarrow \log n! = \sum_{i=1}^{n} \log i = \Theta(n \log n)$$

[3] Life imitates art. After publishing this example in the previous edition, I was approached by the U.S. Sentencing Commission seeking insights to improve these guidelines.

provides another way logarithms pop up in algorithm analysis.

Stop and Think: Importance of an Even Split *Problem: How many queries does binary search take on the million-name Manhattan phone book if each split were* $1/3$*-to-* $2/3$ *instead of* $1/2$*-to-* $1/2$ *?*

Solution: When performing binary searches in a telephone book, how important is it that each query split the book exactly in half? Not very much. For the Manhattan telephone book, we now use $\log_{3/2}(1,000,000) \approx 35$ *queries in the worst case, not a significant change from* $\log_2(1,000,000) \approx 20$*. Changing the base of the log does not affect the asymptotic complexity. The effectiveness of binary search comes from its logarithmic running time, not the base of the log.*

5.8 War Story: Mystery of the Pyramids

That look in his eyes should have warned me off even before he started talking. "We want to use a parallel supercomputer for a numerical calculation up to $1,000,000,000$*, but we need a faster algorithm to do it."*

I'd seen that distant look before. Eyes dulled from too much exposure to the raw horsepower of supercomputers-machines so fast that brute force seemed to eliminate the need for clever algorithms; at least until the problems got hard enough.

"I am working with a Nobel prize winner to use a computer on a famous problem in number theory. Are you familiar with Waring's problem?"

I knew some number theory. "Sure. Waring's problem asks whether every integer can be expressed at least one way as the sum of at most four integer squares. For example, $78 = 8^2 + 3^2 + 2^2 + 1^2 = 7^2 + 5^2 + 2^2$*. I remember proving that four squares suffice to represent any integer in my undergraduate number theory class. Yes, it's a famous problem but one that got solved 200 years ago."*

"No, we are interested in a different version of Waring's problem. A pyramidal number is a number of the form $\left(m^3 - m\right)/6$*, for* $m \geq 2$*. Thus, the first several pyramidal numbers are* $1, 4, 10, 20, 35, 56, 84, 120$*, and 165 . The conjecture since 1928 is that every integer can be represented by the sum of at most five such pyramidal numbers. We want to use a supercomputer to prove this conjecture on all numbers from 1 to* $1,000,000,000$*."*

"Doing a billion of anything will take a substantial amount of time," I warned. "The time you spend to compute the minimum representation of each number will be critical, since you are going to do it one billion times. Have you thought about what kind of an algorithm you are going to use?"

"We have already written our program and run it on a parallel supercomputer. It works very fast on smaller numbers. Still, it takes much too much time as soon as we get to 100,000 or so."

"Terrific," I thought. Our supercomputer junkie had discovered asymptotic growth. No doubt his algorithm ran in something like quadratic time, and went

into vapor lock as soon as n got large.

"We need a faster program in order to get to a billion. Can you help us? Of

course, we can run it on our parallel supercomputer when you are ready."

I am a sucker for this kind of challenge, finding fast algorithms to speed up programs. I agreed to think about it and got down to work.

I started by looking at the program that the other guy had written. He had built an array of all the $\Theta\left(n^{1/3}\right)$ *pyramidal numbers from 1 to n inclusive* $\int^4$ *To test each number k in this range, he did a brute force test to establish whether it was the sum of two pyramidal numbers. If not, the program tested whether it was the sum of three of them, then four, and finally five, until it first got an answer. About* 45% *of the integers are expressible as the sum of three pyramidal numbers. Most of the remaining* 55% *require the sum of four, and usually each of these can be represented in many different ways. Only 241 integers are known to require the sum of five pyramidal numbers, the largest being 343,867. For about half of the n numbers, this algorithm presumably went through all of the three-tests and at least some of the four-tests before terminating. Thus, the total time for this algorithm would be at least* $O\left(n \times \left(n^{1/3}\right)^3\right) = O\left(n^2\right)$ *time, where* $n = 1,000,000,000$. *No wonder his program cried "Uncle."*

Anything that was going to do significantly better on a problem this large had to avoid explicitly testing all triples. For each value of k, we were seeking the smallest set of pyramidal numbers that add up to exactly to k. This problem is called the knapsack problem, and is discussed in Section 16.10 (page 497). In our case, the weights are the pyramidal numbers no greater than n, with an additional constraint that the knapsack holds exactly k items.

A standard approach to solving knapsack precomputes the sum of smaller subsets of the items for use in computing larger subsets. If we have a table of all sums of two numbers and want to know whether k is expressible as the sum of three numbers, we can ask whether k is expressible as the sum of a single number plus a number in this two-table.

Therefore, I needed a table of all integers less than n that can be expressed as the sum of two of the 1,816 non-trivial pyramidal numbers less than $1,000,000,000$. *There can be at most* $1,816^2 = 3,297,856$ *of them. Actually, there are only about half this many, after we eliminate duplicates and any sum bigger than our target. Building a sorted array storing these numbers would be no big deal. Let's call this sorted data structure of all pair-sums the two-table.*

To find the minimum decomposition for a given k, I would first check whether it was one of the 1,816 pyramidal numbers. If not, I would then check whether k was in the sorted table of the sums of two pyramidal numbers. To see whether k was expressible as the sum of three such numbers, all I had to do was check whether $k - p[i]$ *was in the two-table for* $1 \leq i \leq 1,816$. *This could be done quickly using binary search. To see whether k was expressible as the sum of four pyramidal numbers, I had to check whether* $k - two[i]$ *was in the two-table*

for any $1 \leq i \leq |two|$. *However, since almost every k was expressible in many ways as the sum of four pyramidal numbers, this test would termi-*

[4] Why $n^{1/3}$? Recall that pyramidal numbers are of the form $\left(m^3 - m\right)/6$. The largest m such that the resulting number is at most n is roughly $\sqrt[3]{6n}$, so there are $\Theta\left(n^{1/3}\right)$ such numbers.

nate quickly, and the total time taken would be dominated by the cost of the threes. Testing whether k *was the sum of three pyramidal numbers would take* $O\left(n^{1/3}\lg n\right)$. *Running this on each of the* n *integers gives an* $O\left(n^{4/3}\lg n\right)$ *algorithm for the complete job. Comparing this to his* $O\left(n^2\right)$ *algorithm for* $n = 1,000,000,000$ *suggested that my algorithm was a cool 30,000 times faster than his original!*

My first attempt to code this solved up to $n = 1,000,000$ *on my ancient Sparc ELC in about 20 minutes. From here, I experimented with different data structures to represent the sets of numbers and different algorithms to search these tables. I tried using hash tables and bit vectors instead of sorted arrays, and experimented with variants of binary search such as interpolation search (see Section 17.2 (page 510p). My reward for this work was solving up to* $n = 1,000,000$ *in under three minutes, a factor of six improvement over my original program.*

With the real thinking done, I worked to tweak a little more performance out of the program. I avoided doing a sum-of-four computation on any k *when* $k-1$ *was the sum-of-three, since 1 is a pyramidal number, saving about* 10% *of the total run time using this trick alone. Finally, I got out my profiler and tried some low-level tricks to squeeze a little more performance out of the code. For example, I saved another* 10% *by replacing a single procedure call with inline code.*

At this point, I turned the code over to the supercomputer guy. What he did with it is a depressing tale, which is reported in Section 5.8 (page 161.).

In writing up this story, I went back to rerun this program, which is now older than my current graduate students. Even though single-threaded, it ran in 1.113 seconds. Turning on the compiler optimizer reduced this to a mere 0.334 seconds: this is why you need to remember to turn your optimizer on when you are trying to make your program run fast. This code has gotten hundreds of times faster by doing nothing, except waiting for 25 years of hardware improvements. Indeed a server in our lab can now run up to a billion in under three hours (174 minutes and 28.4 seconds) using only a single thread. Even more amazingly, I can run this code to completion in 9 hours, 37 minutes, and 34.8 seconds on the same crummy Apple MacBook laptop that I am writing this book on, despite its keys falling off as I type.

The primary lesson of this war story is to show the enormous potential for algorithmic speedups, as opposed to the fairly limited speedup obtainable via more expensive hardware. I sped his program up by about 30,000 times. His million-dollar computer (at that time) had 16 processors, each reportedly five times faster on integer computations than the \$3,000 *machine on my desk. That gave him a maximum potential speedup of less than 100 times. Clearly, the algorithmic improvement was the big winner here, as it is certain to be in any sufficiently large computation.*

5.9 Advanced Analysis (*)

Ideally, each of us would be fluent in working with the mathematical techniques of asymptotic analysis. And ideally, each of us would be rich and good looking as well.

In this section I will survey the major techniques and functions employed in advanced algorithm analysis. I consider this optional material-it will not be used elsewhere in the textbook section of this book. That said, it will make some of the complexity functions reported in the Hitchhiker's Guide a little less mysterious.

5.9.1 Esoteric Functions

The bread-and-butter classes of complexity functions were presented in Section 2.3.1 (page 38. More esoteric functions also make appearances in advanced algorithm analysis. Although we will not see them much in this book, it is still good business to know what they mean and where they come from.

- *Inverse Ackermann's function* $f(n) = \alpha(n)$ *: This function arises in the detailed analysis of several algorithms, most notably the Union-Find data structure discussed in Section 8.1.3 (page 250. It is sufficient to think of this as geek talk for the slowest growing complexity function. Unlike the constant function* $f(n) = 1, \alpha(n)$ *eventually gets to infinity as* $n \to \infty$*, but it certainly takes its time about it. The value of* $\alpha(n)$ *is smaller than 5 for any value of* n *that can be written in this physical universe.*

- $f(n) = \log \log n$ *: The "log log" function is just that-the logarithm of the logarithm of* n*. One natural example of how it might arise is doing a binary search on a sorted array of only* $\lg n$ *items.*

- $f(n) = \log n / \log \log n$ *: This function grows a little slower than* $\log n$*, because it is divided by an even slower growing function. To see where this arises, consider an* n*-leaf rooted tree of degree* d*. For binary trees, that is, when* $d = 2$*, the height* h *is given*

$$n = 2^h \rightarrow h = \lg n$$

by taking the logarithm of both sides of the equation. Now consider the height of such a tree when the degree $d = \log n$*. Then*

$$n = (\log n)^h \rightarrow h = \log n / \log \log n$$

- $f(n) = \log^2 n$ *: This is the product of two* $\log$ *functions,* $(\log n) \times (\log n)$*. It might arise if we wanted to count the bits looked at when doing a binary search on* n *items, each of which was an integer from 1 to (say)* n^2*. Each such integer requires a* $\lg\left(n^2\right) = 2 \lg n$ *bit representation, and we look at* $\lg n$ *of them, for a total of* $2 \lg^2 n$ *bits.*

The "log squared" function typically arises in the design of intricate nested data structures, where each node in (say) a binary tree represents another data structure, perhaps ordered on a different key.

- $f(n) = \sqrt{n}$ *: The square root is not very esoteric, but represents the class of "sublinear polynomials" since* $\sqrt{n} = n^{1/2}$*. Such functions arise in building d-dimensional grids that contain n points. A* $\sqrt{n} \times \sqrt{n}$ *square has area n, and an* $n^{1/3} \times n^{1/3} \times n^{1/3}$ *cube has volume n. In general, a d-dimensional hypercube of length* $n^{1/d}$ *on each side has volume n.*

- $f(n) = n^{(1+\epsilon)}$ *: Epsilon* (ϵ) *is the mathematical symbol to denote a constant that can be made arbitrarily small but never quite goes away.*
  It arises in the following way. Suppose I design an algorithm that runs in $2^c \cdot n^{(1+1/c)}$ *time, and I get to pick whichever c I want. For* $c = 2$*, this is* $4n^{3/2}$ *or* $O\left(n^{3/2}\right)$*. For* $c = 3$*, this is* $8n^{4/3}$ *or* $O\left(n^{4/3}\right)$*, which is better. Indeed, the exponent keeps getting better the larger I make c.*
  The problem is that I cannot make c arbitrarily large before the 2^c *term begins to dominate. Instead, we report this algorithm as running in* $O\left(n^{1+\epsilon}\right)$*, and leave the best value of* ϵ *to the beholder.*

5.9.2 Limits and Dominance Relations

The dominance relation between functions is a consequence of the theory of limits, which you may recall from taking calculus. We say that $f(n)$ *dominates* $g(n)$ *if* $\lim_{n\to\infty} g(n)/f(n) = 0$*.*

Let's see this definition in action. Suppose $f(n) = 2n^2$ *and* $g(n) = n^2$*. Clearly* $f(n) > g(n)$ *for all n, but it does not dominate since*

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = \lim_{n\to\infty} \frac{n^2}{2n^2} = \lim_{n\to\infty} \frac{1}{2} \neq 0$$

This is to be expected because both functions are in the class $\Theta\left(n^2\right)$*. What about* $f(n) = n^3$ *and* $g(n) = n^2$ *? Since*

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = \lim_{n\to\infty} \frac{n^2}{n^3} = \lim_{n\to\infty} \frac{1}{n} = 0$$

the higher-degree polynomial dominates. This is true for any two polynomials, that is, n^a *dominates* n^b *if* $a > b$ *since*

$$\lim_{n\to\infty} \frac{n^b}{n^a} = \lim_{n\to\infty} n^{b-a} \to 0$$

Thus, $n^{1.2}$ *dominates* $n^{1.1999999}$*.*
Now consider two exponential functions, say $f(n) = 3^n$ *and* $g(n) = 2^n$*. Since*

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = \frac{2^n}{3^n} = \lim_{n\to\infty} \left(\frac{2}{3}\right)^n = 0$$

the exponential with the higher base dominates.
Our ability to prove dominance relations from scratch depends upon our ability to prove limits. Let's look at one important pair of functions. Any polynomial (say $f(n) = n^\epsilon$) dominates logarithmic functions (say $g(n) = \lg n$). Since $n = 2^{\lg n}$,

$$f(n) = \left(2^{\lg n}\right)^\epsilon = 2^{\epsilon \lg n}$$

Now consider

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = \lg n / 2^{\epsilon \lg n}$$

In fact, this does go to 0 as $n \to \infty$.
Take-Home Lesson: By interleaving the functions here with those of Section 2.3.1 (page 38, we see where everything fits into the dominance pecking order:

$$n! \gg c^n \gg n^3 \gg n^2 \gg n^{1+\epsilon} \gg n \log n \gg n \gg \sqrt{n} \gg 1$$
$$\log^2 n \gg \log n \gg \log n / \log\log n \gg \log\log n \gg \alpha(n) \gg 1$$

Chapter Notes *Most other algorithm texts devote considerably more efforts to the formal analysis of algorithms than I have here, and so I refer the theoretically inclined reader elsewhere for more depth. Algorithm texts more heavily stressing analysis include Cormen et al. citecormen2009introduction and Kleinberg and Tardos citekleinberg2006algorithm.*

The book Concrete Mathematics by Graham, Knuth, and Patashnik [citegraham1989concrete] offers an interesting and thorough presentation of mathematics for the analysis of algorithms. Niven and Zuckerman [citeniven1991introduction is my favorite introduction to number theory, including Waring's problem, discussed in the war story.

The notion of dominance also gives rise to the "Little Oh" notation. We say that $f(n) = o(g(n))$ iff $g(n)$ dominates $f(n)$. Among other things, the Little Oh proves useful for asking questions. Asking for an $o\left(n^2\right)$ algorithm means you want one that is better than quadratic in the worst case - and means you would be willing to settle for $O\left(n^{1.999} \log^2 n\right)$.

5.10 Exercises

Program Analysis 2-1. [3] What value is returned by the following function? Express your answer as a function of n. Give the worst-case running time using the Big Oh notation.

Mystery(n)

$r = 0$
for $i = 1$ *to* $n - 1$ *do*
for $j = i + 1$ *to* n *do*
for $k = 1$ *to* j *do*

$r = r + 1$
return (r)

2-2. [3] What value is returned by the following function? Express your answer as a function of n. Give the worst-case running time using Big Oh notation.

Pesky(n)

$r = 0$
for $i = 1$ *to* n *do*
for $j = 1$ *to* i *do*
for $k = j$ *to* $i + j$ *do*
$r = r + 1$
return(r)

2-3. [5] What value is returned by the following function? Express your answer as a function of n. Give the worst-case running time using Big Oh notation.

Pestiferous (n)

$r = 0$
for $i = 1$ *to* n *do*
for $j = 1$ *to* i *do*
for $k = j$ *to* $i + j$ *do*
for $l = 1$ *to* $i + j - k$ *do*
$r = r + 1$
return(r)

2-4. [8] What value is returned by the following function? Express your answer as a function of n. Give the worst-case running time using Big Oh notation.

Conundrum (n)

$r = 0$
for $i = 1$ *to* n *do*
for $j = i + 1$ *to* n *do*
for $k = i + j - 1$ *to* n *do*
$r = r + 1$
return(r)

2-5. [5] Consider the following algorithm: (the print operation prints a single asterisk; the operation $x = 2x$ doubles the value of the variable x).

$$\textit{for } k = 1 \textit{ to } n :$$
$$x = k$$
$$\textit{while } (x < n)$$
$$\textit{print } ',$$
$$x = 2x$$

Let $f(n)$ be the time complexity of this algorithm (or equivalently the number of times $*$ is printed). Provide correct bounds for $O(f(n))$ and $\Omega(f(n))$, ideally converging on $\Theta(f(n))$.
2-6. [5] Suppose the following algorithm is used to evaluate the polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$$

$p = a_0$;
xpower $= 1$;
for $i = 1$ *to* n *do*

$$\textit{xpower } = x \cdot \textit{ xpower };$$
$$p = p + a_i * \textit{ xpower}$$

(a) How many multiplications are done in the worst case? How many additions?

(b) How many multiplications are done on the average?
(c) Can you improve this algorithm?
2-7. [3] Prove that the following algorithm for computing the maximum value in an array $A[1..n]$ is correct.

$$\max(A)$$
$$m = A[1]$$
$$\textit{for } i = 2 \textit{ to } n \textit{ do}$$
$$\textit{if } A[i] > m \textit{ then } m = A[i]$$
$$\textit{return } (m)$$

Big Oh 2-8. [3] (a) Is $2^{n+1} = O(2^n)$?
(b) Is $2^{2n} = O(2^n)$?
2-9. [3] For each of the following pairs of functions, $f(n)$ is in $O(g(n)), \Omega(g(n))$, or $\Theta(g(n))$. Determine which relationships are correct and briefly explain why.
(a) $f(n) = \log n^2; g(n) = \log n + 5$
(b) $f(n) = \sqrt{n}; g(n) = \log n^2$
(c) $f(n) = \log^2 n; g(n) = \log n$
(d) $f(n) = n; g(n) = \log^2 n$
(e) $f(n) = n \log n + n; g(n) = \log n$
(f) $f(n) = 10; g(n) = \log 10$

(g) $f(n) = 2^n; g(n) = 10n^2$
(h) $f(n) = 2^n; g(n) = 3^n$

2-10. [3] For each of the following pairs of functions $f(n)$ *and* $g(n)$, *determine whether* $f(n) = O(g(n)), g(n) = O(f(n))$, *or both.*
(a) $f(n) = \left(n^2 - n\right)/2, g(n) = 6n$
(b) $f(n) = n + 2\sqrt{n}, g(n) = n^2$
(c) $f(n) = n\log n, g(n) = n\sqrt{n}/2$
(d) $f(n) = n + \log n, g(n) = \sqrt{n}$
(e) $f(n) = 2(\log n)^2, g(n) = \log n + 1$
(f) $f(n) = 4n\log n + n, g(n) = \left(n^2 - n\right)/2$

2-11. [5] For each of the following functions, which of the following asymptotic bounds hold for $f(n) : O(g(n)), \Omega(g(n))$, *or* $\Theta(g(n))$ *?*
(a) $f(n) = 3n^2, g(n) = n^2$
(b) $f(n) = 2n^4 - 3n^2 + 7, g(n) = n^5$
(c) $f(n) = \log n, g(n) = \log n + \frac{1}{n}$
(d) $f(n) = 2^{k\log n}, g(n) = n^k$
(e) $f(n) = 2^n, g(n) = 2^{2n}$

2-12. [3] Prove that $n^3 - 3n^2 - n + 1 = \Theta\left(n^3\right)$.
2-13. [3] Prove that $n^2 = O\left(2^n\right)$.
2-14. [3] Prove or disprove: $\Theta\left(n^2\right) = \Theta\left(n^2 + 1\right)$.
2-15. [3] Suppose you have algorithms with the five running times listed below. (Assume these are the exact running times.) How much slower do each of these algorithms get when you (a) double the input size, or (b) increase the input size by one?
(a) n^2 *(b)* n^3
(c) $100n^2$
(d) $n\log n$ *(e)* 2^n

2-16. [3] Suppose you have algorithms with the six running times listed below. (Assume these are the exact number of operations performed as a function of the input size n.) Suppose you have a computer that can perform 10^{10} *operations per second. For each algorithm, what is the largest input size n that you can complete within an hour? (a)* n^2 *(b)* n^3 *(c)* $100n^2$ *(d)* $n\log n$ *(e)* 2^n *(f)* 2^{2^n}
2-17. [3] For each of the following pairs of functions $f(n)$ *and* $g(n)$, *give an appropriate positive constant c such that* $f(n) \leq c \cdot g(n)$ *for all* $n > 1$.
(a) $f(n) = n^2 + n + 1, g(n) = 2n^3$
(b) $f(n) = n\sqrt{n} + n^2, g(n) = n^2$
(c) $f(n) = n^2 - n + 1, g(n) = n^2/2$

2-18. [3] Prove that if $f_1(n) = O\left(g_1(n)\right)$ *and* $f_2(n) = O\left(g_2(n)\right)$, *then* $f_1(n) + f_2(n) = O\left(g_1(n) + g_2(n)\right)$.
2-19. [3] Prove that if $f_1(n) = \Omega\left(g_1(n)\right)$ *and* $f_2(n) = \Omega\left(g_2(n)\right)$, *then* $f_1(n) + f_2(n) = \Omega\left(g_1(n) + g_2(n)\right)$.
2-20. [3] Prove that if $f_1(n) = O\left(g_1(n)\right)$ *and* $f_2(n) = O\left(g_2(n)\right)$, *then* $f_1(n) \cdot f_2(n) = O\left(g_1(n) \cdot g_2(n)\right)$.
2-21. [5] Prove that for all $k \geq 0$ *and all sets of real constants* $\{a_k, a_{k-1}, \ldots, a_1, a_0\}$,

$$a_k n^k + a_{k-1}n^{k-1} + \ldots. + a_1 n + a_0 = O\left(n^k\right)$$

2-22. [5] Show that for any real constants a and $b, b > 0$

$$(n+a)^b = \Theta\left(n^b\right)$$

2-23. [5] List the functions below from the lowest to the highest order. If any two or more are of the same order, indicate which.

n	2^n	$n \lg n$	$\ln n$
$n - n^3 + 7n^5$	$\lg n$	$\sqrt{n}$	e^n
$n^2 + \lg n$	n^2	2^{n-1}	$\lg \lg n$
n^3	$(\lg n)^2$	$n!$	$n^{1+\varepsilon}$ *where* $0 < \varepsilon < 1$

2-24. [8] List the functions below from the lowest to the highest order. If any two or more are of the same order, indicate which.

n^π	π^n	$\binom{n}{5}$	$\sqrt{2\sqrt{n}}$
$\binom{n}{n-4}$	$2^{\log^4 n}$	$n^{5(\log n)^2}$	$n^4\binom{n}{n-4}$

2-25. [8] List the functions below from the lowest to the highest order. If any two or more are of the same order, indicate which.

$\sum_{i=1}^{n} i^i$	n^n	$(\log n)^{\log n}$	$2^{\left(\log n^2\right)}$
$n!$	$2^{\log^4 n}$	$n^{(\log n)^2}$	$\binom{n}{n-4}$

2-26. [5] List the functions below from the lowest to the highest order. If any two or more are of the same order, indicate which.

$\sqrt{n}$	n	2^n
$n \log n$	$n - n^3 + 7n^5$	$n^2 + \log n$
n^2	n^3	$\log n$
$n^{\frac{1}{3}} + \log n$	$(\log n)^2$	$n!$
$\ln n$	$\frac{n}{\log n}$	$\log \log n$
$(1/3)^n$	$(3/2)^n$	*6*

2-27. [5] Find two functions $f(n)$ and $g(n)$ that satisfy the following relationship. If no such f and g exist, write "None."
(a) $f(n) = o(g(n))$ *and* $f(n) \neq \Theta(g(n))$
(b) $f(n) = \Theta(g(n))$ *and* $f(n) = o(g(n))$
(c) $f(n) = \Theta(g(n))$ *and* $f(n) \neq O(g(n))$
(d) $f(n) = \Omega(g(n))$ *and* $f(n) \neq O(g(n))$

2-28. [5] True or False?
(a) $2n^2 + 1 = O\left(n^2\right)$
(b) $\sqrt{n} = O(\log n)$
(c) $\log n = O(\sqrt{n})$
(d) $n^2(1+\sqrt{n}) = O\left(n^2 \log n\right)$
(e) $3n^2 + \sqrt{n} = O\left(n^2\right)$
(f) $\sqrt{n} \log n = O(n)$
(g) $\log n = O\left(n^{-1/2}\right)$

2-29. [5] For each of the following pairs of functions $f(n)$ and $g(n)$, state whether $f(n)=O(g(n)), f(n)=\Omega(g(n)), f(n)=\Theta(g(n))$, or none of the above.
(a) $f(n)=n^2+3n+4, g(n)=6n+7$
(b) $f(n)=n\sqrt{n}, g(n)=n^2-n$
(c) $f(n)=2^n-n^2, g(n)=n^4+n^2$

2-30. [3] For each of these questions, answer yes or no and briefly explain your answer.
(a) If an algorithm takes $O\left(n^2\right)$ worst-case time, is it possible that it takes $O(n)$ on some inputs?
(b) If an algorithm takes $O\left(n^2\right)$ worst-case time, is it possible that it takes $O(n)$ on all inputs?
(c) If an algorithm takes $\Theta\left(n^2\right)$ worst-case time, is it possible that it takes $O(n)$ on some inputs?
(d) If an algorithm takes $\Theta\left(n^2\right)$ worst-case time, is it possible that it takes $O(n)$ on all inputs?
(e) Is the function $f(n)=\Theta\left(n^2\right)$, where $f(n)=100n^2$ for even n and $f(n)=20n^2-n\log_2 n$ for odd n ?
2-31. [3] For each of the following, answer yes, no, or can't tell. Explain your reasoning.
(a) Is $3^n=O\left(2^n\right)$?
(b) Is $\log 3^n=O\left(\log 2^n\right)$?
(c) Is $3^n=\Omega\left(2^n\right)$?
(d) Is $\log 3^n=\Omega\left(\log 2^n\right)$?

2-32. [5] For each of the following expressions $f(n)$ find a simple $g(n)$ such that $f(n)=\Theta(g(n))$.
(a) $f(n)=\sum_{i=1}^{n}\frac{1}{i}$.
(b) $f(n)=\sum_{i=1}^{n}\left\lceil\frac{1}{i}\right\rceil$.
(c) $f(n)=\sum_{i=1}^{n}\log i$.
(d) $f(n)=\log(n!)$.

2-33. [5] Place the following functions into increasing order: $f_1(n)=n^2\log_2 n$, $f_2(n)=n\left(\log_2 n\right)^2, f_3(n)=\sum_{i=0}^{n}2^i$ *and,* $f_4(n)=\log_2\left(\sum_{i=0}^{n}2^i\right)$.
$2-34$. *[5] Which of the following are true?*
(a) $\sum_{i=1}^{n}3^i=\Theta\left(3^{n-1}\right)$.
(b) $\sum_{i=1}^{n}3^i=\Theta\left(3^{n}\right)$.
(c) $\sum_{i=1}^{n}3^i=\Theta\left(3^{n+1}\right)$.

2-35. [5] For each of the following functions f find a simple function g such that $f(n)=\Theta(g(n))$.
(a) $f_1(n)=(1000)2^n+4^n$.
(b) $f_2(n)=n+n\log n+\sqrt{n}$.
(c) $f_3(n)=\log\left(n^{20}\right)+(\log n)^{10}$.
(d) $f_4(n)=(0.99)^n+n^{100}$.

2-36. [5] For each pair of expressions (A,B) below, indicate whether A is O,o,Ω,ω, or Θ of B. Note that zero, one, or more of these relations may hold for a given pair; list all correct ones.

	A	B
(a)	n^{100}	2^n
(b)	$(\lg n)^{12}$	$\sqrt{n}$
(c)	$\sqrt{n}$	$n^{\cos(\pi n/8)}$
(d)	10^n	100^n
(e)	$n^{\lg n}$	$(\lg n)^n$
(f)	$\lg(n!)$	$n \lg n$

Summations 2-37. [5] Find an expression for the sum of the i th row of the following triangle, and prove its correctness. Each entry is the sum of the three entries directly above it. All non-existing entries are considered 0 .

				1				
			1	1	1			
		1	2	3	2	1		
	1	3	6	7	6	3	1	
1	4	10	16	19	16	10	4	1

2-38. [3] Assume that Christmas has n days. Exactly how many presents did my "true love" send to me? (Do some research if you do not understand this question.)

2-39. [5] An unsorted array of size n contains distinct integers between 1 and $n+1$, with one element missing. Give an $O(n)$ algorithm to find the missing integer, without using any extra space.

2-40. [5] Consider the following code fragment:

```
for i=1 to n do
    for j=i to 2*i do
        output ''foobar''
```

Let $T(n)$ denote the number of times 'foobar' is printed as a function of n.

a. Express $T(n)$ as a summation (actually two nested summations).

b. Simplify the summation. Show your work.

2-41. [5] Consider the following code fragment:

```
for i=1 to n/2 do
    for j=i to n-i do
        for k=1 to j do
            output ''foobar')
```

Assume n is even. Let $T(n)$ denote the number of times "foobar" is printed as a function of n.

(a) Express $T(n)$ as three nested summations.

(b) Simplify the summation. Show your work.

2-42. [6] When you first learned to multiply numbers, you were told that $x \times y$ means adding x a total of y times, so $5 \times 4 = 5+5+5+5 = 20$. What is the time *complexity of multiplying two n-digit numbers in base b (people work in base 10, of course, while computers work in base 2) using the repeated addition method, as a function of n and b. Assume that single-digit by single-digit addition or multiplication takes $O(1)$ time. (Hint: how big can y be as a function of n and b?)*

2-43. [6] In grade school, you learned to multiply long numbers on a digit-by-digit basis, so that $127 \times 211 = 127 \times 1 + 127 \times 10 + 127 \times 200 = 26,797$. Analyze the time complexity of multiplying two n-digit numbers with this method as a function of n (assume constant base size). Assume that single-digit by singledigit addition or multiplication takes $O(1)$ time.

Logarithms 2-44. [5] Prove the following identities on logarithms:
(a) $\log_a(xy) = \log_a x + \log_a y$
(b) $\log_a x^y = y \log_a x$
(c) $\log_a x = \frac{\log_b x}{\log_b a}$
(d) $x^{\log_b y} = y^{\log_b x}$

2-45. [3] Show that $\lceil \lg(n+1) \rceil = \lfloor \lg n \rfloor + 1$
2-46. [3] Prove that that the binary representation of $n \geq 1$ has $\lfloor \lg_2 n \rfloor + 1$ bits.
2-47. [5] In one of my research papers I give a comparison-based sorting algorithm that runs in $O(n \log(\sqrt{n}))$. Given the existence of an $\Omega(n \log n)$ lower bound for sorting, how can this be possible?

Interview Problems 2-48. [5] You are given a set S of n numbers. You must pick a subset S' of k numbers from S such that the probability of each element of S occurring in S' is equal (i.e., each is selected with probability k/n). You may make only one pass over the numbers. What if n is unknown?
2-49. [5] We have 1,000 data items to store on 1,000 nodes. Each node can store copies of exactly three different items. Propose a replication scheme to minimize data loss as nodes fail. What is the expected number of data entries that get lost when three random nodes fail?

2-50. [5] Consider the following algorithm to find the minimum element in an array of numbers $A[0, \ldots, n]$. One extra variable tmp is allocated to hold the current minimum value. Start from A[0]; tmp is compared against $A[1], A[2], \ldots, A[N]$ in order. When $A[i] < tmp, tmp = A[i]$. What is the expected number of times that the assignment operation $\mathrm{tmp} = A[i]$ *is performed?*
2-51. [5] You are given ten bags of gold coins. Nine bags contain coins that each weigh 10 grams. One bag contains all false coins that weigh 1 gram less. You must identify this bag in just one weighing. You have a digital balance that reports the weight of what is placed on it.
2-52. [5] You have eight balls all of the same size. Seven of them weigh the same, and one of them weighs slightly more. How can you find the ball that is heavier by using a balance and only two weighings?
2-53. [5] Suppose we start with n companies that eventually merge into one big company. How many different ways are there for them to merge?
2-54. [7] Six pirates must divide \$300 *among themselves. The division is to proceed as follows. The senior pirate proposes a way to divide the money. Then the pirates vote. If the senior pirate gets at least half the votes he wins, and that*

division remains. If he doesn't, he is killed and then the next senior-most pirate gets a chance to propose the division. Now tell what will happen and why (i.e. how many pirates survive and how the division is done)? All the pirates are intelligent and the first priority is to stay alive and the next priority is to get as much money as possible.

2-55. [7] Reconsider the pirate problem above, where we start with only one indivisible dollar. Who gets the dollar, and how many are killed?

LeetCode *2-1. https://leetcode.com/problems/remove-k-digits/*

2-2. https://leetcode.com/problems/counting-bits/

2-3. https://leetcode.com/problems/4sum/

HackerRank *2-1. https://www.hackerrank.com/challenges/pangrams/*

2-2. https://www.hackerrank.com/challenges/the-power-sum/

2-3. https://www.hackerrank.com/challenges/magic-square-forming/

Programming Challenges *These programming challenge problems with robot judging are available at https://onlinejudge.org;*

2-1. "Primary Arithmetic"-Chapter 5, problem 10035.

2-2. "A Multiplication Game"-Chapter 5, problem 847.

2-3. "Light, More Light" - Chapter 7, problem 10110.

Chapter 3

Chapter 6

Data Structures

Putting the right data structure into a slow program can work the same wonders as transplanting fresh parts into a sick patient. Important classes of abstract data types such as containers, dictionaries, and priority queues have many functionally equivalent data structures that implement them. Changing the data structure does not affect the correctness of the program, since we presumably replace a correct implementation with a different correct implementation. However, the new implementation may realize different trade-offs in the time to execute various operations, so the total performance can improve dramatically. Like a patient in need of a transplant, only one part might need to be replaced in order to fix the problem.

But it is better to be born with a good heart than have to wait for a replacement. The maximum benefit from proper data structures results from designing your program around them in the first place. We assume that the reader has had some previous exposure to elementary data structures and pointer manipulation. Still, data structure courses (CS II) focus more on data abstraction and object orientation than the nitty-gritty of how structures should be represented in memory. This material will be reviewed here to make sure you have it down.

As with most subjects, in data structures it is more important to really understand the basic material than to have exposure to more advanced concepts. This chapter will focus on each of the three fundamental abstract data types (containers, dictionaries, and priority queues) and show how they can be implemented with arrays and lists. Detailed discussion of the trade-offs between more sophisticated implementations is deferred to the relevant catalog entry for each of these data types.

6.1 Contiguous vs. Linked Data Structures

Data structures can be neatly classified as either contiguous or linked, depending upon whether they are based on arrays or pointers. Contiguously allocated structures are composed of single slabs of memory, and include arrays, matrices, *heaps, and hash tables. Linked data structures are composed of distinct chunks of memory bound together by pointers, and include lists, trees, and graph adjacency lists.*

In this section, I review the relative advantages of contiguous and linked data structures. These trade-offs are more subtle than they appear at first glance, so I encourage readers to stick with me here even if you may be familiar with both types of structures.

6.1.1 Arrays

The array is the fundamental contiguously allocated data structure. Arrays are structures of fixed-size data records such that each element can be efficiently located by its index or (equivalently) address.

A good analogy likens an array to a street full of houses, where each array element is equivalent to a house, and the index is equivalent to the house number. Assuming all the houses are of equal size and numbered sequentially from 1 to n, we can compute the exact position of each house immediately from its address. 1

Advantages of contiguously allocated arrays include:

- *Constant-time access given the index - Because the index of each element maps directly to a particular memory address, we can access arbitrary data items instantly provided we know the index.*
- *Space efficiency - Arrays consist purely of data, so no space is wasted with links or other formatting information. Further, end-of-record information is not needed because arrays are built from fixed-size records.*
- *Memory locality - Many programming tasks require iterating through all the elements of a data structure. Arrays are good for this because they exhibit excellent memory locality. Physical continuity between successive data accesses helps exploit the high-speed cache memory on modern computer architectures.*

The downside of arrays is that we cannot adjust their size in the middle of a program's execution. Our program will fail as soon as we try to add the $(n+1)$ st customer, if we only allocated room for n records. We can compensate by allocating extremely large arrays, but this can waste space, again restricting what our programs can do.

Actually, we can efficiently enlarge arrays as we need them, through the miracle of dynamic arrays. Suppose we start with an array of size 1, and double its size from m to $2m$ whenever we run out of space. This doubling process allocates a new contiguous array of size $2m$, copies the contents of the old array to the lower half of the new one, and then returns the space used by the old array to the storage allocation system.

[1] Houses in Japanese cities are traditionally numbered in the order they were built, not by their physical location. This makes it extremely difficult to locate a Japanese address without a detailed map.

```
typedef struct list {
    item_type item;             /* data item */
    struct list *next;          /* point to successor */
} list;
```

The apparent waste in this procedure involves recopying the old contents on each expansion. How much work do we really do? It will take $\log_2 n$ *(also known as* $\lg n$ *) doublings until the array gets to have n positions, plus one final doubling on the last insertion when* $n = 2^j$ *for some* j*. There are recopying operations after the first, second, fourth, eighth, ...,* n*th insertions. The number of copy operations at the* i *th doubling will be* 2^{i-1}*, so the total number of movements* M *will be:*

$$M = n + \sum_{i=1}^{\lg n} 2^{i-1} = 1 + 2 + 4 + \ldots + \frac{n}{2} + n = \sum_{i=i}^{\lg n} \frac{n}{2^i} \leq n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

Thus, each of the n *elements move only two times on average, and the total work of managing the dynamic array is the same* $O(n)$ *as it would have been if a single array of sufficient size had been allocated in advance!*

The primary thing lost in using dynamic arrays is the guarantee that each insertion takes constant time in the worst case. Note that all accesses and most insertions will be fast, except for those relatively few insertions that trigger array doubling. What we get instead is a promise that the n*th element insertion will be completed quickly enough that the total effort expended so far will still be* $O(n)$*. Such amortized guarantees arise frequently in the analysis of data structures.*

Pointers and Linked Structures Pointers are the connections that hold the pieces of linked structures together. Pointers represent the address of a location in memory. A variable storing a pointer to a given data item can provide more freedom than storing a copy of the item itself. A cell-phone number can be thought of as a pointer to its owner as they move about the planet.

Pointer syntax and power differ significantly across programming languages, so we begin with a quick review of pointers in C language. A pointer p is assumed to give the address in memory where a particular chunk of data is located n^2 *Pointers in C have types declared at compile time, denoting the data type of the items they can point to. We use* $*\mathrm{p}$ *to denote the item that is pointed to by pointer* p *, and* $\&x$ *to denote the address of (i.e.pointer to) a particular variable* x *. A special NULL pointer value is used to denote structure-terminating or unassigned pointers.*

All linked data structures share certain properties, as revealed by the following type declaration for linked lists:

[2]C permits direct manipulation of memory addresses in ways that may horrify Java programmers, but I will avoid doing any such tricks.

Figure 6.1: Linked list example showing data and pointer fields.

```
typedef struct list {
    item_type item; /* data item */
    struct list *next; /* point to successor */
} list;
```

In particular:

- *Each node in our data structure (here list) contains one or more data fields (here item) that retain the data that we need to store.*
- *Each node contains a pointer field to at least one other node (here next). This means that much of the space used in linked data structures is devoted to pointers, not data.*
- *Finally, we need a pointer to the head of the structure, so we know where to access it.*

The list here is the simplest linked structure. The three basic operations supported by lists are searching, insertion, and deletion. In doubly linked lists, each node points both to its predecessor and its successor element. This simplifies certain operations at a cost of an extra pointer field per node.

Searching a List Searching for item x in a linked list can be done iteratively or recursively. I opt for recursively in the implementation below. If x is in the list, it is either the first element or located in the rest of the list. Eventually, the problem is reduced to searching in an empty list, which clearly cannot contain x.

```
list *search_list(list *l, item_type x) {
    if (l == NULL) {
        return(NULL);
    }
    if (l->item == x) {
        return(l);
    } else {
        return(search_list(l->next, x));
    }
}
```

Insertion into a List *Insertion into a singly linked list is a nice exercise in pointer manipulation, as shown below. Since we have no need to maintain the list in any particular order, we might as well insert each new item in the* *most convenient place. Insertion at the beginning of the list avoids any need to traverse the list, but does require us to update the pointer (denoted l) to the head of the data structure.*

```
void insert_list(list **l, item_type x) {
    list *p; /* temporary pointer */\index{priority queues}
    p = malloc(sizeof(list));
    p->item = x;
    p->next = *l;
    *l = p;
}
```

Two C-isms to note. First, the malloc function allocates a chunk of memory of sufficient size for a new node to contain x. Second, the funny double star in $**l$ *denotes that 1 is a pointer to a pointer to a list node. Thus, the last line,* $*l = \mathrm{p}$*; copies p to the place pointed to by l, which is the external variable maintaining access to the head of the list.*

Deletion From a List *Deletion from a linked list is somewhat more complicated. First, we must find a pointer to the predecessor of the item to be deleted. We do this recursively:*

```
list *item_ahead(list *l, list *x) {
    if ((l == NULL) || (l->next == NULL)) {
        return(NULL);
    }
    if ((l->next) == x) {
        return(l);
    } else {
        return(item_ahead(l->next, x));
    }
}
```

The predecessor is needed because it points to the doomed node, so its next pointer must be changed. The actual deletion operation is simple, once ruling out the case that the to-be-deleted element does not exist. Special care must be taken to reset the pointer to the head of the list (1) when the first element is deleted:

```
void delete_list(list **l, list **x) {
    list *p; /* item pointer */
    list *pred; /* predecessor pointer */
    p = *l;
    pred = item_ahead(*l, *x);
    if (pred == NULL) { /* splice out of list */
        *l = p->next;
    } else {
        pred->next = (*x)->next;
```

```
    }
    free(*x); /* free memory used by node */
}
```

C language requires explicit deallocation of memory, so we must free the deleted node after we are finished with it in order to return the memory to the system. This leaves the incoming pointer as a dangling reference to a location that no longer exists, so care must be taken not to use this pointer again. Such problems can generally be avoided in Java because of its stronger memory management model.

6.1.2 Comparison

The advantages of linked structures over static arrays include:

- *Overflow on linked structures never occurs unless the memory is actually full.*
- *Insertion and deletion are simpler than for static arrays.*
- *With large records, moving pointers is easier and faster than moving the items themselves.*

Conversely, the relative advantages of arrays include:

- *Space efficiency: linked structures require extra memory for storing pointer fields.*
- *Efficient random access to items in arrays.*
- *Better memory locality and cache performance than random pointer jumping.*

Take-Home Lesson: Dynamic memory allocation provides us with flexibility on how and where we use our limited storage resources.

One final thought about these fundamental data structures is that both arrays and linked lists can be thought of as recursive objects:

- *Lists - Chopping the first element off a linked list leaves a smaller linked list. This same argument works for strings, since removing characters from a string leaves a string. Lists are recursive objects.*
- *Arrays - Splitting the first k elements off of an n element array gives two smaller arrays, of size k and $n - k$, respectively. Arrays are recursive objects.*

This insight leads to simpler list processing, and efficient divide-and-conquer algorithms such as quicksort and binary search.

Containers: Stacks and Queues I use the term container to denote an abstract data type that permits storage and retrieval of data items independent of content. By contrast, dictionaries are abstract data types that retrieve based on key values or content, and will be discussed in Section 3.3 (page 76).

Containers are distinguished by the particular retrieval order they support. In the two most important types of containers, this retrieval order depends on the insertion order:

- *Stacks support retrieval by last-in, first-out (LIFO) order. Stacks are simple to implement and very efficient. For this reason, stacks are probably the right container to use when retrieval order doesn't matter at all, such as when processing batch jobs. The put and get operations for stacks are usually called push and pop:*
- *Push* (x, s) : *Insert item* x *at the top of stack* s*.*
- *Pop(s): Return (and remove) the top item of stack* s*.*

LIFO order arises in many real-world contexts. People crammed into a subway car exit in LIFO order. Food inserted into my refrigerator usually exits the same way, despite the incentive of expiration dates. Algorithmically, LIFO tends to happen in the course of executing recursive algorithms.

- *Queues support retrieval in first-in, first-out (FIFO) order. This is surely the fairest way to control waiting times for services. Jobs processed in FIFO order minimize the maximum time spent waiting. Note that the average waiting time will be the same regardless of whether FIFO or LIFO is used. Many computing applications involve data items with infinite patience, which renders the question of maximum waiting time moot.*
  Queues are somewhat trickier to implement than stacks and thus are most appropriate for applications (like certain simulations) where the order is important. The put and get operations for queues are usually called enqueue and dequeue.
- *Enqueue* (x, q) *: Insert item* x *at the back of queue* q*.*
- *Dequeue (q): Return (and remove) the front item from queue* q*.*

We will see queues later as the fundamental data structure controlling breadth-first search (BFS) in graphs.

Stacks and queues can be effectively implemented using either arrays or linked lists. The key issue is whether an upper bound on the size of the container is known in advance, thus permitting the use of a statically allocated array.

6.2 Dictionaries

The dictionary data type permits access to data items by content. You stick an item into a dictionary so you can find it when you need it. The primary operations dictionaries support are:

- Search(D, k) - *Given a search key k, return a pointer to the element in dictionary D whose key value is k, if one exists.*

- Insert(D, x) - *Given a data item x, add it to the dictionary D.*

- *Delete* (D, x) - *Given a pointer x to a given data item in the dictionary D, remove it from D.*

Certain dictionary data structures also efficiently support other useful operations:

- Max(D) *or* Min(D) - *Retrieve the item with the largest (or smallest) key from D. This enables the dictionary to serve as a priority queue, as will be discussed in Section 3.5 (page 87).*

- *Predecessor* (D, x) *or* Successor(D, x) - *Retrieve the item from D whose key is immediately before (or after) item x in sorted order. These enable us to iterate through the elements of the data structure in sorted order.*

Many common data processing tasks can be handled using these dictionary operations. For example, suppose we want to remove all duplicate names from a mailing list, and print the results in sorted order. Initialize an empty dictionary D, whose search key will be the record name. Now read through the mailing list, and for each record search to see if the name is already in D. If not, insert it into D. After reading through the mailing list, we print the names in the dictionary. By starting from the first item Min(D) *and repeatedly calling Successor until we obtain* Max(D), *we traverse all elements in sorted order.*

By defining such problems in terms of abstract dictionary operations, we can ignore the details of the data structure's representation and focus on the task at hand.

In the rest of this section, we will carefully investigate simple dictionary implementations based on arrays and linked lists. More powerful dictionary implementations such as binary search trees (see Section 3.4 (page 81) and hash tables (see Section 3.7 (page 93) are also attractive options in practice. A complete discussion of different dictionary data structures is presented in the catalog in Section 15.1 (page 440). I encourage the reader to browse through the data structures section of the catalog to better learn what your options are.

Stop and Think: Comparing Dictionary Implementations (I) *Problem: What are the asymptotic worst-case running times for all seven fundamental dictionary operations (search, insert, delete, successor, predecessor, minimum, and maximum) when the data structure is implemented as:*

- *An unsorted array.*

- *A sorted array.*

Dictionary operation	*Unsorted array*	*Sorted array*
Search(A, k)	$O(n)$	$O(\log n)$
Insert(A, x)	$O(1)$	$O(n)$
Delete (A, x)	$O(1)^*$	$O(n)$
Successor (A, x)	$O(n)$	$O(1)$
Predecessor (A, x)	$O(n)$	$O(1)$
Minimum (A)	$O(n)$	$O(1)$
Maximum (A)	$O(n)$	$O(1)$
height		

We must understand the implementation of each operation to see why. First, let's discuss the operations when maintaining an unsorted array A.

- *Search is implemented by testing the search key k against (potentially) each element of an unsorted array. Thus, search takes linear time in the worst case, which is when key k is not found in A.*

- *Insertion is implemented by incrementing n and then copying item x to the nth cell in the array, $A[n]$. The bulk of the array is untouched, so this operation takes constant time.*

- *Deletion is somewhat trickier, hence the asterisk in the table above. The definition states that we are given a pointer x to the element to delete, so we need not spend any time searching for the element. But removing the x th element from the array A leaves a hole that must be filled. We could fill the hole by moving each of the elements from $A[x+1]$ to $A[n]$ down one position, but this requires $\Theta(n)$ time when the first element is deleted. The following idea is better: just overwrite $A[x]$ with $A[n]$, and decrement n. This only takes constant time.*

- *The definitions of the traversal operations, Predecessor and Successor, refer to the item appearing before/after x in sorted order. Thus, the answer is not simply $A[x-1]$ (or $A[x+1]$), because in an unsorted array an element's physical predecessor (successor) is not necessarily its logical predecessor (successor). Instead, the predecessor of $A[x]$ is the biggest element smaller than $A[x]$. Similarly, the successor of $A[x]$ is the smallest element larger than $A[x]$. Both require a sweep through all n elements of A to determine the winner.*

Solution: This problem (and the one following it) reveals some of the inherent trade-offs of data structure design. A given data representation may permit efficient implementation of certain operations at the cost that other operations are expensive.

In addition to the array in question, we will assume access to a few extra variables such as n, the number of elements currently in the array. Note that we must *maintain* the value of these variables in the operations where they change (e.g., insert and delete), and charge these operations the cost of this maintenance.

The basic dictionary operations can be implemented with the following costs on unsorted and sorted arrays. The starred element indicates cleverness.

- *Minimum* and *Maximum* are similarly defined with respect to sorted order, and so require linear-cost sweeps to identify in an unsorted array. It is tempting to set aside extra variables containing the current minimum and maximum values, so we can report them in $O(1)$ time. But this is incompatible with constant-time deletion, as deleting the minimum valued item mandates a linear-time search to find the new minimum.

Implementing a dictionary using a sorted array completely reverses our notions of what is easy and what is hard. Searches can now be done in $O(\log n)$ time, using binary search, because we know the median element sits in $A[n/2]$. Since the upper and lower portions of the array are also sorted, the search can continue recursively on the appropriate portion. The number of halvings of n until we get to a single element is $\lceil \lg n \rceil$.

The sorted order also benefits us with respect to the other dictionary retrieval operations. The minimum and maximum elements sit in $A[1]$ and $A[n]$, while the predecessor and successor to $A[x]$ are $A[x-1]$ and $A[x+1]$, respectively.

Insertion and deletion become more expensive, however, because making room for a new item or filling a hole may require moving many items arbitrarily. Thus, both become linear-time operations.

Take-Home Lesson: Data structure design must balance all the different operations it supports. The fastest data structure to support both operations A and B may well not be the fastest structure to support just operation A or B.

Stop and Think: Comparing Dictionary Implementations (II) Problem: What are the asymptotic worst-case running times for each of the seven fundamental dictionary operations when the data structure is implemented as

- A singly linked unsorted list.
- A doubly linked unsorted list.
- A singly linked sorted list.
- A doubly linked sorted list.

Solution: Two different issues must be considered in evaluating these implementations: singly vs. doubly linked lists and sorted vs. unsorted order. Operations with subtle implementations are denoted with an asterisk:

	Singly linked		Doubly linked	
Dictionary operation	unsorted	sorted	unsorted	sorted
Search (L, k)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Insert (L, x)	$O(1)$	$O(n)$	$O(1)$	$O(n)$
Delete (L, x)	$O(n)^*$	$O(n)^*$	$O(1)$	$O(1)$
Successor (L, x)	$O(n)$	$O(1)$	$O(n)$	$O(1)$
Predecessor (L, x)	$O(n)$	$O(n)^*$	$O(n)$	$O(1)$
Minimum (L)	$O(1)^*$	$O(1)$	$O(n)$	$O(1)$
Maximum (L)	$O(1)^*$	$O(1)^*$	$O(n)$	$O(1)$

As with unsorted arrays, search operations are destined to be slow while maintenance operations are fast.

- *Insertion/Deletion - The complication here is deletion from a singly linked list. The definition of the Delete operation states we are given a pointer x to the item to be deleted. But what we really need is a pointer to the element pointing to x in the list, because that is the node that needs to be changed. We can do nothing without this list predecessor, and so must spend linear time searching for it on a singly linked list. Doubly linked lists avoid this problem, since we can immediately retrieve the list predecessor of x*

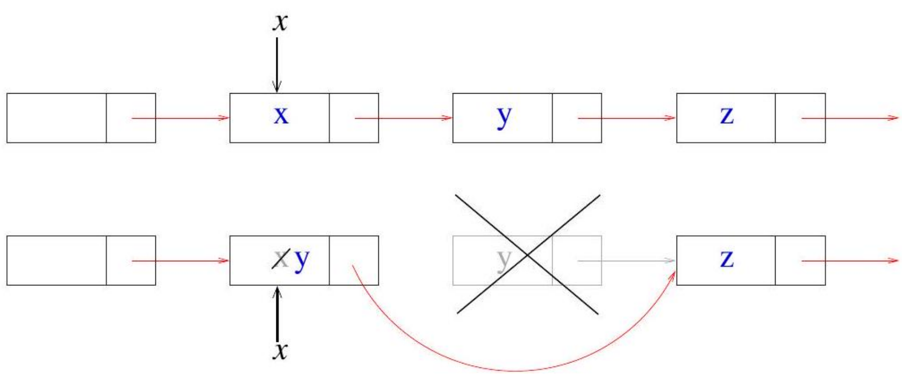


Figure 6.2: By overwriting the contents of a node-to-delete, and deleting its original successor, we can delete a node without access to its list predecessor.

Deletion is faster for sorted doubly linked lists than sorted arrays, because splicing out the deleted element from the list is more efficient than filling the hole by moving array elements. The predecessor pointer problem again complicates deletion from singly linked sorted lists.

- *Search - Sorting provides less benefit for linked lists than it did for arrays. Binary search is no longer possible, because we can't access the median element without traversing all the elements before it. What sorted lists do provide is quick termination of unsuccessful searches, for if we have not found Abbott by the time we hit Costello we can deduce that he doesn't exist in the list. Still, searching takes linear time in the worst case.*

[3] Actually, there is a way to delete an element from a singly linked list in constant time, as shown in Figure 3.2 Overwrite the node that x points to with the contents of what x.next points to, then deallocate the node that x.next originally pointed to. Special care must be taken if x is the first node in the list, or the last node (by employing a permanent sentinel element that is always the last node in the list). But this would prevent us from having constant-time minimum/maximum operations, because we no longer have time to find new extreme elements after deletion.

- *Traversal operations - The predecessor pointer problem again complicates implementing Predecessor with singly linked lists. The logical successor is equivalent to the node successor for both types of sorted lists, and hence can be implemented in constant time.*

- *Minimum/Maximum - The minimum element sits at the head of a sorted list, and so is easily retrieved. The maximum element is at the tail of the list, which normally requires $\Theta(n)$ time to reach in either singly or doubly linked lists.*

However, we can maintain a separate pointer to the list tail, provided we pay the maintenance costs for this pointer on every insertion and deletion. The tail pointer can be updated in constant time on doubly linked lists: on insertion check whether last->next still equals NULL, and on deletion set last to point to the list predecessor of last if the last element is deleted.

We have no efficient way to find this predecessor for singly linked lists. So why can we implement Maximum in $O(1)$? The trick is to charge the cost to each deletion, which already took linear time. Adding an extra linear sweep to update the pointer does not harm the asymptotic complexity of Delete, while gaining us Maximum (and similarly Minimum) in constant time as a reward for clear thinking.

```
typedef struct tree {
    item_type item;             /* data item */
    struct tree *parent;        /* pointer to parent */
    struct tree *left;          /* pointer to left child */
    struct tree *right;         /* pointer to right child */
} tree;
```

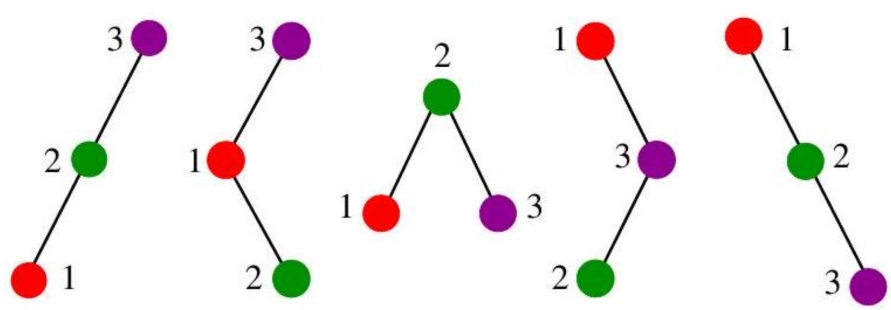


Figure 6.3: The five distinct binary search trees on three nodes. All nodes in the *left* (resp. *right*) subtree of node x have keys $< x$ (resp. $> x$).

6.3 Binary Search Trees

We have seen data structures that allow fast search or flexible update, but not fast search and flexible update. Unsorted, doubly linked lists supported insertion and deletion in $O(1)$ time but search took linear time in the worst case. Sorted arrays support binary search and logarithmic query times, but at the cost of linear-time update.

Binary search requires that we have fast access to two elements-specifically the median elements above and below the given node. To combine these ideas, we need a "linked list" with two pointers per node. This is the basic idea behind binary search trees.

A rooted binary tree is recursively defined as either being (1) empty, or (2) consisting of a node called the root, together with two rooted binary trees called the left and right subtrees, respectively. The order among "sibling" nodes matters in rooted trees, that is, left is different from right. Figure 3.3 gives the shapes of the five distinct binary trees that can be formed on three nodes.

A binary search tree labels each node in a binary tree with a single key such that for any node labeled x, all nodes in the left subtree of x have keys $< x$ while all nodes in the right subtree of x have keys $> xx$[4] *This search tree labeling scheme is very special. For any binary tree on n nodes, and any set of n keys, there is exactly one labeling that makes it a binary search tree. Allowable labelings for three-node binary search trees are given in Figure 3.3*

6.3.1 Implementing Binary Search Trees

Binary tree nodes have left and right pointer fields, an (optional) parent pointer, and a data field. These relationships are shown in Figure 3.4 a type declaration for the tree structure is given below:

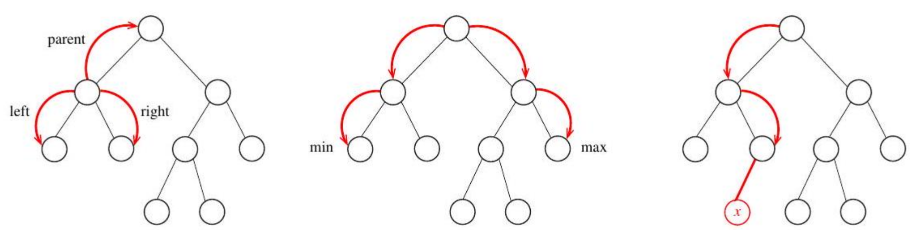


Figure 6.4: Relationships in a binary search tree. Parent and sibling pointers (left). Finding the minimum and maximum elements in a binary search tree (center). Inserting a new node in the correct position (right).

[4] Allowing duplicate keys in a binary search tree (or any other dictionary structure) is bad karma, often leading to very subtle errors. To better support repeated items, we can add a third pointer to each node, explicitly maintaining a list of all items with the given key.

```
typedef struct tree {
    item_type item; /* data item */
    struct tree *parent; /* pointer to parent */
    struct tree *left; /* pointer to left child */
    struct tree *right; /* pointer to right child */
} tree;
```

The basic operations supported by binary trees are searching, traversal, insertion, and deletion.

Searching in a Tree The binary search tree labeling uniquely identifies where each key is located. Start at the root. Unless it contains the query key x, proceed either left or right depending upon whether x occurs before or after the root key. This algorithm works because both the left and right subtrees of a binary search tree are themselves binary search trees. This recursive structure yields the recursive search algorithm below:

```
tree *search_tree(tree *l, item_type x) {
    if (l == NULL) {
        return(NULL);
    }
    if (l->item == x) {
        return(l);
    }
    if (x < l->item) {
        return(search_tree(l->left, x));
    } else {
        return(search_tree(l->right, x));
    }
}
```

This search algorithm runs in $O(h)$ time, where h denotes the height of the tree.

Finding Minimum and Maximum Elements in a Tree By definition, the smallest key must reside in the left subtree of the root, since all keys in the left subtree have values less than that of the root. Therefore, as shown in Figure 3.4 (center), the minimum element must be the left-most descendant of the root. Similarly, the maximum element must be the right-most descendant of the root.

```
tree *find_minimum(tree *t) {
    tree *min; /* pointer to minimum */
    if (t == NULL) {
        return(NULL);
    }
    min = t;
    while (min->left != NULL) {
        min = min->left;
    }
```

```
    return(min);
}
```

Traversal in a Tree Visiting all the nodes in a rooted binary tree proves to be an important component of many algorithms. It is a special case of traversing all the nodes and edges in a graph, which will be the foundation of Chapter 7

A prime application of tree traversal is listing the labels of the tree nodes. Binary search trees make it easy to report the labels in sorted order. By definition, all the keys smaller than the root must lie in the left subtree of the root, and all keys bigger than the root in the right subtree. Thus, visiting the nodes recursively, in accord with such a policy, produces an in-order traversal of the search tree:

```
void traverse_tree(tree *l) {
    if (l != NULL) {
        traverse_tree(l->left);
        process_item(l->item);
        traverse_tree(l->right);
    }
}
```

Each item is processed only once during the course of traversal, so it runs in $O(n)$ time, where n denotes the number of nodes in the tree.

Different traversal orders come from changing the position of process_item relative to the traversals of the left and right subtrees. Processing the item first yields a pre-order traversal, while processing it last gives a post-order traversal. These make relatively little sense with search trees, but prove useful when the rooted tree represents arithmetic or logical expressions.

Insertion in a Tree *There is exactly one place to insert an item x into a binary search tree T so we can be certain where to find it again. We must replace the NULL pointer found in T after an unsuccessful query for the key of x.*

This implementation uses recursion to combine the search and node insertion stages of key insertion. The three arguments to insert_tree are (1) a pointer 1 to the pointer linking the search subtree to the rest of the tree, (2) the key x to be inserted, and (3) a parent pointer to the parent node containing l. The node is allocated and linked in after hitting the NULL pointer. Note that we pass the pointer to the appropriate left/right pointer in the node during the search, so the assignment $*l = p$; links the new node into the tree:

```
void insert_tree(tree **l, item_type x, tree *parent) {
    tree *p; /* temporary pointer */
    if (*l == NULL) {
        p = malloc(sizeof(tree));
        p->item = x;
        p->left = p->right = NULL;
        p->parent = parent;
```

```
        *l = p;
        return;
    }
    if (x < (*l)->item) {
            insert_tree(&((*l)->left), x, *l);
    } else {
        insert_tree(&((*l)->right), x, *l);
    }
}
```

Allocating the node and linking it into the tree is a constant-time operation, after the search has been performed in $O(h)$ time. Here h denotes the height of the search tree.

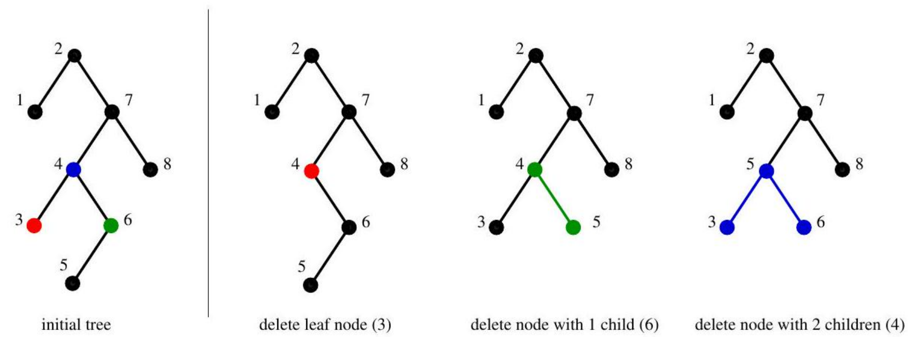


Figure 6.5: Deleting tree nodes with 0,1 , and 2 children. Colors show the nodes affected as a result of the deletion.

Deletion from a Tree *Deletion is somewhat trickier than insertion, because removing a node means appropriately linking its two descendant subtrees back into the tree somewhere else. There are three cases, illustrated in Figure 3.5 Study this figure. Leaf nodes have no children, and so may be deleted simply by clearing the pointer to the given node.*

The case of a doomed node having one child is also straightforward. We can link this child to the deleted node's parent without violating the in-order labeling property of the tree.

But what of a node with two children? Our solution is to relabel this node with the key of its immediate successor in sorted order. This successor must be the smallest value in the right subtree, specifically the left-most descendant in the right subtree p . Moving this descendant to the point of deletion results in a properly labeled binary search tree, and reduces our deletion problem to physically removing a node with at most one child-a case that has been resolved above. The full implementation has been omitted here because it looks a little ghastly, but the code follows logically from the description above.

The worst-case complexity analysis is as follows. Every deletion requires the cost of at most two search operations, each taking $O(h)$ time where h is the height of the tree, plus a constant amount of pointer manipulation.

6.3.2 How Good are Binary Search Trees?

When implemented using binary search trees, all three dictionary operations take $O(h)$ time, where h is the height of the tree. The smallest height we can hope for occurs when the tree is perfectly balanced, meaning that $h = \lceil \log n \rceil$. This is very good, but the tree must be perfectly balanced.

Our insertion algorithm puts each new item at a leaf node where it should have been found. This makes the shape (and more importantly height) of the tree determined by the order in which we insert the keys.

Unfortunately, bad things can happen when building trees through insertion. The data structure has no control over the order of insertion. Consider what happens if the user inserts the keys in sorted order. The operations insert (a), followed by insert (b), insert (c), insert (d), ... will produce a skinny, linearheight tree where only right pointers are used.

Thus, binary trees can have heights ranging from $\lg n$ to n. But how tall are they on average? The average case analysis of algorithms can be tricky because we must carefully specify what we mean by average. The question is well defined if we assume each of the n ! possible insertion orderings to be equally likely, and average over those. If this assumption is valid then we are in luck, because with high probability the resulting tree will have $\Theta(\log n)$ height. This will be shown in Section 4.6 (page 130).

This argument is an important example of the power of randomization. We can often develop simple algorithms that offer good performance with high probability. We will see that a similar idea underlies the fastest known sorting algorithm, quicksort.

6.3.3 Balanced Search Trees

Random search trees are usually good. But if we get unlucky with our order of insertion, we can end up with a linear-height tree in the worst case. This worst case is outside of our direct control, since we must build the tree in response to the requests given by our potentially nasty user.

What would be better is an insertion/deletion procedure that adjusts the tree a little after each insertion, keeping it close enough to be balanced that the maximum height is logarithmic. Sophisticated balanced binary search tree data structures have been developed that guarantee the height of the tree always to be $O(\log n)$. Therefore, all dictionary operations (insert, delete, query) take $O(\log n)$ time each. Implementations of balanced tree data structures such as red-black trees and splay trees are discussed in Section 15.1 (page 440 .

From an algorithm design viewpoint, it is important to know that these trees exist and that they can be used as black boxes to provide an efficient dictionary implementation. When figuring the costs of dictionary operations for algorithm *analysis, we can assume the worst-case complexities of balanced binary trees to be a fair measure.*

Take-Home Lesson: Picking the wrong data structure for the job can be disastrous in terms of performance. Identifying the very best data structure is usually not as critical, because there can be several choices that perform in a similar manner.

Stop and Think: Exploiting Balanced Search Trees *Problem: You are given the task of reading n numbers and then printing them out in sorted order. Suppose you have access to a balanced dictionary data structure, which supports the operations search, insert, delete, minimum,*

maximum, successor, and predecessor each in $O(\log n)$ time. How can you sort in $O(n \log n)$ time using only:

1. *insert and in-order traversal?*
2. *minimum, successor, and insert?*
3. *minimum, insert, and delete?*

Solution: Every algorithm for sorting items using a binary search tree has to start by building the actual tree. This involves initializing the tree (basically setting the pointer t to NULL), and then reading/inserting each of the n items into t. This costs $O(n \log n)$, since each insertion takes at most $O(\log n)$ time. Curiously, just building the data structure is a rate-limiting step for each of our sorting algorithms!

The first problem allows us to do insertion and in-order traversal. We can build a search tree by inserting all n elements, then do a traversal to access the items in sorted order.

The second problem allows us to use the minimum and successor operations after constructing the tree. We can start from the minimum element, and then repeatedly find the successor to traverse the elements in sorted order.

The third problem does not give us successor, but does allow us delete. We can repeatedly find and delete the minimum element to once again traverse all the elements in sorted order.

In summary, the solutions to the three problems are:

Sort1()	*Sort2() So*	
initialize-tree(t)	*initialize-tree(t)*	*initi*
While (not EOF) read(x); insert (x, t)	*While (not EOF) read(x); insert(* x, t *);*	*While (not EO*
Traverse(t)	y = Minimum(t)	y =
	While (y $\neq$ NULL *) do print* (y $\rightarrow$ *item)*	*While (y* $\neq$ NUL
	y = Successor(y, t)	*Dele* y =

Each of these algorithms does a linear number of logarithmic-time operations, and hence runs in $O(n \log n)$ time. The key to exploiting balanced binary search trees is using them as black boxes.

6.4 Priority Queues

Many algorithms need to process items in a specific order. For example, suppose you must schedule jobs according to their importance relative to other jobs. Such

scheduling requires sorting the jobs by importance, and then processing them in this sorted order.

The priority queue is an abstract data type that provides more flexibility than simple sorting, because it allows new elements to enter a system at arbitrary intervals. It can be much more cost-effective to insert a new job into a priority queue than to re-sort everything on each such arrival.

The basic priority queue supports three primary operations:

- *Insert(Q, x)-Given item x, insert it into the priority queue Q.*
- *Find-Minimum (Q) or Find-Maximum (Q) - Return a pointer to the item whose key value is smallest (or largest) among all keys in priority queue Q.*
- *Delete-Minimum (Q) or Delete-Maximum (Q)- Remove the item whose key value is minimum (or maximum) from priority queue Q.*

Naturally occurring processes are often informally modeled by priority queues. Single people maintain a priority queue of potential dating candidates, mentally if not explicitly. One's impression on meeting a new person maps directly to an attractiveness or desirability score, which serves as the key field for inserting this new entry into the "little black book" priority queue data structure. Dating is the process of extracting the most desirable person from the data structure (Find-Maximum), spending an evening to evaluate them better, and then reinserting them into the priority queue with a possibly revised score.

Take-Home Lesson: Building algorithms around data structures such as dictionaries and priority queues leads to both clean structure and good performance.

Stop and Think: Basic Priority Queue Implementations *Problem: What is the worst-case time complexity of the three basic priority queue operations (insert, find-minimum, and delete-minimum) when the basic data structure is as follows:*

- *An unsorted array.*
- *A sorted array.*
- *A balanced binary search tree.*

Solution: There is surprising subtlety when implementing these operations, even using a data structure as simple as an unsorted array. The unsorted array dictionary (discussed on page 77) implements insertion and deletion in constant time, and search and minimum in linear time. A linear-time implementation of delete-minimum can be composed from find-minimum, followed by delete.

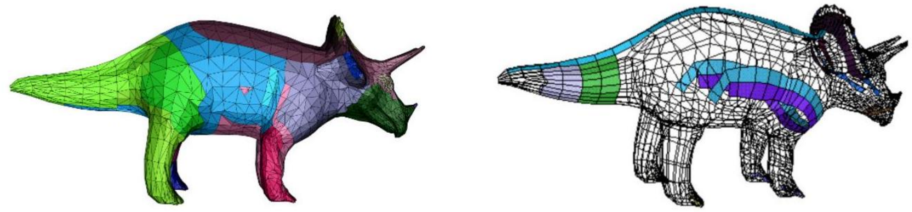


Figure 6.6: A triangulated model of a dinosaur (l), with several triangle strips peeled off the model (r).

For sorted arrays, we can implement insert and delete in linear time, and minimum in constant time. However, priority queue deletions involve only the minimum element. By storing the sorted array in reverse order (largest value on top), the minimum element will always be the last one in the array. Deleting the tail element requires no movement of any items, just decrementing the number of remaining items n, and so delete-minimum can be implemented in constant time.

All this is fine, yet the table below claims we can implement find-minimum in constant time for each data structure:

	Unsorted array	*Sorted array*	*Balanced tree*
Insert (Q, x)	$O(1)$	$O(n)$	$O(\log n)$
Find-Minimum (Q)	$O(1)$	$O(1)$	$O(1)$
Delete-Minimum (Q)	$O(n)$	$O(1)$	$O(\log n)$

The trick is using an extra variable to store a pointer/index to the minimum entry in each of these structures, so we can simply return this value whenever we are asked to find-minimum. Updating this pointer on each insertion is easy-we update it iff the newly inserted value is less than the current minimum. But what happens on a delete-minimum? We can delete that minimum element we point to, and then do a search to restore this canned value. The operation to identify the new minimum takes linear time on an unsorted array and logarithmic time on a tree, and hence can be folded into the cost of each deletion.

Priority queues are very useful data structures. Indeed, they will be the hero of two of our war stories. A particularly nice priority queue implementation (the heap) will be discussed in the context of sorting in Section 4.3 (page 115 . Further, a complete set of priority queue implementations is presented in Section 15.2 (page 445) of the catalog.

6.5 War Story: Stripping Triangulations

Geometric models used in computer graphics are commonly represented by a triangulated surface, as shown in Figure 3.6(1). High-performance rendering

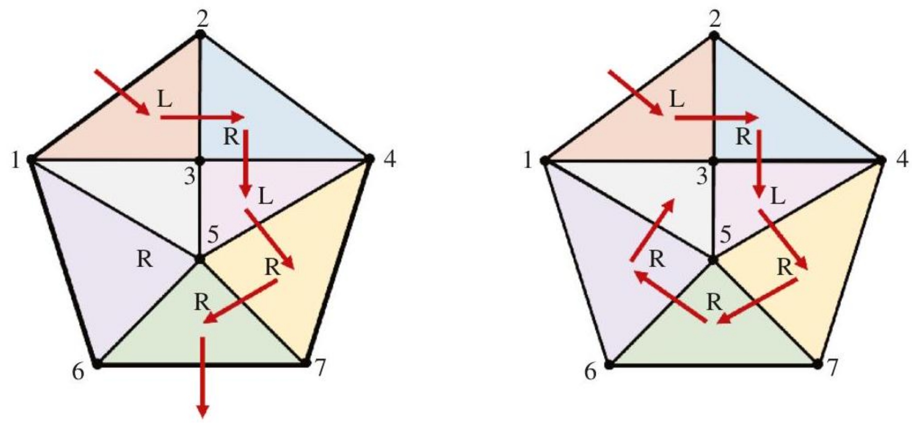


Figure 6.7: A strip with partial coverage using alternating left and right turns (left), and a strip with complete coverage by exploiting the flexibility of arbitrary turns (right).

engines have special hardware for rendering and shading triangles. This rendering hardware is so fast that the computational bottleneck becomes the cost of feeding the triangulation structure into the hardware engine.

Although each triangle can be described by specifying its three endpoints, an alternative representation proves more efficient. Instead of specifying each triangle in isolation, suppose that we partition the triangles into strips of adjacent triangles and walk along the strip. Since each triangle shares two vertices in common with its neighbors, we save the cost of retransmitting the two extra vertices and any associated information. To make the description of the triangles unambiguous, the Open GL triangular-mesh renderer assumes that all turns alternate left and right. The strip in Figure 3.7 (left) completely describes five triangles of the mesh with the vertex sequence $[1, 2, 3, 4, 5, 7, 6]$ *and the implied left/right order. The strip on the right describes all seven triangles with specified turns:* $[1, 2, 3, l-4, r-5, l-7, r-6, r-1, r-3]$.

The task was to find a small number of strips that together cover all the triangle in a mesh, without overlap. This can be thought of as a graph problem. The graph of interest has a vertex for every triangle of the mesh, and an edge between every pair of vertices representing adjacent triangles. This dual graph representation captures all the information about the triangulation (see Section 18.12 (page 581) needed to partition it into strips.

Once we had the dual graph, the project could begin in earnest. We sought to partition the dual graph's vertices into as few paths or strips as possible. Partitioning it into one path implied that we had discovered a Hamiltonian path, which by definition visits each vertex exactly once. Since finding a Hamiltonian *path is NP-complete (see Section 19.5 (page 598), we knew not to look for an optimal algorithm, but concentrate instead on heuristics.*

The simplest approach for strip cover would start from an arbitrary triangle and then do a left-right walk until the walk ends, either by hitting the boundary of the object or a previously visited triangle. This heuristic had the advantage

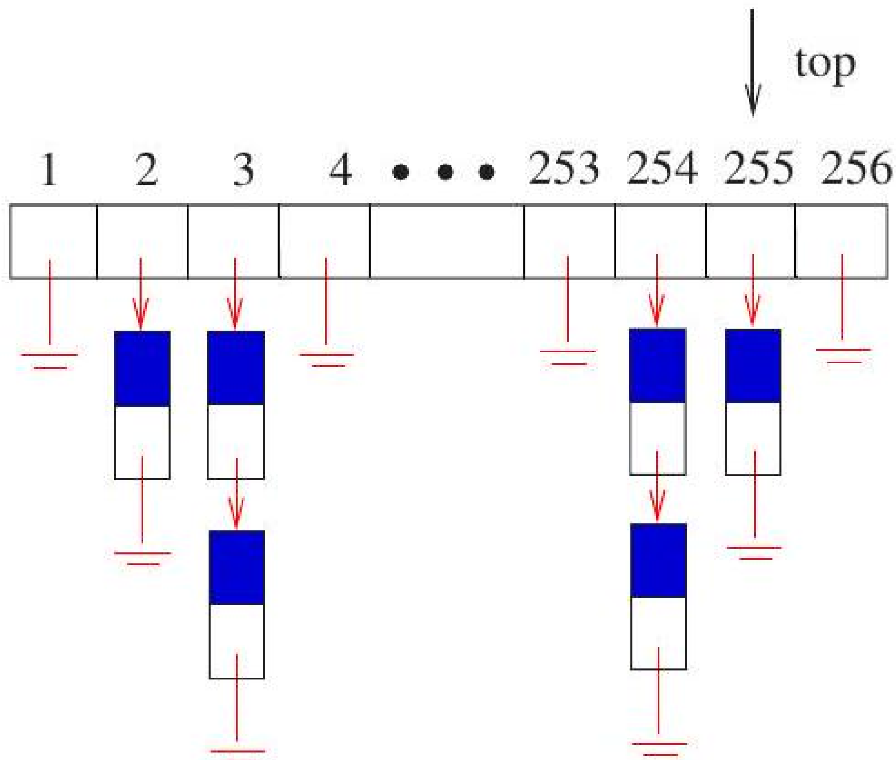


Figure 6.8: A bounded-height priority queue for triangle strips.

that it would be fast and simple, although there is no reason to expect that it must find the smallest possible set of left-right strips for a given triangulation.

The greedy heuristic should result in a relatively small number of strips, however. Greedy heuristics always try to grab the best possible thing first. In the case of the triangulation, the natural greedy heuristic would be to identify the starting triangle that yields the longest left-right strip, and peel that one off first.

Being greedy does not guarantee you the best possible solution overall, since the first strip you peel off might break apart a lot of potential strips we would have wanted to use later. Still, being greedy is a good rule of thumb if you want to get rich. Removing the longest strip leaves the fewest number of triangles *remaining for later strips, so greedy should outperform the naive heuristic of pick anything.*

But how much time does it take to find the largest strip to peel off next? Let k be the length of the walk possible from an average vertex. Using the simplest possible implementation, we could walk from each of the n vertices to find the largest remaining strip to report in $O(kn)$ time. Repeating this for each of the roughly n/k strips we extract yields an $O\left(n^2\right)$-time implementation, which would be hopelessly slow on even a small model of 20,000 triangles.

How could we speed this up? It seems wasteful to re-walk from each triangle after deleting a single strip. We could maintain the lengths of all the possible future strips in a data structure. However, whenever we peel off a strip, we must update the lengths of all affected strips. These strips will be shortened because they walked through a triangle that now no longer exists. There are two aspects of such a data structure:

- *Priority queue - Since we were repeatedly identifying the longest remaining strip, we needed a priority queue to store the strips ordered according to length. The next strip to peel always sits at the top of the queue. Our priority queue had to permit reducing the priority of arbitrary elements of the queue whenever we updated the strip lengths to reflect what triangles were peeled away. Because all of the strip lengths were bounded by a fairly*

Model name	*Triangle count*	*Naive cost*	*Greedy cost*	*Greedy time*
Diver	*3,798*	*8,460*	*4,650*	*6.4 sec*
Heads	*4,157*	*10,588*	*4,749*	*9.9 sec*
Framework	*5,602*	*9,274*	*7,210*	*9.7 sec*
Bart Simpson	*9,654*	*24,934*	*11,676*	*20.5 sec*
Enterprise	*12,710*	*29,016*	*13,738*	*26.2 sec*
Torus	*20,000*	*40,000*	*20,200*	*272.7 sec*
Jaw	*75,842*	*104,203*	*95,020*	*136.2 sec*

Figure 3.9: A comparison of the naive and greedy heuristics for several triangular meshes. Cost is the number of strips. Running time generally scales with triangle count, except for the highly symmetric torus with very long strips.

small integer (hardware constraints prevent any strip from having more than 256 vertices), we used a bounded-height priority queue (an array of buckets, shown in Figure 3.8 and described in Section 15.2 (page 445). An ordinary heap would also have worked just fine.

To update the queue entry associated with each triangle, we needed to quickly find where it was. This meant that we also needed a ...

- *Dictionary - For each triangle in the mesh, we had to find where it was in the queue. This meant storing a pointer to each triangle in a dictionary. By integrating this dictionary with the priority queue, we built a data structure capable of a wide range of operations.*

Although there were various other complications, such as quickly recalculating the length of the strips affected by the peeling, the key idea needed to obtain better performance was to use the priority queue. Run time improved by several orders of magnitude after employing this data structure.

How much better did the greedy heuristic do than the naive heuristic? Study the table in Figure 3.9. In all cases, the greedy heuristic led to a set of strips that cost less, as measured by the total number of vertex occurrences in the strips. The savings ranged from about 10% *to* 50%*, which is quite remarkable since the greatest possible improvement (going from three vertices per triangle down to one) yields a savings of only* 66.6%*.*

After implementing the greedy heuristic with our priority queue data structure, the program ran in $O(n \cdot k)$ *time, where* n *is the number of triangles and* k *is the length of the average strip. Thus, the torus, which consisted of a small number of very long strips, took longer than the jaw, even though the latter contained over three times as many triangles.*

There are several lessons to be gleaned from this story. First, when working with a large enough data set, only linear or near-linear algorithms are likely to be fast enough. Second, choosing the right data structure is often the key to getting the time complexity down. Finally, using the greedy heuristic can

significantly improve performance over the naive approach. How much this improvement will be can only be determined by experimentation.

6.6 Hashing

Hash tables are a very practical way to maintain a dictionary. They exploit the fact that looking an item up in an array takes constant time once you have its index. A hash function is a mathematical function that maps keys to integers. We will use the value of our hash function as an index into an array, and store our item at that position.

The first step of the hash function is usually to map each key (here the string S*) to a big integer. Let* α *be the size of the alphabet on which* S *is written. Let char (c) be a function that maps each symbol of the alphabet to a unique integer from 0 to* $\alpha - 1$*. The function*

$$H(S) = \alpha^{|S|} + \sum_{i=0}^{|S|-1} \alpha^{|S|-(i+1)} \times \mathrm{char}\,(s_i)$$

maps each string to a unique (but large) integer by treating the characters of the string as "digits" in a base- α *number system.*

This creates unique identifier numbers, but they are so large they will quickly exceed the number of desired slots in our hash table (denoted by m *). We must reduce this number to an integer between 0 and* $m - 1$*, by taking the remainder* $H'(S) = H(S) \bmod m$*. This works on the same principle as a roulette wheel. The ball travels a long distance, around and around the circumference-* m *wheel*

$\lfloor H(S)/m \rfloor$ times before settling down to a random bin. If the table size is selected with enough finesse (ideally m is a large prime not too close to $2^i - 1$), the resulting hash values should be fairly uniformly distributed.

6.6.1 Collision Resolution

No matter how good our hash function is, we had better be prepared for collisions, because two distinct keys will at least occasionally hash to the same value. There are two different approaches for maintaining a hash table:

- *Chaining represents a hash table as an array of m linked lists ("buckets"), as shown in Figure 3.10. The i th list will contain all the items that hash to the value of i. Search, insertion, and deletion thus reduce to the corresponding problem in linked lists. If the n keys are distributed uniformly in a table, each list will contain roughly n/m elements, making them a constant size when $m \approx n$.*
  Chaining is very natural, but devotes a considerable amount of memory to pointers. This is space that could be used to make the table larger, which reduces the likelihood of collisions. In fact, the highest-performing hash tables generally rely on an alternative method called open addressing.

0	1	2	3	4	5	6	7	8	9
34	5	55	21		2		3	8	13

Figure 3.11: Collision resolution by open addressing and sequential probing, after inserting the first eight Fibonacci numbers in increasing order with $H(x) = (2x + 1) \bmod 10$. The red elements have been bumped to the first open slot after the desired location.

- *Open addressing maintains the hash table as a simple array of elements (not buckets). Each cell is initialized to null, as shown in Figure 3.11 On each insertion, we check to see whether the desired cell is empty; if so, we insert the item there. But if the cell is already occupied, we must find some other place to put the item. The simplest possibility (called sequential probing) inserts the item into the next open cell in the table. Provided the table is not too full, the contiguous runs of non-empty cells should be fairly short, hence this location should be only a few cells away from its intended position.*
  Searching for a given key now involves going to the appropriate hash value and checking to see if the item there is the one we want. If so, return it. Otherwise we must keep checking through the length of the run. Deletion in an open addressing scheme can get ugly, since removing one element might break a chain of insertions, making some elements inaccessible. We have no alternative but to reinsert all the items in the run that follows the new hole.

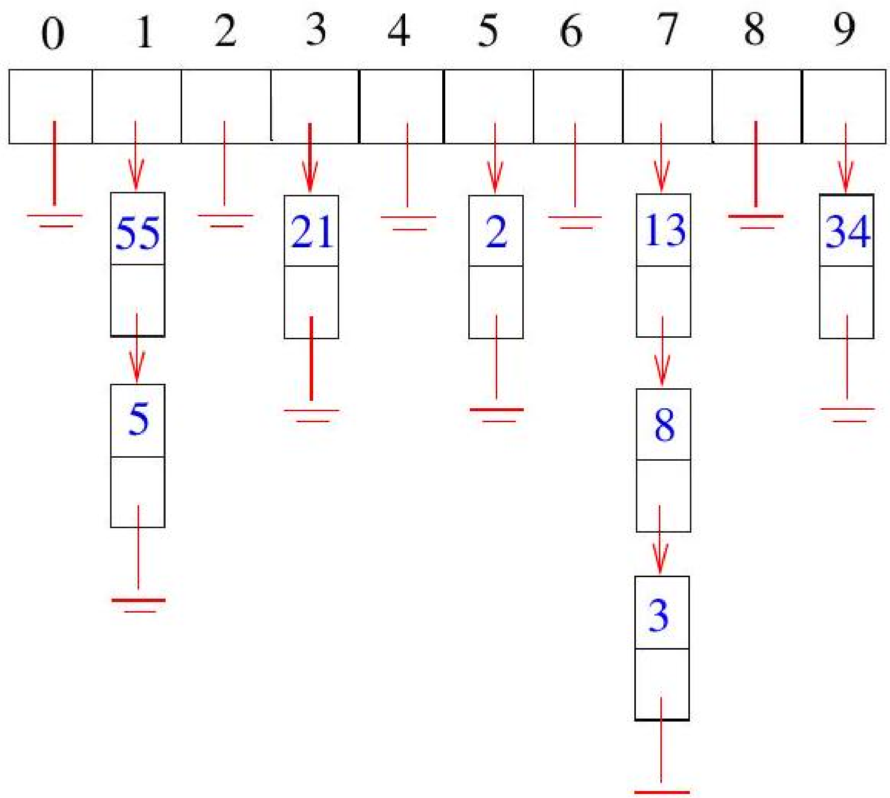


Figure 6.9: Collision resolution by chaining, after hashing the first eight Fibonacci numbers in increasing order, with hash function $H(x) = (2x + 1) \bmod 10$. Insertions occur at the head of each list in this figure.

Chaining and open addressing both cost $O(m)$ to initialize an m-element hash table to null elements prior to the first insertion.

When using chaining with doubly linked lists to resolve collisions in an m element hash table, the dictionary operations for n items can be implemented in the following expected and worst case times:

	Hash table (expected)	*Hash table (worst case)*
Search (L, k)	$O(n/m)$	$O(n)$
Insert(L, x)	$O(1)$	$O(1)$
Delete (L, x)	$O(1)$	$O(1)$
Successor (L, x)	$O(n + m)$	$O(n + m)$
Predecessor (L, x)	$O(n + m)$	$O(n + m)$
Minimum (L)	$O(n + m)$	$O(n + m)$
Maximum (L)	$O(n + m)$	$O(n + m)$

Traversing all the elements in the table takes $O(n + m)$ time for chaining, since we have to scan all m buckets looking for elements, even if the actual number of inserted items is small. This reduces to $O(m)$ time for open addressing, since n must be at most m.

Pragmatically, a hash table often is the best data structure to maintain a dictionary. The applications of hashing go far beyond dictionaries, however, as we will see below.

6.6.2 Duplicate Detection via Hashing

The key idea of hashing is to represent a large object (be it a key, a string, or a substring) by a single number. We get a representation of the large object by a value that can be manipulated in constant time, such that it is relatively unlikely that two different large objects map to the same value.

Hashing has a variety of clever applications beyond just speeding up search. I once heard Udi Manber - at one point responsible for all search products at Google - talk about the algorithms employed in industry. The three most important algorithms, he explained, were "hashing, hashing, and hashing."

Consider the following problems with nice hashing solutions:

- *Is a given document unique within a large corpus? - Google crawls yet another webpage. How can it tell whether this is new content never seen before, or just a duplicate page that exists elsewhere on the web?*

Explicitly comparing the new document D against all n previous documents is hopelessly inefficient for a large corpus. However, we can hash D to an integer, and compare $H(D)$ to the hash codes of the rest of the corpus. Only if there is a collision might D be a possible duplicate. Since we expect few spurious collisions, we can explicitly compare the few documents sharing a particular hash code with little total effort.

- *Is part of this document plagiarized? - A lazy student copies a portion of a web document into their term paper. "The web is a big place," he smirks. "How will anyone ever find the page I stole this from?"*

This is a more difficult problem than the previous application. Adding, deleting, or changing even one character from a document will completely change its hash code. The hash codes produced in the previous application thus cannot help for this more general problem.

However, we could build a hash table of all overlapping windows (substrings) of length w in all the documents in the corpus. Whenever there is a match of hash codes, there is likely a common substring of length w between the two documents, which can then be further investigated. We should choose w to be long enough so such a co-occurrence is very unlikely to happen by chance.

The biggest downside of this scheme is that the size of the hash table becomes as large as the document corpus itself. Retaining a small but well-chosen subset of these hash codes is exactly the goal of min-wise hashing, discussed in Section 6.6

- *How can I convince you that a file isn't changed? - In a closed-bid auction, each party submits their bid in secret before the announced deadline. If you knew what the other parties were bidding, you could arrange to bid* $1 *more than the highest opponent and walk off with the prize as cheaply as possible. The "right" auction strategy would thus be to hack into the computer containing the bids just prior to the deadline, read the bids, and then magically emerge as the winner.*
  How can this be prevented? What if everyone submits a hash code of their actual bid prior to the deadline, and then submits the full bid only after the deadline? The auctioneer will pick the largest full bid, but checks to make sure the hash code matches what was submitted prior to the deadline. Such cryptographic hashing methods provide a way to ensure that the file you give me today is the same as the original, because any change to the file will change the hash code.

Although the worst-case bounds on anything involving hashing are dismal, with a proper hash function we can confidently expect good behavior. Hashing is a fundamental idea in randomized algorithms, yielding linear expected-time algorithms for problems that are $\Theta(n \log n)$ or $\Theta\left(n^2\right)$ in the worst case.

6.6.3 Other Hashing Tricks

Hash functions provide useful tools for many things beyond powering hash tables. The fundamental idea is of many-to-one mappings, where many is controlled so it is very unlikely to be too many.

6.6.4 Canonicalization

Consider a word game that gives you a set of letters S, and asks you to find all dictionary words that can be made by reordering them. For example, I can *make three words from the four letters in* $S = (a, e, k, l)$*, namely kale, lake, and leak.*

Think how you might write a program to find the matching words for S*, given a dictionary* D *of* n *words. Perhaps the most straightforward approach is to test each word* $d \in D$ *against the characters of* S*. This takes time linear in* n *for each* S*, and the test is somewhat tricky to program.*

What if we instead hash every word in D *to a string, by sorting the word's letters. Now kale goes to aekl, as do lake and leak. By building a hash table with the sorted strings as keys, all words with the same letter distribution get hashed to the same bucket. Once you have built this hash table, you can use it for different query sets* S*. The time for each query will be proportional to the number of matching words in* D*, which is a lot smaller than* n*.*

Which set of k *letters can be used to make the most dictionary words? This seems like a much harder problem, because there are* α^k *possible letter sets, where* α *is the size of the alphabet. But observe that the answer is simply the hash code with the largest number of collisions. Sweeping over a sorted array of hash codes (or walking through each bucket in a chained hash table) makes this fast and easy.*

This is a good example of the power of canonicalization, reducing complicated objects to a standard (i.e. "canonical") form. String transformations like reducing letters to lower case or stemming (removing word suffixes like -ed, -s, or -ing) result in increased matches, because multiple strings collide on the same code. Soundex is a canonicalization scheme for names, so spelling variants of "Skiena" like"Skina,", "Skinnia," and "Schiena" all get hashed to the same Soundex code, S25. Soundex is described in more detail in Section 21.4

For hash tables, collisions are very bad. But for pattern matching problems like these, collisions are exactly what we want.

6.6.5 Compaction

Suppose that you wanted to sort all n *books in the library, not by their titles but by the contents of the actual text. Bulwer-Lytton's BL30] "It was a dark and stormy night..." would appear before this book's "What is an algorithm?..." Assuming the average book is* $m \approx 100,000$ *words long, doing this sort seems an expensive and clumsy job since each comparison involves two books.*

But suppose we instead represent each book by the first (say) 100 characters, and sort these strings. There will be collisions involving duplicates of the same prefix, involving multiple editions or perhaps plagiarism, but these will be quite rare. After sorting the prefixes, we can then resolve the collisions by comparing the full texts. The world's fastest sorting programs use this idea, as discussed in Section 17.1

This is an example of hashing for compaction, also called fingerprinting, where we representing large objects by small hash codes. It is easier to work with small objects than large ones, and the hash code generally preserves the identity of each item. The hash function here is trivial (just take the prefix) but *it is designed to accomplish a specific goal - not to maintain a hash table. More sophisticated hash functions can make the probability of collisions between even slightly different objects vanishingly low.*

6.7 Specialized Data Structures

The basic data structures described thus far all represent an unstructured set of items so as to facilitate retrieval operations. These data structures are well known to most programmers. Not as well known are data structures for representing more specialized kinds of objects, such as points in space, strings, and graphs.

The design principles of these data structures are the same as for basic objects. There exists a set of basic operations we need to perform repeatedly. We seek a data structure that allows these operations to be performed very efficiently. These specialized data structures are important for efficient graph and geometric algorithms, so one should be aware of their existence:

- *String data structures - Character strings are typically represented by arrays of characters, perhaps with a special character to mark the end of the string. Suffix trees/arrays are special data structures that preprocess strings to make pattern matching operations faster. See Section 15.3 (page 448) for details.*

- *Geometric data structures - Geometric data typically consists of collections of data points and regions. Regions in the plane can be described by polygons, where the boundary of the polygon is a closed chain of line segments. A polygon P can be represented using an array of points $(v_1, \ldots, v_n, v_1)$, such that (v_i, v_{i+1}) is a segment of the boundary of P. Spatial data structures such as kd-trees organize points and regions by geometric location to support fast search operations. See Section 15.6 (page 460 .*

- *Graph data structures - Graphs are typically represented using either adjacency matrices or adjacency lists. The choice of representation can have a substantial impact on the design of the resulting graph algorithms, and will be discussed in Chapter 7 and in the catalog in Section 15.4*

- *Set data structures - Subsets of items are typically represented using a dictionary to support fast membership queries. Alternatively, bit vectors are Boolean arrays such that the i th bit is 1 if i is in the subset. Data structures for manipulating sets is presented in the catalog in Section 15.5*

6.8 War Story: String 'em Up

The human genome encodes all the information necessary to build a person. Sequencing the genome has had an enormous impact on medicine and molecular

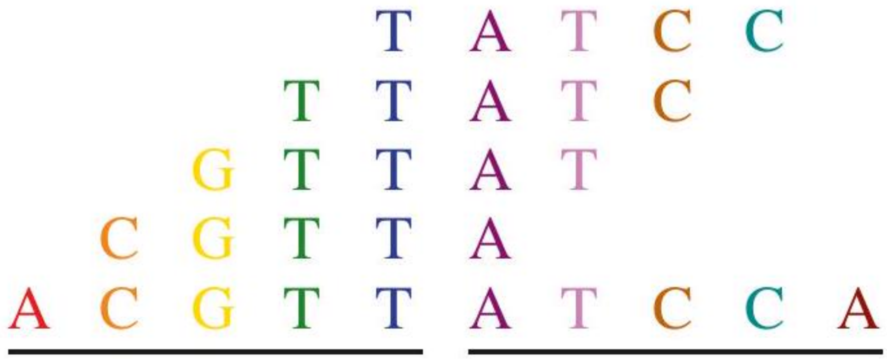


Figure 6.10: The concatenation of two fragments can be in S only if all subfragments are.

biology. Algorists like me have become interested in bioinformatics, for several reasons:

- *DNA sequences can be accurately represented as strings of characters on the four-letter alphabet (A,C,T,G). The needs of biologist have sparked new interest in old algorithmic problems such as string matching (see Section 21.3 (page 685) as well as creating new problems such as shortest common superstring (see Section 21.9 (page 709).*
- *DNA sequences are very long strings. The human genome is approximately three billion base pairs (or characters) long. Such large problem size means that asymptotic (Big Oh) complexity analysis is fully justified on biological problems.*
- *Enough money is being invested in genomics for computer scientists to want to claim their piece of the action.*

One of my interests in computational biology revolved around a proposed technique for DNA sequencing called sequencing by hybridization (SBH). This procedure attaches a set of probes to an array, forming a sequencing chip. Each of these probes determines whether or not the probe string occurs as a substring of the DNA target. The target DNA can now be sequenced based on the constraints of which strings are (and are not) substrings of the target.

We sought to identify all the strings of length $2k$ *that are possible substrings of an unknown string* S*, given the set of all length-* k *substrings of* S*. For example, suppose we know that* AC, CA*, and* CC *are the only length-* 2 *substrings of* S*. It is possible that* $ACCA$ *is a substring of* S*, since the center substring is one of our possibilities. However,* $CAAC$ *cannot be a substring of* S*, since* AA *is not a substring of* S*. We needed to find a fast algorithm to construct all the consistent length-* $2k$ *strings, since* S *could be very long.*

The simplest algorithm to build the $2k$ *strings would be to concatenate all* $O\left(n^2\right)$ *pairs of* k*-strings together, and then test to make sure that all* $(k-1)$ *length-* k *substrings spanning the boundary of the concatenation were in fact substrings, as shown in Figure 3.12 For example, the nine possible concatenations of* AC, CA, *and* CC *are* $ACAC, ACCA, ACCC, CAAC, CACA, CACC$, $CCAC, CCCA$, *and* $CCCC$. *Only* $CAAC$ *can be eliminated, because of the absence of* AA *as a substring of* S.

We needed a fast way of testing whether the $k-1$ *substrings straddling the concatenation were members of our dictionary of permissible* k*-strings. The time it takes to do this depends upon which dictionary data structure we use. A binary search tree could find the correct string within* $O(\log n)$ *comparisons, where each comparison involved testing which of two length-* k *strings appeared first in alphabetical order. The total time using such a binary search tree would be* $O(k \log n)$.

That seemed pretty good. So my graduate student, Dimitris Margaritis, used a binary search tree data structure for our implementation. It worked great up until the moment we ran it.

"I've tried the fastest computer we have, but our program is too slow," Dimitris complained. "It takes forever on string lengths of only 2,000 characters. We will never get up to $n=50,000$*."*

We profiled our program and discovered that almost all the time was spent searching in this data structure. This was no surprise, since we did this $k-$*1 times for each of the* $O\left(n^2\right)$ *possible concatenations. We needed a faster dictionary data structure, since search was the innermost operation in such a deep loop.*

"How about using a hash table?" I suggested. "It should take $O(k)$ *time to hash a* k*-character string and look it up in our table. That should knock off a factor of* $O(\log n)$*."*

Dimitris went back and implemented a hash table implementation for our dictionary. Again, it worked great, up until the moment we ran it.

"Our program is still too slow," Dimitris complained. "Sure, it is now about ten times faster on strings of length 2,000 . So now we can get up to about 4,000 characters. Big deal. We will never get up to 50,000 ."

"We should have expected this," I mused. "After all, $\lg_2(2,000) \approx 11$. *We need a faster data structure to search in our dictionary of strings."*

"But what can be faster than a hash table?" Dimitris countered. "To look up a k*-character string, you must read all* k *characters. Our hash table already does* $O(k)$ *searching."*

"Sure, it takes k *comparisons to test the first substring. But maybe we can do better on the second test. Remember where our dictionary queries are coming from. When we concatenate* $ABCD$ *with* $EFGH$, *we are first testing whether* $BCDE$ *is in the dictionary, then* $CDEF$. *These strings differ from each other by only one character. We should be able to exploit this so each subsequent test takes constant time to perform. . ."*

"We can't do that with a hash table," Dimitris observed. "The second key is not going to be anywhere near the first in the table. A binary search tree won't help,

either. Since the keys $ABCD$ and $BCDE$ differ according to the first character, the two strings will be in different parts of the tree."
"But we can use a suffix tree to do this," I countered. "A suffix tree is a trie containing all the suffixes of a given set of strings. For example, the suffixes of $ACAC$ are $\{ACAC, CAC, AC, C\}$. Coupled with suffixes of string $CACT$,

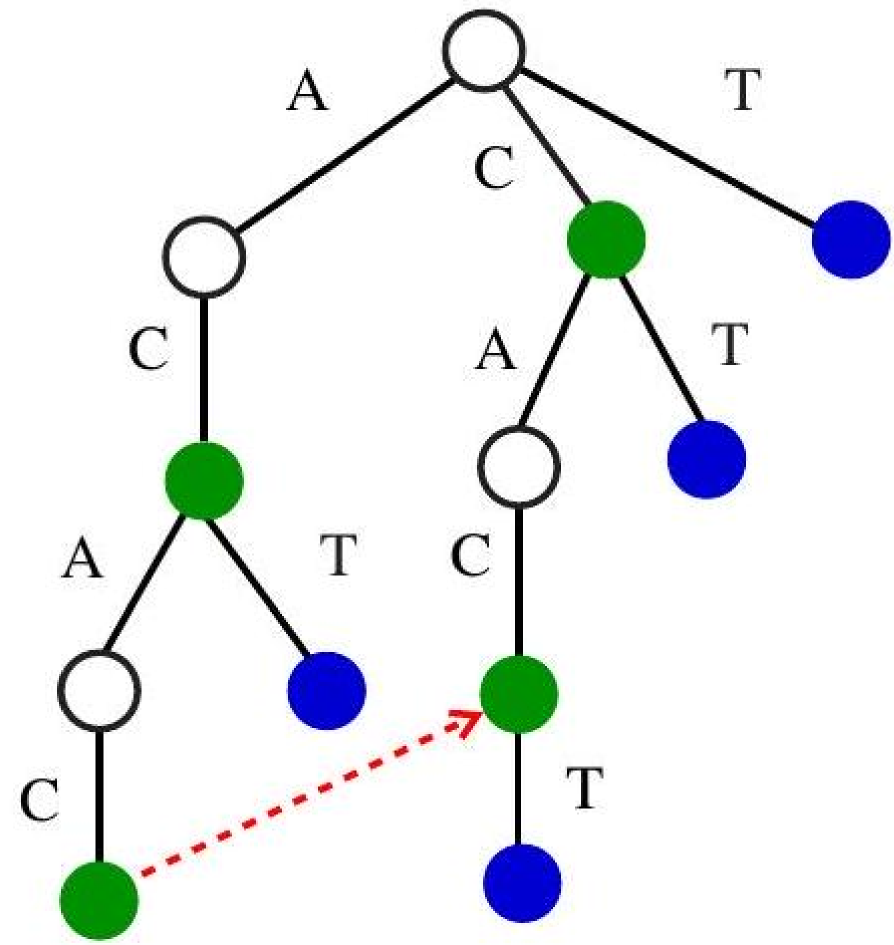


Figure 6.11: Suffix tree on $ACAC$ and $CACT$, with the pointer to the suffix of $ACAC$. Green nodes correspond to suffixes of $ACAC$, with blue nodes to suffixes of $CACT$.

we get the suffix tree of Figure 3.13 By following a pointer from $ACAC$ to its longest proper suffix CAC, we get to the right place to test whether $CACT$

is in our set of strings. One character comparison is all we need to do from there."

Suffix trees are amazing data structures, discussed in considerably more detail in Section 15.3 (page 448). Dimitris did some reading about them, then built a nice suffix tree implementation for our dictionary. Once again, it worked great up until the moment we ran it.

"Now our program is faster, but it runs out of memory," Dimitris complained. "The suffix tree builds a path of length k for each suffix of length k, so all told there can be $\Theta\left(n^2\right)$ *nodes in the tree. It crashes when we go beyond 2,000 characters. We will never get up to strings with 50,000 characters."*

I wasn't ready to give up yet. "There is a way around the space problem, by using compressed suffix trees," I recalled. "Instead of explicitly representing long paths of character nodes, we can refer back to the original string." Compressed suffix trees always take linear space, as described in Section 15.3 (page 448).

Dimitris went back one last time and implemented the compressed suffix tree data structure. Now it worked great! As shown in Figure 3.14 we ran our simulation for strings of length $n = 65,536$ *without incident. Our results showed that interactive SBH could be a very efficient sequencing technique. Based on these simulations, we were able to arouse interest in our technique from biologists. Making the actual wet laboratory experiments feasible provided another computational challenge, which is reported in Section 12.8 (page 414 .*

The take-home lessons for programmers should be apparent. We isolated a single operation (dictionary string search) that was being performed repeatedly and optimized the data structure to support it. When an improved dictionary structure still did not suffice, we looked deeper into the kind of queries we were performing, so that we could identify an even better data structure. Finally, we didn't give up until we had achieved the level of performance we needed. In algorithms, as in life, persistence usually pays off.

String length	*Binary tree*	*Hash table*	*Suffix tree*	*Compressed tree*
8	0.0	0.0	0.0	0.0
16	0.0	0.0	0.0	0.0
32	0.1	0.0	0.0	0.0
64	0.3	0.4	0.3	0.0
128	2.4	1.1	0.5	0.0
256	17.1	9.4	3.8	0.2
512	31.6	67.0	6.9	1.3
1,024	1,828.9	96.6	31.5	2.7
2,048	11,441.7	941.7	553.6	39.0
4,096	> 2 days	5,246.7	out of	45.4
8,192		> 2 days	memory	642.0
16,384				1,614.0
32,768				13,657.8
65,536				39,776.9

Figure 3.14: Run times (in seconds) for the SBH simulation using various

data structures, as a function of the string length n.

Chapter Notes *Optimizing hash table performance is surprisingly complicated for such a conceptually simple data structure. The importance of short runs in open addressing has led to more sophisticated schemes than sequential probing for optimal hash table performance. For more details, see Knuth citeknuth1998taocp3.*

My thinking on hashing was profoundly influenced by a talk by Mihai Ptracu, a brilliant theoretician who sadly died before he turned 30. More detailed treatments on hashing and randomized algorithms include Motwani and Raghavan citemotwani1995randomized and Mitzenmacher and Upfal citemitzenmacher2017probability.

Our triangle strip optimizing program, stripe, is described in Evans et al. citeevans1996optimizing. Hashing techniques for plagiarism detection are discussed in Schlieimer et al. citeschlieimer2003winnowing].

Surveys of algorithmic issues in DNA sequencing by hybridization include Chetverin and Kramer [citechetverin1994oligonucleotide] and Pevzner and Lipshutz [citepevzner1994towards]. Our work on interactive SBH reported in the war story is reported in Margaritis and Skiena citemargaritis1995reconstructing.

6.9 Exercises

Stacks, Queues, and Lists 3-1. [3] A common problem for compilers and text editors is determining whether the parentheses in a string are balanced and properly nested. For example, the string ((())())() contains properly nested pairs of parentheses, while the strings)()(and ()) do not. Give an algorithm that returns true if a string contains properly nested and balanced parentheses, and false if otherwise. For full credit, identify the position of the first offending parenthesis if the string is not properly nested and balanced.

3-2. [5] Give an algorithm that takes a string S consisting of opening and closing parentheses, say)()(())()()))())))(, and finds the length of the longest balanced parentheses in S, which is 12 in the example above. (Hint: The solution is not necessarily a contiguous run of parenthesis from S.)

3-3. [3] Give an algorithm to reverse the direction of a given singly linked list. In other words, after the reversal all pointers should now point backwards. Your algorithm should take linear time.

3-4. [5] Design a stack S that supports S.push(x), S.pop(), and S.findmin(), which returns the minimum element of S. All operations should run in constant time.

3-5. [5] We have seen how dynamic arrays enable arrays to grow while still achieving constant-time amortized performance. This problem concerns extending dynamic arrays to let them both grow and shrink on demand.

(a) Consider an underflow strategy that cuts the array size in half whenever the

array falls below half full. Give an example sequence of insertions and deletions where this strategy gives a bad amortized cost.
(b) Then, give a better underflow strategy than that suggested above, one that achieves constant amortized cost per deletion.

3-6. [3] Suppose you seek to maintain the contents of a refrigerator so as to minimize food spoilage. What data structure should you use, and how should you use it?

3-7. [5] Work out the details of supporting constant-time deletion from a singly linked list as per the footnote from page 79 ideally to an actual implementation. Support the other operations as efficiently as possible.

Elementary Data Structures $3-8$. *[5] Tic-tac-toe is a game played on an* $n \times n$ *board (typically* $n=3$ *) where two players take consecutive turns placing " O " and "X" marks onto the board cells. The game is won if* n *consecutive " O " or " X " marks are placed in a row, column, or diagonal. Create a data structure with* $O(n)$ *space that accepts a sequence of moves, and reports in constant time whether the last move won the game.*
$3-9$. *[3] Write a function which, given a sequence of digits 2-9 and a dictionary of* n *words, reports all words described by this sequence when typed in on a standard telephone keypad. For the sequence 269 you should return any, box, boy, and cow, among other words.*

3-10. [3] Two strings X *and* Y *are anagrams if the letters of* X *can be rearranged to form* Y. *For example, silent/listen, and incest/insect are anagrams. Give an efficient algorithm to determine whether strings* X *and* Y *are anagrams.*

Trees and Other Dictionary Structures 3-11. [3] Design a dictionary data structure in which search, insertion, and deletion can all be processed in $O(1)$ *time in the worst case. You may assume the set elements are integers drawn from a finite set* $1, 2, .., n$, *and initialization can take* $O(n)$ *time.*

3-12. [3] The maximum depth of a binary tree is the number of nodes on the path from the root down to the most distant leaf node. Give an $O(n)$ *algorithm to find the maximum depth of a binary tree with* n *nodes.*

3-13. [5] Two elements of a binary search tree have been swapped by mistake. Give an $O(n)$ *algorithm to identify these two elements so they can be swapped back.*
3-14. [5] Given two binary search trees, merge them into a doubly linked list in sorted order.
3-15. [5] Describe an $O(n)$*-time algorithm that takes an* n*-node binary search tree and constructs an equivalent height-balanced binary search tree. In a height-balanced binary search tree, the difference between the height of the left and right subtrees of every node is never more than 1.*

3-16. [3] Find the storage efficiency ratio (the ratio of data space over total space) for each of the following binary tree implementations on n *nodes:*
(a) All nodes store data, two child pointers, and a parent pointer. The data field requires 4 bytes and each pointer requires 4 bytes.
(b) Only leaf nodes store data; internal nodes store two child pointers. The data field requires four bytes and each pointer requires two bytes.

3-17. [5] Give an $O(n)$ algorithm that determines whether a given n-node binary tree is height-balanced (see Problem 3-15.

3-18. [5] Describe how to modify any balanced tree data structure such that search, insert, delete, minimum, and maximum still take $O(\log n)$ time each, but successor and predecessor now take $O(1)$ time each. Which operations have to be modified to support this?

3-19. [5] Suppose you have access to a balanced dictionary data structure that supports each of the operations search, insert, delete, minimum, maximum, successor, and predecessor in $O(\log n)$ time. Explain how to modify the insert and delete operations so they still take $O(\log n)$ but now minimum and maximum take $O(1)$ time. (Hint: think in terms of using the abstract dictionary operations, instead of mucking about with pointers and the like.)

3-20. [5] Design a data structure to support the following operations:

- $\text{insert}(x, T)$ *- Insert item x into the set T.*
- *delete (k, T) - Delete the k th smallest element from T.*
- *member (x, T) - Return true iff $x \in T$.*

All operations must take $O(\log n)$ time on an n-element set.

3-21. [8] A concatenate operation takes two sets S_1 and S_2, where every key in S_1 is smaller than any key in S_2, and merges them. Give an algorithm to concatenate two binary search trees into one binary search tree. The worst-case running time should be $O(h)$, where h is the maximal height of the two trees.

Applications of Tree Structures 3-22. [5] Design a data structure that supports the following two operations:

- *insert (x) - Insert item x from the data stream to the data structure.*
- *median() - Return the median of all elements so far.*

All operations must take $O(\log n)$ time on an n-element set.

3-23. [5] Assume we are given a standard dictionary (balanced binary search tree) defined on a set of n strings, each of length at most l. We seek to print out all strings beginning with a particular prefix p. Show how to do this in $O(ml \log n)$ time, where m is the number of strings.

3-24. [5] An array A is called k-unique if it does not contain a pair of duplicate elements within k positions of each other, that is, there is no i and j such that $A[i] = A[j]$ and $|j - i| \leq k$. Design a worst-case $O(n \log k)$ algorithm to test if A is k-unique.

3-25. [5] In the bin-packing problem, we are given n objects, each weighing at most 1 kilogram. Our goal is to find the smallest number of bins that will hold the n objects, with each bin holding 1 kilogram at most.

- *The best-fit heuristic for bin packing is as follows. Consider the objects in the order in which they are given. For each object, place it into the partially filled bin with the smallest amount of extra room after the object is inserted.*

If no such bin exists, start a new bin. Design an algorithm that implements the best-fit heuristic (taking as input the n weights $w_1, w_2, \ldots, w_n$ and outputting the number of bins used) in $O(n \log n)$ time.

- *Repeat the above using the worst-fit heuristic, where we put the next object into the partially filled bin with the largest amount of extra room after the object is inserted.*

3-26. [5] Suppose that we are given a sequence of n values $x_1, x_2, \ldots, x_n$ and seek to quickly answer repeated queries of the form: given i and j, find the smallest value in $x_i, \ldots, x_j$.
(a) Design a data structure that uses $O\left(n^2\right)$ space and answers queries in $O(1)$ time.
(b) Design a data structure that uses $O(n)$ space and answers queries in $O(\log n)$ time. For partial credit, your data structure can use $O(n \log n)$ space and have $O(\log n)$ query time.

3-27. [5] Suppose you are given an input set S of n integers, and a black box that if given any sequence of integers and an integer k instantly and correctly answers whether there is a subset of the input sequence whose sum is exactly k. Show how to use the black box $O(n)$ times to find a subset of S that adds up to k.

3-28. [5] Let $A[1..n]$ be an array of real numbers. Design an algorithm to perform any sequence of the following operations:

- *Add(i, y) - Add the value y to the i th number.*
- *Partial-sum(i) - Return the sum of the first i numbers, that is, $\sum_{j=1}^{i} A[j]$.*

There are no insertions or deletions; the only change is to the values of the numbers. Each operation should take $O(\log n)$ steps. You may use one additional array of size n as a work space.

3-29. [8] Extend the data structure of the previous problem to support insertions and deletions. Each element now has both a key and a value. An element is accessed by its key, but the addition operation is applied to the values. The Partial_sum operation is different.

- Add(k, y) *- Add the value y to the item with key k.*
- *Insert (k, y) - Insert a new item with key k and value y.*
- *Delete (k) - Delete the item with key k.*
- *Partial-sum (k) - Return the sum of all the elements currently in the set whose key is less than k, that is, $\sum_{i<k} x_i$.*

The worst-case running time should still be $O(n \log n)$ for any sequence of $O(n)$ operations.

3-30. [8] You are consulting for a hotel that has n one-bed rooms. When a guest

checks in, they ask for a room whose number is in the range $[l, h]$*. Propose a data structure that supports the following data operations in the allotted time:*
(a) Initialize (n): Initialize the data structure for empty rooms numbered $1, 2, \ldots, n$*, in polynomial time.*
(b) Count (l, h) *: Return the number of available rooms in* $[l, h]$*, in* $O(\log n)$ *time.*
(c) Checkin (l, h) *: In* $O(\log n)$ *time, return the first empty room in* $[l, h]$ *and mark it occupied, or return NIL if all the rooms in* $[l, h]$ *are occupied.*
(d) Checkout(x): Mark room x *as unoccupied, in* $O(\log n)$ *time.*

3-31. [8] Design a data structure that allows one to search, insert, and delete an integer X *in* $O(1)$ *time (i.e., constant time, independent of the total number of integers stored). Assume that* $1 \leq X \leq n$ *and that there are* $m + n$ *units of space available, where* m *is the maximum number of integers that can be in the table at any one time. (Hint: use two arrays* $A[1..n]$ *and* $B[1..m]$*.) You are not allowed to initialize either* A *or* B*, as that would take* $O(m)$ *or* $O(n)$ *operations. This means the arrays are full of random garbage to begin with, so you must be very careful.*

Implementation Projects 3-32. [5] Implement versions of several different dictionary data structures, such as linked lists, binary trees, balanced binary search trees, and hash tables. Conduct experiments to assess the relative performance of these data structures in a simple application that reads a large text file and reports exactly one instance of each word that appears within it. This application can be efficiently implemented by maintaining a dictionary of all distinct words that have appeared thus far in the text and inserting/reporting each new word that appears in the stream. Write a brief report with your conclusions.

3-33. [5] A Caesar shift (see Section 21.6 (page 697) is a very simple class of ciphers for secret messages. Unfortunately, they can be broken using statistical properties of English. Develop a program capable of decrypting Caesar shifts of sufficiently long texts.

Interview Problems 3-34. [3] What method would you use to look up a word in a dictionary?
3-35. [3] Imagine you have a closet full of shirts. What can you do to organize your shirts for easy retrieval?
$3 - 36$*. [4] Write a function to find the middle node of a singly linked list.*
3-37. [4] Write a function to determine whether two binary trees are identical. Identical trees have the same key value at each position and the same structure.
3-38. [4] Write a program to convert a binary search tree into a linked list.
3-39. [4] Implement an algorithm to reverse a linked list. Now do it without recursion.
3-40. [5] What is the best data structure for maintaining URLs that have been visited by a web crawler? Give an algorithm to test whether a given URL has already been visited, optimizing both space and time.
$3 - 41$*. [4] You are given a search string and a magazine. You seek to generate all the characters in the search string by cutting them out from the magazine. Give an algorithm to efficiently determine whether the magazine contains all the letters in the search string.*

$3-42$. *[4] Reverse the words in a sentence - that is, "My name is Chris" becomes "Chris is name My." Optimize for time and space.*
3-43. [5] Determine whether a linked list contains a loop as quickly as possible without using any extra storage. Also, identify the location of the loop.
3-44. [5] You have an unordered array X of n integers. Find the array M containing n elements where M_i is the product of all integers in X except for X_i. You may not use division. You can use extra memory. (Hint: there are solutions faster than $O\left(n^2\right)$.)
3-45. [6] Give an algorithm for finding an ordered word pair (e.g. "New York") occurring with the greatest frequency in a given webpage. Which data structures would you use? Optimize both time and space.

LeetCode *3-1. https://leetcode.com/problems/validate-binary-search-tree/*
3-2. https://leetcode.com/problems/count-of-smaller-numbers-after-self/
3-3. https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/

HackerRank *3-1. https://www.hackerrank.com/challenges/is-binary-search-tree/*
3-2. https://www.hackerrank.com/challenges/queue-using-two-stacks/
3-3. https://www.hackerrank.com/challenges/detect-whether-a-linked-list-contains-a-cycle/problem

Programming Challenges *These programming challenge problems with robot judging are available at https://onlinejudge.org*
3-1. "Jolly Jumpers" - Chapter 2, problem 10038.
3-2. "Crypt Kicker" - Chapter 2, problem 843.
3-3. "Where's Waldorf?"-Chapter 3, problem 10010.
3-4. "Crypt Kicker II" - Chapter 3, problem 850.

Chapter 4

Chapter 7

Sorting

Typical computer science students study the basic sorting algorithms at least three times before they graduate: first in introductory programming, then in data structures, and finally in their algorithms course. Why is sorting worth so much attention? There are several reasons:

- *Sorting is the basic building block that many other algorithms are built around. By understanding sorting, we obtain an amazing amount of power to solve other problems.*
- *Most of the interesting ideas used in the design of algorithms appear in the context of sorting, such as divide-and-conquer, data structures, and randomized algorithms.*
- *Sorting is the most thoroughly studied problem in computer science. Literally dozens of different algorithms are known, most of which possess some particular advantage over all other algorithms in certain situations.*

This chapter will discuss sorting, stressing how sorting can be applied to solving other problems. In this sense, sorting behaves more like a data structure than a problem in its own right. I then give detailed presentations of several fundamental algorithms: heapsort, mergesort, quicksort, and distribution sort as examples of important algorithm design paradigms. Sorting is also represented by Section 17.1 (page 506 in the problem catalog.

7.1 Applications of Sorting

I will review several sorting algorithms and their complexities over the course of this chapter. But the punch line is this: clever sorting algorithms exist that run in $O(n \log n)$. This is a big improvement over naive $O\left(n^2\right)$ sorting algorithms, for large values of n. Consider the number of steps done by two different sorting algorithms for reasonable values of n :

n	$n^2/4$	$n \lg n$
10	*25*	*33*
100	*2,500*	*664*
1,000	*250,000*	*9,965*
10,000	$25,000,000$	*132,877*
100,000	$2,500,000,000$	$1,660,960$

You might survive using a quadratic-time algorithm even if $n = 10,000$*, but the slow algorithm clearly gets ridiculous once* $n \geq 100,000$*.*

Many important problems can be reduced to sorting, so we can use our clever $O(n \log n)$ *algorithms to do work that might otherwise seem to require a quadratic algorithm. An important algorithm design technique is to use sorting as a basic building block, because many other problems become easy once a set of items is sorted.*

Consider the following applications:

- *Searching - Binary search tests whether an item is in a dictionary in* $O(\log n)$ *time, provided the keys are all sorted. Search preprocessing is perhaps the single most important application of sorting.*

- *Closest pair - Given a set of* n *numbers, how do you find the pair of numbers that have the smallest difference between them? Once the numbers are sorted, the closest pair of numbers must lie next to each other somewhere in sorted order. Thus, a linear-time scan through the sorted list completes the job, for a total of* $O(n \log n)$ *time including the sorting.*

- *Element uniqueness - Are there any duplicates in a given set of* n *items? This is a special case of the closest-pair problem, where now we ask if there is a pair separated by a gap of zero. An efficient algorithm sorts the numbers and then does a linear scan checking all adjacent pairs.*

- *Finding the mode - Given a set of* n *items, which element occurs the largest number of times in the set? If the items are sorted, we can sweep from left to right and count the number of occurrences of each element, since all identical items will be lumped together after sorting.*
  To find out how often an arbitrary element k *occurs, look up* k *using binary search in a sorted array of keys. By walking to the left of this point until the first element is not* k *and then doing the same to the right, we can find this count in* $O(\log n + c)$ *time, where* c *is the number of occurrences of* k*. Even better, the number of instances of* k *can be found in* $O(\log n)$ *time by using binary search to look for the positions of both* $k - \epsilon$ *and* $k + \epsilon$ *(where* ϵ *is suitably small) and then taking the difference of these positions.*

- *Selection - What is the* k *th largest item in an array? If the keys are placed in sorted order, the* k *th largest item can be found in constant time because it must sit in the* k *th position of the array. In particular,*

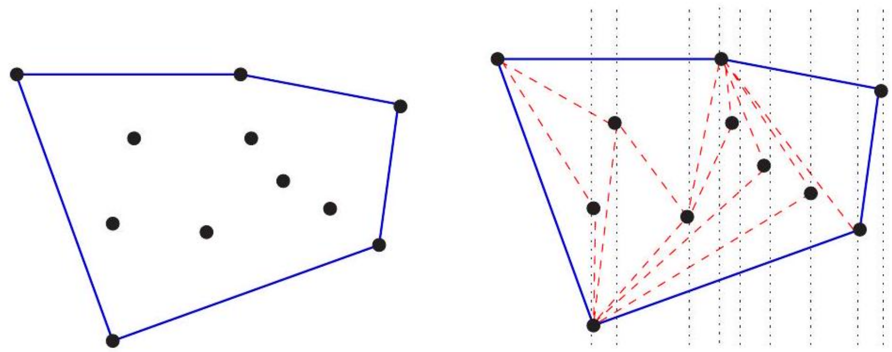


Figure 7.1: The convex hull of a set of points (left), constructed by left-to-right insertion (right).

the median element (see Section 17.3 (page 514) appears in the $(n/2)$ *nd position in sorted order.*

- *Convex hulls - What is the polygon of smallest perimeter that contains a given set of* n *points in two dimensions? The convex hull is like a rubber band stretched around the points in the plane and then released. It shrinks to just enclose the points, as shown in Figure 4.1(1). The convex hull gives a nice representation of the shape of the point set and is an important building block for more sophisticated geometric algorithms, as discussed in the catalog in Section 20.2 (page 626.*

But how can we use sorting to construct the convex hull? Once you have the points sorted by x*-coordinate, the points can be inserted from left to right into the hull. Since the right-most point is always on the boundary, we know that it must appear in the hull. Adding this new right-most point may cause others to be deleted, but we can quickly identify these points because they lie inside the polygon formed by adding the new point. See the example in Figure 4.1(r). These points will be neighbors of the previous point we inserted, so they will be easy to find and delete. The total time is linear after the sorting has been done.*

While a few of these problems (namely median and selection) can be solved in linear time using more sophisticated algorithms, sorting provides quick and easy solutions to all of these problems. It is a rare application where the running time of sorting proves to be the bottleneck, especially a bottleneck that could have otherwise been removed using more clever algorithmics. Never be afraid to spend time sorting, provided you use an efficient sorting routine.

Take-Home Lesson: Sorting lies at the heart of many algorithms. Sorting the data is one of the first things any algorithm designer should try in the quest for efficiency.

Stop and Think: Finding the Intersection *Problem: Give an efficient* *algorithm to determine whether two sets (of size m and n, respectively) are disjoint. Analyze the worst-case complexity in terms of m and n, considering the case where $m \ll n$.*

Solution: At least three algorithms come to mind, all of which are variants of sorting and searching:

- *First sort the big set - The big set can be sorted in $O(n \log n)$ time. We can now do a binary search with each of the m elements in the second, looking to see if it exists in the big set. The total time will be $O((n + m) \log n)$.*
- *First sort the small set - The small set can be sorted in $O(m \log m)$ time. We can now do a binary search with each of the n elements in the big set, looking to see if it exists in the small one. The total time will be $O((n + m) \log m)$.*
- *Sort both sets - Observe that once the two sets are sorted, we no longer have to do binary search to detect a common element. We can compare the smallest elements of the two sorted sets, and discard the smaller one if they are not identical. By repeating this idea recursively on the now smaller sets, we can test for duplication in linear time after sorting. The total cost is $O(n \log n + m \log m + n + m)$.*

So, which of these is the fastest method? Clearly small-set sorting trumps big-set sorting, since $\log m < \log n$ when $m < n$. Similarly, $(n + m) \log m$ must be asymptotically smaller than $n \log n$, since $n + m < 2n$ when $m < n$. Thus, sorting the small set is the best of these options. Note that this is linear when m is constant in size.

Note that expected linear time can be achieved by hashing. Build a hash table containing the elements of both sets, and then explicitly check whether collisions in the same bucket are in fact identical elements. In practice, this may be the best solution.

Stop and Think: Making a Hash of the Problem? *Problem: Fast sorting is a wonderful thing. But which of these tasks can be done as fast or faster (in expected time) using hashing instead of sorting?*

- *Searching?*
- *Closest pair?*
- *Element uniqueness?*
- *Finding the mode?*
- *Finding the median?*
- *Convex hull?*

Solution: Hashing can solve some these problems efficiently, but is inappropriate for others. Let's consider them one by one:

- *Searching - Hash tables are a great answer here, enabling you to search for items in constant expected time, as opposed to* $O(\log n)$ *with binary search.*

- *Closest pair - Hash tables as so far defined cannot help at all. Normal hash functions scatter keys around the table, so a pair of similar numerical values are unlikely to end up in the same bucket for comparison. Bucketing values by numerical ranges will ensure that the closest pair lie within the same bucket, or at worst neighboring buckets. But we cannot also force only a small number of items to lie in this bucket, as will be discussed with respect to bucketsort in Section 4.7*

- *Element uniqueness - Hashing is even faster than sorting for this problem. Build a hash table using chaining, and then compare each of the (expected constant) pairs of items within a bucket. If no bucket contains a duplicate pair, then all the elements must be unique. The table construction and sweeping can be completed in linear expected time.*

- *Finding the mode - Hashing leads to a linear expected-time algorithm here. Each bucket should contain a small number of distinct elements, but may have many duplicates. We start from the first element in a bucket and count/delete all copies of it, repeating this sweep the expected constant number of passes until the bucket is empty.*

- *Finding the median - Hashing does not help us, I am afraid. The median might be in any bucket of our table, and we have no way to judge how many items lie before or after it in sorted order.*

- *Convex hull - Sure, we can build a hash table on points just as well as any other data type. But it isn't clear what good that does us for this problem: certainly it can't help us order the points by* x*-coordinate.*

7.2 Pragmatics of Sorting

We have seen many algorithmic applications of sorting, and we will see several efficient sorting algorithms. One issue stands between them: in what order do we want our items sorted? The answer to this basic question is application specific. Consider the following issues:

- *Increasing or decreasing order? - A set of keys* S *are sorted in ascending order when* $S_i \leq S_{i+1}$ *for all* $1 \leq i < n$*. They are in descending order when* $S_i \geq S_{i+1}$ *for all* $1 \leq i < n$*. Different applications call for different orders.*

- *Sorting just the key or an entire record? - Sorting a data set requires maintaining the integrity of complex data records. A mailing list of names, addresses, and phone numbers may be sorted by names as the key field, but* *it had better retain the linkage between names and addresses. Thus, we need to specify which is the key field in any complex record, and understand the full extent of each record.*

- *What should we do with equal keys? - Elements with equal key values all bunch together in any total order, but sometimes the relative order among these keys matters. Suppose an encyclopedia contains articles on both Michael Jordan (the basketball player) and Michael Jordan (the actor)* [1] *Which entry should appear first? You may need to resort to secondary keys, such as article size, to resolve ties in a meaningful way.*
  Sometimes it is required to leave the items in the same relative order as in the original permutation. Sorting algorithms that automatically enforce this requirement are called stable. Few fast algorithms are naturally stable. Stability can be achieved for any sorting algorithm by adding the initial position as a secondary key.

Of course we could make no decision about equal key order and let the ties fall where they may. But beware, certain efficient sort algorithms (such as quicksort) can run into quadratic performance trouble unless explicitly engineered to deal with large numbers of ties.

- *What about non-numerical data? - Alphabetizing defines the sorting of text strings. Libraries have very complete and complicated rules concerning the relative collating sequence of characters and punctuation. Is Skiena the same key as skiena? Is Brown-Williams before or after Brown America, and before or after Brown, John?*

The right way to specify such details to your sorting algorithm is with an application-specific pairwise-element comparison function. Such a comparison function takes pointers to record items a and b and returns " $<$ " *if* $a < b$, " $>$ " *if* $a > b$, *or* " $=$ " *if* $a = b$.

By abstracting the pairwise ordering decision to such a comparison function, we can implement sorting algorithms independently of such criteria. We simply pass the comparison function in as an argument to the sort procedure. Any reasonable programming language has a built-in sort routine as a library function. You are usually better off using this than writing your own routine. For example, the standard library for C contains the qsort function for sorting:

```
#include <stdlib.h>
void qsort(void *base, size_t nel, size_t width,
    int (*compare) (const void *, const void *));
```

The key to using qsort is realizing what its arguments do. It sorts the first nel elements of an array (pointed to by base), where each element is widthbytes

[1] Not to mention *Michael Jordan (the statistician).*

long. We can thus sort arrays of 1-byte characters, 4-byte integers, or 100 -byte records, all by changing the value of width.

The desired output order is determined by the compare function. It takes as arguments pointers to two width-byte elements, and returns a negative number if the first belongs before the second in sorted order, a positive number if the second belongs before the first, or zero if they are the same. Here is a comparison function to sort integers in increasing order:

```
int intcompare(int *i, int *j)
{
    if (*i > *j) return (1);
    if (*i < *j) return (-1);
    return (0);
}
```

This comparison function can be used to sort an array a, of which the first n elements are occupied, as follows:

```
qsort(a, n, sizeof(int), intcompare);
```

The name qsort suggests that quicksort is the algorithm implemented in this library function, although this is usually irrelevant to the user.

Heapsort: Fast Sorting via Data Structures Sorting is a natural laboratory for studying algorithm design paradigms, since many useful techniques lead to interesting sorting algorithms. The next several sections will introduce algorithmic design techniques motivated by particular sorting algorithms.

The alert reader may ask why I review all the standard sorting algorithms after saying that you are usually better off not implementing them, and using library functions instead. The answer is that the underlying design techniques are very important for other algorithmic problems you are likely to encounter.

We start with data structure design, because one of the most dramatic algorithmic improvements via appropriate data structures occurs in sorting. Selection sort is a simple-to-code algorithm that repeatedly extracts the smallest remaining element from the unsorted part of the set:

```
typedef struct {
    item_type q[PQ_SIZE+1];      /* body of queue */
    int n;                       /* number of queue elements */
} priority_queue;
```

```
SelectionSort $(A)$
    For $i=1$ to $n$ do
        Sort $[i]=$ Find-Minimum from $A$
        Delete-Minimum from $A$
        Return(Sort)
```

A C language implementation of selection sort appeared back in Section 2.5.1 (page 41). There we partitioned the input array into sorted and unsorted regions. To find the smallest item, we performed a linear sweep through the unsorted portion of the array. The smallest item is then swapped with the i th item in the array before moving on to the next iteration. Selection sort performs n iterations, where the average iteration takes $n/2$ steps, for a total of $O\left(n^2\right)$ time.

But what if we improve the data structure? It takes $O(1)$ time to remove a particular item from an unsorted array after it has been located, but $O(n)$ time to find the smallest item. These are exactly the operations supported by priority queues. So what happens if we replace the data structure with a better priority queue implementation, either a heap or a balanced binary tree? The operations within the loop now take $O(\log n)$ time each, instead of $O(n)$. Using such a priority queue implementation speeds up selection sort from $O\left(n^2\right)$ to $O(n \log n)$.

The name typically given to this algorithm, heapsort, obscures the fact that the algorithm is nothing but an implementation of selection sort using the right data structure.

7.2.1 Heaps

Heaps are a simple and elegant data structure for efficiently supporting the priority queue operations insert and extract-min. Heaps work by maintaining a partial order on the set of elements that is weaker than the sorted order (so it can be efficient to maintain) yet stronger than random order (so the minimum element can be quickly identified).

Power in any hierarchically structured organization is reflected by a tree, where each node in the tree represents a person, and edge (x, y) implies that x directly supervises (or dominates) y. The person at the root sits at the "top of the heap."

In this spirit, a heap-labeled tree is defined to be a binary tree such that the key of each node dominates the keys of its children. In a min-heap, a node dominates its children by having a smaller key than they do, while in a maxheap parent nodes dominate by being bigger. Figure 4.2(1) presents a min-heap ordered tree of noteworthy years in American history.

The most natural implementation of this binary tree would store each key in a node with pointers to its two children. But as with binary search trees, the memory used by the pointers can easily outweigh the size of keys, which is the data we are really interested in. The heap is a slick data structure that enables us to represent binary trees without using any pointers. We store data as an

array of keys, and use the position of the keys to implicitly play the role of the pointers.

We store the root of the tree in the first position of the array, and its left and right children in the second and third positions, respectively. In general, we store the 2^{l-1} keys of the l th level of a complete binary tree from left to right

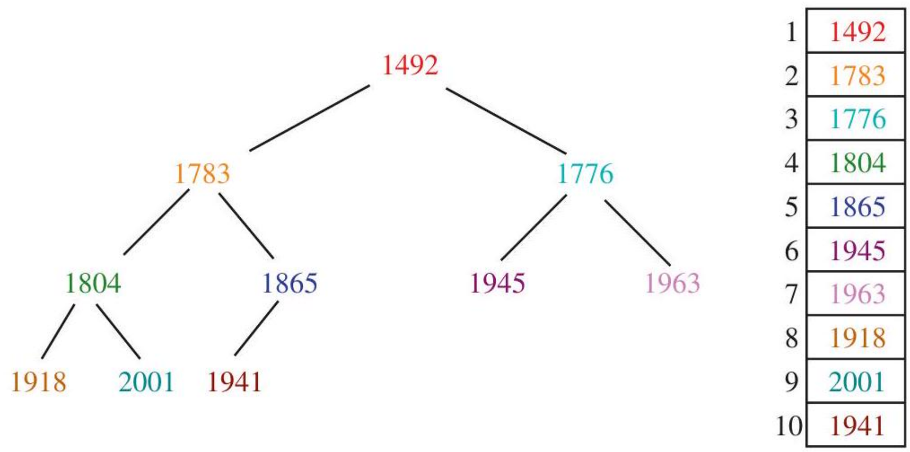


Figure 7.2: A heap-labeled tree of important years from American history (left), with the corresponding implicit heap representation (right).

in positions 2^{l-1} to $2^l - 1$, as shown in Figure 4.2 right). We assume that the array starts with index 1 to simplify matters.

```
typedef struct {
    item_type q[PQ_SIZE+1]; /* body of queue */
    int n; /* number of queue elements */
} priority_queue;
```

What is especially nice about this representation is that the positions of the parent and children of the key at position k are readily determined. The left child of k sits in position $2k$ and the right child in $2k+1$, while the parent of k holds court in position $\lfloor k/2 \rfloor$. Thus, we can move around the tree without any pointers.

```
int pq_parent(int n) {
    if (n == 1) {
        return(-1);
    }
}

int pq_young_child(int n) {
```

int pq_young_child(int n) {
*return(2 * n);*
*return(2 * n);*
}

```
}
```

```
return((int) n/2); /* implicitly take floor(n/2) */
```

This approach means that we can store any binary tree in an array without pointers. What is the catch? Suppose our height h tree was sparse, meaning that the number of nodes $n \ll 2^h - 1$. All missing internal nodes still take up space in our structure, since we must represent a full binary tree to maintain the positional mapping between parents and children.

Space efficiency thus demands that we not allow holes in our tree - meaning that each level be packed as much as it can be. Then only the last level may be incomplete. By packing the elements of the last level as far left as possible, we can represent an n-key tree using the first n elements of the array. If we did not enforce these structural constraints, we might need an array of size $2^n - 1$ to store n elements: consider a right-going twig using positions $1, 3, 7, 15, 31 \ldots$ With heaps all but the last level are filled, so the height h of an n element heap is logarithmic because:

$$\sum_{i=0}^{h-1} 2^i = 2^h - 1 \geq n$$

implying that $h = \lceil \lg(n+1) \rceil$.
This implicit representation of binary trees saves memory, but is less flexible than using pointers. We cannot store arbitrary tree topologies without wasting large amounts of space. We cannot move subtrees around by just changing a single pointer, only by explicitly moving all the elements in the subtree. This loss of flexibility explains why we cannot use this idea to represent binary search trees, but it works just fine for heaps.

Stop and Think: Who's Where in the Heap? *Problem: How can we efficiently search for a particular key k in a heap?*

Solution: We can't. Binary search does not work because a heap is not a binary search tree. We know almost nothing about the relative order of the $n/2$ leaf elements in a heap-certainly nothing that lets us avoid doing linear search through them.

7.2.2 Constructing Heaps

Heaps can be constructed incrementally, by inserting each new element into the left-most open spot in the array, namely the $(n+1)$ st position of a previously n-element heap. This ensures the desired balanced shape of the heap-labeled tree, but does not maintain the dominance ordering of the keys. The new key might be less than its parent in a min-heap, or greater than its parent in a max-heap.

The solution is to swap any such dissatisfied element with its parent. The old parent is now happy, because it is properly dominated. The other child of the old parent is still happy, because it is now dominated by an element even more

extreme than before. The new element is now happier, but may still dominate its new parent. So we recur at a higher level, bubbling up the new key to its proper *position in the hierarchy. Since we replace the root of a subtree by a larger one at each step, we preserve the heap order elsewhere.*

```
void pq_insert(priority_queue *q, item_type x) {
    if (q->n >= PQ_SIZE) {
        printf("Warning: priority queue overflow! \n");
    } else {
        q->n = (q->n) + 1;
        q->q[q->n] = x;
        bubble_up(q, q->n);
    }
}
void bubble_up(priority_queue *q, int p) {
    if (pq_parent(p) == -1) {
        return; /* at root of heap, no parent */
    }
    if (q->q[pq_parent(p)] > q->q[p]) {
        pq_swap(q, p, pq_parent(p));
        bubble_up(q, pq_parent(p));
    }
}
```

This swap process takes constant time at each level. Since the height of an n-element heap is $\lfloor \lg n \rfloor$, each insertion takes at most $O(\log n)$ time. A heap of n elements can thus be constructed in $O(n \log n)$ time through n such insertions:

```
void pq_init(priority_queue *q) {
    q->n = 0;
}
void make_heap(priority_queue *q, item_type s[], int n) {
    int i; /* counter */
    pq_init(q);
    for (i = 0; i < n; i++) {
        pq_insert(q, s[i]);
    }
}
```

7.2.3 Extracting the Minimum

The remaining priority queue operations are identifying and deleting the dominant element. Identification is easy, since the top of the heap sits in the first position of the array.

Removing the top element leaves a hole in the array. This can be filled by moving the element from the right-most leaf (sitting in the nth position of the array) into the first position.

The shape of the tree has been restored but (as after insertion) the labeling of the root may no longer satisfy the heap property. Indeed, this new root may *be dominated by both of its children. The root of this min-heap should be the smallest of three elements, namely the current root and its two children. If the current root is dominant, the heap order has been restored. If not, the dominant child should be swapped with the root and the problem pushed down to the next level.*

This dissatisfied element bubbles down the heap until it dominates all its children, perhaps by becoming a leaf node and ceasing to have any. This percolatedown operation is also called heapify, because it merges two heaps (the subtrees below the original root) with a new key.

```
item_type extract_min(priority_queue *q) \{
    int $\min =-1 ; \quad / *$ minimum value */
    if ( $\mathrm{q}^{->\mathrm{n}}<=0$ ) \{
        printf("Warning: empty priority queue. \n");
    \} else \{
        min $=$ q->q[1];
        $\mathrm{q}->\mathrm{q}[1]=\mathrm{q}->\mathrm{q}\left[\mathrm{q}^{->\mathrm{n}
        $\mathrm{q}->\mathrm{n}=\mathrm{q}->\mathrm{n}-1$;
        bubble_down(q, 1);
    \}
    return(min);
\}
```

void bubble_down(priority_queue q, int p) {
int c; / child index /
int i; / counter /
*int min_index; / index of lightest child */*
c = pq_young_child(p);
min_index $= p$;
for $(i = 0$; $i <= 1; i++$) {

```
    if ((c + i) <= q->n) {
        if (q->q[min_index] > q->q[c + i]) {
            min_index = c + i;
        }
        }
    }
    if (min_index != p) {
        pq_swap(q, p, min_index);
        bubble_down(q, min_index);
    }
}
```

We will reach a leaf after $\lfloor \lg n \rfloor$ *bubble_down steps, each constant time. Thus, root deletion is completed in* $O(\log n)$ *time.*

Repeatedly exchanging the maximum element with the last element and calling heapify yields an $O(n \log n)$ *sorting algorithm, named heapsort.*

```
void heapsort_(item_type s[], int n) {
    int i; /* counters */
    priority_queue q; /* heap for heapsort */
    make_heap(&q, s, n);
    for (i = 0; i < n; i++) {
        s[i] = extract_min(&q);
    }
}
```

Heapsort is a great sorting algorithm. It is simple to program; indeed, the complete implementation has been presented above. It runs in worst-case $O(n \log n)$ time, which is the best that can be expected from any sorting algorithm. It is an in-place sort, meaning it uses no extra memory over the array containing the elements to be sorted. Admittedly, as implemented here, my heapsort is not in-place because it creates the priority queue in q, not s. But each newly extracted element fits perfectly in the slot freed up by the shrinking heap, leaving behind a sorted array. Although other algorithms prove slightly faster in practice, you won't go wrong using heapsort for sorting data that sits in the computer's main memory.

Priority queues are very useful data structures. Recall they were the hero of the war story described in Section 3.6 (page 89. A complete set of priority queue implementations is presented in the catalog, in Section 15.2 (page 445 .

7.2.4 Faster Heap Construction (*)

As we have seen, a heap can be constructed on n elements by incremental insertion in $O(n \log n)$ time. Surprisingly, heaps can be constructed even faster, by using our bubble_down procedure and some clever analysis.

Suppose we pack the n keys destined for our heap into the first n elements of our priority-queue array. The shape of our heap will be right, but the dominance order will be all messed up. How can we restore it?

Consider the array in reverse order, starting from the last (n th) position. It represents a leaf of the tree and so dominates its non-existent children. The same is the case for the last $n/2$ positions in the array, because all are leaves. If we continue to walk backwards through the array we will eventually encounter an internal node with children. This element may not dominate its children, but its children represent well-formed (if small) heaps.

This is exactly the situation the bubble_down procedure was designed to handle, restoring the heap order of an arbitrary root element sitting on top of two sub-heaps. Thus, we can create a heap by performing $n/2$ non-trivial calls to the bubble_down procedure:

```
void make_heap_fast(priority_queue *q, item_type s[], int n) {
    int i; /* counter */
    q->n = n;
    for (i = 0; i < n; i++) {
```

```
        q->q[i + 1] = s[i];
    }
    for (i = q->n/2; i >= 1; i--) {
        bubble_down(q, i);
    }
}
```

Multiplying the number of calls to bubble_down (n) times an upper bound on the cost of each operation $(O(\log n))$ *gives us a running time analysis of* $O(n \log n)$*. This would make it no faster than the incremental insertion algorithm described above.*

But note that it is indeed an upper bound, because only the last insertion will actually take $\lfloor \lg n \rfloor$ *steps. Recall that bubble_down takes time proportional to the height of the heaps it is merging. Most of these heaps are extremely small. In a full binary tree on n nodes, there are* $n/2$ *nodes that are leaves (i.e. height 0),* $n/4$ *nodes that are height* $1, n/8$ *nodes that are height 2 , and so on. In general, there are at most* $\lceil n/2^{h+1} \rceil$ *nodes of height h, so the cost of building a heap is:*

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \lceil n/2^{h+1} \rceil h \leq n \sum_{h=0}^{\lfloor \lg n \rfloor} h/2^h \leq 2n$$

Since this sum is not quite a geometric series, we can't apply the usual identity to get the sum, but rest assured that the puny contribution of the numerator (h) *is crushed by the denominator* $\left(2^h\right)$*. The series quickly converges to linear.*

Does it matter that we can construct heaps in linear time instead of $O(n \log n)$ *? Not really. The construction time did not dominate the complexity of heapsort, so improving the construction time does not improve its worst-case performance. Still, it is an impressive display of the power of careful analysis, and the free lunch that geometric series convergence can sometimes provide.*

Stop and Think: Where in the Heap? *Problem: Given an array-based heap on n elements and a real number x, efficiently determine whether the k th smallest element in the heap is greater than or equal to x. Your algorithm should be* $O(k)$ *in the worst case, independent of the size of the heap. (Hint: you do not have to find the k th smallest element; you need only to determine its relationship to x.)*

Solution: There are at least two different ideas that lead to correct but inefficient algorithms for this problem:

- *Call extract-min k times, and test whether all of these are less than x. This explicitly sorts the first k elements and so gives us more information than the desired answer, but it takes* $O(k \log n)$ *time to do so.*

- *The k th smallest element cannot be deeper than the k th level of the heap, since the path from it to the root must go through elements of decreasing value. We can thus look at all the elements on the first k levels of the heap, and count how many of them are less than x, stopping when we* *either find k of them or run out of elements. This is correct, but takes $O\left(\min\left(n, 2^k\right)\right)$ time, since the top k elements have $2^k - 1$ elements.*

An $O(k)$ solution can look at only k elements smaller than x, plus at most $O(k)$ elements greater than x. Consider the following recursive procedure, called at the root with $i = 1$ and $count = k$:

```
int heap_compare(priority_queue *q, int i, int count, int x) {
    if ((count <= 0) || (i > q->n)) {
        return(count);
    }
    if (q->q[i] < x) {
        count = heap_compare(q, pq_young_child(i), count-1, x);
        count = heap_compare(q, pq_young_child(i)+1, count, x);
    }
    return(count);
}
```

If the root of the min-heap is $\geq x$, then no elements in the heap can be less than x, as by definition the root must be the smallest element. This procedure searches the children of all nodes of weight smaller than x until either (a) we have found k of them, when it returns 0 , or (b) they are exhausted, when it returns a value greater than zero. Thus, it will find enough small elements if they exist.

But how long does it take? The only nodes whose children we look at are those $< x$, and there are at most k of these in total. Each have visited at most two children, so we visit at most $2k + 1$ nodes, for a total time of $O(k)$.

7.2.5 Sorting by Incremental Insertion

Now consider a different approach to sorting via efficient data structures. Select the next element from the unsorted set, and put it into it's proper position in the sorted set:

```
for (i = 1; i < n; i++) {
    j = i;
    while ((j > 0) && (s[j] < s[j - 1])) {
        swap(&s[j], &s[j - 1]);
        j = j-1;
    }\index{Grinch, The}
}
```

Although insertion sort takes $O\left(n^2\right)$ in the worst case, it performs considerably better if the data is almost sorted, since few iterations of the inner loop suffice to sift it into the proper position.

Insertion sort is perhaps the simplest example of the incremental insertion technique, where we build up a complicated structure on n items by first building it on $n - 1$ items and then making the necessary changes to add the last item.

Note that faster sorting algorithms based on incremental insertion follow from more efficient data structures. Insertion into a balanced search tree takes

$O(\log n)$ *per operation, or a total of* $O(n \log n)$ *to construct the tree. An inorder traversal reads through the elements in sorted order to complete the job in linear time.*

7.3 War Story: Give me a Ticket on an Airplane

I came into this particular job seeking justice. I'd been retained by an air travel company to help design an algorithm to find the cheapest available airfare from city x *to city* y*. Like most of you, I suspect, I'd been baffled at the crazy price fluctuations of ticket prices under modern "yield management." The price of flights seems to soar far more efficiently than the planes themselves. The problem, it seemed to me, was that airlines never wanted to show the true cheapest price. If I did my job right, I could make damned sure they would show it to me next time.*

"Look," I said at the start of the first meeting. "This can't be so hard. Construct a graph with vertices corresponding to airports, and add an edge between each airport pair (u, v) *that shows a direct flight from* u *to* v*. Set the weight of this edge equal to the cost of the cheapest available ticket from* u *to* v*. Now the cheapest fair from* x *to* y *is given by the shortest* $x - y$ *path in this graph. This path/fare can be found using Dijkstra's shortest path algorithm. Problem solved!" I announced, waving my hand with a flourish.*

The assembled cast of the meeting nodded thoughtfully, then burst out laughing. It was I who needed to learn something about the overwhelming complexity of air travel pricing. There are literally millions of different fares available at any time, with prices changing several times daily. Restrictions on the availability of a particular fare in a particular context are enforced by a vast set of pricing rules. These rules are an industry-wide kludge - a complicated structure with little in the way of consistent logical principles. My favorite rule exceptions applied only to the country of Malawi. With a population of only 18 million and per-capita income of $\$1,234$ *(180 th in the world), they prove to be an unexpected powerhouse shaping world aviation price policy. Accurately pricing any air itinerary requires at least implicit checks to ensure the trip doesn't take us through Malawi.*

The real problem is that there can easily be 100 different fares for the first flight leg, say from Los Angeles (LAX) to Chicago (ORD), and a similar number for each subsequent leg, say from Chicago to New York (JFK). The cheapest possible LAX-ORD fare (maybe an AARP children's price) might not be combinable with the cheapest ORD-JFK fare (perhaps a pre-Ramadan special that can only be used with subsequent connections to Mecca).

After being properly chastised for oversimplifying the problem, I got down to work. I started by reducing the problem to the simplest interesting case. "So, you need to find the cheapest two-hop fare that passes your rule tests. Is there a

way to decide in advance which pairs will pass without explicitly testing them?"
"No, there is no way to tell," they assured me. "We can only consult a

		X+Y
		$\$150(1,1)$
		X
		$\$100$
		$\$160(2,1)$
$\$110$	$\$50$	
		$\$130$
	$\$75$	$\$180(3,1)$
	$\$125$	$\$185(2,2)$
		$\$205(3,2)$
		$\$235(2,3)$
		$\$255(3,3)$

Figure 4.3: Sorting the pairwise sums of lists X and Y.

black box routine to decide whether a particular price is available for the given itinerary/travelers."
"So our goal is to call this black box on the fewest number of combinations. This means evaluating all possible fare combinations in order from cheapest to most expensive, and stopping as soon as we encounter the first legal combination."
"Right."
"Why not construct the $m \times n$ possible price pairs, sort them in terms of cost, and evaluate them in sorted order? Clearly this can be done in $O(nm \log(nm))$ time. [2]
"That is basically what we do now, but it is quite expensive to construct the full set of $m \times n$ pairs, since the first one might be all we need."

I caught a whiff of an interesting problem. "So what you really want is an efficient data structure to repeatedly return the next most expensive pair without constructing all the pairs in advance."

This was indeed an interesting problem. Finding the largest element in a set under insertion and deletion is exactly what priority queues are good for. The catch here is that we could not seed the priority queue with all values in advance. We had to insert new pairs into the queue after each evaluation.

I constructed some examples, like the one in Figure 4.3. We could represent each fare by the list indexes of its two components. The cheapest single fare will certainly be constructed by adding up the cheapest component from both lists, described $(1,1)$. The second cheapest fare would be made from the head of one list and the second element of another, and hence would be either $(1,2)$ or $(2,1)$. Then it gets more complicated. The third cheapest could either be the unused pair above or $(1,3)$ or $(3,1)$. Indeed it would have been $(3,1)$ in the example above if the third fare of X had been $\$120$.
"Tell me," I asked. "Do we have time to sort the two respective lists of fares in increasing order?"

[2] The question of whether all such sums can be sorted faster than nm arbitrary integers

"Don't have to," the leader replied. "They come out in sorted order from the database."

That was good news! It meant there was a natural order to the pair values. We never need to evaluate the pairs $(i+1, j)$ *or* $(i, j+1)$ *before* (i, j)*, because they clearly defined more expensive fares.*
"Got it!," I said. "We will keep track of index pairs in a priority queue, with the sum of the fare costs as the key for the pair. Initially we put only the pair $(1,1)$ *in the queue. If it proves not to be feasible, we put in its two successors-namely* $(1,2)$ *and* $(2,1)$*. In general, we enqueue pairs* $(i+1, j)$ *and* $(i, j+1)$ *after evaluating/rejecting pair* (i, j)*. We will get through all the pairs in the right order if we do so."*

The gang caught on quickly. "Sure. But what about duplicates? We will construct pair (x, y) *two different ways, both when expanding* $(x-1, y)$ *and (* $x, y-1$ *)."*
"You are right. We need an extra data structure to guard against duplicates. The simplest might be a hash table to tell us whether a given pair exists in the priority queue before we insert a duplicate. In fact, we will never have more than n *active pairs in our data structure, since there can only be one pair for each distinct value of the first coordinate."*

And so it went. Our approach naturally generalizes to itineraries with more than two legs, with complexity that grows with the number of legs. The bestfirst evaluation inherent in our priority queue enabled the system to stop as soon as it found the provably cheapest fare. This proved to be fast enough to provide interactive response to the user. That said, I never noticed airline tickets getting cheaper as a result.

7.4 Mergesort: Sorting by Divide and Conquer

Recursive algorithms reduce large problems into smaller ones. A recursive approach to sorting involves partitioning the elements into two groups, sorting each of the smaller problems recursively, and then interleaving the two sorted lists to totally order the elements. This algorithm is called mergesort, recognizing the importance of the interleaving operation:

$$\text{Mergesort}(A[1,\ldots,n])$$
$$\text{Merge}(\text{MergeSort}(A[1,\ldots,\lfloor n/2\rfloor]), \text{MergeSort}(A[\lfloor n/2\rfloor+1,\ldots,n]))$$

The basis case of the recursion occurs when the subarray to be sorted consists of at most one element, so no rearrangement is necessary. A trace of the execution of mergesort is given in Figure 4.4 Picture the action as it happens

is a notorious open problem in algorithm theory. See
citefredman1976how
citelambert1992sorting for more on $X+Y$ sorting, as the problem is known.

during an in-order traversal of the tree, with each merge occurring after the two child calls return sorted subarrays.

The efficiency of mergesort depends upon how efficiently we can combine the two sorted halves into a single sorted list. We could concatenate them into one

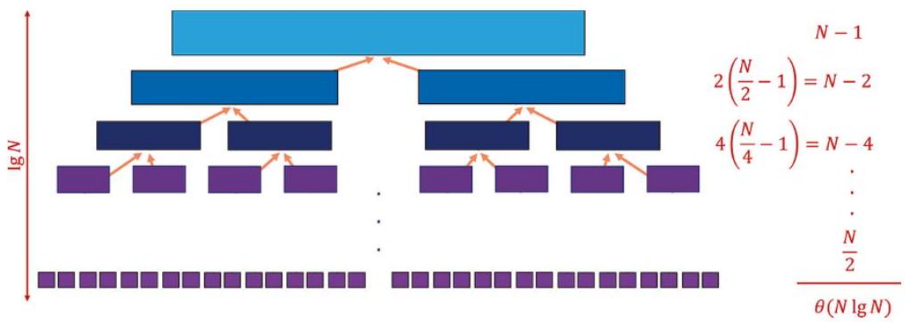


Figure 7.3: The recursion tree for mergesort. The tree has height $\lceil \log_2 n \rceil$, and the cost of the merging operations on each level are $\Theta(n)$, yielding an $\Theta(n \log n)$ time algorithm.

list and call heapsort or some other sorting algorithm to do it, but that would just destroy all the work spent sorting our component lists.

Instead we can merge the two lists together. Observe that the smallest overall item in two lists sorted in increasing order (as above) must sit at the top of one of the two lists. This smallest element can be removed, leaving two sorted lists behind - one slightly shorter than before. The second smallest item overall must now be atop one of these lists. Repeating this operation until both lists are empty will merge two sorted lists (with a total of n *elements between them) into one, using at most* $n-1$ *comparisons or* $O(n)$ *total work.*

What is the total running time of mergesort? It helps to think about how much work is done at each level of the execution tree, as shown in Figure 4.4 If we assume for simplicity that n *is a power of two, the* k *th level consists of all the* 2^k *calls to mergesort processing subranges of* $n/2^k$ *elements.*

The work done on the zeroth level (the top) involves merging one pair of sorted lists, each of size $n/2$*, for a total of at most* $n-1$ *comparisons. The work done on the first level (one down) involves merging two pairs of sorted lists, each of size* $n/4$*, for a total of at most* $n-2$ *comparisons. In general, the work done on the* k *th level involves merging* 2^k *pairs of sorted lists, each of size* $n/2^{k+1}$*, for a total of at most* $n-2^k$ *comparisons. Linear work is done merging all the elements on each level. Each of the* n *elements appears in exactly one subproblem on each level. The most expensive case (in terms of comparisons) is actually the top level.*

The number of elements in a subproblem gets halved at each level. The number of times we can halve n *until we get to 1 is* $\lceil \lg n \rceil$*. Because the recursion* *goes* $\lg n$ *levels deep, and a linear amount of work is done per level, mergesort takes* $O(n \log n)$ *time in the worst case.*

Mergesort is a great algorithm for sorting linked lists, because it does not rely on random access to elements like heapsort and quicksort. Its primary disadvantage is the need for an auxiliary buffer when sorting arrays. It is easy to merge two sorted linked lists without using any extra space, just by rearranging the pointers. However, to merge two sorted arrays (or portions of an array), we need to use a third array to store the result of the merge to avoid stepping on the

component arrays. Consider merging $\{4, 5, 6\}$ *with* $\{1, 2, 3\}$*, packed from left to right in a single array. Without the buffer, we would overwrite the elements of the left half during merging and lose them.*

Mergesort is a classic divide-and-conquer algorithm. We are ahead of the game whenever we can break one large problem into two smaller problems, because the smaller problems are easier to solve. The trick is taking advantage of the two partial solutions to construct a solution of the full problem, as we did with the merge operation. Divide and conquer is an important algorithm design paradigm, and will be the subject of Chapter 5

Implementation The divide-and-conquer mergesort routine follows naturally from the pseudocode:

```
void merge_sort(item_type s[], int low, int high) {
    int middle; /* index of middle element */
    if (low < high) {
        middle = (low + high) / 2;
        merge_sort(s, low, middle);
        merge_sort(s, middle + 1, high);
        merge(s, low, middle, high);
    }
}
```

More challenging turns out to be the details of how the merging is done. The problem is that we must put our merged array somewhere. To avoid losing an element by overwriting it in the course of the merge, we first copy each subarray to a separate queue and merge these elements back into the array. In particular:

```
void merge(item_type s[], int low, int middle, int high) {
    int i; /* counter */
    queue buffer1, buffer2; /* buffers to hold elements for merging */
    init_queue(&buffer1);
    init_queue(&buffer2);
    for (i = low; i <= middle; i++) enqueue(&buffer1, s[i]);
    for (i = middle + 1; i <= high; i++) enqueue(&buffer2, s[i]);
    i = low;
    while (!(empty_queue(&buffer1) || empty_queue(&buffer2))) {
        if (headq(&buffer1) <= headq(&buffer2)) {
            s[i++] = dequeue(&buffer1);
```

```
        } else {
        s[i++] = dequeue(&buffer2);
        }
    }
    while (!empty_queue(&buffer1)) {
        s[i++] = dequeue(&buffer1);
    }
    while (!empty_queue(&buffer2)) {
        s[i++] = dequeue(&buffer2);
    }
}
```

Quicksort: Sorting by Randomization Suppose we select an arbitrary item p from the n items we seek to sort. Quicksort (shown in action in Figure 4.5) separates the $n-1$ other items into two piles: a low pile containing all the elements that are $< p$, and a high pile containing all the elements that are $\geq p$. Low and high denote the array positions into which we place the respective piles, leaving a single slot between them for p.

Such partitioning buys us two things. First, the pivot element p ends up in the exact array position it will occupy in the final sorted order. Second, after partitioning no element flips to the other side in the final sorted order. Thus, we can now sort the elements to the left and the right of the pivot independently! This gives us a recursive sorting algorithm, since we can use the partitioning approach to sort each subproblem. The algorithm must be correct, because each element ultimately ends up in the proper position:

```
void quicksort(item_type s[], int l, int h) {
    int p; /* index of partition */
    if (l < h) {
        p = partition(s, l, h);
        quicksort(s, l, p - 1);
        quicksort(s, p + 1, h);
    }
}
```

We can partition the array in one linear scan for a particular pivot element by maintaining three sections of the array: less than the pivot (to the left of firsthigh), greater than or equal to the pivot (between firsthigh and i), and unexplored (to the right of i), as implemented below:

```
int partition(item_type s[], int l, int h) {
    int i; /* counter */
    int p; /* pivot element index */
    int firsthigh; /* divider position for pivot element */
    p = h; /* select last element as pivot */
    firsthigh = l;
```

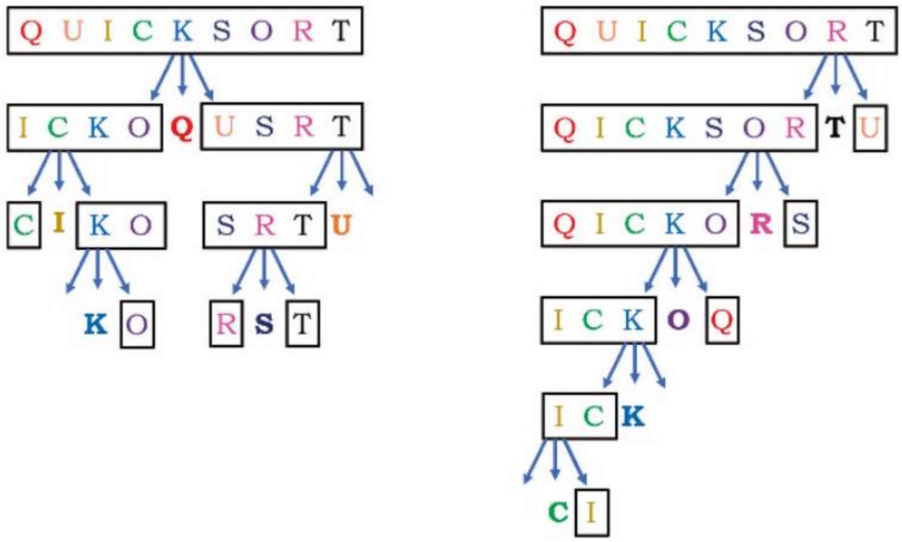


Figure 7.4: first selecting the first element in each subarray as pivot (on left), and then selecting the last element as pivot (on right).

```
    for (i = l; i < h; i++) {
        if (s[i] < s[p]) {
            swap(&s[i], &s[firsthigh]);
            firsthigh++;
        }
    }
    swap(&s[p], &s[firsthigh]);
    return(firsthigh);
}
```

Since the partitioning step consists of at most n swaps, it takes time linear in the number of keys. But how long does the entire quicksort take? As with mergesort, quicksort builds a recursion tree of nested subranges of the n element array. As with mergesort, quicksort spends linear time processing (now partitioning instead of mergeing) the elements in each subarray on each level. As with mergesort, quicksort runs in $O(n \cdot h)$ time, where h is the height of the recursion tree.

The difficulty is that the height of the tree depends upon where the pivot element ends up in each partition. If we get very lucky and happen to repeatedly pick the median element as our pivot, the subproblems are always half the size of those at the previous level. The height represents the number of times we

can halve n until we get down to 1 , meaning $h = \lceil \lg n \rceil$. This happy situation is shown in Figure 4.6(left), and corresponds to the best case of quicksort.

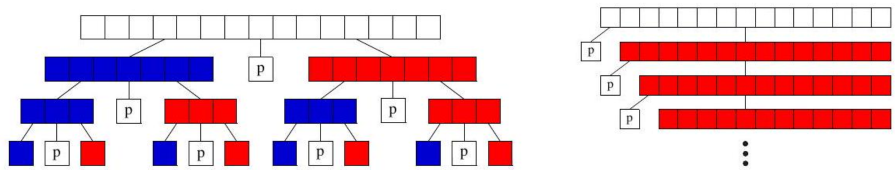


Figure 7.5: The best-case (left) and worst-case (right) recursion trees for quicksort. The left partition is in blue and the right partition in red.

Now suppose we consistently get unlucky, and our pivot element always splits the array as unequally as possible. This implies that the pivot element is always the biggest or smallest element in the sub-array. After this pivot settles into its position, we will be left with one subproblem of size $n-1$. *After doing linear work we have reduced the size of our problem by just one measly element, as shown in Figure 4.6 (right). It takes a tree of height* $n-1$ *to chop our array down to one element per level, for a worst case time of* $\Theta\left(n^2\right)$.

Thus, the worst case for quicksort is worse than heapsort or mergesort. To justify its name, quicksort had better be good in the average case. Understanding why requires some intuition about random sampling.

7.4.1 Intuition: The Expected Case for Quicksort

The expected performance of quicksort depends upon the height of the partition tree constructed by pivot elements at each step. Mergesort splits the keys into two equal halves, sorts both of them recursively, and then merges the halves in linear time - and hence runs in $O(n \log n)$ *time. Thus, whenever our pivot element is near the center of the sorted array (meaning the pivot is close to the median element), we get the same good split realizing the same running time as mergesort.*

I will give an intuitive explanation of why quicksort runs in $O(n \log n)$ *time in the average case. How likely is it that a randomly selected pivot is a good one? The best possible selection for the pivot would be the median key, because exactly half of elements would end up left, and half the elements right, of the pivot. However, we have only a probability of* $1/n$ *that a randomly selected pivot is the median, which is quite small.*

Suppose we say a key is a good enough pivot if it lies in the center half of the sorted space of keys - those ranked from $n/4$ *to* $3n/4$ *in the space of all keys to be sorted. Such good enough pivot elements are quite plentiful, since half the elements lie closer to the middle than to one of the two ends (see Figure 4.7). Thus, on each selection we will pick a good enough pivot with probability of* $1/2$. *We will make good progress towards sorting whenever we pick a good enough*

pivot.

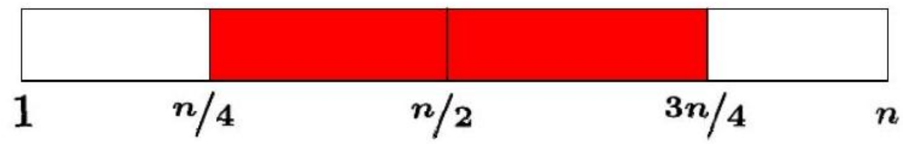


Figure 7.6: Half the time, a *randomly* chosen pivot is close to the median element.

The worst possible good enough pivot leaves the bigger of the two partitions with $3n/4$ *items. This happens also to be the expected size of the larger partition left after picking a random pivot* p*, at the median between the worst possible pivot (* $p = 1$ *or* $p = n$ *leaving a partition of size* $n - 1$ *) and the best possible pivot (*$p = n/2$ *leaving two partitions of size* $n/2$*. So what is the height* h_g *of a quicksort partition tree constructed repeatedly from the expected pivot value? The deepest path through this tree passes through partitions of size* $n, (3/4)n, (3/4)^2 n, \ldots,$ *down to 1 . How many times can we multiply* n *by* $3/4$ *until it gets down to 1 ?*

$$(3/4)^{h_g} n = 1 \Longrightarrow n = (4/3)^{h_g}$$

so $h_g = \log_{4/3} n$*.*
On average, random quicksort partition trees (and by analogy, binary search trees under random insertion) are very good. More careful analysis shows the average height after n *insertions is approximately* $2 \ln n$*. Since* $2 \ln n \approx 1.386 \lg n$*, this is only* 39% *taller than a perfectly balanced binary tree. Since quicksort does* $O(n)$ *work partitioning on each level, the average time is* $O(n \log n)$*. If we are extremely unlucky, and our randomly selected elements are always among the largest or smallest element in the array, quicksort turns into selection sort and runs in* $O\left(n^2\right)$*. But the odds against this are vanishingly small.*

7.4.2 Randomized Algorithms

There is an important subtlety about the expected case $O(n \log n)$ *running time for quicksort. Our quicksort implementation above selected the last element in each sub-array as the pivot. Suppose this program were given a sorted array as input. Then at each step it would pick the worst possible pivot, and run in quadratic time.*

For any deterministic method of pivot selection, there exists a worst-case input instance which will doom us to quadratic time. The analysis presented above made no claim stronger than:
"Quicksort runs in $\Theta(n \log n)$ *time, with high probability, if you give it randomly ordered data to sort."*

But now suppose we add an initial step to our algorithm where we randomly *permute the order of the n elements before we try to sort them. Such a permutation can be constructed in $O(n)$ time (see Section 16.7 for details).*

This might seem like wasted effort, but it provides the guarantee that we can expect $\Theta(n \log n)$ running time whatever the initial input was. The worst case performance still can happen, but it now depends only upon how unlucky we are. There is no longer a well-defined "worst-case" input. We now can claim that:

> *"Randomized quicksort runs in $\Theta(n \log n)$ time on any input, with high probability."*

Alternately, we could get the same guarantee by selecting a random element to be the pivot at each step.

Randomization is a powerful tool to improve algorithms with bad worstcase but good average-case complexity. It can be used to make algorithms more robust to boundary cases and more efficient on highly structured input instances that confound heuristic decisions (such as sorted input to quicksort). It often lends itself to simple algorithms that provide expected-time performance guarantees, which are otherwise obtainable only using complicated deterministic algorithms. Randomized algorithms will be the topic of Chapter 6

Proper analysis of randomized algorithms requires some knowledge of probability theory, and will be deferred to Chapter 6 However, some of the basic approaches to designing efficient randomized algorithms are readily explainable:

- *Random sampling - Want to get an idea of the median value of n things, but don't have either the time or space to look at them all? Select a small random sample of the input and find the median of those. The result should be representative for the full set.*
  This is the idea behind opinion polling, where we sample a small number of people as a proxy for the full population. Biases creep in unless you take a truly random sample, as opposed to the first x people you happen to see. To avoid bias, actual polling agencies typically dial random phone numbers and hope someone answers.

- *Randomized hashing - We have claimed that hashing can be used to implement dictionary search in $O(1)$ "expected time." However, for any hash function there is a given worst-case set of keys that all get hashed to the same bucket. But now suppose we randomly select our hash function from a large family of good ones as the first step of our algorithm. We get the same type of improved guarantee that we did with randomized quicksort.*

- *Randomized search - Randomization can also be used to drive search techniques such as simulated annealing, as will be discussed in detail in Section 12.6 .3 (page 406).*

Stop and Think: Nuts and Bolts *Problem: The nuts and bolts problem is defined as follows. You are given a collection of n bolts of different widths, and n corresponding nuts. You can test* *whether a given nut and bolt fit together, from which you learn whether the nut is too large, too small, or an exact match for the bolt. The differences in size between pairs of nuts or bolts are too small to see by eye, so you cannot compare the sizes of two nuts or two bolts directly. You are asked to match each bolt to each nut as efficiently as possible.*

Give an $O\left(n^2\right)$ *algorithm to solve the nuts and bolts problem. Then give a randomized* $O(n \log n)$ *expected-time algorithm for the same problem.*

Solution: The brute force algorithm for matching nuts and bolts starts with the first bolt and compares it to each nut until a match is found. In the worst case, this will require n comparisons. Repeating this for each successive bolt on all remaining nuts yields an algorithm with a quadratic number of comparisons.

But what if we pick a random bolt and try it? On average, we would expect to get about halfway through the set of nuts before we found the match, so this randomized algorithm would do half the work on average as the worst case. That counts as some kind of improvement, although not an asymptotic one.

Randomized quicksort achieves the desired expected-case running time, so a natural idea is to emulate it on the nuts and bolts problem. The fundamental step in quicksort is partitioning elements around a pivot. Can we partition nuts and bolts around a randomly selected bolt b ?

Certainly we can partition the nuts into those of size less than b and greater than b. But decomposing the problem into two halves requires partitioning the bolts as well, and we cannot compare bolt against bolt. But once we find the matching nut to b, we can use it to partition the bolts accordingly. In $2n - 2$ *comparisons, we partition the nuts and bolts, and the remaining analysis follows directly from randomized quicksort.*

What is interesting about this problem is that no simple deterministic algorithm for nut and bolt sorting is known. It illustrates how randomization makes the bad case go away, leaving behind a simple and beautiful algorithm.

7.4.3 Is Quicksort Really Quick?

There is a clear, asymptotic difference between a $\Theta(n \log n)$ *algorithm and one that runs in* $\Theta\left(n^2\right)$*. Only the most obstinate reader would doubt my claim that mergesort, heapsort, and quicksort will all outperform insertion sort or selection sort on large enough instances.*

But how can we compare two $\Theta(n \log n)$ *algorithms to decide which is faster? How can we prove that quicksort is really quick? Unfortunately, the RAM model and Big Oh analysis provide too coarse a set of tools to make that type of distinction. When faced with algorithms of the same asymptotic complexity, implementation details and system quirks such as cache performance and memory size often prove to be the decisive factor.*

What we can say is that experiments show that when quicksort is implemented well, it is typically two to three times faster than mergesort or heapsort. The primary reason is that the operations in the innermost loop are simpler.

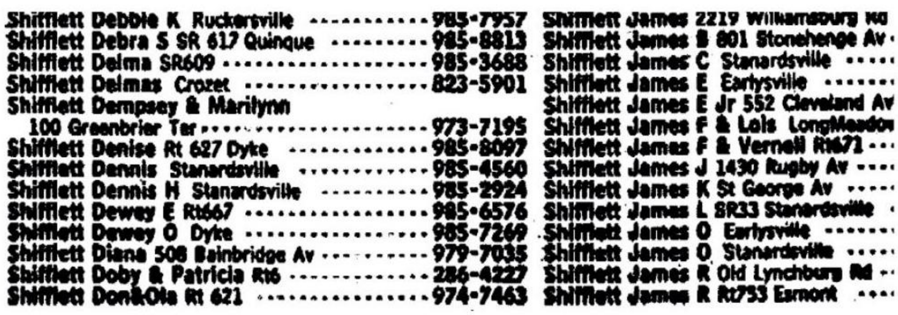

Shifflett Debbie K Ruckersville 985-7957
Shifflett Debra S SR 617 Quinque 985-8813
Shifflett Delma SR609 985-3688
Shifflett Delmas Crozet 823-5901
Shifflett Dempsey & Marilynn
100 Greenbrier Ter 973-7195
Shifflett Denise Rt 627 Dyke 985-8097
Shifflett Dennis Stanardsville 985-4560
Shifflett Dennis H Stanardsville 985-2924
Shifflett Dewey E Rt667 985-6576
Shifflett Dewey O Dyke 985-7269
Shifflett Diana 508 Bainbridge Av 979-7035
Shifflett Doby & Patricia Rt6 286-4227
Shifflett Don&Ola Rt 621 974-7463
Shifflett James 2219 Williamsburg Rd
Shifflett James B 801 Stonehenge Av
Shifflett James C Stanardsville
Shifflett James E Earlysville
Shifflett James E Jr 552 Cleveland Av
Shifflett James F & Lois LongMeadow
Shifflett James F & Vernell Rt671
Shifflett James J 1430 Rugby Av
Shifflett James K St George Av
Shifflett James L SR33 Stanardsville
Shifflett James O Earlysville
Shifflett James O Stanardsville
Shifflett James R Old Lynchburg Rd
Shifflett James R Rt753 Esmont

Figure 7.7: A small subset of Charlottesville *Shiffletts*.

But I can't argue if you don't believe me when I say quicksort is faster. It is a question whose solution lies outside the analytical tools we are using. The best way to tell is to implement both algorithms and experiment.

7.5 Distribution Sort: Sorting via Bucketing

To sort names for a class roster or the telephone book, we could first partition them according to the first letter of the last name. This will create twenty-six different piles, or buckets, of names. Observe that any name in the J pile must occur after all names in the I pile, and before any name in the K pile. Therefore, we can proceed to sort each pile individually and just concatenate the sorted piles together at the end.

Assuming the names are distributed evenly among the buckets, the resulting twenty-six sorting problems should each be substantially smaller than the original problem. By now further partitioning each pile based on the second letter of each name, we can generate smaller and smaller piles. The set of names will be completely sorted as soon as every bucket contains only a single name. Such an algorithm is commonly called bucketsort or distribution sort.

Bucketing is a very effective idea whenever we are confident that the distribution of data will be roughly uniform. It is the idea that underlies hash tables, kd-trees, and a variety of other practical data structures. The downside of such techniques is that the performance can be terrible when the data distribution is not what we expected. Although data structures such as balanced binary trees offer guaranteed worst-case behavior for any input distribution, no such promise exists for heuristic data structures on unexpected input distributions.

Non-uniform distributions do occur in real life. Consider Americans with the uncommon last name of Shifflett. When last I looked, the Manhattan telephone directory (with over one million names) contained exactly five Shiffletts. So how many Shiffletts should there be in a small city of 50,000 people? Figure 4.8 shows a small portion of the two and a half pages of Shiffletts in the Charlottesville, Virginia telephone book. The Shifflett clan is a fixture of the region, but it would *play havoc with any distribution sort program, as refining buckets from S to Sh to Shi to Shif to ... to Shifflett results in no significant partitioning.*

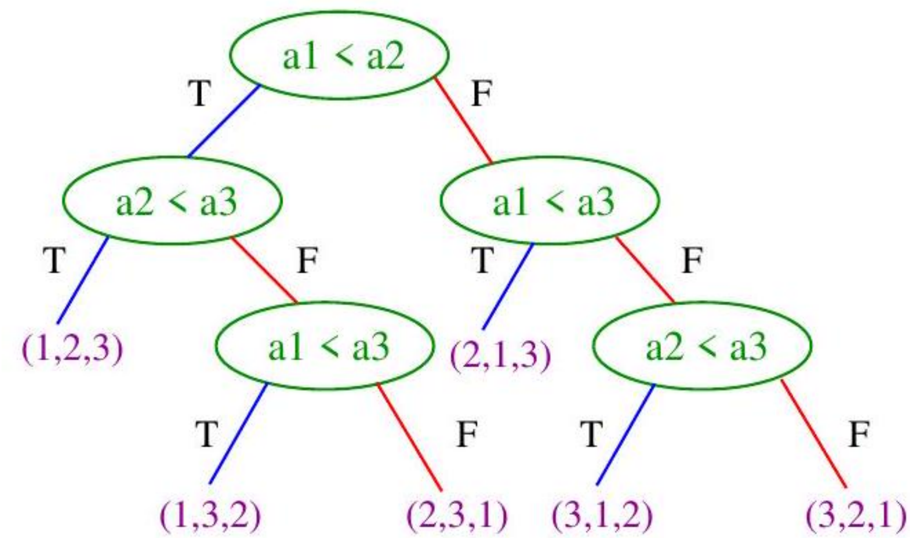


Figure 7.8: Interpreting insertion sort of input array a as a decision tree. Each leaf represents a given input permutation, while the root-to-leaf path describes the sequence of comparisons the algorithm does to sort it.

Take-Home Lesson: Sorting can be used to illustrate many algorithm design paradigms. Data structure techniques, divide and conquer, randomization, and incremental construction all lead to efficient sorting algorithms.

7.5.1 Lower Bounds for Sorting

One last issue on the complexity of sorting. We have seen several sorting algorithms that run in worst-case $O(n \log n)$ time, but none that is linear. To sort n items certainly requires looking at all of them, so any sorting algorithm must be $\Omega(n)$ in the worst case. Might sorting be possible in linear time?

The answer is no, presuming that your algorithm is based on comparing pairs of elements. An $\Omega(n \log n)$ lower bound can be shown by observing that any sorting algorithm must behave differently during execution on each of the n ! possible permutations of n keys. If an algorithm did exactly the same thing with two different input permutations, there is no way that both of them could correctly come out sorted. The outcome of each pairwise comparison governs the run-time behavior of any comparison-based sorting algorithm. We can think of the set of all possible executions for such an algorithm as a tree with n ! leaves, each of which correspond to one input permutation, and each root-to-leaf path *describes the comparisons performed to sort the given input. The minimum height tree corresponds to the fastest possible algorithm, and it happens that* $\lg(n!) = \Theta(n \log n)$.

Figure 4.9 presents the decision tree for insertion sort on three elements. To interpret it, simulate what insertion sort does on the input $a = (3, 1, 2)$. *Because* $a_1 \geq a_2$, *these elements must be swapped to produce a sorted order. Insertion sort then compares the end of the sorted array (the original input* a_1 *) against* a_3. *If* $a_1 \geq a_3$, *the final test of* a_3 *against the head of the sorted part (original input* a_2 *) decides whether to put* a_2 *first or second in sorted order.*

This lower bound is important for several reasons. First, the idea can be

extended to give lower bounds for many applications of sorting, including element uniqueness, finding the mode, and constructing convex hulls. Sorting has one of the few non-trivial lower bounds among algorithmic problems. We will present a different approach to arguing that fast algorithms are unlikely to exist in Chapter 11 .

Note that hashing-based algorithms do not perform such element comparisons, putting them outside the scope of this lower bound. But hashing-based algorithms can get unlucky, and with worst-case luck the running time of any randomized algorithm for one of these problems will be $\Omega(n \log n)$.

7.6 War Story: Skiena for the Defense

I lead a quiet, reasonably honest life. One reward for this is that I don't often find myself on the business end of surprise calls from lawyers. Thus, I was astonished to get a call from a lawyer who not only wanted to talk with me, but wanted to talk to me about sorting algorithms.

It turned out that her firm was working on a case involving high-performance programs for sorting, and needed an expert witness who could explain technical issues to the jury. They knew I knew something about algorithms, but before taking me on they demanded to see my teaching evaluations to prove that I could explain things to people 3^3 *It promised to be an opportunity to learn about how fast sorting programs really worked. I figured I would learn which in-place sorting algorithm was fastest in practice. Was it heapsort or quicksort? What subtle, secret algorithmics made the difference to minimize the number of comparisons in practice?*

The answer was quite humbling. Nobody cared about in-place sorting. The name of the game was sorting huge files, much bigger than can fit in main memory. All the important action was in getting the data on and off a disk. Cute algorithms for doing internal (in-memory) sorting were not the bottleneck, because the real problem lies in sorting gigabytes at a time.

Recall that disks have relatively long seek times, reflecting how long it takes the desired part of the disk to rotate under the read/write head. Once the head is in the right place, the data moves relatively quickly, and it costs about the same to read a large data block as it does to read a single byte. Thus, the goal is *minimizing the number of blocks read/written, and coordinating these operations so the sorting algorithm is never waiting to get the data it needs.*

The disk-intensive nature of sorting is best revealed by the annual Minutesort competition. The goal is to sort as much data in one minute as possible. As of this writing, the current champion is Tencent Sort, which managed to sort 55 terabytes of data in under a minute on a little old 512 -node cluster, each with 20 cores and 512 GB RAM. You can check out the current records at http://sortbenchmark.org/.

That said, which algorithm is best for external sorting? It basically turns out to be a multiway mergesort, employing a lot of engineering and special tricks. You build a heap with members of the top block from each of k sorted lists. By repeatedly plucking the top element off this heap, you build a sorted list merging these k lists. Because this heap is sitting in main memory, these operations are fast. When you have a large enough sorted run, you write it to disk, and free up memory for more data. When you get close to emptying out the elements from the top block of one of the k sorted lists you are merging, load the next block.

It proves very hard to benchmark sorting programs/algorithms at this level and decide which really is fastest. Is it fair to compare a commercial program designed to handle general files with a stripped-down code optimized for integers? The Minutesort competition employs randomly generated 100-byte records. This is a different world than sorting names: Shiffletts are not randomly distributed. For example, one widely employed trick is to strip off a relatively short prefix of the key and initially sort only on that, just to avoid lugging all those extra bytes around.

What lessons did I learn from this? The most important, by far, is to do everything you can to avoid being involved in a lawsuit either as a plaintiff or defendant $\frac{4}{4}$ Courts are not instruments for resolving disputes quickly. Legal battles have a lot in common with military battles: they escalate very quickly, become very expensive in time, money, and soul, and usually end only when both sides are exhausted and compromise. Wise are the parties who can work out their problems without going to court. Properly absorbing this lesson now could save you thousands of times the cost of this book.

On technical matters, it is important to worry about external memory performance whenever you combine very large datasets with low-complexity algorithms (say linear or $\Theta(n \log n)$). Constant factors of even 5 or 10 can make a big difference here between what is feasible and what is hopeless. Of course, quadratic-time algorithms are doomed to fail on large datasets, regardless of data access times.

Chapter Notes *Several interesting sorting algorithms have not been discussed in this section including shellsort, a substantially more efficient version of insertion sort, and radix sort, an efficient algorithm for sorting strings. You can learn more about these and every other sorting algorithm by browsing through Knuth*

[3] One of my more cynical faculty colleagues said this was the first time anyone, anywhere, had ever actually looked at university teaching evaluations.

citeknuth1998taocp3. This includes external sorting, the subject of this chapter's legal war story.

As implemented here, mergesort copies the merged elements into an auxiliary buffer, to avoid overwriting the original elements to be sorted. Through clever but complicated buffer manipulation, mergesort can be implemented in an array without using too much extra storage. Kronrod's algorithm for in-place merging is presented in
citeknuth1998taocp3.

Randomized algorithms are discussed in greater detail in the books by Motwani and Raghavan
citemotwani1995randomized] and Mitzenmacher and Upfal
citemitzenmacher2017probability. The problem of nut and bolt sorting was introduced by Rawlins
citerawlins1992compared. A complicated but deterministic $\Theta(n \log n)$ *algorithm is due to Komlos, Ma, and Szemeredi*
citekomlos1998matching.

7.7 Exercises

Applications of Sorting: Numbers 4-1. [3] The Grinch is given the job of partitioning $2n$ *players into two teams of* n *players each. Each player has a numerical rating that measures how good he or she is at the game. The Grinch seeks to divide the players as unfairly as possible, so as to create the biggest possible talent imbalance between the teams. Show how the Grinch can do the job in* $O(n \log n)$ *time.*

4-2. [3] For each of the following problems, give an algorithm that finds the desired numbers within the given amount of time. To keep your answers brief, feel free to use algorithms from the book as subroutines. For the example, $S = \{6, 13, 19, 3, 8\}, 19 - 3$ *maximizes the difference, while* $8 - 6$ *minimizes the difference.*
(a) Let S *be an unsorted array of* n *integers. Give an algorithm that finds the pair* $x, y \in S$ *that maximizes* $|x - y|$*. Your algorithm must run in* $O(n)$ *worst-case time.*
(b) Let S *be a sorted array of* n *integers. Give an algorithm that finds the pair* $x, y \in S$ *that maximizes* $|x - y|$*. Your algorithm must run in* $O(1)$ *worst-case time.*
(c) Let S *be an unsorted array of* n *integers. Give an algorithm that finds the pair* $x, y \in S$ *that minimizes* $|x - y|$*, for* $x \neq y$*. Your algorithm must run in* $O(n \log n)$ *worst-case time.*
(d) Let S *be a sorted array of* n *integers. Give an algorithm that finds the pair* $x, y \in S$ *that minimizes* $|x - y|$*, for* $x \neq y$*. Your algorithm must run in* $O(n)$ *worst-case time.*
4-3. [3] Take a list of $2n$ *real numbers as input. Design an* $O(n \log n)$ *algorithm that partitions the numbers into* n *pairs, with the property that the partition*

[4] However, it is actually quite interesting serving as an expert witness.

minimizes the maximum sum of a pair. For example, say we are given the numbers $(1,3,5,9)$. *The possible partitions are* $((1,3),(5,9)),((1,5),(3,9))$, *and* $((1,9),(3,5))$. *The pair sums for these partitions are* $(4,14),(6,12)$, *and* $(10,8)$. *Thus, the third partition has 10 as its maximum sum, which is the minimum over the three partitions.*

4-4. [3] Assume that we are given n *pairs of items as input, where the first item is a number and the second item is one of three colors (red, blue, or yellow). Further assume that the items are sorted by number. Give an* $O(n)$ *algorithm to sort the items by color (all reds before all blues before all yellows) such that the numbers for identical colors stay sorted.*

For example: (1,blue), (3,red), (4,blue), (6,yellow), (9 ,red) should become (3,red), (9 ,red), (1 ,blue), (4,blue), (6 ,yellow).

4-5. [3] The mode of a bag of numbers is the number that occurs most frequently in the set. The set $\{4,6,2,4,3,1\}$ *has a mode of 4 . Give an efficient and correct algorithm to compute the mode of a bag of* n *numbers.*

4-6. [3] Given two sets S_1 *and* S_2 *(each of size* n *), and a number* x, *describe an* $O(n \log n)$ *algorithm for finding whether there exists a pair of elements, one from* S_1 *and one from* S_2, *that add up to* x. *(For partial credit, give a* $\Theta\left(n^2\right)$ *algorithm for this problem.)*

4-7. [5] Give an efficient algorithm to take the array of citation counts (each count is a non-negative integer) of a researcher's papers, and compute the researcher's h-index. By definition, a scientist has index h *if* h *of his or her* n *papers have been cited at least* h *times, while the other* $n-h$ *papers each have no more than* h *citations.*

4-8. [3] Outline a reasonable method of solving each of the following problems. Give the order of the worst-case complexity of your methods.

(a) You are given a pile of thousands of telephone bills and thousands of checks sent in to pay the bills. Find out who did not pay.

(b) You are given a printed list containing the title, author, call number, and publisher of all the books in a school library and another list of thirty publishers. Find out how many of the books in the library were published by each company.

(c) You are given all the book checkout cards used in the campus library during the past year, each of which contains the name of the person who took out the book. Determine how many distinct people checked out at least one book.

4-9. [5] Given a set S *of* n *integers and an integer* T, *give an* $O\left(n^{k-1} \log n\right)$ *algorithm to test whether* k *of the integers in* S *add up to* T.

$4-10$. *[3] We are given a set of* S *containing* n *real numbers and a real number* x, *and seek efficient algorithms to determine whether two elements of* S *exist whose sum is exactly* x.

(a) Assume that S *is unsorted. Give an* $O(n \log n)$ *algorithm for the problem.*

(b) Assume that S *is sorted. Give an* $O(n)$ *algorithm for the problem.*

4-11. [8] Design an $O(n)$ *algorithm that, given a list of* n *elements, finds all the elements that appear more than* $n/2$ *times in the list. Then, design an* $O(n)$ *algorithm that, given a list of* n *elements, finds all the elements that appear more than* $n/4$ *times.*

Applications of Sorting: Intervals and Sets 4-12. [3] Give an efficient al-

gorithm to compute the union of sets A and B, where $n = \max(|A|, |B|)$. *The output should be an array of distinct elements that form the union of the sets.*
(a) Assume that A and B are unsorted arrays. Give an $O(n \log n)$ *algorithm for the problem.*
(b) Assume that A and B are sorted arrays. Give an $O(n)$ *algorithm for the problem.*

4-13. [5] A camera at the door tracks the entry time a_i *and exit time* b_i *(assume* $b_i > a_i$*) for each of* n *persons* p_i *attending a party. Give an* $O(n \log n)$ *algorithm that analyzes this data to determine the time when the most people were simultaneously present at the party. You may assume that all entry and exit times are distinct (no ties).*
4-14. [5] Given a list I *of* n *intervals, specified as* (x_i, y_i) *pairs, return a list where the overlapping intervals are merged. For* $I = \{(1,3),(2,6),(8,10),(7,18)\}$ *the output should be* $\{(1,6),(7,18)\}$. *Your algorithm should run in worst-case* $O(n \log n)$ *time complexity.*
4-15. [5] You are given a set S *of* n *intervals on a line, with the* i *th interval described by its left and right endpoints* (l_i, r_i). *Give an* $O(n \log n)$ *algorithm to identify a point* p *on the line that is in the largest number of intervals.*
As an example, for $S = \{(10,40),(20,60),(50,90),(15,70)\}$ *no point exists in all four intervals, but* $p = 50$ *is an example of a point in three intervals. You can assume an endpoint counts as being in its interval.*
4-16. [5] You are given a set S *of* n *segments on the line, where segment* S_i *ranges from* l_i *to* r_i. *Give an efficient algorithm to select the fewest number of segments whose union completely covers the interval from 0 to* m.

Heaps 4-17. [3] Devise an algorithm for finding the k *smallest elements of an unsorted set of* n *integers in* $O(n + k \log n)$.
4-18. [5] Give an $O(n \log k)$*-time algorithm that merges* k *sorted lists with a total of* n *elements into one sorted list. (Hint: use a heap to speed up the obvious* $O(kn)$*-time algorithm).*
4-19. [5] You wish to store a set of n *numbers in either a max-heap or a sorted array. For each application below, state which data structure is better, or if it does not matter. Explain your answers.*
(a) Find the maximum element quickly.
(b) Delete an element quickly.
(c) Form the structure quickly.
(d) Find the minimum element quickly.

4-20. [5] (a) Give an efficient algorithm to find the second-largest key among n *keys. You can do better than* $2n - 3$ *comparisons.*
(b) Then, give an efficient algorithm to find the third-largest key among n *keys. How many key comparisons does your algorithm do in the worst case? Must your algorithm determine which key is largest and second-largest in the process?*

Quicksort 4-21. [3] Use the partitioning idea of quicksort to give an algorithm that finds the median element of an array of n *integers in expected* $O(n)$ *time. (Hint: must you look at both sides of the partition?)*
$4 - 22$. *[3] The median of a set of* n *values is the* $\lceil n/2 \rceil$ *th smallest value.*
(a) Suppose quicksort always pivoted on the median of the current sub-array.

How many comparisons would quicksort make then in the worst case?

(b) Suppose quicksort always pivoted on the $\lceil n/3 \rceil$ *th smallest value of the current sub-array. How many comparisons would be made then in the worst case?*

4-23. [5] Suppose an array A consists of n *elements, each of which is red, white, or blue. We seek to sort the elements so that all the reds come before all the whites, which come before all the blues. The only operations permitted on the keys are:*

- *Examine* (A, i) *- report the color of the* i *th element of* A*.*
- $\mathrm{Swap}(A, i, j)$ *- swap the* i *th element of* A *with the* j *th element.*

Find a correct and efficient algorithm for red-white-blue sorting. There is a linear-time solution.

4-24. [3] Give an efficient algorithm to rearrange an array of n *keys so that all the negative keys precede all the non-negative keys. Your algorithm must be in-place, meaning you cannot allocate another array to temporarily hold the items. How fast is your algorithm?*

4-25. [5] Consider a given pair of different elements in an input array to be sorted, say z_i *and* z_j*. What is the most number of times* z_i *and* z_j *might be compared with each other during an execution of quicksort?*

4-26. [5] Define the recursion depth of quicksort as the maximum number of successive recursive calls it makes before hitting the base case. What are the minimum and maximum possible recursion depths for randomized quicksort?

4-27. [8] Suppose you are given a permutation p *of the integers 1 to* n*, and seek to sort them to be in increasing order* $[1, \ldots, n]$*. The only operation at your disposal is reverse* (p, i, j)*, which reverses the elements of a subsequence* $p_i, \ldots, p_j$ *in the permutation. For the permutation* $[1,4,3,2,5]$ *one reversal (of the second through fourth elements) suffices to sort.*

- *Show that it is possible to sort any permutation using* $O(n)$ *reversals.*
- *Now suppose that the cost of* $\operatorname{reverse}(p, i, j)$ *is equal to its length, the number of elements in the range,* $|j-i|+1$*. Design an algorithm that sorts* p *in* $O\left(n \log^2 n\right)$ *cost. Analyze the running time and cost of your algorithm and prove correctness.*

Mergesort 4-28. [5] Consider the following modification to merge sort: divide the input array into thirds (rather than halves), recursively sort each third, and finally combine the results using a three-way merge subroutine. What is the worst-case running time of this modified merge sort?

4-29. [5] Suppose you are given k *sorted arrays, each with* n *elements, and you want to combine them into a single sorted array of* kn *elements. One approach is to use the merge subroutine repeatedly, merging the first two arrays, then merging the result with the third array, then with the fourth array, and so*

on until you merge in the k th and final input array. What is the running time?
4-30. [5] Consider again the problem of merging k sorted length- n arrays into a single sorted length-kn array. Consider the algorithm that first divides the k arrays into $k/2$ pairs of arrays, and uses the merge subroutine to combine each pair,

resulting in $k/2$ sorted length- $2n$ arrays. The algorithm repeats this step until there is only one length- kn sorted array. What is the running time as a function of n and k ?

Other Sorting Algorithms 4-31. [5] Stable sorting algorithms leave equal-key items in the same relative order as in the original permutation. Explain what must be done to ensure that mergesort is a stable sorting algorithm.
4-32. [5] Wiggle sort: Given an unsorted array A, reorder it such that $A[0] < A[1] > A[2] < A[3] \ldots$. For example, one possible answer for input $[3,1,4,2,6,5]$ is $[1,3,2,5,4,6]$. Can you do it in $O(n)$ time using only $O(1)$ space?
4-33. [3] Show that n positive integers in the range 1 to k can be sorted in $O(n \log k)$ time. The interesting case is when $k \ll n$.
4-34. [5] Consider a sequence S of n integers with many duplications, such that the number of distinct integers in S is $O(\log n)$. Give an $O(n \log \log n)$ worst-case time algorithm to sort such sequences.
4-35. [5] Let $A[1..n]$ be an array such that the first $n - \sqrt{n}$ elements are already sorted (though we know nothing about the remaining elements). Give an algorithm that sorts A in substantially better than $n \log n$ steps.
4-36. [5] Assume that the array $A[1..n]$ only has numbers from $\left\{1, \ldots, n^2\right\}$ but that at most $\log \log n$ of these numbers ever appear. Devise an algorithm that sorts A in substantially less than $O(n \log n)$.
4-37. [5] Consider the problem of sorting a sequence of $n0$'s and 1 's using comparisons. For each comparison of two values x and y, the algorithm learns which of $x < y, x = y,$ or $x > y$ holds.
(a) Give an algorithm to sort in $n-1$ comparisons in the worst case. Show that your algorithm is optimal.
(b) Give an algorithm to sort in $2n/3$ comparisons in the average case (assuming each of the n inputs is 0 or 1 with equal probability). Show that your algorithm is optimal.
4-38. [6] Let P be a simple, but not necessarily convex, n-sided polygon and q an arbitrary point not necessarily in P. Design an efficient algorithm to find a line segment originating from q that intersects the maximum number of edges of P. In other words, if standing at point q, in what direction should you aim a gun so the bullet will go through the largest number of walls. A bullet through a vertex of P gets credit for only one wall. An $O(n \log n)$ algorithm is possible.

Lower Bounds 4-39. [5] In one of my research papers [citeskiena1988encroaching], I discovered a comparison-based sorting algorithm that runs in $O(n \log(\sqrt{n}))$. Given the existence of an $\Omega(n \log n)$ lower bound for sorting, how can this be possible?
4-40. [5] Mr. B. C. Dull claims to have developed a new data structure for priority queues that supports the operations Insert, Maximum, and Extract-Max-

all in $O(1)$ worst-case time. Prove that he is mistaken. (Hint: the argument does not involve a lot of gory details-just think about what this would imply about the $\Omega(n \log n)$ lower bound for sorting.)

7.8 Searching

4-41. [3] A company database consists of 10,000 sorted names, 40% of whom are known as good customers and who together account for 60% of the accesses to the database. There are two data structure options to consider for representing the database:

- *Put all the names in a single array and use binary search.*
- *Put the good customers in one array and the rest of them in a second array. Only if we do not find the query name on a binary search of the first array do we do a binary search of the second array.*

Demonstrate which option gives better expected performance. Does this change if linear search on an unsorted array is used instead of binary search for both options?

4-42. [5] A Ramanujan number can be written two different ways as the sum of two cubes-meaning there exist distinct positive integers a, b, c, and d such that $a^3 + b^3 = c^3 + d^3$. For example, 1729 is a Ramanujan number because $1729 = 1^3 + 12^3 = 9^3 + 10^3$.

(a) Give an efficient algorithm to test whether a given single integer n is a Ramanujan number, with an analysis of the algorithm's complexity.

(b) Now give an efficient algorithm to generate all the Ramanujan numbers between 1 and n, with an analysis of its complexity.

Implementation Challenges 4-43. [5] Consider an $n \times n$ array A containing integer elements (positive, negative, and zero). Assume that the elements in each row of A are in strictly increasing order, and the elements of each column of A are in strictly decreasing order. (Hence there cannot be two zeros in the same row or the same column.) Describe an efficient algorithm that counts the number of occurrences of the element 0 in A. Analyze its running time.

4-44. [6] Implement versions of several different sorting algorithms, such as selection sort, insertion sort, heapsort, mergesort, and quicksort. Conduct experiments to assess the relative performance of these algorithms in a simple application that reads a large text file and reports exactly one instance of each word that appears within it. This application can be efficiently implemented by sorting all the words that occur in the text and then passing through the sorted sequence to identify one instance of each distinct word. Write a brief report with your conclusions.

4-45. [5] Implement an external sort, which uses intermediate files to sort files bigger than main memory. Mergesort is a good algorithm to base such an implementation on. Test your program both on files with small records and on files with large records.

4-46. [8] Design and implement a parallel sorting algorithm that distributes data across several processors. An appropriate variation of mergesort is a likely candidate. Measure the speedup of this algorithm as the number of processors increases. Then compare the execution time to that of a purely sequential mergesort implementation. What are your experiences?

Interview Problems 4-47. [3] If you are given a million integers to sort, what algorithm would you use to sort them? How much time and memory would that consume?

4-48. [3] Describe advantages and disadvantages of the most popular sorting algorithms.

4-49. [3] Implement an algorithm that takes an input array and returns only the unique elements in it.

4-50. [5] You have a computer with only 4 GB of main memory. How do you use it to sort a large file of 500 GB that is on disk?

4-51. [5] Design a stack that supports push, pop, and retrieving the minimum element in constant time.

4-52. [5] Given a search string of three words, find the smallest snippet of the document that contains all three of the search words - that is, the snippet with the smallest number of words in it. You are given the index positions where these words occur in the document, such as word1: (1, 4, 5), word2: (3, 9, 10), and word3: (2, 6, 15). Each of the lists are in sorted order, as above.

$4 - 53$. *[6] You are given twelve coins. One of them is heavier or lighter than the rest. Identify this coin in just three weighings with a balance scale.*

LeetCode *4-1. https://leetcode.com/problems/sort-list/*

4-2. https://leetcode.com/problems/queue-reconstruction-by-height/

4-3. https://leetcode.com/problems/merge-k-sorted-lists/

4-4. https://leetcode.com/problems/find-k-pairs-with-smallest-sums/

HackerRank *4-1. https://www.hackerrank.com/challenges/quicksort3/*

4-2. https://www.hackerrank.com/challenges/mark-and-toys/

4-3. https://www.hackerrank.com/challenges/organizing-containers-of-balls/

Programming Challenges *These programming challenge problems with robot judging are available at https://onlinejudge.org.*

4-1. "Vito's Family" - Chapter 4, problem 10041.

4-2. "Stacks of Flapjacks" - Chapter 4, problem 120.

4-3. "Bridge" - Chapter 4, problem 10037.

4-4. "ShoeMaker's Problem"-Chapter 4, problem 10026.

4-5. "ShellSort"-Chapter 4, problem 10152.

Chapter 5

Chapter 8

Divide and Conquer

One of the most powerful techniques for solving problems is to break them down into smaller, more easily solved pieces. Smaller problems are less overwhelming, and they permit us to focus on details that are lost when we are studying the whole thing. A recursive algorithm starts to become apparent whenever we can break the problem into smaller instances of the same type of problem. Multicore processors now sit in almost every computer, but effective parallel processing requires decomposing jobs into at least as many tasks as the number of processors.

Two important algorithm design paradigms are based on breaking problems down into smaller problems. In Chapter 10 we will see dynamic programming, which typically removes one element from the problem, solves the smaller problem, and then adds back the element to the solution of this smaller problem in the proper way. Divide and conquer instead splits the problem into (say) halves, solves each half, then stitches the pieces back together to form a full solution.

Thus, to use divide and conquer as an algorithm design technique, we must divide the problem into two smaller subproblems, solve each of them recursively, and then meld the two partial solutions into one solution to the full problem. Whenever the merging takes less time than solving the two subproblems, we get an efficient algorithm. Mergesort, discussed in Section 4.5 (page 127), is the classic example of a divide-and-conquer algorithm. It takes only linear time to merge two sorted lists of $n/2$ *elements, each of which was obtained in* $O(n \lg n)$ *time.*

Divide and conquer is a design technique with many important algorithms to its credit, including mergesort, the fast Fourier transform, and Strassen's matrix multiplication algorithm. Beyond binary search and its many variants, however, I find it to be a difficult design technique to apply in practice. Our ability to analyze divide and conquer algorithms rests on our proficiency in solving the recurrence relations governing the cost of such recursive algorithms, so we will introduce techniques for solving recurrences here.

8.1 Binary Search and Related Algorithms

The mother of all divide-and-conquer algorithms is binary search, which is a fast algorithm for searching in a sorted array of keys S. To search for key q, we compare q to the middle key $S[n/2]$. If q appears before $S[n/2]$, it must reside in the left half of S; if not, it must reside in the right half of S. By repeating this process recursively on the correct half, we locate the key in a total of $\lceil \lg n \rceil$ comparisons-a big win over the $n/2$ comparisons expected using sequential search:

```
int binary_search(item_type s[], item_type key, int low, int high) {
    int middle; /* index of middle element */
    if (low > high) {
        return (-1); /* key not found */
    }
    middle = (low + high) / 2;
    if (s[middle] == key) {
        return(middle);
    }
    if (s[middle] > key) {
        return(binary_search(s, key, low, middle - 1));
    } else {
        return(binary_search(s, key, middle + 1, high));
    }
}
```

This much you probably know. What is important is to understand is just how fast binary search is. Twenty questions is a popular children's game where one player selects a word and the other repeatedly asks true/false questions until they guess it. If the word remains unidentified after twenty questions, the first party wins. But the second player has a winning strategy, based on binary search. Take a printed dictionary, open it in the middle, select a word (say "move"), and ask whether the unknown word is before "move" in alphabetical order. Standard dictionaries contain between 50,000 to 200,000 words, so we can be certain that the process will terminate within twenty questions.

8.1.1 Counting Occurrences

Several interesting algorithms are variants of binary search. Suppose that we want to count the number of times a given key k (say "Skiena") occurs in a

given sorted array. Because sorting groups all the copies of k into a contiguous block, the problem reduces to finding that block and then measuring its size.

The binary search routine presented above enables us to find the index of an element x of the correct block in $O(\lg n)$ time. A natural way to identify the boundaries of the block is to sequentially test elements to the left of x until we *find one that differs from the search key, and then repeat this search to the right of x. The difference between the indices of these boundaries (plus one) gives the number of occurrences of k.*

This algorithm runs in $O(\lg n + s)$, where s is the number of occurrences of the key. But this can be as bad as $\Theta(n)$ if the entire array consists of identical keys. A faster algorithm results by modifying binary search to find the boundary of the block containing k, instead of k itself. Suppose we delete the equality test

```
if (s[middle] == key) return(middle);
```

from the implementation above and return the index high instead of -1 on each unsuccessful search. All searches will now be unsuccessful, since there is no equality test. The search will proceed to the right half whenever the key is compared to an identical array element, eventually terminating at the right boundary. Repeating the search after reversing the direction of the binary comparison will lead us to the left boundary. Each search takes $O(\lg n)$ time, so we can count the occurrences in logarithmic time regardless of the size of the block.

By modifying our binary search routine to return (low+high)/2 instead of -1 on an unsuccessful search, we obtain the location between two array elements where the key k should have been. This variant suggests another way to solve our length of run problem. We search for the positions of keys $k - \epsilon$ and $k + \epsilon$, where ϵ is a tiny enough constant that both searches are guaranteed to fail with no intervening keys. Again, doing two binary searches takes $O(\log n)$ time.

8.1.2 One-Sided Binary Search

Now suppose we have an array A consisting of a run of 0 's, followed by an unbounded run of 1 's, and would like to identify the exact point of transition between them. Binary search on the array would find the transition point in $\lceil \lg n \rceil$ tests, if we had a bound n on the number of elements in the array.

But in the absence of such a bound, we can test repeatedly at larger intervals $(A[1], A[2], A[4], A[8], A[16], \ldots)$ until we find a non-zero value. Now we have a window containing the target and can proceed with binary search. This one-sided binary search finds the transition point p using at most $2\lceil \lg p \rceil$ comparisons, regardless of how large the array actually is. One-sided binary search is useful whenever we are looking for a key that lies close to our current position.

8.1.3 Square and Other Roots

The square root of n is the positive number r such that $r^2 = n$. Square root computations are performed inside every pocket calculator, but it is instructive to develop an efficient algorithm to compute them.

First, observe that the square root of $n \geq 1$ must be at least 1 and at most n. Let $l = 1$ and $r = n$. Consider the midpoint of this interval, $m = (l + r)/2$. How does m^2 compare to n ? If $n \geq m^2$, then the square root must be greater than m, so the algorithm repeats with $l = m$. If $n < m^2$, then the square root *must be less than m, so the algorithm repeats with $r = m$. Either way, we have halved the interval using only one comparison. Therefore, after $\lceil \lg n \rceil$ rounds we will have identified the square root to within $\pm 1/2$.*

This bisection method, as it is called in numerical analysis, can also be applied to the more general problem of finding the roots of an equation. We say that x is a root of the function f if $f(x) = 0$. Suppose that we start with values l and r such that $f(l) > 0$ and $f(r) < 0$. If f is a continuous function, there must exist a root between l and r. Depending upon the sign of $f(m)$, where $m = (l + r)/2$, we can cut the window containing the root in half with each test, and stop as soon as our estimate becomes sufficiently accurate.

Root-finding algorithms converging faster than binary search are known for both of these problems. Instead of always testing the midpoint of the interval, these algorithms interpolate to find a test point closer to the actual root. Still, binary search is simple, robust, and works as well as possible without additional information on the nature of the function to be computed.

Take-Home Lesson: Binary search and its variants are the quintessential divide-and-conquer algorithms.

8.2 War Story: Finding the Bug in the Bug

Yutong stood up to announce the results of weeks of hard work. "Dead," he announced defiantly. Everybody in the room groaned.

I was part of a team developing a new approach to create vaccines: Synthetic Attenuated Virus Engineering or SAVE. Because of how the genetic code works, there are typically about 3^n different possible genes that code for any given protein of length n. To a first approximation, all of these are the same, since they describe exactly the same protein. But each of the 3^n synonymous genes use the biological machinery in somewhat different ways, translating at somewhat different speeds.

By substituting a virus gene with a less dangerous replacement, we hoped to create a vaccine: a weaker version of the disease-causing agent that otherwise did the same thing. Your body could fight off the weak virus without you getting sick, along the way training your immune system to fight off tougher villains. But we needed weak viruses, not dead ones: you can't learn anything fighting off something that is already dead.

"Dead means that there must be one place in this 1,200-base region where the virus evolved a signal, a second meaning of the sequence it needs for survival," said our senior virologist. By changing the sequence at this point, we killed the virus. "We have to find this signal to bring it back to life."

"But there are 1,200 places to look! How can we find it?" Yutong asked.

I thought about this a bit. We had to debug a bug. This sounded similar to the problem of debugging a program. I recall many a lonesome night spent trying to figure out exactly which line number was causing my program to crash. I often stooped to commenting out chunks of the code, and then running it again to test

Figure 8.1: Designs of four synthetic genes to locate a specific sequence signal. The green regions are drawn from a viable sequence, while the red regions are drawn from a lethally defective sequence. Genes II, III, and IV were viable while gene I was defective, an outcome that can only be explained by a lethal signal in the region located fifth from the right.

if it still crashed. It was usually easy to figure out the problem after I got the commented-out region down to a small enough chunk. The best way to search for this region was...

"Binary search!" I announced. Suppose we replace the first half of the coding sequence of the viable gene (shown in green) with the coding sequence of the dead, critical signal-deficient strain (shown in red), as in design II in Figure 5.1. If this hybrid gene is viable, it means the critical signal must occur in the right half of the gene, whereas a dead virus implies the problem must occur in the left half. Through a binary search process the signal can be located to one of n regions in $\lceil \log_2 n \rceil$ sequential rounds of experiment.

"We can narrow the area containing the signal in a length- n gene to $n/16$ by doing only four experiments," I informed them. The senior virologist got excited. But Yutong turned pale.

"Four more rounds of experiments!" he complained. "It took me a full month to synthesize, clone, and try to grow the virus the last time. Now you want me to do it again, wait to learn which half the signal is in, and then repeat three more times? Forget about it!"

Yutong realized that the power of binary search came from interaction: the query that we make in round r depends upon the answers to the queries in rounds 1 through $r-1$. Binary search is an inherently sequential algorithm. When each individual comparison is a slow and laborious process, suddenly $\lg n$ comparisons doesn't look so good. But I had a very cute trick up my sleeve.

"Four successive rounds of this will be too much work for you, Yutong. But might you be able to do four different designs at the same time if we could give them to you all at once?" I asked.

"If I am doing the same things to four different sequences at the same time, it is no big deal," he said. "Not much harder than doing just one of them."

That settled, I proposed that they simultaneously synthesize the four virus designs labeled I, II, III, and IV in Figure 5.1. It turns out you can parallelize binary search, provided your queries can be arbitrary subsets instead of connected halves. Observe that each of the columns defined by these four designs consists of a distinct pattern of red and green. The pattern of living/dead among the four synthetic designs thus uniquely defines the position of the critical signal in one experimental round. In this example, virus I happened to be dead while the other three lived, pinpointing the location of the lethal signal to the fifth region from the right.

Yutong rose to the occasion, and after a month of toil (but not months) discovered a new signal in poliovirus SLW$^+$12*. He found the bug in the bug through the idea of divide and conquer, which works best when splitting the problem in half at each point. Note that all four of our designs consist of half red and half green, arranged so all sixteen regions have a distinct pattern of colors. With interactive binary search, the last test selects between just two remaining regions. By expanding each test to have half the sequence, we eliminated the need for sequential tests, making the entire process much faster.*

8.3 Recurrence Relations

Many divide-and-conquer algorithms have time complexities that are naturally modeled by recurrence relations. The ability to solve such recurrences is important to understanding when divide-and-conquer algorithms perform well, and provides an important tool for analysis in general. The reader who balks at the very idea of analysis is free to skip this section, but important insights come from an understanding of the behavior of recurrence relations.

What is a recurrence relation? It is an equation in which a function is defined in terms of itself. The Fibonacci numbers are described by the recurrence relation

$$F_n = F_{n-1} + F_{n-2}$$

together with the initial values $F_0 = 0$ and $F_1 = 1$, as will be discussed in Section 10.1.1 Many other familiar functions are easily expressed as recurrences. Any polynomial can be represented by a recurrence, such as the linear function:

$$a_n = a_{n-1} + 1, a_1 = 1 \longrightarrow a_n = n$$

Any exponential can be represented by a recurrence:

$$a_n = 2a_{n-1}, a_1 = 1 \longrightarrow a_n = 2^{n-1}$$

Finally, lots of weird functions that cannot be easily described using conventional notation can be represented naturally by a recurrence, for example:

$$a_n = na_{n-1}, a_1 = 1 \longrightarrow a_n = n!$$

This shows that recurrence relations are a very versatile way to represent functions.

The self-reference property of recurrence relations is shared with recursive programs or algorithms, as the shared roots of both terms reflect. Essentially, recurrence relations provide a way to analyze recursive structures, such as algorithms.

8.3.1 Divide-and-Conquer Recurrences

A typical divide-and-conquer algorithm breaks a given problem into a smaller pieces, each of which is of size n/b. It then spends $f(n)$ time to combine these subproblem solutions into a complete result. Let $T(n)$ denote the worst-case time this algorithm takes to solve a problem of size n. Then $T(n)$ is given by the following recurrence relation:

$$T(n) = a \cdot T(n/b) + f(n)$$

Consider the following examples, based on algorithms we have previously seen:

- *Mergesort - The running time of mergesort is governed by the recurrence $T(n) = 2T(n/2)+O(n)$, since the algorithm divides the data into equalsized halves and then spends linear time merging the halves after they are sorted. In fact, this recurrence evaluates to $T(n) = O(n \lg n)$, just as we got by our previous analysis.*

- *Binary search - The running time of binary search is governed by the recurrence $T(n) = T(n/2) + O(1)$, since at each step we spend constant time to reduce the problem to an instance half its size. This recurrence evaluates to $T(n) = O(\lg n)$, just as we got by our previous analysis.*

- *Fast heap construction - The bubble_down method of heap construction (described in Section 4.3.4) builds an n-element heap by constructing two $n/2$ element heaps and then merging them with the root in logarithmic time. The running time is thus governed by the recurrence relation $T(n) = 2T(n/2) + O(\lg n)$. This recurrence evaluates to $T(n) = O(n)$, just as we got by our previous analysis.*

Solving a recurrence means finding a nice closed form describing or bounding the result. We can use the master theorem, discussed in Section 5.4, to solve the recurrence relations typically arising from divide-and-conquer algorithms.

8.4 Solving Divide-and-Conquer Recurrences

Divide-and-conquer recurrences of the form $T(n) = aT(n/b)+f(n)$ are generally easy to solve, because the solutions typically fall into one of three distinct cases:

1. *If $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some constant $\epsilon > 0$, then $T(n) = \Theta\left(n^{\log_b a}\right)$.*

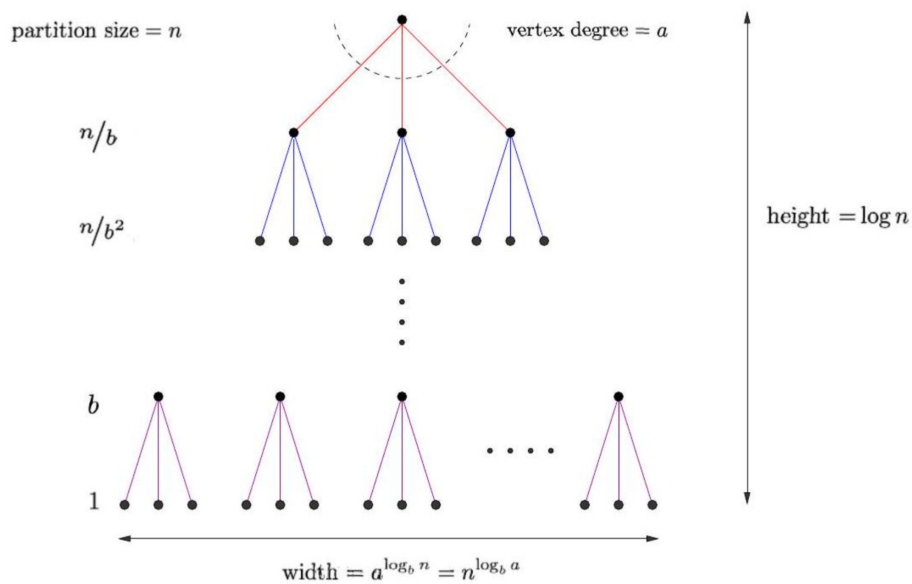


Figure 8.2: The recursion tree resulting from decomposing each problem of size n into a problems of size n/b

2. *If* $f(n) = \Theta\left(n^{\log_b a}\right)$, *then* $T(n) = \Theta\left(n^{\log_b a} \lg n\right)$.

3. *If* $f(n) = \Omega\left(n^{\log_b a+\epsilon}\right)$ *for some constant* $\epsilon > 0$, *and if* $af(n/b) \leq cf(n)$ *for some* $c < 1$, *then* $T(n) = \Theta(f(n))$.

Although this looks somewhat frightening, it really isn't difficult to apply. The issue is identifying which case of this so-called master theorem holds for your given recurrence. Case 1 holds for heap construction and matrix multiplication, while Case 2 holds for mergesort. Case 3 generally arises with clumsier algorithms, where the cost of combining the subproblems dominates everything.

The master theorem can be thought of as a black-box piece of machinery, invoked as needed and left with its mystery intact. However, after a little study it becomes apparent why the master theorem works.

Figure 5.2 shows the recursion tree associated with a typical $T(n) = aT(n/b) + f(n)$ *divide-and-conquer algorithm. Each problem of size n is decomposed into a problems of size* n/b. *Each subproblem of size k takes* $O(f(k))$ *time to deal with internally, between partitioning and merging. The total time for the algorithm is the sum of these internal evaluation costs, plus the overhead of building*

the recursion tree. The height of this tree is $h = \log_b n$ *and the number of leaf nodes is* $a^h = a^{\log_b n}$, *which happens to simplify to* $n^{\log_b a}$ *with some algebraic manipulation.*

The three cases of the master theorem correspond to three different costs, *each of which might be dominant as a function of* a, b, *and* $f(n)$ *:*

- *Case 1: Too many leaves - If the number of leaf nodes outweighs the overall internal evaluation cost, the total running time is* $O\left(n^{\log_b a}\right)$.

- *Case 2: Equal work per level - As we move down the tree, each problem gets smaller but there are more of them to solve. If the sums of the internal evaluation costs at each level are equal, the total running time is the cost per level* $\left(n^{\log_b a}\right)$ *times the number of levels* $(\log_b n)$, *for a total running time of* $O\left(n^{\log_b a} \lg n\right)$.

- *Case 3: Too expensive a root - If the internal evaluation cost grows very rapidly with* n, *then the cost of the root evaluation may dominate everything. Then the total running time is* $O(f(n))$.

Once you accept the master theorem, you can easily analyze any divide-andconquer algorithm, given only the recurrence associated with it. We use this approach on several algorithms below.

8.5 Fast Multiplication

You know at least two ways to multiply integers A *and* B *to get* $A \times B$. *You first learned that* $A \times B$ *meant adding up* B *copies of* A, *which gives an* $O\left(n \cdot 10^n\right)$ *time algorithm to multiply two* n*-digit base-10 numbers. Then you learned to multiply long numbers on a digit-by-digit basis, like*
$9256 \times 5367 = 9256 \times 7 + 9256 \times 60 + 9256 \times 300 + 9256 \times 5000 = 13,787,823$
Recall that those zeros we pad the digit-terms by are not really computed as products. We implement their effect by shifting the product digits to the correct place. Assuming we perform each real digit-by-digit product in constant time, by looking it up in a times table, this algorithm multiplies two n*-digit numbers in* $O\left(n^2\right)$ *time.*

In this section I will present an even faster algorithm for multiplying large numbers. It is a classic divide-and-conquer algorithm. Suppose each number has $n = 2m$ *digits. Observe that we can split each number into two pieces each of* m *digits, such that the product of the full numbers can easily be constructed from the products of the pieces, as follows. Let* $w = 10^{m+1}$, *and represent* $A = a_0 + a_1 w$ *and* $B = b_0 + b_1 w$, *where* a_i *and* b_i *are the pieces of each respective number. Then:*

$$A \times B = (a_0 + a_1 w) \times (b_0 + b_1 w) = a_0 b_0 + a_0 b_1 w + a_1 b_0 w + a_1 b_1 w^2$$

This procedure reduces the problem of multiplying two n*-digit numbers to four products of (* $n/2$ *)-digit numbers. Recall that multiplication by* w *doesn't count: it is simply padding the product with zeros. We also have to add together these four products once computed, which is* $O(n)$ *work.*

Let $T(n)$ denote the amount of time it takes to multiply two n-digit numbers. Assuming we use the same algorithm recursively on each of the smaller products, the running time of this algorithm is given by the recurrence:

$$T(n) = 4T(n/2) + O(n)$$

Using the master theorem (case 1), we see that this algorithm runs in $O\left(n^2\right)$ time, exactly the same as the digit-by-digit method. We divided, but we did not conquer.

Karatsuba's algorithm is an alternative recurrence for multiplication, which yields a better running time. Suppose we compute the following three products:

$$\begin{aligned}
q_0 &= a_0 b_0 \\
q_1 &= (a_0 + a_1)(b_0 + b_1) \\
q_2 &= a_1 b_1
\end{aligned}$$

Note that

$$\begin{aligned}
A \times B = (a_0 + a_1 w) \times (b_0 + b_1 w) &= a_0 b_0 + a_0 b_1 w + a_1 b_0 w + a_1 b_1 w^2 \\
&= q_0 + (q_1 - q_0 - q_2)\, w + q_2 w^2
\end{aligned}$$

so now we have computed the full product with just three half-length multiplications and a constant number of additions. Again, the w terms don't count as multiplications: recall that they are really just zero shifts. The time complexity of this algorithm is therefore governed by the recurrence

$$T(n) = 3T(n/2) + O(n)$$

Since $n = O\left(n^{\log_2 3}\right)$, this is solved by the first case of the master theorem, and $T(n) = \Theta\left(n^{\log_2 3}\right) = \Theta\left(n^{1.585}\right)$. This is a substantial improvement over the quadratic algorithm for large numbers, and indeed beats the standard multiplication algorithm soundly for numbers of 500 digits or so.

This approach of defining a recurrence that uses fewer multiplications but more additions also lurks behind fast algorithms for matrix multiplication. The nested-loops algorithm for matrix multiplication discussed in Section 2.5.4 takes $O\left(n^3\right)$ for two $n \times n$ matrices, because we compute the dot product of n terms for each of the n^2 elements in the product matrix. However, Strassen [citestrassen1969gaussian] discovered a divide-and-conquer algorithm that manipulates the products of seven $n/2 \times n/2$ matrix products to yield the product of two $n \times n$ matrices. This yields a time-complexity recurrence

$$T(n) = 7 \cdot T(n/2) + O\left(n^2\right)$$

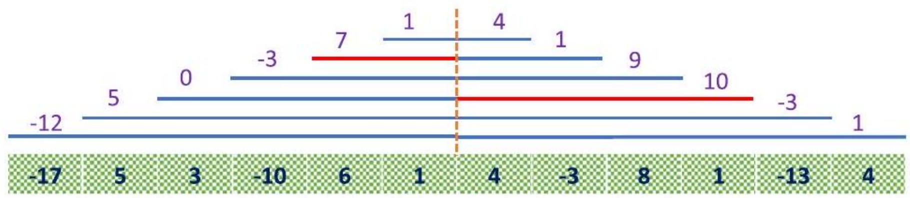


Figure 8.3: The largest subrange sum is either entirely to the left of center or entirely to the right, or (like here) the sum of the largest center-bordering ranges on left and right.

Because $\log_2 7 \approx 2.81, O\left(n^{\log_2 7}\right)$ *dominates* $O\left(n^2\right)$, *so Case 1 of the master theorem applies and* $T(n) = \Theta\left(n^{2.81}\right)$.

This algorithm has been repeatedly "improved" by increasingly complicated recurrences, and the current best is $O\left(n^{2.3727}\right)$. *See Section 16.3 (page 472 for more detail.*

8.6 Largest Subrange and Closest Pair

Suppose you are tasked with writing the advertising copy for a hedge fund whose monthly performance this year was

$$[-17, 5, 3, -10, 6, 1, 4, -3, 8, 1, -13, 4]$$

You lost money for the year, but from May through October you had your greatest gains over any period, a net total of 17 units of gains. This gives you something to brag about.

The largest subrange problem takes an array A *of* n *numbers, and asks for the index pair* i *and* j *that maximizes* $S = \sum_{k=i}^{j} A[k]$. *Summing the entire array does not necessarily maximize* S *because of negative numbers. Explicitly testing each possible interval start-end pair requires* $\Omega\left(n^2\right)$ *time. Here I present a divide-and-conquer algorithm that runs in* $O(n \log n)$ *time.*

Suppose we divide the array A *into left and right halves. Where can the largest subrange be? It is either in the left half or the right half, or includes the middle. A recursive program to find the largest subrange between* $A[l]$ *and* $A[r]$ *can easily call itself to work on the left and right subproblems. How can we find the largest subrange spanning the middle, that is, spanning positions* m *and* $m+1$ *?*

The key is to observe that the largest subrange centered spanning the middle will be the union of the largest subrange on the left ending on m, *and the largest subrange on the right starting from* $m+1$, *as illustrated in Figure 5.3 The value* V_l *of the largest such subrange on the left can be found in linear time with a sweep:*

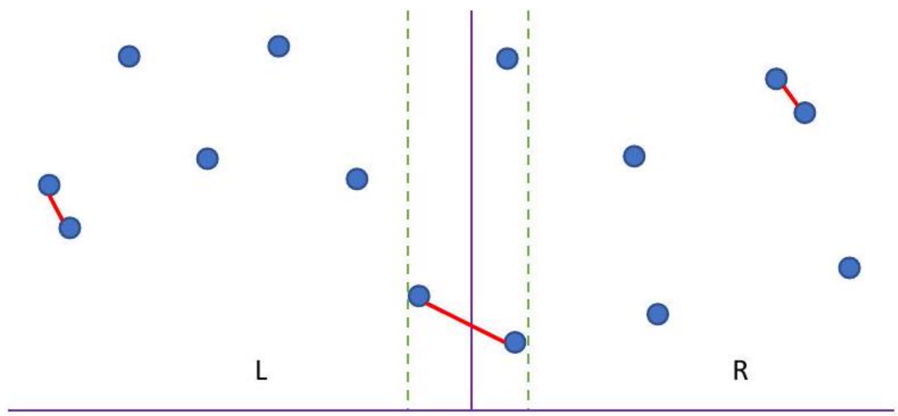


Figure 8.4: The closest pair of points in two dimensions either lie to the left of center, to the right, or in a thin strip straddling the center.

LeftMidMaxRange (A, l, m)
  $S = M = 0$
for $i = m$ *downto* l
  $S = S + A[i]$
   if $(S > M)$ *then* $M = S$
return S

The corresponding value on the right can be found analogously. Dividing n *into two halves, doing linear work, and recurring takes time* $T(n)$*, where*

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

Case 2 of the master theorem yields $T(n) = \Theta(n \log n)$*.*
This general approach of "find the best on each side, and then check what is straddling the middle" can be applied to other problems as well. Consider the problem of finding the smallest distance between pairs among a set of n *points.*

In one dimension, this problem is easy: we saw in Section 4.1 (page 109) that after sorting the points, the closest pair must be neighbors. A linear-time sweep from left to right after sorting thus yields an $\Theta(n \log n)$ *algorithm. But we can replace this sweep by a cute divide-and-conquer algorithm. The closest pair is defined by the left half of the points, the right half, or the pair in the middle, so the following algorithm must find it:*

$$
\begin{aligned}
&\textit{ClosestPair}\ (A, l, r) \\
&\quad \textit{mid}\ = \lfloor (l+r)/2 \rfloor \\
&l_{min}\ = \text{ClosestPair}(A, l,\ \textit{mid}\ ) \\
&r_{min}\ = \text{ClosestPair}(A,\ \textit{mid}\ + 1, r) \\
&\textit{return}\ \min(l_{min}\ , r_{min}\ , A[m+1] - A[m])
\end{aligned}
$$

Because this does constant work per call, its running time is given by the recurrence:

$$T(n) = 2 \cdot T(n/2) + O(1)$$

Case 1 of the master theorem tells us that $T(n) = \Theta(n)$.
This is still linear time and so might not seem very impressive, but let's generalize the idea to points in two dimensions. After we sort the $n(x, y)$ *points according to their* x*-coordinates, the same property must be true: the closest pair is either two points on the left half or two points on the right, or it straddles left and right. As shown in Figure 5.4 these straddling points had better be close to the dividing line (distance* $d < \min(l_{min}\ , r_{min}\ )$ *) and also have very similar* y*-coordinates. With clever bookkeeping, the closest straddling pair can be found in linear time, yielding a running time of*

$$T(n) = 2 \cdot T(n/2) + \Theta(n) = \Theta(n \log n)$$

as defined by Case 2 of the master theorem.

8.7 Parallel Algorithms

Two heads are better than one, and more generally, n *heads better than* $n - 1$. *Parallel processing has become increasingly prevalent, with the advent of multicore processors and cluster computing.*

8.7.1 Data Parallelism

Divide and conquer is the algorithm paradigm most suited to parallel computation. Typically, we seek to partition our problem of size n *into* p *equal-sized parts, and simultaneously feed one to each processor. This reduces the time to completion (or makespan) from* $T(n)$ *to* $T(n/p)$*, plus the cost of combining the results together. If* $T(n)$ *is linear, this gives us a maximum possible speedup of* p. *If* $T(n) = \Theta\left(n^2\right)$ *it may look like we can do even better, but this is generally an illusion. Suppose we want to sweep through all pairs of* n *items. Sure we can partition the items into* p *independent chunks, but* $n^2 - p(n/p)^2$ *of the* n^2 *possible pairs will not ever have both elements on the same processor.*

Multiple processors are typically best deployed to exploit data parallelism, running a single algorithm on different and independent data sets. For example, computer animation systems must render thirty frames per second for realistic *animation. Assigning each frame to a distinct processor, or dividing each image into regions assigned to different processors, might be the best way to get the job done in time. Such tasks are often called embarrassingly parallel.*

Generally speaking, such data parallel approaches are not algorithmically interesting, but they are simple and effective. There is a more advanced world of parallel algorithms where different processors synchronize their efforts so they can together solve a single problem quicker than one can. These algorithms are out of the scope of what we will cover in this book, but be aware of the challenges involved in the design and implementation of sophisticated parallel algorithms.

8.7.2 Pitfalls of Parallelism

There are several potential pitfalls and complexities associated with parallel algorithms:

- *There is often a small upper bound on the potential win - Suppose that you have access to a machine with 24 cores that can be devoted exclusively to your job. These can potentially be used to speed up the fastest sequential program by up to a factor of 24 . Sweet! But even greater performance gains can often result from developing more efficient sequential algorithms. Your time spent parallelizing a code might well be better spent enhancing the sequential version. Performance-tuning tools such as profilers are better developed for sequential machines/programs than for parallel models.*

- *Speedup means nothing - Suppose my parallel program runs 24 times faster on a 24 -core machine than it does on a single processor. That's great, isn't it? If you get linear speedup and can increase the number of processors without bound, you will eventually beat any sequential algorithm. But the one-processor parallel version of your code is likely to be a crummy sequential algorithm, so measuring speedup typically provides an unfair test of the benefits of parallelism. And it is hard to buy machines with an unlimited number of cores.*

The classic example of this phenomenon occurs in the minimax game-tree search algorithm used in computer chess programs. A brute-force tree search is embarrassingly easy to parallelize: just put each subtree on a different processor. However, a lot of work gets wasted because the same positions get considered on different machines. Moving from a brute-force search to the more clever alpha-beta pruning algorithm can easily save 99.99% *of the work, thus dwarfing any benefits of a parallel brute-force search. Alpha-beta can be parallelized, but not easily, and the speedups grow surprisingly slowly as a function of the number of processors you have.*

- *Parallel algorithms are tough to debug - Unless your problem can be decomposed into several independent jobs, the different processors must communicate with each other to end up with the correct final result. Unfortunately, the non-deterministic nature of this communication makes parallel* *programs notoriously difficult to debug, because you will get different results each time you run the code. Data parallel programs typically have no communication except copying the results at the end, which makes things much simpler.*

I recommend considering parallel processing only after attempts at solving a problem sequentially prove too slow. Even then, I would restrict attention to data parallel algorithms where no communication is needed between the processors, except to collect the final results. Such large-grain, naive parallelism

can be simple enough to be both implementable and debuggable, because it really reduces to producing a good sequential implementation. There can be pitfalls even in this approach, however, as shown by the following war story.

8.8 War Story: Going Nowhere Fast

In Section 2.9 (page 54), I related our efforts to build a fast program to test Waring's conjecture for pyramidal numbers. At that point, my code was fast enough that it could complete the job in a few weeks running in the background of a desktop workstation. This option did not appeal to my supercomputing colleague, however.

"Why don't we do it in parallel?" he suggested. "After all, you have an outer loop doing the same calculation on each integer from 1 to $1,000,000,000$*. I can split this range of numbers into different intervals and run each range on a different processor. Divide and conquer. Watch, it will be easy."*

He set to work trying to do our computations on an Intel IPSC-860 hypercube using 32 nodes with 16 megabytes of memory per node - very big iron for the time. However, instead of getting answers, over the next few weeks I was treated to a regular stream of e-mail about system reliability:

- *"Our code is running fine, except one processor died last night. I will rerun."*
- *"This time the machine was rebooted by accident, so our long-standing job was killed."*
- *"We have another problem. The policy on using our machine is that nobody can command the entire machine for more than 13 hours, under any condition."*

Still, eventually, he rose to the challenge. Waiting until the machine was stable, he locked out 16 processors (half the computer), divided the integers from 1 to $1,000,000,000$ *into 16 equal-sized intervals, and ran each interval on its own processor. He spent the next day fending off angry users who couldn't get their work done because of our rogue job. The instant the first processor completed analyzing the numbers from 1 to* $62,500,000$*, he announced to all the people yelling at him that the rest of the processors would soon follow.*

But they didn't. He failed to realize that the time to test each integer increased as the numbers got larger. After all, it would take longer to test whether $1,000,000,000$ *could be expressed as the sum of three pyramidal numbers than it would for 100. Thus, at longer and longer intervals, each new processor would announce its completion. Because of the architecture of the hypercube, he couldn't return any of the processors until our entire job was completed. Eventually, half the machine and most of its users were held hostage by one, final interval.*

What conclusions can be drawn from this? If you are going to parallelize a problem, be sure to balance the load carefully among the processors. Proper

load balancing, using either back-of-the-envelope calculations or the partition algorithm we will develop in Section 10.7 (page 333), would have significantly reduced the length of time we needed the machine, and his exposure to the wrath of his colleagues.

8.9 Convolution (*)

The convolution of two arrays (or vectors) A and B is a new vector C such that

$$C[k] = \sum_{j=0}^{m-1} A[j] \cdot B[k-j]$$

If we assume that A and B are of length m and n respectively, and indexed starting from 0 , the natural range on C is from $C[0]$ to $C[n+m-2]$. The values of all out-of-range elements of A and B are interpreted as zero, so they do not contribute to any product.

An example of convolution that you are familiar with is polynomial multiplication. Recall the problem of multiplying two polynomials, for example:

$$\begin{aligned}\left(3x^2+2x+6\right) \times \left(4x^2+3x+2\right) &= (3\cdot 4)x^4 + (3\cdot 3 + 2\cdot 4)x^3 \\ &+ (3\cdot 2 + 2\cdot 3 + 6\cdot 4)x^2 + (2\cdot 2 + 6\cdot 3)x^1 + (6\cdot 2)x^0\end{aligned}$$

Let $A[i]$ and $B[i]$ denote the coefficients of x^i in each of the polynomials. Then multiplication is a convolution, because the coefficient of the x^k term in the product polynomial is given by the convolution $C[k]$ above. This coefficient is the sum of the products of all terms which have exponent pairs adding to k : for example, $x^5 = x^4 \cdot x^1 = x^3 \cdot x^2$.

The obvious way to implement convolution is by computing the m term dot product $C[k]$ for each $0 \leq k \leq n+m-2$. This is two nested loops, running in $\Theta(nm)$ time. The inner loop does not always involve m iterations because of boundary conditions. Simpler loop bounds could have been employed if A and B were flanked by ranges of zeros.

```
for (i = 0; i < n+m-1; i++) {
    for (j = max(0,i-(n-1)); j <= min(m-1,i); j++) {
        c[i] = c[i] + a[j] * b[i-j];
    }
}
```

Convolution multiplies every possible pair of elements from A and B, and hence it seems like we should require quadratic time to get these $n+m-1$ numbers. But in a miracle akin to sorting, there exists a clever divide-andconquer algorithm that runs in $O(n \log n)$ time, assuming that $n \geq m$. And just like sorting, there are a large number of applications that take advantage of this enormous speedup for large sequences.

Figure 8.5: Convolution of strings becomes equivalent to string matching when the pattern is reversed.

8.9.1 Applications of Convolution

Going from $O\left(n^2\right)$ to $O(n \log n)$ is as big a win for convolution as it was for sorting. Taking advantage of it requires recognizing when you are doing a convolution operation. Convolutions often arise when you are trying all possible ways of doing things that add up to k, for a large range of values of k, or when sliding a mask or pattern A over a sequence B and calculating at each position.

Important examples of convolution operations include:

- *Integer multiplication: We can interpret integers as polynomials in any base b. For example, $632 = 6 \cdot b^2 + 3 \cdot b^1 + 2 \cdot b^0$, where $b = 10$. Polynomial multiplication behaves like integer multiplication without carrying.*
  There are two different ways we can use fast polynomial multiplication to deal with integers. First, we can explicitly perform the carrying operation on the product polynomial, adding $\lfloor C[i]/b \rfloor$ to $C[i+1]$, and then replacing $C[i]$ with $C[i] (\bmod b)$. Alternatively, we could compute the product polynomial and then evaluate it at b to get the integer product $A \times B$.
  With fast convolution, either way gives us an even faster multiplication algorithm than Karatsuba, running in $O(n \log n)$ time on a RAM model of computation.

- *Cross-correlation: For two time series A and B, the cross-correlation function measures the similarity as a function of the shift or displacement of one relative to the other. Perhaps people buy a product on average k*

days after seeing an advertisement for it. Then there should be high correlation between sales and advertising expenditures lagged by k days. This cross-correlation function $C[k]$ can be computed:

$$C[k] = \sum_j A[j]B[j+k]$$

Note that the dot product here is computed over backward shifts of B instead of forward shifts, as in the original definition of a convolution. But we can still use fast convolution to compute this: simply input the reversed sequence B^R instead of B.

- *Moving average filters: Often we are tasked with smoothing time series data by averaging over a window. Perhaps we want $C[i-1] = 0.25B[i-1] + 0.5B[i] + 0.25B[i+1]$ over all positions i. This is just another convolution, where A is the vector of weights within the window around $B[i]$.*

- *String matching: Recall the problem of substring pattern matching, first discussed in Section 2.5.3. We are given a text string S and a pattern string P, and seek to identify all locations in P where P may be found. For $S = abaababa$ and $P = aba$, we can find P in S starting at positions 0,3 , and 5 .*

  The $O(mn)$ algorithm described in Section 2.5.3 works by sliding the length $-m$ pattern over each of the n possible starting points in the text. This sliding window approach is suggestive of being a convolution with the reversed pattern P^R, as shown in Figure 5.5 Can we solve string matching in $O(n \log n)$ by using fast convolution?
  The answer is yes! Suppose our strings have an alphabet of size α. We can represent each character by a binary vector of length α having exactly one non-zero bit. Say $a = 10$ and $b = 01$ for the alphabet $\{a, b\}$. Then we can encode the strings S and P above as

$$S = 1001101001100110$$
$$P = 100110$$

The dot product over a window will be m on an even-numbered position of s iff p starts at that position in the text. So fast convolution can identify all locations of p in s in $O(n \log n)$ time.

Take-Home Lesson: Learn to recognize possible convolutions. A magical $\Theta(n \log n)$ algorithm instead of $O\left(n^2\right)$ is your reward for seeing this.

8.9.2 Fast Polynomial Multiplication (**)

The applications above should whet our interest in efficient ways to compute convolutions. The fast convolution algorithm uses divide and conquer, but a *detailed proof of correctness relies on fairly sophisticated properties of complex numbers and linear algebra that are beyond the scope of what I want to do here. Feel free to skip ahead! But I will provide enough of an overview for you to understand the divide and conquer part.*

We present convolution through a fast algorithm for multiplying polynomials. It is based on a series of observations:

- *Polynomials can be represented either as equations or sets of points: You know that every pair of points defines a line. More generally, any degree-n polynomial $P(x)$ is completely defined by $n+1$ points on the polynomial. For example, the points $(-1,-2),(0,-1)$, and $(1,2)$ define (and are defined by) the quadratic equation $y=x^2+2x-1$.*

- *We can find $n+1$ such points on $P(x)$ by evaluation, but it looks expensive: Generating a point on a given polynomial is easy-simply pick an arbitrary value x and plug it into $P(x)$. The time it takes for one such x will be linear*
  in the degree of $P(x)$, which means n for the problems we are interested in. But doing this $n+1$ times for different values of x would take $O\left(n^2\right)$ time, which is more than we can afford if we want fast multiplication.

- *Multiplying polynomials A and B in a points representation is easy, if they have both been evaluated on the same values of x : Suppose we want to compute the product of $\left(3x^2+2x+6\right)\left(4x^2+3x+2\right)$. The result will be a degree- 4 polynomial, so we need five points to define it. We can evaluate both factors on the same x values:*

$$\begin{aligned}
A(x) &= 3x^2+2x+6 \longrightarrow (-2,14),(-1,7),(0,6),(1,11),(2,22) \\
B(x) &= 4x^2+3x+2 \longrightarrow (-2,12),(-1,3),(0,2),(1,9),(2,24)
\end{aligned}$$

Since $C(x)=A(x)B(x)$, we can now construct points on $C(x)$ by multiplying the corresponding y-values:

$$C(x) \longleftarrow (-2,168),(-1,21),(0,12),(1,99),(2,528)$$

Thus, multiplying points in this representation takes only linear time.

- *We can evaluate a degree-n polynomial $A(x)$ as two degree-$(n/2)$ polynomials in x^2 : We can partition the terms of A into those of even and odd degree, for example:*

$$12x^4+17x^3+36x^2+22x+12=\left(12x^4+36x^2+12\right)+x\left(17x^2+22\right)$$

By replacing x^2 by x', the right side gives us two smaller, lower degree polynomials as promised.

- *This suggests an efficient divide-and-conquer algorithm: We need to evaluate n points of a degree- d polynomial. We need $n \geq 2d+1$ points, since we will be using them to compute the product of two polynomials. We can decompose the problem into doing this evaluation on two polynomials of half the degree, plus a linear amount of work stitching the results together. This defines the recurrence $T(n) = 2T(n/2) + O(n)$, which evaluates to $O(n \log n)$.*

- *Making this work correctly requires picking the right x values to evaluate on: The trick with the squares makes it desirable for our sample points to come in pairs of the form $\pm x$, since their evaluation requires half as much work because they are identical when squared.*

However, this property does not hold recursively, unless the x values are carefully chosen complex numbers. The nth roots of unity are the set of solutions to the equation $x^n = 1$. In reals, we only get $x \in \{-1, 1\}$, but there are n solutions with complex numbers. The k th of these n roots is given by

$$w_k = \cos(2k\pi/n) + i\sin(2k\pi/n)$$

To appreciate the magic properties of these numbers, look at what happens when we raise them to powers:

$$\begin{aligned} w &= \left\{1, \frac{1+i}{\sqrt{2}}, i, -\frac{1-i}{\sqrt{2}}, -1, -\frac{1+i}{\sqrt{2}}, -i, \frac{1-i}{\sqrt{2}}\right\} \\ w^2 &= \{1, i, -1, -i, 1, i, -1, -i\} \\ w^4 &= \{1, -1, 1, -1, 1, -1, 1, -1\} \\ w^8 &= \{1, 1, 1, 1, 1, 1, 1, 1\} \end{aligned}$$

Observe that these terms come in positive/negative pairs, and the number of distinct terms gets halved with each squaring. These are the properties we need to make the divide and conquer work.

The best implementations of fast convolution generally compute the fast Fourier transform (FFT), so usually we seek to reduce our problems to FFTs to take advantage of existing libraries. FFTs are discussed in Section 16.11 (page 501.

Take-Home Lesson: Fast convolution solves many important problems in $O(n \log n)$. The first step is to recognize your problem is a convolution.

Chapter Notes *Several other algorithms texts provide more substantive coverage of divide-and-conquer algorithms, including Kleinberg and Tardos citekleinberg2006algorithm and Manber citemanber1989introduction. See Cormen et al. citecormen2009introduction] for an excellent overview of the master theorem.*

See Skiena citeskiena2012redesigning for an accessible introduction to algorithmic design of vaccines. The bug searching sequences described in Section 5.2 is an example

of a pooling design, enabling the identification of (say) one sick patient out of n *using only* $\lg n$ *blood tests on pooled samples. The theory of these interesting designs is surveyed by Du and Hwang [*
citedu2000combinatorial. The left-right order of the subsets on these designs reflects a Gray code, in which neighboring subsets differ in exactly one element. Gray codes are discussed in Section 17.5

Our parallel computations on pyramidal numbers were reported in Deng and Yang
citedeng1994warings. My treatment of convolutions and the FFT was based on Avrim Blum's 15-451/651 algorithm lecture notes.

8.10 Exercises

Binary Search 5-1. [3] Suppose you are given a sorted array A *of size* n *that has been circularly shifted* k *positions to the right. For example,* $[35, 42, 5, 15, 27, 29]$ *is a sorted array that has been circularly shifted* $k = 2$ *positions, while* $[27, 29, 35, 42, 5, 15]$ *has been shifted* $k = 4$ *positions.*

- *Suppose you know what* k *is. Give an* $O(1)$ *algorithm to find the largest number in* A.

- *Suppose you do not know what* k *is. Give an* $O(\lg n)$ *algorithm to find the largest number in* A. *For partial credit, you may give an* $O(n)$ *algorithm.* $5-2$. *[3] A sorted array of size* n *contains distinct integers between 1 and* $n+1$, *with one element missing. Give an* $O(\log n)$ *algorithm to find the missing integer, without using any extra space.*
  5-3. [3] Consider the numerical Twenty Questions game. In this game, the first player thinks of a number in the range 1 to n. *The second player has to figure out this number by asking the fewest number of true/false questions. Assume that nobody cheats.*
  (a) What is an optimal strategy if n *in known?*
  (b) What is a good strategy if n *is not known?*

5-4. [5] You are given a unimodal array of n *distinct elements, meaning that its entries are in increasing order up until its maximum element, after which its elements are in decreasing order. Give an algorithm to compute the maximum element of a unimodal array that runs in* $O(\log n)$ *time.*
5-5. [5] Suppose that you are given a sorted sequence of distinct integers $[a_1, a_2, \ldots, a_n]$. *Give an* $O(\lg n)$ *algorithm to determine whether there exists an index* i *such that* $a_i = i$. *For example, in* $[-10, -3, 3, 5, 7], a_3 = 3$. *In* $[2, 3, 4, 5, 6, 7]$, *there is no such* i.
5-6. [5] Suppose that you are given a sorted sequence of distinct integers $a = [a_1, a_2, \ldots, a_n]$, *drawn from 1 to* m *where* $n < m$. *Give an* $O(\lg n)$ *algorithm to find an integer* $\leq m$ *that is not present in* a. *For full credit, find the smallest such integer* x *such that* $1 \leq x \leq m$.
5-7. [5] Let M *be an* $n \times m$ *integer matrix in which the entries of each row are*

sorted in increasing order (from left to right) and the entries in each column are in increasing order (from top to bottom). Give an efficient algorithm to find the position of an integer x in M, or to determine that x is not there. How many comparisons of x with matrix entries does your algorithm use in worst case?

Divide and Conquer Algorithms 5-8. [5] Given two sorted arrays A and B of size n and m respectively, find the median of the $n+m$ elements. The overall run time complexity should be $O(\log(m+n))$.

5-9. [8] The largest subrange problem, discussed in Section 5.6 takes an array A of n numbers, and asks for the index pair i and j that maximizes $S = \sum_{k=i}^{j} A[k]$. Give an $O(n)$ algorithm for largest subrange.

5-10. [8] We are given n wooden sticks, each of integer length, where the i th piece has length $L[i]$. We seek to cut them so that we end up with k pieces of exactly the same length, in addition to other fragments. Furthermore, we want these k pieces to be as large as possible.

(a) Given four wood sticks, of lengths $L = \{10, 6, 5, 3\}$, what are the largest sized pieces you can get for $k = 4$? (Hint: the answer is not 3).

(b) Give a correct and efficient algorithm that, for a given L and k, returns the maximum possible length of the k equal pieces cut from the initial n sticks.

5-11. [8] Extend the convolution-based string-matching algorithm described in the text to the case of pattern matching with wildcard characters "", which match any character. For example, "sht" should match both "shot" and "shut".

Recurrence Relations 5-12. [5] In Section 5.3 it is asserted that any polynomial can be represented by a recurrence. Find a recurrence relation that represents the polynomial $a_n = n^2$.

5-13. [5] Suppose you are choosing between the following three algorithms:

- *Algorithm A solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.*
- *Algorithm B solves problems of size n by recursively solving two subproblems of size $n-1$ and then combining the solutions in constant time.*
- *Algorithm C solves problems of size n by dividing them into nine subproblems of size $n/3$, recursively solving each subproblem, and then combining the solutions in $\Theta\left(n^2\right)$ time.*

What are the running times of each of these algorithms (in big O notation), and which would you choose?

5-14. [5] Solve the following recurrence relations and give a Θ bound for each of them:

(a) $T(n) = 2T(n/3) + 1$

(b) $T(n) = 5T(n/4) + n$

(c) $T(n) = 7T(n/7) + n$

(d) $T(n) = 9T(n/3) + n^2$

5-15. [3] Use the master theorem to solve the following recurrence relations:

(a) $T(n) = 64T(n/4) + n^4$

(b) $T(n) = 64T(n/4) + n^3$
(c) $T(n) = 64T(n/4) + 128$

5-16. [3] Give asymptotically tight upper (Big Oh) bounds for $T(n)$ *in each of the following recurrences. Justify your solutions by naming the particular case of the master theorem, by iterating the recurrence, or by using the substitution method:*
(a) $T(n) = T(n-2) + 1$.
(b) $T(n) = 2T(n/2) + n\lg^2 n$.
(c) $T(n) = 9T(n/4) + n^2$.

LeetCode *5-1. https://leetcode.com/problems/median-of-two-sorted-arrays/*
5-2. https://leetcode.com/problems/count-of-range-sum/
5-3. https://leetcode.com/problems/maximum-subarray/

HackerRank *5-1. https://www.hackerrank.com/challenges/unique-divide-and-conquer*
5-2. https://www.hackerrank.com/challenges/kingdom-division/
5-3. https://www.hackerrank.com/challenges/repeat-k-sums/

Programming Challenges *These programming challenge problems with robot judging are available at https://onlinejudge.org:*

5-1. "Polynomial Coefficients"-Chapter 5, problem 10105.
5-2. "Counting"-Chapter 6, problem 10198.
5-3. "Closest Pair Problem"-Chapter 14, problem 10245.

Chapter 6

Chapter 9

Hashing and Randomized Algorithms

Most of the algorithms discussed in previous chapters were designed to optimize worst-case performance: they are guaranteed to return optimal solutions for every problem instance within a specified running time.

This is great, when we can do it. But relaxing the demand for either always correct or always efficient can lead to useful algorithms that still have performance guarantees. Randomized algorithms are not merely heuristics: any bad performance is due to getting unlucky on coin flips, rather than adversarial input data.

We classify randomized algorithms into two types, depending upon whether they guarantee correctness or efficiency:

- *Las Vegas algorithms: These randomized algorithms guarantee correctness, and are usually (but not always) efficient. Quicksort is an excellent example of a Las Vegas algorithm.*
- *Monte Carlo algorithms: These randomized algorithms are provably efficient, and usually (but not always) produce the correct answer or something close to it. Representative of this class are random sampling methods discussed in Section 12.6.1, where we return the best solution found in the course of (say)* $1,000,000$ *random samples.*

We will see several examples of both types of algorithm in this chapter.
One blessing of randomized algorithms is that they tend to be very simple to describe and implement. Eliminating the need to worry about rare or unlikely situations makes it possible to avoid complicated data structures and other contortions. These clean randomized algorithms are often intuitively appealing, and relatively easy to design.

However, randomized algorithms are frequently quite difficult to analyze rigorously. Probability theory is the mathematics we need for the analysis of randomized algorithms, and is of necessity both formal and subtle. Probabilistic *analysis often involves algebraic manipulation of long chains of inequalities that looks frightening, and relies on tricks and experience.*

This makes it difficult to provide satisfying analysis on the level of this book, which maintains a strict no-theorem/proof policy. But I will try to provide intuition where I can, so you can appreciate why these algorithms are usually correct or efficient.

We have had initial peeks at randomized algorithms through our discussions of hash tables (Section 3.7) and quicksort (Section 4.6). Review these now to give yourself the best chance of understanding what is to come.

Stop and Think: Quicksort City *Problem: Why is randomized quicksort a Las Vegas algorithm, as opposed to a Monte Carlo algorithm?*

Solution: Recall that Monte Carlo algorithms are always fast and usually correct, while Las Vegas algorithms are always correct and usually fast.

Randomized quicksort always produces a sorted permutation, so we know it is always correct. Picking a very bad series of pivots might cause the running to exceed $O(n \log n)$, but we are always going to end up sorted. Thus, quicksort is a nice example of a Las Vegas-style algorithm.

9.1 Probability Review

I will resist the temptation to give a thorough review of probability theory here, as part of my objective to keep this book to a manageable size. My presumptions are: (1) you have had some previous exposure to probability theory, and (2) you know where to look if you feel you need more. I will therefore limit myself to a few basic definitions and properties we will use.

9.1.1 Probability

Probability theory provides a formal framework for reasoning about the likelihood of events. Because it is a formal discipline, there is a thicket of associated definitions to instantiate exactly what we are reasoning about:

- *An experiment is a procedure that yields one of a set of possible outcomes. As our ongoing example, consider the experiment of tossing two six-sided dice, one red and one blue, with each face bearing a distinct integer $\{1, \ldots, 6\}$.*

- *A sample space S is the set of possible outcomes of an experiment. In our dice example, there are thirty-six possible outcomes, namely*

$$S = \{(1,1),(1,2),(1,3),(1,4),(1,5),(1,6),\\ (2,1),(2,2),(2,3),(2,4),(2,5),(2,6),\\ (3,1),(3,2),(3,3),(3,4),(3,5),(3,6),\\ (4,1),(4,2),(4,3),(4,4),(4,5),(4,6),\\ (5,1),(5,2),(5,3),(5,4),(5,5),(5,6),\\ (6,1),(6,2),(6,3),(6,4),(6,5),(6,6)\}.$$

- *An event E is a specified subset of the outcomes of an experiment. The event that the sum of the dice equals 7 or 11 (the conditions to win at craps on the first roll) is the subset*

$$E = \{(1,6),(2,5),(3,4),(4,3),(5,2),(6,1),(5,6),(6,5)\}.$$

- *The probability of an outcome s, denoted $p(s)$, is a number with the two properties:*
- *For each outcome s in sample space $S, 0 \leq p(s) \leq 1$.*
- *The sum of probabilities of all outcomes adds to one: $\sum_{s \in S} p(s) = 1$.*

If we assume two distinct fair dice, the probability $p(s) = (1/6) \times (1/6) = 1/36$ for all outcomes $s \in S$.

- *The probability of an event E is the sum of the probabilities of the outcomes of the event. Thus,*

$$P(E) = \sum_{s \in E} p(s)$$

An alternative formulation is in terms of the complement of the event $\bar{E}$, the case when E does not occur. Then

$$P(E) = 1 - P(\bar{E})$$

This is useful, because often it is easier to analyze $P(\bar{E})$ than $P(E)$ directly.

- *A random variable V is a numerical function on the outcomes of a probability space. The function "sum the values of two dice" ($V((a,b)) = a + b$) produces an integer result between 2 and 12 . This implies a probability distribution of the possible values of the random variable. The probability $P(V(s) = 7) = 1/6$, while $P(V(s) = 12) = 1/36$.*
- *The expected value of a random variable V defined on a sample space S, $E(V)$, is defined*

$$E(V) = \sum_{s \in S} p(s) \cdot V(s)$$

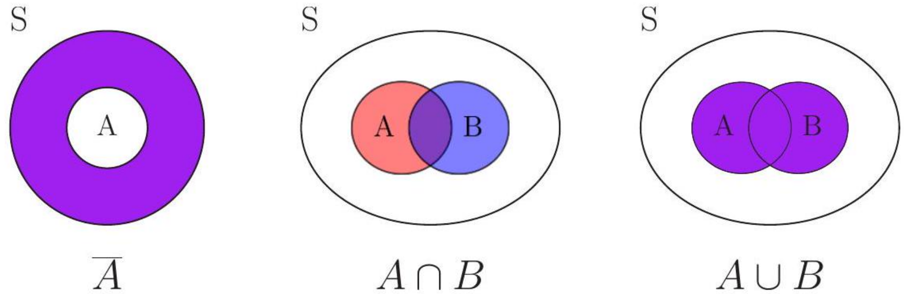


Figure 9.1: Venn diagrams illustrating *set difference* (left), *intersection* (middle), and *union* (right).

9.1.2 Compound Events and Independence

We will be interested in complex events computed from simpler events A and B on the same set of outcomes. Perhaps event A is that at least one of two dice be an even number, while event B denotes rolling a total of either 7 or 11. Note that there exist outcomes of A that are not outcomes of B, specifically

$$\begin{aligned} A - B = \{&(1,2),(1,4),(2,1),(2,2),(2,3),(2,4),(2,6),(3,2),(3,6),(4,1) \\ &(4,2),(4,4),(4,5),(4,6),(5,4),(6,2),(6,3),(6,4),(6,6)\} \end{aligned}$$

This is the set difference operation. Observe that here $B - A = \{\}$, because every pair adding to 7 or 11 must contain one odd and one even number.

The outcomes in common between both events A and B are called the intersection, denoted $A \cap B$. This can be written as

$$A \cap B = A - (S - B)$$

Outcomes that appear in either A or B are called the union, denoted $A \cup B$. The probability of the union and intersection are related by the formula

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

With the complement operation $\bar{A} = S - A$, we get a rich language for combining events, shown in Figure 6.1. We can readily compute the probability of any of these sets by summing the probabilities of the outcomes in the defined sets.

The events A and B are said to be independent if

$$P(A \cap B) = P(A) \times P(B)$$

This means that there is no special structure of outcomes shared between events A and B. Assuming that half of the students in my class are female, and *half the students in my class are above average, we would expect that a quarter of my students are both female and above average if the events are independent.*

Probability theorists love independent events, because it simplifies their calculations. For example, if A_i denotes the event of getting an even number on

the i th dice throw, then the probability of obtaining all evens in a throw of two dice is $P(A_1 \cap A_2) = P(A_1) P(A_2) = (1/2)(1/2) = 1/4$. Then, the probability of A, that at least one of two dice is even, is

$$P(A) = P(A_1 \cup A_2) = P(A_1) + P(A_2) - P(A_1 \cap A_2) = 1/2 + 1/2 - 1/4 = 3/4$$

That independence often doesn't hold explains much of the subtlety and difficulty of probabilistic analysis. The probability of getting n heads when tossing n independent coins is $1/2^n$. But it would be $1/2$ if the coins were perfectly correlated, since the only possibilities would be all heads or all tails. This computation would become very hard if there were complex dependencies between the outcomes of the i th and j th coins.

Randomized algorithms are typically designed around samples drawn independently at random, so that we can safely multiply probabilities to understand compound events.

9.1.3 Conditional Probability

Presuming that $P(B) > 0$, the conditional probability of A given $B, P(A \mid B)$ is defined as follows

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

In particular, if events A and B are independent, then

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A)P(B)}{P(B)} = P(A)$$

and B has absolutely no impact on the likelihood of A. Conditional probability becomes interesting only when the two events have dependence on each other.

Recall the dice-rolling events from Section 6.1.2 namely:

- *Event A : at least one of two dice is an even number.*
- *Event B : the sum of the two dice is either 7 or 11 .*

Observe that $P(A \mid B) = 1$, because any roll summing to an odd value must consist of one even and one odd number. Thus, $A \cap B = B$. For $P(B \mid A)$, note that $P(A \cap B) = P(B) = 8/36$ and $P(A) = 27/36$, so $P(B \mid A) = 8/27$.

Our primary tool to compute conditional probabilities will be Bayes' theorem, which reverses the direction of the dependencies:

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)}$$

Often it proves easier to compute probabilities in one direction than another, as in this problem. By Bayes' theorem $P(B \mid A) = (1 \cdot 8/36)/(27/36) = 8/27$, *exactly what we got before.*

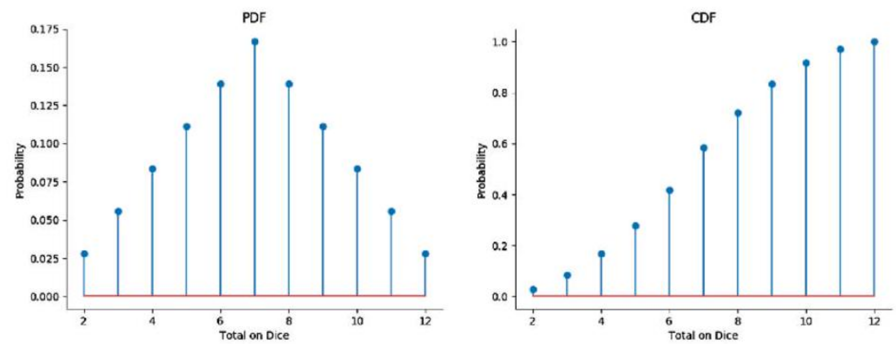


Figure 9.2: The *probability density function* (pdf) of the sum of two dice contains exactly the same information as the cumulative density function (cdf), but looks very different.

9.1.4 Probability Distributions

Random variables are numerical functions where the values are associated with probabilities of occurrence. In our example where $V(s)$ *is the sum of two tossed dice, the function produces an integer between 2 and 12 . The probability of a particular value* $V(s) = X$ *is the sum of the probabilities of all the outcomes whose components add up to* X.

Such random variables can be represented by their probability density function, or pdf. This is a graph where the x*-axis represents the values the random variable can take on, and the* y*-axis denotes the probability of each given value. Figure 6.2 (left) presents the pdf of the sum of two fair dice. Observe that the peak at* $X = 7$ *corresponds to the most probable dice total, with a probability of* $1/6$.

Mean and Variance There are two main types of summary statistics, which together tell us an enormous amount about a probability distribution or a data set:

- *Central tendency measures, which capture the center around which the random samples or data points are distributed.*

- *Variation or variability measures, which describe the spread, that is, how far the random samples or data points can lie from the center.*

The primary centrality measure is the mean. The mean of a random variable V, denoted $E(V)$ and also known as the expected value, is given by

$$E(V) = \sum_{s \in S} V(s)p(s)$$

When the elementary events are all of equal probability, the mean or average, computed as

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

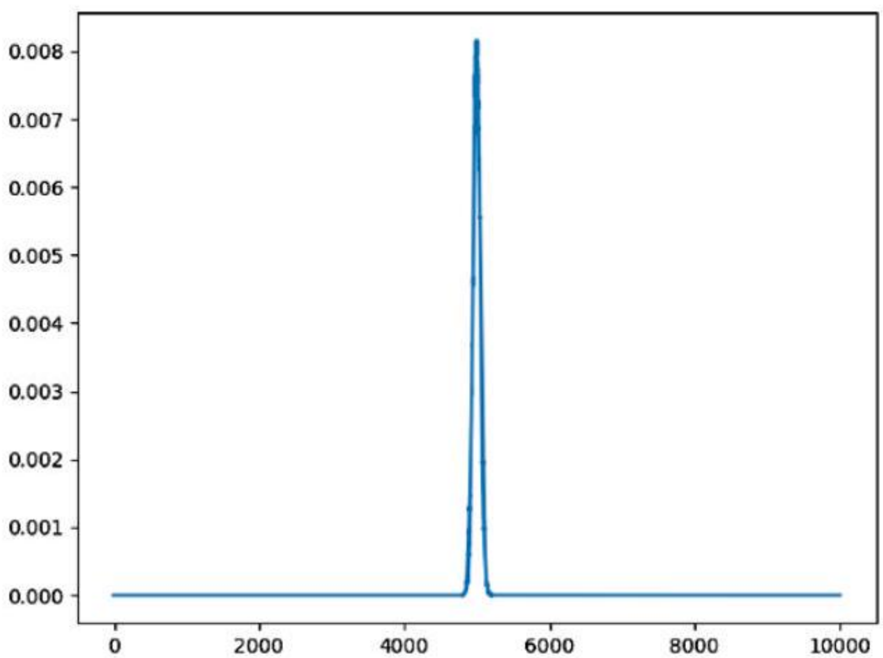


Figure 9.3: The probability distribution of getting h heads in $n = 10,000$ tosses of a fair coin is tightly centered around the mean, $n/2 = 5,000$.

The most common measure of variability is the standard deviation σ. The standard deviation of a random variable V is given by $\sigma = \sqrt{E\left((V - E(V))^2\right.}$. For a data set, the standard deviation is computed from the sum of squared differences between the individual elements and the mean:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n}\left(x_i - \bar{X}\right)^2}{n-1}}$$

A related statistic, the variance $V = \sigma^2$, is the square of the standard deviation. Sometimes it is more convenient to talk about variance than standard deviation, because the term is ten characters shorter. But they measure exactly the same thing.

9.1.5 Tossing Coins

You probably have a fair degree of intuition about the distribution of the number of heads and tails when you toss a fair coin 10,000 times. You know that the expected number of heads in n tosses, each with probability $p = 1/2$ of heads, is pn, or 5,000 for this example. You likely know that the distribution for h heads out of n is a binomial distribution, where

$$P(X - h) = \frac{\binom{n}{h}}{\sum_{x=0}^{n}\binom{n}{x}} = \frac{\binom{n}{h}}{2^n}$$

and that it is a bell-shaped symmetrical distribution about the mean. But you may not appreciate just how narrow this distribution is, as shown in Figure 6.3. Sure, anywhere from 0 to n heads can result from n fair coin tosses. But they won't: the number of heads we get will almost always be within a few standard deviations of the mean, where the standard deviation σ for the binomial distribution is given by $\sigma = \sqrt{np(1-p)} = \Theta(\sqrt{n})$.

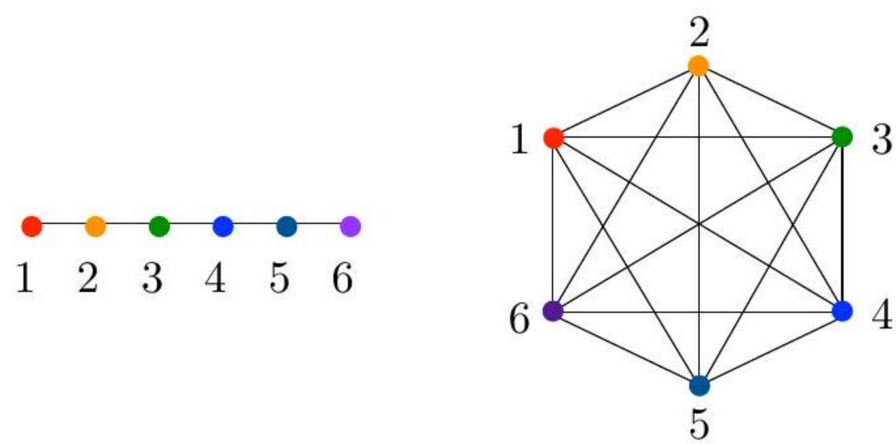


Figure 9.4: How long does a random walk take to visit *all n nodes of a path (left) or a complete graph (right)?*

Indeed, for any probability distribution, at least $1-\left(1/k^2\right)$ of the mass of the distribution lies within $\pm k\sigma$ of the mean μ. Typically σ is small relative to μ for the distributions arising in the analysis of randomized algorithms and processes.

Take-Home Lesson: Students often ask me "what happens" when randomized quicksort runs in $\Theta\left(n^2\right)$. The answer is that nothing happens, in exactly the same way nothing happens when you buy a lottery ticket: you almost certainly just lose. With a randomized quicksort you almost certainly just win: the probability distribution is so tight that you nearly always run in time very close to expectation.

Stop and Think: Random Walks on a Path *Problem: Random walks on graphs are important processes to understand. The expected covering time (the number of moves until we have visited all vertices) differs depending upon the topology of the graph (see Figure 6.4). What is it for a path?*

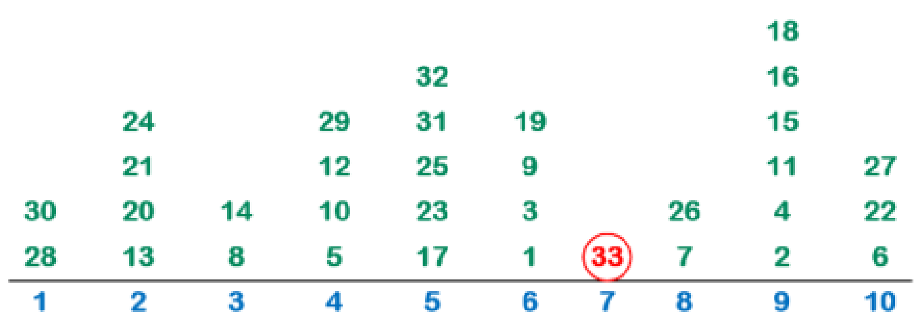
Suppose we start at the left end of an m-vertex path. We repeatedly flip a fair coin, moving one step right on heads and one step left on tails (staying where we are if we were to fall off the path). How many coin flips do we expect it will take, as a function of m, until we get to the right end of the path?

Solution: To get to the right end of the path, we need $m-1$ more heads than tails after n coin flips, assuming we don't bother to flip when on the left-most vertex where we can only move right. We expect about half of the flips to be heads, with a standard deviation of $\sigma=\Theta(\sqrt{n})$. This σ describes the spread of the difference in the number of the heads and tails we are likely to have. We must flip enough times for σ to be on the order of m, so

$$m=\Theta(\sqrt{n}) \rightarrow n=\Theta\left(m^2\right)$$

9.2 Understanding Balls and Bins

Ball and bin problems are classics of probability theory. We are given x identical balls to toss at random into y labeled bins. We are interested in the resulting distribution of balls. How many bins are expected to contain a given number of balls?



Hashing can be thought of as a ball and bin process. Suppose we hash n

balls/keys into n bins/buckets. We expect an average of one ball per bin, but note that this will be true regardless of how good or bad the hash function is.

A good hash function should behave like a random number generator, selecting integers/bin IDs with equal probability from 1 to n. But what happens when we draw n such integers from a uniform distribution? The ideal would be for each of the n items (balls) to be assigned to a different bin, so that each bucket contains exactly one item to search. But is this really what happens?

To help develop your own intuition, I encourage you to code a little simulation and run your own experiment. I did, and got the following results, for hash table sizes from one million to one hundred million items:

	Number of Buckets with k Items		
k	$n = 10^6$	$n = 10^7$	$n = 10^8$
0	*367,899*	$3,678,774$	$36,789,634$
1	*367,928*	$3,677,993$	$36,785,705$
2	*183,926*	$1,840,437$	$18,392,948$
3	*61,112*	*613,564*	$6,133,955$
4	*15,438*	*152,713*	$1,531,360$
5	*3130*	*30,517*	*306,819*
6	*499*	*5,133*	*51,238*
7	*56*	*754*	*7,269*
8	*12*	*107*	*972*
9		*8*	*89*
10			*10*
11			*1*

We see that 36.78% *of the buckets are empty in all three cases. That can't be a coincidence. The first bucket will be empty iff each of the n balls gets assigned to one of the other $n-1$ buckets. The probability p of missing for each particular ball is $p = (n-1)/n$, which approaches 1 as n gets large. But we must miss for all n balls, the probability of which is p^n. What happens when we multiply a large number of large probabilities? You actually saw the answer back when you studied limits:*

$$P(|B_1| = 0) = \left(\frac{n-1}{n}\right)^n \rightarrow \frac{1}{e} = 0.367879$$

Thus, 36.78% *of the buckets in a large hash table will be empty. And, as it turns out, exactly the same fraction of buckets is expected to contain one element.*

If so many buckets are empty, others must be unusually full. The fullest bucket gets fuller in the table above as n increases, from 8 to 9 to 11 . In fact, the expected value of the longest list is $O(\log n / \log \log n)$, which grows slowly but is not a constant. Thus, I was a little too glib when I said in Section 3.7.1

								18	
			32			16			
	24		29	31	19			15	
	21		12	25	9			11	27
30	20	14	10	23	3		26	4	22
28	13	8	5	17	1	33	7	2	6
1	2	3	4	5	6	7	8	9	10

Figure 6.5: Illustrating the coupon collectors problem through an experiment tossing balls at random into ten bins. It is not until the 33rd toss that all bins are non-empty.

that the worst-case access time for hashing is $O(1)$[1]

Take-Home Lesson: Precise analysis of random process requires formal probability theory, algebraic skills, and careful asymptotics. We will gloss over such issues in this chapter, but you should appreciate that they are out there.

9.2.1 The Coupon Collector's Problem

As a final hashing warmup, let's keep tossing balls into these n bins until none of them are empty, that is, until we have at least one ball in each bin. How many tosses do we expect this should take? As shown in Figure 6.5 it may require considerably more than n tosses until every bin is occupied.

We can split such a sequence of balls into n runs, where run r_i consists of the balls we toss after we have filled i buckets until the next time we hit an empty bucket. The expected number of balls to fill all n slots $E(n)$ will be the sum of the expected lengths of all runs. If you are flipping a coin with probability p of coming up heads, the expected number of flips until you get your first head is $1/p$; this is a property of the geometric distribution. After filling i buckets, the probability that our next toss will hit an empty bucket is $p = (n-i)/n$. Putting this together, we get the following

$$E(n) = \sum_{i=0}^{n-1} |r_i| = \sum_{i=0}^{n-1} \frac{n}{n-i} = n \sum_{i=0}^{n-1} \frac{1}{n-i} = nH_n \approx n \ln n$$

The trick here is to remember that the harmonic number $H_n = \sum_{i=1}^{n} 1/i$, and that $H_n \approx \ln n$.

***Stop and Think: Covering Time for K_n** Problem: Suppose we start on vertex 1 of a complete n-vertex graph (see Figure 6.4. We take a random walk on this graph, at each step going to a randomly*

selected neighbor of our current position. What is the expected number of steps until we have visited all vertices of the graph?

Solution: This is exactly the same question as the previous stop and think, but with a different graph and thus with a possibly different answer.

[1] To be precise, the expected search time for hashing is $O(1)$ averaged over all n keys, but we also expect there will a few keys unlucky enough to require $\Theta(\log n/\log\log n)$ time.

Indeed, the random process here of independently generating random integers from 1 to n looks essentially the same as the coupon collector's problem. This suggests that the expected length of the covering walk is $\Theta(n \log n)$.

The only hitch in this argument is that the random walk model does not permit us to stay at the same vertex for two successive steps, unless the graph has edges from a vertex to itself (self-loops). A graph without such self-loops should have a slightly faster covering time, since a repeat visit does not make progress in any way, but not enough to change the asymptotics. The probability of discovering one of $n-i$ untouched vertices in the next step changes from $(n-i)/n$ to $(n-i)/(n-1)$, reducing the total covering time analysis from nH_n to $(n-1)H_n$. But these are asymptotically the same. Covering the complete graph takes $\Theta(n \log n)$ steps, much faster than the covering time of the path.

9.3 Why is Hashing a Randomized Algorithm?

Recall that a hash function $h(s)$ maps keys s to integers in a range from 0 to $m-1$, ideally uniformly over this interval. Because good hash functions scatter keys around this integer range in a manner similar to that of a uniform random number generator, we can analyze hashing by treating the values as drawn from tosses of an m-sided die.

But just because we can analyze hashing in terms of probabilities doesn't make it a randomized algorithm. As discussed so far, hashing is completely deterministic, involving no random numbers. Indeed, hashing must be deterministic, because we need $h(x)$ to produce exactly the same result whenever called with a given x, or else we can never hope to find x in a hash table.

One reason we like randomized algorithms is that they make the worst case input instance go away: bad performance should be a result of extremely bad luck, rather than some joker giving us data that makes us do bad things. But it is easy (in principle) to construct a worst case example for any hash function h. Suppose we take an arbitrary set S of nm distinct keys, and hash each $s \in S$. Because the range of this function has only m elements, there must be many collisions. Since the average number of items per bucket is $nm/m = n$, it follows from the pigeonhole principle that there must be a bucket with at least n items in it. The n items in this bucket, taken by themselves, will prove a nightmare for hash function h.

How can we make such a worst-case input go away? We are protected if we pick our hash function at random from a large set of possibilities, because we

can only construct such a bad example by knowing the exact hash function we will be working with.

So how can we construct a family of random hash functions? Recall that typically $h(x) = f(x)(\mathrm{mod} m)$, where $f(x)$ turns the key into a huge value, and taking the remainder mod m reduces it to the desired range. Our desired range is typically determined by application and memory constraints, so we would not want to select m at random. But what about first reducing with a larger integer *p ? Observe that in general*

$$f(x)(\text{mod} m) \neq (f(x) \bmod p)(\text{mod} m)$$

For example:
$21347895537127(\text{mod}17) = 8 \neq (21347895537127(\text{mod}2342343))(\text{mod}17) = 12$
Thus, we can select p at random to define the hash function

$$h(x) = ((f(x) \bmod p) \bmod m)$$

and things will work out just fine provided (a) $f(x)$ is large relative to p, (b) p is large relative to m, and (c) m is relatively prime to p.

This ability to select random hash functions means we can now use hashing to provide legitimate randomized guarantees, thus making the worst-case input go away. It also lets us build powerful algorithms involving multiple hash functions, such as Bloom filters, discussed in Section 6.4

9.4 Bloom Filters

Recall the problem of detecting duplicate documents faced by search engines like Google. They seek to build an index of all the unique documents on the web. Identical copies of the same document often exist on many different websites, including (unfortunately) pirated copies of my book. Whenever Google crawls a new link, they need to establish whether what they found is a not previously encountered document worth adding to the index.

Perhaps the most natural solution here is to build a hash table of the documents. Should a freshly crawled document hash to an empty bucket, we know it must be new. But when there is a collision, it does not necessarily mean we have seen this document before. To be sure, we must explicitly compare the new document against all other documents in the bucket, to detect spurious collisions between a and b, where $h(a) = s$ and $h(b) = s$, but $a \neq b$. This is what was discussed back in Section 3.7.2

But in this application, spurious collisions are not really a tragedy: they only mean that Google will fail to index a new document it has found. This can be an acceptable risk, provided the probability of it happening is low enough. Removing the need to explicitly resolve collisions has big benefits in making the table smaller. By reducing each bucket from a pointer link to a single bit (is this bucket occupied or not?), we reduce the space by a factor of 64 on typical

machines. Some of this space can then be taken back to make the hash table larger, thus reducing the probability of collisions in the first place.

Now suppose we build such a bit-vector hash table, with a capacity of n bits. If we have distinct bits corresponding to m documents occupied, the probability that a new document will spuriously hash to one of these bits is $p = m/n$. Thus, even if the table is only 5% full, there is still a $p = 0.05$ probability that we will falsely discard a new discovery, which is much higher than is acceptable.

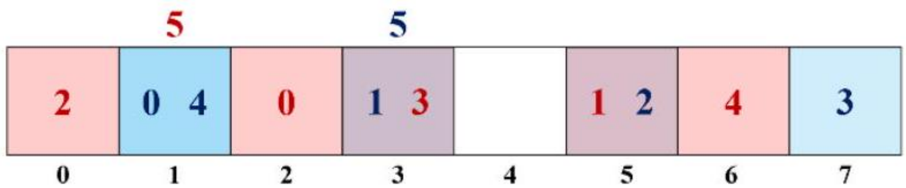


Figure 9.5: Hashing the integers $0,1,2,3$, and 4 into an $n=8$ bit Bloom filter, using hash functions $h_1(x)=2x+1$ (blue) and $h_2(x)=3x+2$ (red). Searching for $x=5$ would yield a false positive, since the two corresponding bits have been set by other elements.

Much better is to employ a Bloom filter, which is also just a bit-vector hash table. But instead of each document corresponding to a single position in the table, a Bloom filter hashes each key k times, using k different hash functions. When we insert document s into our Bloom filter, we set all the bits corresponding to $h_1(s), h_2(s), \ldots h_k(s)$ to be 1 , meaning occupied. To test whether a query document s is present in the data structure, we must test that all k of these bits equal 1 . For a document to be falsely convicted of already being in the filter, it must be unlucky enough that all k of these bits were set in hashes of previous documents, as in the example of Figure 6.6

What are the chances of this? Hashes of m documents in such a Bloom filter will occupy at most km bits, so the probability of a single collision rises to $p_1=km/n$, which is k times greater than the single hash case. But all k bits must collide with those of our query document, which only happens with probability $p_k=(p_1)^k=(km/n)^k$. This is a peculiar expression, because a probability raised to the k th power quickly becomes smaller with increasing k, yet here the probability being raised simultaneously increases with k. To find the k that minimizes p_k, we could take the derivative and set it to zero.

Figure 6.7 graphs this error probability $\left((\mathrm{km}/\mathrm{n})^k\right)$ as a function of load (m/n), with a separate line for each k from 1 to 5 . It is clear that using a large number of hash functions (increased k) reduces false positive error substantially over a conventional hash table (the blue line, $k=1$), at least for small loads. But observe that the error rate associated with larger k increases rapidly with load, so for any given load there is always a point where adding more hash functions becomes counter-productive.

For a 5% load, the error rate for a simple hash table of $k=1$ will be 51.2 times larger than a Bloom filter with $k=5\left(9.77\times10^{-4}\right)$, even though they use exactly the same amount of memory. A Bloom filter is an excellent data structure for maintaining an index, provided you can live with occasionally

saying yes when the answer is no.

The Birthday Paradox and Perfect Hashing Hash tables are an excellent data structure in practice for the standard dictionary operations of insert, delete, and

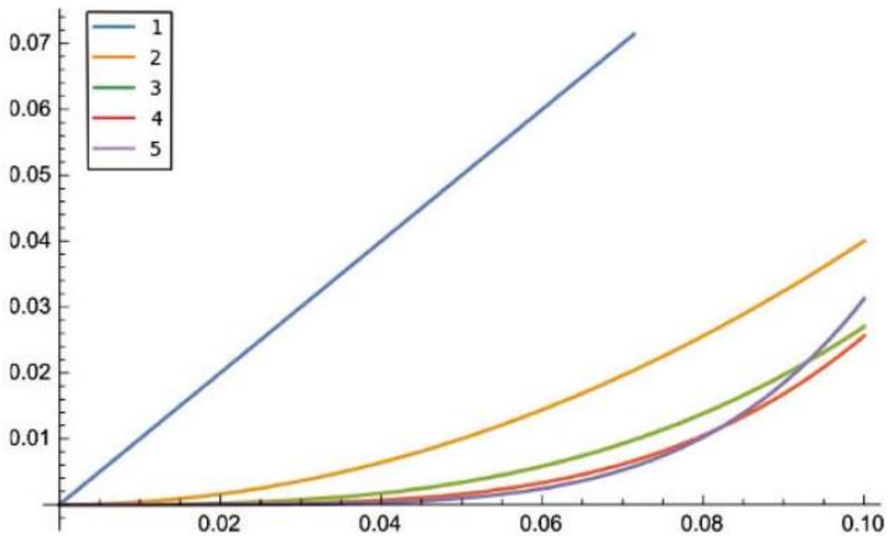


Figure 9.6: Bloom filter error probability as a function of load (m/n) for k from 1 to 5 . By selecting the right k for the given load, we can dramatically reduce false positive error rate with no increase in table space.

search. However, the $\Theta(n)$ worst-case search time for hashing is an annoyance, no matter how rare it is. Is there a way we can guarantee worst-case constant time search?

Perfect hashing offers us this possibility for static dictionaries. Here we are given all possible keys in one batch, and are not allowed to later insert or delete items. We can thus build the data structure once and use it repeatedly for search/membership testing. This is a fairly common use case, so why pay for the flexibility of dynamic data structures when you don't need them?

One idea for how this might work would be to try a given hash function $h(x)$ on our set of n keys S and hope it creates a hash table with no collisions, that is, $h(x) \neq h(y)$ for all pairs of distinct $x, y \in S$. It should be clear that our chances of getting lucky improve as we increase the size of our table relative to n : the more empty slots there are available for the next key, the more likely we find one.

How large a hash table m do we need before we can expect zero collisions among n keys? Suppose we start from an empty table, and repeatedly insert keys. For the $(i+1)$ th insertion, the probability that we hit one of the $m-i$ still-open slots in the table is $(m-i)/m$. For a perfect hash, all n inserts must succeed, so

$$P(\text{ no collision }) = \prod_{i=0}^{n-1}\left(\frac{m-i}{m}\right) = \frac{m!}{m^n((m-n)!)}$$

What happens when you evaluate this is famously called the birthday paradox. How many people do you need in a room before it is likely that at least two of them share the same birthday? Here the table size is $m = 365$. The

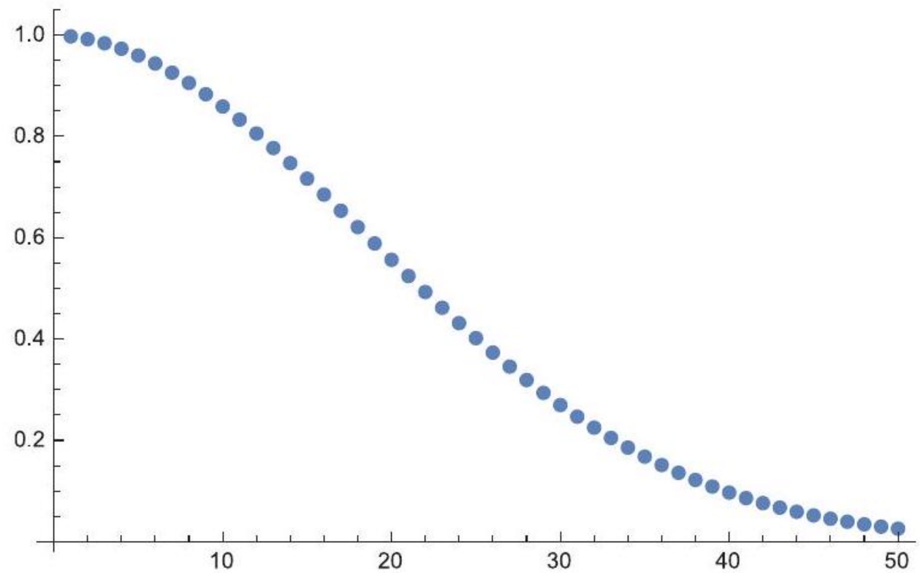


Figure 9.7: The probability of no collisions in a hash table decreases rapidly with n, the number of keys to hash. Here the hash table size is $m = 365$.

probability of no collisions drops below $1/2$ for $n = 23$ and is less than 3% when $n \geq 50$, as shown in Figure 6.8 . In other words, odds are we have a collision when only 23 people are in the room. Solving this asymptotically, we begin to expect collisions when $n = \Theta(\sqrt{m})$ or equivalently when $m = \Theta\left(n^2\right)$.

But quadratic space seems like an awful large penalty to pay for constant time access to n elements. Instead, we will create a two-level hash table. First, we hash the n keys of set S into a table with n slots. We expect collisions, but unless we are very unlucky all the lists will be short enough.

Let l_i be the length of the i th list in this table. Because of collisions, many lists will be of length longer than 1. Our definition of short enough is that n items are distributed around the table such that the sum of squares of the list lengths is linear, that is,

$$N = \sum_{i=1}^{n} {l_i}^2 = \Theta(n)$$

Suppose that it happened that all elements were in lists of length l, meaning that we have n/l non-empty lists. The sum of squares of the list lengths is $N = (n/l)l^2 = nl$, which is linear because l is a constant. We can even get away with a fixed number of lists of length $\sqrt{n}$ and still use linear space.

In fact, it can be shown that $N \leq 4n$ with high probability. So if this isn't true on S for the first hash function we try, we can just try another. Pretty soon we will find one with short-enough list lengths that we can use.

We will use an array of length N for our second-level table, allocating ${l_i}^2$ space for the elements of the i th bucket. Note that this is big enough relative to the number of elements to avoid the birthday paradox-odds are we will have no collision in any given hash function. And if we do, simply try another hash function until all elements end up in unique places.

The complete scheme is illustrated in Figure 6.9 The contents of the i th entry in the first-level hash table include the starting and ending positions for the ${l_i}^2$ entries in the second-level table corresponding to this list. It also contains

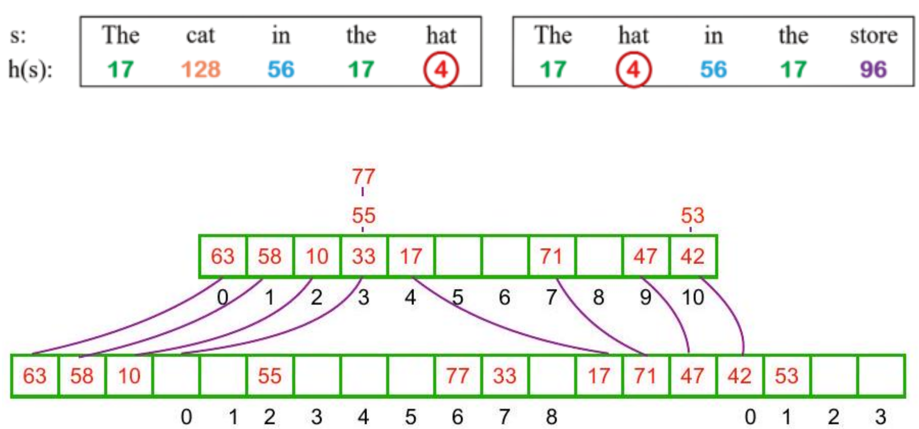


Figure 9.8: Perfect hashing uses a two-level hash table to guarantee lookup in constant time. The first-level table encodes an index range of ${l_i}^2$ elements in the second-level table, allocated to store the l_i items in first-level bucket i.

a description (or identifier) of the hash function that we will use for this region of the second-level table.

Lookup for an item s starts by calling hash function $h_1(s)$ to compute the right spot in the first table, where we will find the appropriate start/stop position and hash function parameters. From this we can compute start + $(h_2(s)(\text{mod}(stop - start)))$, *the location where item s will appear in the second table. Thus, search can always be performed in $\Theta(1)$ time, using linear storage space between the two tables.*

Perfect hashing is a very useful data structure in practice, ideal for whenever you will be making large numbers of queries to a static dictionary. There is a lot of fiddling you can do with this basic scheme to minimize space demands and construction/search cost, such as working harder to find second-level hash functions with fewer holes in the table. Indeed, minimum perfect hashing guarantees constant time access with zero empty hash table slots, resulting in an n-element second hash table for n keys.

9.5 Minwise Hashing

Hashing lets you quickly test whether a specific word w in document D_1 also occurs in document D_2 : build a hash table on the words of D_2 and then hunt for $h(w)$ in this table T. For simplicity and efficiency we can remove duplicate words from each document, so each contains only one entry for each vocabulary term used. By so looking up all the vocabulary words $w_i \in D_1$ in T, we can get a count of the intersection, and compute the Jaccard similarity $J\left(D_1, D_2\right)$ of the two documents, where

$$J\left(D_1, D_2\right)=\frac{\left|D_1 \cap D_2\right|}{\left|D_1 \cup D_2\right|}$$

This similarity measure ranges from 0 to 1 , sort of like a probability that the two documents are similar.

But what if you want to test whether two documents are similar without looking at all the words? If we are doing this repeatedly, on a web scale, effi-

s:	*The*	*cat*	*in*	*the*	*hat*	*Th*	*hat*	*in*	*the*	*store*
h(s *:*	*17*	*128*	*56*	*17*	*(4)*	*17*	*(4)*	*56*	*17*	*96*

Figure 6.10: The words associated with the minimum hash code from each of two documents are likely to be the same, if the documents are very similar.
ciency matters. Suppose we are allowed to look at only one word per document to make a decision. Which word should we pick?

A first thought might be to pick the most frequent word in the original document, but it is likely to be "the" and tell you very little about similarity. Picking the most representative word, perhaps according to the TD-IDF statistic, would be better. But it still makes assumptions about the distribution of words, which may be unwarranted.

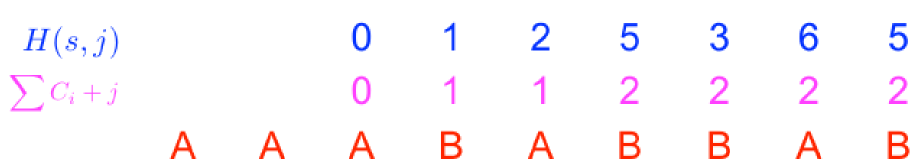
The key idea is to synchronize, so we pick the same word out of the two documents while looking at the documents separately. Minwise hashing is a clever but simple idea to do this. We compute the hash code $h\left(w_i\right)$ of every word in D_1, and select the code with smallest value among all the hash codes. Then we do the same thing with the words in D_2, using the same hash function.

What is cool about this is that it gives a way to pick the same random word in both documents, as shown in Figure 6.10 Suppose the vocabularies of each document were identical. Then the word with minimum hash code will be the same in both D_1 and D_2, and we get a match. In contrast, suppose we had picked completely random words from each document. Then the probability of picking the same word would be only $1/v$, where v is the vocabulary size.

Now suppose that D_1 and D_2 do not have identical vocabularies. The probability that the minhash word appears in both documents depends on the number of words in common, that is, the intersection of the vocabularies, and also on the total vocabulary size of the documents. In fact, this probability is exactly the Jaccard similarity described above.

Sampling a larger number of words, say, the k smallest hash values in each document, and reporting the size of intersection over k gives us a better estimate of Jaccard similarity. But the alert reader may wonder why we bother. It takes time linear in the size of D_1 and D_2 to compute all the hash values just to find the minhash values, yet this is the same running time that it would take to compute the exact size of intersection using hashing!

The value of minhash comes in building indexes for similarity search and clustering over large corpora of documents. Suppose we have N documents, each with an average of m vocabulary words in them. We want to build an index to help us determine which of these is most similar to a new query document Q. Hashing all words in all documents gives us a table of size $O(Nm)$. Storing $k \ll m$ minwise hash values from each document will be much smaller at $O(Nk)$, but the documents at the intersection of the buckets associated with the k minwise hashes of Q are likely to contain the most similar documents-particularly if the Jaccard similarity is high.



Stop and Think: Estimating Population Size *Problem: Suppose we will receive a stream S of n numbers one by one. This stream will contain many duplicates, possibly even a single number repeated n times. How can we estimate the number of distinct values in S using only a constant amount of memory?*

Solution: If space was not an issue, the natural solution would be to build a dictionary data structure on the distinct elements from the stream, each with an associated count of how often it has occurred. For the next element we see in the stream, we add one to the count if it exists in the dictionary, or insert it if it is not found. But we only have enough space to store a constant number of elements. What can we do?

Minwise hashing comes to the rescue. Suppose we hash each new element s of S as it comes in, and only save $h(s)$ if it is smaller than the previous minhash.

Why is this interesting? Suppose the range of possible hash values is between

0 and $M-1$, and we select k values in this range uniformly at a random. What is the expected minimum of these k values? If $k = 1$, the expected value will (obviously) be $M/2$. For general k, we might hand wave and say that if our k values were equally spaced in the interval, the minhash should be $M/(k+1)$.

In fact, this hand waving happens to produce the right answer. Define X as the smallest of k samples. Then

$$P(X=i) = P(X \geq i) - P(X \geq i+1) = \left(\frac{M-i}{M}\right)^k - \left(\frac{M-i-1}{M}\right)^k$$

Taking the limit of the expected value as M gets large gives the result

$$E(X) = \sum_{i=0}^{M-1} i \left(\left(\frac{M-i}{M}\right)^k - \left(\frac{M-i-1}{M}\right)^k \right) \longrightarrow \frac{M}{k+1}$$

The punch line is that M divided by the minhash value gives an excellent estimate of the number of distinct values we have seen. This method will not be fooled by repeated values in the stream, since repeated occurrences will yield precisely the same value every time we evaluate the hash function.

9.6 Efficient String Matching

Strings are sequences of characters where the order of the characters matters: the string ALGORITHM is different than LOGARITHM. Text strings are fundamental to a host of computing applications, from programming language parsing/compilation, to web search engines, to biological sequence analysis.

The primary data structure for representing strings is an array of characters. This allows us constant-time access to the i th character of the string. Some

$H(s,j)$			0	1	2	5	3	6	5
$\sum C_i + j$			0	1	1	2	2	2	2
	A	A	A	B	A	B	B	A	B

Figure 6.11: The Rabin-Karp hash function $H(s,j)$ gives distinctive codes to different substrings (in blue), while a less powerful hash function that just adds the character codes yields many collisions (shown in purple). Here the pattern string (BBA) has length $m = 3$, and the character codes are $A = 0$ and $B = 1$.
auxiliary information must be maintained to mark the end of the string: either a special end-of-string character or (perhaps more usefully) a count n of the characters in the string.

The most fundamental operation on text strings is substring search, namely:
Problem: Substring Pattern Matching
Input: A text string t and a pattern string p.
Output: Does t contain the pattern p as a substring, and if so where?
The simplest algorithm to search for the presence of pattern string p in text t

overlays the pattern string at every position in the text, and checks whether every pattern character matches the corresponding text character. As demonstrated in Section 2.5 .3 (page 43), this runs in $O(nm)$ *time, where* $n = |t|$ *and* $m = |p|$.

This quadratic bound is worst case. More complicated, worst-case lineartime search algorithms do exist: see Section 21.3 (page 685 for a complete discussion. But here I give a linear expected-time algorithm for string matching, called the Rabin-Karp algorithm. It is based on hashing. Suppose we compute a given hash function on both the pattern string p *and the* m*-character substring starting from the* i *th position of* t*. If these two strings are identical, clearly the resulting hash values must be the same. If the two strings are different, the hash values will almost certainly be different. These false positives should be so rare that we can easily spend the* $O(m)$ *time it takes to explicitly check the identity of two strings whenever the hash values agree.*

This reduces string matching to $n - m + 2$ *hash value computations (the* $n - m + 1$ *windows of* t*, plus one hash of* p *), plus what should be a very small number of* $O(m)$ *time verification steps. The catch is that it takes* $O(m)$ *time to compute a hash function on an* m*-character string, and* $O(n)$ *such computations seems to leave us with an* $O(mn)$ *algorithm again.*

But let's look more closely at our previously defined (in Section 3.7 hash function, applied to the m *characters starting from the* j *th position of string* S *:*

$$H(S, j) = \sum_{i=0}^{m-1} \alpha^{m-(i+1)} \times \operatorname{char}(s_{i+j})$$

What changes if we now try to compute $H(S, j+1)$ *- the hash of the next window of* m *characters? Note that* $m - 1$ *characters are the same in both windows, although this differs by one in the number of times they are multiplied by* α*. A little algebra reveals that*

$$H(S, j+1) = \alpha \left(H(S, j) - \alpha^{m-1} \operatorname{char}(s_j)\right) + \operatorname{char}(s_{j+m})$$

This means that once we know the hash value from the j *th position, we can find the hash value from the* $(j+1)$ *th position for the cost of two multiplications, one addition, and one subtraction. This can be done in constant time (the value of* α^{m-1} *can be computed once and used for all hash value computations). This math works even if we compute* $H(S, j) \bmod M$*, where* M *is a reasonably large prime number. This keeps the size of our hash values small (at most* M *) even when the pattern string is long.*

Rabin-Karp is a good example of a randomized algorithm (if we pick M *in some random way). We get no guarantee the algorithm runs in* $O(n+m)$ *time, because we may get unlucky and have the hash values frequently collide with spurious matches. Still, the odds are heavily in our favor-if the hash function returns values uniformly from 0 to* $M - 1$*, the probability of a false collision should be* $1/M$*. This is quite reasonable: if* $M \approx n$*, there should only be one false collision per string, and if* $M \approx n^k$ *for* $k \geq 2$*, the odds are great we will never see any false collisions.*

9.7 Primality Testing

One of the first programming assignments students get is to test whether an integer n is a prime number, meaning that its only divisors are 1 and itself. The sequence of prime numbers starts with $2, 3, 5, 7, 11, 13, 17, \ldots,$ and never ends.

That program you presumably wrote employed trial division as the algorithm: using a loop where i runs from 2 to $n-1$, and check whether n/i is an integer. If so, then i is a factor of n, and so n must be composite. Any integer that survives this gauntlet of tests is prime. In fact, the loop only needs to run up to $\lceil\sqrt{n}\rceil$, since that is the largest possible value of the smallest non-trivial factor of n.

Still, such trial division is not cheap. If we assume that each division takes constant time this gives an $O(\sqrt{n})$ algorithm, but here n is the value of the integer being factored. A 1024-bit number (the size of a small RSA encryption key) encodes numbers up to $2^{1024}-1$, with the security of RSA depending on factoring being hard. Observe that $\sqrt{2^{1024}} = 2^{512}$, which is greater than the number of atoms in the universe. So expect to spend some time waiting before you get the answer.

Randomized algorithms for primality testing (not factoring) turn out to be much faster. Fermat's little theorem states that if n is a prime number then

$$a^{n-1} = 1(\bmod n) \textit{ for all } a \textit{ not divisible by } n$$

For example, when $n = 17$ and $a = 3$, observe that $\left(3^{17-1}-1\right)/17 = 2,532,160$, so $3^{17-1} = 1(\bmod 17)$. But for $n = 16, 3^{16-1} = 11(\bmod 16)$, which proves that 16 cannot be prime.

What makes this interesting is that the mod of this big power always is 1 if n is prime. This is a pretty good trick, because the odds of it being 1 by chance should be very small-only $1/n$ if the residue was uniform in the range.

Let's say we can argue that the probability of a composite giving a residue of 1 is less than $1/2$. This suggests the following algorithm: Pick 100 random integers a_j, each between 1 and $n-1$. Verify that none of them divide n. Then compute $(a_j)^{n-1}(\bmod n)$. If all hundred of these come out to be 1 , then the probability that n is not prime must be less than $(1/2)^{100}$, which is vanishingly small. Because the number of tests (100) is fixed, the running time is always fast, which makes this a Monte Carlo type of randomized algorithm.

There is a minor issue in our probability analysis, however. It turns out that a very small fraction of integers (roughly 1 in 50 billion up to 10^{21}) are not prime, yet also satisfy the Fermat congruence for all a. Such Carmichael numbers like 561 and 1105 are doomed to be always be misclassified as prime. Still, this randomized algorithm proves very effective at distinguishing likely primes from composite integers.

Take-Home Lesson: Monte Carlo algorithms are always fast, usually correct, and most of them are wrong in only one direction.

One issue that might concern you is the time complexity of computing $a^{n-1}(\bmod n)$. *In fact, it can be done in* $O(\log n)$ *time. Recall that we can compute* a^{2m} *as* $(a^m)^2$ *by divide and conquer, meaning we only need a number of multiplications logarithmic in the size of the exponent. Further, we don't have to work with excessively large numbers to do it. Because of the properties of modular arithmetic,*

$$(x \cdot y) \bmod n = ((x \bmod n) \cdot (y \bmod n)) \bmod n$$

so we never need multiply numbers larger than n *over the course of the computation.*

9.8 War Story: Giving Knuth the Middle Initial

The great Donald Knuth is the seminal figure in creating computer science as an intellectually distinct academic discipline. The first three volumes of his Art of Computer Programming series (now four), published between 1968 and 1973, revealed the mathematical beauty of algorithm design, and still make fun and exciting reading. Indeed, I give you my blessing to put my book aside to pick up one of his, at least for a little while.

Knuth is also a co-author of the textbook Concrete Mathematics, which focuses on mathematical analysis techniques for algorithms and discrete mathematics. Like his other books it contains open research questions in addition

to homework problems. One problem that caught my eye concerned middle binomial coefficients, asking whether it is true that

$$\binom{2n}{n} = (-1)^n (\bmod (2n+1)) \textit{ iff } 2n+1 \textit{ is prime}$$

This is suggestive of Fermat's little theorem, discussed in Section 6.8 (page 190 .
The congruence is readily shown to hold whenever $2n+1$ *is prime. By basic modular arithmetic,*

$$(2n)(2n-1)\ldots(n+1) = (-1)(-2)\ldots(-n) = (-1)^n \cdot n! (\bmod (2n+1))$$

Since $ad = bd(\bmod m)$ *implies* $a = b(\bmod m)$ *if* d *is relatively prime to* m *and* n *! divides* $(2n)!/n!, n!$ *can be divided from both sides, giving the result.*

But does this formula hold only when $2n+1$ *is prime, as conjectured? That didn't sound right to me, for logic that is dual to the randomized primality testing algorithm. If we treat the residue* $\bmod 2n{+}1$ *as a random integer, the probability that it would happen to be* $(-1)^n$ *is very small, only* $1/n$. *Thus, not seeing a counterexample over a small number of tests is not very impressive evidence, because chance counterexamples should be rare.*

So I wrote a 16 -line Mathematica program, and left it running for the weekend. When I got back, the program has stopped at $n = 2,953$. *It turns out that* $\binom{5906}{2953} \approx 7.93285 \times 10^{1775}$ *is congruent to 5,906 when taken modulo 5,907 . But since* $5,907 = 3 \cdot 11 \cdot 179$, *this shows that* $2n+1$ *is not prime and the conjecture is refuted.*

It was a big thrill sending this result to Knuth himself, who said he would put my name in the next edition of his book. A notorious stickler for detail, he asked me to give him my middle initial. I proudly replied "S", and asked him when he would send me my check. Knuth famously offered $2.56 *checks to anyone who found mistakes in one of his books* [2] *and I wanted one as a souvenir. But he nixed it, explaining that solving an open problem did not count as fixing an error in his book. I have always regretted that I did not send him my middle initial as "T", because then I would have had an error for him to correct in a future printing.*

9.9 Where do Random Numbers Come From?

All the clever randomized algorithms discussed in this chapter raises a question: Where do we get random numbers? What happens when you call the random number generator associated with your favorite programming language?

We are used to employing physical processes to generate randomness, such as flipping coins, tossing dice, or even monitoring radioactive decay using a Geiger counter. We trust these events to be unpredictable, and hence indicative of true randomness.

But this is not what your random number generator does. Most likely it employs what is essentially a hash function, called a linear congruential generator. The n*th random number* R_n *is a simple function of the previous random number* R_{n-1} *:*

$$R_n = (aR_{n-1} + c) \bmod m$$

where $a, c, m,$ *and* R_0 *are large and carefully selected constants. Essentially, we hash the previous random number (* R_{n-1} *) to get the next one.*

The alert reader may question exactly how random such numbers really are. Indeed, they are completely predictable, because knowing R_{n-1} *provides enough information to construct* R_n. *This predictability means that a sufficiently determined adversary could in principle construct a worst-case input to a randomized algorithm provided they know the current state of your random number generator.*

Linear congruential generators are more accurately called pseudo-random number generators. The stream of numbers produced looks random, in that they have the same statistical properties as would be expected from a truly random source. This is generally good enough for randomized algorithms to work well in practice. However, there is a philosophical sense of randomness which has

[2] I would go broke were I ever to make such an offer.

been lost that occasionally comes back to bite us, typically in cryptographic applications whose security guarantees rest on an assumption of true randomness.

Random number generation is a fascinating problem. Look ahead to section 16.7 in the Hitchhiker's Guide for a more detailed discussion of how random numbers should and should not be generated.

***Chapter Notes** Readers interested in more formal and substantial treatments of randomized algorithms are referred to the book of Mitzenmacher and Upfal*
citemitzenmacher2017probability] and the older text by Motwani and Raghavan [
citemotwani1995randomized. Minwise hashing was invented by Broder [
citebroder1997resemblance.

9.10 Exercises

Probability 6-1. [5] You are given n unbiased coins, and perform the following process to generate all heads. Toss all n coins independently at random onto a table. Each round consists of picking up all the tails-up coins and tossing them onto the table again. You repeat until all coins are heads.
(a) What is the expected number of rounds performed by the process?
(b) What is the expected number of coin tosses performed by the process?

6-2. [5] Suppose we flip n coins each of known bias, such that p_i is the probability of the i th coin being a head. Present an efficient algorithm to determine the exact probability of getting exactly k heads given $p_1, \ldots, p_n \in [0, 1]$.

6-3. [5] An inversion of a permutation is a pair of elements that are out of order.
(a) Show that a permutation of n items has at most $n(n-1)/2$ inversions. Which permutation(s) have exactly $n(n-1)/2$ inversions?
(b) Let P be a permutation and P^r be the reversal of this permutation. Show that P and P^r have a total of exactly $n(n-1)/2$ inversions.
(c) Use the previous result to argue that the expected number of inversions in a random permutation is $n(n-1)/4$.

6-4. [8] A derangement is a permutation p of $\{1, \ldots, n\}$ such that no item is in its proper position, that is, $p_i \neq i$ for all $1 \leq i \leq n$. What is the probability that a random permutation is a derangement?

Hashing 6-5. [easy] An all-Beatles radio station plays nothing but recordings by the Beatles, selecting the next song at random (uniformly with replacement). They get through about ten songs per hour. I listened for 90 minutes before hearing a repeated song. Estimate how many songs the Beatles recorded.
6-6. [5] Given strings S and T of length n and m respectively, find the shortest window in S that contains all the characters in T in expected $O(n+m)$ time.
6-7. [8] Design and implement an efficient data structure to maintain a least recently used (LRU) cache of n integer elements. A LRU cache will discard the least recently accessed element once the cache has reached its capacity, supporting the following operations:

- get(k)− *Return the value associated with the key k if it currently exists in the cache, otherwise return -1 .*

- put(k, v) - *Set the value associated with key k to v, or insert if k is not already present. If there are already n items in the queue, delete the least recently used item before inserting* (k, v).

Both operations should be done in $O(1)$ *expected time.*

Randomized Algorithms 6-8. [5] A pair of English words (w_1, w_2) *is called a rotodrome if one can be circularly shifted (rotated) to create the other word. For example, the words (windup, upwind) are a rotodrome pair, because we can rotate "windup" two positions to the right to get "upwind." Give an efficient algorithm to find all rotodrome pairs among n words of length k, with a worst-case analysis. Also give a faster expected-time algorithm based on hashing.*

6-9. [5] Given an array w of positive integers, where $w[i]$ *describes the weight of index i, propose an algorithm that randomly picks an index in proportion to its weight.*
6-10. [5] You are given a function rand7, which generates a uniform random integer in the range 1 to 7 . Use it to produce a function rand10, which generates a uniform random integer in the range 1 to 10 .

6-11. [5] Let $0 < \alpha < 1/2$ *be some constant, independent of the input array length n. What is the probability that, with a randomly chosen pivot element, the partition subroutine from quicksort produces a split in which the size of both the resulting subproblems is at least* α *times the size of the original array?*
6-12. [8] Show that for any given load m/n, the error probability of a Bloom filter is minimized when the number of hash functions is $k = \exp(-1)/(m/n)$.

LeetCode *6-1. https://leetcode.com/problems/random-pick-with-blacklist/*
6-2. https://leetcode.com/problems/implement-strstr/
6-3. https://leetcode.com/problems/random-point-in-non-overlapping-rectangles/

HackerRank *6-1. https://www.hackerrank.com/challenges/ctci-ransom-note/*
6-2. https://www.hackerrank.com/challenges/matchstick-experiment/
6-3. https://www.hackerrank.com/challenges/palindromes/

Programming Challenges *These programming challenge problems with robot judging are available at https://onlinejudge.org;*

6-1. "Carmichael Numbers"-Chapter 10, problem 10006.
6-2. "Expressions" - Chapter 6, problem 10157.
6-3. "Complete Tree Labeling" - Chapter 6, problem 10247.

Chapter 7

Chapter 10

Graph Traversal

Graphs are one of the unifying themes of computer science - an abstract representation that describes the organization of transportation systems, human interactions, and telecommunication networks. That so many different structures can be modeled using a single formalism is a source of great power to the educated programmer.

More precisely, a graph $G = (V, E)$ consists of a set of vertices V together with a set E of vertex pairs or edges. Graphs are important because they can be used to represent essentially any relationship. For example, graphs can model a network of roads, with cities as vertices and roads between cities as edges, as shown in Figure 7.1. Electrical circuits can also be modeled as graphs, with junctions as vertices and components as edges (or alternately, electrical components as vertices and direct circuit connections as edges).

The key to solving many algorithmic problems is to think of them in terms of graphs. Graph theory provides a language for talking about the properties of relationships, and it is amazing how often messy applied problems have a simple description and solution in terms of classical graph properties.

Designing truly novel graph algorithms is a difficult task, but usually unnecessary. The key to using graph algorithms effectively in applications lies in correctly modeling your problem so you can take advantage of existing algorithms. Becoming familiar with many different algorithmic graph problems is

Figure 7.2: Important properties / flavors of graphs.

more important than understanding the details of particular graph algorithms, particularly since Part II of this book will point you to an implementation as soon as you know the name of your problem.

Here I present the basic data structures and traversal operations for graphs, which will enable you to cobble together solutions for elementary graph problems. Chapter 8 will present more advanced graph algorithms that find minimum spanning trees, shortest paths, and network flows, but I stress the primary importance of correctly modeling your problem. Time spent browsing through the catalog now will leave you better informed of your options when a real job arises.

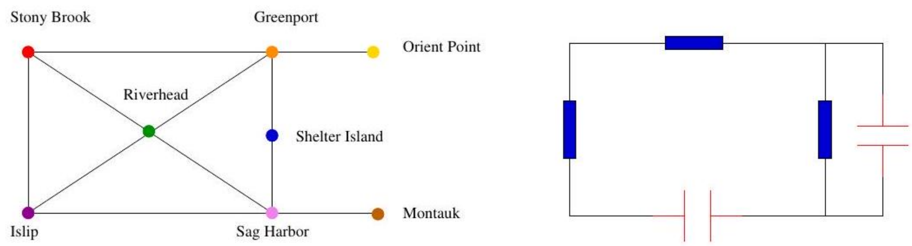


Figure 10.1: Modeling road networks and electrical circuits as graphs.

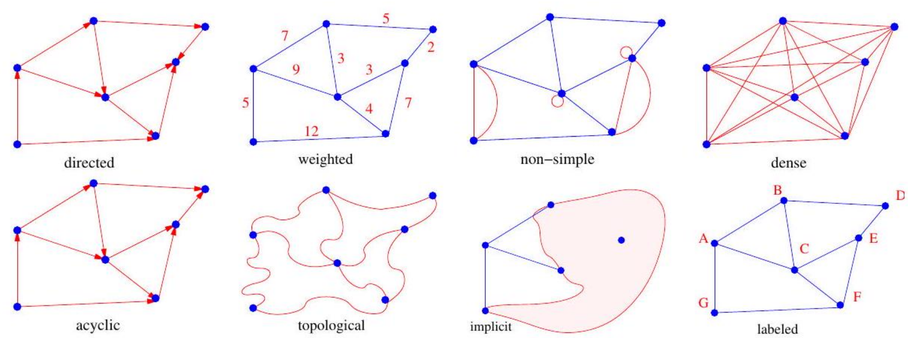


Figure 10.2: Modeling road networks and electrical circuits as graphs.

10.1 Flavors of Graphs

A graph $G = (V, E)$ is defined on a set of vertices V, and contains a set of edges E of ordered or unordered pairs of vertices from V. In modeling a road network, the vertices may represent the cities or junctions, certain pairs of which are connected by roads/edges. In analyzing the source code of a computer program, the vertices may represent lines of code, with an edge connecting lines x and y if y is the next statement executed after x. In analyzing human interactions, the vertices typically represent people, with edges connecting pairs of related souls.

Several fundamental properties of graphs impact the choice of the data structures used to represent them and algorithms available to analyze them. The first step in any graph problem is determining the flavors of the graphs that you will be dealing with (see Figure 7.2):

- *Undirected vs. directed - A graph $G = (V, E)$ is undirected if the presence of edge (x, y) in E implies that edge (y, x) is also in E. If not, we say that the graph is directed. Road networks between cities are typically undirected, since any large road has lanes going in both directions. Street networks within cities are almost always directed, because there are at least a few one-way streets lurking somewhere. Program-flow graphs are typically directed, because the execution flows from one line into the next and changes direction **only** at branches. Most graphs of graph-theoretic interest are undirected.*

- *Weighted vs. unweighted - Each edge (or vertex) in a weighted graph G is assigned a numerical value, or weight. The edges of a road network graph might be weighted with their length, drive time, or speed limit, depending upon the application. In unweighted graphs, there is no cost distinction between various edges and vertices.*

The difference between weighted and unweighted graphs becomes particularly apparent in finding the shortest path between two vertices. For unweighted graphs, a shortest path is one that has the fewest number of edges, and can be found using a breadth-first search (BFS) as discussed in this chapter. Shortest paths in weighted graphs requires more sophisticated algorithms, as discussed in Chapter 8

- *Simple vs. non-simple - Certain types of edges complicate the task of working with graphs. A self-loop is an edge (x, x) involving only one vertex. An edge (x, y) is a multiedge if it occurs more than once in the graph. Both of these structures require special care in implementing graph algorithms. Hence any graph that avoids them is called simple. I confess that all implementations in this book are designed to work only on simple graphs.*

- *Sparse vs. dense: Graphs are sparse when only a small fraction of the possible vertex pairs actually have edges defined between them. Graphs* *where a large fraction of the vertex pairs define edges are called dense. A graph is complete if it contains all possible edges; for a simple undirected graph on n vertices that is* $\binom{n}{2} = \left(n^2 - n\right)/2$ *edges. There is no official boundary between what is called sparse and what is called dense, but dense graphs typically have* $\Theta\left(n^2\right)$ *edges, while sparse graphs are linear in size.*

Sparse graphs are usually sparse for application-specific reasons. Road networks must be sparse because of the complexity of road junctions. The ghastliest intersection I have ever managed to identify is the endpoint of just seven different roads. Junctions of electrical components are similarly limited to the number of wires that can meet at a point, perhaps except for power and ground.

- *Cyclic vs. acyclic - A cycle is a closed path of 3 or more vertices that has no repeating vertices except the start/end point. An acyclic graph does not contain any cycles. Trees are undirected graphs that are connected and acyclic. They are the simplest interesting graphs. Trees are inherently recursive structures, because cutting any edge leaves two smaller trees.*

Directed acyclic graphs are called DAGs. They arise naturally in scheduling problems, where a directed edge (x, y) *indicates that activity* x *must occur before* y*. An operation called topological sorting orders the vertices of a DAG to respect these precedence constraints. Topological sorting is typically the first step of any algorithm on a DAG, as will be discussed in Section 7.10.1 (page 231).*

- *Embedded vs. topological - The edge-vertex representation* $G = (V, E)$ *describes the purely topological aspects of a graph. We say a graph is embedded if the vertices and edges are assigned geometric positions. Thus, any drawing of a graph is an embedding, which may or may not have algorithmic significance.*

Occasionally, the structure of a graph is completely defined by the geometry of its embedding. For example, if we are given a collection of points in the plane, and seek the minimum cost tour visiting all of them (i.e., the traveling salesman problem), the underlying topology is the complete graph connecting each pair of vertices. The weights are typically defined by the Euclidean distance between each pair of points.

Grids of points are another example of topology from geometry. Many problems on an $n \times m$ *rectangular grid involve walking between neighboring points, so the edges are implicitly defined from the geometry.*

- *Implicit vs. explicit - Certain graphs are not explicitly constructed and then traversed, but built as we use them. A good example is in backtrack search. The vertices of this implicit search graph are the states of the search vector, while edges link pairs of states that can be directly generated from each other. Web-scale analysis is another example, where you should try to dynamically crawl and analyze the small relevant portion of interest instead of initially downloading the entire web. The cartoon in Figure 7.2* *tries to capture this distinction between the part of the graph you explicitly know from the fog that covers the rest, which dissipates as you explore it. It is often easier to work with an implicit graph than to explicitly construct and store the entire thing prior to analysis.*

- *Labeled vs. unlabeled - Each vertex is assigned a unique name or identifier in a labeled graph to distinguish it from all other vertices. In unlabeled graphs, no such distinctions have been made.*

Graphs arising in applications are often naturally and meaningfully labeled, such as city names in a transportation network. A common problem is that of isomorphism testing-determining whether the topological structures of two graphs are identical either respecting or ignoring any labels, as discussed in Section 19.9

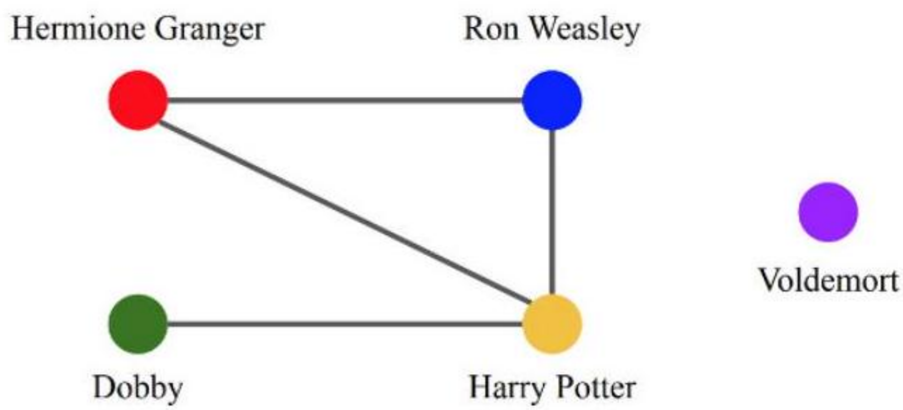


Figure 10.3: A portion of the friendship graph from *Harry Potter*.

10.1.1 The Friendship Graph

To demonstrate the importance of proper modeling, let us consider a graph where the vertices are people, and edge between two people indicates that they are friends. Such graphs are called social networks and are well defined on any set of people - be they the people in your neighborhood, at your school/business, or even spanning the entire world. An entire science analyzing social networks has sprung up in recent years, because many interesting aspects of people and their behavior are best understood as properties of this friendship graph.

We use this opportunity to demonstrate the graph theory terminology described above. "Talking the talk" proves to be an important part of "walking the walk":

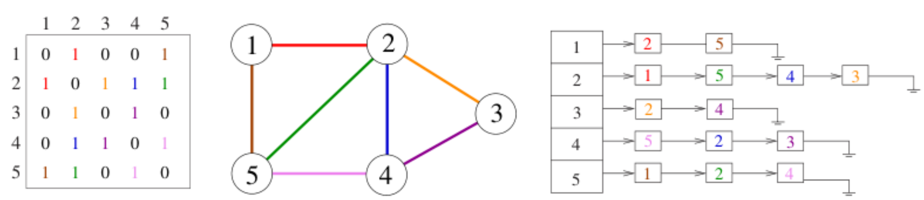
- *If I am your friend, does that mean you are my friend? - This question really asks whether the graph is directed. A graph is undirected if edge*

(x, y) always implies (y, x). Otherwise, the graph is said to be directed. The "heard-of" graph is directed, since I have heard of many famous people who have never heard of me! The "had-sex-with" graph is presumably undirected, since the critical operation always requires a partner. I'd like to think that the "friendship" graph is also an undirected graph.

- *How close a friend are you? - In weighted graphs, each edge has an associated numerical attribute. We could model the strength of a friendship by associating each edge with an appropriate value, perhaps from -100 (enemies) to 100 (blood brothers). The edges of a road network graph might be weighted with their length, drive time, or speed limit, depending upon the application. A graph is said to be unweighted if all edges are assumed to be of equal weight.*

- *Am I my own friend? - This question addresses whether the graph is simple, meaning it contains no loops and no multiple edges. An edge of the form (x, x) is said to be a self-loop. Sometimes people are friends in several different ways. Perhaps x and y were college classmates that now work together at the same company. We can model such relationships using multiedges - multiple edges (x, y) perhaps distinguished by different labels.*

Simple graphs really are simpler to work with in practice. Therefore, we are generally better off declaring that no one is their own friend.



- *Who has the most friends? - The degree of a vertex is the number of edges adjacent to it. The most popular person is identified by finding the vertex of highest degree in the friendship graph. Remote hermits are associated with degree-zero vertices.*

Most of the graphs that one encounters in real life are sparse. The friendship graph is a good example. Even the most gregarious person on earth knows only an insignificant fraction of the world's population.

In dense graphs, most vertices have high degrees, as opposed to sparse graphs with relatively few edges. In a regular graph, each vertex has exactly the same degree. A regular friendship graph is truly the ultimate in social-ism.

- *Do my friends live near me? - Social networks are strongly influenced by geography. Many of your friends are your friends only because they happen to live near you (neighbors) or used to live near you (old college roommates).*

Thus, a full understanding of social networks requires an embedded graph, where each vertex is associated with the point on this world where they live. This geographic information may not be explicitly encoded, but the fact that the graph is inherently embedded on the surface of a sphere shapes our interpretation of the network.

- *Oh, you also know her? - Social networking services such as Instagram and LinkedIn explicitly define friendship links between members. Such graphs consist of directed edges from person/vertex x professing his or her friendship with person/vertex y.*

That said, the actual friendship graph of the world is represented implicitly. Each person knows who their friends are, but cannot find out about other people's friendships except by asking them. The "six degrees of separation" theory argues that there is a short path linking every two people in the world (e.g. Skiena and the President) but offers us no help in actually finding this path. The shortest such path I know of contains three hops:

Steven Skiena → Mark Fasciano → Michael Ashner → Donald Trump

but there could be a shorter one (say if Trump went to college with my dentist) ↑ *The friendship graph is stored implicitly, so I have no way of easily checking.*

	1	2	3	4		
1	0	1	0	0		
2	1	0	1	1		1
3	0	1	0	1		0
4	0	1	1	0		1
5	1	1	0			

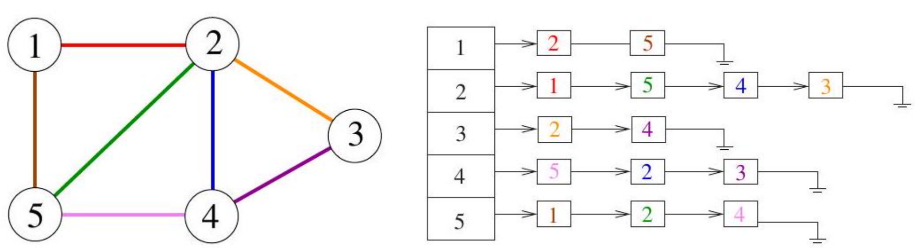


Figure 10.4: The adjacency matrix and adjacency list representation of a given graph. Colors encode specific edges.

[1] There is also a path Steven Skiena → Steve Israel → Joe Biden, so I am covered regardless of the outcome of the 2020 election.

- *Are you truly an individual, or just one of the faceless crowd? - This question boils down to whether the friendship graph is labeled or unlabeled. Are the names or vertex IDs important for our analysis?*
  Much of the study of social networks is unconcerned with labels on graphs. Often the ID number given to a vertex in the graph data structure serves as its label, either for convenience or the need for anonymity. You may assert that you are a name, not a number - but try protesting to the fellow who implements the algorithm. Someone studying how rumors or infectious diseases spread through a friendship graph might examine network properties such as connectedness, the distribution of vertex degrees, or the distribution of path lengths. These properties aren't changed by a scrambling of the vertex IDs.

Take-Home Lesson: Graphs can be used to model a wide variety of structures and relationships. Graph-theoretic terminology gives us a language to talk about them.

10.2 Data Structures for Graphs

Selecting the right graph data structure can have an enormous impact on performance. Your two basic choices are adjacency matrices and adjacency lists, illustrated in Figure 7.4 We assume the graph $G = (V, E)$ contains n vertices and m edges.

- *Adjacency matrix - We can represent G using an $n \times n$ matrix M, where element $M[i, j] = 1$ if (i, j) is an edge of G, and 0 if it isn't. This allows fast answers to the question "is (i, j) in G ?", and rapid updates for edge insertion and deletion. It may use excessive space for graphs with many vertices and relatively few edges, however.*
  Consider a graph that represents the street map of Manhattan in New York City. Every junction of two streets will be a vertex of the graph. Neighboring junctions are connected by edges. How big is this graph? Manhattan is basically a grid of 15 avenues each crossing roughly 200

Comparison	*Winner*
Faster to test if (x, y) is in graph?	*adjacency matrices*
Faster to find the degree of a vertex?	*adjacency lists*
Less memory on sparse graphs?	*adjacency lists $(m+n)$ vs. (n^2)*
Less memory on dense graphs?	*adjacency matrices (a small win)*
Edge insertion or deletion?	*adjacency matrices $O(1)$ vs. $O(d)$*
Faster to traverse the graph?	*adjacency lists $\Theta(m+n)$ vs. $\Theta(n^2)$*
Better for most problems?	*adjacency lists*

Figure 7.5: Relative advantages of adjacency lists and matrices.

streets. This gives us about 3,000 vertices and 6,000 edges, since almost all vertices neighbor four other vertices and each edge is shared between two vertices.

This is a modest amount of data to store, yet an adjacency matrix would have $3,000 \times 3,000 = 9,000,000$ *elements, almost all of them empty!*

There is some potential to save space by packing multiple bits per word or using a symmetric-matrix data structure (e.g. triangular matrix) for undirected graphs. But these methods lose the simplicity that makes adjacency matrices so appealing and, more critically, remain inherently quadratic even for sparse graphs.

- *Adjacency lists - We can more efficiently represent sparse graphs by using linked lists to store the neighbors of each vertex. Adjacency lists require pointers, but are not frightening once you have experience with linked structures.*

Adjacency lists make it harder to verify whether a given edge (i, j) *is in* G*, since we must search through the appropriate list to find the edge. However, it is surprisingly easy to design graph algorithms that avoid any need for such queries. Typically, we sweep through all the edges of the graph in one pass via a breadth-first or depth-first traversal, and update the implications of the current edge as we visit it. Table 7.5 summarizes the tradeoffs between adjacency lists and matrices.*

Take-Home Lesson: Adjacency lists are the right data structure for most applications of graphs.

We will use adjacency lists as our primary data structure to represent graphs. We represent a graph using the following data type. For each graph, we keep a count of the number of vertices, and assign each vertex a unique identification number from 1 to nvertices. We represent the edges using an array of linked lists:

```
#define MAXV 100 /* maximum number of vertices */
typedef struct edgenode {
    int y; /* adjacency info */
    int weight; /* edge weight, if any */
    struct edgenode *next; /* next edge in list */
} edgenode;
typedef struct {
    edgenode *edges[MAXV+1]; /* adjacency info */
    int degree[MAXV+1]; /* outdegree of each vertex */
    int nvertices; /* number of vertices in the graph */
    int nedges; /* number of edges in the graph */
    int directed; /* is the graph directed? */
} graph;
```

We represent directed edge (x, y) *by an edgenode y in x 's adjacency list. The degree field of the graph counts the number of meaningful entries for the given vertex. An undirected edge (* x, y *) appears twice in any adjacency-based graph structure, once as y in x 's list, and the other as x in y 's list. The Boolean* *flag directed identifies whether the given graph is to be interpreted as directed or undirected.*

To demonstrate the use of this data structure, we show how to read a graph from a file. A typical graph file format consists of an initial line giving the number of vertices and edges in the graph, followed by a list of the edges, one vertex pair per line. We start by initializing the structure:

```
void initialize_graph(graph *g, bool directed) {
    int i; /* counter */
    g->nvertices = 0;
    g->nedges = 0;
    g->directed = directed;
    for (i = 1; i <= MAXV; i++) {\index{bipartite graph recognition}\index{two-coloring
        g->degree[i] = 0;
    }
    for (i = 1; i <= MAXV; i++) {
        g->edges[i] = NULL;
    }
}
```

Then we actually read the graph file, inserting each edge into this structure:

```
void read_graph(graph *g, bool directed) {
    int i; /* counter */
    int m; /* number of edges */
    int x, y; /* vertices in edge (x,y) */
    initialize_graph(g, directed);
    scanf("%d %d", &(g->nvertices), &m);
    for (i = 1; i <= m; i++) {
        scanf("%d %d", &x, &y);
        insert_edge(g, x, y, directed);
    }
}
```

The critical routine is insert_edge. The new edgenode is inserted at the beginning of the appropriate adjacency list, since order doesn't matter. We parameterize our insertion with the directed Boolean flag, to identify whether we need to insert two copies of each edge or only one. Note the use of recursion to insert the copy:

```
void insert_edge(graph *g, int x, int y, bool directed) {
    edgenode *p; /* temporary pointer */
    p = malloc(sizeof(edgenode)); /* allocate edgenode storage */
    p->weight = 0;
    p->y = y;
    p->next = g->edges[x];
    g->edges[x] = p; /* insert at head of list */
```

```
    g->degree [x] ++;
    if (!directed) {
        insert_edge(g, y, x, true);
    } else {
        g->nedges++;
    }
}
```

Printing the associated graph is just a matter of two nested loops: one through the vertices, and the second through adjacent edges:

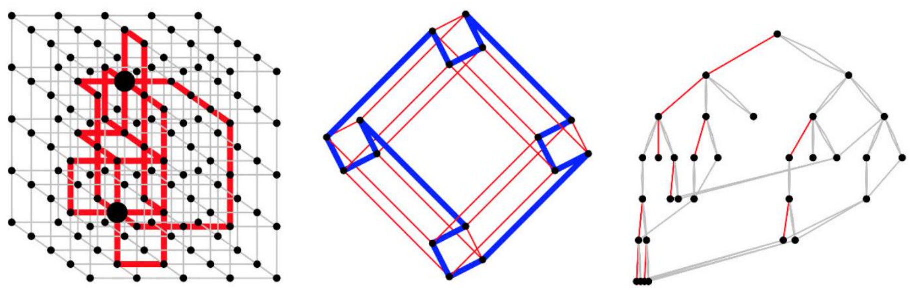


Figure 10.5: edge-disjoint paths (left), Hamiltonian cycle in a hypercube (center), animated depth-first search tree traversal (right).

```
void print_graph(graph *g) {
    int i; /* counter */
    edgenode *p; /* temporary pointer */
    for (i = 1; i <= g->nvertices; i++) {
        printf("%d: ", i);
        p = g->edges[i];
        while (p != NULL) {
            printf(" %d", p->y);
        p = p->next;
        }
        printf("\n");
    }
}
```

It is a good idea to use a well-designed graph data type as a model for building your own, or even better as the foundation for your application. I recommend LEDA (see Section 22.1.1 (page 713)) or Boost (see Section 22.1.3 (page 714) as the best-designed general-purpose graph data structures currently available. They may be more powerful (and hence somewhat slower/larger) than you need, but they do so many things right that you are likely to lose most of the potential do-it-yourself benefits through clumsiness.

10.3 War Story: I was a Victim of Moore's Law

I am the author of a popular library of graph algorithms called Combinatorica (www.combinatorica.com), which runs under the computer algebra system Mathematica. Efficiency is a great challenge in Mathematica, due to its applicative model of computation (it does not support constant-time write operations

command/machine	*Sun-3*	*Sun-4*	*Sun-5*	*Ultra 5*	*Blade*
PlanarQ[GridGraph[4,4]]	*234.10*	*69.65*	*27.50*	*3.60*	*0.40*
Length[Partitions[30]]	*289.85*	*73.20*	*24.40*	*3.44*	*1.58*
VertexConnectivity[GridGraph[3,3]]	*239.67*	*47.76*	*14.70*	*2.00*	*0.91*
RandomPartition[1000]	*831.68*	*267.5*	*22.05*	*3.12*	*0.87*

Table 7.1: Old Combinatorica benchmarks on five generations of workstations (running time in seconds).

to arrays) and the overhead of interpretation (as opposed to compilation). Mathematica code is typically 1,000 to 5,000 times slower than C code.

Such slowdowns can be a tremendous performance hit. Even worse, Mathematica is a memory hog, needing a then-outrageous 4 MB of main memory to run effectively when I completed Combinatorica in 1990. Any computation on large structures was doomed to thrash in virtual memory. In such an environment, my graph package could only hope to work effectively on very small graphs.

One design decision I made as a result was to use adjacency matrices as the basic Combinatorica graph data structure instead of lists. This may sound peculiar. If pressed for memory, wouldn't it pay to use adjacency lists and conserve every last byte? Yes, but the answer is not so simple for very small graphs. An adjacency list representation of a weighted n-vertex, m-edge directed graph should use about $n + 2m$ words to represent; the $2m$ comes from storing the endpoint and weight components of each edge. Thus, the space advantages of adjacency lists only kick in when $n + 2m$ is substantially smaller than n^2. The adjacency matrix is still manageable in size for $n \leq 100$ and, of course, about half the size of adjacency lists on dense graphs.

My more immediate concern was dealing with the overhead of using a slow interpreted language. Check out the benchmarks reported in Table 7.1 Two particularly complex but polynomial-time problems on 9 and 16 vertex graphs took several minutes to complete on my desktop machine in 1990! The quadratic-sized data structure certainly could not have had much impact on these running times, since 9×9 equals only 81 . From experience, I knew the Mathematica programming language handled regular structures like adjacency matrices better than irregular-sized adjacency lists.

Still, Combinatorica proved to be a very good thing despite these performance problems. Thousands of people have used my package to do all kinds of interesting things with graphs. Combinatorica was never intended to be a high-performance algorithms library. Most users quickly realized that computations on large graphs were out of the question, but were eager to take advantage of

Combinatorica as a mathematical research tool and prototyping environment. Everyone was happy.

But over the years, my users started asking why it took so long to do a modest-sized graph computation. The mystery wasn't that my program was slow, because it had always been slow. The question was why did it take them so many years to figure this out?

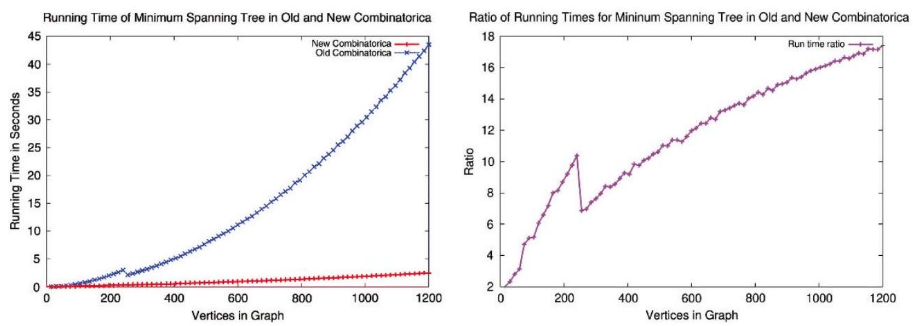


Figure 10.6: absolute running times (left), and the ratio of these times (right).

The reason is that computers keep doubling in speed every two years or so. People's expectation of how long something should take moves in concert with these technology improvements. Partially because of Combinatorica's dependence on a quadratic-size graph data structure, it didn't scale as well as it should on sparse graphs.

As the years rolled on, user demands became more insistent. Combinatorica needed to be updated. My collaborator, Sriram Pemmaraju, rose to the challenge. We (mostly he) completely rewrote Combinatorica to take advantage of faster graph data structures ten years after the initial version.

The new Combinatorica uses a list of edges data structure for graphs, largely motivated by increased efficiency. Edge lists are linear in the size of the graph (edges plus vertices), just like adjacency lists. This makes a huge difference on most graph-related functions-for large enough graphs. The improvement is most dramatic in "fast" graph algorithms - those that run in linear or nearlinear time, such as graph traversal, topological sort, and finding connected or biconnected components. The implications of this change are felt throughout the package in running time improvements and memory savings. Combinatorica can now work with graphs that are fifty to a hundred times larger than what the old package could deal with.

Figure 7.7(left) plots the running time of the MinimumSpanningTree functions for both Combinatorica versions. The test graphs were sparse (grid graphs), designed to highlight the difference between the two data structures. Yes, the new version is much faster, but note that the difference only becomes important for graphs larger than the old Combinatorica was designed for. However, the relative difference in run time keeps growing with increasing n. Figure 7.7(right) *plots the ratio of the running times as a function of graph size. The difference between linear size and quadratic size is asymptotic, so the consequences become ever more important as n gets larger.*

What is the weird bump in running times that occurs around $n \approx 250$? This likely reflects a transition between levels of the memory hierarchy. Such bumps are not uncommon in today's complex computer systems. Cache performance in data structure design should be an important but not overriding consideration. The asymptotic gains due to adjacency lists more than trumped any impact of the cache.

Three main lessons can be taken away from our experience developing Combinatorica:

- *To make a program run faster, just wait - Sophisticated hardware eventually trickles down to everybody. We observe a speedup of more than 200 -fold for the original version of Combinatorica as a consequence of 15 years of hardware evolution. In this context, the further speedups we obtained from upgrading the package become particularly dramatic.*

- *Asymptotics eventually do matter - It was my mistake not to anticipate future developments in technology. While no one has a crystal ball, it is fairly safe to say that future computers will have more memory and run faster than today's. This gives the edge to asymptotically more efficient algorithms/data structures, even if their performance is close on today's instances. If the implementation complexity is not substantially greater, play it safe and go with the better algorithm.*

- *Constant factors can matter - With the growing importance of the study of networks, Wolfram Research has recently moved basic graph data structures into the core of Mathematica. This permits them to be written in a compiled instead of interpreted language, speeding all operations by about a factor of 10 over Combinatorica.*

Speeding up a computation by a factor of 10 is often very important, taking it from a week down to a day, or a year down to a month. This book focuses largely on asymptotic complexity, because we seek to teach fundamental principles. But constants can matter in practice.

10.4 War Story: Getting the Graph

"It takes five minutes just to read the data. We will never have time to make it do something interesting."

The young graduate student was bright and eager, but green to the power of data structures. She would soon come to appreciate their power.

As described in a previous war story (see Section 3.6 (page 89), we were experimenting with algorithms to extract triangular strips for the fast rendering of triangulated surfaces. The task of finding a small number of strips that cover *each triangle in a mesh can be modeled as a graph problem. The graph has a vertex for every triangle of the mesh, with an edge between every pair of vertices representing adjacent triangles. This dual graph representation (see*

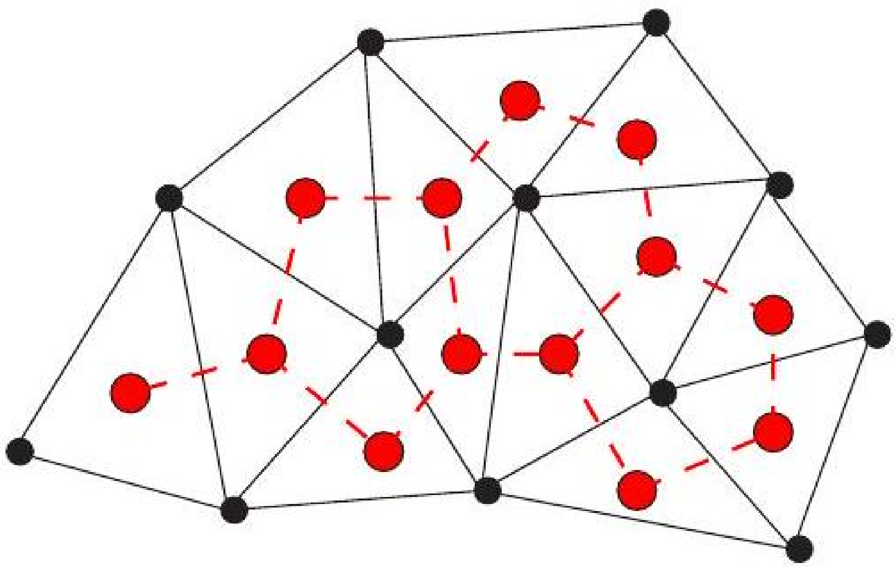


Figure 10.7: The dual graph (dashed lines) of a triangulation

Figure 7.8 captures all information needed to partition the triangulation into triangle strips.

The first step in crafting a program that constructs a good set of strips was to build the dual graph of the triangulation. This I sent the student off to do. A few days later, she came back and announced that it took her machine over five minutes to construct the dual graph of a few thousand triangles.
"Nonsense!" I proclaimed. "You must be doing something very wasteful in building the graph. What format is the data in?"
"Well, it starts out with a list of the three-dimensional coordinates of the vertices used in the model and then follows with a list of triangles. Each triangle is described by a list of three indices into the vertex coordinates. Here is a small example,"

```
\index{articulation vertex}\index{connected component}\index{connectivity}\index{network design}\
VERTICES 4
0.000000 240.000000 0.000000
204.000000 240.000000 0.000000
204.000000 0.000000 0.000000
0.000000 0.000000 0.000000
```

```
TRIANGLES 2
0 13
123
```

"I see. So the first triangle uses all but the third point, since all the indices start from zero. The two triangles must share an edge formed by points 1 and 3."
"Yeah, that's right," she confirmed.
"OK. Now tell me how you built your dual graph from this file."
"Well, the geometric position of the points doesn't affect the structure of the graph, so I can ignore it. My dual graph is going to have as many vertices as the number of triangles. I set up an adjacency list data structure with that many vertices. As I read in each triangle, I compare it to each of the others to check whether it has two end points in common. If it does, then I add an edge from the new triangle to this one."

I started to sputter. "But that's your problem right there! You are comparing each triangle against every other triangle, so that constructing the dual

graph will be quadratic in the number of triangles. Reading the input graph should take linear time!"
"I'm not comparing every triangle against every other triangle. On average, it only tests against half or a third of the triangles."
"Swell. But that still leaves us with an $O\left(n^2\right)$ *algorithm. That is much too slow."*

She stood her ground. "Well, don't just complain. Help me fix it!"
Fair enough. I started to think. We needed some quick method to screen away most of the triangles that would not be adjacent to the new triangle (i, j, k)*. What we really needed was a separate list of all the triangles that go through each of the points* i, j*, and* k*. By Euler's formula for planar graphs, the average point is incident to fewer than six triangles. This would compare each new triangle against fewer than twenty others, instead of most of them.*
"We are going to need a data structure consisting of an array with one element for every vertex in the original data set. This element is going to be a list of all the triangles that pass through that vertex. When we read in a new triangle, we will look up the three relevant lists in the array and compare each of these against the new triangle. Actually, only two of the three lists need be tested, since any adjacent triangles will share two points in common. We will add an edge to our graph for every triangle pair sharing two vertices. Finally, we will add our new triangle to each of the three affected lists, so they will be updated for the next triangle read."

She thought about this for a while and smiled. "Got it, Chief. I'll let you know what happens."

The next day she reported that the graph could be built in seconds, even for much larger models. From here, she went on to build a successful program for extracting triangle strips, as reported in Section 3.6 (page 89.

Take-Home Lesson: Even elementary problems like initializing data struc-

tures can prove to be bottlenecks in algorithm development. Programs working with large amounts of data must run in linear or near-linear time. Such tight performance demands leave no room to be sloppy. Once you focus on the need for linear-time performance, an appropriate algorithm or heuristic can usually be found to do the job.

10.5 Traversing a Graph

Perhaps the most fundamental graph problem is to visit every edge and vertex in a graph in a systematic way. Indeed, all the basic bookkeeping operations on graphs (such as printing or copying graphs, and converting between alternative representations) are applications of graph traversal.

Mazes are naturally represented by graphs, where each graph vertex denotes a junction of the maze, and each graph edge denotes a passageway in the maze. Thus, any graph traversal algorithm must be powerful enough to get us out of an arbitrary maze. For efficiency, we must make sure we don't get trapped in

the maze and visit the same place repeatedly. For correctness, we must do the traversal in a systematic way to guarantee that we find a way out of the maze. Our search must take us through every edge and vertex in the graph.

The key idea behind graph traversal is to mark each vertex when we first visit it and keep track of what we have not yet completely explored. Bread crumbs and unraveled threads have been used to mark visited places in fairy-tale mazes, but we will rely on Boolean flags or enumerated types.

Each vertex will exist in one of three states:

- *Undiscovered - the vertex is in its initial, virgin state.*
- *Discovered - the vertex has been found, but we have not yet checked out all its incident edges.*
- *Processed - the vertex after we have visited all of its incident edges.*

Obviously, a vertex cannot be processed until after we discover it, so the state of each vertex progresses from undiscovered to discovered to processed over the course of the traversal.

We must also maintain a structure containing the vertices that we have discovered but not yet completely processed. Initially, only the single start vertex is considered to be discovered. To completely explore a vertex v*, we must evaluate each edge leaving* v*. If an edge goes to an undiscovered vertex* x*, we mark* x *discovered and add it to the list of work to do in the future. If an edge goes to a processed vertex, we ignore that vertex, because further contemplation will tell us nothing new about the graph. Similarly, we can ignore any edge going to a discovered but not processed vertex, because that destination already resides on the list of vertices to process.*

Each undirected edge will be considered exactly twice, once when each of its endpoints is explored. Directed edges will be considered only once, when *exploring the source vertex. Every edge and vertex in the connected component must eventually be visited. Why? Suppose that there exists a vertex u that remains unvisited, whose neighbor v was visited. This neighbor v will eventually be explored, after which we will certainly visit u. Thus, we must find everything that is there to be found.*

I describe the mechanics of these traversal algorithms and the significance of the traversal order below.

10.6 Breadth-First Search

The basic breadth-first search algorithm is given below. At some point during the course of a traversal, every node in the graph changes state from undiscovered to discovered. In a breadth-first search of an undirected graph, we assign a direction to each edge, from the discoverer u to the discovered v. We thus denote u to be the parent of v. Since each node has exactly one parent, except for the root, this defines a tree on the vertices of the graph. This tree, illustrated in

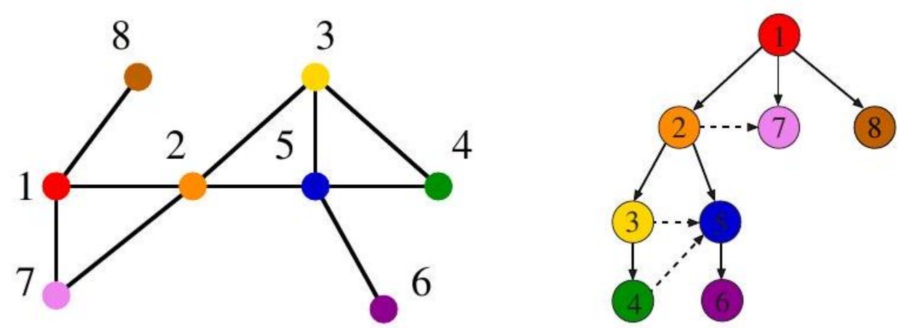


Figure 10.8: An undirected graph and its breadth-first search tree. Dashed lines, which are not part of the tree, show graph edges that go to discovered or processed vertices.

Figure 7.9, defines a shortest path from the root to every other node in the tree. This property makes breadth-first search very useful in shortest path problems.

BFS(G, s)
Initialize each vertex $u \in V[G]$ so
state $[u] =$ "undiscovered"
$p[u] =$ *nil, i.e. no parent is in the BFS tree*
state $[s] =$ "discovered"
$Q = \{s\}$
while $Q \neq \emptyset$ do
$u =$ *dequeue $[Q]$*
process vertex u if desired
for each vertex v that is adjacent to u
process edge (u, v) if desired
if state $[v] =$ "undiscovered" then
state $[v] =$ "discovered"
$p[v] = u$
enqueue $[Q, v]$
state $[u] =$ "processed"

The graph edges that do not appear in the breadth-first search tree also have special properties. For undirected graphs, non-tree edges can point only to vertices on the same level as the parent vertex, or to vertices on the level directly below the parent. These properties follow easily from the fact that each path in the tree must be a shortest path in the graph. For a directed graph, a back-pointing edge (u, v) can exist whenever v lies closer to the root than u does.

Implementation *Our breadth-first search implementation bfs uses two Boolean arrays to maintain our knowledge about each vertex in the graph. A vertex is discovered the first time we visit it. A vertex is considered processed after we have traversed all outgoing edges from it. Thus, each vertex passes from undiscovered*

to discovered to processed over the course of the search. This information could have been maintained using one enumerated type variable, but we used two Boolean variables instead.

```
bool processed[MAXV+1]; /* which vertices have been processed */
bool discovered[MAXV+1]; /* which vertices have been found */
int parent[MAXV+1]; /* discovery relation */
```

Each vertex is initialized as undiscovered:

```
void initialize_search(graph *g) {
    int i; /* counter */
    time = 0;
```

```
    for (i = 0; i <= g->nvertices; i++) {
            processed[i] = false;
            discovered[i] = false;
            parent[i] = -1;
    }
}
```

Once a vertex is discovered, it is placed on a queue. Since we process these vertices in first-in, first-out (FIFO) order, the oldest vertices are expanded first, which are exactly those closest to the root:

```
void bfs(graph *g, int start) {
    queue q; /* queue of vertices to visit */
    int v; /* current vertex */
    int y; /* successor vertex */
    edgenode *p; /* temporary pointer */
    init_queue(&q);
    enqueue(&q, start);
    discovered[start] = true;
    while (!empty_queue(&q)) {
        v = dequeue(&q);
        process_vertex_early(v);
        processed[v] = true;
        p = g->edges[v];
        while (p != NULL) {
            y = p->y;
            if ((!processed[y]) || g->directed) {\index{articulation vertex}\index{bic
                process_edge(v, y);

        }
        if (!discovered[y]) {
            enqueue(&q,y);
            discovered[y] = true;
            parent[y] = v;
        }
        p = p->next;
        }
        process_vertex_late(v);
    }
}
```

10.6.1 Exploiting Traversal

The exact behavior of bfs depends upon the functions process_vertex_early (), process_vertex_late(), and process_edge(). Through these functions, we can customize what the traversal does as it makes its one official visit to each edge *and each vertex. Initially, we will do all vertex processing on entry, so process_vertex_late() returns without action:*

```
void process_vertex_late(int v) {
}
```

By setting the active functions to

```
void process_vertex_early(int v) {
    printf("processed vertex %d\n", v);
}
void process_edge(int x, int y) {
    printf("processed edge (%d,%d)\n", x, y);
}
```

we print each vertex and edge exactly once. If we instead set process_edge to

```
void process_edge(int x, int y) {
    nedges = nedges + 1;
}
```

we get an accurate count of the number of edges. Different algorithms perform different actions on vertices or edges as they are encountered. These functions give us the freedom to easily customize these actions.

10.6.2 Finding Paths

The parent array that is filled over the course of bfs() is very useful for finding interesting paths through a graph. The vertex that first discovered vertex i is defined as the parent [i]. Every vertex is discovered once during the course of traversal, so every node has a parent, except for the start node. This parent relation defines a tree of discovery, with the start node as the root of the tree.

Because vertices are discovered in order of increasing distance from the root, this tree has a very important property. The unique tree path from the root to each node $x \in V$ uses the smallest number of edges (or equivalently, intermediate nodes) possible on any root-to- x path in the graph.

We can reconstruct this path by following the chain of ancestors from x to the root. Note that we have to work backward. We cannot find the path from the root to x, because this does not agree with the direction of the parent pointers. Instead, we must find the path from x to the root. Since this is the reverse of how we normally want the path, we can either (1) store it and then explicitly reverse it using a stack, or (2) let recursion reverse it for us, as follows:

```
void find_path(int start, int end, int parents[]) {
    if ((start == end) || (end == -1)) {\index{DAG}\index{longest path, DAG}\index{scheduling}\ir
        printf("\n%d", start);
```

```
    } else {
        find_path(start, parents[end], parents);
        printf(" %d", end);
    }
}
```

On our breadth-first search graph example (Figure 7.9) our algorithm generated the following parent relation:

vertex	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*
parent	*-1*	*1*	*2*	*3*	*2*	*5*	*1*	*1*

For the shortest path from 1 to 6 , this parent relation yields the path $\{1, 2, 5, 6\}$.
There are two points to remember when using breadth-first search to find a shortest path from x to y : First, the shortest path tree is only useful if BFS was performed with x as the root of the search. Second, BFS gives the shortest path only if the graph is unweighted. We will present algorithms for finding shortest paths in weighted graphs in Section 8.3.1 (page 258).

10.7 Applications of Breadth-First Search

Many elementary graph algorithms perform one or two traversals of the graph, while doing something along the way. Properly implemented using adjacency lists, any such algorithm is destined to be linear, since BFS runs in $O(n + m)$ *time for both directed and undirected graphs. This is optimal, since this is as fast as one can ever hope to just read an n-vertex, m-edge graph.*

The trick is seeing when such traversal approaches are destined to work. I present several examples below.

10.7.1 Connected Components

We say that a graph is connected if there is a path between any two vertices. Every person can reach every other person through a chain of links if the friendship graph is connected.

A connected component of an undirected graph is a maximal set of vertices such that there is a path between every pair of vertices. The components are separate "pieces" of the graph such that there is no connection between the pieces. If we envision tribes in remote parts of the world that have not yet been encountered, each such tribe would form a separate connected component in the friendship graph. A remote hermit, or extremely uncongenial fellow, would represent a connected component of one vertex.

An amazing number of seemingly complicated problems reduce to finding or counting connected components. For example, deciding whether a puzzle such as Rubik's cube or the 15 -puzzle can be solved from any position is really asking whether the graph of possible configurations is connected.

Connected components can be found using breadth-first search, since the vertex order does not matter. We begin by performing a search starting from an arbitrary vertex. Anything we discover during this search must be part of the same connected component. We then repeat the search from any undiscovered vertex (if one exists) to define the next component, and so on until all vertices have been found:

```
void connected_components(graph *g) {
    int c; /* component number */
    int i; /* counter */
    initialize_search(g);
    c = 0;
    for (i = 1; i <= g->nvertices; i++) {
        if (!discovered[i]) {
            c = c + 1;
            printf("Component %d:", c);
            bfs(g, i);
            printf("\n");
        }
    }
}

void process_vertex_early(int v) { /* vertex to process */
    printf(" %d", v);
}
```

void process_edge(int x, int y) {
}

Observe how we increment a counter c denoting the current component number with each call to bfs. Alternatively, we could have explicitly bound each vertex to its component number (instead of printing the vertices in each component) by changing the action of process_vertex.

There are two distinct notions of connectivity for directed graphs, leading to algorithms for finding both weakly connected and strongly connected components. Both of these can be found in $O(n+m)$ time, as discussed in Section 18.1 (page 542 .

10.7.2 Two-Coloring Graphs

The vertex-coloring problem seeks to assign a label (or color) to each vertex of a graph such that no edge links any two vertices of the same color. We can easily avoid all conflicts by assigning each vertex a unique color. However, the goal is to use as few colors as possible. Vertex coloring problems often arise in scheduling applications, such as register allocation in compilers. See Section 19.7 (page 604) for a full treatment of vertex-coloring algorithms and applications.

A graph is bipartite if it can be colored without conflicts while using only two colors. Bipartite graphs are important because they arise naturally in many applications. Consider the "mutually interested-in" graph in a heterosexual world, where people consider only those of opposing gender. In this simple model, gender would define a two-coloring of the graph.

But how can we find an appropriate two-coloring of such a graph, thus separating men from women? Suppose we declare by fiat that the starting vertex is "male." All vertices adjacent to this man must be "female," provided the graph is indeed bipartite.

We can augment breadth-first search so that whenever we discover a new vertex, we color it the opposite of its parent. We check whether any non-tree edge links two vertices of the same color. Such a conflict means that the graph cannot be two-colored. If the process terminates without conflicts, we have constructed a proper two-coloring.

```
void twocolor(graph *g) {
    int i; /* counter */
    for (i = 1; i <= (g->nvertices); i++) {
        color[i] = UNCOLORED;
    }
    bipartite = true;
    initialize_search(g);
    for (i = 1; i <= (g->nvertices); i++) {
        if (!discovered[i]) {
            color[i] = WHITE;
            bfs(g, i);
        }
    }
}
void process_edge(int x, int y) {
    if (color[x] == color [y]) {
        bipartite = false;
        printf("Warning: not bipartite, due to (%d,%d)\n", x, y);
    }
    color [y] = complement(color[x]);
}
int complement(int color) {
    if (color == WHITE) {
        return(BLACK);
    }
    if (color == BLACK) {
        return(WHITE);
    }
    return(UNCOLORED);
}
```

We can assign the first vertex in any connected component to be whatever *color/gender we wish. BFS can separate men from women, but we can't tell which gender corresponds to which color just by using the graph structure. Also, bipartite graphs require distinct and binary categorical attributes, so they don't*

model the real-world variation in sexual preferences and gender identity. Take-Home Lesson: Breadth-first and depth-first search provide mechanisms to visit each edge and vertex of the graph. They prove the basis of most simple, efficient graph algorithms.

10.8 Depth-First Search

There are two primary graph traversal algorithms: breadth-first search (BFS) and depth-first search (DFS). For certain problems, it makes absolutely no difference which you use, but in others the distinction is crucial.

The difference between BFS and DFS lies in the order in which they explore vertices. This order depends completely upon the container data structure used to store the discovered but not processed vertices.

- *Queue - By storing the vertices in a first-in, first-out (FIFO) queue, we explore the oldest unexplored vertices first. Our explorations thus radiate out slowly from the starting vertex, defining a breadth-first search.*
- *Stack - By storing the vertices in a last-in, first-out (LIFO) stack, we explore the vertices by forging steadily along along a path, visiting a new neighbor if one is available, and backing up only when we are surrounded by previously discovered vertices. Our explorations thus quickly wander away from the starting point, defining a depth-first search.*

Our implementation of dfs maintains a notion of traversal time for each vertex. Our time clock ticks each time we enter or exit a vertex. We keep track of the entry and exit times for each vertex.

Depth-first search has a neat recursive implementation, which eliminates the need to explicitly use a stack:

```
$\operatorname{DFS}(G, u)$
    state $[u]=$ "discovered"
    process vertex $u$ if desired
    time $=$ time +1
    entry $[u]=$ time
    for each vertex $v$ that is adjacent to $u$
        process edge $(u, v)$ if desired
        if state $[v]=$ "undiscovered" then
        $p[v]=u$
        $\operatorname{DFS}(G, v)$
    state $[u]=$ "processed"
    $\operatorname{exit}[u]=$ time
    time $=$ time +1
```

The time intervals have interesting and useful properties with respect to depth-first search:

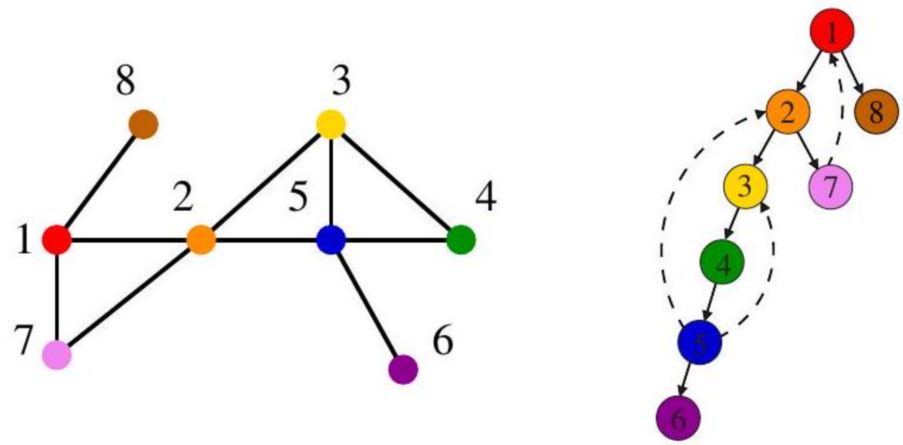


Figure 10.9: An undirected graph and its depth-first search tree. Dashed lines, which are not part of the tree, indicate back edges.

- *Who is an ancestor? - Suppose that x is an ancestor of y in the DFS tree. This implies that we must enter x before y, since there is no way we can be born before our own parent or grandparent! We also must exit y before we exit x, because the mechanics of DFS ensure we cannot exit x until after we have backed up from the search of all its descendants. Thus, the time interval of y must be properly nested within the interval of ancestor x.*

- *How many descendants? - The difference between the exit and entry times for v tells us how many descendants v has in the DFS tree. The clock gets incremented on each vertex entry and vertex exit, so half the time difference denotes the number of descendants of v.*

We will use these entry and exit times in several applications of depthfirst search, particularly topological sorting and biconnected/strongly connected components. We may need to take separate actions on each entry and exit, thus motivating distinct process_vertex_early and process_vertex_late routines called from dfs. For the DFS tree example presented in Figure 7.10 , the parent of each vertex with its entry and exit times are:

vertex	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*
parent	*-1*	*1*	*2*	*3*	*4*	*5*	*2*	*1*
entry	*1*	*2*	*3*	*4*	*5*	*6*	*11*	*14*
exit	*16*	*13*	*10*	*9*	*8*	*7*	*12*	*15*

The other important property of a depth-first search is that it partitions the edges of an undirected graph into exactly two classes: tree edges and back edges. The tree edges discover new vertices, and are those encoded in the parent relation. Back edges are those whose other endpoint is an ancestor of the vertex being expanded, so they point back into the tree.

An amazing property of depth-first search is that all edges fall into one of these two classes. Why can't an edge go to a sibling or cousin node, instead of an ancestor? All nodes reachable from a given vertex v are expanded before we

finish with the traversal from v, so such topologies are impossible for undirected graphs. This edge classification proves fundamental to the correctness of DFSbased algorithms.

Implementation Depth-first search can be thought of as breadth-first search, but using a stack instead of a queue to store unfinished vertices. The beauty of implementing dfs recursively is that recursion eliminates the need to keep an explicit stack:

```
void dfs(graph *g, int v) {
    edgenode *p; /* temporary pointer */
    int y; /* successor vertex */
    if (finished) {
        return; /* allow for search termination */
    }
    discovered[v] = true;
    time = time + 1;
    entry_time[v] = time;
    process_vertex_early(v);
    p = g->edges[v];
    while (p != NULL) {
        y = p->y;
        if (!discovered[y]) {
            parent[y] = v;
            process_edge(v, y);
            dfs(g, y);
        } else if (((!processed[y]) && (parent[v] != y)) || (g->directed)) {
            process_edge(v, y);
        }
        if (finished) {
            return;
        }
        p = p->next;
    }
    process_vertex_late(v);
    time = time + 1;
    exit_time[v] = time;
    processed[v] = true;
```

```
}
```

Depth-first search uses essentially the same idea as backtracking, which we study in Section 9.1 (page 281). Both involve exhaustively searching all possibilities by advancing if it is possible, and backing up only when there is no remaining unexplored possibility for further advance. Both are most easily understood as recursive algorithms.

Take-Home Lesson: DFS organizes vertices by entry/exit times, and edges into tree and back edges. This organization is what gives DFS its real power.

10.9 Applications of Depth-First Search

As algorithm design paradigms go, depth-first search isn't particularly intimidating. It is surprisingly subtle, however, meaning that its correctness requires getting details right.

The correctness of a DFS-based algorithm depends upon specifics of exactly when we process the edges and vertices. We can process vertex v either before we have traversed any outgoing edge from v (process_vertex_early ()), or after we have finished processing all of them (process_vertex_late()). Sometimes we will take special actions at both times, say process_vertex_early() to initialize a vertex-specific data structure, which will be modified on edge-processing operations and then analyzed afterwards using process_vertex_late().

In undirected graphs, each edge (x, y) sits in the adjacency lists of vertex x and y. There are thus two potential times to process each edge (x, y), namely when exploring x and when exploring y. The labeling of edges as tree edges or back edges occurs the first time the edge is explored. This first time we see an edge is usually a logical time to do edge-specific processing. Sometimes, we may want to take different action the second time we see an edge.

But when we encounter edge (x, y) from x, how can we tell if we have previously traversed the edge from y ? The issue is easy if vertex y is undiscovered: (x, y) becomes a tree edge so this must be the first time. The issue is also easy if y has been completely processed: we explored the edge when we explored y so this must be the second time. But what if y is an ancestor of x, and thus in a discovered state? Careful reflection will convince you that this must be our first traversal unless y is the immediate ancestor of x-that is, (y, x) is a tree edge. This can be established by testing if y == *parent* [x].

I find that the subtlety of depth-first search-based algorithms kicks me in the head whenever I try to implement one [2] *I encourage you to analyze these implementations carefully to see where the problematic cases arise and why.*

10.9.1 Finding Cycles

Back edges are the key to finding a cycle in an undirected graph. If there is no back edge, all edges are tree edges, and no cycle exists in a tree. But any back

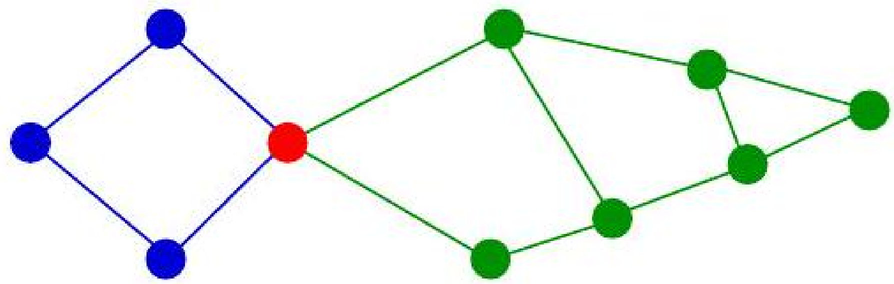


Figure 10.10: An *articulation vertex* is the weakest point in the graph.

edge going from x to an ancestor y creates a cycle with the tree path from y to x. Such a cycle is easy to find using dfs:

```
void process_edge(int x, int y) {
    if (parent[y] != x) { /* found back edge! */
        printf("Cycle from %d to %d:", y, x);
        find_path(y, x, parent);
        finished = true;
    }
}
```

The correctness of this cycle detection algorithm depends upon processing each undirected edge exactly once. Otherwise, a spurious two-vertex cycle (x, y, x) could be composed from the two traversals of any single undirected edge. We use the finished flag to terminate after finding the first cycle. Without it we would waste time discovering a new cycle with every back edge before stopping; a complete graph has $\Theta\left(n^2\right)$ such cycles.

10.9.2 Articulation Vertices

Suppose you are a vandal seeking to disrupt the telephone trunk line network. Which station in Figure 7.11 should you blow up to cause the maximum amount of damage? Observe that there is a single point of failure - a single vertex whose deletion disconnects a connected component of the graph. Such a vertex v is called an articulation vertex or cut-node. Any graph that contains an articulation vertex is inherently fragile, because deleting v causes a loss of connectivity between other nodes.

I presented a breadth-first search-based connected components algorithm in Section 7.7.1 (page 218). In general, the connectivity of a graph is the smallest number of vertices whose deletion will disconnect the graph. The connectivity is 1 if the graph has an articulation vertex. More robust graphs without such

[2] Indeed, the most horrifying errors in the previous edition of this book came in this section.

a vertex are said to be biconnected. Connectivity will be further discussed in Section 18.8 (page 568).

Testing for articulation vertices by brute force is easy. Temporarily delete each candidate vertex v, and then do a BFS or DFS traversal of the remaining graph to establish whether it is still connected. The total time for n such traversals is $O(n(m+n))$. There is a clever linear-time algorithm, however, that tests all the vertices of a connected graph using a single depth-first search.

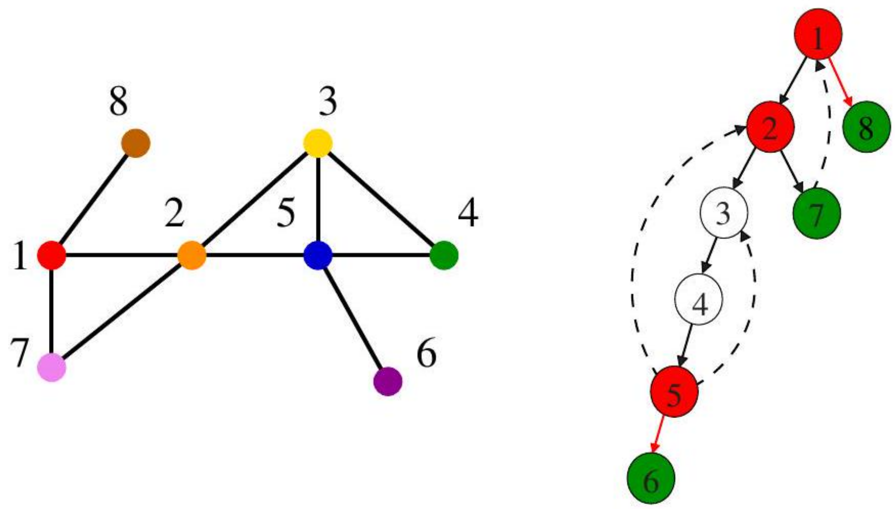


Figure 10.11: DFS tree of a graph containing three articulation vertices (namely 1,2 , and 5). Back edges keep vertices 3 and 4 from being cut-nodes, while green vertices 6,7 , and 8 escape by being leaves of the DFS tree. Red edges $(1, 8)$ and $(5, 6)$ are *bridges* whose deletion disconnect the graph.

What might the depth-first search tree tell us about articulation vertices? This tree connects all the vertices of a connected component of the graph. If the DFS tree represented the entirety of the graph, all internal (non-leaf) nodes would be articulation vertices, since deleting any one of them would separate a leaf from the root. But blowing up a leaf (shown in green in Figure 7.12 would not disconnect the tree, because it connects no one but itself to the main trunk.

The root of the search tree is a special case. If it has only one child, it functions as a leaf. But if the root has two or more children, its deletion disconnects them, making the root an articulation vertex.

General graphs are more complex than trees. But a depth-first search of a general graph partitions the edges into tree edges and back edges. Think of these back edges as security cables linking a vertex back to one of its ancestors. The security cable from x back to y ensures that none of the vertices on the tree path between x and y can be articulation vertices. Delete any of these vertices, and *the security cable will still hold all of them to the rest of the tree.*

Finding articulation vertices requires keeping track of the extent to which back edges (i.e., security cables) link chunks of the DFS tree back to ancestor nodes. Let reachable_ancestor [v] denote the earliest reachable ancestor of vertex v, meaning the oldest ancestor of v that we can reach from a descendant of v by using a back edge. Initially, reachable_ancestor [v] = v *:*

```
int reachable_ancestor[MAXV+1]; /* earliest reachable ancestor of v */
int tree_out_degree[MAXV+1]; /* DFS tree outdegree of v */
```

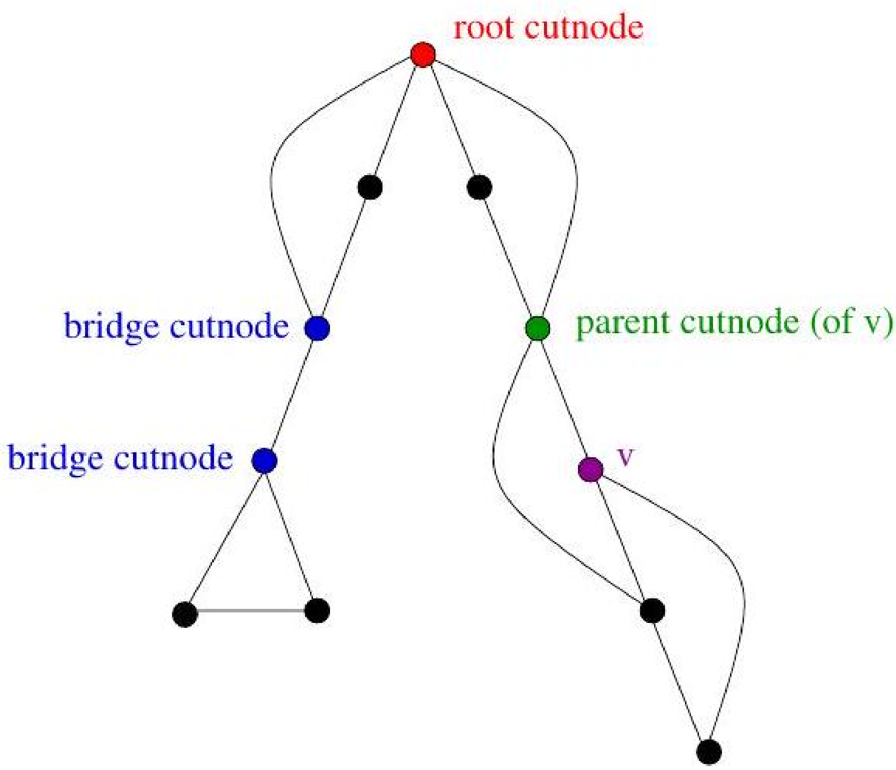


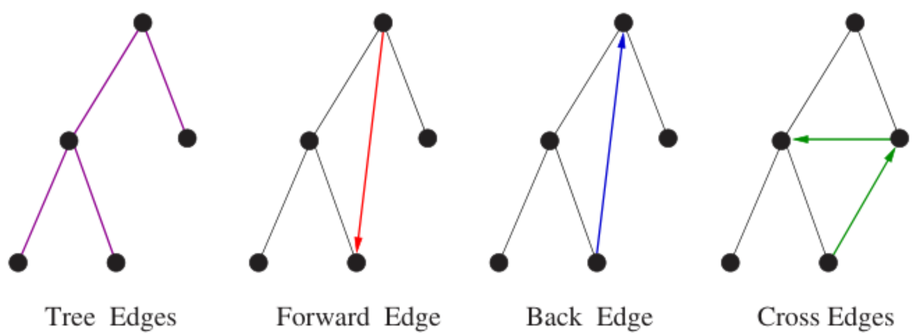
Figure 10.12: root, bridge, and parent cut-nodes.

```
void process_vertex_early(int v) {
    reachable_ancestor[v] = v;
}
```

We update reachable_ancestor [v] whenever we encounter a back edge that takes us to an earlier ancestor than we have previously seen. The relative age/rank of our ancestors can be determined from their entry_time's:

```
void process_edge(int x, int y) {
    int class; /* edge class */
    class = edge_classification(x, y);
    if (class == TREE) {
        tree_out_degree[x] = tree_out_degree[x] + 1;
    }
    if ((class == BACK) && (parent[x] != y)) {
        if (entry_time[y] < entry_time[reachable_ancestor[x]]) {
            reachable_ancestor[x] = y;
        }
    }
}
```

The key issue is determining how the reachability relation impacts whether vertex v is an articulation vertex. There are three cases, illustrated in Figure 7.13 and discussed below. Note that these cases are not mutually exclusive. A single vertex v might be an articulation vertex for multiple reasons:



- *Root cut-nodes - If the root of the DFS tree has two or more children, it must be an articulation vertex. No edges from the subtree of the second child can possibly connect to the subtree of the first child.*

- *Bridge cut-nodes - If the earliest reachable vertex from v is v, then deleting the single edge (parent $[v], v$) disconnects the graph. Clearly parent $[v]$ must be an articulation vertex, since it cuts v from the graph. Vertex v is also an articulation vertex unless it is a leaf of the DFS tree. For any leaf, nothing falls off when you cut it.*

- *Parent cut-nodes - If the earliest reachable vertex from v is the parent of v, then deleting the parent must sever v from the tree unless the parent is the root. This is always the case for the deeper vertex of a bridge, unless it is a leaf.*

The routine below systematically evaluates each of these three conditions as we back up from the vertex after traversing all outgoing edges. We use entry_time [v] to represent the age of vertex v. The reachability time time_v *calculated below denotes the oldest vertex that can be reached using back edges. Getting back to an ancestor above v rules out the possibility of v being a cutnode:*

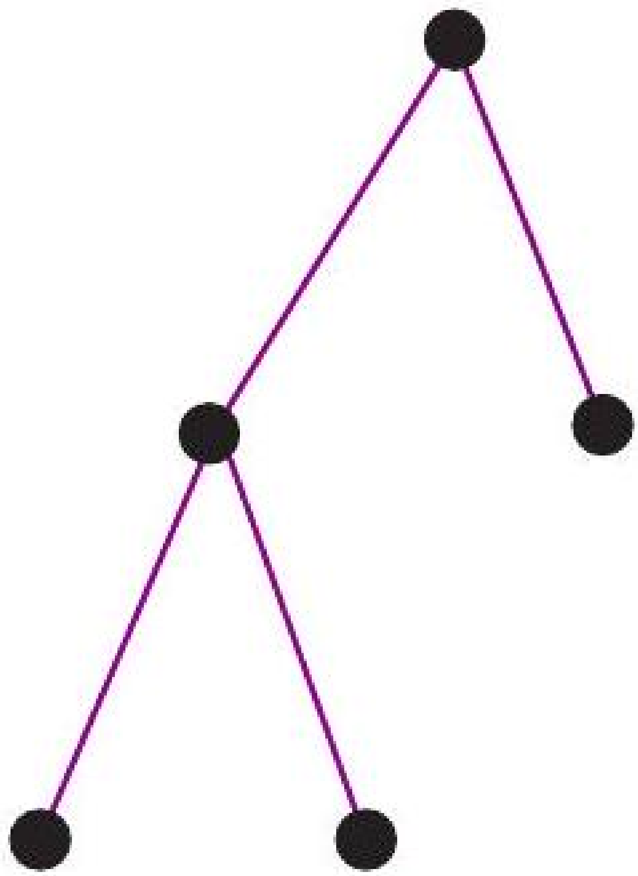
```
void process_vertex_late(int v) {
    bool root; /* is parent[v] the root of the DFS tree? */
    int time_v; /* earliest reachable time for v */
    int time_parent; /* earliest reachable time for parent[v] */
    if (parent[v] == -1) { /* test if v is the root */
        if (tree_out_degree[v] > 1) {
            printf("root articulation vertex: %d \n",v);
        }
        return;
    }
    root = (parent[parent[v]] == -1); /* is parent[v] the root? */
    if (!root) {
        if (reachable_ancestor[v] == parent[v]) {
            printf("parent articulation vertex: %d \n", parent[v]);
        }
        if (reachable_ancestor[v] == v) {
            printf("bridge articulation vertex: %d \n",parent[v]);
            if (tree_out_degree[v] > 0) { /* is v is not a leaf? */
                printf("bridge articulation vertex: %d \n", v);
```

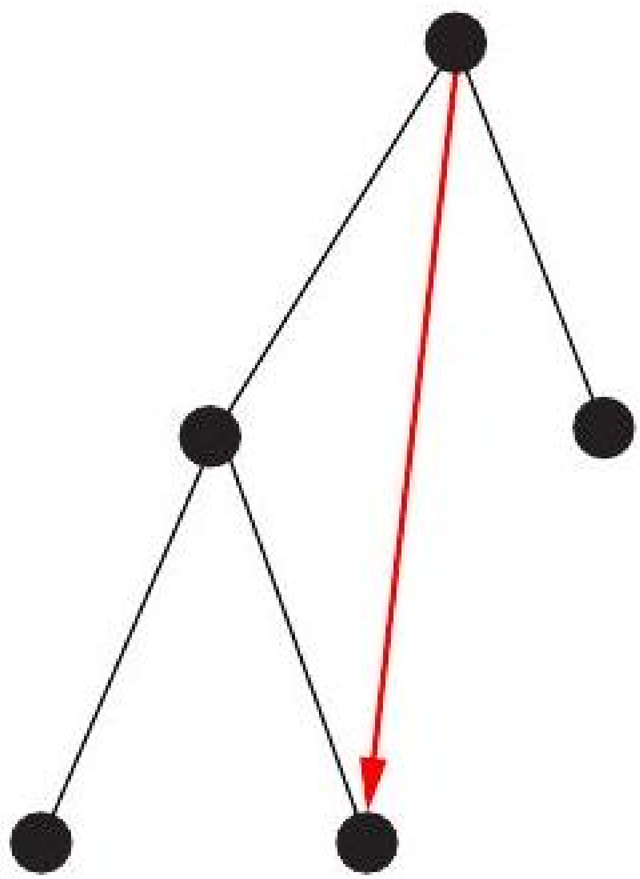
Tree Edges
Forward Edge
Back Edge
Cross Edges }
}
}
time_v = entry_time[reachable_ancestor[v]];
time_parent = entry_time[reachable_ancestor[parent[v]]];
if (time_v < time_parent) { reachable_ancestor[parent[v]] = reachable_ancestor[v];
}

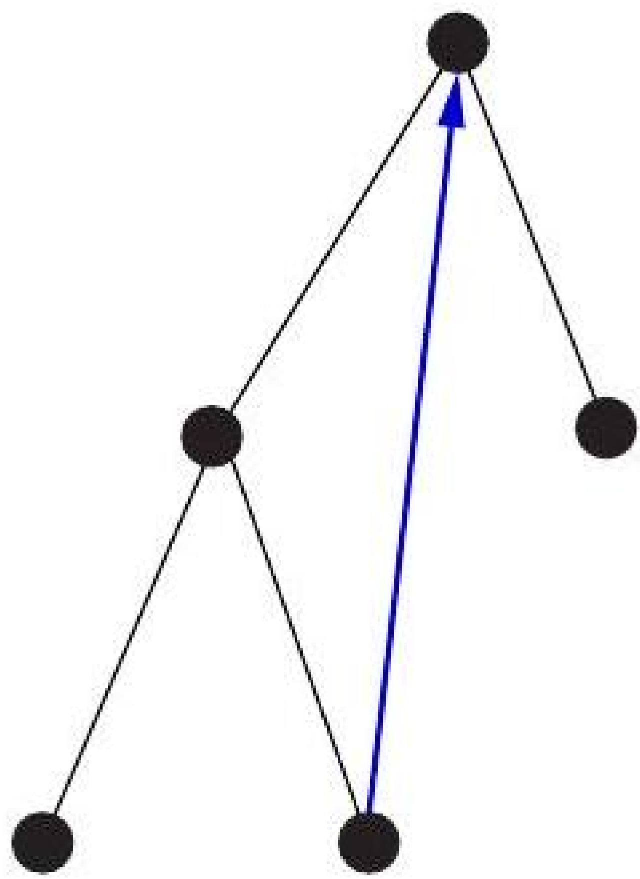
The last lines of this routine govern when we back up from a node's highest reachable ancestor to its parent, namely whenever it is higher than the parent's earliest ancestor to date.

We can alternatively talk about vulnerability in terms of edge failures instead of vertex failures. Perhaps our vandal would find it easier to cut a cable instead of blowing up a switching station. A single edge whose deletion disconnects the graph is called a bridge; any graph without such an edge is said to be edgebiconnected.

Identifying whether a given edge (x, y) *is a bridge is easily done in linear time, by deleting the edge and testing whether the resulting graph is connected. In fact all bridges can be identified in the same* $O(n+m)$ *time using DFS. Edge*







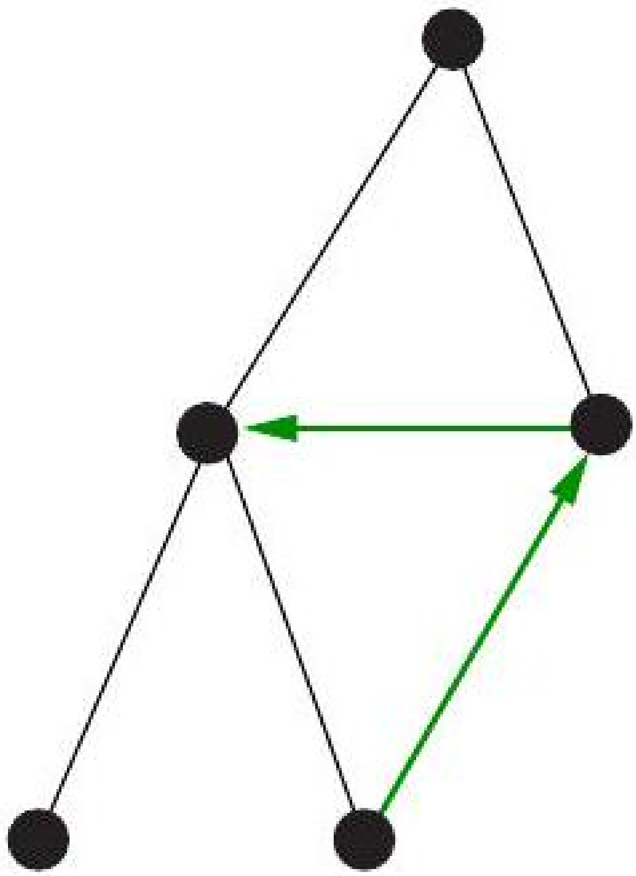


Figure 10.13: The possible edge cases for graph traversal. Forward edges and cross edges can occur in DFS only on *directed graphs*.

(x, y) is a bridge if (1) it is a tree edge, and (2) no back edge connects from y or below to x or above. This can be computed with a appropriate modification to the process_late_vertex function.

10.10 Depth-First Search on Directed Graphs

Depth-first search on an undirected graph proves useful because it organizes the edges of the graph in a very precise way. Over the course of a DFS from a given source vertex, each edge will be assigned one of potentially four labels, as shown in Figure 7.14

When traversing undirected graphs, every edge is either in the depth-first search tree or will be a back edge to an ancestor in the tree. It is important to understand why. Might we encounter a "forward edge" (x, y), directed toward a descendant vertex? No, because in this case, we would have first traversed (x, y) while exploring y, making it a back edge. Might we encounter a "cross edge" (x, y), linking two unrelated vertices? Again no, because we would have first discovered this edge when we explored y, making it a tree edge.

But for directed graphs, depth-first search labelings can take on a wider range of possibilities. Indeed, all four of the edge cases in Figure 7.14 can occur in traversing directed graphs. This classification still proves useful in organizing algorithms on directed graphs, because we typically take a different action on edges from each different class.

The correct labeling of each edge can be readily determined from the state, discovery time, and parent of each vertex, as encoded in the following function:

```
int edge_classification(int x, int y) {
    if (parent[y] == x) {
        return(TREE);
    }
    if (discovered[y] && !processed[y]) {
        return(BACK);
    }
    if (processed[y] && (entry_time[y]>entry_time[x])) {
        return(FORWARD);
    }
    if (processed[y] && (entry_time[y]<entry_time[x])) {
        return(CROSS);
    }
    printf("Warning: self loop (%d,%d)\n", x, y);
    return -1;
}
```

Just as with BFS, this implementation of the depth-first search algorithm includes places to optionally process each vertex and edge - say to copy them, print them, or count them. Both DFS and BFS will traverse all edges in the *same*

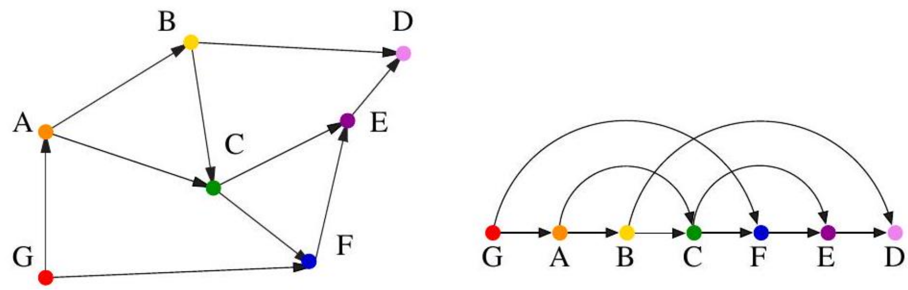


Figure 10.14: A DAG with only one topological sort (G, A, B, C, F, E, D)

connected component as the starting point. Both must start with a vertex in each component to traverse a disconnected graph. The only important difference between them is the way they organize and label the edges.

I encourage the reader to convince themselves of the correctness of the four conditions above. What I said earlier about the subtlety of depth-first search goes double for directed graphs.

10.10.1 Topological Sorting

Topological sorting is the most important operation on directed acyclic graphs (DAGs). It orders the vertices on a line such that all directed edges go from left to right. Such an ordering cannot exist if the graph contains a directed cycle, because there is no way you can keep moving right on a line and still return back to where you started from!

Each DAG has at least one topological sort. The importance of topological sorting is that it gives us an ordering so we can process each vertex before any of its successors. Suppose the directed edges represented precedence constraints, such that edge (x, y) means job x must be done before job y. Any topological sort then defines a feasible schedule. Indeed, there can be many such orderings for a given DAG.

But the applications go deeper. Suppose we seek the shortest (or longest) path from x to y in a DAG. No vertex v appearing after y in the topological order can possibly contribute to any such path, because there will be no way to get from v back to y. We can appropriately process all the vertices from left to right in topological order, considering the impact of their outgoing edges, and know that we will have looked at everything we need before we need it. Topological sorting proves very useful in essentially any algorithmic problem on DAGs, as discussed in the catalog in Section 18.2 (page 546 .)

Topological sorting can be performed efficiently using depth-first search. A directed graph is a DAG iff no back edges are encountered. Labeling the vertices in the reverse order that they are marked processed defines a topological sort of a

DAG. Why? Consider what happens to each directed edge (x, y) as we encounter it exploring vertex x :

- *If y is currently undiscovered, then we start a DFS of y before we can continue with x. Thus, y must be marked processed before x is, so x appears before y in the topological order, as it must.*
- *If y is discovered but not processed, then (x, y) is a back edge, which is impossible in a DAG because it creates a cycle.*
- *If y is processed, then it will have been so labeled before x. Therefore, x appears before y in the topological order, as it must.*

Study the following implementation:

```
void process_vertex_late(int v) {
    push(&sorted, v);
}

void process_edge(int x, int y) {
```

```
    void process_edge(int x, int y) {
int class; /* edge class */
class = edge_classification(x, y);
if (class == BACK) {
printf("Warning: directed cycle found, not a DAG\n");
}
}
```

```
void topsort (graph $*$ g) \{
    int i; /* counter */
    init_stack(\&sorted);
    for (i = 1; i <= g->nvertices; i++) \{
        if (!discovered[i]) \{
            dfs(g, i);\index{Prims algorithm}
        \}
    \}
    print_stack(\&sorted); /* report topological order */
\}
```

We push each vertex onto a stack as soon as we have evaluated all outgoing edges. The

\subsection{Strongly Connected Components}

A directed graph is strongly connected if there is a directed path between any two ver
be places you can drive to but not drive home from without violating one-way signs.

%---- Page End Break Here ---- Page : 246

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{images/algorithms_page_0248_0.png}
\end{figure}
It is straightforward to use graph traversal to test whether a graph $G=$ (V, E) is strongly co

To test if there are paths from every vertex to v, we construct the transpose graph $G^{T}=\lef

graph *transpose(graph *g) {
graph gt; / transpose of graph g /
int x; / counter */
edgenode p; / temporary pointer */
gt = (graph) malloc(sizeof(graph));
initialize_graph(gt, true); / initialize directed graph */
gt->nvertices = g->nvertices;
for (x = 1; x <= g->nvertices; x++) {
p = g->edges[x];
while (p != NULL) {
insert_edge(gt, p->y, x, true);
p = p->next;
}
}
return(gt);
}

Any path from v to z in G^{T} corresponds to a path from z to v in G. By doing a seco

All directed graphs can be partitioned into strongly connected components, such that a directed p

void strong_components(graph *g) {
graph gt; / transpose of graph g /
int i; / counter /
int v; / vertex in component */

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-248.jpg?height=391&width=399

G
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-248.jpg?height=484&width=238
$\operatorname{DFS}(G)$
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-248.jpg?height=319&width=345
$\operatorname{DFS}\left(G^{T}\right)$

Figure 7.16: The strongly connected components of a graph G (left), with its associated DFS tre

init_stack(&dfs1order);

```
initialize_search(g);
for (i = 1; i <= (g->nvertices); i++) {
if (!discovered[i]) {
    dfs(g, i);
}

    }
gt = transpose(g);
initialize_search(gt);
components_found = 0;
while (!empty_stack(&dfs1order)) {
v = pop(&dfs1order);
if (!discovered[v]) {
components_found ++;
printf("Component %d:", components_found);
dfs2(gt, v);
printf("\n");
}
}
}
```

The first traversal pushes the vertices on a stack in the reverse order they were proce
connection makes sense: DAGs are directed graphs where each vertex forms its own strong

```
    void process_vertex_late(int v) {
push(&dfs1order,v);
}
```

The second traversal, on the transposed graph, behaves like the connected component alg

```
    void process_vertex_early2(int v) {
printf(" %d", v);
}
```

The correctness of this is subtle. Observe that first DFS places vertices on the stack

Chapter Notes

Our treatment of graph traversal represents an expanded version of material from chapte

\section{Exercises}

Simulating Graph Algorithms

7-1. [3] For the following weighted graphs G_{1} (left) and G_{2} (right):
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-250.jpg?height=449
(a) Report the order of the vertices encountered on a breadth-first search starting fro

(b) Report the order of the vertices encountered on a depth-first search starting from vertex $A

%---- Page End Break Here ---- Page : 249
7-2. [3] Do a topological sort of the following graph G :
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-250.jpg?height=419&width=527

Traversal

7-3. [3] Prove that there is a unique path between any pair of vertices in a tree.
7-4. [3] Prove that in a breadth-first search on a undirected graph G, every edge is either a t
7-5. [3] Give a linear algorithm to compute the chromatic number of graphs where each vertex has

7-6. [3] You are given a connected, undirected graph G with n vertices and m edges. Give ar
7-7. [5] In breadth-first and depth-first search, an undiscovered node is marked discovered when
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-251.jpg?height=406&width=105

%---- Page End Break Here ---- Page : 250
Figure 7.17: Expression $2+3 * 4+(3 * 4) / 5$ as a tree and a DAG
(a) Describe a graph on n vertices and a particular starting vertex v such that $\Theta(n)$ n
(b) Describe a graph on n vertices and a particular starting vertex v such that $\Theta(n)$ n
(c) Describe a graph on n vertices and a particular starting vertex v such that at some point
7-8. [4] Given pre-order and in-order traversals of a binary tree (discussed in Section 3.4.1), i
7-9. [3] Present correct and efficient algorithms to convert an undirected graph G between the
(a) Convert from an adjacency matrix to adjacency lists.
(b) Convert from an adjacency list representation to an incidence matrix. An incidence matrix $M
(c) Convert from an incidence matrix to adjacency lists.

7-10. [3] Suppose an arithmetic expression is given as a tree. Each leaf is an integer and each i
7-11. [5] Suppose an arithmetic expression is given as a DAG (directed acyclic graph) with common

%---- Page End Break Here ---- Page : 251
7-12. [8] The war story of Section 7.4 (page 210 describes an algorithm for constructing the dual

Applications

7-13. [3] The Chutes and Ladders game has a board with n cells where you seek to travel from ce
7-14. [3] Plum blossom poles are a Kung Fu training technique, consisting of n large posts part
7-15. [5] You are planning the seating arrangement for a wedding given a list of guests, V. For

Algorithm Design

7-16. [5] The square of a directed graph $G=(V, E)$ is the graph $G^{2}=\left(V, E^{2}\right)$ su
Give efficient algorithms for both adjacency lists and matrices.

7-17. [5] A vertex cover of a graph $G=(V, E)$ is a subset of vertices $V^{\prime}$ suc
(a) Give an efficient algorithm to find a minimum-size vertex cover if G is a tree.
(b) Let $G=(V, E)$ be a tree such that the weight of each vertex is equal to the degre
(c) Let $G=(V, E)$ be a tree with arbitrary weights associated with the vertices. Give

7-18. [3] A vertex cover of a graph $G=(V, E)$ is a subset of vertices $V^{\prime}$ suc

%---- Page End Break Here ---- Page : 252
7-19. [5] An independent set of an undirected graph $G=(V, E)$ is a set of vertices $US
(a) Give an efficient algorithm to find a maximum-size independent set if G is a tre
(b) Let $G=(V, E)$ be a tree with weights associated with the vertices such that the we
(c) Let $G=(V, E)$ be a tree with arbitrary weights associated with the vertices. Give

7-20. [5] A vertex cover of a graph $G=(V, E)$ is a subset of vertices $V^{\prime}$ suc
An independent vertex cover is a subset of vertices that is both an independent set and
7-21. [5] Consider the problem of determining whether a given undirected graph $G=$ $(V
(a) Give an $O\left(|V|^{3}\right)$ algorithm to find a triangle if one exists.
(b) Improve your algorithm to run in time $O(|V| \cdot|E|)$. You may assume $|V| \leq|E
Observe that these bounds give you time to convert between the adjacency matrix and adj
7-22. [5] Consider a set of movies $M_{1}, M_{2}, \ldots, M_{k}$. There is a set of cus
You must decide which movies should be televised on Saturday and which on Sunday, so th
$7-23$. [5] The diameter of a tree $T=(V, E)$ is given by
$$
\max _{u, v \in V} \delta(u, v)
$$
(where $\delta(u, v)$ is the number of edges on the path from u to v). Describe an
7-24. [5] Given an undirected graph G with n vertices and m edges, and an integer

7-25. [6] Let v and w be two vertices in an unweighted directed graph $G=(V, E)$. F
7-26. [6] Design a linear-time algorithm to eliminate each vertex v of degree 2 from

%---- Page End Break Here ---- Page : 253
Directed Graphs

7-27. [3] The reverse of a directed graph $G=(V, E)$ is another directed graph $G^{R}=(

7-28. [5] Your job is to arrange n ill-behaved children in a straight line, facing fi
(a) Give an algorithm that orders the line (or says that it is not possible) in $O(m+n)
(b) Suppose instead you want to arrange the children in rows such that if i hates j

7-29. [3] A particular academic program has n required courses, certain pairs of whic
7-30. [5] Gotcha-solitaire is a game on a deck with n distinct cards (all face up) an
7-31. [5] You are given a list of n words each of length k in a language you don't
For example, if the strings are $\{Q Q Z, Q Z Z, X Q Z, X Q X, X X X\}$, the character

(a) Give an algorithm to efficiently reconstruct this character order. (Hint: use a graph structu
(b) What is its running time, as a function of n, k, and α ?

7-32. [3] A weakly connected component in a directed graph is a connected component ignoring the

%---- Page End Break Here ---- Page : 254
7-33. [5] Design a linear-time algorithm that, given an undirected graph G and a particular edg
7-34. [5] An arborescence of a directed graph G is a rooted tree such that there is a directed
7-35. [5] A mother vertex in a directed graph $G=(V, E)$ is a vertex v such that all other vert
(a) Give an $O(n+m)$ algorithm to test whether a given vertex v is a mother of G, where $n=|V
(b) Give an $O(n+m)$ algorithm to test whether graph G contains a mother vertex.

7-36. [8] Let G be a directed graph. We say that G is k-cyclic if every (not necessarily si
7-37. [9] A tournament is a directed graph formed by taking the complete undirected graph and ass

Articulation Vertices

7-38. [5] An articulation vertex of a connected graph G is a vertex whose deletion disconnects
7-39. [5] Following up on the previous problem, give an $O(n+m)$ algorithm that finds a deletion
7-40. [3] Suppose G is a connected undirected graph. An edge e whose removal disconnects the
7-41. [5] A city that only has two-way streets has decided to change them all into oneway streets
(a) Let G be the original undirected graph. Prove that there is a way to properly orient/direct
(b) Give an efficient algorithm to orient the edges of a bridgeless graph G so the result is st

Interview Problems

7-42. [3] Which data structures are used in depth-first and breath-first search?
7-43. [4] Write a function to traverse binary search tree and return the i th node in sorted or

LeetCode

%---- Page End Break Here ---- Page : 255
7-1. https://leetcode.com/problems/minimum-height-trees/
7-2. https://leetcode.com/problems/redundant-connection/
7-3. https://leetcode.com/problems/course-schedule/

HackerRank

7-1. https://www.hackerrank.com/challenges/bfsshortreach/
7-2. https://www.hackerrank.com/challenges/dfs-edges/
7-3. https://www.hackerrank.com/challenges/even-tree/\index{clustering}

Programming Challenges

These programming challenge problems with robot judging are available at https://onlin
7-1. "Bicoloring" - Chapter 9, problem 10004.
7-2. "Playing with Wheels"-Chapter 9, problem 10067.
7-3. "The Tourist Guide" - Chapter 9, problem 10099.
7-4. "Edit Step Ladders" - Chapter 9, problem 10029.
7-5. "Tower of Cubes" - Chapter 9, problem 10051.

%---- Page End Break Here ---- Page : 256

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{algorithms_page_0258_1.png}
\end{figure}
Chapter 8

\chapter{Weighted Graph Algorithms}

The data structures and traversal algorithms of Chapter 7 provide the basic building bl

There is an alternate universe of problems for weighted graphs. The edges of road netw

The graph data structure from Chapter 7 quietly supported edge-weighted graphs, but he

typedef struct {
edgenode edges[MAXV+1]; / adjacency info /
int degree[MAXV+1]; / outdegree of each vertex /
int nvertices; / number of vertices in the graph /
int nedges; / number of edges in the graph /
*int directed; / is the graph directed? */*
} graph;

Each edgenode is a record containing three fields, the first describing the second endp
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-258.jpg?height=51

Figure 8.1: (a) The distances between points define a complete weighted graph, (b) its

typedef struct edgenode {
int y; / adjacency info /*
*int weight; / edge weight, if any */*
*struct edgenode next; / next edge in list */*
} edgenode;

We now describe several sophisticated algorithms for weighted graphs that use this dat

\section{Minimum Spanning Trees}

A spanning tree of a connected graph $G=(V, E)$ is a subset of edges from E forming a tree conn

Minimum spanning trees are the answer whenever we need to connect a set of points (representing c

A minimum spanning tree minimizes the total edge weight over all possi-
ble spanning trees. However, there can be more than one minimum spanning tree of a given graph. I

%---- Page End Break Here ---- Page : 258
\subsection{Prim's Algorithm}

Prim's minimum spanning tree algorithm starts from one vertex and grows the rest of the tree one

Greedy algorithms make the decision of what to do next by selecting the best local option from al

Prim-MST(G)

Select an arbitrary vertex s to start the tree $T_{\text {prim }}$ from.
While (there are still non-tree vertices)
Find the minimum-weight edge between a tree and non-tree vertex Add the selected edge and vertex

Prim's algorithm clearly creates a spanning tree, because no cycle can be introduced by adding ed

We use proof by contradiction. Suppose that there existed a graph G for which Prim's algorithm

But how could we have gone wrong? There must be a path p from x to y in $T_{\text {min }}$,
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-260.jpg?height=423&width=974

%---- Page End Break Here ---- Page : 259
Figure 8.2: Does Prim's algorithm go bad? No, because picking edge (x, y) before ($v_{1}, v_

Implementation

Prim's algorithm grows the minimum spanning tree in stages, starting from a given vertex. At each

Our implementation is somewhat smarter. It keeps track of the cheapest edge linking every non-tre

int prim(graph g, int start) {
*int i; / counter */*
edgenode p; / temporary pointer /
bool intree[MAXV+1]; / is the vertex in the tree yet? /
int distance[MAXV+1]; / cost of adding to tree /
int v; / current vertex to process /
int w; / candidate next vertex /
int dist; / cheapest cost to enlarge tree /

*int weight = 0; / tree weight */*
for (i = 1; i <= g->nvertices; i++) {
intree[i] = false;
distance[i] = MAXINT;
parent[i] = -1;

```
}
distance[start] = 0;
v = start;
while (!intree[v]) {
intree[v] = true;
if (v != start) {
    printf("edge (%d,%d) in tree \n",parent[v],v);
    weight = weight + dist;
}
p = g->edges[v];
while (p != NULL) {
    w = p->y;
    if ((distance[w] > p->weight) && (!intree[w])) {
        distance[w] = p->weight;
        parent[w] = v;
    }
    p = p->next;
}
dist = MAXINT;
for (i = 1; i <= g->nvertices; i++) {
    if ((!intree[i]) && (dist > distance[i])) {
        dist = distance[i];
        v = i;
    }
}
```

}
return(weight);
}

```
Analysis

Prim's algorithm is correct, but how efficient is it? This depends on which data struc

But our implementation avoids the need to test all $m$ edges on each pass. It only cons
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-262.jpg?height=33


%---- Page End Break Here ---- Page : 261
```

Figure 8.3: A graph G (left) with minimum spanning trees produced by Prim's (center) and Kruska

The result is an $O\left(n^{2}\right)$ implementation of Prim's algorithm, and a good illustratic

The minimum spanning tree itself can be reconstructed in two different ways. The simplest method

\subsection{Kruskal's Algorithm}

Kruskal's algorithm is an alternative approach to finding minimum spanning trees that proves more

Kruskal's algorithm builds up connected components of vertices, culminating in the complete minim

Kruskal-MST(G)
Put the edges into a priority queue ordered by increasing weight. count $=0$
while (count $<n-1$) do
get next edge (v, w)
if $(\operatorname{component}(v) \neq \operatorname{component}(w))$
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-263.jpg?height=469&width=106

%---- Page End Break Here ---- Page : 262
Figure 8.4: Could Kruskal's algorithm go bad after selecting red edge (x, y) (on left)? No, bec

> increment count
> add (v, w) to $T_{\text {kruskal }}$
> merge component (v) and $\operatorname{component}(w)$

This algorithm adds $n-1$ edges without creating a cycle, so it must create a spanning tree for a

What is the time complexity of Kruskal's algorithm? Sorting the m edges takes $O(m \lg m)$ time

However, a faster implementation results if we can implement the component test in faster than $O

%---- Page End Break Here ---- Page : 263

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{algorithms_page_0265_1.png}
\end{figure}
Implementation

The implementation of the main routine follows directly from the pseudocode:

int kruskal(graph g) {
int i; / counter /

```
union_find s; / union-find data structure /
edge_pair e[MAXV+1]; / array of edges data structure /
int weight=0; / cost of the minimum spanning tree */
union_find_init(&s, g->nvertices);
to_edge_array(g, e);
qsort(&e,g->nedges, sizeof(edge_pair), &weight_compare);
for (i = 0; i < (g->nedges); i++) {
if (!same_component(&s, e[i].x, e[i].y)) {
printf("edge (%d,%d) in MST\n", e[i].x, e[i].y);
weight = weight + e[i].weight;
union_sets(&s, e[i].x, e[i].y);
}
}
return(weight);
}
```

The Union-Find Data Structure

A set partition parcels out the elements of some universal set (say the integers 1 to

The connected components in a graph can be represented as a set partition. For Kruskal

- Same component $\left(v_{1}, v_{2}\right)$ - Do vertices v_{1} and v_{2} occur i
- Merge components $\left(C_{1}, C_{2}\right)$ - Merge the given pair of connected com

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-265.jpg?height=37

%---- Page End Break Here ---- Page : 264

Figure 8.5: Union-find example - the structure represented as a forest of trees (left)

The two obvious data structures for this task each support only one of these operation

The union-find data structure represents each subset as a "backwards" tree, with point

```
typedef struct {
int p[SET_SIZE+1]; /* parent element /
int size[SET_SIZE+1]; / number of elements in subtree i /
int n; / number of elements in set */
} union_find;
```

We implement our desired component operations in terms of two simpler operations, unio

- Find(i) - Find the root of the tree containing element i, by walking up the pa
- Union (i, j) - Link the root of one of the trees (say containing $i)$ to the root

We seek to minimize the time it takes to execute the worst possible sequence of unions

%---- Page End Break Here ---- Page : 265

To minimize the tree height, it is better to make the smaller tree the subtree of the bigger one.

Implementation

The implementation details are as follows:

```
void union_find_init(union_find s, int n) {
int i; / counter */
for (i = 1; i <= n; i++) {
s->p[i] = i;
s->size[i] = 1;
}
s->n = n;
}
int find(union_find *s, int x) {
if (s->p [x] == x) {
return(x);
}
return(find(s, s->p[x]));
}
void union_sets(union_find s, int s1, int s2) {
int r1, r2; / roots of sets /
r1 = find(s, s1);
r2 = find(s, s2);
if (r1 == r2) {
return; / already in same set */
}
if (s->size[r1] >= s->size[r2]) {
s->size[r1] = s->size[r1] + s->size[r2];
s->p[r2] = r1;
} else {
s->size[r2] = s->size[r1] + s->size[r2];
s->p[r1] = r2;
}
}
```

```
bool same_component(union_find *s, int s1, int s2) {
return (find(s, s1) == find(s, s2));
}
```

Analysis

On each union, the tree with fewer nodes becomes the child. But how tall can such a tree get as a

See the pattern? We must double the number of nodes in the tree to get an extra unit of height. H

\subsection{Variations on Minimum Spanning Trees}

The algorithms that construct minimum spanning trees can also be used to solve several
- Maximum spanning trees - Suppose an evil telephone company is contracted to connect a

Most graph algorithms do not adapt so easily to negative numbers. Indeed, shortest path
- Minimum product spanning trees - Suppose we seek the spanning tree that minimizes the
- Minimum bottleneck spanning tree - Sometimes we seek a spanning tree that minimizes t

%---- Page End Break Here ---- Page : 267
Such bottleneck spanning trees have interesting applications when the edge weights are

The minimum spanning tree of a graph is unique if all m edge weights in the graph are

There are two important variants of a minimum spanning tree that are not solvable with
- Steiner tree - Suppose we want to wire a bunch of houses together, but have the freed
- Low-degree spanning tree - Alternatively, what if we want to find the minimum spannin

\section{War Story: Nothing but Nets}

I'd been tipped off about a small printed circuit board testing company in need of some
"We're leaders in robotic printed circuit board testing devices. Our customers have ver
"How do you do the testing?" I asked.
"We have a robot with two arms, each with electric probes. The arms simultaneously cont
"Wait!" I cried. "What is a net?"
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-269.jpg?height=56

%---- Page End Break Here ---- Page : 268
Figure 8.6: An example net showing (a) the metal connection layer, (b) the contact poin
"Circuit boards have certain sets of points that are all connected together with a meta
"I see. So you have a list of all the connections between pairs of points on the circui

He shook his head. "Not quite. The input for our testing program consists only of the n

I thought for a moment. "OK. So your right arm has to visit all the other points in the

The technical guy spoke up. "Well, we sort the points from left to right and then go in
"Have you ever heard of the traveling salesman problem?" I asked.
He was an electrical engineer, not a computer scientist. "No, what's that?"
"Traveling salesman is the name of the problem that you are trying to solve. Given a se

The president scribbled down some notes and then frowned. "Fine. Maybe
you can order the points in a net better for us. But that's not our real problem. When

%---- Page End Break Here ---- Page : 269
I sat down and thought. The left and right arms each have interlocking TSPs to solve. The left ar
"You are right. We should win if we can break big nets into small nets. We want the nets to be sm

We thus had to break each net into overlapping pieces, where each piece was small. This is a clus

So I explained the idea of constructing the minimum spanning tree of a graph. The boss listened,
"I like your clustering idea. But minimum spanning trees are defined on graphs. All you've got ar
"Oh, we can think of it as a complete graph, where every pair of points is connected. The weight

I went back to thinking. The edge cost should reflect the travel time between two points. While d
"Hey. I have a question about your robot. Does it take the same amount of time to move the arm le

They thought a minute. "Sure it does. We use the same type of motor to control horizontal and ver
"So the time to move both one foot left and one foot up is exactly the same
as just moving one foot left? This means that the weight for each edge should not be the Euclidea
"Well, it takes some time for the arm to come up to full speed. I guess we should factor in its a
"Darn right. The more accurately you can model the time your arm takes to move between two points

%---- Page End Break Here ---- Page : 270
They were somewhat skeptical about whether this approach would do any good. But a few weeks later

The key idea here was modeling the job in terms of classical algorithmic graph problems. I smelle

Take-Home Lesson: Most applications of graphs can be reduced to standard graph properties where w

\section{Shortest Paths}

A path is a sequence of edges connecting two vertices. There are typically an enormous number of

A shortest path from s to t in an unweighted graph can be identified using a breadth-first se

But BFS does not suffice to find shortest paths in weighted graphs. The shortest weighted path mi
route from home to office may involve complicated backroad shortcuts, as shown in Figure 8.7.

%---- Page End Break Here ---- Page : 271
This section will present two distinct algorithms for finding the shortest paths in weighted grap

\subsection{Dijkstra's Algorithm}

Dijkstra's algorithm is the method of choice for finding shortest paths in an edge- and/or vertex

Suppose the shortest path from s to t in graph G passes through a particular int

Dijkstra's algorithm proceeds in a series of rounds, where each round establishes the s

This suggests a dynamic programming-like strategy. The shortest path from s to itsel

ShortestPath-Dijkstra(G, s, t)
known $= \{s\}$
for each vertex v *in* $G, \operatorname{dist}[v]=\infty$
$\operatorname{dist}[s]=0$
for each edge (s, v), *dist* $[v]=w(s, v)$
last $= s$
while (*last* $\neq t$)
select v_{next} , *the unknown vertex minimizing* $\operatorname{dist}[v]$
for each edge $\left(v_{next}, x\right), \operatorname{dist}[x]=\min \left[\operatorname{dist}[x], \operatorname{dist}\left[v_{next}\right]+w\left(v_{next}, x\right)\right]$
last $= v_{next}$
k nown $=$ *known* $\cup \left\{v_{next}\right\}$

The basic idea is very similar to Prim's algorithm. In each iteration, we add exactly o

\footnotetext{
${ }^{1}$ Actually, this is true only when the graph does not contain negative weight e
}
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-273.jpg?height=37

%---- Page End Break Here ---- Page : 272
Figure 8.7: The shortest path from s to t might pass through many intermediate ver

In fact, the only difference between Dijkstra's and Prim's algorithms is how they rate

Implementation

The pseudocode above obscures just how similar the two algorithms are. Below, we give a

```
int dijkstra(graph g, int start) {
int i; / counter */
edgenode p; / temporary pointer /
bool intree[MAXV+1]; / is the vertex in the tree yet? /
int distance[MAXV+1]; / cost of adding to tree /
int v; / current vertex to process /
int w; / candidate next vertex /
int dist; / cheapest cost to enlarge tree /
int weight = 0; / tree weight */
for (i = 1; i <= g->nvertices; i++) {
intree[i] = false;
distance[i] = MAXINT;
```

parent[i] = -1;
}
distance[start] = 0;
v = start;

```
while (!intree[v]) {
intree[v] = true;
if (v != start) {
    printf("edge (%d,%d) in tree \n",parent[v],v);
    weight = weight + dist;
}
p = g->edges[v];
while (p != NULL) {
    w = p->y;
    if (distance[w] > (distance[v]+p->weight)) { /* CHANGED */
            distance[w] = distance[v]+p->weight; /* CHANGED */
            parent[w] = v; /* CHANGED */
    }
    p = p->next;
}
dist = MAXINT;
for (i = 1; i <= g->nvertices; i++) {
    if ((!intree[i]) && (dist > distance[i])) {
        dist = distance[i];
        v = i;
    }
}
```

}
return(weight);
}

```
This algorithm defines a shortest-path spanning tree rooted in $s$. For unweighted graphs, this w

Analysis

What is the running time of Dijkstra's algorithm? As implemented here, the complexity is $O\left(

The length of the shortest path from start to a given vertex $t$ is exactly the value of distance


%---- Page End Break Here ---- Page : 274

\begin{figure}[htbp]
    \centering
```

\includegraphics[width=\linewidth]{algorithms_page_0276_0.png}
\end{figure}
Dijkstra works correctly only on graphs without negative-cost edges. The reason is tha

Fortunately, most applications don't have negative weights, making this discussion lar

Stop and Think: Shortest Path with Node Costs

Problem: Suppose we are given a directed graph whose weights are on the vertices inste

Solution: A natural idea would be to adapt the algorithm we have for edgeweighted grap

However, my preferred approach would leave Dijkstra's algorithm intact and instead con

This technique can be extended to a variety of different domains, including when there

\subsection{All-Pairs Shortest Path}

Suppose you want to find the "center" vertex in a graph-the one that minimizes the lon

%---- Page End Break Here ---- Page : 275
We could solve all-pairs shortest path by calling Dijkstra's algorithm from each of the

Floyd's algorithm is best employed on an adjacency matrix data structure, which is no

```
typedef struct {
int weight[MAXV+1][MAXV+1]; /* adjacency/weight info /
int nvertices; / number of vertices in graph */
} adjacency_matrix;
```

The critical issue in an adjacency matrix implementation is how we denote the edges ab

There are several ways to characterize the shortest path between two nodes in a graph.

What does this mean? When $k=0$, we are allowed no intermediate vertices, so the only

With each iteration, we allow a richer set of possible shortest paths by adding a new
$$
W[i, j]^{k}=\min \left(W[i, j]^{k-1}, W[i, k]^{k-1}+W[k, j]^{k-1}\right)
$$

The correctness of this is somewhat subtle, and I encourage you to convince yourself o

```
void floyd(adjacency_matrix g) {
int i, j; / dimension counters /
int k; / intermediate vertex counter /
```

```
int through_k; / distance through vertex k */
for (k = 1; k <= g->nvertices; k++) {
for (i = 1; i <= g->nvertices; i++) {
for (j = 1; j <= g->nvertices; j++) {
through_k = g->weight [i] [k]+g->weight [k] [j];
if (through_k < g->weight[i][j]) {
g->weight[i][j] = through_k;
}
}
}
}
}
```

The Floyd-Warshall all-pairs shortest-path algorithm runs in $O\left(n^{3}\right)$ time, which is

The output of Floyd's algorithm, as it is written, does not enable one to reconstruct the actual

\subsection{Transitive Closure}

Floyd's algorithm has another important application, that of computing transitive closure. We are

A simplistic answer would be the vertex of highest out-degree, but an even better representative

%---- Page End Break Here ---- Page : 277
The vertices reachable from any single node can be computed using breadthfirst or depth-first sea

Transitive closure is discussed in more detail in the catalog in Section 18.5

\section{War Story: Dialing for Documents}

I was part of a group visiting Periphonics, then an industry leader in building telephone voice-r
"Like typing, my pupik!" came a voice from the rear of our group. "I hate typing on a telephone.
"Maybe you don't have to hit two keys for each letter!" I chimed in. "Maybe the system could figu
"There isn't a whole lot of context when you type in three letters of stock market code."
"Sure, but there would be plenty of context if we typed in English sentences. I'll bet that we co

The guy from Periphonics gave me a disinterested look, then continued the tour. But when I got ba

Not all letters are equally likely to be typed on a telephone. In fact, not all letters can be ty

One reason was clear. This algorithm knew nothing about English words. If we coupled it with a di
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-279.jpg?height=704&width=774

%---- Page End Break Here ---- Page : 278

Figure 8.8: The phases of the telephone code reconstruction process.
next attempt, we reported the unambiguous characters of any words that collided in the

This also did a terrible job. Most words appearing in the text came from ambiguous cod

At this point, I started working with Harald Rau on the project, who proved to be a gr
"We can get good word-use frequencies and grammatical information from a big text data
"Let's think about it as a graph problem," I suggested.
"Graph problem? What graph problem? Where is there even a graph?"
"Think of a sentence as a series of tokens, each representing a word in the sentence.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-280.jpg?height=44

%---- Page End Break Here ---- Page : 279
Figure 8.9: The minimum-cost path defines the best interpretation for a sentence.
word. The cheapest path across this graph defines the best interpretation of the senten
"But all the paths look the same. They have the same number of edges. Now I see! We ha
"Exactly! The cost of an edge will reflect how likely it is that we will travel throug
"It will be hard to keep track of word-pair statistics, since there are so many possibl
"We can pay a cost for walking through a particular vertex that depends upon the freque
"But how do we figure out the relative weights of these factors?"
"Try what seems natural to you and then we can experiment with it."\index{242}
Harald implemented this shortest-path algorithm. With proper grammatical and statistica

FOURSCORE AND SEVEN YEARS AGO OUR FATHERS BROUGHT FORTH ON THIS CONTINENT A NEW NATION

%---- Page End Break Here ---- Page : 280
> LIVING RATHER TO BE DEDICATED HERE TO THE UNFINISHED WORK WHICH THEY WHO FOUGHT HERE

While we still made a few mistakes, we typically guessed about 99% of all characte

The constraints for many pattern recognition problems can be naturally formulated as s

\section{Network Flows and Bipartite Matching}

An edge-weighted graph can be interpreted as a network of pipes, where the weight of a

\subsection{Bipartite Matching}

While the network flow problem is of independent interest, its primary importance lies

Graph G is bipartite or two-colorable if the vertices can be divided into two sets,
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-282.jpg?height=45

```
%---- Page End Break Here ---- Page : 281

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{images/algorithms_page_0283_0.png}
\end{figure}
Figure 8.10: Bipartite graph with a maximum matching (left). The corresponding network flow insta
vertices represent girls, with edges representing compatible pairs. Matchings in these graphs hav

The maximum cardinality bipartite matching can be readily found using network flow. Create a sour

\subsection{Computing Network Flows}

Traditional network flow algorithms are based on the idea of augmenting paths: finding a path of

The key structure is the residual flow graph, denoted as $R(G, f)$, where $G$ is the input graph
(i) an edge $(i, j)$ with weight $c(i, j)-f(i, j)$, if $c(i, j)-f(i, j)>0$ and
(ii) an edge $(j, i)$ with weight $f(i, j)$, if $f(i, j)>0$.

The weight of the edge $(i, j)$ in the residual graph gives the exact amount of extra flow that c
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-283.jpg?height=338&width=525


%---- Page End Break Here ---- Page : 282
G
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-283.jpg?height=398&width=580

R(G,f)

Figure 8.11: Maximum $s-t$ flow in a graph $G$ (on left) showing the associated residual graph $R
from $s$ to $t$ implies that more flow can be pushed from $s$ to $t$. The smallest edge weight on

Figure 8.11 illustrates this idea. The maximum $s-t$ flow in graph $G$ is 7 . Such a flow is reve

Take-Home Lesson: The maximum flow from $s$ to $t$ always equals the weight of the minimum $s-t$

Implementation

We cannot do full justice to the theory of network flows here. However, it is instructive to see

For each edge in the residual flow graph, we must keep track of both the amount of flow currently
```

typedef struct {

int v; / neighboring vertex /*
int capacity; / capacity of edge /

```
int flow; / flow through edge /
int residual; / residual capacity of edge */
struct edgenode next; / next edge in list */
} edgenode;
```

We use a breadth-first search to look for any path from source to sink that increases t

```
    void netflow(flow_graph g, int source, int sink) {
int volume; / weight of the augmenting path */
add_residual_edges(g);
initialize_search(g);
bfs(g, source);
volume = path_volume(g, source, sink);
while (volume > 0) {
augment_path(g, source, sink, volume);
initialize_search(g);
bfs(g, source);
volume = path_volume(g, source, sink);
}
}
```

Any augmenting path from source to sink increases the flow, so we can use bfs to find s

```
    bool valid_edge(edgenode *e) {
return (e->residual > 0);
}
```

Augmenting a path transfers the maximum possible volume from the residual capacity into

```
    int path_volume(flow_graph *g, int start, int end) {
edgenode e; / edge in question */
if (parent[end] == -1) {
return(0);
}
e = find_edge(g, parent[end], end);
if (start == parent[end]) {
return(e->residual);
} else {
return(min(path_volume(g, start, parent[end]), e->residual));
}
}
```

Recall that bfs uses the parent array to record the discoverer of each vertex on the tr

Sending an additional unit of flow along directed edge (i, j) reduces the residual ca

```
    void augment_path(flow_graph *g, int start, int end, int volume) {
edgenode e; / edge in question */
```

```
if (start == end) {
return;
}
e = find_edge(g, parent[end], end);
e->flow += volume;
e->residual -= volume;
e = find_edge(g, end, parent[end]);
e->residual += volume;
augment_path(g, start, parent[end], volume);

    }
```

Initializing the flow graph requires creating directed flow edges (i, j) and (j, i) for each

Analysis

The augmenting path algorithm above eventually converges to the optimal solution. However, each a

However, Edmonds and Karp \\cite{edmonds1972theoretical} proved that always selecting a shortest

\section{Randomized Min-Cut}

Clever randomized algorithms have been developed for many different types of problems. We have so

The minimum-cut problem in graphs seeks to partition the vertices of graph G into sets V_{1}

Suppose the minimum cut C in G is of size k, meaning that k edge deletions are necessary

A contraction operation for edge (x, y) collapses vertices x and y into a single merged ver

What happens to the size of the minimum cut after contracting (x, y) in G ? Each contraction
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-287.jpg?height=235&width=123

%---- Page End Break Here ---- Page : 286
Figure 8.12: If we get lucky, a sequence of random edge contractions does not increase the size o
minimum-cut size is unchanged unless we contract one of the k edges of the optimal cut. If we d

This suggests the following randomized algorithm. Pick a random edge of G and contract it. Repe

What are the chances of success on any given iteration? Consider the initial graph. A contraction

$$
p_{i} \geq 1-\frac{k}{k(n-i+1) / 2}=1-\frac{2}{n-i+1}=\frac{n-i-1}{n-i+1}
$$

The odds on success for all but the last few contractions in a large graph are strongly in our fa

To end up with a minimum cut C for a particular run, we must succeed on every one of
$\prod_{i=1}^{n-2} p_{i}=\prod_{i=1}^{n-2} \frac{n-i-1}{n-i+1}=\left(\frac{n-2}{n}\rig
The product cancels magically, and leaves a success probability of $\Theta\left(1 / n^

Take-Home Lesson: The key to success in any randomized algorithm is setting up a situa

%---- Page End Break Here ---- Page : 287
\section{Design Graphs, Not Algorithms}

Proper modeling is the key to making effective use of graph algorithms. Several proper

The secret is learning to design graphs, not algorithms. We have already seen a few in
- The maximum spanning tree can be found by negating the edge weights of the input gra
- To solve bipartite matching, we constructed a special network flow graph such that t

The applications below demonstrate the power of proper modeling. Each arose in a real-

Stop and Think: The Pink Panther's Passport to Peril

Problem: I'm looking for an algorithm to design natural routes for video-game characte

Solution: Presumably the desired route should look like a path that an intelligent bei

But what is the graph? One approach might be to lay a grid of points in the room. Crea

Stop and Think: Ordering the Sequence

Problem: A DNA sequencing project generates experimental data consisting of small frag
f's right. How can we find a consistent ordering of the fragments from left to right t

%---- Page End Break Here ---- Page : 288
Solution: Create a directed graph, where each fragment is assigned a unique vertex. In

Stop and Think: Bucketing Rectangles

Problem: In my graphics work I must solve the following problem. Given an arbitrary se

Solution: We formulate a graph where each vertex represents a rectangle, and there is a

Stop and Think: Names in Collision

Problem: In porting code from Unix to DOS, I have to shorten several hundred file name

Solution: Construct a bipartite graph with vertices corresponding to each original file name $f_{

%---- Page End Break Here ---- Page : 289
Stop and Think: Separate the Text

Problem: We need a way to separate the lines of text in the optical characterrecognition system t

Solution: Consider the following graph formulation. Treat each pixel in the image as a vertex in

Take-Home Lesson: Designing novel graph algorithms is very hard, so don't do it. Instead, try to

Chapter Notes

Network flows are an advanced algorithmic technique, and recognizing whether a particular problem

The augmenting path method for network flows is due to Ford and Fulkerson [\\cite{ford1962flows}.

The phone code reconstruction system that was the subject of the war story is described in more t

\section{Exercises}

Simulating Graph Algorithms

8-1. [3] For the graphs in Problem 7-1
(a) Draw the spanning forest after every iteration of the main loop in Kruskal's algorithm.
(b) Draw the spanning forest after every iteration of the main loop in Prim's algorithm.
(c) Find the shortest-path spanning tree rooted in A.
(d) Compute the maximum flow from A to H.

%---- Page End Break Here ---- Page : 290
Minimum Spanning Trees

8-2. [3] Is the path between two vertices in a minimum spanning tree necessarily a shortest path
8-3. [3] Assume that all edges in the graph have distinct edge weights (i.e., no pair of edges ha
8-4. [3] Can Prim's and Kruskal's algorithms yield different minimum spanning trees? Explain why
8-5. [3] Does either Prim's or Kruskal's algorithm work if there are negative edge weights? Expla
8-6. [3] (a) Assume that all edges in the graph have distinct edge weights (i.e., no pair of edge
(b) Again, assume that all edges in the graph have distinct edge weights (i.e. no pair of edges h
8-7. [5] Suppose we are given the minimum spanning tree T of a given graph G (with n vertic
8-8. [5] (a) Let T be a minimum spanning tree of a weighted graph G. Construct a new graph $G
(b) Let $P=\{s, \ldots, t\}$ describe a shortest path between vertices s and t of a weighted
8-9. [5] Devise and analyze an algorithm that takes a weighted graph G and finds the smallest c
8-10. [4] Consider the problem of finding a minimum-weight connected subset T of edges from a w
(a) Why is this problem not just the minimum spanning tree problem? (Hint: think negative weight

(b) Give an efficient algorithm to compute the minimum-weight connected subset T.

8-11. [5] Let $T=\left(V, E^{\prime}\right)$ be a minimum spanning tree of a given gra
(a) $e \notin E^{\prime}$ and $\hat{w}(e)>w(e)$
(b) $e \notin E^{\prime}$ and $\hat{w}(e)<w(e)$
(c) $e \in E^{\prime}$ and $\hat{w}(e)<w(e)$
(d) $e \in E^{\prime}$ and $\hat{w}(e)>w(e)$

%---- Page End Break Here ---- Page : 291
8-12. [4] Let $G=(V, E)$ be an undirected graph. A set $F \subseteq E$ of edges is cal
(a) Suppose that G is unweighted. Design an efficient algorithm to find a minimum-si
(b) Suppose that G is a weighted undirected graph with positive edge weights. Design

Union-Find

8-13. [5] Devise an efficient data structure to handle the following operations on a w
(a) Merge two given components.
(b) Locate which component contains a given vertex v.
(c) Retrieve a minimum edge from a given component.

8-14. [5] Design a data structure that enables a sequence of m union and find operat

Shortest Paths

8-15. [3] The single-destination shortest-path problem for a directed graph seeks the
8-16. [3] Let $G=(V, E)$ be an undirected weighted graph, and let T be the shortestp

8-17. [3] (a) Give an example of a weighted connected graph $G=(V, E)$ and a vertex $v
(b) Give an example of a weighted connected directed graph $G=(V, E)$ and a vertex v
(c) Can the two trees be completely disjoint?

8-18. [3] Either prove the following or give a counterexample:
(a) Is the path between a pair of vertices in a minimum spanning tree of an undirected
(b) Suppose that the minimum spanning tree of the graph is unique. Is the path between

%---- Page End Break Here ---- Page : 292
8-19. [3] Give an efficient algorithm to find the shortest path from x to y in an
8-20. [5] In certain graph problems, vertices can have weights instead of or in additi
(a) Suppose that each edge in the graph has a weight of zero (while non-edges have a c
(b) Now suppose that the vertex costs are not constant (but are all positive) and the
(c) Now suppose that both the edge and vertex costs are not constant (but are all posi

8-21. [5] Give an $O\left(n^{3}\right)$ algorithm that takes an n-vertex directed gra

8 -22. [5] A highway network is represented by a weighted graph G, with edges corresponding to
8-23. [5] You are given a directed graph G with possibly negative weighted edges, in which the
8-24. [5] Can we solve the single-source longest-path problem by changing minimum to maximum in D

8-25. [5] Let $G=(V, E)$ be a weighted acyclic directed graph with possibly negative edge weights

8-26. [5] Let $G=(V, E)$ be a directed weighted graph such that all the weights are positive. Let

8-27. [5] Arbitrage is the use of discrepancies in currency-exchange rates to make a profit. For
trade one U.S. dollar and end up with $0.75 \times 2 \times 0.7=1.05$ U.S. dollars-a profit of $5
$$
R\left[c_{1}, c_{i_{1}}\right] \cdot R\left[c_{i_{1}}, c_{i_{2}}\right] \cdots R\left[c_{i_{k-1}}
$$
(Hint: think all-pairs shortest path.)

%---- Page End Break Here ---- Page : 293
Network Flow and Matching

8-28. [3] A matching in a graph is a set of disjoint edges-that is, edges that do not have common
8-29. [5] An edge cover of an undirected graph $G=(V, E)$ is a set of edges such that each vertex

LeetCode

8-1. https://leetcode.com/problems/cheapest-flights-within-k-stops/
8-2. https://leetcode.com/problems/network-delay-time/
8-3. https://leetcode.com/problems/find-the-city-with-the-smallest-number-of-neighbors-at-a-thres

HackerRank

8-1. https://www.hackerrank.com/challenges/kruskalmstrsub/
8-2. https://www.hackerrank.com/challenges/jack-goes-to-rapture/
8-3. https://www.hackerrank.com/challenges/tree-pruning/

Programming Challenges

These programming challenge problems with robot judging are available at https://onlinejudge.org

8-1. "Freckles"-Chapter 10, problem 10034.
8-2. "Necklace" - Chapter 10, problem 10054.
8-3. "Railroads" - Chapter 10, problem 10039.
8-4. "Tourist Guide"-Chapter 10, problem 10199.
8-5. "The Grand Dinner"-Chapter 10, problem 10249.

%---- Page End Break Here ---- Page : 294

Chapter 9

\chapter{Combinatorial Search}

Surprisingly large problems can be solved using exhaustive search techniques, albeit a

Modern computers have clock rates of a few gigahertz, meaning billions of operations p

It is important to realize how big (or how small) one million is. One million permutat

This section introduces backtracking as a technique for listing all possible solutions

\section{Backtracking}

Backtracking is a systematic way to run through all the possible configurations of a s

What these problems have in common is that we must generate each possible configuratio

At each step in the backtracking algorithm, we try to extend a given partial solution

Backtracking constructs a tree of partial solutions, where each node represents a part

Backtrack-DFS}(a,k
if }a=(\mp@subsup{a}{1}{},\mp@subsup{a}{2}{},ůůů,\mp@subsup{a}{k}{})\mathrm{
is a solution, report it.
else
k=k+1
construct Sk
while }\mp@subsup{S}{k}{}\not=\emptyset\mathrm{ do
a
Sk}=\mp@subsup{S}{k}{}-{\mp@subsup{a}{k}{}
Backtrack-DFS(}a,k

Although a breadth-first search could also be used to enumerate solutions, a depth-fir

Implementation

Backtracking ensures correctness by enumerating all possibilities. It ensures efficien

void backtrack(int a[], int k, data input) {
int c[MAXCANDIDATES]; / candidates for next position /*
int nc; / next position candidate count /
int i; / counter /
if (is_a_solution(a, k, input)) {
process_solution(a, k,input);
} else {
k = k + 1;

```
construct_candidates(a, k, input, c, &nc);
for (i = 0; i < nc; i++) {
a[k] = c[i];
make_move(a, k, input);
backtrack(a, k, input);
unmake_move(a, k, input);
if (finished) {
return; / terminate early */
}
}
}
}
```

Study how recursion yields an elegant and easy implementation of the backtracking algorithm. Beca

The application-specific parts of this algorithm consist of five subroutines:
- is_a_solution(a,k,input) - This Boolean function tests whether the first k elements of vector
- construct_candidates(a,k,input, $c, \& n c$) - This routine fills an array c with the comple
- process_solution(a,k,input) - This routine prints, counts, stores, or processes a complete solu
- make_move(a,k,input) and unmake_move(a,k,input) - These routines enable us to modify a data str
as clean up this data structure if we decide to take back the move. Such a data structure can alw

%---- Page End Break Here ---- Page : 297
These calls will function as null stubs in all of this section's examples, but will be employed i

A global finished flag is included to allow for premature termination, which could be set in any

\section{Examples of Backtracking}

To really understand how backtracking works, you must see how such objects as permutations and su

\subsection{Constructing All Subsets}

Designing an appropriate state space to represent combinatorial objects starts by counting how ma

Each subset is described by the elements that are contained in it. To construct all 2^{n} subse

```
int is_a_solution(int a[], int k, int n) {
return (k == n);
}
void construct_candidates(int a[], int k, int n, int c[], int *nc) {
c[0] = true;
c[1] = false;
*nc = 2;
}
```

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-299.jpg?height=44

Figure 9.1: Search tree enumerating all subsets (left) and permutations (right) of $\{

```
    void process_solution(int a[], int k, int input) {
int i; /* counter */
printf("{");
for (i = 1; i <= k; i++) {
if (a[i] == true) {
printf(" %d", i);
}
}
printf(" }\n");
}
```

Ironically, printing out each subset after constructing it proves to be the most comple

Finally, we must instantiate the call to backtrack with the right arguments. Specifical

```
    void generate_subsets(int n) {
int a[NMAX]; /* solution vector */
backtrack(a, 0, n);
}
```

In what order will the subsets of $\{1,2,3\}$ be generated? It depends on the order of

%---- Page End Break Here ---- Page : 299

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{algorithms_page_0301_0.png}
\end{figure}
Trace through this example (shown in Figure 9.1 (left)) carefully to make sure you unde

\subsection{Constructing All Permutations}

Counting permutations of $\{1, \ldots, n\}$ is a necessary prerequisite to generating t

This counting argument suggests a suitable representation. Set up an array/vector a o

In the scheme of the general backtrack algorithm, $S_{k}=\{1, \ldots, n\}-\left\{a_{1}

```
    void construct_candidates(int a[], int k, int n, int c[], int nc) {
int i; / counter /
bool in_perm[NMAX]; / what is now in the permutation? */
for (i = 1; i < NMAX; i++) {
```

```
in_perm[i] = false;
}
for (i = 1; i < k; i++) {
in_perm[a[i]] = true;
}
*nc = 0;
for (i = 1; i <= n; i++) {
if (!in_perm[i]) {
c[ *nc ] = i;
*nc = *nc + 1;
}
}
}
```

Testing whether i is a candidate for the k th slot in the permutation could be done by iterat

%---- Page End Break Here ---- Page : 300

Completing the job requires specifying process_solution and is_a_solution, as well as setting the

```
void process_solution(int a[], int k, int input) {
int i; /* counter */
for (i = 1; i <= k; i++) {
printf(" %d", a[i]);
}
printf("\n");
}
```

```
int is_a_solution(int $a[]$, int $k$, int $n$ ) \{
    return (k == n);
\}
void generate_permutations(int n) \{
    int a[NMAX]; /* solution vector */
    backtrack(a, 0, n);
\}
```

As a consequence of the candidate order, these routines generate permutations in lexicographic, o

\subsection{Constructing All Paths in a Graph}

In a simple path no vertex appears more than once. Enumerating all the simple s to t paths in

The input data we must pass to backtrack to construct the paths consists of the input graph g,

```
typedef struct {
int s; /* source vertex /
int t; / destination vertex /
```

```
graph g; / graph to find paths in */
} paths_data;
```

The starting point of any path from s to t is always s. Thus, s is the only ca

```
    void construct_candidates(int a[], int k, paths_data *g, int c[],
int nc) {
int i; / counters /
bool in_sol[NMAX+1]; / what's already in the solution? */
edgenode p; / temporary pointer /
int last; / last vertex on current path /
for (i = 1; i <= g->g.nvertices; i++) {
in_sol[i] = false;
}
for (i = 0; i < k; i++) {
in_sol[a[i]] = true;
}
if (k == 1) {
c[0] = g->s; / always start from vertex s */
*nc = 1;
} else {
*nc = 0;
last = a[k-1];
p = g->g.edges[last];
while (p != NULL) {
if (!in_sol[ p->y ]) {
c[*nc] = p->y;
*nc= *nc + 1;
}
p = p->next;
}
}
}
```

We report a successful path whenever $a_{k}=t$.

```
    int is_a_solution(int a[], int k, paths_data *g) {
return (a[k] == g->t);

    }
```

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-303.jpg?height=48

Figure 9.2: Search tree (right) enumerating all simple $s-t$ paths in the given graph

The number of paths discovered can be counted in process_solution by incrementing a gl

```
void process_solution(int a[], int k, paths_data input) {
int i; / counter */
solution_count ++;
printf("{");
for (i = 1; i <= k; i++) {
printf(" %d",a[i]);
}
printf(" }\n");
}
```

This solution vector must have room for all n vertices, although most paths should be shorter t

\section{Search Pruning}\index{derangement}\index{multiset}

Backtracking ensures correctness by enumerating all possibilities. A correct algorithm to find th

However, it is wasteful to construct all the permutations first and then analyze them later. Supp
that vertex-pair $\left(v_{1}, v_{2}\right)$ was not an edge in G. The $(n-2)$! permutations e

%---- Page End Break Here ---- Page : 303

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{images/algorithms_page_0305_0.png}
\end{figure}
Pruning is the technique of abandoning a search direction the instant we can establish that a giv

Exploiting symmetry is another avenue for reducing combinatorial search. Pruning away partial sol

Take-Home Lesson: Combinatorial search, when augmented with tree-pruning techniques, can be used

\section{Sudoku}

A Sudoku craze has swept the world. Many newspapers publish daily Sudoku puzzles, and millions of

What is Sudoku? In its most common form, it consists of a 9×9 grid filled with blanks an

Backtracking lends itself nicely to the task of solving Sudoku puzzles. We will
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-305.jpg?height=525&width=527
\begin{tabular}{|l|l|l|l|l|l|l|l|l|}
\hline 6 & 7 & 3 & 8 & 9 & 4 & 5 & 1 & 2 \\
%---- Page End Break Here ---- Page : 304

\hline 9 & 1 & 2 & 7 & 3 & 5 & 4 & 8 & 6 \\
\hline 8 & 4 & 5 & 6 & 1 & 2 & 9 & 7 & 3 \\

```
\hline 7 & 9 & 8 & 2 & 6 & 1 & 3 & 5 & 4 \\
\hline 5 & 2 & 6 & 4 & 7 & 3 & 8 & 9 & 1 \\
\hline 1 & 3 & 4 & 5 & 8 & 9 & 2 & 6 & 7 \\
\hline 4 & 6 & 9 & 1 & 2 & 8 & 7 & 3 & 5 \\
\hline 2 & 8 & 7 & 3 & 5 & 6 & 1 & 4 & 9 \\
\hline 3 & 5 & 1 & 9 & 4 & 7 & 6 & 2 & 8 \\
\hline
\end{tabular}
```

Figure 9.3: A challenging Sudoku puzzle (left) with its completed solution (right).
use Sudoku here to illustrate pruning techniques for combinatorial search. Our state sp

The solution vector a supported by backtrack only accepts a single integer per positio

#define DIMENSION 9 /* 99 board /
#define NCELLS DIMENSIONDIMENSION / 81 cells in 9-by-9-board /
#define MAXCANDIDATES DIMENSION+1 / max digit choices per cell /
bool finished = false;
typedef struct {
int x, y; / row and column coordinates of square */
} point;

typedef struct {

```
typedef struct {
    int m[DIMENSION+1] [DIMENSION+1]; /* board contents */
    int m[DIMENSION+1] [DIMENSION+1]; /* board contents */
    int freecount; /* open square count */
    int freecount; /* open square count */
    point move[NCELLS+1]; /* which cells have we filled? */
    point move[NCELLS+1]; /* which cells have we filled? */
} boardtype;
```

} boardtype;

Constructing the move candidates for the next position requires first picking which op

void construct_candidates(int a[], int k, boardtype *board, int c[],
int nc) {
int i; / counter /
bool possible[DIMENSION+1]; / which digits fit in this square /
next_square(&(board->move[k]), board); / pick square to fill next */
nc = 0;
if ((board->move[k].x < 0) && (board->move[k].y < 0)) {
return; / error condition, no moves possible */

```
}
possible_values(board->move[k], board, possible);
for (i = 1; i <= DIMENSION; i++) {
if (possible[i]) {
c[*nc] = i;
*nc = *nc + 1;
}
}
}
```

We must update our board data structure to reflect the effect of putting a candidate value into a

```
void make_move(int a[], int k, boardtype *board) {
fill_square(board->move[k], a[k], board);
}
void unmake_move(int a[], int k, boardtype *board) {
free_square(board->move[k], board);
}
```

One important job for these board update routines is maintaining how many free squares remain on

```
bool is_a_solution(int a[], int k, boardtype board) {
steps = steps + 1; / count steps for results table */
return (board->freecount == 0);
}
```

We print the configuration and then turn off the backtrack search after finding a solution by set

```
void process_solution(int a[], int k, boardtype *board) {
finished = true;
printf("process solution\n");
print_board(board);
}
```

This completes the program modulo details of identifying the next open square to fill (next_squar
- Arbitrary square selection - Pick the first open square we encounter, be it the first, the last
- Most constrained square selection - Here, we check each open square (i, j) to see how many di

Although both possibilities work correctly, the second option is much, much better. If there are

If the most constrained square has two possibilities, we have a 50% chance of guessing right
\begin{tabular}{|cc|r|r|r|}
\hline \multicolumn{2}{|c}{ Pruning condition } & \multicolumn{3}{c|}{ Puzzle complexity } \\
%---- Page End Break Here ---- Page : 307

next_square & possible_values & Easy & Medium & Hard \\
\hline arbitrary & local count & $1,904,832$ & 863,305 & never finished \\

```
arbitrary & look ahead & 127 & 142 & $12,507,212$ \\
most constrained & local count & 48 & 84 & $1,243,838$ \\
most constrained & look ahead & 48 & 65 & 10,374 \\
\hline
\end{tabular}

Figure 9.4: Sudoku run times (in number of steps) for different pruning strategies.

Our final decision concerns the possible_values we allow for each square. We have two p
- Local count - Our backtrack search works correctly if the routine that generates cand
- Look ahead - But what if our current partial solution has some other open square whe

We will discover this obstruction eventually, when we pick this square for expansion,

Successful pruning requires looking ahead to see when a partial solution is doomed to g

Figure 9.4 presents the number of calls to is_a_solution for all four backtracking vari
- The Easy board was intended to be easy for a human player. Indeed, my program solved
- The Medium board stumped all the contestants at the finals of the World Sudoku Champi

\footnotetext{
${ }^{1}$ This look-ahead condition might have naturally followed from the most-constra
}
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-309.jpg?height=40


%---- Page End Break Here ---- Page : 308
Figure 9.5: Configurations covering 63 but not 64 squares.
- The Hard problem is the board displayed in Figure 9.3 which initially contains only

What is considered to be a "hard" problem instance depends upon the given heuristic. S

What can we learn from these experiments? Looking ahead to eliminate dead positions as

Smart square selection had a similar impact, even though it nominally just rearranges

It took the better part of an hour (48:44) to solve the puzzle in Figure 9.3 when I sel

This is the power of search pruning. Even simple pruning strategies can suffice to red

\section{War Story: Covering Chessboards}

Every researcher dreams of solving a classical problem - one that has remained open and
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-310.jpg?height=39
```

%---- Page End Break Here ---- Page : 309
Figure 9.6: The ten unique positions for the queen, with respect to rotational and reflective sym
pleasant sense of smugness that comes from figuring out how to do something that nobody could do

There are several possible reasons why a problem might stay open for such a long period of time.

Chess is a game that has fascinated people for thousands of years. In addition, it has inspired m

In 1849, Kling posed the question of whether all 64 squares on the board could be simultaneously

How many ways can the eight main chess pieces be positioned on a chessboard? The trivial bound is

%---- Page End Break Here ---- Page : 310
Getting the job done would require significant pruning. Our first idea was to remove symmetries.

Once the queen is placed, there remain $32 \cdot 31$ distinct positions for the bishops, then $61

We could use backtracking to construct all possible chess boards, but we had to find a way to pru

This pruning strategy required carefully ordering the evaluation of the pieces. Each piece can th

When we implemented a backtrack search using this pruning strategy, it eliminated over 95% of

Effective pruning means eliminating large numbers of positions at a single stroke. Our previous a

So in our final version, the nodes of our search tree corresponded to chessboards that could have

Our algorithm consisted of two passes. The first pass listed boards where every square was weakly
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-312.jpg?height=382&width=812

%---- Page End Break Here ---- Page : 311
Figure 9.7: Weakly covering 64 squares.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-312.jpg?height=358&width=367

Figure 9.8: Seven pieces suffice when superimposing queen and knight (shown as a white queen).
to worry about), and any strong attack set is always a subset of a weak attack set. The position

This program was efficient enough to complete the search in under a day. It did not find a single

Take-Home Lesson: Clever pruning can make short work of surprisingly hard combinatorial search pr

\section{Best-First Search}\index{binomial coecients}\index{paths across a grid, counting}\index{

An important idea to speed up search is to explore your best options before the less promising ch

The examples so far in this chapter have focused on existential search problems, where

%---- Page End Break Here ---- Page : 312

Best-first search, also called branch and bound, assigns a cost to every partial soluti

```
    void branch_and_bound (tsp_solution *s, tsp_instance t) {
int c[MAXCANDIDATES]; / candidates for next position /
int nc; / next position candidate count /
int i; / counter */
first_solution(&best_solution,t);
best_cost = solution_cost(&best_solution, t);
initialize_solution(s,t);
extend_solution(s,t,1);
pq_init(&q);
pq_insert(&q,s);
while (top_pq(&q).cost < best_cost) {
*s = extract_min(&q);
if (is_a_solution(s, s->n, t)) {
process_solution(s, s->n, t);
}
else {
construct_candidates(s, (s->n)+1, t, c, &nc);
for (i=0; i<nc; i++) {
extend_solution(s,t,c[i]);
pq_insert(&q,s);
contract_solution(s,t);

    }
}
}
}
```

The extend_solution and contract_solution routines handle the bookkeeping of creating a

```
    void extend_solution(tsp_solution *s, tsp_instance *t, int v) {
s->n++;
s->p[s->n] = v;
s->cost = partial_solution_lb(s,t);
}
void contract_solution(tsp_solution *s, tsp_instance *t) {
s->n–;
s->cost = partial_solution_lb(s,t);
}
```

What should be the cost of a partial solution? There are $(n-1)$! circular permutation

But does the first full solution from a best-first search have to be an optimal solution? No, not

Thus, to get the global optimal, we must continue to explore the partial solutions coming off the

\section{The A* Heuristic}

Best-first search can take a while, even if our partial cost function is a lower bound on the opt
\begin{tabular}{c|rrr|cc}
& \multicolumn{3}{|c}{ Backtracking } & \multicolumn{2}{c}{ Branch and Bound } \\
%---- Page End Break Here ---- Page : 314

n & all & cost $<$ best & $l b<$ best & cost $<$ best & $l b<$ best \\
\hline 5 & 24 & 22 & 17 & 11 & 7 \\
6 & 120 & 86 & 62 & 28 & 20 \\
7 & 720 & 217 & 153 & 51 & 42 \\
8 & 5,040 & 669 & 443 & 111 & 85 \\
9 & 40,320 & 2,509 & 1,619 & 354 & 264 \\
10 & 362,880 & 5,042 & 3,025 & 655 & 475 \\
11 & $3,628,800$ & 12,695 & 6,391 & 848 & 705
\end{tabular}

Figure 9.9: Number of complete TSP solutions evaluated by different search variants. The A^{*}
optimal solution on all n nodes, meaning that we must expand all partial solutions until their

The A^{*} heuristic (pronounced "A-star") is an elaboration on the branch-and-bound search pres

How can we lower bound the cost of the full tour, which contains n edges, from a partial soluti

```
double partial_solution_cost(tsp_solution *s, tsp_instance t) {
int i; / counter /
double cost = 0.0; / cost of solution */
for (i = 1; i < (s->n); i++) {
cost = cost + distance(s, i, i + 1, t);
}
return(cost);
}
double partial_solution_lb(tsp_solution *s, tsp_instance *t) {
return(partial_solution_cost(s,t) + (t->n - s->n + 1) * minlb);

}
```

Figure 9.9 presents the number of full solution cost evaluations in finding the optimal TSP tour

Note that the number of full solutions encountered is a gross underestimate of the total work dor

Best-first search is sort of like breadth-first search. A disadvantage of BFS over DFS

The resulting size of the priority queue for best-first search is a real problem. Cons

Take-Home Lesson: The promise of a given partial solution is not just its cost, but al

The A* heuristic proves useful in a variety of different problems, most notably findin

But that means that half the growth is in a direction away from t, thus moving farth

Chapter Notes

My treatment of backtracking here is partially based on my book Programming Challenges
of eight mutually non-attacking queens on an 8×8 board.
More details on our combinatorial search for optimal chessboard-covering positions app

%---- Page End Break Here ---- Page : 316
\section{Exercises}

Permutations

9-1. [3] A derangement is a permutation p of $\{1, \ldots, n\}$ such that no item is
9-2. [4] Multisets are allowed to have repeated elements. A multiset of n items may

9-3. [5] For a given a positive integer n, find all permutations of the $2 n$ elemen
9-4. [5] Design and implement an algorithm for testing whether two graphs are isomorph
9-5. [5] The set $\{1,2,3, \ldots, n\}$ contains a total of n ! distinct permutation
$$
[123,132,213,231,312,321]
$$

Give an efficient algorithm that returns the k th of n ! permutations in this sequ

Backtracking

9-6. [5] Generate all structurally distinct binary search trees that store values $1 \
9-7. [5] Implement an algorithm to print all valid (meaning properly opened and closed
9-8. [5] Generate all possible topological orderings of a given DAG.
9-9. [5] Given a specified total t and a multiset S of n integers, find all dist

%---- Page End Break Here ---- Page : 317
9-10. [8] Design and implement an algorithm for solving the subgraph isomorphism probl
9-11. [5] A team assignment of $n=2 k$ players is a partitioning of them into two team
9-12. [5] Given an alphabet Σ, a set of forbidden strings S, and a target len

9-13. [5] In the k-partition problem, we need to partition a multiset of positive integers into
9-14. [5] You are given a weighted directed graph G with n vertices and m edges. The mean w
9-15. [8] In the turnpike reconstruction problem, you are given a multiset D of $n(n-$ 1) $/ 2$

Games and Puzzles

9-16. [5] Anagrams are rearrangements of the letters of a word or phrase into a different word or
9-17. [5] Construct all sequences of moves that a knight on an $n \times n$ chessboard can make w
9-18. [5] A Boggle board is an $n \times m$ grid of characters. For a given board, we seek to fin
\begin{tabular}{cccc}
e & t & h & t \\
n & d & t & i \\
a & i & h & n \\
r & h & u & b
\end{tabular}
contains words like tide, dent, raid, and hide. Design an algorithm to construct the most words f

%---- Page End Break Here ---- Page : 318
9-19. [5] A Babbage square is a grid of words that reads the same across as it does down. Given a
\begin{tabular}{cccc|cccc}
h & a & i & r & h & a & i & r \\
a & i & d & e & a & l & t & o \\
i & d & l & e & i & t & e & m \\
r & e & e & f & r & o & m & b
\end{tabular}

9-20. [5] Show that you can solve any given Sudoku puzzle by finding the minimum vertex coloring

Combinatorial Optimization

For problems 9-21 to 9-27 implement a combinatorial search program to solve it for small instance
9-21. [5] Design and implement an algorithm for solving the bandwidth minimization problem discus
9-22. [5] Design and implement an algorithm for solving the maximum satisfiability problem discus
9-23. [5] Design and implement an algorithm for solving the maximum clique problem discussed in S
9-24. [5] Design and implement an algorithm for solving the minimum vertex coloring problem discu
9-25. [5] Design and implement an algorithm for solving the minimum edge coloring problem discuss
9-26. [5] Design and implement an algorithm for solving the minimum feedback vertex set problem d
9-27. [5] Design and implement an algorithm for solving the set cover problem discussed in Sectio

Interview Problems

9-28. [4] Write a function to find all permutations of the letters in a given string.
9-29. [4] Implement an efficient algorithm for listing all k-element subsets of n items.
9-30. [5] An anagram is a rearrangement of the letters in a given string into a sequence of dicti

9-31. [5] Telephone keypads have letters on each numerical key. Write a program that g

9-32. [7] You start with an empty room and a group of n people waiting outside. At ea

%---- Page End Break Here ---- Page : 319
9-33. [4] Use a random number generator (rng04) that generates numbers from $\{0,1,2,3

LeetCode

9-1. https://leetcode.com/problems/subsets/
9-2. https://leetcode.com/problems/remove-invalid-parentheses/
9-3. https://leetcode.com/problems/word-search/

HackerRank

9-1. https://www.hackerrank.com/challenges/sudoku/
9-2. https://www.hackerrank.com/challenges/crossword-puzzle/

Programming Challenges

These programming challenge problems with robot judging are available at https://onlin

9-1. "Little Bishops" - Chapter 8, problem 861.
9-2. "15-Puzzle Problem"-Chapter 8, problem 10181.
9-3. "Tug of War" - Chapter 8, problem 10032.
9-4. "Color Hash" - Chapter 8, problem 704.

%---- Page End Break Here ---- Page : 320
Chapter 10

\chapter{Dynamic Programming}\index{boundaries!programming}

The most challenging algorithmic problems involve optimization, where we seek to find a

Algorithms for optimization problems require proof that they always return the best pos

Dynamic programming combines the best of both worlds. It gives us a way to design cust

After you understand it, dynamic programming is probably the easiest algorithm design t

Dynamic programming is a technique for efficiently implementing a recursive algorithm b

Dynamic programming is generally the right method for optimization prob-
lems on combinatorial objects that have an inherent left-to-right order among componen

\section{Caching vs. Computation}

Dynamic programming is essentially a tradeoff of space for time. Repeatedly computing a given qua

The tradeoff between space and time exploited in dynamic programming is best illustrated when eva

\subsection{Fibonacci Numbers by Recursion}

The Fibonacci numbers were defined by the Italian mathematician Fibonacci in the thirteenth centu
$$
F_{n}=F_{n-1}+F_{n-2}
$$
with basis cases $F_{0}=0$ and $F_{1}=1$. Thus, $F_{2}=1, F_{3}=2$, and the series continues $3,5

That they are defined by a recursive formula makes it easy to write a recursive program to comput

```
long fib_r(int n) {
if (n == 0) {
return(0);
}
if (n == 1) {
return(1);
}
return(fib_r(n-1) + fib_r(n-2));

}
```

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-323.jpg?height=451&width=889

Figure 10.1: The recursion tree for computing Fibonacci numbers.

The course of execution for this recursive algorithm is illustrated by its recursion tree, as ill

Note that $F(4)$ is computed on both sides of the recursion tree, and $F(2)$ is computed no less

How much time does the recursive algorithm take to compute $F(n)$? Since $F_{n+1} / F_{n} \appro

\subsection{Fibonacci Numbers by Caching}

In fact, we can do much better. We can explicitly store (or cache) the results of each Fibonacci

```
#define MAXN 92 /* largest n for which F(n) fits in a long /
#define UNKNOWN -1 / contents denote an empty cell /
long f[MAXN+1]; / array for caching fib values */
```

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-324.jpg?height=503&width=886

%---- Page End Break Here ---- Page : 323
Figure 10.2: The recursion tree for computing Fibonacci numbers with caching.

```
long fib_c(int n) {
if (f[n] == UNKNOWN) {
f[n] = fib_c(n-1) + fib_c(n-2);
}
return(f[n]);
}
long fib_c_driver(int n) {
int i; /* counter */
f[0] = 0;
f[1] = 1;
for (i = 2; i <= n; i++) {
f[i] = UNKNOWN;
}
return(fib_c(n));
}
```

To compute $F(n)$, we call fib_c_driver (n). This initializes our cache to

This cached version runs instantly up to the largest value that can fit in a long integ
return.
What is the running time of this algorithm? The recursion tree provides more of a clue

%---- Page End Break Here ---- Page : 324
This general method of explicitly caching (or tabling) results from recursive calls to

Caching makes sense only when the space of distinct parameter values is modest enough

Take-Home Lesson: Explicit caching of the results of recursive calls provides most of

\subsection{Fibonacci Numbers by Dynamic Programming}

We can calculate F_{n} in linear time more easily by explicitly specifying the order

```
long fib_dp(int n) {
int i; /* counter /
long f[MAXN+1]; / array for caching values */
f[0] = 0;
f[1] = 1;
for (i = 2; i <= n; i++) {
f[i] = f[i-1] + f[i-2];
}
```

```
return(f[n]);
}
```

Observe that we have removed all recursive calls! We evaluate the Fibonacci numbers from smallest
have F_{i-1} and F_{i-2} ready whenever we need to compute F_{i}. The linearity of this alg

%---- Page End Break Here ---- Page : 325
More careful study shows that we do not need to store all the intermediate values for the entire

```
    long fib_ultimate(int n)
{
int i; /* counter /
long back2=0, back1=1; / last two values of f[n] /
long next; / placeholder for sum */
if (n == 0) return (0);
for (i=2; i<n; i++) {
next = back1+back2;
back2 = back1;
back1 = next;
}
return(back1+back2);
}
```

This analysis reduces the storage demands to constant space with no asymptotic degradation in run

\subsection{Binomial Coefficients}

We now show how to compute binomial coefficients as another example of how to eliminate recursion

How do you compute binomial coefficients? First, $\binom{n}{k}=\frac{n!}{k!(n-k)!}$, so in princi

A more stable way to compute binomial coefficients is using the recurrence relation implicit in t
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-326.jpg?height=238&width=397
\begin{tabular}{c|cccccc}
n / k & 0 & 1 & 2 & 3 & 4 & 5 \\
%---- Page End Break Here ---- Page : 326

\hline 0 & A & & & & & \\
1 & B & G & & & & \\
2 & C & 1 & H & & & \\
3 & D & 2 & 3 & I & & \\
4 & E & 4 & 5 & 6 & J & \\
5 & F & 7 & 8 & 9 & 10 & K
\end{tabular}
\begin{tabular}{c|cccccc}
n / k & 0 & 1 & 2 & 3 & 4 & 5 \\

\hline 0 & 1 & & & & & \\
1 & 1 & 1 & & & & \\
2 & 1 & 2 & 1 & & & \\
3 & 1 & 3 & 3 & 1 & & \\
4 & 1 & 4 & 6 & 4 & 1 & \\
5 & 1 & 5 & 10 & 10 & 5 & 1
\end{tabular}

Figure 10.3: Evaluation order for binomial_coefficient at $M[5,4]$ (left). The initiali

Each number is the sum of the two numbers directly above it. The recurrence relation in

$$
\binom{n}{k}=\binom{n-1}{k-1}+\binom{n-1}{k}
$$

Why does this work? Consider whether the nth element appears in one of the $\binom{n}

No recurrence is complete without basis cases. What binomial coefficient values do we k

Figure 10.3 demonstrates a proper evaluation order for the recurrence. The initialized

```
long binomial_coefficient(int n, int k) {
int i, j; /* counters /
long bc[MAXN+1][MAXN+1]; / binomial coefficient table */

for (i = 0; i <= n; i++) {
bc[i][0] = 1;
}

for (j = 0; j <= n; j++) {
bc[j][j] = 1;

}
for (i = 2; i <= n; i++) {
for (j = 1; j < i; j++) {
bc[i][j] = bc[i-1][j-1] + bc[i-1][j];
}
}
return(bc[n][k]);
}
```

Study this function carefully to make sure you see how we did it. The rest of this chap

\section{Approximate String Matching}

Searching for patterns in text strings is a problem of unquestionable importance. Back in Sectior

How can we search for the substring closest to a given pattern, to compensate for spelling errors
- Substitution - Replace a single character in pattern P with a different character, such as ch
- Insertion - Insert a single character into pattern P to help it match text T, such as chang
- Deletion - Delete a single character from pattern P to help it match text T, such as changi

Properly posing the question of string similarity requires us to set the cost of each such transf
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-329.jpg?height=213&width=104

%---- Page End Break Here ---- Page : 328
Figure 10.4: In a single string edit operation, the last character must be either matched/substit

Approximate string matching seems like a difficult problem, because we must decide exactly where

\subsection{Edit Distance by Recursion}

We can define a recursive algorithm using the observation that the last character in the string m

More precisely, let $D[i, j]$ be the minimum number of differences between the substrings $P_{1}
- If $\left(P_{i}=T_{j}\right)$, then $D[i-1, j-1]$, else $D[i-1, j-1]+1$. This means we either m
- $D[i, j-1]+1$. This means that there is an extra character in the text to account for, so we do
- $D[i-1, j]+1$. This means that there is an extra character in the pattern to remove, so we do r

#define MATCH O / enumerated type symbol for match*

```
typedef struct {
    int cost;                /* cost of reaching this cell */
    int parent;              /* parent cell */
} cell;
```

#define INSERT 1 //

```
typedef struct {
    int cost;                /* cost of reaching this cell */
    int parent;              /* parent cell */
} cell;
```

#define INSERT 1 / enumerated type symbol for insert /
*#define DELETE 2 / enumerated type symbol for delete */*

```
int string_compare_r(char *s, char *t, int i, int j) \{
    int k; /* counter */
    int opt[3]; /* cost of the three options */
```

```
    int lowest_cost; /* lowest cost */
    if (i $==0)\{\quad / *$ indel is the cost of an insertion or deletion */
        return(j * indel(' '));
    \}
    if (j == 0) \{
        return(i * indel(' '));
    \}
                    /* match is the cost of a match/substitution */
    opt[MATCH] = string_compare_r(s,t,i-1,j-1) + match(s[i],t[j]);
    opt[INSERT] = string_compare_r(s,t,i,j-1) + indel(t[j]);
    opt[DELETE] = string_compare_r(s,t,i-1,j) + indel(s[i]);
    lowest_cost $=$ opt[MATCH];
    for ( $\mathrm{k}=$ INSERT; $\mathrm{k}<=$ DELETE; $\mathrm{k}++$ ) \{
        if (opt[k] < lowest_cost) \{
            lowest_cost $=$ opt [k];
        \}
    \}
    return(lowest_cost);
\}
```

This program is absolutely correct-convince yourself. It also turns out to be impossibl

Why is the algorithm so slow? It takes exponential time because it recomputes values ag

%---- Page End Break Here ---- Page : 330
\subsection{Edit Distance by Dynamic Programming}

So, how can we make this algorithm practical? The important observation is that most of

A table-based, dynamic programming implementation of this algorithm is given below. The

```
    typedef struct {
int cost; /* cost of reaching this cell /
int parent; / parent cell /
} cell;
cell m[MAXLEN+1][MAXLEN+1]; / dynamic programming table */
```

Our dynamic programming implementation has three differences from the recursive version

Be aware that we adhere to special string and index conventions in the routine below. I

```
    int string_compare(char *s, char t, cell m[MAXLEN+1] [MAXLEN+1]) {
int i, j, k; / counters /
int opt[3]; / cost of the three options */
for (i = 0; i <= MAXLEN; i++) {
row_init(i, m);
```

column_init(i, m);

}

```
for (i = 1; i < strlen(s); i++) {
for (j = 1; j < strlen(t); j++) {
    opt[MATCH] = m[i-1][j-1].cost + match(s[i], t[j]);
    opt[INSERT] = m[i][j-1].cost + indel(t[j]);
    opt[DELETE] = m[i-1][j].cost + indel(s[i]);
    m[i][j].cost = opt[MATCH];
    m[i][j].parent = MATCH;
    for (k = INSERT; k <= DELETE; k++) {
        if (opt[k] < m[i][j].cost) {
        m[i][j].cost = opt[k];
        m[i][j].parent = k;
            }
    }
}
}
goal_cell(s, t, &i, &j);
return(m[i][j].cost);
```

}

```
To determine the value of cell $(i, j)$, we need to have three values sitting and waiting for us

As an example, we show the cost matrix for turning $P=$ "thou shalt" into $T=$ "you should" in fi

\subsection{Reconstructing the Path}

The string comparison function returns the cost of the optimal alignment, but not the alignment i

The possible solutions to a given dynamic programming problem are described by paths through the

\footnotetext{
${ }^{1}$ Suppose we create a graph with a vertex for every matrix cell, and a directed edge $(x,
}
\begin{tabular}{cr|rrrrrrrrrrr}
& $T$ & & y & o & u & - & s & h & o & u & l & d \\
%---- Page End Break Here ---- Page : 332

$P$ & pos & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\
\hline$:$ & & $\underline{\mathbf{0}}$ & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\
t: & 1 & $\mathbf{1}$ & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\
$\mathrm{~h}:$ & 2 & $\underline{2}$ & $\underline{\mathbf{2}}$ & 2 & 3 & 4 & 5 & 5 & 6 & 7 & 8 &
```

```
$\mathrm{o}:$ & 3 & 3 & 3 & $\underline{\mathbf{2}}$ & 3 & 4 & 5 & 6 & 5 & 6 & 7 & 8 \
$\mathrm{u}:$ & 4 & 4 & 4 & 3 & $\underline{\mathbf{2}}$ & 3 & 4 & 5 & 6 & 5 & 6 & 7 \
-: & 5 & 5 & 5 & 4 & 3 & $\underline{\mathbf{2}}$ & 3 & 4 & 5 & 6 & 6 & 7 \\
$\mathrm{~s}:$ & 6 & 6 & 6 & 5 & 4 & 3 & $\underline{\mathbf{2}}$ & 3 & 4 & 5 & 6 & 7 
$\mathrm{~h}:$ & 7 & 7 & 7 & 6 & 5 & 4 & 3 & $\underline{\mathbf{2}}$ & $\underline{\ma
$\mathrm{a}:$ & 8 & 8 & 8 & 7 & 6 & 5 & 4 & 3 & 3 & $\underline{\mathbf{4}}$ & 5 & 6 \
$\mathrm{l}:$ & 9 & 9 & 9 & 8 & 7 & 6 & 5 & 4 & 4 & 4 & $\underline{\mathbf{4}}$ & 5 \
$\mathrm{t}:$ & 10 & 10 & 10 & 9 & 8 & 7 & 6 & 5 & 5 & 5 & 5 & $\underline{\mathbf{5}}
\end{tabular}
```

Figure 10.5: Example of a dynamic programming matrix for editing distance computation, initial configuration (the pair of empty strings $(0,0)$) down to the final goal stat

Reconstructing these decisions is done by walking backward from the goal state, follow

Walking backward reconstructs the solution in reverse order. However, clever use of re

*void reconstruct_path(char *s, char *t, int i, int j,*
cell m[MAXLEN+1] [MAXLEN+1]) {
if (m[i][j].parent == -1) {
return;
}
if (m[i][j].parent == MATCH) {
reconstruct_path(s, t, i-1, j-1, m);

```
\begin{tabular}{lr|rrrrrrrrrrr}
& $T$ & & y & o & u & - & s & h & o & u & l & d \\
$P$ & pos & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\
\hline & 0 & $\mathbf{- 1}$ & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
$\mathrm{t}:$ & 1 & $\underline{\mathbf{2}}$ & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \
$\mathrm{~h}:$ & 2 & 2 & $\mathbf{0}$ & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\
o & 3 & 2 & 0 & $\underline{\mathbf{0}}$ & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\
$\mathrm{u}:$ & 4 & 2 & 0 & 2 & $\underline{\mathbf{0}}$ & 1 & 1 & 1 & 1 & 0 & 1 & 1 \
$\mathrm{-:}$ & 5 & 2 & 0 & 2 & 2 & $\underline{0}$ & 1 & 1 & 1 & 1 & 0 & 0 \\
$\mathrm{~s}:$ & 6 & 2 & 0 & 2 & 2 & 2 & $\underline{\mathbf{0}}$ & 1 & 1 & 1 & 1 & 0 
$\mathrm{~h}:$ & 7 & 2 & 0 & 2 & 2 & 2 & 2 & $\underline{\mathbf{0}}$ & $\underline{\ma
$\mathrm{a}:$ & 8 & 2 & 0 & 2 & 2 & 2 & 2 & 2 & 0 & $\underline{\mathbf{0}}$ & 0 & 0 \
$\mathrm{l}:$ & 9 & 2 & 0 & 2 & 2 & 2 & 2 & 2 & 0 & 0 & $\underline{\mathbf{0}}$ & 1 \
$\mathrm{t}:$ & 10 & 2 & 0 & 2 & 2 & 2 & 2 & 2 & 0 & 0 & 0 & $\underline{\mathbf{0}}$\
\end{tabular}
```

Figure 10.6: Parent matrix for edit distance computation, with the optimal alignment pa

```
    match_out(s, t, i, j);
    return;
}
if (m[i][j].parent == INSERT) {
```

```
    reconstruct_path(s, t, i, j-1, m);
    insert_out(t, j);
    return;
}
if (m[i][j].parent == DELETE) {
reconstruct_path(s, t, i-1, j, m);
delete_out(s, i);
    return;
}

   }
```

For many problems, including edit distance, the solution can be reconstructed from the cost matri

%---- Page End Break Here ---- Page : 334
\subsection{Varieties of Edit Distance}

The string_compare and path reconstruction routines reference several functions that we have not
- Table initialization - The functions row_init and column_init initialize the zeroth row and col

```
   row_init(int i)
{
m[0][i].cost = i;
if (i>0)
m[0][i].parent = INSERT;
else
m[0] [i].parent = - 1;
}
```

```
   column_init(int i)
{
m[i][0].cost = i;
if (i>0)
m[i][0].parent = DELETE;
else
m[i][0].parent = -1;
}
```

- Penalty costs - The functions match (c,d) and indel (c) present the costs for transforming char

```
   int match(char c, char d)
{
if (c == d) return(0);
else return(1);
}
```

- Goal cell identification - The function goal_cell returns the indices of the cell ma

```
void goal_cell(char *s, char *t, int *i, int *j) {
*i = strlen(s) - 1;
*j = strlen(t) - 1;
}
```

- Traceback actions - The functions match_out, insert_out, and delete_out perform the a

%---- Page End Break Here ---- Page : 335
For edit distance, this might mean printing out the name of the operation or character

```
insert_out(char *t, int j)
{
printf("I");
}
delete_out(char *s, int i)
{
printf("D");
}
```

All of these functions are quite simple for edit distance computation. However, we mus

This may seem like a lot of infrastructure to develop for such a simple algorithm. How
- Substring matching - Suppose we want to find where a short pattern P best occurs w

We want an edit distance search where the cost of starting the match is independent of

```
void row_init(int i, cell m[MAXLEN+1][MAXLEN+1]) {
m[0][i].cost = 0; /* NOTE CHANGE /
m[0][i].parent = -1; / NOTE CHANGE */

}
```

```
void goal_cell(char *s, char *t, int *i, int j) {
int k; / counter */
*i = strlen(s) - 1;
*j = 0;
for (k = 1; k < strlen(t); k++) {
if (m[*i][k].cost < m[*i][*j].cost) {
*j = k;
}
}
}
```

- Longest common subsequence - Perhaps we are interested in finding the longest scattered string

A common subsequence is defined by all the identical-character matches in an edit trace. To maxim

```
int match(char c, char d) {
if (c == d) {
return(0);
}
return(MAXLEN);
}
```

Actually, it suffices to make the substitution penalty greater than that of an insertion plus a d
- Maximum monotone subsequence - A numerical sequence is monotonically increasing if the i th e

%---- Page End Break Here ---- Page : 337
In fact, this is just a longest common subsequence problem, where the second string is the elemen

As you can see, our edit distance routine can be made to do many amazing things easily. The trick

The alert reader may notice that it is unnecessary to keep all $O(m n)$ cells to compute the cost

Saving space in dynamic programming is very important. Since memory on any computer is limited, u

\section{Longest Increasing Subsequence}

There are three steps involved in solving a problem by dynamic programming:
1. Formulate the answer you want as a recurrence relation or recursive algorithm.
2. Show that the number of different parameter values taken on by your recurrence is bounded by a
3. Specify an evaluation order for the recurrence so the partial results you need are always avai

To see how this is done, let's see how we would develop an algorithm to find the longest monotoni

We distinguish an increasing sequence from a run, where the elements must be physical neighbors o

$$
S=(2,4,3,5,1,7,6,9,8)
$$

%---- Page End Break Here ---- Page : 338
The longest increasing subsequence of S is of length 5 : for example, ($2,3,5,6,8$). In fact,

Finding the longest increasing run in a numerical sequence is straightforward. Indeed, you should

To apply dynamic programming, we need to design a recurrence relation for the length of the longe
- The length L of the longest increasing sequence in $\left(s_{1}, s_{2}, \ldots, s_{n-1}\right$

Unfortunately, this length L is not enough information to complete the full solution
- We need to know the length of the longest sequence that s_{n} will extend. To be co

This provides the idea around which to build a recurrence. Define L_{i} to be the len
$$
\begin{aligned}
L_{i} & =1+\max _{\substack{0 \leq j<i \\
s_{j}<s_{i}}} L_{j}, \\
L_{0} & =0
\end{aligned}
$$

These values define the length of the longest increasing sequence ending at each sequen
\begin{tabular}{r|ccccccccc}
Index i & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\
Sequence s_{i} & 2 & 4 & 3 & 5 & 1 & 7 & 6 & 9 & 8 \\
\hline Length L_{i} & 1 & 2 & 2 & 3 & 1 & 4 & 4 & 5 & 5 \\
Predecessor p_{i} & - & 1 & 1 & 2 & - & 4 & 4 & 6 & 6
\end{tabular}

What auxiliary information will we need to store to reconstruct the actual sequence ins

%---- Page End Break Here ---- Page : 339
What is the time complexity of this algorithm? Each one of the n values of L_{i} is

Take-Home Lesson: Once you understand dynamic programming, it can be easier to work out

\section{War Story: Text Compression for Bar Codes}

Ynjiun waved his laser wand over the torn and crumpled fragments of a bar code label. T

I was visiting the research laboratories of Symbol Technologies (now Zebra), the world

The ten-digit capacity of conventional bar code labels provides room enough to only sto
"PDF-417 is our new, two-dimensional bar code symbology," Ynjiun explained. A sample la
"How much data can you fit in a typical 1-inch label?" I asked him.
"It depends upon the level of error correction we use, but about 1,000 bytes. That's en
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-341.jpg?height=29

%---- Page End Break Here ---- Page : 340
Figure 10.7: A two-dimensional barcode label of the Gettysburg Address using PDF-417.
"Interesting. You should use some data compression technique to maximize the amount of
"We do incorporate a data compaction method," he explained. "We understand the differen
"I see. So you designed the mode character sets to minimize the number of mode switch c

"Right. We put all the digits in one mode and all the punctuation characters in another. We also
"Wow!" I said. "With all of this mode switching going on, there must be many different ways to en
"We use a greedy algorithm. We look a few characters ahead and then decide which mode we would be

I pressed him on this. "How do you know it works fairly well? There might be significantly better
"I guess I don't know. But it's probably NP-complete to find the optimal coding." Ynjiun's voice

I started to think. Every encoding starts in a given mode and consists of a sequence of intermixe
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-342.jpg?height=508&width=521

%---- Page End Break Here ---- Page : 341
Figure 10.8: Mode switching in PDF-417.
pair, we would have been interested in the cheapest way of getting there, over all possible encod

My eyes lit up so bright they cast shadows on the walls.
"The optimal encoding for any given text in PDF-417 can be found using dynamic programming. For e

Basically,
$$
M[i, j]=\min _{1 \leq m \leq 4}\left(M[i-1, m]+c\left(S_{i}, m, j\right)\right)
$$
where $c\left(S_{i}, m, j\right)$ is the cost of encoding character S_{i} and switching from mo

Ynjiun was skeptical, but he encouraged us to implement an optimal encoder. A few complications a

Our observed impact of replacing a heuristic solution with the global optimum is probably typical
heuristic, you should get a decent solution. Replacing it with an optimal result, however, usuall

%---- Page End Break Here ---- Page : 342
\section{Unordered Partition or Subset Sum}

The knapsack or subset sum problem asks whether there exists a subset $S^{\prime}$ of an input mu

Dynamic programming works best on linearly ordered items, so we can consider them from left to ri

Here is the idea. Either the nth integer s_{n} is part of a subset adding up to k, or it is
$$
T_{n, k}=T_{n-1, k} \vee T_{n-1, k-s_{n}}
$$

This gives an $O(n k)$ algorithm to decide whether target k is realizable:

bool sum[MAXN+1] [MAXSUM+1]; / table of realizable sums /*
int parent[MAXN+1][MAXSUM+1]; / table of parent pointers /

```
bool subset_sum(int s[], int n, int k) {
int i, j; / counters /
sum[0][0] = true;
parent[0][0] = NIL;
for (i = 1; i <= k; i++) {
sum[0][i] = false;
parent[0][i] = NIL;
}
for (i = 1; i <= n; i++) { / build table */
for (j = 0; j <= k; j++) {
sum[i][j] = sum[i-1][j];
parent[i][j] = NIL;
if ((j >= s[i-1]) && (sum[i-1][j-s[i-1]]==true)) {
```

```
        sum[i][j] = true;
        parent[i][j] = j-s[i-1];
    }
}
}
return(sum[n] [k]);
```

```
    }
```

```
The parent table encodes the actual subset of numbers totaling to $k$. An appropriate s
```

```
    void report_subset(int n, int k) {
if (k == 0) {
return;
}
if (parent[n] [k] == NIL) {
report_subset(n-1,k);
}
else {
report_subset(n-1,parent[n] [k]);
printf(" %d ",k-parent[n][k]);
}
}
```

```
Below is an example showing the sum table for input set $S=\{1,2,4,8\}$ and target $k=
\begin{tabular}{cc|cccccccccccc}
$i$ & $s_{i}$ & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\
\hline 0 & 0 & T & F & F & F & F & F & F & F & F & F & F & F \\
1 & 1 & T & T & F & F & F & F & F & F & F & F & F & F \\
2 & 2 & T & T & T & T & F & F & F & F & F & F & F & F \\
3 & 4 & T & T & T & T & T & T & T & T & F & F & F & F \\
```

4 & 8 & T & T & T & T & T & T & T & T & T & T & T & T
\end{tabular}

Below is the corresponding parents array, encoding the solution $1+2+8=$ 11. The 3 in the lower r
\begin{tabular}{cc|cccccccccccc}
i & s_{i} & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\
%---- Page End Break Here ---- Page : 344

\hline 0 & 0 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\
1 & 1 & -1 & 0 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\
2 & 2 & -1 & -1 & 0 & 1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\
3 & 4 & -1 & -1 & -1 & -1 & 0 & 1 & 2 & 3 & -1 & -1 & -1 & -1 \\
4 & 8 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 0 & 1 & 2 & 3
\end{tabular}

The alert reader might wonder how we can have an $O(n k)$ algorithm for subset sum when subset su

Unfortunately, no. Note that the target number k can be specified using $O(\log k)$ bits, meani

Another way to see the problem is to consider what happens to the algorithm when we take a specif

\section{War Story: The Balance of Power}

One of the many (presumably too many) uncharitable suspicions I hold is that most electrical engi

Thus, it was a relief when an EE professor and his students came to me with an honest EE problem,
"Alternating current (AC) power systems transmit electricity on each of three different phases. O
"I guess loads are the machines needing power, right?" I asked insightfully.
"Yeah, think of every house on the street as being a load. Each house will get assigned one of th
"Presumably they connect every third house A, B, C, A, B, C as they wire up the street to balan
"Something like that," the EE professor confirmed. "But not all houses use the same amount of pow
company might just turn on the lights when another runs an arc furnace. After we measure the load

%---- Page End Break Here ---- Page : 345
Now I saw the algorithmic problem. "So given a set of numbers representing the various loads, you
"Yeah. Can you give me a fast algorithm to do this?," he asked.
This seemed clear enough to me. It smelled like an integer partition problem, namely the subset s

The generalization of the problem to partition into three subsets instead of two was straightforw

I broke the bad news gently. "Integer partition is an NP-complete problem, and three-phase balanc

They got up and started to leave. But then I remembered the dynamic programming algorithm for sub
$C\left[n, w_{A}, w_{B}\right]=C\left[n-1, w_{A}-s_{n}, w_{B}\right] \vee C\left[n-1, w_{A}, w_{B
This took constant time per cell to update, but there were $n k^{2}$ cells to update, where k i

This pleased them immensely, and they set to work to implement the algorithm. But I ha
"Uh, maybe impedance matching and, uh, complex numbers?" he fumphered. His advisor sho

But that computer engineering student could code, and that was what mattered here. He

Our dynamic programming algorithm always produced at least as good a solution as sever
could be a problem, but binning the loads by (say) $\left\lfloor s_{i} / 10\right\rflo

%---- Page End Break Here ---- Page : 346
Dynamic programming really proved its worth when our electrical engineers got interest
$$
\begin{gathered}
C\left[n, w_{A}, w_{B}\right]=\min \left(C\left[n-1, w_{A}-s_{n}, w_{B}\right]+1,\righ
C\left[n-1, w_{A}, w_{B}-s_{n}\right]+1, \\
\left.C\left[n-1, w_{A}, w_{B}\right]\right)
\end{gathered}
$$

They then got greedy, and wanted the lowest cost solution that never got seriously unb

That is the power of dynamic programming. Once you can reduce your state space to a sm

\section{The Ordered Partition Problem}

Suppose that three workers are given the task of scanning through a shelf of books in

But what is the fairest way to divide up the shelf? If all books are the same length,
$$
100100100 \text { | } 100100100 \text { | } 100100100
$$
so that everyone has 300 pages to deal with.
But what if the books are not the same length? Suppose we used the same partition when
$$
100200300|400500600| 700800900
$$

I would volunteer to take the first section, with only 600 pages to scan, instead of t
where the largest job is only 1,700 pages.
In general, we have the following problem:
Problem: Integer Partition without Rearrangement
Input: An arrangement S of non-negative numbers $s_{1}, \ldots, s_{n}$ and an intege

%---- Page End Break Here ---- Page : 347

This so-called ordered partition problem arises often in parallel processing. We seek to balance

Stop for a few minutes and try to find an algorithm to solve the linear partition problem.

A novice algorist might suggest a heuristic as the most natural approach to solving the partition

Instead, consider a recursive, exhaustive search approach to solving this problem. Notice that th
- the cost of the last partition $\sum_{j=i+1}^{n} s_{j}$, and
- the cost of the largest partition formed to the left of the last divider.

What is the size of this left partition? To minimize our total, we must use the $k-2$ remaining d

Therefore, define $M[n, k]$ to be the minimum possible cost over all partitionings of $s_{1}, \ld
$$
M[n, k]=\min _{i=1}^{n}\left(\max \left(M[i, k-1], \sum_{j=i+1}^{n} s_{j}\right)\right)
$$

We also need to specify the boundary conditions of the recurrence relation. These boundary condit
\begin{tabular}{|l|lll|l|l|lll|}
\hline$M$ & \multicolumn{3}{|c|}{k} & k & & D & & k \\
\hline$s$ & 1 & 2 & 3 & & s & 1 & 2 & 3 \\
1 & 1 & 1 & 1 & & 1 & - & - & - \\
1 & 2 & 1 & 1 & & 1 & - & 1 & 1 \\
1 & 3 & 2 & 1 & & 1 & - & 1 & 2 \\
1 & 4 & 2 & 2 & & 1 & - & 2 & 2 \\
1 & 5 & 3 & 2 & & 1 & - & 2 & 3 \\
1 & 6 & 3 & 2 & & 1 & - & $\mathbf{3}$ & 4 \\
1 & 7 & 4 & 3 & & 1 & - & 3 & 4 \\
1 & 8 & 4 & 3 & & 1 & - & 4 & 5 \\
1 & 9 & 5 & 3 & & 1 & - & 4 & $\mathbf{6}$ \\
\hline
\end{tabular}
\begin{tabular}{|l|rrr|l|l|lll|}
\hline$M$ & \multicolumn{3}{|c|}{k} & & D & & k & \\
\hline$s$ & 1 & 2 & 3 & & s & 1 & 2 & 3 \\
1 & 1 & 1 & 1 & & 1 & - & - & - \\
2 & 3 & 2 & 2 & & 2 & - & 1 & 1 \\
3 & 6 & 3 & 3 & & 3 & - & 2 & 2 \\
4 & 10 & 6 & 4 & & 4 & - & 3 & 3 \\
5 & 15 & 9 & 6 & & 5 & - & 3 & 4 \\
6 & 21 & 11 & 9 & & 6 & - & 4 & 5 \\
7 & 28 & 15 & 11 & & 7 & - & $\mathbf{5}$ & 6 \\
8 & 36 & 21 & 15 & & 8 & - & 5 & 6 \\
9 & 45 & 24 & 17 & & 9 & - & 6 & $\mathbf{7}$ \\
\hline
\end{tabular}

Figure 10.9: Dynamic programming matrices M and D for two instances of the ordered
many dividers are used. The smallest reasonable value of the second argument is $k=1$,
$$
\begin{aligned}
& M[1, k]=s_{1}, \text { for all } k>0 \\
%---- Page End Break Here ---- Page : 348

& M[n, 1]=\sum_{i=1}^{n} s_{i}
\end{aligned}
$$

How long does it take to compute this when we store the partial results? There are a to

The evaluation order computes the smaller values before the bigger values, so that each

```
void partition(int s[], int n, int k) {
int p[MAXN+1]; /* prefix sums array /
int m[MAXN+1][MAXK+1]; / DP table for values /
int d[MAXN+1][MAXK+1]; / DP table for dividers /
int cost; / test split cost /
int i,j,x; / counters /
p[0] = 0; / construct prefix sums */
for (i = 1; i <= n; i++) {
p[i] = p[i-1] + s[i];

}
```

```
for (i = 1; i <= n; i++) {
m[i][1] = p[i]; /* initialize boundaries */
}
for (j = 1; j <= k; j++) {
    m[1][j] = s[1];
}
for (i = 2; i <= n; i++) { /* evaluate main recurrence */
    for (j = 2; j <= k; j++) {
    m[i][j] = MAXINT;
    for (x = 1; x <= (i-1); x++) {
        cost = max(m[x][j-1], p[i]-p[x]);
        if (m[i][j] > cost) {
            m[i][j] = cost;
            d[i][j] = x;
            }
        }
}
```

```
    }
reconstruct_partition(s, d, n, k); /* print book partition */
}
```

This implementation above, in fact, runs faster than advertised. Our original analysis assumed th

By studying the recurrence relation and the dynamic programming matrices of these two examples, y

The second matrix, D, is used to reconstruct the optimal partition. Whenever we update the valu

```
    void reconstruct_partition(int s[],int d[MAXN+1][MAXK+1], int n, int k)
{
if (k == 1) {
print_books(s, 1, n);
} else {
reconstruct_partition(s, d, d[n] [k], k-1);
print_books(s, d[n][k]+1, n);
}
}
```

```
void print_books(int s[], int start, int end) \{
    int i; /* counter */
    printf("\\{"); }
    for (i = start; i <= end; i++) \{
        printf(" \%d ", s[i]);
    \}
    printf("\}\n");
\}
```

\section{Parsing Context-Free Grammars}

Compilers identify whether a particular program is a legal expression in a particular programming

Parsing a given text sequence S as per a given context-free grammar G is the algorithmic prob

Parsing seemed like a horribly complicated subject when I took a compilers course as a graduate s

We assume that the sequence S has length n while the grammar G itself is of constant size.

Further, we assume that the definitions of each rule are in Chomsky normal form, like the example

sentence ::= noun-phrase
verb-phrase
noun-phrase ::= article noun
verb-phrase ::= verb noun-phrase
article ::= the, a
noun ::= cat, milk
verb ::= drank

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-352.jpg?height=34

Figure 10.10: A context-free grammar (on left) with an associated parse tree (right)
(b) exactly one terminal symbol, $X \rightarrow \alpha$. Any context-free grammar can

So how can we efficiently parse S using a context-free grammar where each interestin

This suggests a dynamic programming algorithm, where we keep track of all nonterminals
$$
M[i, j, X]=\bigvee_{(X \rightarrow Y Z) \in G}\left(\bigvee_{k=i}^{j-1} M[i, k, Y] \we
$$
where $\vee$ denotes the logical or over all productions and split positions, and $\we

The terminal symbols define the boundary conditions of the recurrence. In particular,

What is the complexity of this algorithm? The size of our state-space is $O\left(n^{2}

Stop and Think: Parsimonious Parserization

%---- Page End Break Here ---- Page : 352
Problem: Programs often contain trivial syntax errors that prevent them from compiling

Solution: This problem seemed extremely difficult when I first encountered it. But on

Define $M^{\prime}[i, j, X]$ to be an integer function that reports the minimum number
$$
M^{\prime}[i, j, X]=\min _{(X \rightarrow Y Z) \in G}\left(\min _{k=i}^{j-1} M^{\prime
$$

The boundary conditions also change mildly. If there exists a production $X \rightarro

Take-Home Lesson: For optimization problems on left-to-right objects, such as characte

\section{Limitations of Dynamic Programming: TSP}

Dynamic programming doesn't always work. It is important to see why it can fail, to he

Our algorithmic poster child will once again be the traveling salesman problem, where

Problem: Longest Simple Path
Input: A weighted graph $G=(V, E)$, with specified start and end vertices s and t.
Output: What is the most expensive path from s to t that does not visit any vertex

This problem differs from TSP in two quite unimportant ways. First, it asks for a path

```
%---- Page End Break Here ---- Page : 353
For unweighted graphs (where each edge has cost 1), the longest possible simple path from $s$ to

\subsection{When is Dynamic Programming Correct?}

Dynamic programming algorithms are only as correct as the recurrence relations they are based on.
$$
L P[i, j]=\max _{\substack{x \in V \\(x, j) \in E}} L P[i, x]+c(x, j)
$$

This idea seems reasonable, but can you see the problem? I see at least two of them.

First, this recurrence does nothing to enforce simplicity. How do we know that vertex $j$ has not

The second problem concerns evaluation order. What can you evaluate first? Because there is no le

Dynamic programming can be applied to any problem that obeys the principle of optimality. Roughly


%---- Page End Break Here ---- Page : 354
Problems do not satisfy the principle of optimality when the specifics of the operations matter,

\subsection{When is Dynamic Programming Efficient?}

The running time of any dynamic programming algorithm is a function of two things: (1) the number

In all of the examples we have seen, the partial solutions are completely described by specifying

When the objects are not firmly ordered, however, we likely have an exponential number of possibl
$$
L P\left[i, j, P_{i j}\right]=\max _{\substack{j \notin P_{i x} \\(x, j) \in E \\ P_{i j}=P_{i x}
$$

This formulation is correct, but how efficient is it? The path $P_{i j}$ consists of an ordered s

We can do something better with this idea, however. Let $L P^{\prime}\left[i, j, S_{i j}\right]$
$$
L P^{\prime}\left[i, j, S_{i j}\right]=\max _{\substack{j \notin S_{i x} \\(x, j) \in E \\ S_{i j
$$
where $S \cup\{x\}$ denotes unioning $S$ with $x$.
The longest simple path from $i$ to $j$ can then be found by maximizing over all possible interme
$$
L P[i, j]=\max _{S} L P^{\prime}[i, j, S]
$$
```

%---- Page End Break Here ---- Page : 355
There are only 2^{n} subsets of n vertices, so this is a big improvement over enume

Take-Home Lesson: Without an inherent left-to-right ordering on the objects, dynamic p

\section{War Story: What's Past is Prolog}
"But our heuristic works very, very well in practice." My colleague was simultaneously

Unification is the basic computational mechanism in logic programming languages like P

An execution of a Prolog program starts by specifying a goal, say $p(a, X, Y)$, where
$$
\begin{aligned}
& p(a, a, a):=h(a) ; \\
& p(b, a, a):=h(a) * h(b) ; \\
& p(c, b, b):=h(b)+h(c) ; \\
& p(d, b, b):=h(d)+h(b) ;
\end{aligned}
$$
"In order to speed up unification, we want to preprocess the set of rule heads so that

Tries are extremely useful data structures in working with strings, as discussed in Se
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-357.jpg?height=47

%---- Page End Break Here ---- Page : 356
Figure 10.11: Two different tries for the same set of Prolog rule heads, where the tri
"I agree. A trie is a natural way to represent your rule heads. Building a trie on a s
"The efficiency of our unification algorithm depends very much on minimizing the numbe

He showed me the example in Figure 10.11. A trie constructed according to the original
"Interesting. . ." I started to reply before he cut me off again.
"There's one other constraint. We must keep the leaves of the trie ordered, so that th

Then came my mission.
"We have a greedy heuristic for building good, but not optimal, tries that picks as th

I agreed to try to prove the hardness of the problem, and chased him from my office. T
complete problem that had such a left-to-right ordering constraint. After all, if a gi

%---- Page End Break Here ---- Page : 357
Bingo! That settled it.
The next day I went back to my colleague and told him. "I can't prove that your proble

My recurrence looked something like this. Suppose that we are given n ordered rule heads $s_{1}

$$
C[i, j]=\min _{p=1}^{m}\left(\sum_{k=1}^{r}\left(C\left[i_{k}, j_{k}\right]+1\right)\right)
$$

A graduate student immediately set to work implementing this algorithm to compare with their heur

The fact that the rules had to remain ordered was the crucial property that we exploited in the d

Take-Home Lesson: The global optimum (found perhaps using dynamic programming) is often noticeabl

%---- Page End Break Here ---- Page : 358

Chapter Notes

Bellman \\cite{bel58} is credited with inventing the technique of dynamic programming. The edit d

Techniques such as dynamic programming and backtracking can be used to generate worst-case effici

More details about the war stories in this chapter are available in published papers. See Dawson

The dynamic programming algorithm presented for parsing is known as the CKY algorithm after its t

\section{Exercises}

Elementary Recurrences

10-1. [3] Up to k steps in a single bound! A child is running up a staircase with n steps and
10-2. [3] Imagine you are a professional thief who plans to rob houses along a street of n home
10-3. [5] Basketball games are a sequence of 2-point shots, 3-point shots, and 1point free throws
10-4. [5] Basketball games are a sequence of 2-point shots, 3-point shots, and 1-point free throw
10-5. [5] Given an $s \times t$ grid filled with non-negative numbers, find a path from top left
(a) Give a solution based on Dijkstra's algorithm. What is its time complexity as a function of $
(b) Give a solution based on dynamic programming. What is its time complexity as a function of $s

%---- Page End Break Here ---- Page : 359

Edit Distance

10-6. [3] Typists often make transposition errors exchanging neighboring characters, such as typi
Incorporate a swap operation into our edit distance function, so that such neighboring transposit
10-7. [4] Suppose you are given three strings of characters: X, Y, and Z, where $|X|=n,|Y|=m$
(a) Show that cchocohilaptes is a shuffle of chocolate and chips, but chocochilatspe is not.
(b) Give an efficient dynamic programming algorithm that determines whether Z is a shuffle of $

10-8. [4] The longest common substring (not subsequence) of two strings X and Y is the longes

(a) Let $n=|X|$ and $m=|Y|$. Give a $\Theta(n m)$ dynamic programming algorithm for lon
(b) Give a simpler $\Theta(n m)$ algorithm that does not rely on dynamic programming.

10-9. [6] The longest common subsequence (LCS) of two sequences T and P is the long
(a) Give efficient algorithms to find the LCS and SCS of two given sequences.
(b) Let $d(T, P)$ be the minimum edit distance between T and P when no substitution

10-10. [5] Suppose you are given n poker chips stacked in two stacks, where the edges
For example, consider the stacks $(R R G G, G B B B)$. Playing red, green, and then blu

%---- Page End Break Here ---- Page : 360
Greedy Algorithms

10-11. [4] Let $P_{1}, P_{2}, \ldots, P_{n}$ be n programs to be stored on a disk wit
(a) Does a greedy algorithm that selects programs in order of non-decreasing s_{i} ma
(b) Does a greedy algorithm that selects programs in order of non-increasing s_{i} us

10-12. [5] Coins in the United States are minted with denominations of $1,5,10,25$, and
(a) The greedy algorithm repeatedly selects the biggest coin no bigger than the amount
(b) Give an efficient algorithm that correctly determines the minimum number of coins r

10-13. [5] In the United States, coins are minted with denominations of 1, 5, 10, 25, a
(a) How many ways are there to make change of 20 units from $\{1,6,10\}$?
(b) Give an efficient algorithm to compute $C(n)$, and analyze its complexity. (Hint: t

10-14. [6] In the single-processor scheduling problem, we are given a set of n jobs S

Show that if a feasible schedule exists, then the schedule produced by this greedy algo

%---- Page End Break Here ---- Page : 361
Number Problems

10-15. [3] You are given a rod of length n inches and a table of prices obtainable fo
\begin{tabular}{l|rrrrrrrr}
length & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\
\hline price & 1 & 5 & 8 & 9 & 10 & 17 & 17 & 20
\end{tabular}
then the maximum obtainable value is 22 , by cutting into pieces of lengths 2 and 6 .

10-16. [5] Your boss has written an arithmetic expression of n terms to compute your
$$
6+2 \times 0-4
$$
there exist parenthesizations with values ranging from -32 to 2 .

```
10-17. [5] Given a positive integer $n$, find an efficient algorithm to compute the smallest numb
10-18. [5] Given an array $A$ of $n$ integers, find an efficient algorithm to compute the largest
10-19. [5] Two drivers have to divide up $m$ suitcases between them, where the weight of the $i$
10-20. [6] The knapsack problem is as follows: given a set of integers $S=\left\{s_{1}, s_{2}, \l
Give a dynamic programming algorithm for knapsack that runs in $O(n T)$ time.
10-21. [6] The integer partition takes a set of positive integers $S=\left\{s_{1}, \ldots, s_{n}\
$$
\sum_{i \in I} s_{i}=\sum_{i \notin I} s_{i}
$$

Let $\sum_{i \in S} s_{i}=M$. Give an $O(n M)$ dynamic programming algorithm to solve the integer
10-22. [5] Assume that there are $n$ numbers (some possibly negative) on a circle, and we wish to

10-23. [5] A certain string processing language allows the programmer to break a string into two
a 20 -character string after characters 3,8 , and 10 . If the breaks are made in left-to-right or
Give a dynamic programming algorithm that takes a list of character positions after which to brea


%---- Page End Break Here ---- Page : 362
10-24. [5] Consider the following data compression technique. We have a table of $m$ text strings

10-25. [5] The traditional world chess championship is a match of 24 games. The current champion
(a) Write a recurrence for the probability that the champion retains the title. Assume that there
(b) Based on your recurrence, give a dynamic programming algorithm to calculate the champion's pr
(c) Analyze its running time for an $n$ game match.

10-26. [8] Eggs break when dropped from great enough height. Specifically, there must be a floor
You seek to find the critical floor $f$ using an $n$-floor building. The only operation you can p
(a) Show that $E(1, n)=n$.
(b) Show that $E(k, n)=\Theta\left(n^{\frac{1}{k}}\right)$.
(c) Find a recurrence for $E(k, n)$. What is the running time of the dynamic program to find $E(k

Graph Problems


%---- Page End Break Here ---- Page : 363
10-27. [4] Consider a city whose streets are defined by an $X \times Y$ grid. We are interested i
Unfortunately, the city has bad neighborhoods, whose intersections we do not want to walk in. We
(a) Give an example of the contents of bad such that there is no path across the grid avoiding ba
(b) Give an $O(X Y)$ algorithm to find a path across the grid that avoids bad neighborhoods.
(c) Give an $O(X Y)$ algorithm to find the shortest path across the grid that avoids bad neighbor
10-28. [5] Consider the same situation as the previous problem. We have a city whose streets are
If there were no bad neighborhoods to contend with, the shortest path across the grid would have
Give an algorithm that takes the array bad and returns the number of safe paths of length $X+Y-2$
10-29. [5] You seek to create a stack out of $n$ boxes, where box $i$ has width $w_{i}$, height $
```

Design Problems

10-30. [4] Consider the problem of storing n books on shelves in a library. The orde
Suppose all the books have the same height h (i.e., $h=h_{i}$ for all i) and the s
10-31. [6] This is a generalization of the previous problem. Now consider the case whe
- Give an example to show that the greedy algorithm of stuffing each shelf as full as
- Give an algorithm for this problem, and analyze its time complexity. (Hint: use dyna

%---- Page End Break Here ---- Page : 364
10-32. [5] Consider a linear keyboard of lowercase letters and numbers, where the left
Give a dynamic programming algorithm that finds the most efficient way to type a given
10-33. [5] You have come back from the future with an array G, where $G[i]$ tells yo
10-34. [8] You are given a string of n characters $S=s_{1} \ldots s_{n}$, which you
(a) You seek to reconstruct the document using a dictionary, which is available in the
(b) Now assume you are given the dictionary as a set of m words each of length at mo
10-35. [8] Consider the following two-player game, where you seek to get the biggest s
10-36. [6] Given an array of n real numbers, consider the problem of finding the max
$$
[31,-41,59,26,-53,58,97,-93,-23,84]
$$
the maximum is achieved by summing the third through seventh elements, where $59+26+(-
- Give a simple and clear $\Theta\left(n^{2}\right)$-time algorithm to find the maximu
- Now give a $\Theta(n)$-time dynamic programming algorithm for this problem. To get p

%---- Page End Break Here ---- Page : 365
10-37. [7] Consider the problem of examining a string $x=x_{1} x_{2} \ldots x_{n}$ fro
\begin{tabular}{c|ccc}
& a & b & c \\
\hline$a$ & a & c & c \\
b & a & a & b \\
c & c & c & c
\end{tabular}

For example, consider the above multiplication table and the string $b b b b a$. Paren
Give an algorithm, with time polynomial in n and k, to decide whether such a paren

10-38. [6] Let α and β be constants. Assume that it costs α to go

Interview Problems

10-39. [5] Given a set of coin denominations, find the minimum number of coins to make

10-40. [5] You are given an array of n numbers, each of which may be positive, negat

10-41. [7] Observe that when you cut a character out of a magazine, the character on the reverse

LeetCode

10-1. https://leetcode.com/problems/binary-tree-cameras/
10-2. https://leetcode.com/problems/edit-distance/
10-3. https://leetcode.com/problems/maximum-product-of-splitted-binary-tree/

%---- Page End Break Here ---- Page : 366
HackerRank

10-1. https://www.hackerrank.com/challenges/ctci-recursive-staircase/
10-2. https://www.hackerrank.com/challenges/coin-change/
10-3. https://www.hackerrank.com/challenges/longest-increasing-subsequent/

Programming Challenges

These programming challenge problems with robot judging are available at https://onlinejudge.org\
10-1. "Is Bigger Smarter?"-Chapter 11, problem 10131.
10-2. "Weights and Measures"-Chapter 11, problem 10154.
10-3. "Unidirectional TSP"-Chapter 11, problem 116.
10-4. "Cutting Sticks"-Chapter 11, problem 10003.
10-5. "Ferry Loading" - Chapter 11, problem 10261.

%---- Page End Break Here ---- Page : 367
Chapter 11

\chapter{NP-Completeness}

I will now introduce techniques for proving that no efficient algorithm can exist for a given pro

The truth is that the theory of NP-completeness is an immensely useful tool for the algorithm des

The theory of NP-completeness also enables us to identify which properties make a particular prob

The fundamental concept we will use here is reduction, showing that two problems are really equiv

\section{Problems and Reductions}

We have encountered several problems in this book where we couldn't find any efficient algorithm.

The key idea to demonstrating the hardness of a problem is that of a re-
duction, or translation, between two problems. The following allegory of NPcompleteness may help

Reductions are algorithms that convert one problem into another. To describe them, we r

Problem: The Traveling Salesman Problem (TSP)
Input: A weighted graph G.
Output: Which tour $\left(v_{1}, v_{2}, \ldots, v_{n}\right)$ minimizes $\sum_{i=1}^{n
Any weighted graph defines an instance of TSP. Each particular instance has at least o

. 1 The Key Idea

Now consider two algorithmic problems, called Bandersnatch and Bo-billy. Suppose that I

Bandersnatch (G)
Translate the input G to an instance Y of the Bo-billy problem. Call the subroutin
Return the answer of Bo-billy (Y) as the answer to Bandersnatch (G).

This algorithm will correctly solve the Bandersnatch problem provided that the transla

$$
\operatorname{Bandersnatch}(G)=\operatorname{Bo-billy}(Y)
$$

A translation of instances from one type of problem to instances of another such that

Now suppose this reduction translates instance G to Y in $O(P(n))$ time. There are
- If my Bo-billy subroutine ran in $O\left(P^{\prime}(n)\right)$, this yields an algor
- If I know that $\Omega\left(P^{\prime}(n)\right)$ is a lower bound on computing Band

%---- Page End Break Here ---- Page : 369
Essentially, this reduction shows that Bo-billy is no easier than Bandersnatch. Theref

Take-Home Lesson: Reductions are a way to show that two problems are essentially ident

\subsection{Decision Problems}

Reductions translate between problems so that their answers are identical in every pro

The simplest interesting class of problems have answers restricted to true and false.

Fortunately, most interesting optimization problems can be phrased as decision problem

Problem: The Traveling Salesman Decision Problem (TSDP)
Input: A weighted graph G and integer k.
Output: Does there exist a TSP tour with cost $\leq k$?
This decision version captures the heart of the traveling salesman problem, in that if

From now on I will generally talk about decision problems, because they prove easier t

%---- Page End Break Here ---- Page : 370
\section{Reductions for Algorithms}

Reductions are an honorable way to generate new algorithms from old ones. Whenever we can transla

In this section, we look at several reductions that lead to efficient algorithms. To solve proble

\subsection{Closest Pair}

The closest-pair problem asks to find the pair of numbers within a set S that have the smallest

Input: A set S of n numbers, and threshold t.
Output: Is there a pair $s_{i}, s_{j} \in S$ such that $\left|s_{i}-s_{j}\right| \leq t$?
Finding the closest pair is a simple application of sorting, since the closest pair must be neigh

CloseEnoughPair (S, t)
Sort S.
$$
\text { Is } \min _{1 \leq i<n}\left|s_{i+1}-s_{i}\right| \leq t ?
$$

There are several things to note about this simple reduction:
- The decision version captured what is interesting about the general problem, meaning it is no e
- The complexity of this algorithm depends upon the complexity of sorting. Use an $O(n \log n)$ a
- This reduction and the fact that there is an $\Omega(n \log n)$ lower bound on sorting does not
- On the other hand, if we knew that a close-enough pair required $\Omega(n \log n)$ time to solv

%---- Page End Break Here ---- Page : 371
\subsection{Longest Increasing Subsequence}

Recall Chapter 10 where dynamic programming was used to solve a variety of problems, including st

Problem: Edit Distance
Input: Integer or character sequences S and T; penalty costs for each insertion $\left(c_{\te
Output: What is the cost of the least expensive sequence of operations that transforms S to T

It was shown that many other problems can be solved using edit distance. But these algorithms can

Problem: Longest Increasing Subsequence (LIS)
Input: An integer or character sequence S.
Output: What is the length of the longest sequence of positions $p_{1}, \ldots, p_{m}$ such that

In Section 10.3 (page 324) I demonstrated that longest increasing subsequence can be solved as a

$$
\begin{aligned}
& \text { LongestIncreasingSubsequence }(S) \\
& \quad T=\operatorname{Sort}(S) \\
& c_{\text {ins }}=c_{\text {del }}=1 \\
& c_{\text {sub }}=\infty \\
& \text { Return }\left(|S|-\operatorname{EditDistance}\left(S, T, c_{i n s}, c_{d e l}
\end{aligned}
$$

Why does this work? By constructing the second sequence T as the elements of S sor

What are the implications of this reduction? The reduction takes $O(n \log n)$ time be

\subsection{Least Common Multiple}

The least common multiple (lcm) and greatest common divisor (gcd) problems arise often

Problem: Least Common Multiple (lcm)

%---- Page End Break Here ---- Page : 372
Input: Two positive integers x and y.
Output: Return the smallest positive integer m such that m is a multiple of x an

Problem: Greatest Common Divisor (gcd)
Input: Two positive integers x and y.
Output: Return the largest integer d such that d divides both x and y.
For example, $\operatorname{lcm}(24,36)=72$ and $\operatorname{gcd}(24,36)=12$. Both p

$$
\text { if }(b \mid a) \text {, then } \operatorname{gcd}(a, b)=b
$$

This should be pretty clear. if b divides a, then $a=b k$ for some integer k, an

If $a=b t+r$ for integers t and r, then $\operatorname{gcd}(a, b)=g c d(b, r)$.
Then, for $a \geq b$, Euclid's algorithm repeatedly replaces (a, b) by ($b, a \bmod

Since $x \cdot y$ is a multiple of both x and $y, \operatorname{lcm}(x, y) \leq x y$

> LeastCommonMultiple (x, y)
> Return $(x y / \operatorname{gcd}(x, y))$.

This reduction gives us a nice way to reuse Euclid's efforts for $l \mathrm{~cm}$.

\subsection{Convex Hull (*)}

My final example of a reduction from an "easy" problem (meaning one that can be solved in polynom

Problem: Convex Hull

Input: A set S of n points in the plane.
Output: Find the smallest convex polygon containing all the points of S.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-374.jpg?height=466&width=647

%---- Page End Break Here ---- Page : 373
Figure 11.1: Reducing sorting to convex hull by mapping points to a parabola

I will now show how to transform an instance of sorting (say $\{13,5,11,17\}$) to an instance of

Sort(S)
For each $i \in S$, create point $\left(i, i^{2}\right)$.
Call subroutine convex-hull on this point set.
From the left-most point in the hull, read off the points from left to right.

What does this mean? Recall the sorting lower bound of $\Omega(n \log n)$. If we could compute co

\section{Elementary Hardness Reductions}

The reductions in Section 11.2 (page 358) demonstrate transformations between pairs of problems

For now, I want you to trust me when I say that Hamiltonian cycle and vertex cover are hard probl
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-375.jpg?height=674&width=997

%---- Page End Break Here ---- Page : 374
Figure 11.2: A portion of the reduction tree for NP-complete problems. Blue lines denote the redu

\subsection{Hamiltonian Cycle}

The Hamiltonian cycle problem is one of the most famous in graph theory. It seeks a tour that vis

Problem: Hamiltonian Cycle

Input: An unweighted graph G.
Output: Does there exist a simple tour that visits each vertex of G without repetition?

Hamiltonian cycle has some obvious similarity to the traveling salesman problem. Both problems se
$\operatorname{HamiltonianCycle}(G=(V, E))$
Construct a complete weighted graph $G^{\prime}=\left(V^{\prime}, E^{\prime}\right)$ where $V^{\p
$n=|V|$

```
for $i=1$ to $n$ do
for $j=1$ to $n$ do
if $(i, j) \in E$ then $w(i, j)=1$ else $w(i, j)=2$
Return the answer to Traveling-Salesman-Decision-Problem $\left(G^{\prime}, n\right)$.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-376.jpg?height=26


%---- Page End Break Here ---- Page : 375
Figure 11.3: Graphs with (left) and without (right) a Hamiltonian cycle.

The actual reduction is quite simple, with the translation from unweighted to weighted

This reduction is truth preserving and fast, running in $\Theta\left(n^{2}\right)$ tim

\subsection{Independent Set and Vertex Cover}

The vertex cover problem, discussed more thoroughly in Section 19.3 (page 591), asks f

Problem: Vertex Cover

Input: A graph $G=(V, E)$ and integer $k \leq|V|$.
Output: Is there a subset $S$ of at most $k$ vertices such that every $e \in E$ contai

It is trivial to find $a$ vertex cover of a graph: consider the cover that consists of

A set of vertices $S$ of graph $G$ is independent if there are no edges $(x, y)$ where

Problem: Independent Set
Input: A graph $G$ and integer $k \leq|V|$.
Output: Does there exist a set of $k$ independent vertices in $G$ ?
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-377.jpg?height=39


%---- Page End Break Here ---- Page : 376
Figure 11.4: Red vertices form a vertex cover of $G$, so the blue vertices must define

Both vertex cover and independent set are problems that revolve around finding special
$$
\begin{gathered}
\text { VertexCover }(G, k) \\
\begin{array}{l}
G^{\prime}=G \\
k^{\prime}=|V|-k
\end{array}
\end{gathered}
$$
```

```
Return the answer to IndependentSet $\left(G^{\prime}, k^{\prime}\right)$
Again, a simple reduction shows that one problem is at least as hard as the other. Notice how tra

Stop and Think: Hardness of General Movie Scheduling

Problem: Recall the movie scheduling problem, discussed in Section 1.2 (page 8). There, each poss

The general problem allows movie projects to have discontinuous schedules. For example, Project $

Prove that the general movie scheduling problem is NP-complete, with a reduction from independent
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-378.jpg?height=368&width=113


%---- Page End Break Here ---- Page : 377

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{images/algorithms_page_0379_0.png}
\end{figure}
Figure 11.5: Reduction from independent set to generalized movie scheduling, with numbered vertic

Problem: General Movie Scheduling Decision Problem

Input: A set $I=\left\{I_{1}, \ldots, I_{n}\right\}$ of $n$ sets of intervals on the line, intege
Output: Can a subset of at least $k$ mutually non-overlapping interval sets be selected from $I$

Solution: To prove a problem hard, we first need to establish which is Bandersnatch and which is

What is the correspondence between the two problems? Both problems involve selecting the largest

My construction is as follows. Create an interval on the line for each of the $m$ edges of the gr
$$
\begin{aligned}
& \text { IndependentSet }(G, k) \\
& \quad I=\emptyset \\
& \text { For the } i \text { th edge }(x, y), 1 \leq i \leq m \\
& \quad \text { Add interval }[i, i+0.5] \text { to movie } x \text { 's interval set } I_{x} \te
& \text { Add interval }[i, i+0.5] \text { to movie } y \text { 's interval set } I_{y} \text { i
& \text { Return the answer to GeneralMovieScheduling }(I, k)
\end{aligned}
$$

Each pair of vertices sharing an edge (forbidden to be in an independent set) defines a pair of m
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-379.jpg?height=342&width=654
```

%---- Page End Break Here ---- Page : 378
Figure 11.6: A small graph with a four-vertex clique (left), with the corresponding inc

\subsection{Clique}

A social clique is a group of mutual friends who all hang around together. Everyone kno

Problem: Maximum Clique
Input: A graph $G=(V, E)$ and integer $k \leq|V|$.
Output: Does the graph contain a clique of k vertices, meaning is there a subset of v

The graph in Figure 11.6 contains a clique of four blue vertices. Within the friendship

In the independent set problem, we looked for a subset S with no edges between two ve

IndependentSet (G, k)
Construct a graph $G'=(V', E')$ *where* $V'=V$, *and*
For all $(i, j) \notin E$, *add* (i, j) *to* E'
Return Clique (G', k)

These last two reductions provide a chain linking three different problems together. Th

\section{Satisfiability}

%---- Page End Break Here ---- Page : 379
To demonstrate the hardness of problems by using reductions, we must start from a singl

Problem: Satisfiability (SAT)
Input: A set of Boolean variables V and a set of logic clauses C over V.
Output: Does there exist a satisfying truth assignment for C-in other words, a way to

This can be made clear with two examples. Consider $C=\left\{\left\{v_{1}, \bar{v}_{2}\
$$
\left(v_{1} \vee \bar{v}_{2}\right) \wedge\left(\bar{v}_{1} \vee v_{2}\right)
$$
and can be satisfied either by setting $v_{1}=v_{2}=$ true or $v_{1}=v_{2}=$ false.
However, consider the set of clauses $\left\{\left\{v_{1}, v_{2}\right\},\left\{v_{1},

For a combination of social and technical reasons, it is well accepted that satisfiabil

\subsection{3-Satisfiability}

Satisfiability's role as the first NP-complete problem implies that the problem is hard
that directly contradict each other, such as $C=\left\{\left\{v_{1}\right\},\left\{\ba

%---- Page End Break Here ---- Page : 380
Since it is so easy to determine whether clause sets with exactly one literal per clause are sati

Problem: 3-Satisfiability (3-SAT)
Input: A collection of clauses C where each clause contains exactly 3 literals, over a set of B
Output: Is there a truth assignment to V such that each clause is satisfied?
Since 3-SAT is a restricted case of satisfiability, the hardness of 3-SAT would imply that genera

This reduction transforms each clause independently based on its length, by adding new clauses an
- $k=1$, say $C_{i}=\left\{z_{1}\right\}$ - We create two new variables v_{1}, v_{2} and four n
- $k=2$, say $C_{i}=\left\{z_{1}, z_{2}\right\}$ - We create one new variable v_{1} and two new
- $k=3$, say $C_{i}=\left\{z_{1}, z_{2}, z_{3}\right\}$ - We copy C_{i} into the 3-SAT instance
- $k>3$, say $C_{i}=\left\{z_{1}, z_{2}, \ldots, z_{k}\right\}$ - Here we create $k-3$ new variab
$$
C_{i}=\left\{z_{1}, z_{2}, z_{3}, z_{4}, z_{5}, z_{6}\right\}
$$
gets transformed into the following set of four 3-literal clauses with three new Boolean variable
$$
\left\{\left\{z_{1}, z_{2}, \bar{v}_{i, 1}\right\},\left\{v_{i, 1}, z_{3}, \bar{v}_{i, 2}\right\}
$$

The most complicated case is that of the large clauses. If none of the original literals in $C_{i
to satisfy all the new subclauses. You can satisfy $C_{i, 1}$ by setting $v_{i, 1}=$ false, but t

%---- Page End Break Here ---- Page : 381
This transform takes $O(n+c)$ time if there were c clauses and n total literals in the SAT in

Note that a slight modification to this construction would serve to prove that 4-SAT, 5 -SAT, or

\section{Creative Reductions from SAT}

Since both satisfiability and 3-SAT are known to be hard, we can use either of them in future red

One perpetual point of confusion is getting the direction of the reduction right. Recall that we

\subsection{Vertex Cover}

Algorithmic graph theory proves to be a fertile ground for hard problems. The prototypical NP-com

Problem: Vertex Cover
Input: A graph $G=(V, E)$ and integer $k \leq|V|$.
Output: Is there a subset S of at most k vertices such that every $e \in E$ has at least one

Demonstrating the hardness of vertex cover proves more difficult than the previous redu
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-383.jpg?height=28

%---- Page End Break Here ---- Page : 382
Figure 11.7: Reducing 3-SAT instance $\left\{\left\{v_{1}, \overline{v_{3}}, \overline-
construct a graph G and bound k from the variables and clauses of the satisfiabili

Here is a way to do it. First, we translate the variables of the 3-SAT problem. For ea

Second, we translate the clauses of the 3-SAT problem. For each of the c clauses, we

Finally, we will connect these two sets of components together. Each literal in a vert

This graph has been very carefully designed to have a vertex cover of size $n+2 c$ iff
- Every satisfying truth assignment gives a vertex cover of size $n+2 c-$ Given a satis
- Every vertex cover of size $n+2 c$ gives a satisfying truth assignment - In any vert
for otherwise a clause-gadget edge would go uncovered. These clausegadget vertices can

%---- Page End Break Here ---- Page : 383
This proof of the hardness of vertex cover, chained with the clique and independent se

Take-Home Lesson: A small set of NP-complete problems (3-SAT, vertex cover, integer pa

\subsection{Integer Programming}

As discussed in Section 16.6 (page 482), integer programming is a fundamental combina

Problem: Integer Programming (IP)
Input: A set of integer variables V, a set of linear inequalities over V, a linear
Output: Does there exist an assignment of integers to V such that all inequalities a

Consider the following two examples. Suppose
$$
\begin{gathered}
V_{1} \geq 1, \quad V_{2} \geq 0 \\
V_{1}+V_{2} \leq 3 \\
f(V): 2 V_{2}, \quad B=3
\end{gathered}
$$

A solution to this would be $V_{1}=1, V_{2}=2$. Note that this respects integrality, a
$$
\begin{gathered}
V_{1} \geq 1, \quad V_{2} \geq 0 \\

V_{1}+V_{2} \leq 3 \\
f(V): 2 V_{2}, \quad B=5
\end{gathered}
$$
the maximum possible value of $f(V)$ given the constraints is $2 \times 2=4$, so there can be no

We show that integer programming is hard using a reduction from general satisfiability. 3-SAT ger

%---- Page End Break Here ---- Page : 384
In which direction must the reduction go? We want to prove integer programming is hard, and know

What should the translation look like? Every satisfiability instance contains Boolean (true/false

Our translated integer programming problem will have twice as many variables as the SAT instance
- We restrict each integer programming variable V_{i} to values of either 0 or 1 , by adding co
- We ensure that exactly one of the two integer programming variables associated with a given SAT

For each clause $C_{i}=\left\{z_{1}, \ldots, z_{k}\right\}$, construct the constraint
$$
Z_{1}+\ldots+Z_{k} \geq 1
$$

To satisfy this constraint, at least one literal per clause must be set to 1 , thus corresponding

The maximization function and bound prove relatively unimportant here, because we have already en
- Every SAT solution gives a solution to the IP problem - In any SAT solution, a true literal cor
- Every IP solution gives a solution to the original SAT problem - All variables must be set to e

This reduction works both ways, so integer programming must be hard. Notice the following propert
- This reduction preserved the structure of the problem. It did not solve the problem, just put i
- The possible IP instances resulting from this transformation represent only a small subset of a
- The transformation captures the essence of why IP is hard. It has nothing to do with big coeffi

%---- Page End Break Here ---- Page : 385
\section{The Art of Proving Hardness}

Proving that problems are hard is a skill. But once you get the hang of it, reductions can be sur

It takes experience to judge which problems are likely to be hard. The quickest way to gain this

The first place to look when you suspect a problem might be NP-complete is Garey and Johnson's bo

Otherwise I offer the following advice to those seeking to prove the hardness of a given problem:
- Make your source problem as simple (meaning restricted) as possible Never try to use the genera

As another example, never try to use full satisfiability to prove hardness. Start with
satisfiability. Instead, you can use planar 3-satisfiability, where there must exist a
- Make your target problem as hard as possible - Don't be afraid to add extra constrai
- Select the right source problem for the right reason - Selecting the right source pr
I use four (and only four) problems as candidates for my hard source problem. Limiting
- 3-SAT: The old reliable. When none of the three problems below seem appropriate, I g
- Integer partition: This is the one and only choice for problems whose hardness seems
- Vertex cover: This is the answer for any graph problem whose hardness depends upon s
- Hamiltonian path: This is my choice for any graph problem whose hardness depends upo
- Amplify the penalties for making the undesired selection - Many people are too timid
sharper the consequences for doing what is undesired, the easier it is to prove the eq
- Think strategically at a high level, then build gadgets to enforce tactics You shoul
1. How can I force that A or B is chosen but not both?
2. How can I force that A is taken before B ?
3. How can I clean up the things I did not select?

%---- Page End Break Here ---- Page : 386

%---- Page End Break Here ---- Page : 387
Once you know what you want your gadgets to do, you can then worry about how to actual
- When you get stuck, switch between looking for an algorithm and a reduction - Someti

\section{War Story: Hard Against the Clock}

My class's attention span was running down like sand through an hourglass. Eyes were s

There were twenty minutes left to go in my lecture on NP-completeness, and I couldn't

I reached for my trusty copy of Garey and Johnson's book [\\cite{garey1979computers},
"Enough of this!" I announced loudly enough to startle those in the back row. "NP-comp

A few students in the front held up their hands. A few students in the back held up th
"Select a problem at random from the back of this book. I can prove the hardness of an

I had definitely gotten their attention. But I could have done that by offering to jug

%---- Page End Break Here ---- Page : 388
The student picked out a problem. "OK, prove that Inequivalence of Programs with Assig
"Huh? I've never heard of that problem before. What is it? Read me the entire problem

Problem: Inequivalence of Programs with Assignments

Input: A finite set X of variables, two programs P_{1} and P_{2}, each a sequenc

$$
x_{0} \leftarrow \text { if }\left(x_{1}=x_{2}\right) \text { then } x_{3} \text { else } x_{4}
$$
where the x_{i} are in X; and a value set V.
Output: Is there an initial assignment of a value from V to each variable in X such that prog

I looked at my watch. Fifteen minutes to go. But everything was now on the table. I was faced wit
"First things first. We need to select a source problem for our reduction. Do we start with integ

Since I had an audience, I tried thinking out loud. "Our target is not a graph problem or a numer

My watch said fourteen minutes left. "So, class, which way does the reduction go? 3-SAT to progra

The front row correctly murmured, " 3 -SAT to program."
"Right. So we have to translate our set of clauses into two programs. How can we do that? We migh

Instead, let's try something else. We can translate all the clauses into one program, and then le

I was still talking out loud to myself, which wasn't that unusual. But I had people listening to
"Now, how can we turn a set of clauses into a program? We want to know whether the set of clauses

It took me a few minutes of scratching before I found the right program to simulate a clause. I a
$$
\begin{aligned}
& C_{1}=\text { if }\left(x_{1}=\text { true }\right) \text { then true else false } \\
%---- Page End Break Here ---- Page : 389

& C_{1}=\text { if }\left(x_{2}=\text { false }\right) \text { then true else } C_{1} \\
& C_{1}=\text { if }\left(x_{3}=\text { true }\right) \text { then true else } C_{1}
\end{aligned}
$$
"Great. Now I have a way to evaluate the truth of each clause. I can do the same thing at the end

sat = if (C}\mp@subsup{C}{1}{}=\mathrm{ true) then true else false
sat = if (C2= true) then sat else false
\vdots
sat = if (C}\mp@subsup{C}{c}{}=\mathrm{ true) then sat else false

Now the back of the classroom was getting excited. They were starting to see a ray of hope that o
"Great. So now we have a program that can evaluate to be true if and only if there is a way to as

I lifted my arms in triumph. "And so, the problem is neat, sweet, and NP-complete." I got the las

\section{War Story: And Then I Failed}

This exercise of picking a random NP-complete problem from Garey and Johnson's book and proving h

The class had voted to see a reduction from the graph theory section of the catalog, ar

Problem: Uniconnected Subgraph
Input: A directed graph $G=(V, A)$, positive integer $k \leq|A|$.
Output: Is there a subset of arcs $A^{\prime} \subseteq A$ with $\left|A^{\prime}\right

It took a while for me to grok this problem. An undirected version of this would be fin
any pair of vertices. Adding even a single edge (x, y) to this tree would create a cy

%---- Page End Break Here ---- Page : 390
Any form of directed tree would also be uniconnected. But this problem asks for the lar
"It is a selection problem," I realized after grokking. After all, we had to select the

I worked through how the two problems stacked up. Both sought subsets, although vertex

I had to do something to direct the edges of the vertex cover graph. I could try to rep

I realized I could direct the edges so the resulting graph was a DAG. But then, so what

Alternately, I could try to replace each undirected edge (x, y) with two arcs, from $

Meanwhile, the clock was running and I knew it. A sense of panic set in during the last

There is no feeling worse for a professor than botching up a lecture. You stand up ther

I promised them a solution for the next class, but somehow I kept getting stuck in the

I dreaded returning to give my next class, the last lecture of the semester. But the ni
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-392.jpg?height=49

%---- Page End Break Here ---- Page : 391
Figure 11.8: Reducing vertex cover to unidirected subgraph, by dividing edges and addin

I sat up in bed and scratched out the proof. Suppose I replace each undirected edge $(x

But now add a sink node s with edges from all the original vertices. There are exactl

Presenting this proof in class provided some personal vindication, because it validates

\section{P vs. NP}

The theory of NP-completeness rests on a foundation of rigorous but subtle definitions
foundations. These details are not really essential to the practical aspects of design

```
%---- Page End Break Here ---- Page : 392
\subsection{Verification vs. Discovery}

The primary issue in P vs. NP is whether verification is really an easier task than initial disco

For the NP-complete decision problems we have studied here, the answer seems obvious:
- Can you verify that a proposed TSP tour has weight of at most $k$ in a given graph? Sure. Just
- Can you verify that a given truth assignment represents a solution to a given satisfiability pr
- Can you verify that a given $k$-vertex subset $S$ is a vertex cover of graph $G$ ? Sure. Just t

At first glance, this seems obvious. The given solutions can be verified in linear time for all t

\subsection{The Classes P and NP}

Every well-defined algorithmic problem must have an asymptotically fastestpossible algorithm solv

We can think of the class $P$ as an exclusive club for algorithmic problems where there exists a


%---- Page End Break Here ---- Page : 393
A less exclusive club welcomes all the algorithmic problems whose solutions can be verified in po

We call this less-exclusive club NP. You can think of this as standing for Not necessarily Polyno

The $\$ 1,000,000$ question is whether there are problems in NP that are not members of P. If no

\subsection{Why Satisfiability is Hard}

An enormous tree of NP-completeness reductions has been established that entirely rests on the ha

This may seem like a fragile foundation. What would it mean if someone did find a polynomial-time

Fear not. There exists an extraordinary super-reduction called Cook's theorem reducing all the pr

Cook's theorem proves that satisfiability is as hard as any problem in NP. Furthermore, it proves

\footnotetext{
${ }^{1}$ In fact, it stands for non-deterministic polynomial time. This is in the sense of nonde
}


%---- Page End Break Here ---- Page : 394
\subsection{NP-hard vs. NP-complete?}
```

The final technicality we will discuss is the difference between a problem being NP-ha

We say that a problem is NP-hard if, like satisfiability, it is at least as hard as an

That said, there are some problems that appear to be NP-hard yet are not in NP. These

Chapter Notes

The notion of NP-completeness was first developed by Cook [\\cite{cook1971complexity}.

Karp \\cite{karp1972reducibility}] showed the importance of Cook's result by providing

The best introduction to the theory of NP-completeness remains Garey and Johnson's boo

Factor Man \\cite{ginsberg2018factor} is an exciting novel about a man who discovers a
time algorithm for satisfiability, and must dodge government agents and assassins for

%---- Page End Break Here ---- Page : 395
A few catalog problems exist in a limbo state where it is not yet known whether the pr

For an alternative and inspiring view of NP-completeness, check out the videos of Erik

\section{Exercises}

Transformations and Satisfiability

11-1. [2] Give the 3-SAT formula that results from applying the reduction of satisfiab
$(x \vee y \vee \bar{z} \vee w \vee u \vee \bar{v}) \wedge(\bar{x} \vee \bar{y} \vee z
11-2. [3] Draw the graph that results from the reduction of 3-SAT to vertex cover for
$$
(x \vee \bar{y} \vee z) \wedge(\bar{x} \vee y \vee \bar{z}) \wedge(\bar{x} \vee y \vee
$$

11-3. [3] Prove that 4-SAT is NP-hard.
11-4. [3] Stingy SAT is the following problem: given a set of clauses (each a disjunct

11-5. [3] The Double SAT problem asks whether a given satisfiability problem has at le

11-6. [4] Suppose we are given a subroutine that can solve the traveling salesman deci

11-7. [7] Implement a SAT to 3-SAT reduction that translates satisfiability instances

%---- Page End Break Here ---- Page : 396
11-8. [7] Design and implement a backtracking algorithm to test whether a set of claus

11-9. [8] Implement the vertex cover to satisfiability reduction, and run the resulting clauses t

Basic Reductions

11-10. [4] An instance of the set cover problem consists of a set X of n elements, a family $
For example, if $X=\{1,2,3,4\}$ and $F=\{\{1,2\},\{2,3\},\{4\},\{2,4\}\}$, there does not exist a
Prove that set cover is NP-hard with a reduction from vertex cover.
11-11. [4] The baseball card collector problem is as follows. Given packets $P_{1}, \ldots, P_{m}
For example, if the players are $\{$ Aaron, Mays, Ruth,Skiena $\}$ and the packets are
$$
\{\{\text { Aaron, Mays }\},\{\text { Mays, Ruth }\},\{\text { Skiena }\},\{\text { Mays, Skiena
$$
there does not exist a solution for $k=2$, but there does for $k=3$, such as
$$
\{\text { Aaron, Mays }\},\{\text { Mays, Ruth }\},\{\text { Skiena }\}
$$

Prove that the baseball card collector problem is NP-hard using a reduction from vertex cover.

11-12. [4] The low-degree spanning tree problem is as follows. Given a graph G and an integer $
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-397.jpg?height=186&width=243
(a) Prove that the low-degree spanning tree problem is NP-hard with a reduction from Hamiltonian
(b) Now consider the high-degree spanning tree problem, which is as follows. Given a graph G an

%---- Page End Break Here ---- Page : 397
11-13. [5] In the minimum element set cover problem, we seek a set cover $S \subseteq C$ of a uni
11-14. [3] The half-Hamiltonian cycle problem is, given a graph G with n vertices, determine
11-15. [5] The 3-phase power balance problem asks for a way to partition a set of n positive in
11-16. [4] Show that the following problem is NP-hard:
Problem: Dense Subgraph
Input: A graph G, and integers k and y.
Output: Does G contain a subgraph of exactly k vertices and at least y edges?
11-17. [4] Show that the following problem is NP-hard:
Problem: Clique, No-clique
Input: An undirected graph $G=(V, E)$ and an integer k.
Output: Does G contain both a clique of size k and an independent set of size k ?

11-18. [5] An Eulerian cycle is a tour that visits every edge in a graph exactly once. An Euleria
11-19. [5] Show that the following problem is NP-hard:
Problem: Maximum Common Subgraph
Input: Two graphs $G_{1}=\left(V_{1}, E_{1}\right)$ and $G_{2}=\left(V_{2}, E_{2}\right)$, and a
Output: Two sets of nodes $S_{1} \subseteq V_{1}$ and $S_{2} \subseteq V_{2}$ whose deletion leav

11-20. [5] A strongly independent set is a subset of vertices S in a graph G such that for an

11-21. [5] A kite is a graph on an even number of vertices, say $2 n$, in which n of

%---- Page End Break Here ---- Page : 398
Creative Reductions

11-22. [5] Prove that the following problem is NP-hard:
Problem: Hitting Set
Input: A collection C of subsets of a set S, positive integer k.
Output: Does S contain a subset $S^{\prime}$ such that $\left|S^{\prime}\right| \leq

11-23. [5] Prove that the following problem is NP-hard:
Problem: Knapsack
Input: A set S of n items, such that the i th item has value v_{i} and weight $
Output: Does there exist a subset $S^{\prime} \subseteq S$ such that $\sum_{i \in S^{\

11-24. [5] Prove that the following problem is NP-hard:
Problem: Hamiltonian Path
Input: A graph G, and vertices s and t.
Output: Does G contain a path that starts from s, ends at t, and visits all vert

11-25. [5] Prove that the following problem is NP-hard:\index{heuristics}
Problem: Longest Path
Input: A graph G and positive integer k.
Output: Does G contain a path that visits at least k different vertices without vi

11-26. [6] Prove that the following problem is NP-hard:
Problem: Dominating Set
Input: A graph $G=(V, E)$ and positive integer k.
Output: Is there a subset $V^{\prime} \subseteq V$ such that $\left|V^{\prime}\right| \

11-27. [7] Prove that the vertex cover problem (does there exist a subset S of k ve
11-28. [7] Prove that the following problem is NP-hard:
Problem: Set Packing
Input: A collection C of subsets of a set S, positive integer k.
Output: Does C contain at least k disjoint subsets (i.e., such that no pair of subs

%---- Page End Break Here ---- Page : 399
11-29. [7] Prove that the following problem is NP-hard:
Problem: Feedback Vertex Set
Input: A directed graph $G=(V, A)$ and positive integer k.
Output: Is there a subset $V^{\prime} \subseteq V$ such that $\left|V^{\prime}\right| \

11-30. [8] Give a reduction from Sudoku to the vertex coloring problem in graphs. Spec

Algorithms for Special Cases

11-31. [5] A Hamiltonian path P is a path that visits each vertex exactly once. The problem of

Give an $O(n+m)$-time algorithm to test whether a directed acyclic graph G (a DAG) contains a H

11-32. [3] Consider the k-clique problem, which is the general clique problem restricted to gra

11-33. [8] The 2-SAT problem is, given a Boolean formula in 2-conjunctive normal form (CNF), to d

$$
\left(x_{1} \vee x_{2}\right) \wedge\left(\bar{x}_{2} \vee x_{3}\right) \wedge\left(x_{1} \vee
$$

Give a polynomial-time algorithm to solve 2-SAT.

$P=NP$?

11-34. [4] Show that the following problems are in NP:
- Does graph G have a simple path (i.e., with no vertex repeated) of length k ?
- Is integer n composite (i.e., not prime)?
- Does graph G have a vertex cover of size k ?

11-35. [7] Until 2002, it was an open question whether the decision problem "Is integer n a com

$$
\begin{aligned}
& \text { PrimalityTesting }(n) \\
& \text { composite }=\text { false } \\
& \text { for } \mathrm{i}:=2 \text { to } n-1 \text { do } \\
& \text { if }(n \bmod i)=0 \text { then } \\
& \quad \text { composite }=\text { true }
\end{aligned}
$$

LeetCode

11-1. https://leetcode.com/problems/target-sum/
11-2. https://leetcode.com/problems/word-break-ii/
11-3. https://leetcode.com/problems/number-of-squareful-arrays/

HackerRank

11-1. https://www.hackerrank.com/challenges/spies-revised
11-2. https://www.hackerrank.com/challenges/brick-tiling/
11-3. https://www.hackerrank.com/challenges/tbsp/

Programming Challenges

These programming challenge problems with robot judging are available at https://onlinejudge.org.

```
11-1. "The Monocycle" - Chapter 12, problem 10047.
11-2. "Dog and Gopher"-Chapter 13, problem 111301.
11-3. "Chocolate Chip Cookies"-Chapter 13, problem 10136.
11-4. "Birthday Cake"-Chapter 13, problem 10167.
These are not particularly relevant to NP-completeness, but are added for completeness


%---- Page End Break Here ---- Page : 401

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{images/algorithms_page_0403_0.png}
\end{figure}
Chapter 12

\chapter{Dealing with Hard Problems}

For the practical person, demonstrating that a problem is NP-complete is never the end
- Algorithms fast in the average case - Examples of such algorithms include backtrackin
- Heuristics - Heuristic methods like simulated annealing or greedy approaches can be u
- Approximation algorithms - The theory of NP-completeness stipulates that it is hard t

This chapter will investigate these possibilities deeper. I include a brief introductio

\section{Approximation Algorithms}

Approximation algorithms produce solutions with a guarantee attached, namely that the o
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-403.jpg?height=274

Figure 12.1: Failing to pick the center vertex leads to a terrible vertex cover.
correct answer. Furthermore, provably good approximation algorithms are often conceptua

One thing that is usually not clear, however, is how well the solution from an approxim

One way to get the best of approximation algorithms and unwashed heuristics is to run b

\section{Approximating Vertex Cover}

Recall the vertex cover problem, where we seek a small subset $S$ of the vertices of a
$$
\begin{aligned}
& \text { VertexCover }(G=(V, E)) \\
& \text { While }(E \neq \emptyset) \text { do: } \\
& \text { Select an arbitrary edge }(u, v) \in E \\
& \text { Add both } u \text { and } v \text { to the vertex cover } \\
& \text { Delete all edges from } E \text { that are incident to either } u \text { or
```

\end{aligned}
$$

It should be apparent that this procedure always produces a vertex cover, since each edge is dele
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-404.jpg?height=320&width=885

%---- Page End Break Here ---- Page : 403
Figure 12.2: A bad example for the greedy heuristic for vertex cover. The optimal cover of this b
include at least one vertex per edge, which makes it at least half the size of this $2 k$-vertex

There are several interesting things to notice about this algorithm:
- Although the procedure is simple, it is not stupid - Many seemingly smarter heuristics can give
- Greedy isn't always the answer - Perhaps the most natural heuristic for vertex cover would repe
- Making a heuristic more complicated does not necessarily make it better It is easy to complicat
- A post-processing cleanup step can't hurt - The flip side of designing simple heuristics is tha
solutions without weakening the approximation bound. For example, a post-processing step that del

%---- Page End Break Here ---- Page : 404
The important property of approximation algorithms is relating the size of the solution produced

Stop and Think: Leaving Behind a Vertex Cover

Problem: Suppose we do a depth-first search of graph G, naturally building a depth-first search

Solution: Why must the set of all non-leaf nodes in the DFS tree T form a vertex cover? Recall

But why is the set of non-leaves at most twice the size of the optimal cover? Start from any leaf

\subsection{A Randomized Vertex Cover Heuristic}

Although we proved that our original vertex cover heuristic of selecting arbitrary uncovered edge

Such a horrible performance requires making the wrong decision $n-1$ times in a row, which implie
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-406.jpg?height=199&width=520

%---- Page End Break Here ---- Page : 405
Figure 12.3: The triangle inequality, that $d(u, w) \leq d(u, v)+d(v, w)$, holds for distances de
$$
\begin{aligned}
& \text { VertexCover }(G=(V, E)) \\
& \text { While }(E \neq \emptyset) \text { do: } \\
& \text { Select an arbitrary edge }(u, v) \in E \\
& \text { Randomly pick either } u \text { or } v \text {, and add it to the vertex cover } \\

& \text { Delete all edges from } E \text { that are incident to the selected vertex.
\end{aligned}
$$

At the end of this procedure, we will end up with a vertex cover, but how well does it

Randomization is a very powerful tool for developing approximation algorithms. Its rol

\section{Euclidean TSP}

In most natural applications of the traveling salesman problem (TSP), direct routes ar

The edge weights induced by Euclidean geometry satisfy the triangle inequality, namely

The traveling salesman problem remains hard when the edge weights are defined by Eucli
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-407.jpg?height=47

%---- Page End Break Here ---- Page : 406
Figure 12.4: A depth-first traversal of a spanning tree, with the shortcut tour (left)
optimal traveling salesman tour on such graphs that obey the triangle inequality using

Consider now what happens when performing a depth-first traversal of a spanning tree.
$$
1,2,1,3,5,8,5,9,5,3,6,3,1,4,7,10,7,11,7,4,1
$$

This circuit travels along each edge of the minimum spanning tree twice, and hence cos

However, many vertices will be repeated on this depth-first search circuit. To remove
$$
1,2,3,5,8,9,6,4,7,10,11,1
$$

Because we have replaced a chain of edges by a single direct edge, the triangle inequa

\subsection{The Christofides Heuristic}

There is another way of looking at this minimum spanning tree doubling idea, which wil
an Eulerian cycle in graph G is a circuit traversing each edge of G exactly once $

%---- Page End Break Here ---- Page : 407
We can reinterpret the minimum spanning tree heuristic for TSP in terms of Eulerian cy

This suggests that we might find an even better approximation for TSP if we could find

```
So let's start by identifying the odd-degree vertices in the minimum spanning tree of $G$, which

The Christofides heuristic constructs a multigraph $M$ consisting of the minimum spanning tree of

Note that the cost of this matching of just the odd-degree vertices must be a lower bound on the

Observe in Figure 12.5 that the alternating edges of any TSP tour must define a matching, because

In conclusion, the total weight of $M$ must be at most $(1+(1 / 2))=(3 / 2)$ times that of the op

\footnotetext{
${ }^{1}$ Or, if you don't recall this, tour Section 18.7 (page 565 for a refresher.
}
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-409.jpg?height=310&width=308


%---- Page End Break Here ---- Page : 408
Figure 12.5: Any TSP tour in a graph with an even number of vertices that observes the triangle i
structs a tour of weight at most $3 / 2$ times that of the optimal tour. As with the minimum span

\section{When Average is Good Enough}

In the mythical land of Lake Wobegon, all the children are above average. For certain optimizatio

Maximum $k$-SAT

Recall the problem of 3-SAT discussed in Section 11.4.1 (page 367), where we are given a set of t
$$
v_{3} \text { or } \bar{v}_{17} \text { or } v_{24}
$$
and asked to find an assignment of either true or false to each variable $v_{i}$ so as to make al

A more general problem is maximum 3-SAT, where we seek the Boolean variable assignment that makes

What happens when we flip a coin to decide the value of each variable $v_{i}$, and thus construct


%---- Page End Break Here ---- Page : 409
That seems pretty good for a mindless approach to an NP-complete problem. For a maximum $k$-SAT i

\subsection{Maximum Acyclic Subgraph}

Directed acyclic graphs (DAGs) are easier to work with than general directed graphs. Sometimes it

Here we consider an interesting problem in this class, where we seek to retain as many edges as p
```

Problem: Maximum Directed Acyclic Subgraph
Input: A directed graph $G=(V, E)$.
Output: Find the largest possible subset $E^{\prime} \subseteq E$ such that $G^{\prime

In fact, there is a very simple algorithm that guarantees you a solution with at least

Problem: Construct any permutation of the vertices, and interpret it as a left-to-righ

One of these two edge subsets must be at least as large as the other. This means it co

This approximation algorithm is simple almost to the point of being stupid. But note t

\section{Set Cover}

The previous sections may encourage a false belief that every problem can be approxima

Set cover occupies a middle ground between these extremes, having a factor$\Theta(\lg
\begin{tabular}{r|c|c|ccccccccc|c}
milestone class & 6 & \multicolumn{8}{|c|}{5} & \multicolumn{5}{c|}{4} & & 3 & 2 & 1 &
%---- Page End Break Here ---- Page : 410

\hline uncovered elements & 64 & 51 & 40 & 30 & 25 & 22 & 19 & 16 & 13 & 10 & 7 & 4 \\
2 & 1 \\
selected subset size & 13 & 11 & 10 & 5 & 3 & 3 & 3 & 3 & 3 & 3 & 3 & 2
\end{tabular} 1

Figure 12.6: The coverage process for the greedy heuristic on a particular instance of

Problem: Set Cover
Input: A collection of subsets $S=\left\{S_{1}, \ldots, S_{m}\right\}$ of the universa
Output: What is the smallest subset T of S whose union equals the universal set-i.

The natural heuristic is greedy. Repeatedly select the subset that covers the largest
$$
\begin{aligned}
& \operatorname{SetCover}(S) \\
& \text { While }(U \neq \emptyset) \text { do: } \\
& \quad \text { Identify the subset } S_{i} \text { with the largest intersection with
& \text { Select } S_{i} \text { for the set cover } \\
& U=U-S_{i}
\end{aligned}
$$

One consequence of this selection process is that the number of freshly covered elemen

Thus we can view this heuristic as reducing the number of uncovered elements from n down to zer

Let w_{i} denote the number of subsets that were selected by the heuristic to cover elements be

Since there are at most $\lg n$ such milestones, the solution produced by the greedy heuristic mu

Why? Consider the average number of new elements covered as we move between milestones $2^{i+1}-1
subsets can cover at most μ_{i} subsets. Thus, no subset exists in S that can cover more th

%---- Page End Break Here ---- Page : 411
The surprising thing here is that there are set cover instances where the greedy heuristic finds

Take-Home Lesson: Approximation algorithms guarantee answers that are always close to the optimal

\section{Heuristic Search Methods}

Backtracking gave us a method to find the best of all possible solutions, as scored by a given ob

In this section, I will discuss approaches to heuristic search. The bulk of our attention will be

In particular, we will look at three different heuristic search methods: random sampling, gradien
- Solution candidate representation - This is a complete yet concise description of possible solu
- Cost function - Search methods need a cost or evaluation function to assess the quality of each

%---- Page End Break Here ---- Page : 412
\subsection{Random Sampling}

The simplest approach to search in a solution space uses random sampling, also known as the Monte

True random sampling requires that we select elements from the solution space uniformly at random

```
void random_sampling(tsp_instance *t, int nsamples, tsp_solution s) {
tsp_solution s_now; / current tsp solution /
double best_cost; / best cost so far /
double cost_now; / current cost /
int i; / counter */
initialize_solution(t->n, &s_now);
best_cost = solution_cost(&s_now, t);
copy_solution(&s_now, s);
for (i = 1; i <= nsamples; i++) {
random_solution(&s_now);
cost_now = solution_cost(&s_now, t);
if (cost_now < best_cost) {
best_cost = cost_now;
```

```
copy_solution(&s_now, s);
}
solution_count_update(&s_now, t);
}
}
```

When might random sampling do well?
- When there is a large proportion of acceptable solutions - Finding a piece of hay in
Finding prime numbers is a domain where a random search proves successful. Generating
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-414.jpg?height=53

%---- Page End Break Here ---- Page : 413
Figure 12.7: Search time/quality tradeoffs for TSP using random sampling. Progress is
- When there is no coherence in the solution space - Random sampling is the right thin

Consider again the problem of hunting for a large prime number. Primes are scattered q

How does random sampling do on TSP? Pretty lousy. The best solution I found after test

Most problems we encounter, like TSP, have relatively few good solutions and a highly

Stop and Think: Picking the Pair

Problem: We need an efficient and unbiased way to generate random pairs of vertices to
to generate elements from the $\binom{n}{2}$ unordered pairs on $\{1, \ldots, n\}$ uni

%---- Page End Break Here ---- Page : 414
Solution: Uniformly generating random structures is a surprisingly subtle problem. Con

```
    i = random_int(1,n-1);
j = random_int(i+1,n);
```

It is clear that this indeed generates unordered pairs, since $i<j$. Further, it is cl

But are they uniform? The answer is no. What is the probability that pair $(1,2)$ is g

The problem is that fewer pairs start with big numbers than little numbers. We could s

But instead of working through the math, let's exploit the fact that randomly generati

```
    do {
i = random_int(1,n);
j = random_int(1,n);
if (i > j) swap(&i,&j);
} while (i==j);
```

\subsection{Local Search}

Now suppose you want to hire an algorithms expert as a consultant to solve your problem. You coul
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-416.jpg?height=471&width=656

%---- Page End Break Here ---- Page : 415
Figure 12.8: Improving a candidate TSP tour by swapping vertices 3 and 7 replaces four old tour e
on the phone for someone more likely to be an algorithms expert, and call them up instead.

A local search scans the neighborhood around elements in the solution space. Think of each such c

We certainly do not want to explicitly construct this neighborhood graph for any sizable solution

Instead, we want a general transition mechanism that takes us to a nearby solution by slightly mo

A reasonable transition mechanism for TSP would be to swap the current tour positions of a random

Local search heuristics start from an arbitrary element of the solution space, and then scan the
that leads in the direction we want to travel. We repeat until we have reached a point where all

%---- Page End Break Here ---- Page : 416
But unfortunately, we are probably not King of the Mountain. Suppose you wake up in a ski lodge,

```
void hill_climbing(tsp_instance *t, tsp_solution s) {
double cost; / best cost so far /
double delta; / swap cost /
int i, j; / counters /
bool stuck; / did I get a better solution? */
initialize_solution(t->n, s);
random_solution(s);
cost = solution_cost(s, t);
do {
stuck = true;
for (i = 1; i < t->n; i++) {
for (j = i + 1; j <= t->n; j++) {
delta = transition(s, t, i, j);
if (delta < 0) {
stuck = false;
cost = cost + delta;
} else {
transition(s, t, j, i);
}
solution_count_update(s, t);
}
```

}
} while (stuck);
}

When does local search do well?
- When there is great coherence in the solution space - Hill climbing is at its best w
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-418.jpg?height=44

%---- Page End Break Here ---- Page : 417
Figure 12.9: Search time/quality tradeoffs for TSP using hill climbing.

Many natural problems have this property. We can think of a binary search as starting
- Whenever the cost of incremental evaluation is much cheaper than global evaluation -
If we are given a very large value of n and a very small budget of how much time we

The primary drawback of a local search is that there isn't anything more for us to do

How does local search do on TSP? Much better than random sampling for a similar amount

This is good, but not great. You would not be happy to learn you are paying twice the

\footnotetext{
${ }^{2}$ This is still more than double the optimal solution cost of 6,828 , so the m
}
fairly similar quality. We need more powerful methods to get closer to the optimal sol

%---- Page End Break Here ---- Page : 418
\subsection{Simulated Annealing}

Simulated annealing is a heuristic search procedure that allows occasional transitions

The inspiration for simulated annealing comes from the physical process of cooling mol
$$
P\left(e_{i}, e_{j}, T\right)=e^{\left(e_{i}-e_{j}\right) /\left(k_{B} T\right)}
$$

Here k_{B} is a positive constant-called Boltzmann's constant, used to tune the desi

What does this formula mean? Transitioning from a low energy state to higher energy st
$$
\begin{aligned}
& \text { Simulated-Annealing }() \\
& \text { Create initial solution } s \\
& \text { Initialize temperature } T \\

```
& \text { repeat } \\
& \quad \text { for } i=1 \text { to iteration-length do } \\
& \quad \text { Randomly select a neighbor of } s \text { to be } s_{i} \\
& \text { If }\left(C(s) \geq C\left(s_{i}\right)\right) \text { then } s=s_{i} \\
& \text { else if }\left(e^{\left(C(s)-C\left(s_{i}\right)\right) /\left(k_{B} \cdot T\right)}>\c
& \text { Reduce temperature } T \\
& \text { until (no change in } C(s)) \\
& \text { Return } s
\end{aligned}
$$

What relevance does this have for combinatorial optimization? A physical system, as it cools, see


%---- Page End Break Here ---- Page : 419
Through random transitions generated according to the given probability distribution, we can mimi

Take-Home Lesson: Don't worry about this molten metal business. Simulated annealing is effective

As with a local search, the problem representation includes both a representation of the solution

At the beginning of the search, we are eager to use randomness to explore the search space widely
- Initial system temperature - Typically $T_{1}=1$.
- Temperature decrement function - Typically $T_{i}=\alpha \cdot T_{i-1}$, where $0.8 \leq \alpha
- Number of iterations between temperature change - Typically, 1,000 iterations or so might be pe
- Acceptance criteria - A typical criterion is to accept any good transition, and also accept a b
$$
e^{\frac{C\left(s_{i-1}\right)-C\left(s_{i}\right)}{k_{B} T}}>r
$$
where $r$ is a random number $0 \leq r<1$. The "Boltzmann" constant $k_{B}$ scales this cost func
- Stop criteria - Typically, when the value of the current solution has not changed or improved w

Creating the proper cooling schedule is a trial-and-error process of mucking with constants and s
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-421.jpg?height=525&width=697


%---- Page End Break Here ---- Page : 420
Figure 12.10: Search time/quality tradeoffs for TSP using simulated annealing.

Compare the time/quality profiles of our three heuristics. Simulated annealing does best of all.

After ten million iterations simulated annealing gave us a solution of cost 7,212 - only $10.4 \%

In expert hands, the best problem-specific heuristics for TSP will slightly outperform simulated

Implementation
```

The implementation follows the pseudocode quite closely:

```
void anneal(tsp_instance *t, tsp_solution s) {
int x, y; / pair of items to swap /
int i, j; / counters /
bool accept_win, accept_loss; / conditions to accept transition /
double temperature; / the current system temp /
double current_value; / value of current state /
double start_value; / value at start of loop /
double delta; / value after swap /
double exponent; / exponent for energy funct */
temperature = INITIAL_TEMPERATURE;
initialize_solution(t->n, s);
current_value = solution_cost(s, t);
for (i = 1; i <= COOLING_STEPS; i++) {
temperature = COOLING_FRACTION;
start_value = current_value;
for (j = 1; j <= STEPS_PER_TEMP; j++) {
/ pick indices of elements to swap /
x = random_int(1, t->n);
y = random_int(1, t->n);
delta = transition(s, t, x, y);
accept_win = (delta < 0); / did swap reduce cost? /
exponent = (-delta / current_value) / (K * temperature);
accept_loss = (exp(exponent) > random_float(0,1));
if (accept_win || accept_loss) {
current_value += delta;
} else {
transition(s, t, x, y); / reverse transition /
}
solution_count_update(s, t);
}
if (current_value < start_value) { / rerun at this temp */
temperature /= COOLING_FRACTION;

}
}
}
```

\subsection{Applications of Simulated Annealing}

We provide several examples to demonstrate how careful modeling of the state representa

Maximum Cut

The maximum cut problem seeks to partition the vertices of a weighted graph G into sets $V_{1}

How can we formulate maximum cut for simulated annealing? The solution space consists of all $2^

This kind of simple, natural modeling represents the right type of heuristic to seek in practice.

\section{Independent Set}

An independent set of a graph G is a subset of vertices S such that there is no edge with bo

The natural state space for a simulated annealing solution would consist of all 2^{n} possible

One natural objective function for subset S might be 0 if the S-induced subgraph contains an
edges, and the dependence of $C(S)$ on T ensures that the search will eventually drive the edg

%---- Page End Break Here ---- Page : 423
Circuit Board Placement

In designing printed circuit boards, we are faced with the problem of positioning modules (typic

Formally, we are given a collection of rectangular modules $r_{1}, \ldots, r_{n}$, each with ass

The state space for this problem must describe the positions of each rectangle on the board. To
$$
C(S)=\lambda_{\text {area }}\left(S_{\text {height }} \cdot S_{\text {width }}\right)+\sum_{i=1}
$$
where $\lambda_{\text {area }}, \lambda_{\text {wire }}$, and $\lambda_{\text {overlap }}$ are w

Take-Home Lesson: Simulated annealing is a simple but effective technique for efficiently obtain

\section{War Story: Only it is Not a Radio}
"Think of it as a radio," he chuckled. "Only it is not a radio."
I'd been whisked by corporate jet to the research center of a large but very secretive company l

The issue concerned a manufacturing technique known as selective assembly. Eli Whitney helped ki
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-425.jpg?height=396&width=95

%---- Page End Break Here ---- Page : 424
Figure 12.11: Part assignments for three not-radios, such that each had at most fifty defect poi
any legal cog-widget could be used to replace any other legal cog-widget. This greatly sped up t

Unfortunately, it also resulted in large piles of cog-widgets that were slightly outside the man
"Each not-radio is made up of n different types of not-radio parts," he told me. For the i t

The situation is illustrated in Figure 12.11. Each not-radio consists of three parts, a

I thought about the problem. The simplest procedure would take the best part for each p

The goal was to match up good parts and bad parts so the total amount of badness wasn't
"I can solve your problem using bipartite matching," I announced, "provided not-radios

%---- Page End Break Here ---- Page : 425
There was silence. Then they all started laughing at me. "Everyone knows not-radios hav

That spelled the end of this algorithmic approach. Extending to more than two parts tur

I went back to the drawing board. They wanted to put parts into assemblies so that no a

It wasn't pure bin packing, however, because parts came in different types, and the tas

Bin packing is NP-complete, but is a natural candidate for a heuristic search approach

The local neighborhood operation involves moving parts around from one bin to another.

The key decision was the cost function to use. They supplied the hard limit k on the

My final cost function was as follows. I gave one point for every working

\footnotetext{
${ }^{3}$ A hypergraph is made up of edges that can contain more than two vertices each
}
assembly, and a significantly smaller credit for each non-working assembly based on how

%---- Page End Break Here ---- Page : 426
I implemented this algorithm, and then ran the search on the test case they provided. I

I watched as simulated annealing chugged and bubbled on this problem instance. The numb

I called and tried to admit defeat, but they wouldn't hear of it. It turned out that th

\section{War Story: Annealing Arrays}

The war story of Section 3.9 (page 98 reported how we used advanced data structures to

A biochemist at Oxford University got interested in our technique, and moreover he had

However, we had to provide the proper programming. Fabricating complicated arrays requi
\begin{tabular}{c|cccc}

& \multicolumn{4}{|c}{ suffix } \\
%---- Page End Break Here ---- Page : 427

prefix & $A A$ & $A G$ & $G A$ & $G G$ \\
\hline \hline$A A$ & $A A A A$ & $A A A G$ & $A A G A$ & $A A G G$ \\
$A G$ & $A G A A$ & $A G A G$ & $A G G A$ & $A G G G$ \\
$G A$ & $G A A A$ & $G A A G$ & $G A G A$ & $G A G G$ \\
$G G$ & $G G A A$ & $G G A G$ & $G G G A$ & $G G G G$
\end{tabular}

Figure 12.12: A prefix-suffix array of all purine 4-mers.
"We are going to have to use a heuristic," I told him. "So how can we model this problem?"
"Well, each string can be partitioned into prefix and suffix pairs that realize it. For example,
"Good. This gives us a natural representation for simulated annealing. The state space will consi
"What's a good cost function?" he asked.
"Well, we need as small an array as possible that covers all the strings. How about taking the ma

Ricky went off and implemented a simulated annealing program along these lines. It printed out th
"The program doesn't seem to recognize when it is making progress. The evaluation function only g

Ricky changed the evaluation function, and we tried again. This time, the program did not hesitat
"OK. Let's add another term to the evaluation function to give it points for being roughly square

Ricky tried again. Now the arrays were the right shape, and progress was in the right direction.
"Too many of the insertion moves don't affect many strings. Maybe we should skew the random selec
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-429.jpg?height=345&width=135

%---- Page End Break Here ---- Page : 428
Figure 12.13: Compression of the HIV array by simulated annealing-after 0 , $500,1,000$, and 5,75
picked more often."
Ricky tried again. Now it converged faster, but sometimes it still got stuck. We changed the cool

Our final solution refined the initial array by applying the following random moves:
- Swap - swap a prefix/suffix on the array with one that isn't.
- $A d d$ - add a random prefix/suffix to the array.
- Delete - delete a random prefix/suffix from the array.
- Useful add - add the prefix/suffix with the highest usefulness to the array.
- Useful delete - delete the prefix/suffix with the lowest usefulness from the array.
- String add - randomly select a string not on the array, and add the most useful prefix and/or s

We used a standard cooling schedule, with an exponentially decreasing temperature (dependent upon
$$
\operatorname{cost}=2 \times \max +\min +\frac{(\max -\min)^{2}}{4}+4\left(s t r_{t o t a l}-s t
$$
where max is the size of the maximum chip dimension, $\min$ is the size of the minimum chip dimen

How well did we do? Figure 12.13 shows the convergence of an array consisting of the 5
of the state of the chip at four points during the annealing process, after 0,500 , 1,0

%---- Page End Break Here ---- Page : 429
But how well did we do? Since simulated annealing is only a heuristic, we really don't

\section{Genetic Algorithms and Other Heuristics}

Many heuristic search methods have been proposed for combinatorial optimization problem

The intuition behind these methods is highly appealing, but skeptics decry them as vood

The question isn't whether you can get decent answers for many problems given enough e

In general, I don't believe that they do. But in the spirit of free inquiry, I introdu

Genetic Algorithms

Genetic algorithms draw their inspiration from evolution and natural selection. Throug

Genetic algorithms maintain a "population" of solution candidates for the given proble

%---- Page End Break Here ---- Page : 430
The idea behind genetic algorithms is extremely appealing. However, they just don't se

I will not discuss genetic algorithms further, except to discourage you from consideri

Take-Home Lesson: I have never encountered any problem where genetic algorithms seemed

\section{Quantum Computing}

We live in an era where the random access machine (RAM) model of computation introduce

Quantum mechanics is well known for being so completely unintuitive that no one really

I presume that you the reader may well never have taken a physics course, and are like
exciting. I provide a taste of how three of the most famous quantum algorithms work. F

%---- Page End Break Here ---- Page : 431
\subsection{Properties of "Quantum" Computers}

Consider a conventional deterministic computer with n bits of memory labeled $b_{0}, We can think of a conventional deterministic computer as maintaining a probability distribution o

Quantum computers come with n qubits of memory, $q_{0}, \ldots, q_{n-1}$. Thus, there are also
"Quantum" computers support the following operations:
- Initialize-state (Q, n, D) - Initialize the probability distribution of the n qubits of mac
- Quantum-gate (Q, c) - Change the probability distribution of machine Q according to a quant
- $\operatorname{Jack}(Q, c)$ - Increase the probabilities of all states defined by condition c.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-433.jpg?height=520&width=735

%---- Page End Break Here ---- Page : 432
Figure 12.14: Jacking the probability of all states where qubit $q_{2}=1$.
where $c=$ "qubit $q_{2}=1$ ", as shown in Figure 12.14 This takes time proportional to the numbe
That this can be done in constant time should be recognized as surprising. Even if the condition
- Sample (Q) - Select exactly one of the 2^{n} states at random as per the current probabil
"Quantum" algorithms are sequences of these powerful operations, perhaps augmented with control l

\subsection{Grover's Algorithm for Database Search}

The first algorithm we will consider is for database search, or more generally function inversion

We can create such a system Q using the Initialize-state (Q, n, D) instruction, with an app

The instruction set described above does not give us a print statement, other than Sample (Q).
must jack up the probability of all strings with the right value qubits. This suggests an algorit

Search(Q,S)
Repeat
Jack(Q, "all strings where $S = q_n \ldots q_{n+m-1}$")
Until probability of success is high enough
Return the first n bits of Sample(Q)

Each such Jack operation takes constant time, which is fast. But it increases the probabilities a

Solving Satisfiability?

An interesting application of Grover's database searching algorithm is an algorithm for solving s

Now let's add an $(n+1)$ st qubit to the system, to store whether the i th string satisfies a g

Now suppose we perform a $\operatorname{Search}\left(Q,\left(q_{n}==1\right)\right)$ operation, r

Grover's search algorithm runs in $O(\sqrt{N})$ time, where $N=2^{n}$. Since $\sqrt{N}=$ $\sqrt{2

Take-Home Lesson: Despite their powers, quantum computers cannot solve NP-complete problems in po

![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-435.jpg?height=27

%---- Page End Break Here ---- Page : 434
Figure 12.15: Transforming the integers from the frequency to the time domain computes

\subsection{The Faster "Fourier Transform"}

The fast Fourier Transform (FFT) is the most important algorithm in signal processing,

Computing Fourier transforms is most simply done using an $O\left(N^{2}\right)$ algori

It happens that each of these stages of the FFT circuit can be implemented using $\log

But there is a catch. We now have an n-qubit quantum system Q, where the Fourier c

This is a very restricted type of "Fourier transform," returning just the index of wha

\subsection{Shor's Algorithm for Integer Factorization}

There is an interesting connection between periodic functions (meaning they repeat at
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-436.jpg?height=13

%---- Page End Break Here ---- Page : 435
Figure 12.16: Sampling from multiples of the factors of M (here 21) is unlikely to d
occur k positions apart on a number line. What are the integers that have 7 as facto

The FFT enables us to convert a series in the "time domain" (here consisting of all mu

Now suppose we are interested in factoring a given integer $M<N$. Through quantum magi

The greatest common divisor $\operatorname{gcd}(x, y)$ is the largest integer d such

The complete Shor's algorithm for factoring, in pseudocode, is as follows:

Factor (M)

Set up an n-qubit quantum system Q, where $N=2^{n}$ and $M<N$.
Initialize Q so that $p(i)=1 / 2^{n}$ for all $0 \leq i \leq N-1$.
Repeat
$$
\operatorname{Jack}(Q, " \text { all } i \text { such that }(\operatorname{gcd}(i, M)>
$$

Until the probabilities of all terms relatively prime to M are very small. FFT(Q).

For $j=1$ to n
$$
S_{j}=\operatorname{Sample}(Q)
$$
$$
\text { If } \left.\left(\left(d=G C D\left(S_{j}, S_{k}\right)\right)>1\right) \text { and }(d \
$$

Return(d) as a factor of M
Otherwise report no factor was found
Each of these operations takes time proportional to n, not $M=\Theta\left(2^{n}\right)$,
so this is an exponential-time improvement over divide-and-test factoring. No polynomial-time alg

\subsection{Prospects for Quantum Computing}

What are the prospects for quantum computing? I write this in the year of normal vision (2020), a
- Quantum computing is a real thing, and is gonna happen-One develops a reasonably trustworthy bu
- Quantum computing is unlikely to impact the problems considered in this book - The value of you
The fastest technology does not necessarily take over the world. The highest achievable data tran
- The big wins are likely to be in problems computer scientists don't really care about - It is r

We shall see. I look forward to writing the fourth edition of this book, perhaps in 2035, to lear

You should be aware that the "quantum" computing model I describe here differs from real quantum
- The role of probabilities are played by complex numbers - Probabilities are real numbers betwee
- Reading the state of a quantum system destroys it - When we randomly sample from the state of a

%---- Page End Break Here ---- Page : 437
The key hurdle of quantum computing is how to extract the answer we want, because this measuremen
- Real quantum systems breakdown (or decohere) easily - Manipulating individual atoms to do compl
- Initializing quantum states and the powers of quantum gates are somewhat different than describ

Chapter Notes

Kirkpatrick et al.'s original paper on simulated annealing \\cite{kirkpatrick1983optimization} in

The heuristic TSP solutions presented here employ vertex-swap as the local neighborhood operation
edges with a vertex-swap. This improves the possibility of a local improvement. However, more sop

%---- Page End Break Here ---- Page : 438
The different heuristic search techniques are ably presented in Aarts and Lenstra \\cite{aarts199

Our work using simulated annealing to compress DNA arrays was reported in Bradley and Skiena \\ci

If my introduction to quantum computing whetted your interest, I would encourage you to

\section{Exercises}

Special Cases of Hard Problems

12-1. [5] Dominos are tiles represented by integer pairs $\left(x_{i}, y_{i}\right)$,
(a) Prove that finding the longest domino chain is NP-complete.
(b) Give an efficient algorithm to find the longest domino chain where the numbers inc
12-2. [5] Let $G=(V, E)$ be a graph and x and y be two distinct vertices of G. E
(a) Prove that it is NP-complete to find the path from x to y where you can collec
(b) Give an efficient algorithm if G is a directed acyclic graph (DAG).

12-3. [8] The Hamiltonian completion problem takes a given graph G and seeks an algo
an efficient algorithm if G is a tree. Give an efficient and provably correct algori

%---- Page End Break Here ---- Page : 439
Approximation Algorithms

12-4. [4] In the maximum satisfiability problem, we seek a truth assignment that satis

12-5. [5] Consider the following heuristic for vertex cover. Construct a DFS tree of t
12-6. [5] The maximum cut problem for a graph $G=(V, E)$ seeks to partition the vertic

12-7. [5] In the bin-packing problem, we are given n objects with weights $w_{1}, w_
The first-fit heuristic considers the objects in the order in which they are given. Fo

12-8. [5] For the first-fit heuristic described just above, give an example where the
12-9. [5] Given an undirected graph $G=(V, E)$ in which each node has degree $\leq d$,
12-10. [5] A vertex coloring of graph $G=(V, E)$ is an assignment of colors to vertice

12-11. [5] Show that you can solve any given Sudoku puzzle by finding the minimum vert

Combinatorial Optimization

For each of the problems below, design and implement a simulated annealing heuristic t
12-12. [5] Design and implement a heuristic for the bandwidth minimization problem dis
12-13. [5] Design and implement a heuristic for the maximum satisfiability problem dis

12-14. [5] Design and implement a heuristic for the maximum clique problem discussed i
12-15. [5] Design and implement a heuristic for the minimum vertex coloring problem di
12-16. [5] Design and implement a heuristic for the minimum edge coloring problem disc
12-17. [5] Design and implement a heuristic for the minimum feedback vertex set proble
12-18. [5] Design and implement a heuristic for the set cover problem discussed in Sec

%---- Page End Break Here ---- Page : 440
"Quantum" Computing

12-19. [5] Consider an n qubit "quantum" system Q, where each of the $N=2^{n}$ states start c
12-20. [5] For the satisfiability problem, construct (a) an instance on n variables that has ex
12-21. [3] Consider the first ten multiples of 11 , namely $11,22, \ldots 110$. Pick two of them
12-22. [8] IBM quantum computing (https://www.ibm.com/quantum-computing/) offers the opportunity

LeetCode

12-1. https://leetcode.com/problems/split-array-with-same-average/\index{array}
12-2. https://leetcode.com/problems/smallest-sufficient-team/
12-3. https://leetcode.com/problems/longest-palindromic-substring/

HackerRank

12-1. https://www.hackerrank.com/challenges/mancala6/
12-2. https://www.hackerrank.com/challenges/sams-puzzle/
12-3. https://www.hackerrank.com/challenges/walking-the-approximate longest-path/

Programming Challenges\index{RAM}

These programming challenge problems with robot judging are available at https://onlinejudge.org
12-1. "Euclid Problem" - Chapter 7, problem 10104.
12-2. "Chainsaw Massacre"-Chapter 14, problem 10043.
12-3. "Hotter Colder"-Chapter 14, problem 10084.
12-4. "Useless Tile Packers"-Chapter 14, problem 10065.

%---- Page End Break Here ---- Page : 441
Chapter 13

\chapter{How to Design Algorithms}

Designing the right algorithm for a given application is a major creative actthat of taking a pro

This book has been designed to make you a better algorithm designer. The techniques presented in

The key to algorithm design (or any other problem-solving task) is to proceed by asking yourself

Towards this end, I provide a sequence of questions designed to guide your search for the right a

The distinction between strategy and tactics is important to keep aware of during any design proc
to win the minor battles we must fight along the way. In problem-solving, it is important to repe

Too many people freeze up in their thinking when faced with a design problem. After rea

Obviously, the more experience you have with algorithm design techniques such as dynami

This list of questions was inspired by a passage in The Right Stuff [Wol79-a wonderful

I hope this book has provided you with the right stuff to be a successful algorithm des
1. Do I really understand the problem?
(a) What exactly does the input consist of?
(b) What exactly are the desired results or output?\index{ASCII}\index{bucketing techni
(c) Can I construct an input example small enough to solve by hand? What happens when I
(d) How important is it to my application that I always find the optimal answer? Might
(e) How large is a typical instance of my problem? Will I be working on 10 items? 1,000
(f) How important is speed in my application? Must the problem be solved within one sec
(g) How much time and effort can I invest in implementation? Will I be limited to simpl
(h) Am I trying to solve a numerical problem? A graph problem? A geometric problem? A s
2. Can I find a simple algorithm or heuristic for my problem?
(a) Will brute force solve my problem correctly by searching through all subsets or ar
i. If so, why am I sure that this algorithm always gives the correct answer?
ii. How do I measure the quality of a solution once I construct it?
iii. Does this simple, slow solution run in polynomial or exponential time? Is my probl
iv. Am I certain that my problem is sufficiently well defined to actually have a correc
(b) Can I solve my problem by repeatedly trying some simple rule, like picking the bigg
i. If so, on what types of inputs does this heuristic work well? Do these correspond to
ii. On what inputs does this heuristic work badly? If no such examples can be found, ca
iii. How fast does my heuristic come up with an answer? Does it have a simple implement
3. Is my problem in the catalog of algorithmic problems in the back of this book?
(a) What is known about the problem? Is there an available implementation that I can us
(b) Did I look in the right place for my problem? Did I browse through all the pictures
(c) Are there relevant resources available on the World Wide Web? Did I do a Google Sch
4. Are there special cases of the problem that I know how to solve?
(a) Can I solve the problem efficiently when I ignore some of the input parameters?
(b) Does the problem become easier to solve when some of the input parameters are set t
(c) How can I simplify the problem to the point where I can solve it efficiently? Why c
(d) Is my problem a special case of a more general problem in the catalog?
5. Which of the standard algorithm design paradigms are most relevant to my problem?
(a) Is there a set of items that can be sorted by size or some key? Does this sorted or
(b) Is there a way to split the problem into two smaller problems, perhaps by doing a b
(c) Does the set of input objects have a natural left-to-right order among its componen
(d) Are there certain operations being done repeatedly, such as searching, or finding t
(e) Can I use random sampling to select which object to pick next? What about construct
(f) Can I formulate my problem as a linear program? How about an integer program?
(g) Does my problem resemble satisfiability, the traveling salesman problem, or some ot
6. Am I still stumped?
(a) Am I willing to spend money to hire an expert (like the author) to tell me what to
(b) Go back to the beginning and work through these questions again. Did any of my answ

%---- Page End Break Here ---- Page : 443\index{amortized analysis}\index{average-case analysis}\

%---- Page End Break Here ---- Page : 444\index{dictionary}\index{future events}\index{244}\index

%---- Page End Break Here ---- Page : 445
Problem-solving is not a science, but part art and part skill. It is one of the skills most worth

\section{Preparing for Tech Company Interviews}

I gather that many of you reading this book have been inspired more by a fear of technical job in

First, with respect to algorithm design people know what they know, and hence cramming the night

Algorithm design problems tend to creep into the interview process in two ways: preliminary codin

That said, performance on these programming challenge problems improves with practice. If you are

For self-study, I recommend solving some coding problems on judging sites like HackerRank and Lee

After you get past the preliminary screening, you will be granted a video or on-site interview. H
generally be like the exercises at the end of each chapter. Some have been designated interview q

%---- Page End Break Here ---- Page : 446
What should you do at the whiteboard to look like you know what you are talking about? First, I e

Students of mine who take these interviews often report that they are given incorrect solutions b

Finally, I have a confession to make. For several years, I served as Chief Scientist at a startup

Of course I strongly encourage that other companies continue to base their screening on algorithm

Good luck to you on your job quest, and may what you learn from this book help you with your job,

%---- Page End Break Here ---- Page : 447
Part II\index{prex string}\index{string data structures}\index{string matching}\index{substring

The Hitchhiker's Guide to Algorithms

%---- Page End Break Here ---- Page : 448
Chapter 14

\chapter{A Catalog of Algorithmic Problems}

This is a catalog of algorithmic problems that arise commonly in practice. It describes

What is the best way to use this catalog? First, think about your problem. If you reca

The catalog entries contain a variety of different types of information that were neve

To make this catalog more accessible, I introduce each problem with a pair of graphics

Once you have identified your problem, the discussion section tells you what you shoul

Available software implementations are discussed in the implementation field of each c

Finally, in deliberately smaller print, the history of each problem will be discussed,

Caveats

This is a catalog of algorithmic problems. It is not a cookbook. It cannot be, because
- For each problem, I suggest algorithms and directions to attack it. These recommenda
- The implementations I recommend are not necessarily complete solutions to your probl
- Please respect the licensing conditions for any implementations you use commercially
- I would be interested in hearing about your experiences with my recommendations, bot

%---- Page End Break Here ---- Page : 450
Chapter 15

\chapter{Data Structures}

Data structures are not so much algorithms as they are the fundamental constructs arou

This puts data structures slightly out of sync with the rest of the catalog. Perhaps t

There are a large number of books on elementary data structures available. My favorite
- Sedgewick [\\cite{sedgewick2011algorithms} - This comprehensive introduction to algo
- Weiss \\cite{weiss2011data} - A nice text, emphasizing data structures more than alg
- Goodrich and Tamassia \\cite{goodrich2014data} - The Java edition makes particularly
- Brass [\\cite{brass2008advanced} - This is a good treatment of more advanced data st

The Handbook of Data Structures and Applications \\cite{mehta2018handbook} provides a
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-452.jpg?height=80

\section{Dictionaries}

Input description: A set of n records, each identified by one or more keys.

Problem description: Build and maintain a data structure to efficiently locate, insert, and delet

Discussion: The abstract data type "dictionary" is one of the most important structures in comput

An essential piece of advice is to carefully isolate the implementation of your dictionary data s

To choose the right data structure for your dictionary, answer the following questions:
- How many items will you have in your data structure? - Will you know this number in advance? Ar
- Do you know the relative frequencies of insert, delete, and search operations? - Static data st
- Will the access pattern for keys be uniform and random? - Search queries exhibit a skewed acces
- Is it critical that each individual operation be fast, or only that the total amount of work do

%---- Page End Break Here ---- Page : 452
An object-oriented generation has emerged that is no more likely to write their own container cla
- Unsorted linked lists or arrays - For small data sets, an unsorted array is probably the easies

A particularly interesting and useful variant is the self-organizing list. Whenever a key is acce
- Sorted linked lists or arrays - Maintaining a sorted linked list is usually not worth the effor
- Hash tables - For applications involving a moderate-to-large number of keys, a hash table is pr
A well-tuned hash table will outperform a sorted array in most applications. However, several des
- How do I deal with collisions? Open addressing can lead to more concise tables with better cach
- How big should the table be? With bucketing, m should be about the same as the maximum number
- What hash function should I use? For strings, something like
$$
H(S)=\alpha^{|S|}+\sum_{i=0}^{|S|-1} \alpha^{|S|-(i+1)} \times \operatorname{char}\left(s_{i}\rig
$$
should work, where α is the size of the alphabet and $\operatorname{char}(x)$ is the funct

%---- Page End Break Here ---- Page : 453
When evaluating a hash function/table implementation, print statistics on the distribution of key
- Binary search trees - Binary search trees are elegant data structures that support fast inserti
Balanced search trees use local rotation operations for restructuring, moving more distant nodes

%---- Page End Break Here ---- Page : 454
Bottom line: Which tree is best for your application? Probably the one of which you have the best
- B-trees - For data sets so large that they will not fit in main memory your best bet will be so

The idea behind a B-tree is to collapse several levels of a binary search tree into a single larg

Even for modest-sized data sets, unexpectedly poor performance of a data structure may result fro
- Skip lists - These are somewhat of a cult data structure. A hierarchy of sorted linked lists is
total expected $O(\lg n)$ query time. The primary benefits of skip lists are ease of analysis and

%---- Page End Break Here ---- Page : 455
Implementations: Modern programming languages provide libraries offering complete and e

LEDA (see Section 22.1.1 (page 713) provides a complete collection of dictionary data s

Notes: Knuth \\cite{knuth1997taocp1} provides the most detailed analysis and expositio

The Handbook of Data Structures and Applications \\cite{mehta2018handbook} provides up-

The 1996 DIMACS implementation challenge focused on elementary data structures, includi

The cost of transferring data back and forth between levels of the memory hierarchy (R

Splay trees and other modern data structures have been studied using amortized analysi

Related problems: Sorting (see page 506), searching (see page 510).

%---- Page End Break Here ---- Page : 456
October 30
December 7
July 4
January 1
February 2
December 25
January 30
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-457.jpg?height=40

Input
Output

\section{Priority Queues}

Input description: A set of records with totally ordered keys.
Problem description: Build and maintain a data structure for providing quick access to

Discussion: Priority queues are useful data structures in simulations, particularly fo

If your application will perform no insertions after the initial query, there is no ne

However, you will need a real priority queue when mixing queries with insertions and d
- What other operations do you need? - Will you be searching for arbitrary keys, or jus
- Do you know the maximum data structure size in advance? - The issue here is whether y
- Might you raise or lower the priority of elements already in the queue? - Changing t

%---- Page End Break Here ---- Page : 457
Your choices are between the following basic priority queue implementations:
- Sorted array or list - A sorted array is very efficient to both identify the smallest element a
- Binary heaps - This simple, elegant data structure supports both insertion and extract-min in $
Binary heaps are the right answer when you know an upper bound on the number of items in your pri
- Bounded-height priority queue - This array-based data structure permits constant-time insertion
Bounded-height priority queues are very useful to maintain the vertices of a graph sorted by degr
- Binary search trees - Binary search trees make effective priority queues, since the smallest el
- Fibonacci and pairing heaps - These complicated priority queues are designed to speed up decrea
v than previously established. Properly implemented and used, they lead to better performance o

%---- Page End Break Here ---- Page : 458
Implementations: Modern programming languages provide libraries offering complete and efficient p

LEDA (see Section 22.1.1 (page 713) provides a complete collection of priority queues in C++, inc

Notes: The Handbook of Data Structures and Applications \\cite{mehta2018handbook} provides severa

Double-ended priority queues extend the basic heap operations to simultaneously support both find

Bounded-height priority queues are useful data structures in practice, but do not promise good wo

Fibonacci heaps [\\cite{fredman1987fibonacci} BLT12 support insert and decrease-key operations in

Heaps define a partial order that can be built using a linear number of comparisons. The familiar

Related problems: Dictionaries (see page 440), sorting (see page 506, shortest path (see page 554
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-460.jpg?height=708&width=124

%---- Page End Break Here ---- Page : 459
\section{Suffix Trees and Arrays}

Input description: A reference string S.
Problem description: Build a data structure for quickly finding all places where an arbitrary que

Discussion: Suffix trees and arrays are phenomenally useful data structures for solving string pr

In its simplest instantiation, a suffix tree is simply a trie of the n suffixes of an n-chara

Tries are useful in testing whether a given query string q is in the set of strings. We travers

A suffix tree is simply a trie of all proper suffixes of S. The suffix tree enables
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-461.jpg?height=487&width=762

%---- Page End Break Here ---- Page : 460
Figure 15.1: A trie on strings the, their, there, was, and when (on left). The suffix a
you to test whether q is a substring of S, because any substring of S is the pre

The catch is that constructing a full suffix tree in this manner can require $O\left(n

Even better, there exist $O(n)$ algorithms to construct this collapsed tree, by making

But what can you do with suffix trees? Consider the following applications:
- Find all occurrences of q as a substring of S - Just as with a trie, we can walk
- Longest substring common to a set of strings - Build a single collapsed suffix tree
- Find the longest palindrome in S - A palindrome is a string that reads the same if

%---- Page End Break Here ---- Page : 461
Since linear-time suffix tree construction algorithms are non-trivial, I recommend usi

A suffix array is in principle just an array that contains all the n suffixes of S

In practice, suffix arrays are typically as fast or faster to search than suffix trees

Some care must be taken to construct suffix arrays efficiently, because there are $O\l

Implementations: There now exist a wealth of suffix array implementations available. I

No less than eight different $\mathrm{C} / \mathrm{C}++$ implementations of compressed

Suffix tree implementations are also readily available. A SuffixTree class is provided
an implementation of suffix trees. It is available at https://web.cs.ucdavis. edu/~gus

%---- Page End Break Here ---- Page : 462
Notes: Tries were first proposed by Fredkin \\cite{fredkin1962trie}, the name coming f

Efficient algorithms for suffix tree construction are due to Weiner \\cite{weiner1973l

Suffix arrays were invented by Manber and Myers [\\cite{manber1993suffix}], although a

Recent work has resulted in the development of compressed full text indexes that offer

The power of suffix trees can be further augmented by using a data structure to comput

Related problems: String matching (see page 685), text compression (see page 693), lon
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-464.jpg?height=58

%---- Page End Break Here ---- Page : 463
\section{Graph Data Structures}

Input description: A graph G.
Problem description: Represent the graph G using a flexible, efficient data structure.

Discussion: The two basic data structures for representing graphs are adjacency matrices and adja

The issues in deciding which data structure to use include:
- How big will your graph be? - How many vertices will it have, both typically and in the worst c
- How dense will your graph be? - If your graph is very dense, meaning that a large fraction of t
- Which algorithms will you be implementing? - Certain algorithms are more natural on adjacency m
- Will you modify the graph over the course of the computation? - Efficient static graph implemen
- Will your graph be a persistent, online structure? - Data structures and databases are two diff

%---- Page End Break Here ---- Page : 464
Building a good general purpose graph type is a substantial project. I thus suggest that you chec

Planar graphs are those that can be drawn in the plane so no two edges cross. Graphs arising in m

Hypergraphs are generalized graphs where each edge may link subsets of more than two vertices. Su

Two basic data structures for hypergraphs are:
- Incidence matrices, which are analogous to adjacency matrices. They require $n \times m$ space,
- Bipartite incidence structures, which are analogous to adjacency lists, and thus suited for spa
add an edge (i, j) in the incidence structure whenever vertex i of the hypergraph appears in

%---- Page End Break Here ---- Page : 465
Special efforts must be taken to represent very large graphs efficiently. However, interesting pr

If your graph is extremely large, it may become necessary to switch to a hierarchical representat

Implementations: LEDA (see Section 22.1.1 (page 713) is a commercial product that provides the be

The C++ Boost Graph Library \\cite{siek2002boost} (http://www.boost.org/libs/ graph) is more read

Neo4j (https://neo4j.com/) is a widely used graph database, where the J stands for Java. Needham

The Stanford Graphbase (see Section 22.1.7 (page 715) provides a simple but flexible graph data s

%---- Page End Break Here ---- Page : 466

My (biased) preferences in C language graph types include the libraries from this book

Notes: The advantages of adjacency list data structures for graphs became apparent wit

The improved efficiency of static graph types was revealed by Naher and Zlotowski \\ci

Matrix representations of graphs can exploit the power of linear algebra in problems r

Dynamic graph algorithms \\cite{eppstein1998dynamic} are data structures that maintain

Hierarchically defined graphs often arise in VLSI design problems, because designers m

Related problems: Set data structures (see page 456), graph partition (see page 601.\i
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-468.jpg?height=58

%---- Page End Break Here ---- Page : 467
\section{Set Data Structures}

Input description: A universe of items $U=\left\{u_{1}, \ldots, u_{n}\right\}$ on whic

Problem description: Represent each subset so as to efficiently (1) test whether $u_{i

Discussion: In mathematical terms, a set is an unordered collection of objects drawn f

We distinguish sets from two other types of objects: dictionaries and strings. A colle

Multisets permit elements to have more than one occurrence. Data structures for sets c

When every subset contains exactly two elements, they can be thought of as edges in a
connected components or shortest path in a graph/hypergraph.
Your primary alternatives for representing arbitrary subsets are:
- Bit vectors - An n-bit vector or array can represent any subset S on a universal
- Containers or dictionaries - A subset S can also be represented using a linked lis
- Bloom filters - We can emulate a bit vector in the absence of a fixed universal set
A Bloom filter uses several (say k) different hash functions $H_{1}, \ldots H_{k}$,
This hashing-based data structure is much more space-efficient than dictionaries, for

%---- Page End Break Here ---- Page : 468
Many applications involve collections of subsets that are pairwise disjoint, meaning t

The primary issue with set partition data structures is maintaining changes over time,
- Collection of containers - Representing each subset in its own container or dictiona
- Generalized bit vector - Let the i th element of an array denote the number/name o
- Dictionary with a subset attribute - Similarly, each item in a binary tree can be as

- Union-find data structure - We represent a subset using a rooted tree where each node points to

%---- Page End Break Here ---- Page : 469
Implementation details have a big impact on asymptotic performance here. Always selecting the lar

Implementations: Modern programming languages provide libraries offering complete and efficient s

An implementation of union-find underlies any implementation of Kruskal's minimum spanning tree a

%---- Page End Break Here ---- Page : 470
The computer algebra system REDUCE (http://www.reduce-algebra. com/) contains SETS, a package sup

Notes: Optimal algorithms for set operations such as intersection and union were presented in Rei

Certain balanced tree data structures support merge/meld/link/cut operations, which permit fast w

Galil and Italiano \\cite{galil1991data} survey data structures for disjoint set union. The upper

The power set of a set S is the collection of all $2^{|S|}$ subsets of S. Explicit manipulati

Related problems: Generating subsets (see page 521), generating partitions (see page 524), set co
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-472.jpg?height=704&width=124

%---- Page End Break Here ---- Page : 471
\section{Kd-Trees}

Input description: A set S of n points (or more complicated geometric objects) in k dimensi

Problem description: Construct a tree that partitions space by half-planes such that each object

Discussion: Kd-trees and related spatial data structures hierarchically decompose space into a sm

Typical algorithms construct kd-trees by partitioning point sets. Each node in the tree is define

The cutting planes along any path from the root to another node defines a unique box-shaped regio

Flavors of kd-trees differ in exactly how the splitting plane is selected, but options include:
- Cycling through the dimensions - partition first on d_{1}, then d_{2}, \ldots, d_{k}$ before
- Cutting along the largest dimension - select the partition dimension that makes the resulting b
- Quadtrees or octtrees - Instead of partitioning with single planes, use all axis-parallel plane
- Random projection trees - Here the cutting plane for a node is defined by a random slope/direct
- BSP-trees - Binary space partitions use general (i.e.not just axis-parallel) cutting planes to
- Ball trees - A ball tree is a hierarchical data structure on points, where each node is associa

%---- Page End Break Here ---- Page : 472
Ideally, our partitions will split both the space (ensuring fat, regular regions) and
- Point location - To identify which cell a query point q lies in, start at the root
- Nearest-neighbor search - To find the point in S closest to a given query point $q

%---- Page End Break Here ---- Page : 473
Point p is likely close to q, but it might not be the absolute closest neighbor. W
- Range search - Which points lie within a query box or region? Starting from the root
- Partial key search - Suppose we want to find a point p in S, but we do not have

Kd-trees are most useful for a small to moderate number of dimensions, say from two up

The bottom line is you should try to avoid working in high-dimensional spaces, perhaps

Implementations: KDTREE 2 contains C++ and Fortran 95 implementations of $k d$-trees f

Samet's spatial index demos (http://donar.umiacs.umd.edu/quadtree/) provide a series o

The 1999 DIMACS implementation challenge focused on data structures for nearest-neighb

%---- Page End Break Here ---- Page : 474
Notes: Samet \\cite{samet2006foundations} is the best reference on kd-trees and other

The performance of spatial data structures degrades with high dimensionality. Balltree

Projecting high-dimensional spaces onto a random lower-dimensional hyperplane has rece

Algorithms that quickly produce a point provably close to the query point are an impor

Related problems: Nearest-neighbor search (see page 637), point location (see page 644

%---- Page End Break Here ---- Page : 475
Chapter 16

\chapter{Numerical Problems}\index{geometric degeneracy!problems}\index{solving linear

If most problems you encounter are numerical in nature, there is an excellent chance t

Numerical computation has been of increasing importance because of machine learning, w
- Issues of precision and error - Numerical algorithms typically perform repeated floa
A simple and dependable way to test for round-off errors in numerical programs is to r

```
- Extensive libraries of codes - Large, high-quality libraries of numerical algorithms have exist

Regardless of why, you should exploit this software base. There is probably
no reason to implement algorithms for any of the problems in this section, as opposed to using ex

Many scientists and engineers have ideas about algorithms that culturally derive from the simple

There is a vast literature on numerical algorithms. In addition to Numerical Recipes, recommended
- Chapra and Canale \\cite{chapra2016numerical} - The contemporary market leader in numerical ana
- Mak \\cite{mak2002java} - This enjoyable text introduces Java to the world of numerical computa
- Hamming \\cite{hamming1987numerical} - This oldie but goodie provides a clear and lucid treatme
- Skeel and Keiper \\cite{skeel2000elementary} - A readable and interesting treatment of basic nu
- Cheney and Kincaid \\cite{cheney2012numerical} - A traditional Fortran-based numerical analysis
- Buchanan and Turner \\cite{buchanan1992numerical} - Thorough language-independent treatment of
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-478.jpg?height=496&width=123


%---- Page End Break Here ---- Page : 477
\section{Solving Linear Equations}

Input description: An $m \times n$ matrix $A$ and an $m \times 1$ vector $b$, together representi

Problem description: What is the vector $x$ such that $A \cdot x=b$ ?
Discussion: The need to solve linear systems arises in an estimated $75 \%$ of all scientific com

Not all systems of equations have solutions. Just try to solve $2 x+3 y=5$ and $2 x+3 y=6$. Some

Solving linear systems is a problem of such scientific and commercial importance that excellent c

Gaussian elimination is based on the observation that the solution to a system of linear equation

The time complexity of Gaussian elimination on an $n \times n$ system of equations is $O\left(n^{
other equations to clear the $i$ th (of $n$ ) variable. But on this problem, constants matter. Al


%---- Page End Break Here ---- Page : 478
Issues to worry about include:
- Are round-off errors and numerical stability affecting my solution? Gaussian elimination would
To eliminate the danger of numerical errors, it pays to substitute the solution back into each of

The key to minimizing round-off errors in Gaussian elimination is selecting the right equations a
- Which routine in the library should I use? - Selecting the right code is also somewhat of an ar
- Is my system sparse? - A key to recognizing that you have a special-case linear system is estab
- Will I be solving many systems using the same coefficient matrix? - In applications such as lea
$$
A \cdot x=(L \cdot U) \cdot x=L \cdot(U \cdot x)=b
```

$$

This is efficient because back substitution solves a triangular system of equations in

%---- Page End Break Here ---- Page : 479
The problem of solving linear systems is equivalent to that of matrix inversion, since

Implementations: The library of choice for solving linear systems is apparently LAPACK

JScience provides an extensive linear algebra package (including determinants) as part

Numerical Recipes \\cite{press2007numerical} (http://numerical.recipes/) provides guida

Notes: Golub and van Loan [\\cite{golub1996matrix} is the standard reference on algori

Parallel algorithms for linear systems are discussed in \\cite{gallivan1990parallel} \\

Matrix inversion and (hence) linear systems solving can be done in matrix multiplicati

The HHL quantum computing algorithm has been proposed to solve $n \times n$ linear sys

Related problems: Matrix multiplication (see page 472), determinant/permanent (see page
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-481.jpg?height=41

%---- Page End Break Here ---- Page : 480
\section{Bandwidth Reduction}

Input description: A graph $G=(V, E)$, representing an $n \times n$ matrix M of zero

Problem description: Which permutation p of the vertices minimizes the length of the

Discussion: Bandwidth reduction lurks as a hidden but important problem for both graph

Bandwidth minimization on graphs arises in more subtle ways. Arranging n circuit com

The bandwidth problem seeks a linear ordering of the vertices, which minimizes the len

Unfortunately, all these variants of bandwidth minimization are NP-complete. It stays

Fortunately, ad hoc heuristics have been well studied and production-quality implement
left-most point of the ordering. All of the vertices that are distance 1 from v are

%---- Page End Break Here ---- Page : 481

Implementations of the most popular heuristics-the Cuthill-McKee and Gibbs-Poole-Stockmeyer algor

Brute-force search programs can find the exact minimum bandwidth, by backtracking through the set

Implementations: Del Corso and Manzini's \\cite{delcorso1999bandwidth} code for exact solutions t

Fortran language implementations of both the Cuthill-McKee algorithm \\cite{cuthill1969bandwidth}

Notes: Diaz et al. \\cite{diaz2002survey} provide an excellent survey on algorithms for bandwidth

The hardness of the bandwidth problem was first established by Papadimitriou \\cite{papadimitriou

Related problems: Solving linear equations (see page 467), topological sorting (see page 546).
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-483.jpg?height=586&width=123

%---- Page End Break Here ---- Page : 482
\section{Matrix Multiplication}

Input description: An $x \times y$ matrix A and a $y \times z$ matrix B.
Problem description: Compute the $x \times z$ matrix $A \times B$.
Discussion: Matrix multiplication is a fundamental problem in linear algebra. Its main significan

Matrix multiplication is often used for data rearrangement problems instead of hard-coded logic.

The following tight algorithm computes the product of $x \times y$ matrix A and $y \times z$ ma

```
$$
\begin{aligned}
& \text { for } i=1 \text { to } x \text { do } \\
& \qquad \begin{array}{l}
\text { for } j=1 \text { to } z \\
\quad \text { for } k=1 \text { to } y \\
\quad M[i, j]=M[i, j]+A[i, k] \cdot B[k, j]
\end{array}
\end{aligned}
$$
```

An implementation in C appears in Section 2.5.4 (page 45). This straightforward algorithm would s
answer. Such a permutation will change the memory access patterns and thus how effectively the ca

%---- Page End Break Here ---- Page : 483
When multiplying two bandwidth- b matrices, a speedup to $O(x b z)$ is possible. Zero values ca

Asymptotically faster algorithms for matrix multiplication exist, using clever divide-and-conquer

There is a better way to save computation when you are multiplying a chain of more tha

Matrix multiplication has a particularly interesting interpretation in counting the nu

Implementations: D'Alberto and Nicolau \\cite{dalberto2007adaptive} have engineered a

An $O\left(n^{3}\right)$ algorithm will likely be your best bet unless your matrices a

%---- Page End Break Here ---- Page : 484
Notes: Winograd's algorithm Win68 for fast matrix multiplication reduces the number of

In my opinion, the history of theoretical algorithm design began when Strassen \\cite{

The problem of Boolean matrix multiplication can be reduced to that of general matrix

Engineering efficient matrix multiplication algorithms requires careful management of

The inverse of multiplying matrices is factoring them, reducing M to A and B suc

The interest in the squares of graphs goes beyond counting paths. Fleischner \\cite{re

Good expositions of the matrix-chain algorithm include \\cite{baase1999computer} \\cit

Related problems: Solving linear equations (see page 467), shortest path (see page 554
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-486.jpg?height=59

%---- Page End Break Here ---- Page : 485
\section{Determinants and Permanents}

Input description: An $n \times n$ matrix M.
Problem description: What is the determinant $|M|$ or permanent $\operatorname{perm}(M

Discussion: Determinants of matrices provide a clean and useful abstraction that can b
- Testing whether a matrix is singular, meaning that the matrix does not have an inver
- Testing whether a set of d points lies on a plane in fewer than d dimensions. If
- Testing whether a point lies to the left or right of a line or plane. This problem r
- Computing the area or volume of a triangle, tetrahedron, or other simplicial complex

The determinant of a matrix M is defined as a sum over all n ! possible permutatio
$$
|M|=\sum_{i=1}^{n!}(-1)^{\operatorname{sign}\left(\pi_{i}\right)} \prod_{j=1}^{n} M\le
$$
where $\operatorname{sign}\left(\pi_{i}\right)$ denotes the number of pairs of element

Directly implementing this definition yields an $O(n!)$ algorithm, as does the cofactor expansion

%---- Page End Break Here ---- Page : 486
The permanent is a closely related function that arises in combinatorial problems. For example, t
$$
\operatorname{perm}(M)=\sum_{i=1}^{n!} \prod_{j=1}^{n} M\left[j, \pi_{j}\right]
$$
differing from the determinant only in that all products are positive.
Surprisingly, it is NP-hard to compute the permanent, even though the determinant can easily be c

Implementations: The linear algebra package LINPACK contains a variety of Fortran routines for co

Nijenhuis and Wilf \\cite{nijenhuis1978combinatorial} provide an efficient Fortran routine to com

Two different codes for approximating the permanent are provided by Barvinok. The first, based on

Notes: Cramer's rule reduces the problems of matrix inversion and solving linear systems to that

Determinants can be computed in $o\left(n^{3}\right)$ time using fast matrix multiplication, as s

The problem of computing the permanent was shown to be \#P-complete by Valiant \\cite{valiant1979
mial time. A counting machine returns the number of distinct solutions to a problem. Counting the

%---- Page End Break Here ---- Page : 487
Minc \\cite{minc1978permanents} is the primary reference on permanents. A variant of an $O\left(n

Probabilistic algorithms have been developed for estimating the permanent, culminating in a fully

Related problems: Solving linear systems (see page 467), matching (see page 562 , geometric primi
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-489.jpg?height=527&width=122

%---- Page End Break Here ---- Page : 488
\section{Constrained/Unconstrained Optimization}

Input description: A function $f\left(x_{1}, \ldots, x_{n}\right)$.
Problem description: Which point $p=\left(p_{1}, \ldots, p_{n}\right)$ maximizes (or minimizes) t

Discussion: Of the seventy-five problems in this catalog, this is the one whose importance has gr

Optimization arises whenever you have an objective function that must be tuned for optimal perfor
$$
\text { stock-goodness }(t)=c_{1} \times \operatorname{price}(t)+c_{2} \times \operatorname{inter
$$

We seek the numerical values c_{1}, c_{2}, c_{3} whose stock-goodness function does

Unconstrained optimization problems also arise in scientific computation. Physical sys

Global optimization problems tend to be hard, and there are many ways to go about them
- Am I doing constrained or unconstrained optimization? - In unconstrained optimizatio
- Is the function I am trying to optimize described by a formula? - Sometimes you need
- Is your function convex? The main difficulty of global optimization is getting trapp
Convex functions have exactly one maxima (or minima), corresponding to a world with a

%---- Page End Break Here ---- Page : 489
How do you know whether your function is convex? This is beyond the scope of my book,
- How continuous or smooth is my function? - Even if your function is non-convex, ther
- Is it expensive to compute the function at a given point? - Sometimes we are given a

%---- Page End Break Here ---- Page : 490
Our freedom to search in such a situation depends upon how efficiently we can evaluate
Such a situation arises in tuning evaluation functions for games. Suppose that $f\left

The most efficient algorithms for convex optimization use derivatives and partial deri

For constrained optimization, finding a point that satisfies all the constraints is of

%---- Page End Break Here ---- Page : 491
At the end, the penalties should be very high to ensure that all constraints are satis

Simulated annealing is a fairly robust approach to constrained optimization, particula

It is a good idea to try several different methods on any given optimization problem.

Implementations: The world of constrained/unconstrained optimization is sufficiently c

NEOS (Network-Enabled Optimization System) provides a unique servicethe opportunity to

General purpose simulated annealing implementations are available, and are likely the

Notes: Bertsekas \\cite{bertsekas2015convex}, Boyd and Vandenberghe \\cite{boyd2004conv

The full objective functions associated with machine learning problems are often linea

Simulated annealing was devised by Kirkpatrick et al. [\\cite{kirkpatrick1983optimizat

```
Genetic algorithms were developed and popularized by Holland \\cite{holland1975adaptation} \\cite

Related problems: Linear programming (see page 482), satisfiability (see page 537 .
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-493.jpg?height=591&width=123


%---- Page End Break Here ---- Page : 492
\section{Linear Programming}

Input description: A set $S$ of $n$ linear inequalities on $m$ variables\index{NP}
$$
S_{i}=\sum_{j=1}^{m} c_{i j} \cdot x_{j} \geq b_{i}, 1 \leq i \leq n
$$
and a linear optimization function $f(X)=\sum_{j=1}^{m} c_{j} \cdot x_{j}$.
Problem description: Which variable assignment $X^{\prime}$ maximizes the objective function $f$

Discussion: Linear programming is the most important problem in mathematical optimization and ope
- Resource allocation - We seek to invest a given amount of money to maximize our return. Often c
- Approximating the solution of inconsistent equations - A set of $n$ linear equations on $m$ var
- Graph algorithms - Many graph problems described in this book, including shortest path, biparti
special cases of linear programming. Most of the rest, including traveling salesman, set cover, a


%---- Page End Break Here ---- Page : 493
The simplex method is the standard algorithm for linear programming. Each constraint in a linear

While the basic simplex algorithm is not particularly complex, there is considerable art to produ

The bottom line on linear programming is that you are much better off using an existing LP code t

Issues that arise in linear programming include:
- Do any variables have integrality constraints? - It is impossible to send 6.54 airplanes from N
Unfortunately, it is NP-complete to solve integer or mixed programs to optimality. But there are
- Do I have more variables or constraints? - Any linear program with $m$ variables and $n$ inequa
program with $n$ variables and $m$ inequalities. This is important to know, because the running t
- What if my optimization function or constraints are not linear? - In least-squares curve fittin
There are three possible courses of action when you must solve a nonlinear program. The best is t
- What if my model does not match the input format of my LP solver? Many linear programming imple
Do not fear. There exist standard transformations to map arbitrary LP models into standard form.


%---- Page End Break Here ---- Page : 494
Implementations: There are at least three reasonable choices in free LPsolvers. Lp_solve, written

Benchmark studies $\mathrm{GAD}^{+} 13$, \\cite{meindl2012analysis}] agree that commercial solver
```

```
%---- Page End Break Here ---- Page : 495
NEOS (Network-Enabled Optimization System) provides a unique servicethe opportunity to

Notes: The need for optimization via linear programming arose in logistics problems in

Smoothed analysis measures the complexity of algorithms assuming that their inputs are

Khachian's ellipsoid algorithm [\\cite{khachian1979polynomial}] first proved that linea

Semidefinite programming deals with optimization problems over symmetric positive semic

Linear programming is P-complete under log-space reductions \\cite{dobkin1979linear}. T

Related problems: Constrained and unconstrained optimization (see page 478, network flo
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-497.jpg?height=638


%---- Page End Break Here ---- Page : 496
\section{Random Number Generation}

Input description: Nothing, or perhaps a seed.
Problem description: Generate a sequence of random integers.
Discussion: Random numbers have a surprising variety of interesting and important appli

Unfortunately, generating random numbers looks much easier than it really is. Indeed, i

There can be serious consequences to using a bad random-number generator. In one famous
- Should my program produce the same "random" numbers each time it runs? - A poker game
you play quickly loses interest. One common solution uses the lower-order bits of the m


%---- Page End Break Here ---- Page : 497
Such methods are adequate for games, but not for serious simulations. There are liable
- How good is my compiler's built-in random number generator? - If you need uniformly g

If you are going to bet the farm on the results of your simulation, you had better tes
- What if I must implement my own random-number generator? - The standard algorithm of
$$
R_{n}=\left(a R_{n-1}+c\right) \quad \bmod m
$$

Linear congruential generators work the same way roulette wheels do. The long path of t

A substantial theory has been developed to properly select the constants $a, c, m$, and
```

%---- Page End Break Here ---- Page : 498
Note that the stream of numbers produced by a linear congruential generator starts to cycle the i
- What if I don't want such large random numbers? - The linear congruential generator R_{n} pro
- What if I need non-uniformly distributed random numbers? - Generating random numbers according

This method is correct, but can be slow. When the volume of the region of interest is small relat

Be cautious about inventing your own technique, because it can be tricky to obtain the right prob
- How long should I run my Monte Carlo simulation? - The longer you run a simulation, the more ac
Instead of jacking up the length of a simulation run to the max, it is usually more informative t
results are repeatable. This exercise corrects the natural tendency to see a simulation as giving

%---- Page End Break Here ---- Page : 499
Implementations: Seehttps://www.agner.org/random for an excellent website on random-number genera

Parallel simulations make special demands on random-number generators. How can we ensure that ran

The National Institute of Standards [$\left.\mathrm{BRS}^{+} 10\right]$ has prepared an extensiv

True random-number generators extract random bits by observing physical processes. The website ht

Notes: Knuth \\cite{knuth1997taocp2} provides a thorough treatment of random-number generation, w

That said, see \\cite{gentle2006random} L'E12 for more recent developments in random number gener

Tables of random numbers appear in most mathematical handbooks as relics from the days before the

The deep relationship between randomness and information is explored within the theory of Kolmogo

Related problems: Constrained/unconstrained optimization (see page 478), generating permutations
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-501.jpg?height=632&width=123

%---- Page End Break Here ---- Page : 500
\section{Factoring and Primality Testing}

Input description: An integer n.
Problem description: Is n a prime number? If not, what are its factors?
Discussion: The dual problems of integer factorization and primality testing have surprisingly ma

Factoring and primality testing are closely related problems, although they are quite different a

The simplest algorithm for both of these problems is brute-force trial division. To factor n, c

Such algorithms can be sped up by using a precomputed table of small primes to avoid t
vector of all odd numbers less than 1,000,000 fits in under 64 kilobytes. Even tighter

%---- Page End Break Here ---- Page : 501
Although trial division runs in $O(\sqrt{n})$ time, it is not a polynomial-time algori

Randomized algorithms make it much easier to test whether an integer is prime. Fermat'

Although the primes are scattered in a seemingly random way throughout the integers, t

Quantum computers are (theoretically!) capable of factoring large integers very fast,

Implementations: Several general systems for computational number theory are available

A Library for doing Number Theory (NTL) is a high-performance, portable C++ library pr
bitrary length integers and vectors, matrices, and polynomials over the integers and o

%---- Page End Break Here ---- Page : 502
Finally, MIRACL (Multiprecision Integer and Rational Arithmetic C/C++ Library) impleme

Notes: Expositions on modern algorithms for factoring and primality testing include Cr

Agrawal, Kayal, and Saxena \\cite{agrawal2004primes} solved a long-standing open probl

The complexity class co-NP is the set of problems for which a polynomial-time algorith

Shor \\cite{shor1999polynomial} sparked great interest in quantum computing with his a

The Miller-Rabin \\cite{miller1976riemann}, \\cite{rabin1980probabilistic} randomized

Mechanical sieving devices provided the fastest way to factor integers surprisingly fa

The integer RSA-129 was factored in eight months using over 1,600 computers. This was

Related problems: Cryptography (see page 697), high precision arithmetic (see page 493

%---- Page End Break Here ---- Page : 503
\author{
49578291287491495151508905425869578
}

74367436931237242727263358138804367

$2 / 3$
Input
Output

\section{Arbitrary-Precision Arithmetic}

Input description: Two very large integers, x and y.
Problem description: What is $x+y, x-y, x \times y$, and x / y ?
Discussion: Every programming language rising above basic assembler supports single- and perhaps

Other applications require much larger integers. The RSA algorithm for public-key cryptography re

What should you do when you need large integers?
- Am I solving a problem instance requiring large integers, or do I have an embedded application?
If you have an embedded application requiring high-precision arithmetic instead, you should use a
- Do I need high- or arbitrary-precision arithmetic? - Is there an upper bound on how big your in
- What base should I do arithmetic in? - It is perhaps simplest to implement your own high-precis
Why? The higher the base, the fewer digits we need to represent a number. Compare 64 decimal with
The primary complication of using a larger base is that integers must be converted to and from ba
- How low-level are you willing to go for fast computation? - Hardware addition is much faster th

%---- Page End Break Here ---- Page : 504
The algorithms of choice for the basic arithmetic operations are as follows:
- Addition - The basic schoolhouse method of lining up the decimal points and then adding the dig
- Subtraction - By fooling with the sign bits of the numbers, subtraction can be a special consid
- Multiplication - The repeated addition method takes exponential time on large integers, so stay
method is reasonable to program and runs in $O\left(n^{2}\right)$ time to multiply two n-digit
- Division - Repeated subtraction takes exponential time, so the easiest reasonable algorithm to

%---- Page End Break Here ---- Page : 505
In fact, integer division can be reduced to integer multiplication, although in a non-trivial way
- Exponentiation - We can evaluate a^{n} using $n-1$ multiplications, by computing $a \times a

function power(a, n)
if $(n=0)$ *return(1)*
$x=\operatorname{power}(a,\lfloor n/2\rfloor)$
if (n *is even) then return* $\left(x^{2}\right)$
else return $\left(a \times x^{2}\right)$

High- but not arbitrary-precision arithmetic can be conveniently performed using the Chinese rema

Many algorithms on long integers can be directly applied to computations on polynomials. A partic

Implementations: Python plus commercial computer algebra systems like Maple and Mathematica incor best option for a quick, non-embedded application if you have access. The rest of this

%---- Page End Break Here ---- Page : 506
The premier C/C++ library for fast, arbitrary-precision is the GNU Multiple Precision A

The java.math BigInteger class provides arbitrary-precision analogues to all of Java's

A lower-performance, less well-tested but more personal implementation of high-precisio

Several general systems for computational number theory are available. Each of these su

Notes: Knuth \\cite{knuth1997taocp2} is the primary reference on algorithms for all bas

Expositions on the $O\left(n^{1.59}\right)$-time divide-and-conquer algorithm for multi

Good expositions of algorithms for modular arithmetic and the Chinese remainder theorem

Euclid's algorithm for computing the greatest common divisor of two numbers is perhaps

Related problems: Factoring integers (see page 490), cryptography (see page 697).\index
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-508.jpg?height=588

%---- Page End Break Here ---- Page : 507
\section{Knapsack Problem}

Input description: A set of items $S=\{1, \ldots, n\}$, where item i has size s_{i}

Problem description: Find the subset $S^{\prime}$ of S that maximizes the value of $\

Discussion: The knapsack problem arises in resource allocation with financial constrain

The most common formulation is the $0 / 1$ knapsack problem, where each item must eithe

Issues that arise in selecting the best algorithm include:
- Does every item have the same cost, or perhaps the same size? - When all items are wo
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-509.jpg?height=288

%---- Page End Break Here ---- Page : 508
Figure 16.1: Integer partition is a special case of the knapsack problem.
have the same size. Sort by cost, and take the most valuable elements first. These are
- Do all items have the same "price per pound"? - If so, our problem is equivalent to i

An important special case of a constant "price-per-pound" knapsack is the integer parti

```
The constant "price-per-pound" knapsack problem is often called the subset sum problem, because w
- Are all the sizes relatively small integers? - When the sizes of the items and the knapsack cap
The algorithm works as follows: Let $S^{\prime}$ be a set of items, and let $C\left[i, S^{\prime}
$$
C\left[i, S^{\prime} \cup s_{j}\right]=\text { true iff }\left(C\left[i, S^{\prime}\right]=\text
$$
because we either use $s_{j}$ in realizing the sum or we don't. We identify all sums that can be
solution is given by the index of the true element of largest realizable size. To reconstruct thi
This dynamic programming formulation ignores the values of the items. To generalize the algorithm
- What if I have multiple knapsacks? - When there are multiple knapsacks, your problem might be b


%---- Page End Break Here ---- Page : 509
Exact solutions for large capacity knapsacks can be found using integer programming or backtracki

Heuristics must be used when exact solutions prove too costly to compute. The simple greedy heuri

Another heuristic is based on scaling. Dynamic programming works well if the knapsack capacity is

Implementations: Martello and Toth's collection of Fortran implementations of algorithms for a va

David Pisinger maintains a well-organized collection of C-language codes for knapsack problems an

Algorithm 632 \\cite{martello1985program} of the Collected Algorithms of the ACM is a Fortran cod


%---- Page End Break Here ---- Page : 510
Notes: Keller, Pferschy, and Pisinger [\\cite{kellerer2004knapsack}] is the most current referenc

A polynomial-time approximation scheme (PTAS) is an algorithm that approximates the optimal solut

An interesting special variant of the knapsack problem is the 3-sum problem, where we are given t

Vector bin packing is a generalization of knapsack where the knapsack has capacity constraints al

The first algorithm for generalized public key encryption by Merkle and Hellman \\cite{merkle1978

Related problems: Bin packing (see page 652), integer programming (see page 482 .
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-512.jpg?height=584&width=123


%---- Page End Break Here ---- Page : 511
\section{Discrete Fourier Transform}

Input description: A sequence of $n$ real or complex values $h_{i}, 0 \leq i \leq n-1$, sampled a
```

Problem description: The discrete Fourier transform $H_{m}=\sum_{k=0}^{n-1} h_{k} e^{-

Discussion: Although computer scientists tend to be fairly ignorant about Fourier trans
- Filtering - Taking the Fourier transform of a function is equivalent to representing
- Image compression - A smoothed, filtered image contains less information than the or
- Convolution and deconvolution - Fourier transforms can efficiently compute convoluti
n-variable polynomials f and g or comparing two character strings. Implementing
Another example comes from image processing. Because a scanner measures the intensity
- Computing the correlation of functions - The correlation function of two functions $

$$
z(t)=\int_{-\infty}^{\infty} f(\tau) g(t+\tau) d \tau
$$

and can be easily computed using Fourier transforms. When functions $f(t)$ and $g(t)$

%---- Page End Break Here ---- Page : 512

The discrete Fourier transform takes as input n complex numbers $h_{k}, 0 \leq$ $k \

$$
H_{m}=\sum_{k=0}^{n-1} h_{k} \cdot e^{-2 \pi i k m / n}=\sum_{k=0}^{n-1} h_{k}\left[\c
$$

and the inverse Fourier transform is defined by

$$
h_{m}=\frac{1}{n} \sum_{k=0}^{n-1} H_{k} \cdot e^{2 \pi i k m / n}=\frac{1}{n} \sum_{k
$$

which enables us to move easily between h and H.
Since the output of the discrete Fourier transform consists of n numbers, each of wh

The FFT usually assumes that n is a power of two. If this is not the case, you are u

%---- Page End Break Here ---- Page : 513

Many signal-processing systems have strong real-time constraints, so FFTs are often im

Implementations: FFTW is a C subroutine library for computing the discrete Fourier tra

FFTPACK is a package of Fortran subprograms for the fast Fourier transform of periodic

Notes: Bracewell [\\cite{bracewell1999fourier}] and Brigham [\\cite{brigham1988fast}]

A cache-oblivious algorithm for the fast Fourier transform is given in [\\cite{refFLPR

An interesting divide-and-conquer algorithm for polynomial multiplication \\cite{karat

The quantum Fourier transform provides an exponential speedup over the classical FFT,

It is an open question of whether complex variables are really fundamental to fast alg

In recent years, wavelets have been proposed to replace Fourier transforms in filtering. See \\ci

Related problems: Data compression (see page 693), high-precision arithmetic (see page 493).

%---- Page End Break Here ---- Page : 514
Chapter 17

\chapter{Combinatorial Problems}

We will now consider several algorithmic problems of a purely combinatorial nature. These include

The rest of this section deals with combinatorial objects, such as permutations, partitions, subs

I conclude with the problem of generating graphs. Graph algorithms are more fully presented in su

Books on general combinatorial algorithms, in this restricted sense, include:
- Knuth - The standard reference on sorting and searching \\cite{knuth1998taocp3}. New material c
- Ruskey \\cite{ruskey2003combinatorial} - He never officially completed it, but this manuscript
- Kreher and Stinson \\cite{kreher1999combinatorial} - This book on combinatorial generation algo
- Pemmaraju and Skiena \\cite{pemmaraju2003computational} - This description of Combinatorica, a
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-516.jpg?height=473&width=123

\section{Sorting}

Input description: A set of n items.
Problem description: Arrange the items in increasing order.
Discussion: Sorting is the most fundamental algorithmic problem in computer science. Learning the

Sorting also illustrates most of the standard paradigms of algorithm design. Programmers are gene
- How many keys will you be sorting? - For small amounts of data (say $n \leq 100$), it doesn't
But if you have more than one hundred items to sort, it becomes important to use an $O(n \lg n)$-
Once you get past (say) 100,000,000 items, it is important to start thinking about external-memor
- Will there be duplicate keys? - The sorted order is completely defined if all items have distin
matters. Ties are often broken by sorting on a secondary key, like the first name or initial wher

%---- Page End Break Here ---- Page : 516
Ties sometimes get broken by their initial position in the data set. Suppose the 5 th and 27 th i
- What do you know about your data? - Perhaps you can exploit special knowledge about your data t
- Has the data already been partially sorted? If so, certain algorithms like insertion sort perfo
- Do you know the distribution of the keys? If the keys are randomly or uniformly distributed, a
- Are your keys very long or hard to compare? When sorting long text strings, it might pay to use
Another idea might be to use radix sort. This always takes time linear in the number of character
- Is the range of possible keys very small? If you want to sort a subset of (say) $n / 2$ distinc

- Should I worry about disk accesses? - In massive sorting problems, it is not possibl
describes a variety of intricate algorithms for efficiently merging data from differen
The simplest approach to external sorting loads the data into a B-tree, and then does

%---- Page End Break Here ---- Page : 517
The best general-purpose internal sorting algorithm is quicksort (see Section 4.2 (pag
- Use randomization - By randomly permuting (see Section 17.4 (page 517)) the keys bef
- Median of three - For your pivot element, use the median of the first, last, and mid
- Leave small subarrays for insertion sort - Terminating the quicksort recursion and s
- Do the smaller partition first - You can minimize run-time memory by processing the

Before you get started, see Bentley's article on building a faster quicksort \\cite{be

Implementations: The best freely available sort program is presumably GNU sort, part o

%---- Page End Break Here ---- Page : 518
Modern programming languages provide libraries so you should never need to implement y

High-performance sorting systems distribute the work over many machines. Map-Reduce sy

Numerous websites provide animations of all the basic sorting algorithms, many quite i

Notes: Knuth \\cite{knuth1998taocp3} is the best book that will ever be written on sor

Expositions on the basic internal sorting algorithms appear in every algorithms textbo

The primary competitive forum for high-performance sorting is an annual competition in

Sorting has a well-known $\Omega(n \lg n)$ lower bound under the algebraic decision tr

This lower-bound does not hold under different models of computation. Fredman and Will

Related problems: Dictionaries (see page 440), searching (see page 510, topological so
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-520.jpg?height=58

%---- Page End Break Here ---- Page : 519
Searching

Input description: A set of n keys S, and a query key q.
Problem description: Where is q in S ?
Discussion: "Searching" is a word that means different things to different people. Sea

But here we consider the task of searching for a key in a list, array, or tree. Dictio

I treat searching as a problem distinct from dictionaries because simpler and more efficient solu

The two basic approaches we consider are sequential search and binary search. Both are simple, ye
big win over the $n / 2$ comparisons we expect with sequential search. See Section 5.1 (page 148

%---- Page End Break Here ---- Page : 520
A sequential search is the simplest algorithm, and likely to be fastest on up to about twenty ele
- How much time can you spend programming? - A binary search is a notoriously tricky algorithm to
- Are certain items accessed more often than other ones? - Certain English words (such as "the")

Knowledge of access frequencies is easy to exploit with sequential search. But the issue is more
- Might access frequencies change over time? - Reordering a list or tree to exploit a skewed acce
accesses to any given key are likely to occur in clusters. A hot key will be maintained near the

%---- Page End Break Here ---- Page : 521
Self-organization can extend the useful size range of sequential search, although you should swit
- Is the key close by? - Suppose we know that the target key is to the right of position p, and

Such a one-sided binary search finds the target at position $p+l$ using at most $2\lceil\lg l\rce
- Is my data structure sitting on external memory? - Once the number of keys grows too large, bin
- Can I guess where the key should be? - In interpolation search, we exploit our understanding of

Although an interpolation search is an appealing idea, I caution against it for three reasons: Fi

%---- Page End Break Here ---- Page : 522
Implementations: The basic sequential and binary search algorithms are simple enough that you sho

Many data structure textbooks provide extensive and illustrative implementations. Sedgewick (http

Notes: The Handbook of Data Structures and Applications \\cite{mehta2018handbook} provides up-to-

The next position probed in linear interpolation search on an array of sorted numbers is given by
$$
\text { next }=(\text { low }-1)+\left\lceil\frac{q-S[\text { low }-1]}{S[\text { high }+1]-S[\te
$$
where q is the query numerical key and S the sorted numerical array. If the keys are drawn in

Non-uniform access patterns can be exploited in binary search trees by structuring them so that p

The van Emde Boas layout of a binary tree (or sorted array) offers better external memory perform

Grover's algorithm permits searching in an unsorted database in $O(\sqrt{n})$ time on a quantum c

Related problems: Dictionaries (see page 440), sorting (see page 506).
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-524.jpg?height=469

%---- Page End Break Here ---- Page : 523
\section{Median and Selection}

Input description: A set of n numbers or keys, and an integer k.
Problem description: Find the key greater than or equal to exactly k of the n keys

Discussion: Median finding is an essential problem in statistics, where it provides a r

Median finding is a special case of the more general selection problem, which seeks the
- Filtering outlying elements - When dealing with noisy data, it is often a good idea t
- Identifying the most promising candidates - A computer chess program might quickly as
- Deciles and related divisions - A useful way to present income distribution in a popu
- Order statistics - Particularly interesting special cases of selection include findin

The mean of n numbers can be computed in linear time by summing the elements and divi
- How fast does it have to be? - The most elementary median-finding algorithm sorts the

%---- Page End Break Here ---- Page : 524
In particular, quick-select is an $O(n)$ expected-time algorithm based on quicksort. Se

More complicated algorithms can find the median in worst-case linear time. However, thi
- What if you only get to see each element once? - Selection and median finding become
One solution to such a problem is random sampling. Flip a coin for each value to decide
- How fast can you find the mode? - Beyond mean and median lies a third notion of avera
the length of the longest run and hence compute the mode in a total of $O(n \log n)$ ti
The mode can also be found in expected linear time using hashing, but no such worst-cas

%---- Page End Break Here ---- Page : 525\index{LempelZiv algorithms!525}
Implementations: The C++ Standard Template Library (STL) provides a general selection m

Notes: The linear expected-time algorithm for median and selection is due to Hoare \\ci

Streaming algorithms have extensive applications to large data sets, and are well surve

A sport of considerable theoretical interest is determining exactly how many comparison

Tight combinatorial bounds for selection problems are presented in Aigner \\cite{aigner

Related problems: Priority queues (see page 445), sorting (see page 506).

```
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-527.jpg?height=583&width=108


%---- Page End Break Here ---- Page : 526
\section{Generating Permutations}

Input description: An integer $n$.
Problem description: Generate (1) all, or (2) a random, or (3) the next permutation of length $n$

Discussion: A permutation describes an arrangement or ordering of items. Many algorithmic problem

There are $n$ ! permutations of $n$ items. This grows so quickly that you can't generate all perm

Fundamental to any permutation-generation algorithm is the notion of order, the sequence in which

There are two different paradigms for constructing permutations: ranking/unranking and incrementa
but ranking and unranking can be applied to solve a much wider class of problems. The key is to d
- $\operatorname{Rank}(p)$ - What is the position of $p=\left\{p_{1}, \ldots, p_{n}\right\}$ in t
$$
\operatorname{Rank}(p)=\left(p_{1}-1\right) \cdot(|p|-1)!+\operatorname{Rank}\left(p_{2}, \ldots,
$$


%---- Page End Break Here ---- Page : 527
We assume that any permutation $p$ is an arrangement of distinct integers 1 trough $|p|$, so gett
$$
\operatorname{Rank}(\{2,1,3\})=(2-1) \cdot 2!+\operatorname{Rank}(\{1,2\})=2+(1-1) \cdot 1!+\oper
$$
- Unrank $(m, n)$ - Which permutation is in position $m$ of the $n$ ! permutations of $n$ items?

What the rank and unrank functions actually do does not matter so long as they are inverses of ea
- Sequencing permutations - To determine the next permutation that occurs in order after $p$, we
- Generating random permutations - Selecting a random integer from 0 to $n!-1$ and then unranking
- Keep track of a set of permutations - Suppose we are generating random permutations, but want t


%---- Page End Break Here ---- Page : 528
This rank/unrank method is best suited for small values of $n$, since $n$ ! quickly exceeds the c

Incremental change algorithms for sequencing permutations are elegant but tricky, generally so co

Throughout this discussion, we have assumed that the items we are permuting are all distinguishab

Generating random permutations is an important little problem that people often stumble across, a
$$
\begin{aligned}
```

```
& \text { for } i=1 \text { to } n \text { do } a[i]=i \\
& \text { for } i=1 \text { to } n-1 \text { do } \operatorname{swap}[a[i], a[\operato
\end{aligned}
$$

That this algorithm generates all permutations uniformly at random is not obvious. If y
$$
\begin{aligned}
& \text { for } i=1 \text { to } n \text { do } a[i]=i \\
& \text { for } i=1 \text { to } n-1 \text { do } \operatorname{swap}[a[i], a[\operato
\end{aligned}
$$

Such subtleties demonstrate why you must be very careful with random generation algorit

Implementations: The C++ Standard Template Library (STL) provides two functions (next_p
of combinatorial objects, including permutations and cyclic permutations, are available


%---- Page End Break Here ---- Page : 529

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{images/algorithms_page_0531_0.png}
\end{figure}
The Combinatorial Object Server (http://combos.org/) developed by Frank Ruskey of the U

Nijenhuis and Wilf \\cite{nijenhuis1978combinatorial} is a venerable but still valuable

Combinatorica [\\cite{pemmaraju2003computational}] provides Mathematica implementations

Notes: The best recent reference on permutation generation is Knuth \\cite{knuth2011tac

Fast permutation generation methods make only a single swap between successive permutat

Markov chain generation methods construct random objects through random transitions, li

In the days before ready access to computers, books with tables of random permutations

Related problems: Random-number generation (see page 486, generating subsets (see page
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-531.jpg?height=453


%---- Page End Break Here ---- Page : 530
Output
```

\section{Generating Subsets}

Input description: An integer n.
Problem description: Generate (1) all, or (2) a random, or (3) the next subset of the integers $\

Discussion: A subset describes a selection of objects, where the order among them does not matter

There are 2^{n} distinct subsets of an n-element set, including the empty set as well as the

By definition, the relative order among the elements does not distinguish different subsets. Thus

As with permutations (see Section 17.4 (page 517), the key to subset generation problems is estab
- Lexicographic order - This means sorted order, and is often the most natural way to generate co
- Gray code - A particularly interesting subset sequence is minimum change order, wherein adjacen

%---- Page End Break Here ---- Page : 531\index{generating permutations!531}
Generating subsets in Gray code order can be very fast, because there is a nice recursive constru
Since only one element changes between subsets, exhaustive search algorithms built on Gray codes
- Binary counting - The simplest approach to subset-generation problems is based on the observati
This binary representation is the key to solving all subset generation problems. To generate all
To generate a random subset, you could try to generate a random integer from 0 to $2^{n}-1$ and u

Generation problems for two closely related problems arise often in practice:\index{arbitrary-pre
- K-subsets - Instead of constructing all subsets, we may only be interested in the subsets conta
The best way to construct all k-subsets is in lexicographic order. The ranking function is base
whose smallest element is f. Using this, we can determine the smallest element in the m th $k
- Strings - Generating all subsets is equivalent to generating all 2^{n} strings of true and fa

%---- Page End Break Here ---- Page : 532
Implementations: $\mathrm{C}++$ routines for generating an astonishing variety of combinatorial o

The Combinatorial Object Server (http://combos.org/) developed by Frank Ruskey of the University

Nijenhuis and Wilf \\cite{nijenhuis1978combinatorial} is an excellent reference on generating com

Combinatorica \\cite{pemmaraju2003computational} provides Mathematica implementations of algorith

Notes: The best reference on subset generation is Knuth \\cite{knuth2011taocp4a}. Good exposition

Gray codes were first developed \\cite{gray1953pulse} to transmit digital information in a robust

The popular puzzle Spinout ${ }^{\circledR}$, manufactured by ThinkFun (formerly Binary Arts Corp

Related problems: Generating permutations (see page 517), generating partitions (see page 524).

```
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-534.jpg?height=58


%---- Page End Break Here ---- Page : 533
\section{Generating Partitions}

Input description: An integer $n$.
Problem description: Generate (1) all, or (2) a random, or (3) the next integer or set
Discussion: Two different types of combinatorial objects are denoted by the word "part
- Integer partitions are multisets of non-zero integers that add up exactly to $n$. Fo
- Set partitions divide the elements $\{1, \ldots, n\}$ into non-empty subsets. There 

Although the number of integer partitions grows exponentially with $n$, they do so at 
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-535.jpg?height=49


%---- Page End Break Here ---- Page : 534
Figure 17.1: The Ferrers diagram of a random partition of $n=1,000$.

The easiest way to generate integer partitions constructs them in lexicographically de

This algorithm is sufficiently intricate to program that you should use one of the imp

Generating integer partitions uniformly at random is a trickier business than generati
$$
P_{n, k}=P_{n-k, k}+P_{n, k-1}
$$
because any such partition either contains a part of size $k$ or it doesn't. The two b

Random partitions tend to have large numbers of fairly small parts, best visualized by


%---- Page End Break Here ---- Page : 535
One application arises in evaluating the publication record of academic researchers. R

Set partitions can be generated using techniques akin to integer partitions. Each set 

Since there is a one-to-one equivalence between set partitions and restricted growth f

We can use a similar counting strategy to generate random set partitions as we did wit
$$
\left\{\begin{array}{l}
n \\
k
\end{array}\right\}=\left\{\begin{array}{c}
n-1 \\
```

```
k-1
\end{array}\right\}+k\left\{\begin{array}{c}
n-1 \\
k
\end{array}\right\}
$$
with the boundary conditions $\left\{\begin{array}{l}n \\ n\end{array}

Implementations: $\mathrm{C}++$ routines for generating an astonishing variety of combinatorial c

The Combinatorial Object Server (http://combos.org/) developed by Frank Ruskey of the University

Nijenhuis and Wilf \\cite{nijenhuis1978combinatorial} remains a valuable resource on generating c

Combinatorica \\cite{pemmaraju2003computational} provides Mathematica implementations of algorith
and Young tableaux, as well as to count and manipulate these objects. See Section 22.1.8 (page 71
Notes: The best reference on algorithms for generating both integer and set partitions is Knuth \


%---- Page End Break Here ---- Page : 536
Integer and set partitions are both special cases of multiset partitions, or set partitions of no

The (long) history of combinatorial object generation is detailed by Knuth \\cite{knuth2011taocp4

The 2015 film The Man Who Knew Infinity, about the life of the great Indian mathematician Ramanuj

Two related combinatorial objects are Young tableaux and integer compositions, although they are

Young tableaux are two-dimensional configurations of integers $\{1, \ldots, n\}$ where the number

Compositions represent the possible assignments of $n$ indistinguishable balls to $k$ distinguish

Related problems: Generating permutations (see page 517), generating subsets (see page 521).\inde
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-538.jpg?height=590&width=124


%---- Page End Break Here ---- Page : 537
\section{Generating Graphs}

Input description: Parameters describing the desired graph, including the number of vertices $n$,

Problem description: Generate (1) all, or (2) a random, or (3) the next graph satisfying the para

Discussion: Graph generation typically arises when constructing test data for programs. Perhaps y

Many factors complicate the problem of generating graphs. First, make sure you know exactly what
```

- Do I want labeled or unlabeled graphs? - The issue here is whether the names of the v
- Do I want directed or undirected graphs? - Most natural generation algorithms generat

%---- Page End Break Here ---- Page : 538
You also must define what you mean by random. There are three primary models of random
- Random edge generation - The Erds-Rényi model is parameterized by a given edge probab
- Random edge selection - The second model is parameterized by the desired number of ed
- Preferential attachment - Under a rich-get-richer model, newly created edges are more

Which of these options best models your application? Probably none of them. By definiti

An alternative to random graphs is "organic" graphs-graphs that reflect the relationshi

Two classes of graphs have particularly interesting generation algorithms:

\footnotetext{
${ }^{1}$ Please link to us from your homepage to correct this travesty.
}
- Trees - Prüfer codes provide a simple way to rank and unrank labeled trees and thus s
The key to Prüfer's bijection is the observation that every tree has at least two verti
Algorithms for efficiently generating unlabeled rooted trees are discussed in the Imple
- Fixed degree sequence graphs - The degree sequence of a graph G is an integer parti
Not all partitions correspond to degree sequences of graphs. However, there is a recurs

%---- Page End Break Here ---- Page : 539
Although this construction is deterministic, a semi-random collection of graphs realizi

Implementations: The Stanford GraphBase \\cite{knuth1994graphbase} is perhaps most usef

Resources for real-world networks include the Network Data Repository (http://networkre

%---- Page End Break Here ---- Page : 540
Combinatorica \\cite{pemmaraju2003computational} provides Mathematica generators for su

The Combinatorial Object Server (http://combos.org/) developed by Frank Ruskey of the U

The graph isomorphism testing program Nauty (see Section 19.9 (page 610p) includes a su

Nijenhuis and Wilf \\cite{nijenhuis1978combinatorial} provide efficient Fortran routine

Notes: Extensive literature exists on generating graphs uniformly at random. Surveys in

Knuth \\cite{knuth2011taocp4a} is the best recent reference on generating trees. The bi

Random graph theory is concerned with the properties of random graphs. Threshold laws in random g

The preferential attachment model of graphical evolution has emerged relatively recently in the s

An integer partition is graphic if there exists a simple graph with that degree sequence. Erds ar
$$
\sum_{i=1}^{r} d_{i} \leq r(r-1)+\sum_{i=r+1}^{n} \min \left(r, d_{i}\right)
$$

Related problems: Generating permutations (see page 517), graph isomorphism (see page 610).
\begin{tabular}{c|l}
December 21, 2012? & \begin{tabular}{l}
5773 Teveth 8 (Hebrew) \\
%---- Page End Break Here ---- Page : 541

(Gregorian) \\
1434 Safar 7 (Islamic) \\
1934 Agrahayana 30 (Indian Civil) \\
13.0.0.0 (Mayan Long Count)
\end{tabular} \\
\hline Input & Output
\end{tabular}

\section{Calendrical Calculations}

Input description: A particular calendar date d : month, day, and year.
Problem description: Which day of the week did d fall on according to the given calendar system

Discussion: Business applications often need to perform calendrical calculations. Perhaps we want

More complicated questions arise in international applications, because different nations and eth

Calendrical calculations differ from other problems in this book because calendars are historical

The basic approach underlying calendar systems is to start from a particular reference date (call

That the solar year is not an integer number of days long is the major source of complications in
four years leaves an extra 10 minutes and 48 seconds unaccounted for every year.

%---- Page End Break Here ---- Page : 542
The original Julian calendar (from Julius Caesar) ignored these extra minutes, which had accumula

The rules for most calendrical systems are sufficiently complicated and pointless that you should

There are a variety of "impress your friends" algorithms that enable you to compute in

Implementations: Readily available calendar libraries exist in both C++ and Java. The

Dershowitz and Reingold provide a uniform algorithmic presentation \\cite{reingold2018

C and Java implementations of international calendars of unknown reliability are readi

Notes: A comprehensive discussion of calendrical computation algorithms appear in the

Related problems: Arbitrary-precision arithmetic (see page 493), generating permutatio
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-544.jpg?height=59

%---- Page End Break Here ---- Page : 543
\section{Job Scheduling}

Input description: A directed acyclic graph $G=(V, E)$, with vertices representing job

Problem description: Which schedule of tasks completes the job using the minimum amoun

Discussion: Devising the best schedule to satisfy a set of constraints is fundamental

Scheduling problems differ widely in the nature of the constraints that must be satisf
- Topological sort constructs a schedule consistent with precedence constraints in a D
- Bipartite matching assigns jobs to workers with appropriate skills for them. See Sec
- Vertex/edge coloring assigns jobs to time slots such that no two interfering jobs ar
- Traveling salesman identifies the most efficient delivery route to visit a given set
- Eulerian cycle defines the most efficient route for a snowplow or mailman to travers

%---- Page End Break Here ---- Page : 544
Here the focus is on precedence-constrained scheduling problems for directed acyclic g

Several problems on these task networks are of interest:
- Critical path - The longest path from the start vertex to the completion vertex defi
- Minimum completion time - What is the fastest we can get this job completed while re
The minimum completion time for a DAG can be computed in $O(n+m)$ time, because it is
- What is the tradeoff between number of workers and completion time? What we really a

Real scheduling applications often present constraints that may be difficult or imposs

Another fundamental scheduling problem takes a set of jobs without precedence constrai
as bin packing (see Section 20.9 (page 652). Each job is associated with the time it w

%---- Page End Break Here ---- Page : 545
More sophisticated variations of job-shop scheduling provide each task with allowable start and r

Implementations: JOBSHOP is a collection of C programs for job-shop scheduling created for a comp

UniTime (https://www. unitime. org/is a comprehensive educational scheduling system that supports

LEKIN is a flexible job-shop scheduling system designed for educational use \\cite{pinedo2016sche

For commercial scheduling applications, ILOG CP has been reflective of the state-of-the-art for c

Notes: The literature on scheduling algorithms is vast. Brucker \\cite{brucker2007scheduling} and

A well-defined taxonomy covers thousands of job-shop scheduling variants, which classifies each p

Gantt charts provide visual representations of job-shop scheduling solutions, where the x-axis

Timetabling is a term often used in discussion of classroom and related scheduling problems. PATA

Related problems: Topological sorting (see page 546), matching (see page 562), vertex coloring (s
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-547.jpg?height=602&width=123

%---- Page End Break Here ---- Page : 546
\section{Satisfiability}

Input description: A set of clauses in conjunctive normal form.
Problem description: Is there a truth assignment to the Boolean variables such that every clause

Discussion: Satisfiability (SAT) arises whenever we seek a configuration or object that must be c

Satisfiability is the original NP-complete problem. Despite its applications to constraint satisf

Issues in satisfiability testing include:
- Is your formula the "AND of ORs" or the "OR of ANDs"? - In satisfiability, the constraints are
literals (terms of the form v_{i} or $\overline{v_{i}}$, the latter denoting "not v_{i} "), I
$$
\left(v_{1} \text { or } \overline{v_{2}}\right) \text { and }\left(v_{2} \text { or } v_{3}\righ
$$

%---- Page End Break Here ---- Page : 547
With DNF formulas, it suffices to satisfy any single clause, because they will be or-ed together.
($\overline{v_{1}}$ and $\overline{v_{2}}$ and v_{3}) or ($\overline{v_{1}}$ and v_{2} and
$\left(\overline{v_{1}}\right.$ and v_{2} and $\left.v_{3}\right)$ or (v_{1} and $\overline{
Solving DNF-satisfiability is trivial, because every DNF formula can be satisfied unless all clau

- How big are your clauses? - k-SAT is a special case of satisfiability, where each c
- Does it suffice to satisfy most of the clauses? - If you are determined to solve a SA
Still, we might benefit by relaxing the problem so that the goal becomes satisfying as

When faced with a problem of unknown complexity, proving it NP-complete can be an impor
first principles, using the basic problems of 3-SAT, vertex cover, independent set, int

%---- Page End Break Here ---- Page : 548
Implementations: Recent years have seen tremendous progress in the performance of satis

SAT Live! (http://www.satlive.org/) is the most up-to-date source for papers, programs

Notes: The most comprehensive overview of satisfiability testing in practice is Kautz,

The first part of the second part of Knuth's Volume 4 is on satisfiability algorithms,

An algorithm for solving 3-SAT in worst-case $O^{*}\left(1.4802^{n}\right)$ appears in

The primary reference on NP-completeness is \\cite{garey1979computers}, featuring a lis

A linear-time algorithm for 2-SAT appears in \\cite{aspvall1979linear}. See \\cite{wagn

Related problems: Constrained optimization (see page 478), traveling salesman problem

%---- Page End Break Here ---- Page : 549
Chapter 18

\chapter{Graph Problems: Polynomial Time}

Algorithmic graph problems constitute roughly one third of the material in this catalog

In this section, we will deal with problems that have efficient algorithms to solve the

Graphs are often best understood as drawings. Many interesting graph properties follow

Many advanced graph algorithms are difficult to program, but good implementations are a

See Atallah \\cite{atallah2017algorithms}, Thulasiraman \\cite{thulasiraman2016handbook
- Sedgewick \\cite{sedgewick2011algorithms} - The graph algorithms volume of this algor
- Ahuja, Magnanti, and Orlin \\cite{ahuja1993network} - While purporting to be a book o
- Even \\cite{even2011graph} - A respected advanced text on graph algorithms, with a pa
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-551.jpg?height=593

\section{Connected Components}

Input description: A directed or undirected graph G.
Problem description: Identify the different pieces or components of G, where vertices x and $

Discussion: The connected components of a graph represent in a gross sense the separate pieces of

Finding connected components is at the heart of many important graph applications. For example, c

Testing whether a graph is connected is an essential preprocessing step for every graph algorithm

Testing the connectivity of any undirected graph is a job for either depthfirst or breadth-first

Other notions of connectivity also arise in practice:
- What if my graph is directed? - There are two distinct notions of connected components for dire
Weakly and strongly connected components define unique partitions of the vertices. The output fig
Testing whether a directed graph is weakly connected can be done easily in linear time. Simply tu

%---- Page End Break Here ---- Page : 551
All the strongly connected components of G can be extracted in linear time using more sophistic
1. Perform a DFS, starting from an arbitrary vertex in G, and labeling each vertex in order of
2. Reverse the direction of each edge in G, yielding $G^{\prime}$.
3. Perform a DFS of $G^{\prime}$, starting from the highest numbered vertex in G. If this searc
4. Each DFS tree created in Step 3 defines a strongly connected component.

My implementation of this two-pass algorithm appears in Section 7.10.2 (page 232). In either case
- What is the weakest point in my graph/network? - A chain is only as strong as its weakest link.
chain to become disconnected. The connectivity of graphs measures the strength of a graph-how man
Algorithmic connectivity problems are discussed in Section 18.8 (page 568. In particular, biconne
- Is the graph a tree? How can I find a cycle if one exists? - The problem of cycle identificatio

%---- Page End Break Here ---- Page : 552
For undirected graphs, the analogous problem is tree identification. A tree is, by definition, an

Depth-first search can be used to find cycles in both directed and undirected graphs. Whenever we

Implementations: The graph data structure implementations of Section 15.4 (page 452 all include i

With respect to Java, JUNG (http://jung.sourceforge.net/) also provides biconnected component alg

My (biased) preference for C language implementations of all basic graph connectivity algorithms,

%---- Page End Break Here ---- Page : 553
Notes: Depth-first search was first used to find paths out of mazes, and dates back to the ninete

Hopcroft and Tarjan \\cite{hopcroft1973efficient} \\cite{tarjan1972-depth-first} establ

The first linear-time algorithm for strongly connected components is due to Tarjan \\c

DFS is hard to parallelize. Parallel algorithms for connected components on MapReduce a

Random graphs exhibit interesting connectivity properties, such that once the number o

Related problems: Edge-vertex connectivity (see page 568), shortest path (see page 554
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-555.jpg?height=58

%---- Page End Break Here ---- Page : 554
\section{Topological Sorting}

Input description: A directed acyclic graph $G=(V, E)$, also known as a partial order

Problem description: Find a linear ordering of the vertices of V such that for each

Discussion: Topological sort arises as a subproblem in most algorithms on directed acy

Topological sort can be used to schedule jobs under precedence constraints. Suppose we

Three important facts about topological sorting are:
1. Only DAGs can be topologically sorted, because any directed cycle provides an inhere
2. Every DAG can be topologically sorted, so there always is at least one schedule for
3. DAGs can usually be topologically sorted in many different ways, especially when th

The conceptually simplest linear-time algorithm for topological sorting performs a dep
in any DAG. Source vertices can appear at the front of any schedule without violating

%---- Page End Break Here ---- Page : 555
An alternate algorithm makes use of the observation that ordering the vertices in term

Two special considerations with respect to topological sorting are:
- What if I need all the linear extensions, instead of just one of them? Sometimes we
Algorithms for listing all linear extensions in a DAG are based on backtracking. They
Algorithms to construct random linear extensions start from an arbitrary linear extens
- What if your graph is not acyclic? - When a set of constraints contains inherent con
Because the DFS-based topological sorting algorithm gets stuck as soon as it identifie

Implementations: Essentially all the graph data structure implementations of Section 1
doc) and LEDA (see Section 22.1.1 (page 713) for C++. For Java, check out JGraphT (htt

```
%---- Page End Break Here ---- Page : 556
The Combinatorial Object Server (http://combos.org/) provides C language programs to generate lin

My (biased) preference for C language implementations of all basic graph algorithms, including to
Notes: Good expositions on topological sorting include \\cite{cormen2009introduction} \\cite{manb

Pruesse and Ruskey \\cite{pruesse1994generating} give an algorithm that generates linear extensio

Huber \\cite{huber2006fast} gives an algorithm to sample linear extensions uniformly at random fr

Related problems: Sorting (see page506), feedback edge/vertex set (see page 618.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-558.jpg?height=592&width=124


%---- Page End Break Here ---- Page : 557
\section{Minimum Spanning Tree}

Input description: A graph $G=(V, E)$ with weighted edges.
Problem description: Find a subset of edges $E^{\prime} \subset E$ that define a tree of minimum

Discussion: The minimum spanning tree (MST) of a graph defines the cheapest subset of edges that

Minimum spanning trees prove important for several reasons:
- They can be computed quickly and easily, and create a sparse subgraph that reflects a lot about
- They provide a way to identify clusters in sets of points. Deleting all long edges from a minim
- They can be used to give approximate solutions to hard problems such as Steiner tree and travel
- As an educational tool, MST algorithms provide graphic evidence that greedy algorithms sometime
Three classical algorithms efficiently construct minimum spanning trees. Detailed implementations
- Kruskal's algorithm - Every vertex starts as a separate tree. These trees are merged together b


%---- Page End Break Here ---- Page : 558
Kruskal $(G)$
Sort the edges in order of increasing weight
count $=0$
while (count $<n-1$ ) do
get next edge $(v, w)$
if (component $(v) \neq \operatorname{component}(w))$ add to $T$
component $(v)=\operatorname{component}(w)$
count++

The "component $(x)$ ?" test can be efficiently implemented using the unionfind data structure (S
- Prim's algorithm - Starts from an arbitrary vertex $v$ to "grow" a tree, repeatedly finding the
$\operatorname{Prim}(G)$
Select an arbitrary vertex to start
```

While (there are fringe vertices)
select minimum-weight edge between tree and fringe add the selected edge and vertex to

This creates a spanning tree for any connected graph, since no cycle can be introduced
- Boruvka's algorithm - This rests on the observation that the lowest-weight edge incic

%---- Page End Break Here ---- Page : 559
Boruvka (G)
Initialize spanning forest F to n single-vertex trees
While (F has more than one tree)
for each T in F, find the smallest edge from T to $G-T$ add all selected edges to

The number of trees are at least halved in each round, so we get the MST after at most

MST is only one of several spanning tree problems that arise in practice. The following
- Are all edges of your graph of identical weight? - Every spanning tree on n points
- Should I use Prim's or Kruskal's algorithm? - As implemented in Section 8.1 (page 24
- What if my input consists of points in the plane, instead of a graph? Geometric insta
- How can I find a spanning tree that avoids vertices of high degree? Another common go

Implementations: All the graph data structure implementations of Section 15.4 (page 45

%---- Page End Break Here ---- Page : 560
Timing experiments on minimum spanning tree algorithms produce contradicting results, s

Combinatorica \\cite{pemmaraju2003computational} provides Mathematica implementations o

My (biased) preference for C language implementations of all basic graph algorithms, in
Notes: The minimum spanning tree problem dates back to Boruvka's algorithm in 1926, wel

The fastest implementations of Prim's and Kruskal's algorithms use Fibonacci heaps \\ci

A simple combination of Boruvka's algorithm with Prim's algorithm yields an $O(m \lg \l

The best theoretical bounds on finding minimum spanning trees tell a complicated story

A spanner $S(G)$ of a given graph G is a subgraph that offers an effective compromise

%---- Page End Break Here ---- Page : 561
The $O(n \log n)$ algorithm for Euclidean MSTs is due to Shamos, and discussed in compu

Fürer and Raghavachari \\cite{furer1994approximating} give an algorithm that constructs

Minimum spanning tree algorithms have an interpretation in terms of matroids, which are systems c

Dynamic graph algorithms seek to maintain a graph invariant (such as the MST) efficiently under e

Algorithms for generating spanning trees in order from minimum to maximum weight are presented ir

Related problems: Steiner tree (see page 614), traveling salesman (see page 594.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-563.jpg?height=593&width=123

%---- Page End Break Here ---- Page : 562
\section{Shortest Path}

Input description: An edge-weighted graph G, with vertices s and t.
Problem description: Find the shortest path from s to t in G.
Discussion: The problem of finding shortest paths in a graph has many applications, some quite su
- Shortest paths often arise in transportation or communications problems, such as finding the be
- Segmentation is the task of partitioning a digitized image into regions containing distinct obj
- Recall the dialing for documents war story of Section 8.4 (page 264, where we sought to use som
- In an informative drawing of a graph, the "center" of the graph should appear near the center c

%---- Page End Break Here ---- Page : 563
The primary algorithm for finding shortest paths is Dijkstra's algorithm, which efficiently compu
$$
\operatorname{dist}(x, v)=\min _{u \in S}(\operatorname{dist}(x, u)+\operatorname{weight}(u, v))
$$

This edge (u, v) gets added to a shortest path tree, whose root is x and describes all the sh

An $O\left(n^{2}\right)$ implementation of Dijkstra's algorithm appears in Section 8.3.1 (page 25

Dijkstra's algorithm is the right choice to compute single-source shortest path on positively wei
- Is your graph weighted or unweighted? - If your graph is unweighted, a simple breadth-first sea
- Does your graph have negative cost weights? - Dijkstra's algorithm assumes that all edges have
Note that adding a fixed amount of weight to make each edge positive does not solve the problem.
- Is your input a set of geometric obstacles, instead of a graph? - Many applications seek the sh
algorithm. Vertices correspond to the boundary vertices of the obstacles, with edges defined betw
There are more efficient geometric algorithms that compute the shortest path directly from the ar
- Is your graph acyclic - that is, a DAG? - Shortest paths in directed acyclic graphs can be four
$$
\operatorname{dist}(s, j)=\min _{(i, j) \in E}(\operatorname{dist}(s, i)+\operatorname{weight}(i,
$$
since we already know the shortest path $\operatorname{dist}(s, i)$ for all vertices i to the l
- Do you need the shortest path between all pairs of points? - The naive approach to calculate th
$$

```
\begin{aligned}
& D^{0}=M \\
%---- Page End Break Here ---- Page : 564

& \text { for } k=1 \text { to } n \text { do } \\
& \quad \text { for } i=1 \text { to } n \text { do } \\
& \quad \text { for } j=1 \text { to } n \text { do } \\
& \quad D_{i j}^{k}=\min \left(D_{i j}^{k-1}, D_{i k}^{k-1}+D_{k j}^{k-1}\right)
\end{aligned}
$$

The key to understanding Floyd's algorithm is that $D_{i j}^{k}$ denotes "the length o
- How do I find the shortest cycle in a graph? - One application of allpairs shortest p

This might be what you want. But the shortest cycle through $x$ is likely to go from $x
The problem of finding the longest cycle in a graph includes Hamiltonian cycle as a spe


%---- Page End Break Here ---- Page : 565
The all-pairs shortest path matrix can be used to compute several useful invariants rel

Implementations: The highest performance shortest path codes are due to Andrew Goldberg

All the C++ and Java graph libraries discussed in Section 15.4 (page 452 ) include at l

Shortest-path algorithms was the subject of the 9th DIMACS Implementation Challenge, ho

Notes: Good expositions on Dijkstra's algorithm \\cite{dijkstra1959note}, the Bellman-F

The fastest algorithm known for single-source shortest-path is Dijkstra's algorithm wit


%---- Page End Break Here ---- Page : 566
Online services like Google Maps quickly find at least an approximate shortest path bet

The $A^{*}$-algorithm performs a best-first search for the shortest path coupled with a

Many applications demand multiple short alternative paths, in addition to the optimal p

Fast algorithms for computing the girth are known for both general [\\cite{itai1978find

Related problems: Network flow (see page 571), motion planning (see page 667).
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-568.jpg?height=586


%---- Page End Break Here ---- Page : 567
```

\section{Transitive Closure and Reduction}

Input description: A directed graph $G=(V, E)$.
Problem description: For transitive closure, construct a graph $G^{\prime}$ with edge $(i, j) \in

Discussion: Transitive closure can be thought of as establishing a data structure to efficiently

Transitive closure arises in propagating the consequences of modified attributes of a graph G.

There are three basic algorithms for computing transitive closure:
- The simple algorithm performs a breadth-first or depth-first search from each vertex, and keeps
- Warshall's algorithm constructs the transitive closure in $O\left(n^{3}\right)$ time, using a s
resulting paths, we can reduce storage by retaining only one bit per matrix element. Thus, $D_{i
- Matrix multiplication can also be used to solve transitive closure. Let M^{1} be the adjacenc
This will be faster for large enough n if using Strassen's fast matrix multiplication algorithm

%---- Page End Break Here ---- Page : 568
The running time for all three of these methods can be substantially improved on many graphs. Rec

Transitive reduction (also known as minimum equivalent digraph) is the inverse operation to trans

Although the transitive closure of G is uniquely defined, a graph may have many different trans
- A quick-and-dirty, linear-time transitive reduction algorithm identifies the strongly connected
One catch with this heuristic is that it might add edges to the transitive reduction of G that
- If we are restricted to only using edges from G in our transitive reduction, we must abandon
see why, consider a directed graph consisting of one strongly connected component, where every ve
A heuristic for edge-preserving transitive reduction successively considers each edge, deleting i
- The minimum size reduction using arbitrary pairs of vertices as edges can be found in $O\left(n

%---- Page End Break Here ---- Page : 569
Implementations: The Boost Graph Library \\cite{siek2002boost} (http://www.boost. org/libs/graph)

Combinatorica \\cite{pemmaraju2003computational} provides Mathematica implementations of transiti

Notes: Van Leeuwen \\cite{vanleeuwen1990graph} provides an excellent survey on transitive closure

There is a surprising amount of more recent activity on transitive closure, much of it captured i

The equivalence between transitive closure and reduction was established in \\cite{aho1972transit

Estimating the size of the transitive closure is important in database query optimization. A line

Related problems: Connected components (see page 542), shortest path (see page 554.).\index{conne
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-571.jpg?height=588&width=123

```
%---- Page End Break Here ---- Page : 570
\section{Matching}

Input description: A (weighted) graph $G=(V, E)$.
Problem description: Find the largest set of edges $E^{\prime}$ from $E$ such that ever

Discussion: Suppose you manage a team of workers, each capable of performing a distinct

Matching is a very powerful piece of algorithmic magic, so powerful that it is quite su

Marrying off a set of boys to a set of girls such that each couple is happy is another

This basic matching framework can be enhanced in several ways, while remaining essentia
- Is your graph bipartite? - Many matching problems involve bipartite graphs, as in the
marriages represent a matching on a more general, non-bipartite graph. Efficient algori
- What if employees can be given multiple jobs? - Natural generalizations of matching i
- Is your graph weighted or unweighted? - The matching applications discussed so far ar
But other applications augment each edge with a weight, perhaps reflecting the suitabil


%---- Page End Break Here ---- Page : 571
Efficient algorithms for constructing matchings work by finding augmenting paths in gra

General graphs prove trickier to match than bipartite ones, because it is possible to l

The standard algorithms for bipartite matching are based on network flow, using a simpl

Be warned that different approaches are needed to solve weighted matching problems, mos

Implementations: High-performance codes for both weighted and unweighted bipartite matc


%---- Page End Break Here ---- Page : 572
The First DIMACS Implementation Challenge \\cite{johnson1993network} focused on network

LEDA (see Section 22.1.1 (page 713) provides efficient implementations in C++ for both

The Stanford GraphBase (see Section 22.1.7 (page 715) contains an implementation of the

Notes: Lovász and Plummer \\cite{lovasz2009matching} is the definitive reference on mat

Edmond's algorithm \\cite{edmonds1965paths} for maximum-cardinality matching is of grea

Consider a matching of boys to girls containing edges $\left(B_{1}, G_{1}\right)$ and
```

Online matching problems arise when edge selection must take place without full information about

The maximum matching is equal in size to the minimum vertex cover in bipartite graphs. This impli

Related problems: Network flow (see page 571. vertex cover (see page 591.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-574.jpg?height=597&width=125

%---- Page End Break Here ---- Page : 573
\section{Eulerian Cycle/Chinese Postman}

Input description: A graph $G=(V, E)$.
Problem description: Find the shortest tour visiting every edge of G.
Discussion: Suppose you are charged with designing the daily routes for garbage trucks, snow plow

Alternatively, consider a human-factors validation of telephone menu systems. Each "Press 4 for m

Both tasks are variants of the Eulerian cycle problem, best characterized by the puzzle that asks

Well-known conditions exist for determining whether a graph contains an Eulerian cycle:
- An undirected graph contains an Eulerian cycle iff it is connected, and each vertex is of even
- A directed graph contains an Eulerian cycle iff it is strongly connected, and each vertex has t

For Eulerian paths, which cover all edges but do not return to the starting vertex, the degree co
contains an Eulerian path iff all but two vertices are of even degree. These will serve as the st

%---- Page End Break Here ---- Page : 574
These characterizations of Eulerian graphs make it is easy to test whether such a path/cycle exis

An Eulerian cycle, if one exists, solves the motivating snowplow problem, since any tour that vis

The optimal postman tour can be constructed by adding the appropriate edges to the graph G to m

Finding the best set of shortest paths to add to G reduces to identifying a minimum-weight perf

Implementations: Several graph libraries provide implementations of Eulerian cycles, but Chinese

GOBLIN (http://goblin2.sourceforge.net/) is an extensive C++ library dealing with all of the star
cycles, matching, and shortest path in both bipartite and general graphs. Combinatorica \\cite{pe

%---- Page End Break Here ---- Page : 575
Notes: The history of graph theory began in 1736, when Euler first solved the seven bridges of Kö

Corberán and Laporte \\cite{corberan2013arc} and Toth and Vigo \\cite{toth2014-vehicle-

The Euler's tour technique is an important paradigm in parallel graph algorithms. Many

The problem of finding the shortest tour traversing all edges in a graph was introduce

A de Bruijn sequence S of span k on an alphabet Σ of size α is a cir

De Bruijn sequences can be constructed by building a directed graph whose vertices rep

A cute algorithm for optimizing embroidery patterns based on Eulerian cycles is presen

Related problems: Matching (see page 562, Hamiltonian cycle (see page 598.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-577.jpg?height=39

%---- Page End Break Here ---- Page : 576
\section{Edge and Vertex Connectivity}

Input description: A graph G. Optionally, a pair of vertices s and t.
Problem description: What is the smallest subset of vertices (or edges) whose deletion

Discussion: Graph connectivity arises in problems related to network reliability. In t

The edge (vertex) connectivity of a graph G is the smallest number of edge (vertex)

Several connectivity problems prove to be of interest:
- Is the graph already disconnected? - The simplest connectivity problem is testing wh
- Is there a weak link in my graph? - We say that G is biconnected if there is no si

The simplest algorithm to identify articulation vertices (or bridges) tries deleting v
- What if I want to split the graph into equal-sized pieces? - Often we seek a small c
This is the graph partition problem, discussed in Section 19.6 (page 601). Although th
- Are arbitrary cuts OK, or must I separate a given pair of vertices? There are two fl

%---- Page End Break Here ---- Page : 577
Edge and vertex connectivity can both be found using network-flow techniques, which in

Vertex connectivity is characterized by Menger's theorem, which states that a graph is

To exploit Menger's theorem, we construct a graph $G^{\prime}$ such that any set of ed

Implementations: MINCUTLIB is a collection of high-performance codes for several diffe
perimental study of these algorithms and the heuristics needed to make them run fast $

```
%---- Page End Break Here ---- Page : 578
Most of the graph data structure libraries of Section 18.1 (page 542 ) include routines for conne

GOBLIN (http://goblin2.sourceforge.net/) is an extensive C++ library dealing with all of the star

Notes: Good expositions on the network-flow approach to edge and vertex connectivity include \\ci

The theoretically fastest algorithms for minimum-cut/edge connectivity are based on graph contrac

The fastest version of Karger's algorithm runs in $\left(m \lg ^{3} n\right)$ expected time \\cit

Minimum-cut methods have found many applications in computer vision, including image segmentation

A non-flow-based algorithm for edge $k$-connectivity in $O\left(k n^{2}\right)$ is due to Matula

Related problems: Connected components (see page 542), network flow (see page 571, graph partitio
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-580.jpg?height=586&width=123


%---- Page End Break Here ---- Page : 579
\section{Network Flow}

Input description: A directed graph $G$, where each edge $e=(i, j)$ has a capacity $c_{e}$. A sou

Problem description: What is the maximum flow you can route from $s$ to $t$ while respecting the

Discussion: Applications of network flow go far beyond plumbing. Finding the most cost-effective

But the real power of network flow is that many linear programming problems arising in practice c

The key to exploiting this power is recognizing that your problem can be modeled as network flow.
- Maximum flow - Here we seek the heaviest possible flow from $s$ to $t$, given the edge capacity
$$
0 \leq x_{i j} \leq c_{i j} \text { for } 1 \leq i, j \leq n
$$

Furthermore, an equal flow comes in as goes out at each non-source or non-sink vertex, so
$$
\sum_{j=1}^{n} x_{j i}-\sum_{j=1}^{n} x_{i j}=0 \text { for all } 1 \leq i \leq n
$$


%---- Page End Break Here ---- Page : 580
We seek the assignment that maximizes the flow into sink $t$, namely $\sum_{i=1}^{n} x_{i t}$.
- Minimum cost flow - Here we have an extra parameter for each edge $(i, j)$, namely the cost $\l
```

```
$$
\sum_{i=1}^{n} \sum_{j=1}^{n} d_{i j} \cdot x_{i j}
$$
subject to the edge and vertex capacity constraints of maximum flow, plus the additiona

Special considerations include:
- What if I have multiple sources and/or sinks? - No problem. We can modify the networl
- What if all arc capacities are identical, either 0 or 1? - Faster algorithms exist fo
- What if all my edge costs are identical? - Use the simpler and faster algorithms for
- What if I have multiple types of material moving through the network? Every message s
Linear programming will suffice for multicommodity flow if fractional flows are permit

Network flow algorithms can be complicated, and significant engineering is required to
- Augmenting path methods - These algorithms repeatedly find a path of positive capaci
- Preflow-push methods - These algorithms push flows from one vertex to another, initia


%---- Page End Break Here ---- Page : 581
Implementations: High-performance codes for both maximum flow and minimum cost flow we

The C++ Boost Graph Library [\\cite{siek2002boost}] (http://www.boost.org/libs/ graph)

The First DIMACS Implementation Challenge on Network Flows and Matching \\cite{johnson

Notes: Excellent books on network flows and its applications include \\cite{ahuja1993n

Conventional wisdom has long held that network flow should be computable in $O(n m)$ t

Information flows through a network can be modeled as multicommodity flows, with the o

Related problems: Linear programming (see page 482), matching (see page 562 , connecti
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-583.jpg?height=69


%---- Page End Break Here ---- Page : 582
\section{Drawing Graphs Nicely}

Input description: A graph $G$.
Problem description: Draw a graph $G$ to accurately reflect its structure.
Discussion: Graph drawing is a natural problem, yet it is inherently ill-defined. What

Unfortunately, these are "soft" criteria for which it is impossible to design an optim

Several "hard" criteria can help assess the quality of a drawing:
- Crossings - We seek a drawing with as few pairs of crossing edges as possible, becaus
- Area - We seek a drawing that uses as little paper as possible relative to the short
```

- Edge length - We seek a drawing that avoids long edges, because they tend to obscure other feat
- Angular resolution - We seek a drawing that avoids small angles between two edges incident on a
- Aspect ratio - We seek a drawing whose aspect ratio (width/height) reflects the desired output

%---- Page End Break Here ---- Page : 583

Unfortunately, these goals are mutually contradictory, and the problem of finding the best drawin

Two final warnings before getting down to business. For graphs without any inherent symmetries or

Once all this is understood, it must be admitted that graph-drawing algorithms can be quite effec
- Must the edges be straight, or can I have curves and/or bends? - Straightline drawing algorithm
- Is there a natural, application-specific drawing? - If your graph represents a network of citie
- Is your graph either planar or a tree? - If so, use one of the special planar graph or tree-dra
- Is your graph directed? - Edge direction has a significant impact on the nature of the desired
- How fast must your algorithm be? - Your graph drawing algorithm must be very fast if it will be
- Does your graph contain symmetries? - The output drawing above is attractive because the graph

%---- Page End Break Here ---- Page : 584

For a quick-and-dirty drawing, I recommend simply spacing the vertices evenly on a circle, and th

A good, general purpose graph-drawing heuristic models the graph as a system of springs and then

If you need a polyline graph-drawing algorithm, my recommendation is that you study the systems p

Drawing your graph opens another can of worms, namely where to place the edge/vertex labels. We s

Implementations: GraphViz (http://www.graphviz.org) is a popular and well-supported graph-drawing

All of the graph data structure libraries of Section 15.4 (page 452) devote some effort to visua
output graphs in GraphViz's Dot format, instead of reinventing the wheel.
VivaGraph (https://github.com/anvaka/VivaGraphJS) and Sigma (http: //sigmajs.org/) are popular Ja

%---- Page End Break Here ---- Page : 585

Graph drawing is a problem where very good commercial products exist, including those from Tom Sa

Combinatorica \\cite{pemmaraju2003computational}] provides Mathematica implementations of several
Notes: There exists a significant community of researchers in graph drawing, whose annual confere

Two excellent books on graph-drawing algorithms are Battista et al. [\\cite{dibattista1999graph}]

Graph embeddings encode the structural information associated with each vertex as a short vector,

It is trivial to space n points evenly along the boundary of a circle. However, the problem is

```
Related problems: Drawing trees (see page 578), planarity testing (see page 581.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-587.jpg?height=58


%---- Page End Break Here ---- Page : 586
\section{Drawing Trees}

Input description: A tree $T$, which is a graph without any cycles.
Problem description: Create a nice drawing of the tree $T$.
Discussion: Many applications require drawing pictures of trees. Tree diagrams are oft

Different aesthetics are associated with each application, making it difficult to gene
- Rooted trees define a hierarchical order, emanating from a single source node identi
- Free trees do not encode any structure beyond their connection topology. There is no

Trees are always planar graphs, and hence can and should be drawn so no two edges cros
because there are much simpler ways of constructing planar drawings of trees. The spri


%---- Page End Break Here ---- Page : 587
The most natural tree-drawing algorithms assume rooted trees. However, they can be use

Your two primary options for drawing rooted trees are ranked and radial embeddings:
- Ranked embeddings - Place the root in the top center of your page, and then partitio
Such ranked embeddings are particularly effective to represent a hierarchybe it a fami
- Radial embeddings - Free trees are better drawn using a radial embedding, where the 

Implementations: GraphViz (http://www.graphviz.org) is a popular and well-supported gr

Very good commercial products exist for graph/tree, including those from Tom Sawyer So


%---- Page End Break Here ---- Page : 588
Combinatorica \\cite{pemmaraju2003computational} provides Mathematica implementations 

Notes: All books and surveys on graph drawing include discussions of tree-drawing algo

A comprehensive resource of tree visualization is https://treevis.net/, with an associ

Heuristics for tree layout have been studied by several researchers, with Buchheim, et

Certain tree layout algorithms arise from non-drawing applications. The van Emde Boas 

Related problems: Drawing graphs (see page 574), planar drawings (see page 581.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-590.jpg?height=60
```

%---- Page End Break Here ---- Page : 589
\section{Planarity Detection and Embedding}

Input description: A graph G.
Problem description: Can G be drawn in the plane such that no two edges cross? If so, produce s

Discussion: Planar drawings (or embeddings) make clear the structure of a given graph by eliminat

Planar graphs have nice properties that can be exploited to yield faster algorithms for many prob

To gain a better appreciation of the subtleties of planar drawings, I encourage the reader to con

The study of planarity has motivated much of the development of graph theory. It must be confesse

%---- Page End Break Here ---- Page : 590
\\cite{skiena1999who}. That said, it is still very useful to know how to deal with planar graphs

It pays to distinguish the problem of planarity testing (does my graph have a planar drawing?) fr

Algorithms for planarity testing begin by embedding an arbitrary cycle from the graph in the plan

Such path-crossing algorithms can be used to construct a planar embedding by inserting the paths

For non-planar graphs, what is often sought is a drawing that minimizes the number of crossings.

Implementations: LEDA (see Section 22.1.1 (page 713) includes linear-time algorithms for both pla

The Open Graph Drawing Framework (http://www.ogdf.net), presented in CGJ ${ }^{+}$13], is a C++ g

PIGALE (http://pigale.sourceforge.net/) is a C++ graph editor and
algorithm library focusing on planar graphs. It contains a variety of algorithms for constructing

%---- Page End Break Here ---- Page : 591
Greedy randomized adaptive search procedure (GRASP) heuristics to find the largest planar subgrap

Notes: Kuratowski \\cite{kuratowski1930sur} gave the first characterization of planar graphs, nam

Hopcroft and Tarjan \\cite{hopcroft1974efficient} gave the first linear-time algorithm for drawin

Outerplanar graphs are those that can be drawn such that all vertices lie on the outer face of th

Generalizations of planarity revolve around embedding graphs in more complicated surfaces than th

```
Related problems: Graph drawing (see page 574), drawing trees (see page 578.


%---- Page End Break Here ---- Page : 592
Chapter 19

\chapter{Graph Problems: NP-Hard}

A cynical view of graph algorithms is that "everything we want to do is hard." Indeed,

Still, do not abandon hope if your problem resides in this chapter. I provide a recomm

The following books will help you deal with NP-complete problems:
- Garey and Johnson \\cite{garey1979computers} - This is the classic reference on the
- Crescenzi and Kann \\cite{crescenzi1997approximation} - This website (www.nada.kth.s
- Williamson and Shmoys \\cite{williamson2011design} - The most comprehensive textbook
- Vazirani \\cite{vazirani2004approximation} - A complete treatment of the theory of a
- Gonzalez \\cite{gonzalez2018handbook} - This handbook contains current surveys on a
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-594.jpg?height=58

\section{Clique}

Input description: A graph $G=(V, E)$.
Problem description: What is the largest subset $S$ of vertices such that all pairs ar

Discussion: In high school, everybody complained about the "clique" - a group of frien

Identifying "clusters" of related objects often reduces to finding large cliques in gr

Since every single edge in a graph represents a clique of two vertices, the challenge
- Will a maximal clique suffice? - A clique is maximal if it cannot be enlarged by add
in fact be the largest possible clique, but it probably won't be, and could well be mu
- What if I will settle for a large, dense subgraph? - Insisting on cliques to define
The largest set of vertices whose induced subgraph has vertex degree $\geq k$ can be f
- What if the graph is planar? - Planar graphs cannot have cliques of a size larger th


%---- Page End Break Here ---- Page : 594
If you really need to find the largest clique in a graph, an exhaustive search via bac

Heuristics for finding large cliques based on randomized techniques, such as simulated


%---- Page End Break Here ---- Page : 595
Implementations: Cliquer is a set of C routines for finding cliques in arbitrary weigh
```

Programs for finding cliques and independent sets were sought for the Second DIMACS Implementatio

Kreher and Stinson \\cite{kreher1999combinatorial}] provide branch-and-bound programs in C for fi

GOBLIN (http://goblin2.sourceforge.net/) employs branch-and-bound algorithms to find large clique

Notes: Bomze, et al. \\cite{bbpp99} and Wu, et al. WH15 give comprehensive surveys on the problem

The proof that clique is NP-complete is due to Karp \\cite{karp1972reducibility}. His reduction (

The densest subgraph problem seeks the subset of vertices whose induced subgraph has the highest

That clique cannot be approximated to within a factor of $n^{1 / 2-\epsilon}$ unless $P=N P$ (and

Related problems: Independent set (see page 589), vertex cover (see page 591.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-597.jpg?height=602&width=125

%---- Page End Break Here ---- Page : 596
Independent Set

Input description: A graph $G=(V, E)$.
Problem description: What is the largest subset S of vertices of V such that there is no edge

Discussion: The need to find large independent sets arises in facility dispersion problems, where

Independent sets (also known as stable sets) avoid conflicts between elements, and hence often ar

Independent set is closely related to two other NP-complete problems:\index{approximation scheme}
- Clique - A clique is a subset of pairwise connected vertices, while an independent set is a sub
- Vertex coloring - The vertex coloring of a graph $G=(V, E)$ is a partition of V into a k se
Indeed, one heuristic to find a large independent set is to use any vertex coloring algorithm/heu

%---- Page End Break Here ---- Page : 597
The simplest reasonable heuristic is to find the lowest-degree vertex, add it to the independent

The independent set problem is in some sense dual to graph matching. The former asks for a large

The maximum independent set of a tree can be found in linear time by (1) stripping off the leaf n

Implementations: Any program for computing the maximum clique in a graph can find maximum indepen

GOBLIN (http://goblin2.sourceforge.net/) implements a branch-andbound algorithm for finding indep

Greedy randomized adaptive search (GRASP) heuristics for independent set have been impl
Notes: That independent set is NP-complete was shown by Karp \\cite{karp1972reducibilit

Finding maximal independent sets is a challenging problem in parallel and distributed m

Related problems: Clique (see page 586, vertex coloring (see page 604, vertex cover (s
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-599.jpg?height=58

%---- Page End Break Here ---- Page : 598
\section{Vertex Cover}

Input description: A graph $G=(V, E)$.
Problem description: What is the smallest subset $C \subseteq V$ such that every edge $

Discussion: Vertex cover is a special case of the more general set cover problem, whic

To turn vertex cover into a set cover problem, let universal set U represent the set

Vertex cover and independent set are very closely related problems. Since every edge i

The simplest heuristic for vertex cover selects the vertex with highest degree,
adds it to the cover, deletes all adjacent edges, and then repeats until the graph is e

%---- Page End Break Here ---- Page : 599
Fortunately, we can always find a vertex cover whose size is at most twice as large as

Taking both of the vertices for each edge in this maximal matching gives us a vertex co

This heuristic can be tweaked to perform somewhat better in practice, if not in theory

The vertex cover problem seeks to cover all edges using few vertices. Two other importa
- Cover all vertices using few vertices - The dominating set problem seeks the smallest

Dominating set problems can also be expressed as instances of set cover (see Section 21
- Cover all vertices using few edges - The edge cover problem seeks the smallest set of
of edge cover and vertex cover have such different fates: one NP-complete and the other

%---- Page End Break Here ---- Page : 600
Implementations: Any program for computing the maximum clique in a graph can be applied

JGraphT (https://jgrapht.org) is a Java graph library that contains greedy and 2-approx

Notes: Karp \\cite{karp1972reducibility} first proved that vertex-cover is NP-complete

Whether there exists a better than 2-factor approximation for vertex cover has long been one of t

The primary reference on dominating sets is the monograph of Haynes et al. \\cite{haynes1998funda

Related problems: Independent set (see page 589), set cover (see page 678).\index{edge tour!probl
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-602.jpg?height=588&width=124

%---- Page End Break Here ---- Page : 601
\section{Traveling Salesman Problem}

Input description: A weighted graph G.
Problem description: Find the cycle of minimum cost, visiting each vertex of G exactly once.

Discussion: The traveling salesman problem (TSP) is the most notorious NPcomplete problem. This i

The traveling salesman problem arises in many transportation and routing problems. Optimizing too

Several issues arise in solving TSPs:
- Is the graph unweighted? - If the graph is unweighted, or is a complete graph where all the edg
- Does your input satisfy the triangle inequality? - Our sense of how proper distance measures be
cheapest airfare between two points. TSP heuristics work much better on sensible graphs that do o
- Are you given n points as input or a weighted graph? - Geometric instances are often easier t
- Can you visit a vertex more than once? - The restriction that the tour not revisit any vertex i

%---- Page End Break Here ---- Page : 602
TSP with repeated vertices is easily solved by using any conventional TSP code on a new cost matr
- Is your distance function symmetric? - A distance function is asymmetric when there exists $x,
- How important is it to find the optimal tour? - Heuristic solutions will suffice for most appli

Almost any flavor of TSP is going to be NP-complete, so the right way to proceed is with heuristi

%---- Page End Break Here ---- Page : 603
Empirical results in the literature are sometimes contradictory. However, we recommend choosing f
- Minimum spanning trees - Start by finding the minimum spanning tree of the sites, and then do a
- Incremental insertion methods - A different class of heuristics starts from a single vertex, an
$$
\max _{v \in V} \min _{i=1}^{|T|}\left(d\left(v, v_{i}\right)+d\left(v, v_{i+1}\right)\right)
$$

The "min" ensures that we insert the vertex in the position that adds the smallest amount of dist
- K-optimal tours - More powerful are the Kernighan-Lin or k-opt heuristics. The method applies

Implementations: Concorde is a program for the symmetric traveling salesman problem and
among available TSP codes. Their website features very interesting material on the his

%---- Page End Break Here ---- Page : 604
Lodi and Punnen \\cite{lodi2007tsp}] put together an excellent survey of available sof

TSPLIB \\cite{reinelt1991tsplib}] provides the standard collection of hard instances o

Notes: The book by Applegate et al. \\cite{applegate2007traveling} documents the techn

Experimental results on heuristic methods for solving large TSPs include \\cite{ben92a

The Christofides heuristic \\cite{christofides1976heuristic} is an improvement over th

Polynomial-time approximation schemes for Euclidean TSP have been developed by Arora \

The history of progress on optimal TSP solutions is inspiring. In 1954, Dantzig, Fulke

For sets of n points in convex position in the plane, the minimum TSP tour is descri

Related problems: Hamiltonian cycle (see page 598), minimum spanning tree (see page 54
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-606.jpg?height=59

%---- Page End Break Here ---- Page : 605
\section{Hamiltonian Cycle}

Input description: A graph $G=(V, E)$.
Problem description: Find a tour of the vertices using only edges from G, such that

Discussion: Hamiltonian cycle or path in a graph G is a special case of the travelin

Hamiltonian cycles are fundamental structures in graph theory, and a useful way to mod

Closely related is the problem of finding the longest path or cycle in a graph. In pre

The problems of finding longest cycles and paths are both NP-complete, even on very re
- Is there a serious penalty for visiting vertices more than once? - Reformulating the
and approximation algorithms. Finding a spanning tree of the graph and doing a depth-f
- Am I seeking the longest path in a directed acyclic graph (DAG)? - The problem of fi
- Is my graph dense? - Sufficiently dense graphs always contain Hamiltonian cycles. Fu
- Are you visiting all the vertices or all the edges? - Verify that you really have a

%---- Page End Break Here ---- Page : 606

If you really must know whether your graph is Hamiltonian, backtracking with pruning is your only

Implementations: The reduction described above (weight 1 for an edge and 2 for a non-edge) turns

An effective program for solving Hamiltonian cycle problems resulted from the masters thesis of V

Lodi and Punnen \\cite{lodi2007tsp}] put together an excellent survey of available TSP software, are maintained athttp://or.deis.unibo.it/research_pages/tspsoft.html. Nijenhuis and Wilf \\cite{n

%---- Page End Break Here ---- Page : 607
The football program of the Stanford GraphBase (see Section 22.1.7 (page 715) uses a stratified

Notes: Hamiltonian cycles first arose in Euler's study of the knight's tour problem, although the

Finding long paths in graphs is very difficult. Although fast algorithms exist to find paths of l

Techniques for solving optimization problems in the laboratory using biological processes have at

Related problems: Eulerian cycle (see page 565), traveling salesman (see page 594.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-609.jpg?height=590&width=123

%---- Page End Break Here ---- Page : 608
\section{Graph Partition}

Input description: A (weighted) graph $G=(V, E)$ and integers k and m.
Problem description: Partition the vertices into m roughly equal-sized subsets such that the to

Discussion: Graph partitioning arises in many divide-and-conquer algorithms, which gain their eff

Graph partition also arises when clustering vertices into logical components. If edges link "simi

Finally, graph partition is a critical step in many parallel algorithms. Consider the finite elem

Several different flavors of graph partitioning arise, depending on the desired objective functio
- Minimum cut set - The smallest set of edges to cut so as to disconnect a graph can be efficient
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-610.jpg?height=561&width=785

%---- Page End Break Here ---- Page : 609
Figure 19.1: The maximum cut of a bipartite graph cuts all the edges.
- Graph partition - A better partition criterion seeks a small cut that partitions the vertices i
Certain types of graphs always have small separators that partition the vertices into balanced pi
Each planar graph has a set of $O(\sqrt{n})$ vertices whose deletion leaves no component with mor
- Maximum cut - Given an electronic circuit specified by a graph, the maximum cut defines the lar

The basic approach for dealing with graph partitioning or maximum cut problems constru
iterations are needed before the process converges on a local optimum. Even so, such a

%---- Page End Break Here ---- Page : 610
There are many variations of this basic procedure, by changing the order we test the v

Spectral partitioning methods use sophisticated linear algebra techniques to obtain a g

Implementations: A very popular code for graph partitioning is METIS (http://glaros.dt

Scotch (http://www.labri.fr/perso/pelegrin/scotch/) is another wellrespected code to c

The Tenth DIMACS Challenge (https://www.cc.gatech.edu/dimacs10/) revolved around the r

Notes: Recent surveys of graph partitioning algorithms include [\\cite{bichot2013graph

The planar separator theorem and an efficient algorithm for finding such a separator a

Any random vertex partition will expect to cut half of the edges in the graph, because

Related problems: Edge/vertex connectivity (see page 568), network flow (see page 571)
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-612.jpg?height=68

%---- Page End Break Here ---- Page : 611
\section{Vertex Coloring}

Input description: A graph $G=(V, E)$.
Problem description: Color the vertices of V using the minimum number of colors such

Discussion: Vertex coloring arises in scheduling and clustering applications. Register

Of course, conflicts will never occur if each vertex is colored using a distinct color

Several special cases of interest arise in practice:
- Can I color the graph using only two colors? - An important special case is testing

%---- Page End Break Here ---- Page : 612
It is easy to test whether a graph G is bipartite. Color the first vertex blue, and
- Is the graph planar, or are all vertices of low degree? - The famous fourcolor theor

But there is a very simple algorithm that will vertex color any planar graph using at
- Is this an edge-coloring problem? - Certain vertex coloring problems can be modeled

Computing the chromatic number of a graph is NP-complete. If you need an exact solution you must

Incremental methods prove to be the heuristic of choice for vertex coloring. As in the previously

Incremental methods can be further improved by using color interchange. Taking a properly colored
the red vertices blue and the blue vertices red) leaves a proper vertex coloring. Now suppose we

%---- Page End Break Here ---- Page : 613
Color interchange is a win in terms of producing better colorings, at a cost of increased time an

Implementations: Graph coloring has been blessed with two useful web resources. Culberson's graph

Programs for the closely related problems of finding cliques and vertex coloring graphs were soug

The C++ Boost Graph Library \\cite{siek2002boost} (http://www.boost.org/libs/ graph) and Java JGr

Nijenhuis and Wilf \\cite{nijenhuis1978combinatorial} provide an efficient Fortran implementation

Notes: Recent survey articles on graph coloring include \\cite{galinier2013recent} \\cite{malagut

Wilf Wil84 proved that backtracking to test whether a random graph has chromatic number k runs

Paschos \\cite{paschos2003polynomial} reviews what is known about provably good approximation alg
$n^{1-1 /(\chi(G)-1)}$ approximation algorithm, where $\chi(G)$ is the chromatic number of G.
Brook's theorem states that the chromatic number $\chi(G) \leq \Delta(G)+1$, where $\Delta(G)$ is

%---- Page End Break Here ---- Page : 614
The four-color problem is the most famous problem in the history of graph theory, first posed in

Related problems: Independent set (see page 589), edge coloring (see page 608).\index{minimum spa
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-616.jpg?height=593&width=124

%---- Page End Break Here ---- Page : 615
\section{Edge Coloring}

Input description: A graph $G=(V, E)$.
Problem description: What is the smallest set of colors needed to color the edges of G such tha

Discussion: Edge coloring arises in scheduling applications, typically associated with minimizing

The National Football League solves such an edge-coloring problem to make up its schedule each se

```
The minimum number of colors needed to edge color a graph is called its edge-chromatic

Edge coloring has a better (if less famous) theorem associated with it than vertex colo
perspective, note that any edge coloring must have at least $\Delta$ colors, because al


%---- Page End Break Here ---- Page : 616

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{algorithms_page_0618_1.png}
\end{figure}
The proof of Vizing's theorem is constructive, meaning it can be turned into an $O(n m

Edge-coloring on graph $G$ can be converted to the problem of finding a vertex coloring

Implementations: The C++ Boost Graph Library \\cite{siek2002boost} (http://www. boost.c

See Section 19.7 (page 604) for a larger collection of vertex-coloring codes and heuris

Notes: Stiebitz et al. [\\cite{stiebitz2012graph}] is a recent book on edge coloring. C

Whitney, in introducing line graphs Whi32, showed that any two connected graphs with is

Related problems: Vertex coloring (see page 604), scheduling (see page 534).
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-618.jpg?height=595


%---- Page End Break Here ---- Page : 617
\section{Graph Isomorphism}

Input description: Two graphs, $G$ and $H$.
Problem description: Find a mapping $f$ from the vertices of $G$ to the vertices of $H$

Discussion: Isomorphism is the problem of testing whether two graphs are really the sam

Many pattern recognition problems can be mapped to graph or subgraph isomorphism. For e

What exactly is meant when we say that two graphs are the same? Two labeled graphs $G=V

Identifying symmetries is another important application of graph isomorphism. A mapping

Several variations of graph isomorphism arise in practice:
- Is graph $G$ contained in graph H? - Instead of testing equality, we are often intere
There are two distinct graph-theoretic notions of "contained in." Subgraph isomorphism
```

%---- Page End Break Here ---- Page : 618
Be aware of this distinction in your application. Subgraph isomorphism problems tend to be much h
- Are your graphs labeled or unlabeled? - In many applications, vertices or edges of the graphs a
Labels and related constraints can be factored into any backtracking algorithm. Further, such con
- Are you testing whether two trees are isomorphic? - Faster algorithms exist for special cases o
Efficient algorithms for tree isomorphism work inward toward the center, starting from the leaves
- How many graphs do you have? - Many data mining applications search for all instances of a part

%---- Page End Break Here ---- Page : 619
No polynomial-time algorithm is known for graph isomorphism, but neither is it known to be NP-com

Although no worst-case polynomial-time algorithm is known, testing isomorphism is usually not ver

But the real key to efficient isomorphism testing is preprocessing the vertices into "equivalence
- Vertex degree - Two vertices of different degrees can never be identical, or mapped to each oth
- Shortest path distance - The all-pairs shortest path matrix (see Section 18.4 (page 554) define
- Counting length-k paths - Taking the k th power of the adjacency matrix of G yields a matri

By using these invariants, you should be able to partition the vertices of most graphs into a lar

%---- Page End Break Here ---- Page : 620
Implementations: The best isomorphism testing program is nauty (No AUTomorphisms, Yes?) - a set o

Valiente \\cite{valiente2002algorithms} has made available the implementations of graph/subgraph

Kreher and Stinson \\cite{kreher1999combinatorial}] compute isomorphisms of graphs in addition to

Notes: Graph isomorphism is an important problem in computational complexity theory because of it

Monographs on isomorphism detection include \\cite{hoffmann1982group} \\cite{kobler1993graph}]. V

Polynomial-time algorithms are known for planar graph isomorphism \\cite{hopcroft1974linear} and

A problem is said to be isomorphism-complete if it is provably as hard as isomorphism. Bipartite

Related problems: Shortest path (see page 554), string matching (see page 685.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-622.jpg?height=589&width=124

%---- Page End Break Here ---- Page : 621
\section{Steiner Tree}

Input description: A graph $G=(V, E)$ and specified subset of vertices $T \subseteq V$. Or set of

Problem description: Find the smallest tree connecting the vertices of T.
Discussion: Steiner trees arise in network design problems, because the minimum Steine

The Steiner tree problem is distinguished from the minimum spanning tree problem in gr
- How many points must you connect? - The Steiner tree of just a pair of vertices is t
- Is the input a set of geometric points? - The geometric version of Steiner tree work
of points. These Steiner points must satisfy several geometric properties, which can b
- Are there constraints on the edges we define? - In many wiring problems, all the edg
- Do I really need an optimal tree? - Certain Steiner tree applications justify invest
There are many opportunities for pruning search based on geometric and graph-theoretic
- What is the meaning of the Steiner vertices? - A very special type of Steiner tree a

%---- Page End Break Here ---- Page : 622
Many phylogenic tree construction algorithms have been developed that differ in what d

Fortunately, there is a good, efficient heuristic for finding Steiner trees that works

%---- Page End Break Here ---- Page : 623
The worst case for the minimum spanning tree approximation of the Euclidean Steiner tr

For geometric instances, any suboptimal tree can be refined by inserting a Steiner poi

Note that we are only interested here in the subtree connecting the terminal vertices.

An alternative heuristic for graphs is based on finding shortest paths. Start with a t

Implementations: GeoSteiner is a package for solving both Euclidean and rectilinear St

Steiner tree algorithms were the subject of the 11th DIMACS Implementation Challenge,

FLUTE (http://home.eng.iastate.edu/~cnchu/flute.html) computes rectilinear Steiner tre

The programs PHYLIP (http://evolution.genetics.washington.edu/ phylip.html) and PAUP (

%---- Page End Break Here ---- Page : 624
Notes: Monographs on the Steiner tree problem include Hwang, Richards, and Winter \\ci

The Euclidean Steiner problem dates back to Fermat, who asked how to find a point p

Gilbert and Pollak \\cite{gilbert1968steiner} first conjectured that the ratio of the

Arora \\cite{arora1998polynomial} gave a polynomial-time approximation scheme (PTAS) f

The hardness of Steiner tree for Euclidean and rectilinear metrics was established in \\cite{gare

Analogies can be drawn between minimum Steiner trees and minimum energy configurations in certai

Related problems: Minimum spanning tree (see page 549), shortest path (see page 554 .\index{mail
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-626.jpg?height=595&width=12

%---- Page End Break Here ---- Page : 625
\section{Feedback Edge/Vertex Set}

Input description: A (directed) graph $G=(V, E)$.
Problem description: What is the smallest set of edges $E^{\prime}$ or vertices $V^{\prime}$ whos

Discussion: Feedback set problems arise because many things are easier to do on directed acyclic

The feedback set identifies a small number of constraints that can be dropped to permit a valid s

Similar considerations are involved in eliminating race conditions from electronic circuits. This

One final application has to do with ranking tournaments. Suppose we want to order the skills of

Issues in feedback set problems include:
- Do any constraints have to be dropped? - No changes are needed if the graph is already a DAG, w
- How can I find a good feedback edge set? - An effective linear-time heuristic constructs a vert

%---- Page End Break Here ---- Page : 626
But what is the right vertex order to start with? A good heuristic is to sort the vertices in ter
- How can I find a good feedback vertex set? - The heuristics above yield vertex orders defining
- How do I break all cycles in an undirected graph? - The problem of finding feedback sets in und
The feedback vertex set problem remains NP-complete for undirected graphs, however. A reasonable

It may pay to refine any of these heuristic solutions using randomization or simulated annealing.

%---- Page End Break Here ---- Page : 627
Implementations: Greedy randomized adaptive search procedure (GRASP) heuristics for both feedback

An exact solver for feedback vertex set https://github.com/wata-orz/fvs by Iwata and Imanishi \\c

GOBLIN (http://goblin2.sourceforge.net) includes an approximation heuristic for minimum feedback

The econ_order program of the Stanford GraphBase (see Section 22.1.7 (page 715) permutes the rows

Notes: See \\cite{festa1999feedback}] for a survey on the feedback set problem. Exposi

Bafna, et al. \\cite{bbf99} gives a 2-factor approximation for feedback vertex set in u

Fixed-parameter tractable algorithms are polynomial in the input size n and exponenti

I will confess that I use a feedback edge set approach to grading semester projects in

An interesting application of feedback arc set to economics is presented in \\cite{knu

Related problems: Bandwidth reduction (see page 470), topological sorting (see page 54

%---- Page End Break Here ---- Page : 628
Chapter 20

\chapter{Computational Geometry}

Computational geometry is the algorithmic study of geometric problems. Its emergence co

Good books on computational geometry include:
- de Berg, et al. \\cite{deberg2008computational} - The "three Mark's" book is the bes
- O'Rourke O'R01 - This is the best practical introduction to computational geometry.
- Preparata and Shamos \\cite{preparata1985computational} - Although somewhat out of da
- Goodman, O'Rourke, and Toth \\cite{toth2018-handbook} - This recent collection of su

The leading conference in computational geometry is the ACM Symposium on Computational
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-630.jpg?height=59

\section{Robust Geometric Primitives}

Input description: A point p and line segment l, or two segments l_{1}, l_{2}.
Problem description: Does p lie on, over, or under l ? Does l_{1} intersect $l_{
Discussion: Implementing basic geometric primitives is a task fraught with peril. Even

If you are new to implementing geometric algorithms, I suggest that you see O'Rourke's

There are two different issues at work here: geometric degeneracy and numerical stabil
- Ignore it - Make an operating assumption that your program will work correctly only
for short-term projects where you can live with frequent crashes. The drawback is that
- Fake it - Randomly or symbolically perturb your data so that it becomes non-degenera
- Deal with it - Geometric applications can be made more robust by writing special cod

%---- Page End Break Here ---- Page : 630
Geometric computations often involve floating-point arithmetic, which leads to problems

- Integer arithmetic - By forcing all points of interest to lie on a fixedsize integer grid, you
- Double-precision reals - By using double-precision floating point numbers, you can reduce the c
- Arbitrary precision arithmetic - This is certain to be correct, but also to be slow. Careful an

The best technique to produce robust geometric software is to build your applications around a sm
- Area of a triangle - The area $A(t)$ of a triangle $t=(a, b, c)$ is indeed half the base times
$$
2 \cdot A(t)=\left|\begin{array}{ccc}
a_{x} & a_{y} & 1 \\
%---- Page End Break Here ---- Page : 631

b_{x} & b_{y} & 1 \\
c_{x} & c_{y} & 1
\end{array}\right|=a_{x} b_{y}-a_{y} b_{x}+a_{y} c_{x}-a_{x} c_{y}+b_{x} c_{y}-c_{x} b_{y}
$$

This formula generalizes to compute d ! times the volume of a simplex in d dimensions. Thus,
$$
6 \cdot A(t)=\left|\begin{array}{cccc}
a_{x} & a_{y} & a_{z} & 1 \\
b_{x} & b_{y} & b_{z} & 1 \\
c_{x} & c_{y} & c_{z} & 1 \\
d_{x} & d_{y} & d_{z} & 1
\end{array}\right|
$$

These formulae give signed volumes and hence can be negative, so take the absolute value first. S

The conceptually simplest way to compute the area of a polygon (or polyhedron) is to triangulate
- Above-below-on test - Does a given point c lie above, below, or on a given line l ? A clear

This primitive can be implemented using the sign of the triangle area as computed above. If the a
- Line segment intersection - This above-below primitive can also be used to test whether a line
- In-circle test - Does point d lie inside or outside the circle defined by points a, b, and
$$
\operatorname{incircle}(a, b, c, d)=\left|\begin{array}{cccc}
a_{x} & a_{y} & a_{x}^{2}+a_{y}^{2} & 1 \\
%---- Page End Break Here ---- Page : 632

b_{x} & b_{y} & b_{x}^{2}+b_{y}^{2} & 1 \\
c_{x} & c_{y} & c_{x}^{2}+c_{y}^{2} & 1 \\
d_{x} & d_{y} & d_{x}^{2}+d_{y}^{2} & 1
\end{array}\right|
$$

In-circle will return 0 if all four points are cocircular, a positive value if d is inside the

Check out the implementations below before you build your own.
Implementations: CGAL (www.cgal.org) and LEDA (see Section 22.1.1 (page 713) both provi

O'Rourke O'R01 provides implementations in C of most of the primitives discussed in thi

The Core Library (see http://cs.nyu.edu/exact/) provides an API, which supports the Exa

Shewchuk's \\cite{shewchuk1997robust} robust implementation of basic geometric primiti

Notes: O'Rourke O'R01 provides an implementation-oriented introduction to computational

Yap \\cite{sharma2018robust} gives an excellent survey on techniques for achieving robu

Related problems: Intersection detection (see page 648), maintaining arrangements (see
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-634.jpg?height=58

%---- Page End Break Here ---- Page : 633
\section{Convex Hull}\index{convex hull}

Input description: A set S of n points in d-dimensional space.
Problem description: Find the smallest convex polygon (or polyhedron) containing all th

Discussion: Convex hull is the most important elementary problem in computational geome

Convex hull serves as a preprocessing step to many geometric algorithms. For example, c

There are almost as many convex hull algorithms as sorting algorithms. Answer the follo
- How many dimensions are you working with? - Convex hulls are fast to compute in two a
for the hull points or do you need the defining polyhedron? Be aware of these complica

%---- Page End Break Here ---- Page : 634
Simple $O(n \log n)$ convex-hull algorithms are available for the important special cas

The key to efficiency is making sure that each edge is explored only once. Implemented
- Is your data given as vertices or half-spaces? - The problem of finding the intersect
- How many points are likely to be on the hull? - If your point set was generated "rand

This trick can also be applied beyond two dimensions, although it loses effectiveness a
- How do I find the shape of my point set? - Although convex hulls provide a gross meas

%---- Page End Break Here ---- Page : 635
The Graham scan is the most popular convex-hull algorithm in the plane. It starts with

This Graham scan procedure can also be used to construct a non-selfintersecting (or simple) polyg

The gift-wrapping algorithm becomes especially simple in two dimensions, since each "facet" becom

Implementations: The CGAL library (www.cgal.org) offers C ++ implementations of an extensive vari

Qhull \\cite{bdh97} is a popular low-dimensional, convex-hull code, optimized for two to about ei

O'Rourke O'R01 provides a robust implementation of the Graham scan in two dimensions and an $O\le

Ken Clarkson's higher-dimensional convex-hull code Hull also does alphashapes, and is available a

%---- Page End Break Here ---- Page : 636
Different codes are needed for enumerating the vertices of intersecting halfspaces in higher dime

Notes: Planar convex hull plays the same role in computational geometry that sorting does in algo

Good expositions of the Graham scan algorithm \\cite{graham1972efficient} and the Jarvis march \\

Topology is the study of shape. Edelsbrunner and Harar \\cite{edelsbrunner2010computational} prov

Reverse-search algorithms for constructing convex hulls are effective in higher dimensions \\cite

Dynamic algorithms for convex-hull maintenance are data structures that permit inserting and dele

Related problems: Sorting (see page 506), Voronoi diagrams (see page 634).
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-638.jpg?height=588&width=124

%---- Page End Break Here ---- Page : 637
\section{Triangulation}

Input description: A set of points, a polygon, or a polyhedron.
Problem description: Partition the interior of the point set or polyhedron into triangles.

Discussion: A good first step in working with complicated geometric objects is to break them into

One interesting application of triangulation is surface interpolation. Suppose that we have sampl

A triangulation in the plane is constructed by adding non-intersecting chords between the vertice
- Are you triangulating a point set or a polygon? - Often we are given a set of points to triangu

The simplest such $O(n \lg n)$ algorithm first sorts the points by x-coordinate. It then insert
of page 111, building the triangulation by adding a chord to each point newly cut off from the hu

- Does the shape of the triangles in your triangulation matter? - There are usually mar
Many applications seek to avoid skinny triangles, or equivalently, minimize small angl
- How can I improve the shape of a given triangulation? - Each internal edge of any tri

%---- Page End Break Here ---- Page : 638
This gives us a local "edge-flip" operation for changing and possibly improving a give
- What dimension are we working in? - Three-dimensional problems are usually harder th
- What constraints does the input have? - When we are triangulating a polygon or polyh
- Are you allowed to add extra points, or move input vertices? - When the shape of the
construction of a triangulation (say) with no small angles. As discussed above, you mu

%---- Page End Break Here ---- Page : 639
To triangulate a convex polygon in linear time, just pick an arbitrary starting vertex

Implementations: Triangle, by Jonathan Shewchuk, is an award-winning C language code t

Fortune's Sweep2 is a widely used two-dimensional code for Voronoi diagrams and Delaun

TetGen (http://wias-berlin.de/software/tetgen/) appears to be the software of choice t

Higher-dimensional Delaunay triangulations are a special case of higherdimensional con

Notes: Chazelle \\cite{chazelle1991triangulating} gave a linear-time algorithm for tri

%---- Page End Break Here ---- Page : 640
Wyk \\cite{tarjan1988-triangulation} followed before Chazelle's result. Bern et al. \\

Books on Delaunay triangulations and quality mesh generation include \\cite{aurenhamme

Linear-time algorithms for triangulating monotone polygons have been long known \\cite

A well-studied class of optimal triangulations seeks to minimize the total length of t

Related problems: Voronoi diagrams (see page 634, polygon partitioning (see page 658.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-642.jpg?height=57

%---- Page End Break Here ---- Page : 641
\section{Voronoi Diagrams}

Input description: A set S of points $p_{1}, \ldots, p_{n}$.
Problem description: Decompose space into regions such that all points in the region a

Discussion: Voronoi diagrams represent the region of influence around each of a given set of site

Voronoi diagrams have a surprising variety of applications:
- Nearest-neighbor search - Finding the nearest neighbor of query point q from a fixed set of p
- Facility location - Suppose McDonald's wants to open another restaurant. To minimize interferen
- Largest empty circle - Suppose you seek a large, contiguous, undeveloped piece of land on which
- Path planning - If the sites of S are the centers of obstacles we seek to avoid, the edges of
that maximize the distance to the obstacles. The "safest" path among the obstacles will thus stic
- Quality triangulations - When triangulating a set of points, we often seek nice, fat triangles

%---- Page End Break Here ---- Page : 642
Each edge of a Voronoi diagram is a segment of the perpendicular bisector of the line segment for

However, the method of choice is Fortune's sweep line algorithm, especially since robust implemen

There is an interesting relationship between convex hulls in $d+1$ dimensions and Delaunay triang
$$
\left(x_{1}, x_{2}, \ldots, x_{d}\right) \longrightarrow\left(x_{1}, x_{2}, \ldots, x_{d}, \sum_
$$
then taking the convex hull of this $(d+1)$-dimensional point set, and finally projecting back in

Several important variations of standard Voronoi diagrams arise in practice:
- Non-Euclidean distance metrics - Recall that Voronoi diagrams decompose space into regions of i
- Power diagrams - These structures decompose space into regions of influence around the sites, w
the same power. Imagine a map of radio stations broadcasting at a given frequency. The region of
- Kth-order and furthest-site diagrams - The idea of decomposing space into regions sharing some

%---- Page End Break Here ---- Page : 643
Implementations: Fortune's Sweep2 is a widely used two-dimensional code for Voronoi diagrams and

Both the CGAL (www.cgal.org) and LEDA (see Section 22.1.1 (page 713)) libraries offer $\mathrm{C}

Higher-dimensional and furthest-site Voronoi diagrams can be constructed as a special case of hig

Notes: Voronoi diagrams were studied by Dirichlet in 1850 and are sometimes referred to as Dirich

Two books \\cite{aurenhammer2013voronoi} \\cite{okabe2000spatial} offer a complete treatment of V

In a k th-order Voronoi diagram, we partition the plane such that each point in a region is clo

Related problems: Nearest-neighbor search (see page 637), point location (see page 644), triangul
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-645.jpg?height=593&width=124

%---- Page End Break Here ---- Page : 644
\section{Nearest-Neighbor Search}

Input description: A set S of n points in d dimensions, and query point q.
Problem description: Which point in S is closest to q ?
Discussion: The need to quickly find the nearest neighbor of a query point arises in ma

A nearest-neighbor search is also important in classification. Suppose we are given a c

Such nearest-neighbor classifiers are widely used, often in high-dimensional spaces. Th

Issues arising in nearest-neighbor search include:
- How many points are you searching? - When your data set contains only a small number
ever destined to be performed, the simple approach is best. Compare the query point q
- How many dimensions are you working in? - Nearest-neighbor search gets progressively
In two dimensions, Voronoi diagrams (see Section 20.4 (page 634) provide an efficient d
- Do you really need the exact nearest neighbor? - Finding the absolute nearest neighbo

%---- Page End Break Here ---- Page : 645
One important technique is dimension reduction. Projections exist that map any set of $
Another idea is adding randomness when you search your data structure. A $k d$-tree can
- Is your data set static or dynamic? - Will there be occasional insertions or deletion
rare events, it might pay to rebuild your data structure from scratch each time. But if

%---- Page End Break Here ---- Page : 646
The nearest-neighbor graph on a set S of n points links each vertex to its nearest

As a lower bound, the closest-pair problem in one dimension reduces to sorting. We only

Implementations: $A N N$ is a C++ library for both exact and approximate nearest-neighb

Annoy (Approximate Nearest Neighbors Oh Yeah) is a C ++ library with Python bindings cr

Samet's spatial index demos (http://donar.umiacs.umd.edu/quadtree/) provide a series of

Section 20.4 (page 634) gives a complete collection of Voronoi diagram implementations

Notes: Approximate nearest-neighbor search in high dimensions has been a very active ar

The theoretical guarantees underlying Arya and Mount's approximate nearestneighbor code
data structures hold promise for other problems in high-dimensional spaces. Nearestneig

%---- Page End Break Here ---- Page : 647

Samet \\cite{samet2006foundations}] is the best reference on kd-trees and other spatial data stru

Good expositions on finding the closest pair of points in the plane \\cite{bentley1976divide}] in

Related problems: Kd-trees (see page 460), Voronoi diagrams (see page 634, range search (see page
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-649.jpg?height=593&width=124

%---- Page End Break Here ---- Page : 648
\section{Range Search}

Input description: A set S of n points in d dimensions, and a query region Q.

Problem description: Which points in S lie within Q ?
Discussion: Range search problems arise in database and geographic information system (GIS) appli

The difficulty of range search depends on several factors:
- How many range queries will you perform? - The simplest approach to range search tests each of
- What is the shape of your query polygon? - The easiest regions to query against are axis-parall
When querying against a non-convex polygon, it pays to partition the polygon into convex pieces o
- How many dimensions? - A general approach to range queries builds a kd-tree on the point set, a
- Is your point set static, or might there be insertions/deletions? - A clever practical approach
One nice thing about this approach is that it is relatively easy to employ "edge-flip" operations
- Can I just count the number of points in a region, or must I identify them? - It often suffices
A nice data structure for efficiently answering such aggregate range queries is based on the domi
$$
m=D\left(x_{\max }, y_{\max }\right)-D\left(x_{\max }, y_{\min }\right)-D\left(x_{\min }, y_{\max
$$

%---- Page End Break Here ---- Page : 649
The last additive term corrects for the points for the lower left-hand corner that have been subt
We can partition the space into n^{2} rectangles by drawing a horizontal and vertical line thro
to binary search and thus take $O(\lg n)$ time. This data structure takes quadratic space, which

%---- Page End Break Here ---- Page : 650
Implementations: Both CGAL (www.cgal.org) and LEDA (see Section 22.1.1 (page 713p) use a dynamic
$A N N$ is a C++ library for both exact and approximate nearest-neighbor searching in arbitrarily

Annoy (Approximate Nearest Neighbors Oh Yeah) is a C++ library with Python bindings created by Sp

Notes: Good expositions on data structures with worst-case $O(\lg n+k)$ performance for orthogona

The problem becomes considerably more difficult for non-orthogonal range queries, where the query

Related problems: Kd-trees (see page 460, point location (see page 644.\index{mail rou
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-652.jpg?height=59

%---- Page End Break Here ---- Page : 651
\section{Point Location}

Input description: A decomposition of the plane into polygonal regions, and a query po

Problem description: Which region contains the query point q ?
Discussion: Point location is often needed as an ingredient to solve larger geometric p
- Is the query point inside or outside of polygon P? - The simplest version of point lo
- How many queries will be performed? - We could perform this insidepolygon test separa
better to construct a grid-like or tree-like data structure on top of our subdivision t
- How complicated are the regions of your subdivision? - More sophisticated inside-outs
- How regularly sized and spaced are your regions? - When all resulting triangles are r
Such grid files can perform very well, provided that each triangular region overlaps re
- How many dimensions will you be working in? - In three or more dimensions, some flavo

%---- Page End Break Here ---- Page : 652
Kd-trees, described in Section 15.6 (page 460, decompose space into a hierarchy of rect
- Am I close to the right cell? - Walking is a simple point-location technique that mig

%---- Page End Break Here ---- Page : 653
Proceeding to the neighboring cell through this face gets us one step closer to the ta

The simplest algorithm that guarantees $O(\lg n)$ worst-case access is the slab method

Point location methods that are efficient in the worst case tend to require a lot of me

Implementations: Both CGAL (www.cgal.org) and LEDA (see Section 22.1.1 (page 713) provi
$A N N$ is a C++ library for both exact and approximate nearest-neighbor searching in a

Arrange is a package for maintaining arrangements of polygons in either the plane or on

Routines (in C) to test whether a point lies in a simple polygon have been provided by

Notes: Snoeyink [\\cite{snoeyink2018point} gives an excellent survey of the state of th

Tamassia and Vismara \\cite{tamassia2001-algorithm-engineering} use planar point locati
point location is described in \\cite{edahiro1984point}. The winner was a bucketing tec

%---- Page End Break Here ---- Page : 654

Kirkpatrick's elegant triangle refinement method \\cite{kirkpatrick1983optimal}] builds a hierarc

More recently, there has been interest in dynamic data structures for point location, which suppo

Related problems: Kd-trees (see page 460), Voronoi diagrams (see page 634), nearest-neighbor sear
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-656.jpg?height=595&width=124

%---- Page End Break Here ---- Page : 655
\section{Intersection Detection}

Input description: A set S of lines and line segments $l_{1}, \ldots, l_{n}$, or a pair of poly

Problem description: Which pairs of line segments intersect each other? What is the intersection

Discussion: Intersection detection is a fundamental geometric primitive with many applications. F

Another application arises in design rule checking for integrated circuit layout. A minor design

Issues arising in intersection detection include:
- Do you want to compute the intersection or just report it? - We distinguish between intersectio
- Are you intersecting lines or line segments? - The big difference here is that any two lines wi
lines provides more information than just the intersection points, and is discussed in Section 20
Finding all the intersections between n line segments proves considerably more challenging. Eve
- How many intersection points do you expect? - In integrated circuit design-rule checking, we ex
Such output-sensitive algorithms exist for line segment intersection. The fastest algorithm takes
- Can you see point x from point y ? - Visibility queries ask whether vertex x has an unobs
- Are the intersecting objects convex? - Better intersection algorithms exist when the line segme

%---- Page End Break Here ---- Page : 656
However, non-convex polygons are not so well behaved. Consider the intersection of two "combs" ge
Intersecting polyhedra is more complicated than polygons, because two polyhedra can intersect eve
- Do the objects move? - In the walk-through application just described, the room and the objects

%---- Page End Break Here ---- Page : 657
One common technique is to approximate the objects in a scene by simpler objects that enclose the

Planar sweep algorithms can be used to efficiently compute the intersections among a set of line
- Insertion - The left-most point of a line segment may be encountered, which now becomes availab
- Deletion - The right-most point of a line segment is encountered, which means that we have comp
- Intersection - If we maintain the active line segments that intersect the sweep line as sorted

Keeping track of what is going on requires two data structures. The future is maintained by an ev

To compute the intersection or union of polygons, we modify the processing of the thre

Implementations: Both LEDA (see Section 22.1.1 (page 713)) and CGAL (www.cgal.org) off

%---- Page End Break Here ---- Page : 658
O'Rourke O'R01 provides a robust program in C to compute the intersection of two conve

Finding the mutual intersection of a collection of half-spaces is a special case of th

Notes: Mount \\cite{mount2018geometric} is an excellent survey of algorithms for compu

An optimal $O(n \lg n+k)$ algorithm for computing line segment intersections is due to

Lin et al. \\cite{lin2018collision}] survey techniques and software for collision dete

Related problems: Maintaining arrangements (see page 671), motion planning (see page 6
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-660.jpg?height=59

%---- Page End Break Here ---- Page : 659
\section{Bin Packing}

Input description: A set of n items with sizes $d_{1}, \ldots, d_{n}$. A set of m

Problem description: Store all the items using the fewest number of bins.
Discussion: Bin packing arises in many packaging and manufacturing problems. Suppose t

Even the most elementary-sounding bin-packing problems are NP-complete; see the discus
- What are the shapes and sizes of the objects? - The character of a binpacking proble

%---- Page End Break Here ---- Page : 660
When all the boxes are of identical size and shape, repeatedly filling each row gives
- Are there constraints on the orientation and placement of objects? - Many shipping b
- Is the problem on-line or off-line? - Do we know the complete set of objects to pack

The standard off-line heuristics for bin packing order the objects by size or shape an

Analytical and empirical results suggest that first-fit decreasing is the best heurist

First-fit decreasing is easily implemented in $O(n \lg n+b n)$ time, where $b \leq$ $\

We can fiddle with the insertion order in such a scheme to deal with problemspecific c

Packing boxes is much easier than packing arbitrary geometric shapes, enough so that o the upper, lower, left, and right tangents in a given orientation. Finding the orientation that m

%---- Page End Break Here ---- Page : 661
In the case of non-convex parts, considerable useful space can be wasted in the holes created by

Implementations: BPPLIB (http://or.dei.unibo.it/library/bpplib) is an extensive collection of bin

Martello and Toth's collection of Fortran implementations of algorithms for a variety of knapsack

A first step towards packing arbitrary shapes packs each in its own minimum volume box. For a cod
html. This algorithm runs in near-linear time [\\cite{barequet2001efficiently}].
Notes: See CJCG $^{+} 13$, \\cite{christensen2017survey}, \\cite{delorme2016bin} WHS07 for survey

Efficient algorithms are known for finding the largest empty rectangle in a polygon \\cite{daniel

Sphere packing is an important and well-studied special case of bin packing, with applications to

Milenkovic has worked extensively on two-dimensional bin-packing problems for the apparel industr

Related problems: Knapsack problem (see page 497), set packing (see page 682 .
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-663.jpg?height=622&width=124

%---- Page End Break Here ---- Page : 662
\section{Medial-Axis Transform}

Input description: A polygon or polyhedron P.
Problem description: Find the skeleton of P, the set of points that have more than one closest

Discussion: The medial-axis transformation is useful in thinning a polygon, or equivalently findi

The medial-axis transformation of a polygon is always a tree, making it fairly easy to use dynami

There are two distinct approaches to computing medial-axis transforms, depending upon whether you
- Geometric data - Recall that the Voronoi diagram of a point set S (see Section 20.4 (page 634
segment $l_{i} \in L$ such that all points within the region around l_{i} are closer to l_{i}

%---- Page End Break Here ---- Page : 663
Polygons are defined by line segments, such that each segment l_{i} shares a vertex with neighb

The straight skeleton is a structure related to the medial-axis transform of a polygon, except th
- Image data - Digitized images can be interpreted as points sitting at the lattice points on an
A direct pixel-based approach for constructing a skeleton implements the "brush fire" view of thi

Algorithms that explicitly manipulate pixels tend to be easy to implement, because they

Implementations: CGAL (www.cgal.org) includes a package for computing the straight skel
structing offset contours defining the polygonal regions within P whose points are a

%---- Page End Break Here ---- Page : 664
VRONI \\cite{held2001vroni} is a robust and efficient program for computing Voronoi dia

Programs that reconstruct or interpolate point clouds often are based on medial-axis t

Notes: The book by Siddiqi and Pizer [\\cite{siddiqi2008medial}] offers a comprehensive

The medial-axis of a polygon can be computed in $O(n \lg n)$ time for arbitrary n-gon

Straight skeletons were introduced in \\cite{aichholzer1995novel}, with a subquadratic

Related problems: Voronoi diagrams (see page 634), Minkowski sum (see page 674 .\index
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-666.jpg?height=55

%---- Page End Break Here ---- Page : 665
\section{Polygon Partitioning}

Input description: A polygon or polyhedron P.
Problem description: Partition P into a small number of simple (typically convex) pi

Discussion: Partitioning is an important pre-processing step for many geometric algori

Carving a nation into states or counties or districts is a classical problem in polygo

Several flavors of polygon partitioning arise, depending upon the particular applicati
- Should all the pieces be triangles? - Triangulation is the mother of all polygon par

Every triangulation of an n-vertex polygon contains exactly $n-2$ triangles. Thus, t
- Do I want to cover or partition my polygon? - Partitioning a polygon means complete
- Am I allowed to add extra vertices? - A final issue is whether we are allowed to add

%---- Page End Break Here ---- Page : 666
The Hertel-Mehlhorn heuristic for convex decomposition using diagonals is simple and e

I recommend using this heuristic unless it is critical for you to absolutely minimize

Dynamic programming can be employed to find the absolute minimum number of diagonals u

An alternate decomposition problem partitions polygons into monotone pieces. The vertices of a $y

Implementations: Many triangulation codes start by finding a trapezoidal or monotone decompositio
of convex decomposition. Check out the codes in Section 20.3 (page 630) as a starting point.

%---- Page End Break Here ---- Page : 667
CGAL (www.cgal.org) contains a polygon-partitioning library that includes (1) the Hertel-Mehlhorn
(2) finding an optimal convex partitioning using the $O\left(n^{4}\right)$ dynamic programming al

A triangulation code of particular relevance here is GEOMPACK - a suite of Fortran 77 and $\mathr

Notes: Survey articles on polygon partitioning include \\cite{keil2000polygon} \\cite{orourke2018

Art gallery problems are an interesting topic related to polygon covering, where we seek to posit

Related problems: Triangulation (see page 630), set cover (see page 678).
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-669.jpg?height=527&width=123

%---- Page End Break Here ---- Page : 668
\section{Simplifying Polygons}

Input description: A polygon or polyhedron p, with n vertices.
Problem description: Find a polygon or polyhedron $p^{\prime}$ containing only $n^{\prime}$ verti

Discussion: Polygon simplification has two primary applications. The first involves cleaning up a

Several issues arise in shape simplification:
- Do you want the convex hull? - The simplest simplification is the convex hull of the object's v
- Am I allowed to insert points, or just delete them? - The typical goal of simplification is to

%---- Page End Break Here ---- Page : 669
Methods that only delete vertices quickly melt shapes into unrecognizability, however. More robus
- Must the resulting polygon be intersection-free? - A serious drawback of incremental procedures
An approach to polygon simplification that guarantees a simple approximation involves computing m
This link distance approach "fattens" the boundary of the polygon by some acceptable error window
- Are you given an image to clean up, instead of a polygon to simplify? The conventional approach

The Douglas-Peucker algorithm for shape simplification starts with a simple approximation and the

Simplification becomes considerably more difficult in three dimensions. Indeed, it is NP-complete

%---- Page End Break Here ---- Page : 670

```
Implementations: The Douglas-Peucker algorithm is readily implemented. For a C implemen

QSlim is a quadric-based simplification algorithm that can produce highquality approxin

Yet another approach to polygonal simplification is based on simplifying and expanding

CGAL (www.cgal.org) provides support for polyline simplification, as well as the most e

Notes: The Douglas-Peucker incremental refinement algorithm \\cite{douglas1973algorithm

Heckbert and Garland \\cite{heckbert1997survey} survey algorithms for shape simplificat

Testing whether a polygon is simple can be performed in linear time, at least in theory

Related problems: Thinning (see page 655), convex hull (see page 626.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-672.jpg?height=584


%---- Page End Break Here ---- Page : 671
\section{Shape Similarity}

Input description: Two polygonal shapes, $P_{1}$ and $P_{2}$.
Problem description: How similar are $P_{1}$ and $P_{2}$ ?
Discussion: Shape similarity is a problem that underlies much of pattern recognition. C

Shape similarity is an inherently ill-defined problem, because what "similar" means is

Among your possible approaches are:
- Hamming distance - Suppose that your two polygons have been properly registered, mean

Computing the area of the symmetric difference reduces to finding the intersection or u
are inherently aligned within lines on the page and are not free to rotate. Efficient a


%---- Page End Break Here ---- Page : 672
Hamming distance is particularly simple and efficient to compute on bitmapped images, s
- Hausdorff distance - An alternative similarity measure (post-registration) is Hausdor

Which is better, Hamming or Hausdorff? It depends upon your application. As with Hammin
- Comparing Skeletons - A more powerful approach to shape similarity uses thinning (see
- Machine learning techniques - A final approach for pattern recognition problems uses
Machine learning proves successful when you have a lot of data to train on and no probl
then takes your training data and produces a classification function, which accepts as
How good are the resulting classifiers? It depends upon the application. Machine learni
```

```
%---- Page End Break Here ---- Page : 673
Implementations: The computational geometry library CGAL (https:// www.cgal.org/) contains a vari

Several excellent support vector machine classifiers are available. These include Python's scikit

Notes: Veltkamp \\cite{veltkamp2001shape} is an excellent survey on shape matching from a computa

The optimal alignment of $n$ and $m$-vertex convex polygons subject to translation (but not rotat

A linear-time algorithm for computing the Hausdorff distance between two convex polygons is given

Related problems: Graph isomorphism (see page 610), thinning (see page 655.\index{motion planning
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-675.jpg?height=591&width=124


%---- Page End Break Here ---- Page : 674
\section{Motion Planning}

Input description: A polygonal-shaped robot starting at position $s$ in a room containing polygon

Problem description: Find the shortest route taking $s$ to $t$ without intersecting any obstacles

Discussion: That motion planning is a complex problem is obvious to anyone who has tried to move

Finally, motion planning provides a tool for computer animation and virtual reality. Given the se

Many factors govern the complexity of motion-planning problems:\index{convolution  polygon}\index
- Is your robot a point? - For point robots, motion planning reduces to finding the shortest path
This visibility graph can be constructed by testing each of the $\binom{n}{2}$ vertexpair edge ca
although faster algorithms are known. Assign each edge of this visibility graph with weight equal
- What motions can your robot perform? - Motion planning becomes considerably more difficult when


%---- Page End Break Here ---- Page : 675
The algorithmic complexity depends upon the number of degrees of freedom that the robot can use t
- Can you simplify the shape of your robot? - Motion planning algorithms tend to be complex and t
- Are motions limited to translation only? - When rotation is not allowed, the expanded obstacles
- Are the obstacles known in advance? - We have assumed that the robot starts out with a map of i
obstacle until the robot is again free to proceed directly towards the goal. Unfortunately, this


%---- Page End Break Here ---- Page : 676
The most practical approach to general motion planning involves randomly sampling the configurati

Construct a set of legal configuration-space points by random sampling. For each pair of points $
```

There are many ways to enhance this basic technique, such as adding additional vertices

Implementations: State-of-the-art sampling-based motion planning algorithms are availab

The Motion Planning Toolkit (MPK) is a C++ library and toolkit for developing single- a

The computational geometry library CGAL (www.cgal.org) contains many algorithms related

Notes: Latombe's book \\cite{latombe1991robot} describes practical approaches to motion

Motion planning was originally studied by Schwartz and Sharir as the "piano mover's pro

The best general result for this free-space approach to motion planning is due to Canny
solved in $O\left(n^{d} \lg n\right)$, although faster algorithms exist for special cas

%---- Page End Break Here ---- Page : 677
The time complexity of algorithms based on the free-space approach to motion planning d

The visibility graph of n line segments with E pairs of visible vertices can be con

Related problems: Shortest path (see page 554), Minkowski sum (see page 674.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-679.jpg?height=595

%---- Page End Break Here ---- Page : 678
\section{Maintaining Line Arrangements}

Input description: A set of lines $l_{1}, \ldots, l_{n}$.
Problem description: What is the decomposition of the plane defined by $l_{1}, \ldots,

Discussion: A fundamental problem in computational geometry is explicitly constructing
- Degeneracy testing - Given a set of n lines in the plane, do any three lines pass t
- Satisfying the maximum number of linear constraints - Suppose that we are given a set

Thinking of geometric problems in terms of features in an arrangement can be very usefu
- What is the right way to construct a line arrangement? - Algorithms for constructing
- How big will your arrangement be? - A geometric fact called the zone theorem implies
- What do you want to do with your arrangement? - Given an arrangement and a query poin
- Does your input consist of points instead of lines? - Although lines and points seem
$$
L: y=2 a x-b \leftrightarrow p:(a, b)
$$

%---- Page End Break Here ---- Page : 679

Duality is important because we can now apply line arrangements to point problems, often with sur
For example, suppose we are given a set of n points, and we want to know whether any three of t

It often becomes useful to traverse each face of an existing arrangement exactly once. Such trave

Implementations: CGAL (www.cgal.org) provides a generic and robust package for arrangements of cu
starting point for any serious project using arrangements. A recent book by Fogel et al. \\cite{r

%---- Page End Break Here ---- Page : 680
A robust code for constructing and topologically sweeping an arrangement in C++ is provided at ht

Arrange is a package for maintaining arrangements of polygons in either the plane or on the spher
Notes: Edelsbrunner \\cite{edelsbrunner1987algorithms} provides a comprehensive treatment of the

Arrangements generalize naturally beyond two dimensions. Instead of lines, the space decompositio

The history of the zone theorem has become somewhat muddled, because the original proofs were lat

The naive algorithm for sweeping an arrangement of lines sorts the n^{2} intersection points by

Related problems: Intersection detection (see page 648), point location (see page 644.
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-682.jpg?height=586&width=123

%---- Page End Break Here ---- Page : 681
\section{Minkowski Sum}

Input description: Point sets or polygons A and B, containing n and m vertices respective

Problem description: What is the convolution of A and B, that is, the Minkowski sum $A+B=\{x+

Discussion: Minkowski sums are useful geometric operations that fatten up objects in appropriate

The definition of a Minkowski sum assumes that the polygons A and B have been positioned on a
$$
A+B=\{x+y \mid x \in A, y \in B\}
$$
where $x+y$ is the vector sum of two points. Thinking of this in terms of translation, the Minkow
- Are your objects rasterized images or explicit polygons? - The definition of Minkowski summatio
of points in A and B, sum up their coordinates and darken the appropriate pixel. These algori
- Do you want to fatten your object by a fixed amount? - The most common fattening operation expa
- Are your objects convex? - The complexity of computing Minkowski sums depends in a serious way

%---- Page End Break Here ---- Page : 682

A straightforward approach to computing the Minkowski sum is based on triangulation and

Computing the Minkowski sum of two convex polygons is easier than the general case, bec

Implementations: The CGAL (www.cgal.org) Minkowski sum package provides an efficient an

An implementation for computing the Minkowski sums of two convex polyhedra in three dim
Notes: Good expositions on algorithms for Minkowski sums include \\cite{deberg2008compu

The practical efficiency of Minkowski sum in the general case depends upon how the poly
sarily the partition with the fewest number of convex pieces. Agarwal et al. \\cite{aga

%---- Page End Break Here ---- Page : 683
The combinatorial complexity of the Minkowski sum of two convex polyhedra in three dime

Related problems: Thinning (see page 655), motion planning (see page 667), simplifying

%---- Page End Break Here ---- Page : 684
Chapter 21

\chapter{Set and String Problems}

Sets and strings both represent collections of objects - the difference is whether orde

The assumption of a fixed order makes it possible to solve string problems much more ef
- Gusfield \\cite{gusfield1997algorithms} - To my taste, this remains the best introduc
- Crochemore, Hancart, and Lecroq \\cite{crochemore2007strings} - A comprehensive treat
- Navarro and Raffinot \\cite{navarro2007flexible} - A concise but practical and implem
- Crochemore and Rytter \\cite{crochemore2003jewels} - A survey of specialized topics i

Theoreticians working in string algorithmics sometimes refer to their field as stringol
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-686.jpg?height=59

\section{Set Cover}

Input description: A collection of subsets $S=\left\{S_{1}, \ldots, S_{m}\right\}$ of t

Problem description: What is the smallest subset T of S whose union equals the univ
$$
\cup_{i=1}^{|T|} T_{i}=U ?
$$

Discussion: Set cover arises when you try to efficiently acquire items that have been p

Boolean logic minimization is another interesting application of set cover. We are given a specif

There are several types of set cover problems you should be aware of:
- Are you allowed to cover elements more than once? - The distinction here is between set cover a
- Are your sets derived from the edges or vertices of a graph? - Set cover is a very general prob

%---- Page End Break Here ---- Page : 686
It is instructive to see how to model vertex cover as an instance of set cover. Let the universal
- Do your subsets contain only two elements each? - You are in luck if all of your subsets have a
- Do you want to cover elements with sets, or sets with elements? - In the hitting set problem, w
Hitting set is dual to set cover, meaning that it is exactly the same problem in disguise. Replac

Set cover is at least as hard as vertex cover, so it is also NP-complete. In fact, it is somewhat
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-688.jpg?height=435&width=977

%---- Page End Break Here ---- Page : 687\index{animation sorting!687}
Figure 21.1: A hitting set instance optimally solved by selecting elements 1 and 3 , or 2 and 3 (
optimal for vertex cover, but the best approximation algorithm for set cover is $\Theta(\lg n)$ t

Greedy is the most natural and effective heuristic for set cover. Begin by selecting the largest

The simplest implementation of the greedy heuristic sweeps through the entire input instance of $

It pays to check whether there exist elements that occur in only a few subsets - ideally just one

Simulated annealing techniques on top of such greedy heuristics is likely to produce somewhat bet

An often more powerful approach rests on the integer linear programming (ILP) formulation of set
$$
\sum_{x \in S_{i}} s_{i} \geq 1
$$

This ensures that x will be covered by at least one selected subset. The minimum
set cover satisfies all of these constraints while minimizing $\sum_{i} s_{i}$. This integer prog

%---- Page End Break Here ---- Page : 688
Implementations: Both the greedy heuristic and the integer linear programming formulation above a

Pascal implementations of an exhaustive search algorithm for set packing, as well as heuristics f

SYMPHONY is a mixed-integer linear programming solver that includes a set partitioning solver. It

Notes: An old but classic survey article on set cover is \\cite{balas1976set}, with more recent a

```
Good expositions of the greedy heuristic for set cover include [\\cite{cormen2009introd

Knuth's volume 4A \\cite{knuth2011taocp4a} contains a fascinating discussion of Boolea

Related problems: Matching (see page 562), vertex cover (see page 591), set packing (s
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-690.jpg?height=59


%---- Page End Break Here ---- Page : 689
\section{Set Packing}

Input description: A set of subsets $S=\left\{S_{1}, \ldots, S_{m}\right\}$ of the uni

Problem description: Select a small collection of mutually disjoint subsets from $S$ w

Discussion: Set packing problems arise in applications where we have strong constraint

Some flavor of this is captured by the independent set problem in graphs, discussed in

Scheduling airline flight crews is another application of set packing. Each airplane i


%---- Page End Break Here ---- Page : 690
We use set packing here to represent several different problems on sets, all of which
- Must every element appear in exactly one selected subset? - In the exact cover probl
Unfortunately, exact cover puts us in the same situation as the Hamiltonian cycle on g
- Does each element have its own singleton set? - Things will be far better if we can
- What is the penalty for covering elements twice? - In set cover (see Section 21.1 (p

The right heuristics for set packing are greedy, and similar to those of set cover (se

A more powerful approach rests on an integer programming formulation akin to that of s
$$
\sum_{x \in S_{i}} s_{i}=1
$$

This ensures that $x$ is covered by exactly one selected subset. Minimizing or maximizi

Implementations: Since set cover is a more popular and more tractable problem than set
to solve the cover problem. Such implementations discussed in Section 21.1 (page 678)


%---- Page End Break Here ---- Page : 691
Pascal implementations of an exhaustive search algorithm for set packing, as well as h
```

SYMPHONY is a mixed-integer linear programming solver that includes a set partitioning solver. It

Notes: Survey articles on set packing include \\cite{balas1976set} \\cite{hoffman2009set} \\cite{

Set-packing relaxations for integer programs are presented in \\cite{borndorfer2000set}. An excel

Related problems: Independent set (see page 589), set cover (see page 678).
"I often repeat repeat myself, I often repeat repeat. I often repeat repeat myself, I often repea
"I often repeat repeat myself, I often repeat repeat.
I often repeat repeat myself,
I often repeat repeat."
- Jack Prelutsky, A Pizza the Size of the Sun

%---- Page End Break Here ---- Page : 692
Input
Output

\section{String Matching}

Input description: A text string t of length n. A pattern p of length m.
Problem description: Find an/all instances of pattern p in the text.
Discussion: String matching arises in almost all text processing applications. Every text editor

String matching is a fundamental algorithmic problem that remains surprisingly active. Several is
- Are your search patterns and/or texts short? - If your strings are short and your queries infre
For very short patterns (say $m \leq 10$), you can't hope to beat this simple algorithm by much,
- What about longer texts and patterns? - String matching can in fact be performed in worst-case
the text in position $i+1$. The Knuth-Morris-Pratt (KMP) algorithm preprocesses the search patter
- Do I expect to find the pattern, or not? - The Boyer-Moore algorithm matches the pattern agains

%---- Page End Break Here ---- Page : 693
Although somewhat more complicated than Knuth-Morris-Pratt, this is worthwhile in practice for pa
- Will you perform multiple queries on the same text? - Suppose you are building a program to rep
- Will you search many texts using the same patterns? - Suppose you are building a program to scr
Performing a linear-time scan for each pattern yields an $O(k(m+n))$ algorithm. But if k is lar

Sometimes multiple patterns are specified not as a list of strings, but concisely as a regular ex

%---- Page End Break Here ---- Page : 694
When the patterns get specified by context-free grammars instead of regular expressions, the prob
- What if our text or pattern contains a spelling error? - The algorithms discussed here work onl

Implementations: Strmat is a collection of C programs implementing exact pattern matching algorit

```
Several versions of the general regular expression pattern matcher (grep) are readily a

The Boost string algorithms library provides C++ routines for basic operations on strin

Notes: All books on string algorithms contain thorough discussions of exact string matc

Aho \\cite{aho1990algorithms} provides a good survey on algorithms for pattern matching

Empirical comparisons of string matching algorithms include \\cite{davies1986algorithms

The Rabin-Karp algorithm \\cite{karp1987randomized}] uses a hash function to perform st

Related problems: Suffix trees (see page 448), approximate string matching (see page 68
\begin{tabular}{|c|c|c|c|c|c|c|c|c|c|}
\hline \multirow{3}{*}{amispelt} & \multirow[t]{3}{*}{\begin{tabular}{l}
misplaced \\
%---- Page End Break Here ---- Page : 695

? misspelled misprinted
\end{tabular}} & \multicolumn{8}{|l|}{\multirow[t]{3}{*}{a m i s p el t}} \\
\hline & & & & & & & & & \\
\hline & & & & & & & & & \\
\hline \multicolumn{10}{|c|}{Input Output} \\
\hline
\end{tabular}

\section{Approximate String Matching}

Input description: A text string $t$ and a pattern string $p$.
Problem description: What is the minimum-cost way to transform $t$ to $p$ using inserti

Discussion: Approximate string matching is important because we live in an error-prone

I once encountered approximate string matching when evaluating the performance of an op

When no changes are permitted, our problem reduces to exact string matching, which is c

Dynamic programming provides the basic approach to approximate string matching. Let $D
- If $p_{i}=t_{j}$ then $D[i-1, j-1]$ else $D[i-1, j-1]+$ substitution cost.
- $D[i-1, j]+$ deletion cost of $p_{i}$.
- $D[i, j-1]+$ deletion cost of $t_{j}$.


%---- Page End Break Here ---- Page : 696
A general implementation in C and more complete discussion appears in Section 10.2 (pag
```

```
- Do I match the pattern against the full text, or against a substring? The boundary condition of
But now suppose that the pattern can occur anywhere within the text. The proper cost of $D[0, j]$
- How should I select the substitution and insertion/deletion costs? - The edit distance algorith
The default choice charges the same for each insertion, deletion, or substitution. Charging a sub
- How do I find the actual alignment of the strings? - The recurrence above only yields the cost
- What if the two strings are very similar to each other? - The dynamic programming algorithm abo
of $O(d n)$ cells within a distance $d$ of the central diagonal. If no low-cost alignment exists
- Is your pattern short or long? - A recent approach to string matching exploits the fact that mo
The basic idea is quite clever. Construct a bit-mask $B_{\alpha}$ for each letter $\alpha$ of the
The agrep program, discussed below, uses such a bit-parallel algorithm generalized to approximate
- How do I minimize the required storage? - The quadratic space needed to store the dynamic progr
But we can use Hirschberg's clever recursive algorithm to efficiently recover the optimal alignme
- Should I score long runs of insertions/deletions differently? - Many string matching applicatio
String matching with gap penalties provides a way to properly account for such changes. Typically
the per-character deletion cost. If $A$ is large relative to $B$, the alignment has an incentive
String matching under such affine gap penalties can be done in the same quadratic time as regular
$$
\begin{aligned}
& V(i, j)=\max (E(i, j), F(i, j), G(i, j)) \\
%---- Page End Break Here ---- Page : 697\index{nal examination}\index{integer factorization}\ind

%---- Page End Break Here ---- Page : 698

& G(i, j)=V(i-1, j-1)+\text { match }(i, j) \\
& E(i, j)=\max (E(i, j-1), V(i, j-1)-A)-B \\
& F(i, j)=\max (F(i-1, j), V(i-1, j)-A)-B
\end{aligned}
$$

With a constant amount of work per cell, this algorithm takes $O(m n)$ time, same as without gap
- Does similarity mean strings that sound alike? - Other models of approximate pattern matching b

Soundex drops vowels and silent letters, removes doubled letters, and then assigns the remaining

Implementations: Several excellent software tools are available for approximate pattern matching.
$T R E$ is a general regular-expression matching library for exact and approximate matching, whic


%---- Page End Break Here ---- Page : 699

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{algorithms_page_0701_0.png}
\end{figure}
Wikipedia gives programs for computing edit (Levenshtein) distance in a dizzying array of languag
```

Notes: There have been many recent advances in approximate string matching, particularl

The basic dynamic programming alignment algorithm is attributed to WF74, although it is

Masek and Paterson \\cite{masek1980faster} compute the edit distance between m - and

The shortest-path formulation leads to a variety of algorithms that are good when the e

Bit-parallel algorithms for approximate matching include Myers's \\cite{myers1999fast}

Soundex was invented and patented by M. K. Odell and R. C. Russell. Expositions on Soun

Related problems: String matching (see page 685), longest common substring (see page 70

%---- Page End Break Here ---- Page : 700
Fourscore and seven years ago our father brought forth on this continent a new nation c

Input

Output

\section{Text Compression}

Input description: A text string S.
Problem description: Create a shorter text string $S^{\prime}$ such that S can be cor

Discussion: Secondary storage devices quickly fill up on most computer systems, even th

People seem to like inventing ad hoc data-compression methods for their particular appl
- Must we recover the exact input text after compression? - Lossy vs. lossless encoding
- Can I simplify my data before I compress it? - The most effective way to free space o
from the file? Might the document be converted entirely to uppercase characters, or hav
A particularly interesting simplification results from applying the BurrowsWheeler tran

%---- Page End Break Here ---- Page : 701
Provided the last character of the input string is unique (an end-of-string symbol), th
- Does it matter whether the algorithm is patented? - Certain data compression algorith
- How do I compress image data - The simplest lossless compression algorithm for image
For serious audio/image/video compression applications, I recommend that you use a popu
- Must compression run in real time? - Fast decompression is often more important than

Literally dozens of text compression algorithms are available, but they can be classifi
build a single coding table by analyzing the entire document. Adaptive algorithms, such
- Huffman codes - Huffman codes replace each alphabet symbol by a variablelength code s

Huffman codes can be constructed using a greedy algorithm. Sort the symbols in increasing order b
Huffman codes are popular but have three disadvantages. Two passes must be made over the document
- Lempel-Ziv algorithms - Lempel-Ziv algorithms (including the popular LZW variant) compress text
Lempel-Ziv algorithms build coding tables of frequent substrings, which can get arbitrarily long.

%---- Page End Break Here ---- Page : 702
The amazing thing about Lempel-Ziv is how robust it is on different types of data. It is quite di

%---- Page End Break Here ---- Page : 703
Implementations: Perhaps the most popular text compression program is gzip, which implements a pu

There is a natural tradeoff between compression ratio and compression time. Another choice is bzi

Notes: Many books on data compression are available. Recent and comprehensive books include Sayoo

Good expositions on Huffman codes \\cite{huffman1952method} include \\cite{aho1983data} \\cite{co

The annual IEEE Data Compression Conference (https://www.cs.brandeis.edu/ $\sim \mathrm{dcc} /)$

Related problems: Shortest common superstring (see page 709, cryptography (see page 697).

%---- Page End Break Here ---- Page : 704
\title{
The magic words are Squeamish Ossifrage.
}

15\&AE<\&UA9VEC' $=0$
<F1s"F\%R92!3<75E96UI<V
V@*3W-S:69R86=E+@K_
Input
Output

\section{Cryptography}

Input description: A plaintext message T or encrypted text E, and key k.
Problem description: Encode T using k giving E, or decode E giving T.
Discussion: Cryptography has grown wildly more important as computer networks increasingly make c

Cryptographic ideas and applications go far beyond the tasks of "encryption" and "decryption." Th

There are three classes of cryptosystems everyone should be aware of:
- Caesar shifts - The oldest ciphers involve mapping each character of the alphabet to a differen
- Block shuffle ciphers - This class of algorithms repeatedly shuffles the bits of your text as g

%---- Page End Break Here ---- Page : 705
But 256-bit AES is still considered very secure. AES libraries are available for all ma
- Public key cryptography - If you fear bad guys reading your messages, you should als
$R S A$ is the classic example of a public key cryptosystem, named after its inventors
RSA is slow relative to other cryptosystems - roughly 100 to 1,000 times slower than AE

The critical issue in selecting a cryptosystem is identifying your paranoia level. Who

That said, I will confess that I use DES to encrypt my final exam each semester. It pro

Most symmetric key encryption mechanisms are harder to crack than public key ones for t

%---- Page End Break Here ---- Page : 706
Simple ciphers like the Caesar shift are fun and easy to program. For this reason, it i

Another thing you should never do is try to develop your own novel cryptosystem. The se

Certain other problems related to cryptography arise often in practice:
- How can I validate the integrity of data against random corruption? There is often a
A more efficient method uses a checksum, a hash of the long text down to a large intege
Cyclic-redundancy check (CRC) - The CRC provides a more powerful method to compute chec
- How can I validate the integrity of data against deliberate corruption? CRC is good a
- How can I prove that a file has not been changed? - If I send you a contract in elec
claiming that your version is what we had agreed to? I need a way to prove that any mod
I can compute a checksum for any given file, and then encrypt this checksum using my ow
- How can I restrict access to copyrighted material? - An important application for cry
High-speed cryptosystems have proven to be relatively easy to break. The state-of-the-a

%---- Page End Break Here ---- Page : 707
Implementations: Nettle is a comprehensive low-level cryptographic library in C. Crypto

Crypto ++ is a large C ++ class library of cryptographic schemes, including all that I

Many popular open source utilities employ serious cryptography, and serve as good model

The Boost CRC Library provides multiple implementations of cyclic redundancy check algo

Notes: The Handbook of Applied Cryptography \\cite{menezes1996handbook} provides techni

%---- Page End Break Here ---- Page : 708
Kahn \\cite{kahn1967code} presents the fascinating history of cryptography from ancient

```
Expositions on the RSA algorithm \\cite{rivest1978method} include \\cite{cormen2009introduction}.

Of course, the National Security Agency is the place to go to learn the real state of the art in

MD5 \\cite{rivest1992md5} is the hashing function used by PGP to compute digital signatures. Expo

Related problems: Factoring and primality testing (see page 490), text compression (see page 693)
![](https://cdn.mathpix.com/cropped/2025_03_24_35eff77eaa66f5ad98acg-710.jpg?height=595&width=124


%---- Page End Break Here ---- Page : 709
\section{Finite State Machine Minimization}

Input description: A deterministic finite automaton $M$.
Problem description: Create the smallest deterministic finite automaton $M^{\prime}$ such that $M

Discussion: Finite state machines are very useful for specifying and recognizing patterns. Modern

Finite state machines are defined by directed graphs. Each vertex represents a state, and each ch

Finite state machines are often used to specify search patterns in the guise of regular expressio


%---- Page End Break Here ---- Page : 710
We consider three different problems on finite automata:
- Minimizing deterministic finite state machines - Transition matrices for finite automata can be
Algorithms to minimize the number of states in a deterministic finite automaton (DFA) appear in a
This algorithm makes a sweep through all the classes looking for a new partition, and repeats the
- Constructing deterministic machines from non-deterministic machines DFAs are simple to work wit
In fact, any NFA can be mechanically converted to an equivalent DFA, which can then be minimized
The proofs of equivalence between NFAs, DFAs, and regular expressions are elementary enough to be
- Constructing machines from regular expressions - There are two approaches for translating a reg
The non-deterministic construction uses $\epsilon$-moves, which are optional transitions that req
construct an automaton using $\epsilon$-moves, from a depth-first traversal of the parse tree of
The deterministic construction starts with the parse tree for the regular expression, observing t


%---- Page End Break Here ---- Page : 711
Implementations: Grail+ is a C++ package for symbolic computation with finite automata and regula

The OpenFst Library (http://www.openfst.org/) is a library for constructing, combining, optimizin

JFLAP (Java Formal Languages and Automata Package) is a package of graphical tools for learning t

Notes: Aho \\cite{aho1990algorithms} provides a good survey on algorithms for pattern matching, w
```

Hopcroft \\cite{hopcroft1971n} gave an optimal $O(n \lg n)$ algorithm for minimizing t
problems of compressing a DFA to a minimum NFA [\\cite{jiang1993minimal}] and testing

%---- Page End Break Here ---- Page : 712
The inefficiencies of algorithmic approaches for regular expression pattern matching ha

Related problems: Satisfiability (see page 537). string matching (see page 685.

%---- Page End Break Here ---- Page : 713
AHAAIGDDETNWORTDTS
HGITGDDEANWTOSRDSG
GTASHIDDENWORDTGAG
HIGSDDEGNWORGADSTA
HAIDAGDENSWORSADTS

AHAAIGDDETNWORTDTS
HGITGDDEANWTOSRDSG
GTASHIDDENWORDTGAG HIGSDDEGNWORGADSTA

HAIDAGDENSWORSADTS

Input

\section{Longest Common Substring/Subsequence}

Input description: A set S of strings $S_{1}, \ldots, S_{n}$.
Problem description: What is the longest string $S^{\prime}$ such that all the charact

Discussion: Longest common substring/subsequence (LCS) arises whenever we search for s

The longest common subsequence problem for two strings is a special case of edit dista

Issues arising in LCS include:
- Are you looking for a common substring? - When detecting plagiarism, we seek to find
The longest common substring of a set of strings can be identified in linear time, as
- Are you looking for a common scattered subsequence? - For the rest of this section w

%---- Page End Break Here ---- Page : 714
Let $M[i, j]$ denote the number of characters in the longest common subsequence of $S[
This recurrence computes the length of the longest common subsequence in $O(n m)$ time
- What if the strings are permutations? - Permutations are strings without repeating c
- What if there are relatively few sets of matching characters? - There is a faster al

```
The complete set of such points can be found in $O(n+m+r)$ time using bucketing techniques. We cr

A common subsequence describes a monotonically non-decreasing path through these points, meaning
- What if we have more than two strings to align? - The basic dynamic programming algorithm can b
Many heuristics have been proposed for multiple sequence alignment. They often start by computing
strings. One approach replaces the two most similar sequences with a single merged sequence, and


%---- Page End Break Here ---- Page : 715

\begin{figure}[htbp]
    \centering
    \includegraphics[width=\linewidth]{algorithms_page_0717_0.png}
\end{figure}
Implementations: Several programs are available for multiple sequence alignment of DNA/protein se

Any of the dynamic programming-based approximate string matching programs of Section 21.4 (page 6

Combinatorica \\cite{pemmaraju2003computational} provides a Mathematica implementation of an algo

Notes: Surveys of algorithmic results on longest common subsequence (LCS) problems include [BHR00

Construct two random $n$-character strings on an alphabet of size $\alpha$. What is the expected

Multiple sequence alignment for computational biology is a large field, with the books of Gusfiel

We motivated the problem of longest common substring with the application of plagiarism detection

Related problems: Approximate string matching (see page 688), shortest common superstring (see pa


%---- Page End Break Here ---- Page : 716
\title{
A B R A C A R A C A D A A C A D A B C A D A B R A D A B R A
}

ABRACADABRA ABRACA

Input
Output

\section{Shortest Common Superstring}

Input description: A set of strings $S=\left\{S_{1}, \ldots, S_{m}\right\}$.
Problem description: Find the shortest string $S^{\prime}$ that contains each string $S_{i}$ as a
```

Discussion: Shortest common superstring/supersequence (SCS) arises in a variety of appl

A more important application of shortest common superstring is data/matrix compression

Perhaps the most compelling application is in DNA sequence assembly. Machines readily s

Finding a superstring of a set of strings is not difficult, since we can simply concate
atic. Indeed, shortest common superstring is NP-complete for all reasonable classes of

%---- Page End Break Here ---- Page : 717
Finding the shortest common superstring can be reduced to the traveling salesman proble

The greedy heuristic provides the standard approach to approximating shortest common su

How well does the greedy heuristic perform? It can certainly be fooled into creating a

Building superstrings becomes difficult when the input contains both positive strings

Implementations: Several high-performance programs for DNA sequence assembly are availa

CAP3 (Contig Assembly Program) \\cite{huang1999cap3} and PCAP HWA ${ }^{+} 03$ are the

The Celera assembler that sequenced the human genome is now available as open source. S

Notes: The shortest common superstring (SCS) problem and its application to DNA shotgun
superstring, where the strings are assumed to have character substitution errors. Thei

%---- Page End Break Here ---- Page : 718
Blum et al. $\mathrm{BJL}^{+} 94$ gave the first constant-factor approximation algorith

Experiments on shortest common superstring heuristics are reported in \\cite{romero2004

Related problems: Suffix trees (see page 448), text compression (see page 693 .

%---- Page End Break Here ---- Page : 719
Chapter 22

\chapter{Algorithmic Resources}

This chapter briefly describes resources that the working algorithm designer should be

\section{Algorithm Libraries}

A good algorithm designer does not reinvent the wheel, and a good programmer does not rewrite cod

But a word of caution about stealing. Many of the codes described in this book have been made ava

Although many of the systems described here may be available by accessing the algorithm reposito

\subsection{LEDA}

LEDA, for Library of Efficient Data types and Algorithms, is perhaps the best single resource ava
nally developed by a group at the Max Planck Institut in Saarbrücken, Germany. LEDA is unique bec

What LEDA offers is a complete collection of well-implemented C++ data structures and types. Part

LEDA is exclusively available from Algorithmic Solutions Software GmbH (https://www.algorithmic-s

\subsection{CGAL}

The Computational Geometry Algorithms Library or CGAL provides efficient and reliable geometric a

CGAL (https://www.cgal.org) should be the first place to go for serious geometric computing. CGAL

\subsection{Boost Graph Library}

Boost (www. boost . org) provides a well-regarded collection of free peer-reviewed portable $\mat

The Boost Graph Library [\\cite{siek2002boost} (http://www.boost.org/libs/graph/ doc) is perhaps

\subsection{Netlib}

Netlib (https://netlib.org/) is an online repository of mathematical software that contains a lar
is important because of its breadth and ease of access. Whenever you need a specialized piece of

%---- Page End Break Here ---- Page : 721
The Guide to Available Mathematical Software (GAMS), is an indexing service for Netlib and other

\subsection{Collected Algorithms of the ACM}

An early mechanism for the distribution of useful algorithm implementations was the Collected Alg

Almost 1,000 algorithms have appeared to date. Most of the codes are in Fortran and are relevant

\subsection{GitHub and SourceForge}

With over 28 million public repositories, the software development platform GitHub (https://githu

SourceForge (http://sourceforge.net/) is an older open source software development webs

\subsection{The Stanford GraphBase}

The Stanford GraphBase is an interesting program for several reasons. First, it was co

The GraphBase contains implementations of several important combinatorial algorithms, i
recreational problems, including constructing word ladders (flour-floor-flood-blood-bro

%---- Page End Break Here ---- Page : 722
Although the GraphBase is fun to play with, it is not really suited for building genera

\subsection{Combinatorica}

Combinatorica \\cite{pemmaraju2003computational} is a collection of over 450 algorithms

Although (in my totally unbiased opinion) Combinatorica is more comprehensive and bette

Check out http://www.combinatorica.com for the latest release and associated resources

\subsection{Programs from Books}

Several books on algorithms include working implementations of the algorithms in a real

The most useful codes of this genre are described below. Most are available from the al
- Programming Challenges - If you like the C code that appeared in the first half of th
https://www.cs.stonybrook.edu/~skiena/392/programs/, and
https://github.com/SkienaBooks/Algorithm-Design-Manual-Programs.
- Combinatorial Algorithms for Computers and Calculators - Nijenhuis and Wilf \\cite{ni
are often very short, but they are hard to locate and usually surprisingly subtle. Fort

%---- Page End Break Here ---- Page : 723
These programs are now available from the algorithm repository website. I tracked them
- Computational Geometry in C - O'Rourke O'R01 is perhaps the best practical introduc
- Algorithms in $C++$ - Sedgewick's popular algorithms text \\cite{sedgewick1998algorit
- Discrete Optimization Algorithms in Pascal - This collection of 28 programs for solvi

\section{Data Sources}

It is often important to have interesting data to feed your algorithms, to serve as tes
- Stanford Network Analysis Project (SNAP) - This resource couples a general purpose ne
- TSPLIB - This well-respected library of test instances for the traveling salesman pro
- Stanford GraphBase - As discussed in Section 22.1.7 this suite of programs by Knuth p

%---- Page End Break Here ---- Page : 724
\section{Online Bibliographic Resources}

The web is a fantastic resource for people interested in algorithms. What follows is a selective
- Google Scholar - This free resource (https://scholar.google.com/) restricts web searches to thi
- ACM Digital Library - This collection of bibliographic references provides links to essentially
- Arxiv - This preprint server with over 1.6 million papers is where researchers disseminate resu

\section{Professional Consulting Services}

Algorist Technologies (http://www.algorist.com) is a consulting firm that provides its clients wi

Algorist Technologies 215 West 92nd St. Suite 1F

New York, NY 10025

%---- Page End Break Here ---- Page : 725
Chapter 23

\section{Bibliography}
[\\cite{aichholzer1995novel}] O. Aichholzer, F. Aurenhammer, D. Alberts, and B. Gartner. A novel
[\\cite{abrahamsen2018art}] M. Abrahamsen, A. Adamaszek, and T. Miltzown. The art gallery problem
[\\cite{aaronson2013quantum}] Scott Aaronson. Quantum Computing since Democritus. Cambridge Unive
[\\cite{aaronson2015quantum}] Scott Aaronson. Quantum machine learning algorithms: Read the fine
[\\cite{arora2009computational}] Sanjeev Arora and Boaz Barak. Computational Complexity: A Modern
[\\cite{atallah2017algorithms}] Mikhail J Atallah and Marina Blanton. Algorithms and Theory of Co
[\\cite{applegate2007traveling}] D. Applegate, R. Bixby, V. Chvatal, and W. Cook. The Traveling S
[\\cite{arge2005cache}] L. Arge, G. Brodal, and R. Fagerberg. Cache-oblivious data structures. In
[\\cite{abboud2015tight}] Amir Abboud, Arturs Backurs, and Virginia Vassilevska Williams. Tight h
[\\cite{aho1975efficient}] A. Aho and M. Corasick. Efficient string matching: an aid to bibliogra
[\\cite{applegate1991computational}] D. Applegate and W. Cook. A computational study of the job-s
[ACFC $\left.{ }^{+} 16\right]$ Alberto Apostolico, Maxime Crochemore, Martin Farach-Colton, Zvi
$\left[\mathrm{ACH}^{+} 91\right] \quad$ E. M. Arkin, L. P. Chew, D. P. Huttenlocher, K. Kedem, a
[\\cite{amenta2001powercrust}] N. Amenta, S. Choi, and R. Kolluri. The power crust. In Proc. 6th
[\\cite{amenta2001powercrustunions}] N. Amenta, S. Choi, and R. Kolluri. The power crust, unions
[\\cite{angel2012implementation}] Eric Angel, Romain Campigotto, and Christian Laforest. Implemen
[\\cite{ajwani2011engineering}] Deepak Ajwani, Adan Cosgaya-Lozano, and Norbert Zeh. Engineering
$\left[\mathrm{ACP}^{+} 07\right] \quad$ H. Ahn, O. Cheong, C. Park, C. Shin, and A. Vigneron. Ma
[\\cite{aleksandrov2007partitioning}] Lyudmil Aleksandrov, Hristo Djidjev, Hua Guo, and Anil Mahe
[\\cite{arlazarov1970economical}] V. Arlazarov, E. Dinic, M. Kronrod, and I. Faradzev. On economi
[Ad194] L. M. Adleman. Molecular computations of solutions to combinatorial problems. Science, 26
[\\cite{andrews2004integer}] G. Andrews and K. Eriksson. Integer Partitions. Cambridge University
[\\cite{avis1996reverse}] D. Avis and K. Fukuda. Reverse search for enumeration. Disc. Applied Ma
[\\cite{abraham2010highway}] Ittai Abraham, Amos Fiat, Andrew V Goldberg, and Renato F Werneck. H

[\\cite{agarwal2002polygon}] P. Agarwal, E. Flato, and D. Halperin. Polygon decomposit
[\\cite{alt2000discrete}] H. Alt and L. Guibas. Discrete geometric shapes: Matching, i
[\\cite{agarwal2018range}] P. Agarwal. Range searching. In J. Goodman, J. O'Rourke, an
[\\cite{amato2001randomized}] N. Amato, M. Goodrich, and E. Ramos. A randomized algori
[\\cite{aggarwal1989linear}] A. Aggarwal, L. Guibas, J. Saxe, and P. Shor. A linear-ti
[\\cite{aho1972transitive}] A. Aho, M. Garey, and J. Ullman. The transitive reduction
[AHK $\left.{ }^{+} 08\right]$ E. Arkin, G. Hart, J. Kim, I. Kostitsyna, J. Mitchell,
[\\cite{aho1990algorithms}] A. Aho. Algorithms for finding patterns in strings. In J.
[\\cite{aho1974design}] A. Aho, J. Hopcroft, and J. Ullman. The Design and Analysis of
[\\cite{aho1983data}] A. Aho, J. Hopcroft, and J. Ullman. Data Structures and Algorith
[\\cite{andoni2006near}] Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorit
[\\cite{andoni2008near}] Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorit
[\\cite{aigner1988combinatorial}] M. Aigner. Combinatorial Search. Wiley-Teubner, 1988
[\\cite{andoni2018approximate}] Alexandr Andoni, Piotr Indyk, and Ilya Razenshteyn. Ap
[\\cite{asahiro2000greedily}] Y. Asahiro, K. Iwama, H. Tamaki, and T. Tokuyama. Greedi
[\\cite{auger2018worst}] Nicolas Auger, Vincent Jugé, Cyril Nicaud, and Carine Pivotea
[\\cite{aarts1989simulated}] E. Aarts and J. Korst. Simulated Annealing and Boltzman M
[\\cite{aurenhammer2013voronoi}] Franz Aurenhammer, Rolf Klein, and Der-Tsai Lee. Voro
[\\cite{agrawal2004primes}] M. Agrawal, N. Kayal, and N. Saxena. PRIMES is in P. Annal
[\\cite{aarts1997local}] E. Aarts and J.K. Lenstra. Local Search in Combinatorial Opti
[\\cite{arya1993approximate}] S. Arya and D. Mount. Approximate nearest neighbor queri
$\left[\mathrm{AMN}^{+} 98\right]$ S. Arya, D. Mount, N. Netanyahu, R. Silverman, and
[\\cite{ahuja1993network}] R. Ahuja, T. Magnanti, and J. Orlin. Network Flows. Prentic
[\\cite{andrews1998theory}] G. Andrews. The Theory of Partitions. Cambridge Univ. Pres
[\\cite{andersson2005searching}] A. Andersson. Searching and priority queues in $o(\lo
[ANN ${ }^{+}$18] Alexandr Andoni, Assaf Naor, Aleksandar Nikolov, Ilya Razenshteyn, a
[\\cite{andoni2014parallel}] Alexandr Andoni, Aleksandar Nikolov, Krzysztof Onak, and
[\\cite{aho1972minimum}] A. Aho and T. Peterson. A minimum distance error-correcting p
[\\cite{agarwal2014near}] Pankaj K Agarwal and Jiangwei Pan. Near-linear algorithms fo
[\\cite{aspvall1979linear}] B. Aspvall, M. Plass, and R. Tarjan. A linear-time algorit
[\\cite{arora1998polynomial}] S. Arora. Polynomial time approximations schemes for Euc
[\\cite{agarwal2000arrangements}] P. Agarwal and M. Sharir. Arrangements. In J. Sack a
[\\cite{atallah1983linear}] M. Atallah. A linear time algorithm for the Hausdorff dist
[$\left.\mathrm{B}^{+} 16\right]$ Albert-László Barabási et al. Network Science. Camb
[\\cite{babai2016graph}] László Babai. Graph isomorphism in quasipolynomial time. In P
[\\cite{bapat2010graphs}] Ravindra B Bapat. Graphs and Matrices, volume 27. Springer,
[\\cite{barabasi2003linked}] A. Barabasi. Linked: How Everything Is Connected to Every
[\\cite{bb14}] Charles H Bennett and Gilles Brassard. Quantum cryptography: public key
[\\cite{bbf99}] V. Bafna, P. Berman, and T. Fujito. A 2-approximation algorithm for th
[\\cite{bbpp99}] I. Bomze, M. Budinich, P. Pardalos, and M. Pelillo. The maximum cliqu
[\\cite{bc19}] Stephan Beyer and Markus Chimani. Strong steiner tree approximations in
[\\cite{bcgm13}] Gunnar Brinkmann, Kris Coolsaet, Jan Goedgebeur, and Hadrien Mélot. H
[\\cite{bcgr92}] D. Berque, R. Cecchini, M. Goldberg, and R. Rivenburgh. The SetPlayer
[\\cite{bcw90}] T. Bell, J. Cleary, and I. Witten. Text Compression. Prentice Hall, 19
[\\cite{bd99}] R. Bubley and M. Dyer. Faster random generation of linear extensions. D
[\\cite{bd11}] Joseph Blitzstein and Persi Diaconis. A sequential importance sampling

[\\cite{bdh97}] C. Barber, D. Dobkin, and H. Huhdanpaa. The Quickhull algorithm for convex hulls.
[\\cite{bdn01}] G. Bilardi, P. D'Alberto, and A. Nicolau. Fractal matrix multiplication: a case s
[\\cite{bdy06}] K. Been, E. Daiches, and C. Yap. Dynamic map labeling. IEEE Trans./ Visualization
[\\cite{beb12}] Tyson Brochu, Essex Edwards, and Robert Bridson. Efficient geometrically exact co
[\\cite{bel58}] R. Bellman. On a routing problem. Quarterly of Applied Mathematics, 16:87-90, 195
[\\cite{ben75}] J. Bentley. Multidimensional binary search trees used for associative searching.
[\\cite{ben90}] J. Bentley. More Programming Pearls. Addison-Wesley, 1990.
[\\cite{ben92a}] J. Bentley. Fast algorithms for geometric traveling salesman problems. ORSA J. C
[\\cite{ben92b}] J. Bentley. Software exploratorium: The trouble with qsort. UNIX Review, 10(2):8
[\\cite{bentley1999programming}] J. Bentley. Programming Pearls. Addison-Wesley, second edition,
[\\cite{bertsekas1982constrained}] D. Bertsekas. Constrained Optimization and Lagrange Multiplier
[\\cite{berge1989hypergraphs}] C. Berge. Hypergraphs. North-Holland, 1989.
[\\cite{bern2002adaptive}] M. Bern. Adaptive mesh generation. In T. Barth and H. Deconinck, edito
[\\cite{bernstein2004fast}] D. Bernstein. Fast multiplication and its applications. http://cr.yp.
[\\cite{bertsekas2015convex}] D. Bertsekas. Convex Optimization Algorithms. Athena Scientific Bel
[\\cite{bernhardt2019quantum}] Chris Bernhardt. Quantum Computing for Everyone. MIT Press, 2019.
[\\cite{dibattista1999graph}] G. Di Battista, P. Eades, R. Tamassia, and I. Tollis. Graph Drawing
[\\cite{bender2000lca}] M. Bender and M. Farach. The LCA problem revisited. In Proc. 4th Latin Am
$\left[\mathrm{BFH}^{+} 18\right]$ Alon Baram, Efi Fogel, Dan Halperin, Michael Hemmer, and Sebas
$\left[\right.$ BFP $\left.^{+} 72\right]$ M. Blum, R. Floyd, V. Pratt, R. Rivest, and R. Tarjan.
[\\cite{blelloch2012greedy}] G. Blelloch, J. Fineman, and J. Shun. Greedy sequential maximal inde
[\\cite{brodal2007engineering}] G. Brodal, R. Fagerberg, and K. Vinther. Engineering a cache-obli
[\\cite{berry1995path}] J. Berry and M. Goldberg. Path optimization and near-greedy analysis for
$\left[\right.$ BGJ $\left.^{+} 12\right]$ Arnab Bhattacharyya, Elena Grigorescu, Kyomin Jung, So
[\\cite{bellare1995free}] M Bellare, O. Goldreich, and M. Sudan. Free bits, PCPs, and non-approxi
[\\cite{buckley1990distances}] F. Buckley and F. Harary. Distances in Graphs. Addison-Wesley, 199
[\\cite{barequet2001efficiently}] G. Barequet and S. Har-Peled. Efficiently approximating the mir
[\\cite{bjorklund2004approximating}] Andreas Björklund, Thore Husfeldt, and Sanjeev Khanna. Appro
[BHR00] L. Bergroth, H. Hakonen, and T. Raita. A survey of longest common subsequence algorithms.
[BHvM09] Armin Biere, Marijn Heule, and Hans van Maaren. Handbook of Satisfiability, volume 185.
[BI15] Arturs Backurs and Piotr Indyk. Edit distance cannot be computed in strongly subquadratic
[BIK $\left.{ }^{+} 04\right]$ H. Bronnimann, J. Iacono, J. Katajainen, P. Morin, J. Morrison, an
[Bis06] Christopher M Bishop. Pattern Recognition and Machine Learning. Springer, 2006.
[BJL $\left.{ }^{+} 94\right]$ A. Blum, T. Jiang, M. Li, J. Tromp, and M. Yanakakis. Linear appro
[BJL06] C. Buchheim, M. Jünger, and S. Leipert. Drawing rooted trees in linear time. Software: Pr
[BJLM83] J. Bentley, D. Johnson, F. Leighton, and C. McGeoch. An experimental study of bin packir
[BK04] Y. Boykov and V. Kolmogorov. An experimental comparison of min-cut/max-flow algorithms for
[BK18] Karl Bringmann and Marvin Künnemann. Multivariate fine-grained complexity of longest commo
[BKRV00] A. Blum, G. Konjevod, R. Ravi, and S. Vempala. Semi-definite relaxations for minimum bar
[BL30] Edward Bulwer Lytton. Paul Clifford. Henry Colburn and Richard Bentley, London, 1830.
[BL76] K. Booth and G. Lueker. Testing for the consecutive ones property, interval graphs, and pl
[BLS91] D. Bailey, K. Lee, and H. Simon. Using Strassen's algorithm to accelerate the solution of
[BLT12] Gerth Stølting Brodal, George Lagogiannis, and Robert E. Tarjan. Strict fibonacci heaps.
[Blu67] H. Blum. A transformation for extracting new descriptions of shape. In W. Wathen-Dunn, ed
[\\cite{biggs1976graph}] N. L. Biggs, E. K. Lloyd, and R. J. Wilson. Graph Theory 1736-1936. sndc
[\\cite{birkhoff1953survey}] G. Birkhoff and S. MacLane. A Survey of Modern Algebra. Macmillian,

[\\cite{boyer1977fast}] R. Boyer and J. Moore. A fast string-searching algorithm. Commu
[\\cite{bingham2001random}] E. Bingham and H. Mannila. Random projection in dimensional
[\\cite{broder2005network}] A. Broder and M. Mitzenmacher. Network applications of blo
[\\cite{buchin2016area}] Kevin Buchin, Wouter Meulemans, André Van Renssen, and Bettin
$\left[\mathrm{BMS}^{+} 16\right]$ Aydn Buluç, Henning Meyerhenke, Ilya Safro, Peter S
[\\cite{bader2013graph}] David A Bader, Henning Meyerhenke, Peter Sanders, and Dorothe
[\\cite{boggess2009first}] Albert Boggess and Francis J Narcowich. A First Course in W
[\\cite{bentley1979algorithms}] J. Bentley and T. Ottmann. Algorithms for reporting an
[\\cite{benor1983lower}] M. Ben-Or. Lower bounds for algebraic computation trees. In P
[\\cite{bollobas2001random}] B. Bollobas. Random Graphs. Cambridge Univ. Press, second
[\\cite{bottou2012stochastic}] Léon Bottou. Stochastic gradient descent tricks. In Neu
[\\cite{balas1976set}] E. Balas and M. Padberg. Set partitioning-a survey. SIAM Review
[\\cite{binstock1995practical}] A. Binstock and J. Rex. Practical Algorithms for Progr
[\\cite{bracewell1999fourier}] R. Bracewell. The Fourier Transform and its Application
[\\cite{brass2008advanced}] Peter Brass. Advanced Data Structures. Cambridge Universit
[Brè79] D. Brèlaz. New methods to color the vertices of a graph. Comm. ACM, 22:251-256
[\\cite{brigham1988fast}] E. Brigham. The Fast Fourier Transform. Prentice, facimile e
[\\cite{brooks1995mythical}] F. Brooks. The Mythical Man-Month. Addison-Wesley, twenti
[\\cite{broder1997resemblance}] Andrei Z. Broder. On the resemblance and containment o
[BRS ${ }^{+}$10] Lawrence E. Bassham, Andrew L. Rukhin, Juan Soto, James R. Nechvatal
[\\cite{brucker2007scheduling}] P. Brucker. Scheduling Algorithms. Springer-Verlag, fi
[\\cite{brzozowski1964derivatives}] J. Brzozowski. Derivatives of regular expressions.
[\\cite{bentley1976divide}] J. Bentley and M. Shamos. Divide-and-conquer in higher-dim
[\\cite{berry1986regular}] G. Berry and R. Sethi. From regular expressions to determin
[\\cite{bach1996algorithmic}] E. Bach and J. Shallit. Algorithmic Number Theory: Effic
[\\cite{bradley1997fabricating}] R. Bradley and S. Skiena. Fabricating arrays of strin
[\\cite{barvinok2007random}] A. Barvinok and A. Samorodnitsky. Random weighting, asymp
[\\cite{bichot2013graph}] Charles-Edmond Bichot and Patrick Siarry. Graph partitioning
[\\cite{bern2018triangulations}] M. Bern, J. Shewchuk, and N. Amenta. Triangulations a
[\\cite{buchanan1992numerical}] J. Buchanan and P. Turner. Numerical Methods and Analy
[\\cite{buttazzo2011hard}] Giorgio C Buttazzo. Hard Real-Time Computing Systems: Predi
[\\cite{boyd2004convex}] Stephen Boyd and Lieven Vandenberghe. Convex Optimization. Ca
[\\cite{baase1999computer}] S. Baase and A. van Gelder. Computer Algorithms. Addison-W
[\\cite{brightwell1991counting}] G. Brightwell and P. Winkler. Counting linear extensi
[\\cite{burrows1994block}] M. Burrows and D. Wheeler. A block sorting lossless data co
[\\cite{borndorfer2000set}] R. Borndorfer and R. Weismantel. Set packing relaxations o
[\\cite{brent2010modern}] Richard P Brent and Paul Zimmermann. Modern computer Arithme
[\\cite{canny1987complexity}] J. Canny. The complexity of robot motion planning. MIT P
[\\cite{cash1995fast}] G. Cash. A fast computer algorithm for finding the permanent of
[\\cite{cong2004euler}] C. Cong and D. Bader. The Euler tour technique and parallel ro
[\\cite{carlsson1992complexity}] S. Carlsson and J. Chen. The complexity of heaps. In
[\\cite{cook1997combinatorial}] W. Cook and W. Cunningham. Combinatorial Optimization.
[\\cite{chapra2016numerical}] S. Chapra and R. Canale. Numerical Methods for Engineers
[\\cite{chinn1982bandwidth}] P. Chinn, J. Chvátolvá, A. K. Dewdney, and N. E. Gibbs. T
[\\cite{cao2015feedback}] Yixin Cao, Jianer Chen, and Yang Liu. On feedback vertex set
[\\cite{chazelle1985optimal}] B. Chazelle and D. Dobkin. Optimal convex decompositions

[CDG $\left.{ }^{+} 18\right]$ Diptarka Chakraborty, Debarati Das, Elazar Goldenberg, Michal Kouc
[\\cite{chazelle1986computing}] B. Chazelle, R. Drysdale, and D. Lee. Computing the largest empty
[\\cite{chazelle1992optimal}] B. Chazelle and H. Edelsbrunner. An optimal algorithm for intersect
[\\cite{caprara1999heuristic}] A. Caprara, M. Fischetti, and P. Toth. A heuristic method for the
[\\cite{caprara2000algorithms}] A. Caprara, M. Fischetti, and P. Toth. Algorithms for the set cov
[\\cite{cherkassky1994implementing}] B. Cherkassky and A. Goldberg. On implementing push-relabel
[\\cite{coullard1998matching}] C.R. Coullard, A.B. Gamble, and P.C. Jones. Matching problems in s
[CGJ ${ }^{+}$13] Markus Chimani, Carsten Gutwenger, Michael Jünger, Gunnar W. Klau, Karsten Klei
$\left[\mathrm{CGK}^{+} 97\right]$ C. Chekuri, A. Goldberg, D. Karger, M. Levine, and C. Stein. E
[\\cite{chazelle1985duality}] B. Chazelle, L. Guibas, and D. T. Lee. The power of geometric duali
$\left[\mathrm{CGM}^{+} 98\right]$ B. Cherkassky, A. Goldberg, P. Martin, J. Setubal, and J. Stol
[\\cite{cherkassky1999shortest}] B. Cherkassky, A. Goldberg, and T. Radzik. Shortest paths algori
[\\cite{cherkassky1999buckets}] B. Cherkassky, A. Goldberg, and C. Silverstein. Buckets, heaps, l
[\\cite{cook2006mining}] D. Cook and L. Holder. Mining Graph Data. John Wiley \& Sons, 2006.
[\\cite{cormode2009frequent}] Graham Cormode and Marios Hadjieleftheriou. Finding the frequent it
[\\cite{chazelle1991triangulating}] B. Chazelle. Triangulating a simple polygon in linear time. D
[\\cite{chazelle2000mst}] B. Chazelle. A minimum spanning tree algorithm with inverse-Ackerman ty
[\\cite{chew1985planing}] L. P. Chew. Planing the shortest path for a disc in $\mathrm{O}\left(n^
[\\cite{chen2015handbook}] Chi-hau Chen. Handbook of Pattern Recognition and Computer Vision. Wor
[\\cite{crochemore2007strings}] M. Crochemore, C. Hancart, and T. Lecroq. Algorithms on Strings.
[\\cite{christofides1976heuristic}] N. Christofides. Worst-case analysis of a new heuristic for t
[\\cite{chung1997spectral}] F. Chung. Spectral Graph Theory. AMS, 1997.
[\\cite{chvatal1983linear}] V. Chvatal. Linear Programming. Freeman, 1983.
[\\cite{crochemore2001lcs}] M. Crochemore, C. Iliopolous, Y. Pinzon, and J. Reid. A fast and prac
$\left[\right.$ CJCG $^{+}$13] Edward G. Coffman Jr, János Csirik, Gábor Galambos, Silvano Martel
[CJL ${ }^{+}$16] Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray Perlner,
[\\cite{chetverin1994oligonucleotide}] A. Chetverin and F. Kramer. Oligonucleotide arrays: New co
[\\cite{crescenzi1997approximation}] Pierluigi Crescenzi and Viggo Kann. Approximation on the web
[\\cite{chekuri2005ptas}] Chandra Chekuri and Sanjeev Khanna. A polynomial time approximation sch
[\\cite{cheney2012numerical}] W. Cheney and D. Kincaid. Numerical Mathematics and Computing. Cent
[\\cite{christensen2017survey}] Henrik I. Christensen, Arindam Khan, Sebastian Pokutta, and Prasa
[\\cite{crochemore1998text}] M. Crochemore and T. Lecroq. Text data compression algorithms. In M.
[\\cite{corberan2013arc}] Ángel Corberán and Gilbert Laporte. Arc Routing: Problems, Methods, and
[\\cite{clarkson1992determinant}] K. L. Clarkson. Safe and effective determinant evaluation. In P
[\\cite{chan2011orthogonal}] Timothy M Chan, Kasper Green Larsen, and Mihai Ptracu. Orthogonal ra
[\\cite{cormen2009introduction}] T. Cormen, C. Leiserson, R. Rivest, and C. Stein. Introduction t
[\\cite{cuthill1969bandwidth}] E. Cuthill and J. McKee. Reducing the bandwidth of sparse symmetri
[\\cite{cheriyan1996dense}] J. Cheriyan and K. Mehlhorn. Algorithms for dense graphs and networks
[\\cite{delcorso1999bandwidth}] G. Del Corso and G. Manzini. Finding exact solutions to the bandw
[\\cite{cabello2013adding}] Sergio Cabello and Bojan Mohar. Adding one edge to planar graphs make
[\\cite{chan2018optimal}] Timothy M. Chan and Yakov Nekrich. Towards an optimal method for dynami
[\\cite{chiba1985linear}] Norishige Chiba, Takao Nishizeki, Shigenobu Abe, and Takao Ozawa. A lin
[\\cite{cohen1994estimating}] E. Cohen. Estimating the size of the transitive closure in linear t
[\\cite{conway1971regular}] J.H. Conway. Regular Algebra and Finite Machines. Chapman \$ Hall, 19
[\\cite{cook1971complexity}] S. Cook. The complexity of theorem proving procedures. In Third ACM
[\\cite{cook2011pursuit}] William J Cook. In Pursuit of the Traveling Salesman: Mathematics at th

[\\cite{carraghan1990exact}] R. Carraghan and P. Pardalos. An exact algorithm for the m
[\\cite{crandall2005prime}] R. Crandall and C. Pomerance. Prime Numbers: A Computationa
[\\cite{compeau2018bioinformatics}] Phillip Compeau and P.A. Pevzner. Bioinformatics Al
[\\cite{chen2018tutorial}] Haochen Chen, Bryan Perozzi, Rami Al-Rfou, and Steven Skiena
[\\cite{chen1998review}] B. Chen, C. Potts, and G. Woeginger. A review of machine sche
[\\cite{cohen1976implementation}] J. Cohen and M. Roth. On the implementation of Stras
[\\cite{cook1999computing}] W. Cook and A. Rohe. Computing minimum-weight perfect match
[\\cite{crochemore2003jewels}] M. Crochemore and W. Rytter. Jewels of Stringology. Worl
[\\cite{conway1993sphere}] J. Conway and N. Sloane. Sphere Packings, Lattices, and Grou
[\\cite{caprara2005laying}] A. Caprara and J. Salazar-González. Laying out sparse graph
[\\cite{cooley1965algorithm}] J. Cooley and J. Tukey. An algorithm for the machine calc
[\\cite{chiang1992dynamic}] Y. Chiang and R. Tamassia. Dynamic algorithms in computatio
[\\cite{coppersmith1990matrix}] D. Coppersmith and S. Winograd. Matrix multiplication v
[\\cite{chan2016adaptive}] Timothy M Chan and Bryan T Wilkinson. Adaptive and approxima
[\\cite{dantzig1963linear}] G. Dantzig. Linear Programming and Extensions. Princeton Un
[\\cite{dancik1994expected}] V. Dancik. Expected length of longest common subsequences
[\\cite{dahlquist1974numerical}] G. Dahlquist and A. Bjorck. Numerical Methods. Prentic
[\\cite{davies1986algorithms}] G. Davies and S. Bowsher. Algorithms for pattern matchin
[dBDK $\left.{ }^{+} 98\right]$ M. de Berg, O. Devillers, M. Kreveld, O. Schwarzkopf, a
[\\cite{deberg2008computational}] M. de Berg, M. van Kreveld, M. Overmars, and O. Schwa
[\\cite{davis2018impact}] James C Davis, Christy A Coghlan, Francisco Servant, and Dong
[\\cite{dyken2009simultaneous}] Christopher Dyken, Morten Dæhlen, and Thomas Sevaldrud
[\\cite{demaine2002cache}] Erik D Demaine. Cache-oblivious algorithms and data structur
[\\cite{deng2005efficient}] L. Y. Deng. Efficient and portable multiple recursive gener
[\\cite{dey2006curve}] T. Dey. Curve and Surface Reconstruction: Algorithms with Mathem
[\\cite{denardo1979shortest}] E. Denardo and B. Fox. Shortest-route methods: 1. reachin
[\\cite{dasgupta2008random}] Sanjoy Dasgupta and Yoav Freund. Random projection trees a
[\\cite{downey2012parameterized}] Rodney G. Downey and Michael Ralph Fellows. Parameter
[\\cite{dantzig1954solution}] G. Dantzig, D. Fulkerson, and S. Johnson. Solution of a l
[\\cite{defraysseix1990how}] H. de Fraysseix, J. Pach, and R. Pollack. How to draw a pl
[\\cite{dubey2011hardness}] Chandan Dubey, Uriel Feige, and Walter Unger. Hardness resu
[DGH $\left.{ }^{+} 02\right]$ E. Dantsin, A. Goerdt, E. Hirsch, R. Kannan, J. Kleinber
[\\cite{dial1979computational}] R. Dial, F. Glover, D. Karney, and D. Klingman. A compu
[\\cite{du1992proof}] D. Du and F. Hwang. A proof of Gilbert and Pollak's conjecture on
[\\cite{du2000combinatorial}] Dingzhu Du and Frank Hwang. Combinatorial Group Testing a
[\\cite{duda2000pattern}] R. Duda, P. Hart, and D. Stork. Pattern Classification. Wiley
[\\cite{dietzfelbinger2004primality}] M. Dietzfelbinger. Primality Testing in Polynomia
[\\cite{dijkstra1959note}] E. W. Dijkstra. A note on two problems in connection with gr
[\\cite{delorme2016bin}] Maxence Delorme, Manuel Iori, and Silvano Martello. Bin packin
[\\cite{delorme2018bpplib}] Maxence Delorme, Manuel Iori, and Silvano Martello. Bpplib
[\\cite{das1992minimum}] G. Das and D. Joseph. Minimum vertex hulls for polyhedral doma
[\\cite{djidjev2000computing}] H. Djidjev. Computing the girth of a planar graph. In Pr
[\\cite{demaine2004interpolation}] E. Demaine, T. Jones, and M. Patrascu. Interpolation
[\\cite{dutta2010steiner}] Prasun Dutta, S. Pratik Khastgir, and Anushree Roy. Steiner
[\\cite{dobkin1976multidimensional}] D. Dobkin and R. Lipton. Multidimensional searchin
[\\cite{dobkin1979linear}] D. Dobkin, R. Lipton, and S. Reiss. Linear programming is lo

[\\cite{dobkin1980determining}] D. Dobkin and J.. Munro. Determining the mode. Theoretical Comput
[\\cite{daniels1997multiple}] K. Daniels and V. Milenkovic. Multiple translational containment. p
[\\cite{dongarra1979linpack}] J. Dongarra, C. Moler, J. Bunch, and G. Stewart. LINPACK User's Gui
[\\cite{demourapinto2009fast}] Francisco de Moura Pinto and Carla Maria Dal Sasso Freitas. Fast m
[\\cite{daniels1997finding}] K. Daniels, V. Milenkovic, and D. Roth. Finding the largest area axi
[\\cite{dalberto2007adaptive}] P. D'Alberto and A. Nicolau. Adaptive Strassen's matrix multiplica
[\\cite{dalberto2009adaptive}] Paolo D'Alberto and Alexandru Nicolau. Adaptive winograd's matrix
[\\cite{denooy2018exploratory}] Wouter De Nooy, Andrej Mrvar, and Vladimir Batagelj. Exploratory
[\\cite{douglas1973algorithms}] D. H. Douglas and T. K. Peucker. Algorithms for the reduction of
[\\cite{dittmer2018counting}] Samuel Dittmer and Igor Pak. Counting linear extensions of restrict
[\\cite{diaz2002survey}] J. Diaz, J. Petit, and M. Serna. A survey of graph layout problems. ACM
[\\cite{dasgupta2008algorithms}] Sanjoy Dasgupta, Christos H Papadimitriou, and Umesh Virkumar Va
[\\cite{dershowitz1990calendrical}] N. Dershowitz and E. Reingold. Calendrical calculations. Soft
[\\cite{dershowitz2002calendrical}] N. Dershowitz and E. Reingold. Calendrical Tabulations: 1900-
[DRR $\left.{ }^{+} 95\right]$ S. Dawson, C. R. Ramakrishnan, I. V. Ramakrishnan, K. Sagonas, S.
[\\cite{dinur2005hardness}] Irit Dinur and Samuel Safra. On the hardness of approximating minimum
[\\cite{du2000advances}] D. Du, J. Smith, and J. Rubinstein. Advances in Steiner Trees. Kluwer, 2
[\\cite{dorigo2004ant}] M. Dorigo and T.Stutzle. Ant Colony Optimization. MIT Press, 2004.
[\\cite{devsmit1982comparison}] G. de V. Smit. A comparison of three string matching algorithms.
[\\cite{devries2003combinatorial}] S. de Vries and R. Vohra. Combinatorial auctions: A survey. In
[\\cite{deng1994warings}] Y. Deng and C. Yang. Waring's problem for pyramidal numbers. Science in
[\\cite{dor1999selecting}] D. Dor and U. Zwick. Selecting the median. SIAM J. Computing, pages 17
[\\cite{dor2001median}] D. Dor and U. Zwick. Median selection requires $(2+\epsilon) n$ compariso
[\\cite{ebert1988computing}] J. Ebert. Computing Eulerian trails. Info. Proc. Letters, 28:93-97,
[\\cite{estivillcastro1992survey}] V. Estivill-Castro and D. Wood. A survey of adaptive sorting a
[\\cite{edelsbrunner1987algorithms}] H. Edelsbrunner. Algorithms for Combinatorial Geometry. Spri
[\\cite{edelsbrunner2006geometry}] H. Edelsbrunner. Geometry and Topology for Mesh Generation. Ca
[\\cite{edmonds1965paths}] J. Edmonds. Paths, trees, and flowers. Canadian J. Math., 17:449-467,
[\\cite{edmonds1971matroids}] J. Edmonds. Matroids and the greedy algorithm. Mathematical Program
[\\cite{eppstein1999raising}] D. Eppstein and J. Erickson. Raising roofs, crashing cycles, and pl
[\\cite{erdos1960graphs}] P. Erds and T. Gallai. Graphs with prescribed degrees of vertices. Mat.
[\\cite{edelsbrunner1989topologically}] H. Edelsbrunner and L. Guibas. Topologically sweeping an
[\\cite{edelsbrunner1991corrigendum}] H. Edelsbrunner and L. Guibas. Corrigendum: Topologically s
[\\cite{eppstein1998dynamic}] David Eppstein, Zvi Galil, and Giuseppe F Italiano. Dynamic graph a
[\\cite{eppstein1997sparsification}] David Eppstein, Zvi Galil, Giuseppe F Italiano, and Amnon Ni
[\\cite{edelsbrunner1986optimal}] H. Edelsbrunner, L. Guibas, and J. Stolfi. Optimal point locati
[\\cite{edelsbrunner2010computational}] Herbert Edelsbrunner and John Harer. Computational Topolo
$\left[\mathrm{EHM}^{+} 12\right]$ Yuval Emek, Magnús M Halldórsson, Yishay Mansour, Boaz PattSha
[\\cite{edmonds1973matching}] J. Edmonds and E. Johnson. Matching, Euler tours, and the Chinese p
[\\cite{edmonds1972theoretical}] J. Edmonds and R. Karp. Theoretical improvements in the algorith
[\\cite{easley2010networks}] David Easley and Jon Kleinberg. Networks, Crowds, and Markets. Cambr
[\\cite{edahiro1984point}] M. I. Edahiro, I. Kokubo, and T. Asano. A new point location algorithm
[\\cite{edelsbrunner1983shape}] H. Edelsbrunner, D. Kirkpatrick, and R. Seidel. On the shape of a
[\\cite{edelsbrunner1994three}] H. Edelsbrunner and E. Mücke. Three-dimensional alpha shapes. ACM
[\\cite{even1998approximating}] G. Even, J. Naor, B. Schieber, and M. Sudan. Approximating minimu
[\\cite{eppstein1998k}] D. Eppstein. Finding the k shortest paths. SIAM J. Computing, 28:652673

[\\cite{edelsbrunner1986voronoi}] H. Edelsbrunner and R. Seidel. Voronoi diagrams and a
[\\cite{edelsbrunner1993zone}] H. Edelsbrunner, R. Seidel, and M. Sharir. On the zone
[\\cite{evans1996optimizing}] F. Evans, S. Skiena, and A. Varshney. Optimizing triangl
[\\cite{everstine1979comparison}] G. Everstine. A comparison of three resequencing alg
[\\cite{even2011graph}] Shimon Even. Graph Algorithms. Cambridge University Press, sec
[F48] I. Fáry. On straight line representation of planar graphs. Acta. Sci. Math. Szeg
[\\cite{fan2014cuckoo}] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mi
[\\cite{feige1998threshold}] U. Feige. A threshold of $\ln n$ for approximating set co
[\\cite{ford1962flows}] L. Ford and D. R. Fulkerson. Flows in Networks. Princeton Univ
[\\cite{feige1995approximating}] U. Feige and M. Goemans. Approximating the value of t
[\\cite{fogel2006exact}] E. Fogel and D. Halperin. Exact and efficient construction of
[\\cite{friedman2001elements}] Jerome Friedman, Trevor Hastie, and Robert Tibshirani.
[\\cite{fogel2007exact}] E. Fogel, D. Halperin, and C. Weibel. On the exact maximum co
[\\cite{refFHW12}] Efi Fogel, Dan Halperin, and Ron Wein. CGAL Arrangements and Their
[\\cite{refFJ05}] M. Frigo and S. Johnson. The design and implementation of FFTW3. Pro
[\\cite{refFJMO93}] M. Fredman, D. Johnson, L. McGeoch, and G. Ostheimer. Data structu
[\\cite{refFK10}] Alireza Fathi and John Krumm. Detecting road intersections from gps
[\\cite{refFK15}] Alan Frieze and Micha Karoski. Introduction to Random Graphs. Cambri
[FKN ${ }^{+}$08] Michael R. Fellows, Christian Knauer, Naomi Nishimura, Prabhakar Rag
[\\cite{refFL07}] Uriel Feige and James R Lee. An improved approximation ratio for the
[\\cite{refFle74}] H. Fleischner. The square of every two-connected graph is Hamiltoni
[\\cite{refFlo62}] R. Floyd. Algorithm 97 (shortest path). Communications of the ACM,
[\\cite{refFlo64}] R. Floyd. Algorithm 245 (treesort). Communications of the ACM, 18:7
[\\cite{refFLPR99}] M. Frigo, C. Leiserson, H. Prokop, and S. Ramachandran. Cacheobliv
[\\cite{refFM71}] M. Fischer and A. Meyer. Boolean matrix multiplication and transitiv
[\\cite{refFM82}] C. Fiduccia and R. Mattheyses. A linear time heuristic for improving
[\\cite{refFN04}] K. Fredriksson and G. Navarro. Average-optimal single and multiple a
[\\cite{refFor87}] S. Fortune. A sweepline algorithm for Voronoi diagrams. Algorithmic
[\\cite{refFor13}] Lance Fortnow. The Golden Ticket: P, NP, and the Search for the Imp
[\\cite{refFor18}] S. Fortune. Voronoi diagrams and Delauney triangulations. In J. Goo
[\\cite{festa1999feedback}] P. Festa, P. Pardalos, and M. Resende. Feedback set proble
[\\cite{festa2001algorithm815}] P. Festa, P. Pardalos, and M. Resende. Algorithm 815:
[\\cite{floyd1975expected}] R. Floyd and R. Rivest. Expected time bounds for selection
[\\cite{furer1994approximating}] M. Fürer and B. Raghavachari. Approximating the minim
[\\cite{fredkin1962trie}] E. Fredkin. Trie memory. Communications of the ACM, 3:490-49
[\\cite{fredman1976how}] M. Fredman. How good is the information theory bound in sorti
[\\cite{ferguson2003practical}] N. Ferguson and B. Schneier. Practical Cryptography. J
[\\cite{foggia2001performance}] P. Foggia, C. Sansone, and M. Vento. A performance com
[\\cite{fredman1987fibonacci}] M. Fredman and R. Tarjan. Fibonacci heaps and their use
[\\cite{fortune1993efficient}] S. Fortune and C. van Wyk. Efficient exact arithmetic f
[\\cite{fiorini1977edge}] S. Fiorini and R. Wilson. Edge-Colourings of Graphs. Researc
[\\cite{fredman1993surpassing}] M. Fredman and D. Willard. Surpassing the information
[\\cite{fogel2004code}] E. Fogel, R. Wein, and D. Halperin. Code flexibility and progr
[\\cite{gabow1977two}] H. Gabow. Two algorithms for generating weighted spanning trees
$\left[\mathrm{GAD}^{+} 13\right]$ Jared L. Gearhart, Kristin L. Adair, Richard J. Det
[\\cite{galil1986efficient}] Z. Galil. Efficient algorithms for finding maximum matchi

[\\cite{gallivan1990parallel}] K. Gallivan. Parallel Algorithms for Matrix Computations. SIAM, 19
[\\cite{ghosh2013fast}] Mukulika Ghosh, Nancy M. Amato, Yanyan Lu, and Jyh-Ming Lien. Fast approx
[\\cite{gass2003linear}] S. Gass. Linear Programming: Methods and Applications. Dover, fifth edit
[\\cite{goodfellow2016deep}] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep Learning. M
[\\cite{gonnet1991handbook}] G. Gonnet and R. Baeza-Yates. Handbook of Algorithms and Data Struct
[\\cite{gidney2019how}] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 h
[\\cite{gentle2006random}] James E Gentle. Random Number Generation and Monte Carlo Methods. Spri
[\\cite{garey1977complexity}] M. Garey, R. Graham, and D. Johnson. The complexity of computing St
[\\cite{garey1978bandwidth}] M. Garey, R. Graham, D. Johnson, and D. Knuth. Complexity results fo
[\\cite{graham1985history}] R. Graham and P. Hell. On the history of the minimum spanning tree pr
[\\cite{galinier2006survey}] P. Galinier and A. Hertz. A survey of local search methods for graph
[\\cite{ghaffari2016improved}] Mohsen Ghaffari. An improved distributed algorithm for maximal ind
[\\cite{galinier2013recent}] Philippe Galinier, Jean-Philippe Hamiez, Jin-Kao Hao, and Daniel Por
[\\cite{guibas1993approximating}] L. J. Guibas, J. E. Hershberger, J. S. B. Mitchell, and J. S. S
[\\cite{greenlaw1995limits}] R. Greenlaw, J. Hoover, and W. Ruzzo. Limits to Parallel Computation
[\\cite{gusfield1989stable}] D. Gusfield and R. Irving. The Stable Marriage Problem: Structure an
[\\cite{galil1991data}] Z. Galil and G. Italiano. Data structures and algorithms for disjoint set
[\\cite{ginsberg2018factor}] M. Ginsberg. Factor Man. Zowie Press, 2018.
[\\cite{garey1977rectilinear}] M. Garey and D. Johnson. The rectilinear Steiner tree problem is N
[\\cite{garey1979computers}] M. R. Garey and D. S. Johnson. Computers and Intractability: A Guide
[\\cite{goldwasser2002data}] M. Goldwasser, D. Johnson, and C. McGeoch, editors. Data Structures,
[\\cite{garey1978triangulating}] M. Garey, D. Johnson, F. Preparata, and R. Tarjan. Triangulating
[\\cite{goldberg1995efficient}] A. Goldberg and R. Kennedy. An efficient cost scaling algorithm f
[\\cite{guha1998approximation}] S. Guha and S. Khuller. Approximation algorithms for connected do
[\\cite{glover1974implementation}] F. Glover, D. Karney, and D. Klingman. Implementation and comp
[\\cite{graham1989concrete}] R. Graham, D. Knuth, and O. Patashnik. Concrete Mathematics. Addison
[\\cite{gupta1995improving}] S. Gupta, J. Kececioglu, and A. Schäffer. Improving the practical sp
[\\cite{gomes2008satisfiability}] Carla P. Gomes, Henry Kautz, Ashish Sabharwal, and Bart Selman.
[\\cite{gibson2005discovering}] D. Gibson, R. Kumar, and A. Tomkins. Discovering large dense subg
[\\cite{goldberg2006reach}] A. Goldberg, H. Kaplan, and R. Werneck. Reach for A*: Efficient point
[\\cite{golub1996matrix}] G. Golub and C. Van Loan. Matrix Computations. Johns Hopkins University
[\\cite{gonnet1986heaps}] G. Gonnet and J.. Munro. Heaps on heaps. SIAM J. Computing, 15:964971,
[\\cite{ghosh1991output}] S. Ghosh and D. Mount. An output-sensitive algorithm for computing visi
[\\cite{gordon2012matroids}] Gary Gordon and Jennifer McNulty. Matroids: a geometric introduction
[\\cite{gomes2006experimental}] F. Gomes, C. Meneses, P. Pardalos, and G. Viana. Experimental ana
[\\cite{gajentaan1995class}] Anka Gajentaan and Mark H Overmars. On a class of $\mathrm{O}\left(n
[\\cite{golub2014scientific}] Gene H. Golub and James M. Ortega. Scientific Computing: An Introdu
[\\cite{goemans1997semidefinite}] M. Goemans. Semidefinite programming in combinatorial optimizat
[\\cite{goldberg1993efficient}] L. Goldberg. Efficient Algorithms for Listing Combinatorial Struc
[\\cite{goldberg1997efficient}] A. Goldberg. An efficient implementation of a scaling minimum-cos
[\\cite{goldberg2001shortest}] A. Goldberg. Shortest path algorithms: Engineering aspects. In 12t
[\\cite{golumbic2004algorithmic}] M. Golumbic. Algorithmic Graph Theory and Perfect Graphs, volum
[\\cite{gonthier2008formal}] Georges Gonthier. Formal proof-the four-color theorem. Notices of th
[\\cite{gonzalez2018handbook}] T. Gonzalez. Handbook of Approximation Algorithms and Metaheuristi
[\\cite{gilbert1968steiner}] E. Gilbert and H. Pollak. Steiner minimal trees. SIAM J. Applied Mat
[\\cite{gates1979bounds}] B. Gates and C. Papadimitriou. Bounds for sorting by prefix reversals.

[\\cite{gutin2007traveling}] G. Gutin and A. Punnen. The Traveling Salesman Problem and
[\\cite{gibbs1976comparison}] N. Gibbs, W. Poole, and P. Stockmeyer. A comparison of s
[\\cite{gray1953pulse}] F. Gray. Pulse code communication. US Patent 2632058, March 17
[\\cite{graham1972efficient}] R. Graham. An efficient algorithm for determining the co
[\\cite{gries1989science}] D. Gries. The Science of Programming. Springer-Verlag, 1989
[\\cite{gale1962college}] D. Gale and L. Shapely. College admissions and the stability
[\\cite{gensen1994graph}] T. Gensen and B. Toft. Graph Coloring Problems. John Wiley \
[\\cite{gross2001topological}] Jonathan L. Gross and Thomas W. Tucker. Topological Gra
[\\cite{goldberg2014efficient}] Andrew V. Goldberg and Robert E. Tarjan. Efficient max
[\\cite{goodrich2014data}] Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldw
[\\cite{gupta1966chromatic}] R. P. Gupta. The chromatic index and the degree of a grap
[\\cite{gusfield1994faster}] D. Gusfield. Faster implementation of a shortest superstr
[\\cite{gusfield1997algorithms}] D. Gusfield. Algorithms on Strings, Trees, and Sequen
[\\cite{goemans1995approximation}] M. Goemans and D. Williamson. .878-approximation al
[\\cite{goldberg1996randomness}] I. Goldberg and D. Wagner. Randomness and the Netscap
[\\cite{grossman1997computational}] T. Grossman and A. Wool. Computational experience
[\\cite{hamming1987numerical}] R. Hamming. Numerical Methods for Scientists and Engine
[\\cite{hastad182clique}] H. Hastad. Clique is hard to approximate within $n^{1-\epsil
[\\cite{hall1980approximate}] P. Hall and G. Dowling. Approximate string matching. ACM
[\\cite{hilgemeier2003fast}] M. Hilgemeier, N. Drechsler, and R. Drechsler. Fast heuri
[\\cite{holm2001poly}] J. Holm, K. de lichtenberg, and M. Thorup. Poly-logarithmic det
[\\cite{held2001vroni}] M. Held. VRONI: An engineering approach to the reliable and ef
[\\cite{hyyro2005increased}] H. Hyyro, K. Fredriksson, and G. Navarro. Increased bit-p
[\\cite{heckbert1997survey}] P. Heckbert and M. Garland. Survey of polygonal surface s
[\\cite{hanniel2000two}] I. Hanniel and D. Halperin. Two-dimensional arrangements in C
[\\cite{haran2009experimental}] Idit Haran and Dan Halperin. An experimental study of
[\\cite{harrow2009quantum}] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum
[\\cite{haynes1998fundamentals}] T. Haynes, S. Hedetniemi, and P. Slater. Fundamentals
[\\cite{hassanieh2012simple}] Haitham Hassanieh, Piotr Indyk, Dina Katabi, and Eric Pr
[\\cite{hirschberg1975linear}] D. Hirschberg. A linear-space algorithm for computing m
[\\cite{hopcroft1973n}] J. Hopcroft and R. Karp. An $n^{5 / 3}$ algorithm for maximum
[\\cite{huttenlocher1990computing}] D. P. Huttenlocher and K. Kedem. Computing the min
[\\cite{holzer2011descriptional}] Markus Holzer and Martin Kutrib. Descriptional and c
[\\cite{hemmer2012improved}] Michael Hemmer, Michal Kleinbort, and Dan Halperin. Impro
[\\cite{hormann2004automatic}] W. Hörmann, J. Leydold, and G. Derfinger. Automatic Non
[\\cite{hertel1983fast}] S. Hertel and K. Mehlhorn. Fast triangulation of simple polyg
[\\cite{huang1999cap3}] X. Huang and A. Madan. Cap3: A DNA sequence assembly program.
[\\cite{hershberger2003finding}] J. Hershberger, M. Maxel, and S. Suri. Finding the k
[\\cite{hopcroft2006introduction}] J. Hopcroft, R. Motwani, and J. Ullman. Introductio
[\\cite{heath1991parallel}] Michael T. Heath, Esmond Ng, and Barry W Peyton. Parallel
[\\cite{henzinger2018practical}] Monika Henzinger, Alexander Noe, Christian Schulz, an
[\\cite{hoare1961algorithm}] C. A. R. Hoare. Algorithm 63 (partition) and algorithm 65
[\\cite{hoare1962quicksort}] C. A. R. Hoare. Quicksort. Computer Journal, 5:10-15, 196
[\\cite{hochbaum1996approximation}] D. Hochbaum, editor. Approximation Algorithms for
[\\cite{hoffmann1982group}] C. M. Hoffmann. Group-theoretic algorithms and graph isomo
[\\cite{holland1975adaptation}] J.H. Holland. Adaptation in Natural and Artificial Sys

[\\cite{holyer1981np}] I. Holyer. The NP-completeness of edge colorings. SIAM J. Computing, 10:71
[\\cite{holland1992genetic}] J.H. Holland. Genetic algorithms. Scientific American, 267(1):66-72,
[\\cite{hopcroft1971n}] J. Hopcroft. An $n \log n$ algorithm for minimizing the states in a finit
[\\cite{horspool1980practical}] R. N. Horspool. Practical fast searching in strings. Software-Pra
[\\cite{harary1973graphical}] F. Harary and E. Palmer. Graphical Enumeration. Academic Press, New
[\\cite{hoffman2009set}] Karla Hoffman and Manfred Padberg. Set covering, packing and partitionin
$\left[\mathrm{HPS}^{+} 05\right]$ M. Holzer, G. Prasinos, F. Schulz, D. Wagner, and C. Zaroliagi
[\\cite{hwang1992steiner}] R. Hwang, D. Richards, and P. Winter. The Steiner Tree Problem, volume
[\\cite{henzinger2017local}] Monika Henzinger, Satish Rao, and Di Wang. Local flow partitioning f
[\\cite{hunt1977fast}] J. Hunt and T. Szymanski. A fast algorithm for computing longest common su
[\\cite{hershberger1994douglas}] J. Hershberger and J. Snoeyink. An $O(n \log n)$ implementation
[\\cite{hershberger1998cartographic}] J. Hershberger and J. Snoeyink. Cartographic line simplific
[\\cite{hershberger1999optimal}] J. Hershberger and S. Suri. An optimal algorithm for Euclidean s
[\\cite{halperin2018arrangements}] D. Halperin and M. Sharir. Arrangements. In J. Goodman, J. O'R
[\\cite{hopcroft1987planning}] J. Hopcroft, J. Schwartz, and M. Sharir. Planning, Geometry, and C
[\\cite{hardin2007maximum}] R. Hardin, N. Sloane, and W. Smith. Maximum volume spherical codes. h
[\\cite{halperin2018algorithmic}] D. Halperin, O. Salzman, and M. Sharir. Algorithmic motion plan
[\\cite{holzer2005combining}] M. Holzer, F. Schultz, D. Wagner, and T. Willhalm. Combining speedu
[\\cite{hopcroft1973dividing}] J. Hopcroft and R. Tarjan. Dividing a graph into triconnected comp
[\\cite{hopcroft1973efficient}] J. Hopcroft and R. Tarjan. Efficient algorithms for graph manipul
[\\cite{hopcroft1974efficient}] J. Hopcroft and R. Tarjan. Efficient planarity testing. J. ACM, 2
[\\cite{harel1984fast}] D. Harel and R. E. Tarjan. Fast algorithms for finding nearest common anc
[\\cite{huber2006fast}] M. Huber. Fast perfect sampling from linear extensions. Disc. Math., 306:
[\\cite{huffman1952method}] D. Huffman. A method for the construction of minimum-redundancy codes
[\\cite{haunschmid2002cache}] E. Haunschmid, C. Ueberhuber, and P. Wurzinger. Cache oblivious hig
[\\cite{harvey2019integer}] David Harvey and Joris Van Der Hoeven. Integer multiplication in time
[\\cite{harvey2016even}] David Harvey, Joris Van Der Hoeven, and Grégoire Lecerf. Even faster int
[\\cite{hopcroft1974linear}] J.. Hopcroft and J. K. Wong. Linear time algorithm for isomorphism o
[HWA $\left.{ }^{+} 03\right]$ X. Huang, J. Wang, S. Aluru, S. Yang, and L. Hillier. PCAP: A whol
[\\cite{ibarra1975fast}] O. Ibarra and C. Kim. Fast approximation algorithms for knapsack and sum
[\\cite{indyk2018low}] P. Indyk, J. Matousek, and A. Sidiropoulous. Low-distortion embeddings of
[\\cite{indyk1998faster}] P. Indyk. Faster algorithms for string matching problems: matching the
[\\cite{itai1978finding}] A. Itai and M. Rodeh. Finding a minimum circuit in a graph. SIAM J. Com
[\\cite{iliopoulos2009new}] Costas S. Iliopoulos and M. Sohel Rahman. A new efficient algorithm f
[\\cite{itai1978two}] A. Itai. Two commodity flow. J. ACM, 25:596-611, 1978.
[\\cite{iwata2016linear}] Yoichi Iwata. Linear-time kernelization for feedback vertex set. arXiv
[J92] J. JáJá. An Introduction to Parallel Algorithms. Addison-Wesley, 1992.
[\\cite{jacobson1989space}] G. Jacobson. Space-efficient static trees and graphs. In Proc. Symp.
[\\cite{johnson1991optimization}] D. Johnson, C. Aragon, C. McGeoch, and D. Schevon. Optimizatior
[\\cite{jarvis1973identification}] R. A. Jarvis. On the identification of the convex hull of a fi
[\\cite{jacob2019dynamic}] Riko Jacob and Gerth Stølting Brodal. Dynamic planar convex hull. arXi
[\\cite{jain1988algorithms}] A. Jain and R. Dubes. Algorithms for Clustering Data. Prentice-Hall,
[\\cite{janson2000random}] S. Janson, T. Luczak, and A. Rucinski. Random Graphs. John Wiley \& So
[\\cite{johnson1993network}] D. Johnson and C. McGeoch, editors. Network Flows and Matching: Firs
[\\cite{junger2012graph}] Michael Jünger and Petra Mutzel. Graph drawing software. Springer Scien
[\\cite{johnson1963generation}] S. M. Johnson. Generation of permutations by adjacent transpositi

[\\cite{johnson1974approximation}] D. Johnson. Approximation algorithms for combinatori
[\\cite{jones1986empirical}] D. W. Jones. An empirical comparison of priority-queue anc
[\\cite{josuttis2012cpp}] N. Josuttis. The C++ Standard Library: A Tutorial and Referer
[\\cite{jiang1993minimal}] T. Jiang and B. Ravikumar. Minimal NFA problems are hard. SI
[\\cite{johnson1991treemaps}] Brian Johnson and Ben Shneiderman. Tree-Maps: A Space-Fil
[\\cite{jagota2001adaptive}] A. Jagota and L. Sanchis. Adaptive, restart, randomized gr
[\\cite{jerrum2004polynomial}] Mark Jerrum, Alistair Sinclair, and Eric Vigoda. A polyn
[\\cite{johnson1996cliques}] D. Johnson and M. Trick. Cliques, Coloring, and Satisfiabi
[\\cite{juhl2018geosteiner}] Daniel Juhl, David M Warme, Pawel Winter, and Martin Zacha
[\\cite{ko2003space}] P. Ko and S. Aluru. Space-efficient linear time construction of s
[\\cite{kodituwakku2010comparison}] S.R. Kodituwakku and U.S. Amarasinghe. Comparison o
[\\cite{kahn1967code}] D. Kahn. The Code Breakers: The Story of Secret Writing. Macmil
[\\cite{karp1972reducibility}] R. M. Karp. Reducibility among combinatorial problems. I
[\\cite{karmarkar1984new}] N. Karmarkar. A new polynomial-time algorithm for linear pro
[\\cite{karloff1996how}] H. Karloff. How good is the Goemans-Williamson MAX CUT algorit
[\\cite{karger200mincuts}] D. Karger. Minimum cuts in near-linear time. J. ACM, 47:46-7
[\\cite{kroller2012artgallery}] Alexander Kröller, Tobias Baumgartner, Sándor P Fekete
[\\cite{koren2009matrix}] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factor
[\\cite{keil2000polygon}] M. Keil. Polygon decomposition. In J.R. Sack and J. Urrutia,
[\\cite{karaman2011sampling}] Sertac Karaman and Emilio Frazzoli. Sampling-based algor
[\\cite{kirkpatrick1983optimization}] S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vec
[\\cite{khachian1979polynomial}] L. Khachian. A polynomial algorithm in linear programm
[\\cite{kirkpatrick1979efficient}] D. Kirkpatrick. Efficient computation of continuous
[\\cite{kirkpatrick1983optimal}] D. Kirkpatrick. Optimal search in planar subdivisions
[\\cite{karger1995randomized}] D. Karger, P. Klein, and R. Tarjan. A randomized linear-
[\\cite{kernighan1970efficient}] B. W. Kernighan and S. Lin. An efficient heuristic pro
$\left[\mathrm{KLM}^{+} 14\right]$ Raimondas Kiveris, Silvio Lattanzi, Vahab Mirrokni,
[\\cite{klee1972how}] V. Klee and G. Minty. How good is the simplex algorithm. In Inequ
[\\cite{kececioglu1995combinatorial}] J.D. Kececioglu and E. W. Myers. Combinatorial al
[\\cite{knuth1977fast}] D. Knuth, J. Morris, and V. Pratt. Fast pattern matching in str
$\left[K_{M P}{ }^{+} 04\right]$ L. Kettner, K. Mehlhorn, S. Pion, S. Schirra, and C. Y
[\\cite{karger1997approximating}] David Karger, Rajeev Motwani, and Gurumurthy D.S. Ran
[\\cite{khanna1997efficiently}] S. Khanna, M. Muthukrishnan, and S. Skiena. Efficiently
[\\cite{komlos1998matching}] János Komlós, Yuan Ma, and Endre Szemerédi. Matching nuts
[\\cite{knuth1994graphbase}] D. Knuth. The Stanford GraphBase: A Platform for Combinato
[\\cite{knuth1997taocp1}] D. Knuth. The Art of Computer Programming, Volume 1: Fundamen
[\\cite{knuth1997taocp2}] D. Knuth. The Art of Computer Programming, Volume 2: Seminume
[\\cite{knuth1998taocp3}] D. Knuth. The Art of Computer Programming, Volume 3: Sorting
[\\cite{knuth2011taocp4a}] D. Knuth. The Art of Computer Programming, Volume 4A: Combin
[\\cite{knuth2015taocp4f6}] D. Knuth. The Art of Computer Programming, Volume 4, Fascic
[\\cite{karatsuba1963multiplication}] A. Karatsuba and Yu. Ofman. Multiplication of mul
[\\cite{koehler2005contraction}] H. Koehler. A contraction algorithm for finding minima
[\\cite{kaul1991minkowski}] A. Kaul, M. A. O'Connor, and V. Srinivasan. Computing Minko
[\\cite{kececioglu1998matchings}] J. Kececioglu and J. Pecqueur. Computing maximum-card
[\\cite{kellerer2004knapsack}] H. Kellerer, U. Pferschy, and P. Pisinger. Knapsack Prob
[\\cite{karp1987randomized}] R. Karp and M. Rabin. Efficient randomized pattern-matchin

[\\cite{kanevsky1991fourconnectivity}] A. Kanevsky and V. Ramachandran. Improved algorithms for g
[\\cite{khot2008vertex}] Subhash Khot and Oded Regev. Vertex cover might be hard to approximate t
[\\cite{kruskal1956spanning}] J.B. Kruskal. On the shortest spanning subtree of a graph and the t
[\\cite{knuth1974topological}] D.E. Knuth and J.L. Szwarcfiter. A structured program to generate
[\\cite{keil1985decomposition}] M. Keil and J.R. Sack. Minimum decomposition of geometric objects
[\\cite{kirkpatrick1986ultimate}] D. Kirkpatrick and R. Siedel. The ultimate planar convex hull a
[\\cite{kreher1999combinatorial}] D. Kreher and D. Stinson. Combinatorial Algorithms: Generation,
[\\cite{keil2002time}] M. Keil and J. Snoeyink. On the time bound for convex decomposition of sim
[\\cite{kaplan2005greedy}] H. Kaplan and N. Shafrir. The greedy algorithm for shortest superstrin
[\\cite{kelner2005randomized}] J. Kelner and D. Spielman. A randomized polynomial-time simplex al
[\\cite{kautz2007state}] H. Kautz and B. Selman. The state of SAT. Disc. Applied Math., 155:1514-
[\\cite{karkkainen2006linear}] Juha Kärkkäinen, Peter Sanders, and Stefan Burkhardt. Linear work
[\\cite{kautz2007satisfiability}] H. Kautz, B. Selman, R. Brachman, and T. Dietterich. Satisfiabi
[\\cite{kim2003linear}] D. Kim, J. Sim, H. Park, and K. Park. Linear-time construction of suffix
[\\cite{kobler1993graph}] J. Köbler, U. Schöning, and J. Túran. The Graph Isomorphism Problem: It
[\\cite{keyes1997parallel}] D. Keyes, A. Sameh, and V. Venkatarishnan. Parallel Numerical Algorit
[\\cite{kleinberg2006algorithm}] J. Kleinberg and E. Tardos. Algorithm Design. Addison Wesley, 20
[\\cite{kuratowski1930sur}] K. Kuratowski. Sur le problème des courbes gauches en topologie. Fund
[\\cite{kaufmann2001drawing}] M. Kaufmann and D. Wagner. Drawing Graphs: Methods and Models. Spri
[\\cite{kwan1962graphic}] M. Kwan. Graphic programming using odd and even points. Chinese Math.,
[\\cite{leung2004handbook}] J. Leung and J. Anderson, editors. Handbook of Scheduling: Algorithms
[\\cite{lien2006approximate}] J. Lien and N. Amato. Approximate convex decomposition of polygons.
[\\cite{lambert1992sorting}] J.-L. Lambert. Sorting the sums $\left(x_{i}+y_{j}\right)$ in $O\lef
[\\cite{latombe1991robot}] J.-C. Latombe. Robot Motion Planning. Kluwer, 1991.
[\\cite{laumond1998robot}] J. Laumond. Robot Motion Planning and Control. Springer-Verlag, Lectur
[\\cite{lavalle2006planning}] S. LaValle. Planning Algorithms. Cambridge University Press, 2006.
[\\cite{lawler2011combinatorial}] E. Lawler. Combinatorial Optimization: Networks and Matroids. D
[\\cite{laycock2003automatically}] R. Laycock and A. Day. Automatically generating roof models fr
[L'E12] Pierre L'Ecuyer. Random number generation. In Handbook of Computational Statistics, pages
[\\cite{lecroq1995experimental}] T. Lecroq. Experimental results on string matching algorithms. S
[\\cite{lee1982medial}] D. T. Lee. Medial axis transformation of a planar shape. IEEE Trans./ Pat
[\\cite{lengauer1987efficient}] T. Lengauer. Efficient algorithms for finding minimum spanning fo
[\\cite{lenstra1987factoring}] H. W. Lenstra. Factoring integers with elliptic curves. Annals of
[\\cite{lengauer1989hierarchical}] T. Lengauer. Hierarchical planarity testing algorithms. J. ACM
[\\cite{lengauer1990combinatorial}] T. Lengauer. Combinatorial Algorithms for Integrated Circuit
[\\cite{lamarca1996influence}] A. LaMarca and R. Ladner. The influence of caches on the performan
[\\cite{lamarca1999influence}] A. LaMarca and R. Ladner. The influence of caches on the performan
[\\cite{lin2012accelerating}] Cheng-Hung Lin, Chen-Hsiung Liu, Lung-Sheng Chien, and Shih-Chieh C
[\\cite{lenstra1983theory}] J.K. Lenstra, E. L. Lawler, and A. Rinnooy Kan. Theory of Sequencing
[\\cite{lawler1985traveling}] E. Lawler, J. Lenstra, A. Rinnooy Kan, and D. Shmoys. The Traveling
[\\cite{lam1992thinning}] L. Lam, S.-W. Lee, and C. Suen. Thinning methodologies-a comprehensive
[\\cite{lin2018collision}] M. Lin, D. Manocha, and Y. Kim. Collision and proximity queries. In J.
$\left[\mathrm{LMM}^{+} 18\right]$ Daniel Lokshtanov, Pranabendu Misra, Joydeep Mukherjee, Geevar
[\\cite{lloyd2006identifying}] L. Lloyd, A. Mehler, and S. Skiena. Identifying co-referential nam
[\\cite{langdon2002foundations}] W. Langdon and R. Poli. Foundations of Genetic Programming. Spri
[\\cite{lodi2007tsp}] A. Lodi and A. Punnen. TSP software. In G. Gutin and A. Punnen, editors, Th

[\\cite{lovasz2009matching}] László Lovász and Michael D. Plummer. Matching Theory, vol
[\\cite{livnat2016sex}] Adi Livnat and Christos H. Papadimitriou. Sex as an algorithm:
[\\cite{lozanoperez1979algorithm}] T. Lozano-Perez and M. Wesley. An algorithm for plan
[\\cite{lang1993finding}] K. Lang and S. Rao. Finding near-optimal cuts: An empirical e
[\\cite{laborie2018ibm}] Philippe Laborie, Jérôme Rogerie, Paul Shaw, and Petr Vilím. I
[\\cite{lumelski1987path}] V. Lumelski and A. Stepanov. Path planning strategies for a
[\\cite{lin1995algorithms}] Y.-L. Lin and S. Skiena. Algorithms for square roots of gra
[\\cite{lecuyer2002object}] P. L'Ecuyer, R. Simard, E. Chen, and W. D. Kelton. An objec
[\\cite{larkin2014back}] Daniel H. Larkin, Siddhartha Sen, and Robert E. Tarjan. A back
$\left[L^{+} Z^{+} 15\right]$ Liang Liu, Yuning Song, Haiyang Zhang, Huadong Ma, and At
[\\cite{lipton1979separator}] R. Lipton and R. Tarjan. A separator theorem for planar g
[\\cite{lipton1980applications}] R. Lipton and R. Tarjan. Applications of a planar sepa
[\\cite{lucas1891recreations}] E. Lucas. Récréations Mathématiques. Gauthier-Villares,
[\\cite{luks1980isomorphism}] E. M. Luks. Isomorphism of bounded valence can be tested
[\\cite{landau1988fast}] G. Landau and U. Vishkin. Fast string matching with k differ
[\\cite{li1997introduction}] M. Li and P. Vitányi. An Introduction to Kolmogorov Comple
[\\cite{lee1977worst}] D. T. Lee and C. K. Wong. Worst-case analysis for region and par
[\\cite{lengauer1988efficient}] T. Lengauer and E. Wanke. Efficient solution of connect
[$\left.\mathrm{M}^{+} 13\right] \quad$ Aranyak Mehta et al. Online matching and ad al
[\\cite{maheshwari1976traversal}] S. Maheshwari. Traversal marker placement problems ar
[\\cite{maier1978complexity}] D. Maier. The complexity of some problems on subsequences
[\\cite{mak2002java}] R. Mak. Java Number Cruncher: The Java Programmer's Guide to Nume
[\\cite{manber1989introduction}] U. Manber. Introduction to Algorithms. Addison-Wesley
[\\cite{mansour2012combinatorics}] Toufik Mansour. Combinatorics of Set Partitions. Cha
[MAR $\left.{ }^{+} 17\right]$ Amgad Madkour, Walid G. Aref, Faizan Ur Rehman, Mohamed
[\\cite{matula1987determining}] D. W. Matula. Determining edge connectivity in $O(n m)$
[\\cite{mccreight1976space}] E. McCreight. A space-economical suffix tree construction
[\\cite{mckay1981practical}] B. McKay. Practical graph isomorphism. Congressus Numerant
[\\cite{musser2001stl}] D. Musser, G. Derge, and A. Saini. STL Tutorial and Reference G
[\\cite{menger1927zur}] K. Menger. Zur allgemeinen Kurventheorie. Fund. Math., 10:96-11
[\\cite{meyers2001effective}] S. Meyers. Effective STL: 50 Specific Ways to Improve You
[\\cite{michalewicz2000how}] Z. Michalewicz and D. Fogel. How To Solve It: Modern Heuri
[\\cite{misra1992constructive}] J. Misra and D. Gries. A constructive proof of Vizing's
[\\cite{matousek2007understanding}] Jiri Matousek and Bernd Gärtner. Understanding and
[\\cite{merkle1978hiding}] R. Merkle and M. Hellman. Hiding and signatures in trapdoor
[\\cite{maaten2008visualizing}] Laurens van der Maaten and Geoffrey Hinton. Visualizing
[\\cite{miller1976riemann}] G. Miller. Riemann's hypothesis and tests for primality. J
[\\cite{milenkovic1997multiple}] V. Milenkovic. Multiple translational containment. par
[\\cite{minc1978permanents}] H. Minc. Permanents, volume 6 of Encyclopedia of Mathemati
[\\cite{mitchell1999guillotine}] J. Mitchell. Guillotine subdivisions approximate polyg
[\\cite{mardis2007advances}] E. Mardis, S. Kim, and H. Tang, editors. Advances in Genom
[\\cite{muja2014scalable}] Marius Muja and David G Lowe. Scalable nearest neighbor algo
[\\cite{minguez2016motion}] Javier Minguez, Florant Lamiraux, and Jean-Paul Laumond. Mo
[\\cite{manber1993suffix}] U. Manber and G. Myers. Suffix arrays: A new method for on-l
[\\cite{mehlhorn1996embedding}] K. Mehlhorn and P. Mutzel. On the embedding phase of th
[\\cite{matula1972graph}] D. Matula, G. Marble, and J. Isaacson. Graph coloring algorit

[\\cite{matsumoto1998mersenne}] M. Matsumoto and T. Nishimura. Mersenne twister: A 623dimensional
[\\cite{mehlhorn1999leda}] K. Mehlhorn and S. Naher. LEDA: A Platform for Combinatorial and Geome
[\\cite{makinen2007compressed}] V. Makinen and G. Navarro. Compressed full text indexes. ACM Comp
$\left[M_{N M}{ }^{+} 16\right]$ Thomas Monz, Daniel Nigg, Esteban A. Martinez, Matthias F. Brand
[\\cite{moses1963tables}] L. E. Moses and R. V. Oakford. Tables of Random Permutations. Stanford
[\\cite{moore1959shortest}] E. F. Moore. The shortest path in a maze. In Proc. International Symp
[\\cite{mehlhorn2011general}] Kurt Mehlhorn, Ralf Osbild, and Michael Sagraloff. A general approa
[\\cite{mount2018geometric}] D. Mount. Geometric intersection. In J. Goodman, J. O'Rourke, and C.
[\\cite{menezes1996handbook}] A. Menezes, P. Oorschot, and S. Vanstone. Handbook of Applied Crypt
[\\cite{masek1980faster}] W. Masek and M. Paterson. A faster algorithm for computing string edit
[\\cite{mckay2014practical}] Brendan D McKay and Adolfo Piperno. Practical graph isomorphism, ii.
[MPC $\left.{ }^{+} 06\right]$ S. Mueller, D. Papamichial, J.R. Coleman, S. Skiena, and E. Wimmer
[\\cite{martello1999dynamic}] S. Martello, D. Pisinger, and P. Toth. Dynamic programming and stro
[\\cite{martello2000new}] S. Martello, D. Pisinger, and P. Toth. New trends in exact algorithms f
[\\cite{motwani1995randomized}] R. Motwani and P. Raghavan. Randomized Algorithms. Cambridge Univ
[\\cite{myrvold2001ranking}] W. Myrvold and F. Ruskey. Ranking and unranking permutations in line
[\\cite{mulzer2006minimum}] W. Mulzer and G. Rote. Minimum weight triangulation is NP-hard. In Pr
[\\cite{mustafa2010improved}] Nabil H Mustafa and Saurabh Ray. Improved results on geometric hitt
[\\cite{metropolis1953equation}] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, and A. H. Tel
[\\cite{moret1991algorithm}] B. Moret and H. Shapiro. Algorithm from P to NP: Design and Efficien
[\\cite{margaritis1995reconstructing}] D. Margaritis and S. Skiena. Reconstructing strings from s
[\\cite{mitchell1995separation}] J. S. B. Mitchell and S. Suri. Separation and approximation of p
[\\cite{mascagni2000sprng}] M. Mascagni and A. Srinivasan. Algorithm 806: Sprng: A scalable libra
[\\cite{mehta2018handbook}] D. Mehta and S. Sahni. Handbook of Data Structures and Applications.
[\\cite{martello1985program}] S. Martello and P. Toth. A program for the $0-1$ multiple knapsack
[\\cite{martello1987algorithms}] S. Martello and P. Toth. Algorithms for knapsack problems. In S.
[\\cite{martello1990knapsack}] S. Martello and P. Toth. Knapsack Problems: Algorithms and Compute
[\\cite{mehlhorn1990data}] K. Mehlhorn and A. Tsakalidis. Data structures. In J. van Leeuwen, edi
[\\cite{malaguti2010survey}] Enrico Malaguti and Paolo Toth. A survey on vertex coloring problems
[\\cite{meindl2012analysis}] Bernhard Meindl and Matthias Templ. Analysis of commercial and free
[\\cite{mitzenmacher2017probability}] M. Mitzenmacher and E. Upfal. Probability and Computing: Ra
[\\cite{mucha2013lyndon}] Marcin Mucha. Lyndon words and short superstrings. In Proceedings of th
[\\cite{mulmuley1994computational}] K. Mulmuley. Computational Geometry: An Introduction Through
[\\cite{muthukrishnan2005data}] S. Muthukrishnan. Data Streams: Algorithms and Applications. Now
[\\cite{micali1980algorithm}] S. Micali and V. Vazirani. An $O(\sqrt{|V|}|E|)$ algorithm for find
[\\cite{mccullough1999numerical}] B. McCullough and H. Vinod. The numerical reliability of econom
[\\cite{mehlhorn2007robust}] K. Mehlhorn and C. Yap. Robust Geometric Computation. manuscript, ht
[\\cite{myers1986difference}] E. Myers. An $\mathrm{O}(n d)$ difference algorithm and its variati
[\\cite{myers1999whole}] E. Myers. Whole-genome DNA sequencing. IEEE Computational Engineering an
[\\cite{myers1999fast}] G. Myers. A fast bit-vector algorithm for approximate string matching bas
[\\cite{navarro2001guided}] G. Navarro. A guided tour to approximate string matching. ACM Computi
[\\cite{navarro2001nr}] G. Navarro. Nr-grep: a fast and flexible pattern matching tool. Software
[\\cite{nielsen2002quantum}] Michael A Nielsen and Isaac Chuang. Quantum Computation and Quantum
[\\cite{nesterov2013introductory}] Yurii Nesterov. Introductory Lectures on Convex Optimization:
[\\cite{vonneumann1963various}] J.on Neumann. Various techniques used in connection with random d
[\\cite{newman2018networks}] Mark Newman. Networks. Oxford university press, 2018.

[\\cite{needham2019graph}] M. Needham and A. Hodler. Graph Algorithms: Practical Exampl
[\\cite{nagamochi2008algorithmic}] Hiroshi Nagamochi and Toshihide Ibaraki. Algorithmic
[\\cite{nobari2011fast}] Sadegh Nobari, Xuesong Lu, Panagiotis Karras, and Stéphane Br
[\\cite{notredame2002recent}] C. Notredame. Recent progress in multiple sequence align
[\\cite{navarro2000fast}] G. Navarro and M. Raffinot. Fast and flexible string matching
[\\cite{nishizeki2004planar}] T. Nishizeki and S. Rahman. Planar Graph Drawing. World S
[\\cite{navarro2007flexible}] G. Navarro and M. Raffinot. Flexible Pattern Matching in
[\\cite{narasimhan2007geometric}] G. Narasimhan and M. Smid. Geometric Spanner Network
[\\cite{nuutila1995efficient}] E. Nuutila. Efficient transitive closure computation in
[\\cite{nijenhuis1978combinatorial}] A. Nijenhuis and H. Wilf. Combinatorial Algorithm
[\\cite{naher2002design}] S. Näher and O. Zlotowski. Design and implementation of effi
[\\cite{niven1991introduction}] I. Niven, H. Zuckerman, and H. Montgomery. An Introduc
[\\cite{okabe2000spatial}] A. Okabe, B. Boots, K. Sugihara, and S. Chiu. Spatial Tesse
[\\cite{ogniewicz1993discrete}] R. Ogniewicz. Discrete Voronoi Skeletons. Hartung-Gorre
[\\cite{omohundro1989five}] Stephen M. Omohundro. Five Balltree Construction Algorithm
[O'R85] J. O'Rourke. Finding minimal enclosing boxes. Int. J. Computer and Information
[O'R87] J. O'Rourke. Art Gallery Theorems and Algorithms. Oxford University Press, 198
[O'R01] J. O'Rourke. Computational Geometry in C. Cambridge University Press, second e
[\\cite{orlin2013max}] James B. Orlin. Max flows in $O(n m)$ time, or better. In Proce
[\\cite{orourke2018polygons}] J. O'Rourke, S. Suri, and C. Toth. Polygons. In J. Goodm
[\\cite{omalley2008terabyte}] Owen OMalley. Terabyte sort on apache hadoop. http:// so
[\\cite{paluch2014better}] Katarzyna Paluch. Better approximation algorithms for maxim
[\\cite{panigrahy2006hashing}] R. Panigrahy. Hashing, Searching, Sketching. PhD thesis
[\\cite{papadimitriou1976edge}] C. Papadimitriou. The complexity of edge traversing. J
[\\cite{papadimitriou1976bandwidth}] C. Papadimitriou. The NP-completeness of the band
[\\cite{parker1990better}] G. Parker. A better phonetic search. C Gazette, 5-4, June/J
[\\cite{perozzi2014deepwalk}] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk
[\\cite{paschos1997survey}] V. Paschos. A survey of approximately optimal solutions to
[\\cite{paschos2003polynomial}] V. Paschos. Polynomial approximation and graph-colorin
[\\cite{patrignani2013planarity}] Maurizio Patrignani. Planarity testing and embedding
[\\cite{pavlidis1982algorithms}] T. Pavlidis. Algorithms for Graphics and Image Proces
$\left[\mathrm{PY}^{+}{ }^{16}]\right.$ Brian Paden, Michal áp, Sze Zheng Yong, Dmitry
[\\cite{peczarski2004new}] M. Peczarski. New results in minimum-comparison sorting. Al
[\\cite{peczarski2007ford}] M. Peczarski. The Ford-Johnson algorithm still unbeaten fo
[\\cite{petit2003experiments}] J. Petit. Experiments on the minimum linear arrangement
[\\cite{press2007numerical}] W. Press, B. Flannery, S. Teukolsky, and W. T. Vetterling
[\\cite{padberg1980symmetric}] M. Padberg and S. Hong. On the symmetric traveling sale
[\\cite{perl1978interpolation}] Y. Perl, A. Itai, and H. Avni. Interpolation search-a
[\\cite{pinedo2016scheduling}] M. Pinedo. Scheduling: Theory, Algorithms, and Systems.
[\\cite{pevzner1994towards}] P. A. Pevzner and R. J. Lipshutz. Towards DNA sequencing
[\\cite{panneton2006improved}] F. Panneton, P. L'Ecuyer, and M. Matsumoto. Improved lo
[\\cite{park1988random}] S. Park and K. Miller. Random number generators: Good ones ar
[\\cite{mitchell2018shortest}] Shortest Paths and Networks. J. Mitchell. In J. Goodman
[\\cite{polya1957how}] G. Polya. How to Solve It. Princeton University Press, second e
[\\cite{pomerance1984quadratic}] C. Pomerance. The quadratic sieve factoring algorithm
[\\cite{penner2006cache}] M. Penner and V. Prasanna. Cache-friendly implementations of

[\\cite{pruesse1994generating}] G. Pruesse and F. Ruskey. Generating linear extensions fast. SIAM
[\\cite{pettie2002optimal}] S. Pettie and V. Ramachandran. An optimal minimum spanning tree algor
[\\cite{pratt1975every}] V. Pratt. Every prime has a succinct certificate. SIAM J. Computing, 4:2
[\\cite{prim1957shortest}] R. C. Prim. Shortest connection networks and some generalizations. Bel
[Prü18] H. Prüfer. Neuer Beweis eines Satzes über Permutationen. Arch. Math. Phys., 27:742-744, 1
[\\cite{preparata1985computational}] F. Preparata and M. Shamos. Computational Geometry. Springer
[\\cite{papadimitriou1998combinatorial}] C. Papadimitriou and K. Steiglitz. Combinatorial Optimiz
[\\cite{proemel2002steiner}] H. Prömel and A. Steger. The Steiner Tree Problem: A Tour Through Gr
[\\cite{pemmaraju2003computational}] S. Pemmaraju and S. Skiena. Computational Discrete Mathemati
[\\cite{pothen1990partitioning}] A. Pothen, H. Simon, and K. Liou. Partitioning sparse matrices w
[\\cite{putze2007cache}] F. Putze, P. Sanders, and J. Singler. Cache-, hash-, and space-efficient
[\\cite{puglisi2007taxonomy}] S. Puglisi, W. Smyth, and A. Turpin. A taxonomy of suffix array con
[\\cite{pavlides1992information}] T. Pavlides, J. Swartz, and Y. Wang. Information encoding with
[\\cite{pothen2005cache}] A. Pothen and S. Toledo. Cache-oblivious data structures. In D. Mehta a
[\\cite{panigrahy2011heuristics}] Rina Panigrahy, Kunal Talwar, Lincoln Uyeda, and Udi Wieder. He
[\\cite{pugh1986partitioning}] G. Allen Pugh. Partitioning for selective assembly. Computers and
[\\cite{pocchiola1996topologically}] M. Pocchiola and G. Vegter. Topologically sweeping visibilit
[\\cite{rabin1980probabilistic}] M. Rabin. Probabilistic algorithm for testing primality. J. Numb
[\\cite{raman2005data}] R. Raman. Data structures for sets. In D. Mehta and S. Sahni, editors, Ha
[\\cite{rawlins1992compared}] G. Rawlins. Compared to What? Computer Science Press, New York, 199
[\\cite{romero2004experimental}] H. Romero, C. Brizuela, and A. Tchernykh. An experimental compar
[\\cite{rand1955million}] Rand-Corporation. A Million Random Digits with 100,000 Normal Deviates.
[\\cite{reingold2018calendrical}] Edward M. Reingold and Nachum Dershowitz. Calendrical Calculati
[\\cite{reingold1993calendrical}] E. Reingold, N. Dershowitz, and S. Clamen. Calendrical calculat
[\\cite{reingold1972optimality}] E. Reingold. On the optimality of some set algorithms. J. ACM, 1
[\\cite{reinelt1991tsplib}] G. Reinelt. TSPLIB-a traveling salesman problem library. ORSA J. Comp
[\\cite{reinelt1994traveling}] G. Reinelt. The traveling salesman problem: Computational solution
[\\cite{roger2006jflap}] S. Roger and T. Finley. JFLAP: An Interactive Formal Languages and Autom
[\\cite{resende1998algorithm787}] M. Resende, T. Feo, and S. Smith. Algorithm 787: Fortran subrou
[\\cite{richter2007stochastic}] S. Richter, M. Helert, and C. Gretton. A stochastic local search
[\\cite{robison1989eight}] A. Robison, B. Hafner, and S. Skiena. Eight pieces cannot cover a ches
[\\cite{rivest1992md5}] R. Rivest. The MD5 message digest algorithm. RFC 1321, 1992.
[\\cite{roughgarden2017algorithms}] T. Roughgarden. Algorithms Illuminated, volume 1. Soundlikeyo
[\\cite{ribeiro1999algorithm797}] C.C. Ribeiro and M.G.C. Resende. Algorithm 797: Fortran subrout
[\\cite{rau1996dialing}] H. Rau and S. Skiena. Dialing for documents: an experiment in informatio
[\\cite{rivest1978method}] R. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital s
[\\cite{rosenkrantz1977analysis}] D. Rosenkrantz, R. Stearns, and P. M. Lewis. An analysis of sev
[\\cite{rafalin2002topological}] E. Rafalin, D. Souvaine, and I. Streinu. Topological sweep in de
[\\cite{robertson1996efficiently}] N. Robertson, D. Sanders, P. Seymour, and R. Thomas. Efficient
[\\cite{ruskey2003combinatorial}] F. Ruskey. Combinatorial Generation. Preliminary working draft.
[\\cite{ruskey2009coolest}] Frank Ruskey and Aaron Williams. The coolest way to generate combinat
[\\cite{raghvendra2018grid}] Sharath Raghvendra and Mariëtte C Wessels. A grid-based approximatio
[\\cite{rytter1985fast}] W. Rytter. Fast recognition of pushdown automata and context-free langua
[\\cite{robins2005improved}] G. Robins and A. Zelikovsky. Improved Steiner tree approximation in
[\\cite{sharir1995davenport}] M. Sharir and P. Agarwal. Davenport-Schinzel Sequences and Their Ge
[\\cite{sahni2005double}] S. Sahni. Double-ended priority queues. In D. Mehta and S. Sahni, edito

[\\cite{salomon2006data}] D. Salomon. Data Compression: The Complete Reference. Spring
[\\cite{samet2005multidimensional}] H. Samet. Multidimensional spatial data structures
[\\cite{samet2006foundations}] H. Samet. Foundations of Multidimensional and Metric Da
[\\cite{sanders2000fast}] P. Sanders. Fast priority queues for cached memory. ACM Jour
[\\cite{savage1997survey}] C. Savage. A survey of combinatorial gray codes. SIAM Revie
[\\cite{saxe1980dynamic}] J.B. Saxe. Dynamic programming algorithms for recognizing sm
[\\cite{sayood2017introduction}] K. Sayood. Introduction to Data Compression. Morgan K
[\\cite{samorodnitsky2001distance}] A. Samorodnitsky and A. Barvinok. The distance app
[\\cite{sun2019parallel}] Yihan Sun and Guy E. Blelloch. Parallel range, segment and r
[\\cite{saha2016survey}] Punam K. Saha, Gunilla Borgefors, and Gabriella Sanniti di Ba
[\\cite{soltanpour2017survey}] Sima Soltanpour, Boubakeur Boufama, and Q.M. Jonathan W
[\\cite{schrijver1998bipartite}] A. Schrijver. Bipartite edge-coloring in $\mathrm{O}(
[\\cite{schulz2011treevis}] Hans-Jorg Schulz. Treevis.net: A tree visualization refere
[\\cite{schneier2015applied}] B. Schneier. Applied Cryptography: Protocols, Algorithms
[\\cite{syslo1975computational}] M. Syslo and J. Dzikiewicz. Computational experiences
[\\cite{schmidt1976fast}] D. C. Schmidt and L. E. Druffel. A fast backtracking algorit
[\\cite{shewchuk2016delaunay}] Jonathan Shewchuk, Tamal K Dey, and Siu-Wing Cheng. Del
[\\cite{syslo1983discrete}] M. Syslo, N. Deo, and J. Kowalik. Discrete Optimization Al
[\\cite{sedgewick1977permutation}] R. Sedgewick. Permutation generation methods. Compu
[\\cite{sedgewick1978implementing}] R. Sedgewick. Implementing quicksort programs. Com
[\\cite{sedgewick1998algorithms}] R. Sedgewick. Algorithms in C++, Parts 1-4: Fundamen
[\\cite{seidel2018convex}] R. Seidel. Convex hull computations. In J. Goodman, J. O'Ro
[\\cite{shore1982algorithm}] M. Shore, L. Foulds, and P. Gibbons. An algorithm for the
[\\cite{shamos1975closest}] M. Shamos and D. Hoey. Closest point problems. In Proc. Si
[\\cite{shih1999new}] W. Shih and W. Hsu. A new planarity test. Theoretical Computer S
[\\cite{sharir1987efficient}] M. Sharir. Efficient algorithms for planning purely tran
[\\cite{shewchuk1997robust}] J.R. Shewchuk. Robust adaptive floating-point geometric p
[\\cite{satish2009designing}] Nadathur Satish, Mark Harris, and Michael Garland. Desig
[\\cite{shor1999polynomial}] Peter W. Shor. Polynomial-time algorithms for prime facto
[\\cite{shoup2009computational}] V. Shoup. A Computational Introduction to Number Theo
[\\cite{si2015tetgen}] Hang Si. Tetgen, a delaunay-based quality tetrahedral mesh gene
[\\cite{sinclair2012algorithms}] Alistair Sinclair. Algorithms for Random Generation a
[\\cite{sipser2005introduction}] M. Sipser. Introduction to the Theory of Computation.
[\\cite{saaty1986four}] T. Saaty and P. Kainen. The Four-Color Problem. Dover, New Yor
[\\cite{sankoff1999time}] D. Sankoff and J. Kruskal. Time Warps, String Edits, and Mac
[\\cite{skeel2000elementary}] R. Skeel and J. Keiper. Elementary Numerical Computing w
[\\cite{skiena1988encroaching}] S. Skiena. Encroaching lists as a measure of presorted
[\\cite{skiena1990implementing}] S. Skiena. Implementing Discrete Mathematics. Addison
[\\cite{skiena1999who}] S. Skiena. Who is interested in algorithms and why?: lessons f
[\\cite{skiena2012redesigning}] S. Skiena. Redesigning viral genomes. IEEE Computer, 4
[\\cite{singh2007approximating}] M. Singh and L. Lau. Approximating minimum bounded de
[\\cite{siek2002boost}] J. Siek, L. Lee, and A. Lumsdaine. The Boost Graph Library: Us
$\left[\right.$ SLW $\left.^{+} 12\right]$ Y. Song, Y. Liu, C. Ward, S. Mueller, B. Fu
[\\cite{stockmeyer1973word}] L. Stockmeyer and A. Meyer. Word problems requiring expon
[\\cite{salomon2010handbook}] David Salomon and Giovanni Motta. Handbook of Data Compr
[MK12] Ioan A. ucan, Mark Moll, and Lydia E. Kavraki. The Open Motion Planning Library

[\\cite{snoeyink2018point}] J. Snoeyink. Point location. In J. Goodman, J. O'Rourke, and C. Toth,
[\\cite{siddiqi2008medial}] Kaleem Siddiqi and Stephen Pizer. Medial Representations: Mathematics
[\\cite{simpson2015theory}] Jared T. Simpson and Mihai Pop. The theory and practice of genome seq
[\\cite{supowit1983complexity}] K. Supowit and E. Reingold. The complexity of drawing trees nicel
[\\cite{skiena2003programming}] S. Skiena and M. Revilla. Programming Challenges: The Programming
[\\cite{slota2014bfs}] George M Slota, Sivasankaran Rajamanickam, and Kamesh Madduri. Bfs and col
[\\cite{schonhage1971schnelle}] A. Schönhage and V. Strassen. Schnelle Multiplikation grosser Zah
[\\cite{schurmann2007incomplex}] K. Schurmann and J. Stoye. An incomplex algorithm for fast suffi
[\\cite{stiebitz2012graph}] Michael Stiebitz, Diego Scheide, Bjarne Toft, and Lene M Favrholdt. G
[\\cite{spielman2004smoothed}] D. Spielman and S. Teng. Smoothed analysis: Why the simplex algori
[\\cite{stallings2006cryptography}] W. Stallings. Cryptography and Network Security: Principles a
[\\cite{strassen1969gaussian}] V. Strassen. Gaussian elimination is not optimal. Numerische Mathe
[\\cite{svensson2017constant}] Ola Svensson, Jakub Tarnawski, and László A Végh. A constant-facto
[\\cite{stasko1987pairing}] J. Stasko and J. Vitter. Pairing heaps: Experiments and analysis. Com
[\\cite{schieber1988finding}] B. Schieber and U. Vishkin. On finding lowest common ancestors: sim
[\\cite{stout1986tree}] Q. Stout and B. Warren. Tree rebalancing in optimal time and space. Comm.
[\\cite{sedgewick2011algorithms}] R. Sedgewick and K. Wayne. Algorithms in Java, Parts 1-4: Funda
[\\cite{sviridenko2013large}] Maxim Sviridenko and Justin Ward. Large neighborhood local search f
[\\cite{schlieimer2003winnowing}] S. Schlieimer, D. Wilkerson, and A. Aiken. Winnowing: Local alg
[\\cite{shallit1995discovery}] J. Shallit, H. Williams, and F. Moraine. Discovery of a lost facto
[\\cite{sharma2018robust}] V. Sharma and C. Yap. Robust geometric computation. In J. Goodman, J.
[\\cite{szpiro2003kepler}] G. Szpiro. Kepler's Conjecture: How Some of the Greatest Minds in Hist
[\\cite{thulasiraman2016handbook}] Krisnaiyan Thulasiraman, Subramanian Arumugam, Andreas Brandst
[\\cite{tamassia2013handbook}] Roberto Tamassia. Handbook of Graph Drawing and Visualization. Cha
[\\cite{tarry1895-labyrinthes}] G. Tarry. Le problème de labyrinthes. Nouvelles Ann. de Math., 14
[\\cite{tarjan1972-depth-first}] R. Tarjan. Depth-first search and linear graph algorithms. SIAM
[\\cite{tarjan1975-set-union}] R. Tarjan. Efficiency of a good but not linear set union algorithm
[\\cite{tarjan1979-disjoint-sets}] R. Tarjan. A class of algorithms which require non-linear time
[\\cite{tarjan1983-data-structures}] R. Tarjan. Data Structures and Network Algorithms. Society f
[TDS ${ }^{+}$16] Andrea Tagliasacchi, Thomas Delame, Michela Spagnuolo, Nina Amenta, and Alexand
[\\cite{tam2003-medial-axis}] R. Tam and W. Heidrich. Shape simplification based on the medial ax
[\\cite{thompson1994-clustalw}] J. Thompson, D. Higgins, and T. Gibson. CLUSTAL W: improving the
[\\cite{thimbleby2003-directed-cpp}] H. Thimbleby. The directed Chinese postman problem. Software
[\\cite{thompson1968-regular-expression}] K. Thompson. Regular expression search algorithm. Commu
[\\cite{tinhofer1990-generating-graphs}] G. Tinhofer. Generating graphs uniformly at random. Comp
[\\cite{toth2018-handbook}] C. Toth, J. O'Rourke, and J. Goodman, editors. Handbook of Discrete a
[\\cite{trotter1962-perm}] H. F. Trotter. Perm (algorithm 115). Comm. ACM, 5:434-435, 1962.
[\\cite{turner1988-k-colorable}] J. Turner. Almost all k-colorable graphs are easy to color. J.
[\\cite{tamassia2001-algorithm-engineering}] R. Tamassia and L. Vismara. A case study in algorith
[\\cite{toth2014-vehicle-routing}] Paolo Toth and Daniele Vigo. Vehicle Routing: Problems, Method
[\\cite{tarjan1988-triangulation}] R. Tarjan and C. Van Wyk. An $\mathrm{O}(n \lg \lg n)$ algorit
[\\cite{ukkonen1992-suffix-trees}] E. Ukkonen. Constructing suffix trees on-line in linear time.
[\\cite{valiant1979-permanent}] L. Valiant. The complexity of computing the permanent. Theoretica
[\\cite{valiente2002algorithms}] G. Valiente. Algorithms on Trees and Graphs. Springer, 2002.
[\\cite{vandegriend1998finding}] B. Vandegriend. Finding hamiltonian cycles: Algorithms, graphs a
[\\cite{vazirani2004approximation}] V. Vazirani. Approximation Algorithms. Springer, 2004.

[\\cite{vandenberghe1996semidefinite}] L. Vandenberghe and S. Boyd. Semidefinite progra
[\\cite{vanemdeboas1977design}] P. van Emde Boas, R. Kaas, and E. Zulstra. Design and
[\\cite{veltkamp2001shape}] Remco C Veltkamp. Shape matching: Similarity measures and a
[\\cite{vitter2001external}] J. Vitter. External memory algorithms and data structures
[\\cite{vizing1964estimate}] V. Vizing. On an estimate of the chromatic class of a p-g
[\\cite{vanleeuwen1990graph}] J. van Leeuwen. Graph algorithms. In J. van Leeuwen, edi
[\\cite{vanleeuwen1990handbook}] J. van Leeuwen, editor. Handbook of Theoretical Compu
[\\cite{viger2005efficient}] Fabien Viger and Matthieu Latapy. Efficient and simple ge
[\\cite{voss1992steiners}] S. Voss. Steiner's problem in graphs: heuristic methods. Di
[\\cite{warshall1962theorem}] S. Warshall. A theorem on boolean matrices. J. ACM, 9:11
[\\cite{watson2003new}] B. Watson. A new algorithm for the construction of minimal acy
[\\cite{watts2004six}] D. Watts. Six Degrees: The Science of a Connected Age. W.W. Nor
[\\cite{wu2004spanning}] B. Wu and K. Chao. Spanning Trees and Optimization Problems.
[WCL $\left.{ }^{+} 14\right]$ Thomas Weise, Raymond Chiong, Jorg Lassig, Ke Tang, Shi
[\\cite{weiner1973linear}] P. Weiner. Linear pattern-matching algorithms. In Proc. 14t
[\\cite{weiss2011data}] M. Weiss. Data Structures and Algorithm Analysis in Java. Pear
[Wel84] T. Welch. A technique for high-performance data compression. IEEE Computer, 17
[Wel13] René Weller. New Geometric Data Structures for Collision Detection and Haptics
[Wes89] Jeffery R Westbrook. Algorithms and data structures for dynamic graph problems
[Wes00] D. West. Introduction to Graph Theory. Prentice-Hall, second edition, 2000.
[WF74] R. A. Wagner and M. J. Fischer. The string-to-string correction problem. J. ACM
[WH15] Qinghua Wu and Jin-Kao Hao. A review on algorithms for maximum clique problems.
[Whi32] H. Whitney. Congruent graphs and the connectivity of graphs. American J. Mathe
[Whi12] Tom White. Hadoop: The Definitive Guide. O'Reilly Media, 2012.
[WHS07] Gerhard Wäscher, Heike Hauner, and Holger Schumann. An improved typology of cu
[Wig83] A. Wigerson. Improving the performance guarantee for approximate graph colorin
[Wil64] J. W. J. Williams. Algorithm 232 (heapsort). Communications of the ACM, 7:347-
[Wil84] H. Wilf. Backtrack: An O(1) expected time algorithm for graph coloring. Info.
[Wil85] D. E. Willard. New data structures for orthogonal range queries. SIAM J. Compu
[Wil89] H. Wilf. Combinatorial Algorithms: An Update. SIAM, 1989.
[Wil12] Virginia Vassilevska Williams. Multiplying matrices faster than Coppersmith-Wi
[Wil19] D. Williamson. Network Flow Algorithms. Cambridge University Press, 2019.
[Win68] S. Winograd. A new algorithm for inner product. IEEE Trans./ Computers, C-17:6
[WM92a] S. Wu and U. Manber. Agrep-a fast approximate pattern-matching tool. In Usenix
[WM92b] S. Wu and U. Manber. Fast text searching allowing errors. Comm. ACM, 35:83-91,
[Woe03] G. Woeginger. Exact algorithms for NP-hard problems: A survey. In Combinatoria
[Wol79] T. Wolfe. The Right Stuff. Bantam Books, Toronto, 1979.
[\\cite{williamson2011design}] David P. Williamson and David B Shmoys. The Design of A
[\\cite{wang2013phase}] Kai Wang, Steven Skiena, and Thomas G Robertazzi. Phase balanc
[\\cite{wang2014hashing}] Jingdong Wang, Heng Tao Shen, Jingkuan Song, and Jianqiu Ji.
[\\cite{wagner1995map}] F. Wagner and A. Wolff. Map labeling heuristics: provably good
[\\cite{wang2005how}] X. Wang and H. Yu. How to break MD5 and other hash functions. In
[\\cite{xie2011improved}] Zhong Xie, Huimin Wang, and Liang Wu. The improved DouglasPe
[\\cite{yan2003primality}] S. Yan. Primality Testing and Integer Factorization in Publ
[\\cite{yao1981lower}] A. C. Yao. A lower bound to finding convex hulls. J. ACM, 28:78
[\\cite{yeung2005network}] R. Yeung, S-Y. Li, N. Cai, and Z. Zhang. Network Coding The

[\\cite{yu2016string}] Minghe Yu, Guoliang Li, Dong Deng, and Jianhua Feng. String similarity sea
[\\cite{yanofsky2008quantum}] Noson S. Yanofsky and Mirco A. Mannucci. Quantum Computing for Comp
[\\cite{younger1967recognition}] D. Younger. Recognition and parsing of context-free languages in
[\\cite{younas1996randomized}] F. Younas and S. Skiena. Randomized algorithms for identifying mir
[\\cite{yang1999shortest}] E. Yang and Z. Zhang. The shortest common superstring problem: Average
[\\cite{ziv1978universal}] J. Ziv and A. Lempel. A universal algorithm for sequential data compre
[\\cite{zaritsky2004preservation}] Z. Zaritsky and M. Sipper. The preservation of favored buildir
[\\cite{zwick2001exact}] U. Zwick. Exact and approximate distances in graphs - a survey. In Proc.

%---- Page End Break Here ---- Page : 727

%---- Page End Break Here ---- Page : 728

%---- Page End Break Here ---- Page : 729

%---- Page End Break Here ---- Page : 730

%---- Page End Break Here ---- Page : 731

%---- Page End Break Here ---- Page : 732

%---- Page End Break Here ---- Page : 733

%---- Page End Break Here ---- Page : 734

%---- Page End Break Here ---- Page : 735

%---- Page End Break Here ---- Page : 736

%---- Page End Break Here ---- Page : 737

%---- Page End Break Here ---- Page : 738

%---- Page End Break Here ---- Page : 739

%---- Page End Break Here ---- Page : 740

%---- Page End Break Here ---- Page : 741

%---- Page End Break Here ---- Page : 742

%---- Page End Break Here ---- Page : 743

%---- Page End Break Here ---- Page : 744

%---- Page End Break Here ---- Page : 745

%---- Page End Break Here ---- Page : 746

%---- Page End Break Here ---- Page : 747

%---- Page End Break Here ---- Page : 748

%---- Page End Break Here ---- Page : 749

%---- Page End Break Here ---- Page : 750

%---- Page End Break Here ---- Page : 751

%---- Page End Break Here ---- Page : 752

%---- Page End Break Here ---- Page : 753

%---- Page End Break Here ---- Page : 754

%---- Page End Break Here ---- Page : 755

%---- Page End Break Here ---- Page : 756

%---- Page End Break Here ---- Page : 757

%---- Page End Break Here ---- Page : 758

%---- Page End Break Here ---- Page : 759

%---- Page End Break Here ---- Page : 760

%---- Page End Break Here ---- Page : 761

%---- Page End Break Here ---- Page : 762

%---- Page End Break Here ---- Page : 763

%---- Page End Break Here ---- Page : 764

%---- Page End Break Here ---- Page : 765

%---- Page End Break Here ---- Page : 766

%---- Page End Break Here ---- Page : 767

%---- Page End Break Here ---- Page : 768

%---- Page End Break Here ---- Page : 769

%---- Page End Break Here ---- Page : 770

%---- Page End Break Here ---- Page : 771

%---- Page End Break Here ---- Page : 772

%---- Page End Break Here ---- Page : 773

%---- Page End Break Here ---- Page : 774

%---- Page End Break Here ---- Page : 775
\section{Index}

%---- Page End Break Here ---- Page : 785

%---- Page End Break Here ---- Page : 786

%---- Page End Break Here ---- Page : 788

%---- Page End Break Here ---- Page : 789

Bibliography